

从两次测向到完整清除： 有界误差下无线电干扰源的鲁棒定位与在线调度

摘要

针对测向误差有界、干扰源位置与数量未知条件下的自动搜索、定位与清除问题，本文以缩短任务总时间为目标，建立位置可行集合与在线任务调度相结合的模型，并利用几何覆盖条件保证搜索与清除的完整性。

针对问题一，将 $\pm 1^\circ$ 测向误差表示为半平面约束，通过求交构造定位区域，采用边界交点枚举与最远顶点对计算区域直径。利用等边三角形反例证明，**以区域直径为直径的圆不一定覆盖整个区域**，进而给出基于指定圆心覆盖半径的判据，为后续光学清除提供依据。

针对问题二，结合首次接收信息与接收半径下界，推导保证再次接收的测点区域，并加入交会角约束，建立以最坏定位直径最小化为目标的选点模型。经网格搜索和场景加密，得到沿首次示向方向前进 **800 m**、**横向偏移 ± 605 m** 的推荐测点；在 $\pm 1^\circ$ 误差下，36,421 个场景中的最大定位直径为 **134.284 m**。

针对问题三，构造七站圆盘覆盖，结合位置区域收缩、跨频道观测共享与滚动任务排序，交错执行未知源搜索和已知源清除。在 120 个独立布局、共 960 次条件运行的自建验证中，联合方案均完成全部清除；相对冻结基线，有界误差和舍入压力两种环境下的平均总时间分别降低 **10.27%** 和 **10.07%**。

针对问题四，利用近距测站凸包建立适用于未知发射方向的连续发现条件，构造 **25 站双环布设**，并在测向失联时采用有限网格光学覆盖完成清除。结合在线排序与低信息量观测筛选，在两批各 96 个新布局的自建配对验证中，相对 31 站初始方案的平均总时间分别降低 **41.83%** 和 **41.24%**。

在官方模拟器中，第三、四问各完成三次正式测试，分别累计清除 38 个和 36 个干扰源，六次均满足停止判据并主动退出。两问逐次每源平均定位清除时间的算术平均分别为 **264.41 s** 和 **514.13 s**。

关键词：有界测向误差；位置可行集合；鲁棒定位；连续覆盖；在线调度

1 问题重述

1.1 问题背景

为代替人工进入危险区域排查，题目要求机器狗利用无线电测向、光学定位和激光清除装置，自动搜索并清除干扰源，尽量缩短任务总时间^[1]。

源位于半径 1800 m 的圆域内，各占 1—20 中的不同频道，接收半径未知且介于 1000—1500 m。机器狗停止后逐频道检测，正常示向度误差为 $\pm 1^\circ$ ，同点同源误差固定；距源 20 m 内可进行光学定位清除。全向源向所有方向发射，定向源仅向轴线两侧各 90° 的闭半平面发射；光学清除不受方向限制。移动、切频、检测及光学清除均计时^[1-3]。

1.2 问题提出

问题一：由检测点坐标和示向度计算交会定位区域的直径，并判断直径圆能否覆盖整个区域。

问题二：已知全向源的首次测向，给出第二检测点的候选区域与选点策略，以改善定位效果。

问题三：对 10—16 个数量未知的全向源，从圆心、频道 1 出发，设计确保全部清除且总时间尽量短的自动算法。

问题四：在其余条件不变、全向源与未知数量及方向的定向源混合时，给出搜索定位与清除算法。第三、四问均需模拟器演练及各三次正式测试。

2 问题分析

2.1 问题一的分析

测向中心线交点忽略了有界误差，应以多个前向楔形的交集表示可能位置。先判断集合是否为空或无界，再由凸多边形顶点计算直径。直径不能直接替代覆盖半径，须另建圆覆盖判据，供后续清除使用。

2.2 问题二的分析

横移可能失联，沿示向前进又可能使交会退化。先利用首次接收和半径下界构造保证接收域，再筛选交会角足够大的测点；以相容源位置和读数中的最坏定位直径比较候选，并记录精度改善所需的移动代价。

2.3 问题三的分析

完整清除同时要求发现未知源和清除已知源。以七站覆盖排除遗漏，以区域收缩指导局部清除；将搜索站与源服务共同排序，每次执行首任务后重排，并在满足接收与信息条件时共享观测。覆盖与源数上限约束停止，整局计时评价排序收益。

2.4 问题四的分析

定向发射使“距离足够近”不再保证接收，单次无信号也不能排除附近源。以近距测站凸包包围候选源，建立任意方向下的连续发现条件；有效测向收缩区域，失联时用有限光学网格完成清除，再通过滚动调度降低时间。

2.5 总体建模思路

四问依次建立定位区域、第二测点、全向多源任务与定向混合任务，形成图 1 所示闭环。“证书”指仅由公开观测和几何条件可核验的充分条件，用于约束动作及停止；效率通过同布局、同误差条件下的整局配对实验判断。

从局部定位到完整清除：问题递进与方法衔接

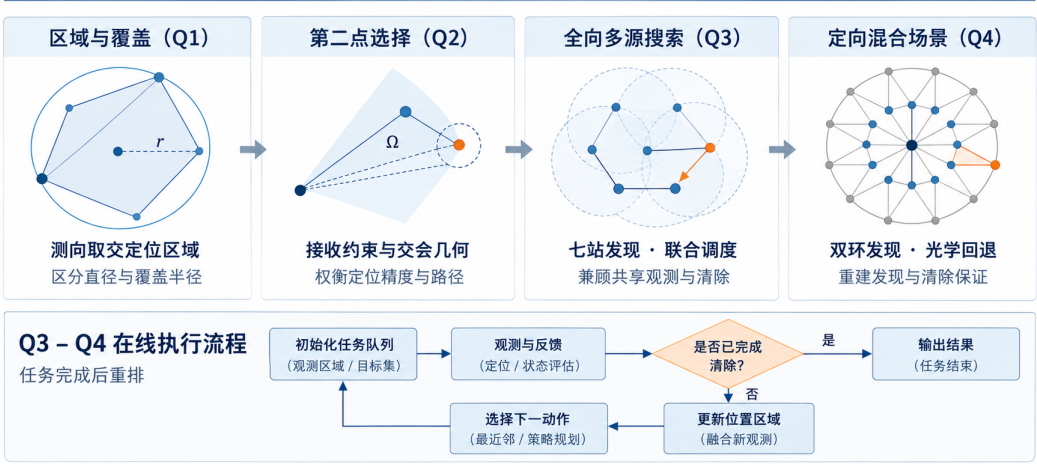


图 1: 整体技术路线。上方按 Q1 至 Q4 展示方法递进，下方概括 Q3 与 Q4 的在线执行反馈。几何小图为非定量示意；流程中的“完成清除”须同时满足正文的停止判据，不能仅凭已知源全部清除就退出。

3 模型假设

依据题面及接口规则^[1-3]，采用以下前提：

1. **静态与独立**。单局中源的位置、频道、半径及方向固定，频道互异且信号互不影响。
2. **平面运动**。移动距离按直线计算；圆域限定源位置，测点可以位于域外。
3. **理想接收**。接收由距离与发射方向决定，不另设遮挡或随机丢包；光学清除只受距离限制。
4. **有界误差**。测向误差为 $\pm 1^\circ$ ，同点同源误差固定，不设概率分布或同点平均降噪。
5. **顺序计时**。停止后检测，各动作依次执行并分别计时。
6. **公开信息决策**。策略仅使用位置、频道、观测反馈及题面范围，不读取源真值、实际半径、方向或总数。

4 符号说明

主要符号及其含义如下；局部辅助量在使用时定义。

表 1: 主要符号说明

符号	含义与层级
Ω	半径 1800 m 的源位置先验圆域
p, S_i, Q	机器人位置、第 i 个检测点、候选第二检测点
g_j, r_j, n_j	第 j 个源的位置、接收半径及定向发射轴单位向量
θ_i, δ	第 i 次示向度及当前采用的误差半宽
$\delta_{\text{th}}, \delta_{\text{num}}$	理论误差半宽 1° ；程序保守误差半宽 1.005°
c, N_{src}	频道编号；一局中的实际源数
L	累计移动距离，单位 m
$N_{\text{sw}}, N_{\text{det}}$	实际测向频道切换次数；成功执行的检测调用次数（包括无信号）
$N_{\text{opt}}, N_{\text{clr}}$	光学调用次数；成功清除的不同源数
$T, \tau_{\text{clr}}, p_{\text{clr}}$	总虚拟时间、逐局每源平均时间及清除比例
$\mathcal{F}_1$	首次正常测向后，与角度、距离及源域相容的真实初始候选集合
P	仅由测向楔形取交得到的角度定位区域，可能无界
P_c	频道 c 当前的保守位置外包，结合源域、距离上界与有效测向
$\mathcal{C}_{\text{rec}}$	对所有初始候选源和相容接收半径均保证再次接收的测点集合
$\mathcal{C}_{\text{safe}}$	在接收域内进一步满足近距排除与交会角条件的充分候选域
$D(P), R(P, q)$	区域直径；区域关于指定点 q 的覆盖半径，单位 m

这里“外包”允许保留部分实际上不可能的位置，以换取不删去真源的可靠性。因此 P_c 不必等于精确可行集合， $\mathcal{C}_{\text{rec}}$ 也属于测点空间，不能与源位置集合混为一谈。

程序使用 $\delta_{\text{num}} = 1.005^\circ$ ，在题面的 1° 误差界上增加两位小数读数可能引入的 0.005° 舍入量。这是实现中的保守包络，并非题目直接给出的误差范围。下文 δ 泛指当前使用的半宽，解析示例与数值复核分别注明取值。

5 模型建立与求解

本章先建立四问共用的接收条件与计时模型，再分别给出定位区域计算、第二测点选择以及全向和混合源场景的完整搜索清除方法。

统一接收模型。设目标圆域为 $\Omega = \{x \in \mathbb{R}^2 : \|x\| \leq 1800\}$ ，第 j 个源位置为 g_j 、接收半径为 $r_j \in [1000, 1500]$ m。全向源在 $\|p - g_j\| \leq r_j$ 时能够接收；对于定向源，若 n_j 为其未知发射轴单位向量，则还需满足

$$\|p - g_j\| \leq r_j, \quad n_j^\top (p - g_j) \geq 0. \quad (1)$$

发射半平面对包含边界。若接口已给出 5 m 内的近场响应，则可直接光学清除；但距离源很近也不能消除背向不可见性。光学清除仅要求距离不超过 20 m，与发射方向无关。因此，无线电接收和光学清除必须按不同条件处理。

统一代价与完成指标。

设累计移动距离为 L ，实际测向频道切换次数为 N_{sw} ，成功执行的检测调用次数为 N_{det} （包括返回无信号的调用），光学调用次数为 N_{opt} ，不同源成功清除次数为 N_{clr} 。按照给定计费规则，总虚拟时间为

$$T = \frac{L}{5} + N_{\text{sw}} + 5N_{\text{det}} + 3N_{\text{opt}} + 2N_{\text{clr}}. \quad (2)$$

光学调用无论成功与否都花费 3 s，成功清除再加 2 s；/clear 不切换测向机频道，因此不计入 N_{sw} 。各动作串行计费，不能用同时执行抵消代价。程序的现实运行时间另行记录，不等于式 (2) 的虚拟时间。

评价首先要求全部源实际清除且停止条件合法，然后比较 T 。这样可以排除通过漏清或提前退出获得较低时间的策略。题目要求的每源平均时间记为 $\tau_{\text{clr}} = T/N_{\text{clr}}$ ，跨局统计先逐局计算再平均，不用 $\sum T / \sum N_{\text{clr}}$ 替代。若 $N_{\text{clr}} = 0$ ，该指标不定义，该局仍保留为失败记录。

成功清除比例定义为 $p_{\text{clr}} = N_{\text{clr}}/N_{\text{src}}$ ；只有 $p_{\text{clr}} = 1$ 且停止依据合法时才算完整完成。 N_{src} 由评价器事后核验，策略执行时不能读取。

5.1 问题一：交会定位区域的直径计算与圆覆盖判定

5.1.1 交会定位区域的数学建模

对于仅给出误差界的观测，集合估计用与误差界相容的集合描述未知量^[4]。本文将这一思想用于测向定位，以各次观测约束的交集表示源的可能位置。

设同一干扰源的 n 组检测数据为 $\{(S_i, \theta_i)\}_{i=1}^n$ ，其中 $S_i = (x_i, y_i)^T$ 为检测点坐标， θ_i 为对应示向度，待定位点记为 $z = (x, y)^T$ 。本问取题面误差半宽 $\delta = 1^\circ$ ，以各次测向扇区的交集作为多边形定位区域；源域和接收距离约束在后续各问中另行加入。

第 i 次测向的两条边界方向为 $u_i^\pm = (\cos(\theta_i \pm \delta), \sin(\theta_i \pm \delta))^T$ 。记二维叉积为 $[a, b] = a_x b_y - a_y b_x$ ，则与该次读数相容的位置满足

$$[u_i^-, z - S_i] \geq 0, \quad [u_i^+, z - S_i] \leq 0. \quad (3)$$

式 (3) 分别限定位于下边界的左侧和上边界的右侧，两者共同构成张角为 2δ 的前向楔形，记为 $W(S_i, \theta_i, \delta)$ 。由于 $2\delta < 180^\circ$ ，该表示同时限定了方位误差和前向方向，且适用于示向度跨越 0° 的情况。

将叉积展开，令两个半平面的行法向量及右端项为

$$\begin{aligned} a_{2i-1} &= (\sin(\theta_i - \delta), -\cos(\theta_i - \delta)), & b_{2i-1} &= a_{2i-1} S_i, \\ a_{2i} &= (-\sin(\theta_i + \delta), \cos(\theta_i + \delta)), & b_{2i} &= a_{2i} S_i. \end{aligned}$$

以 a_j 为矩阵 A 的第 j 行， $b = (b_1, \dots, b_{2n})^T$ ，便得到定位区域模型

$$P = \bigcap_{i=1}^n W(S_i, \theta_i, \delta) = \{z \in \mathbb{R}^2 : Az \leq b\}, \quad A \in \mathbb{R}^{2n \times 2}.$$

半平面的交集具有凸性^[5]。观测与误差假设一致时，真实源位于 P 内；但不同测向的交集可能无界，也可能退化为线段或单点。因此，计算直径前需先判断区域的状态。

5.1.2 定位区域直径的计算

对非空有界的 P ，顶点集为 $V(P) = \{v_1, \dots, v_k\}$ ，则

$$D(P) = \max_{z, w \in P} \|z - w\| = \max_{1 \leq a, b \leq k} \|v_a - v_b\|. \quad (4)$$

任意 z, w 均为顶点凸组合，三角不等式给出 $\|z - w\| \leq \max_{a, b} \|v_a - v_b\|$ ；反向不等式由顶点属于 P 得到。计算步骤如下。

1. **判定状态。** 线性规划检查 $Az \leq b$ 的可行性；为空时直径不定义。分别最小化 $x, -x, y, -y$ ，任一无界则 $D(P) = +\infty$ 。
2. **恢复顶点。** 枚举全部不平行边界对的交点，保留满足所有约束者并去重；有界区域的每个顶点必在其中。
3. **计算直径。** 比较全部顶点对，输出最大距离及端点；线段取两端距离，单点取 0。

除固定次数的二维线性规划外，顶点筛选复杂度为 $O(n^3)$ ，直径比较为 $O(n^2)$ 。图 2 算例以 $G = (600, 180)$ m、 $S_1 = (0, 0)$ 、 $S_2 = (850, -450)$ m 和零中心线误差生成读数，得 $D(P) = 33.691$ m；详细输入、顶点及计算精度见附录 E.1。

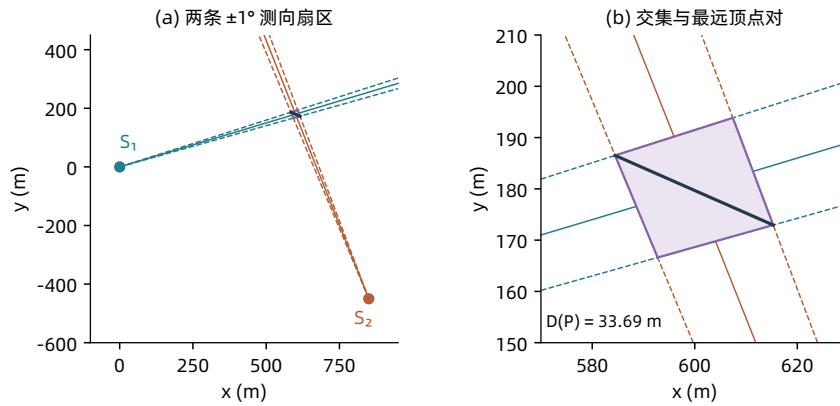


图 2: 两次测向定位算例。左图为两个测向扇区，右图放大其交集；黑色线段连接最远顶点对，长度约为 33.69 m。

5.1.3 直径圆的覆盖判定

对于非空有界的定位区域 P ，以其直径 D 为直径的圆不一定能够覆盖整个区域。为判断具体区域是否满足这一性质，取一对直径端点 v_a, v_b 。若半径为 $D/2$ 、圆心为 q 的圆盘包含 P ，则必须同时包含这两个端点，因而

$$D = \|v_a - v_b\| \leq \|v_a - q\| + \|q - v_b\| \leq D.$$

上式两处不等式均须取等号，故圆心只能是 $q_D = (v_a + v_b)/2$ 。定义区域关于指定点 q 的覆盖半径为

$$R(P, q) = \max_{v \in V(P)} \|v - q\|. \quad (5)$$

圆盘是凸集，包含全部顶点即可包含其凸包 P 。因此，存在直径为 D 的圆盘覆盖 P 的充要条件为

$$R(P, q_D) \leq D/2, \quad q_D = \frac{v_a + v_b}{2}.$$

图 2 算例中， $q_D \approx (599.882, 179.731)$ m，且 $R(P, q_D) = D/2 \approx 16.845$ m，满足上述判据，因此直径圆可以覆盖整个定位区域。

一般情形则不成立。如图 3，取等边三角形顶点 $(0, 0)$ 、 $(39, 0)$ 、 $(19.5, 19.5\sqrt{3})$ ，其直径为 39 m。以底边为直径时，圆心为 $(19.5, 0)$ ，第三顶点距圆心 $19.5\sqrt{3} \approx 33.775$ m，大于圆半径 19.50 m，覆盖判据不成立。由圆心唯一性，改换圆心也无法得到直径为 39 m 的覆盖圆。该三角形的最小覆盖圆为外接圆，其半径为 $39/\sqrt{3} \approx 22.52$ m，严格大于 $D/2$ 。附录 B.1 给出能够产生该三角形的三组合法测向数据。

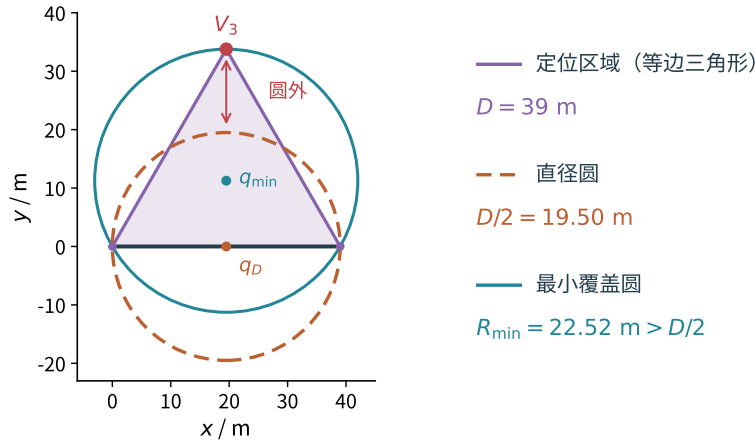


图 3: 直径圆不能覆盖定位区域的反例。橙色虚线圆以三角形底边为直径，半径为 19.50 m，未覆盖上方顶点；蓝色实线圆为最小覆盖圆，半径约为 22.52 m。

这一结论也给出后续清除的判断依据：对于维护的位置外包 P_c ，若 $R(P_c, q) \leq 20$ m，则在 q 可以保证光学清除；仅有 $D(P_c) \leq 40$ m 不足以作出该保证。

5.2 问题二：第二检测点的选择

本问只给出首次检测点及示向度，尚不能确定源距，也没有给出源位置或测向误差的概率分布。本文将“较好的定位效果”具体化为：在保证第二次获得正常示向度、避免近共线交会的条件下，尽量减小两次测向定位区域的最坏直径。移动时间作为候选方案的附加比较指标，第三问再以完整清除时间为目标安排动作。

5.2.1 测向几何关系

设首测点为 S ，测得示向度为 θ ，误差半宽为 δ 。以示向方向为局部横轴，定义正交单位向量

$$u = (\cos \theta, \sin \theta), \quad v = (-\sin \theta, \cos \theta).$$

任一与首次读数相容的候选源 G 和待选第二测点 Q 可表示为

$$G = S + r(\cos \alpha u + \sin \alpha v), \quad Q = S + au + bv.$$

其中, $r = \|G - S\|$ 为首测点到源的距离, α 为真实方向相对示向度的偏差; a, b 分别为第二测点沿示向方向和法向的位移。正常示向度意味着 $5 < r \leq 1500$ m 且 $|\alpha| \leq \delta$ 。结合源位置先验圆域 Ω , 首次候选位置集合为

$$\mathcal{F}_1 = \{S + r(\cos \alpha u + \sin \alpha v) : 5 < r \leq 1500, |\alpha| \leq \delta\} \cap \Omega.$$

记 $d_2 = \|G - Q\|$, $\varphi \in [0, \pi]$ 为向量 $G - S$ 与 $G - Q$ 的夹角。将两条视线的锐交会角定义为 $\beta = \min(\varphi, \pi - \varphi)$ 。当 β 接近 0 时, 两次测向近共线, 角度误差会形成狭长甚至无界的交集; 因此应同时考虑第二点是否仍能接收和交会角是否足够大。

图 4 给出几何关系及两个反例。若源为 (1500, 0) m、接收半径为 1500 m, 从首测点 (0, 0) 横移到 (0, 500) 后, 源距增至 1581.14 m, 发生失联; 前移到 (500, 0) 虽仍能接收, 却与源保持共线, 不能保证纯角度交集有界。这说明单独增加横向位移或沿示向度靠近, 都不足以构成完整的第二测点选择策略。

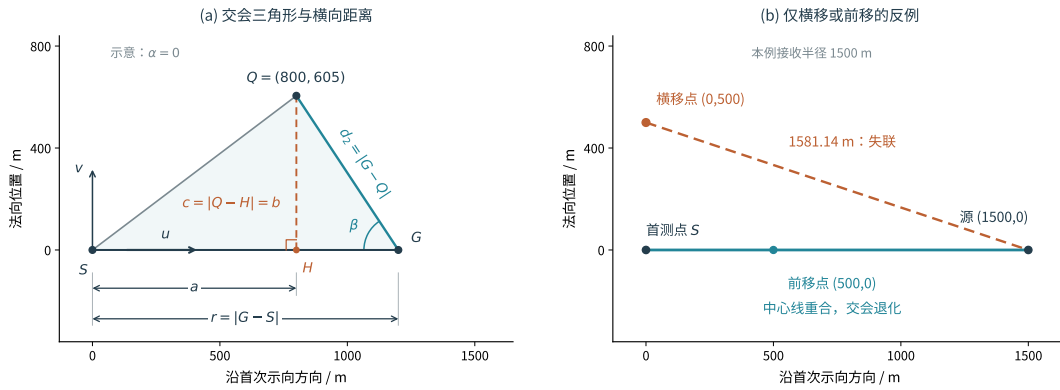


图 4: 第二测点的几何关系与典型反例。(a) 图示取 $\alpha = 0$ 、 $b > 0$, H 为 Q 在真实视线 SG 上的垂足, 此时 $c = b$ 且 $\sin \beta = c/d_2$; (b) 横移可能超出接收范围, 前移可能造成交会退化。坐标均为构造示例, 不代表搜索轨迹。

5.2.2 信号接收条件

首次成功接收说明真实接收半径 $R_G \geq r$, 结合 $R_G \in [1000, 1500]$, 其相容最小值为 $\max(1000, r)$ 。因此, 对固定候选源 G , 第二点保证接收的充要条件为

$$\|Q - G\| \leq \max(1000, \|G - S\|). \quad (6)$$

对全部 $G \in \mathcal{F}_1$ 取交, 得 $\mathcal{C}_{\text{rec}} = \bigcap_{G \in \mathcal{F}_1} B(G, \max(1000, \|G - S\|))$, 其中 B 为闭圆盘。为得到不依赖首测位置的通用策略, 使用不经源域裁剪的闭合扇形外包

$$\mathcal{F}_0 = \{S + r(\cos \alpha u + \sin \alpha v) : 5 \leq r \leq 1500, |\alpha| \leq \delta\} \supseteq \mathcal{F}_1.$$

令 $\ell = \sqrt{a^2 + b^2}$ 、 $t(\alpha) = a \cos \alpha + b \sin \alpha$, 则 $d_2^2 = \ell^2 + r^2 - 2rt(\alpha)$ 。近段 $r \in [5, 1000]$ 的距离平方为凸函数, 只需检查两端; 远段 $r \in [1000, 1500]$ 由 $r = 1000$ 条件控制。可行性要求 $a \geq 0$, 最不利方向投影为 $h = \min t = a \cos \delta - |b| \sin \delta$, 故

$$a \geq 0, \quad \ell^2 - 10h + 25 \leq 10^6, \quad \ell^2 \leq 2000h. \quad (7)$$

式 (7) 对 $\mathcal{F}_0$ 充要, 对裁剪后的 $\mathcal{F}_1$ 可能偏保守; 它也排除了 $a = 0, b \neq 0$ 的纯横移。完整分段推导见附录 E.2。

5.2.3 交会角约束

为获得正常读数并避免共线, 进一步要求 $d_2 > 5$ m、锐交会角 $\beta \geq 20^\circ$ 。图 4(a) 中第二测点到真实视线的垂距为 $c = |b \cos \alpha - a \sin \alpha|$, 且 $\sin \beta = c/d_2$ 。定义

$$c_{\min} = |b| \cos \delta - a \sin \delta,$$

$$M^2 = \max\{\ell^2 + 25 - 10h, \ell^2 + 1500^2 - 3000h\}.$$

由 $c \geq c_{\min}$ 及 $d_2 \leq M$, 以下条件足以保证正常接收距离与交会角:

$$\boxed{c_{\min} > 5, \quad c_{\min} \geq M \sin 20^\circ.} \quad (8)$$

于是 $\mathcal{C}_{\text{safe}} = \{S + au + bv : \text{式 (7)、(8) 成立}\}$, 其关于示向轴对称, 见图 5。M 是最大第二源距; 式 (8) 为充分条件, 20° 是设计参数, 均不宣称最优。真实源保证交集非空, 且 $20^\circ - 2\delta > 2\delta$ ($\delta = 1^\circ$ 或 1.005°), 两楔形无共同非零延伸方向, 故交集有界。

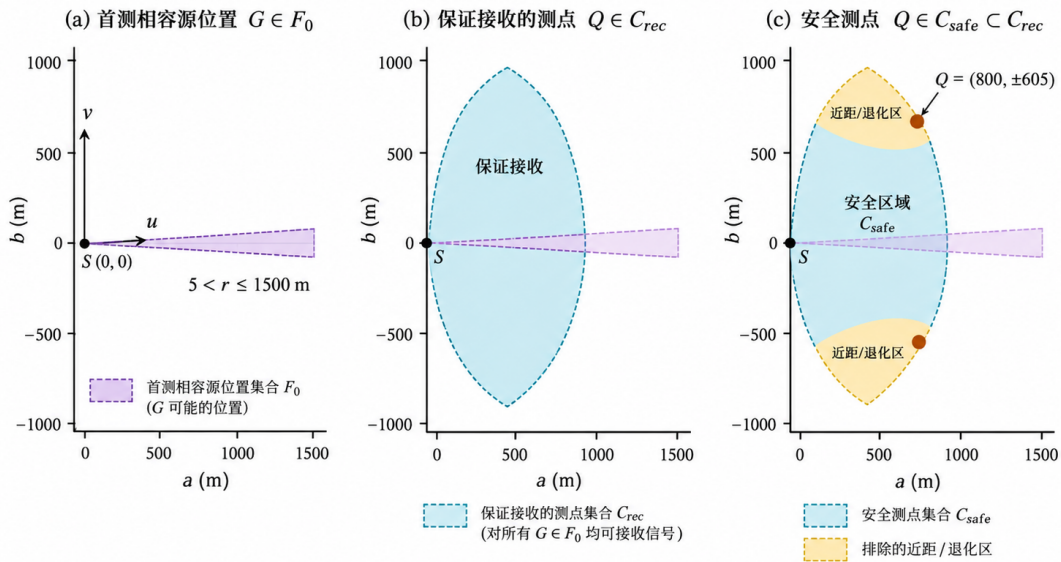


图 5: 第二测点候选区域的构造示意。(a) 首次测向的相容源位置; (b) 保证接收的测点集合; (c) 安全测点筛选及推荐镜像点。填色仅示意集合关系, 不作为边界或点归属的数值判据; 安全区域严格由式 (7) 和式 (8) 共同确定, 轴线上退化测点不属于安全区域。

5.2.4 优化模型及求解

对候选点 Q 、相容源 G 和第二次误差 ε_2 , 读数为 $\theta_2 = \arg(G - Q) + \varepsilon_2$, 定位区域为 $P(Q; G, \varepsilon_2) = W(S, \theta, \delta) \cap W(Q, \theta_2, \delta)$ 。首次误差已由 α 确定, 不再独立设置。建立最坏定位直径模型:

$$J_0(Q) = \sup_{G \in \mathcal{F}_0, |\varepsilon_2| \leq \delta} D(P(Q; G, \varepsilon_2)),$$

$$Q^* \in \arg \min_{Q \in \mathcal{C}_{\text{safe}}} J_0(Q). \quad (9)$$

决策变量为 (a, b) ，只以定位直径为目标； P 沿用第一问的纯角度交集。以 $\mathcal{F}_0$ 代替 $\mathcal{F}_1$ 使策略通用且可能保守，有限场景仅近似上确界。求解分三步：

1. **粗网格筛选。** 令 $S = 0, \theta = 0, \delta = 1^\circ$ ，仅搜 $b > 0$ 。取 $a = 0, 50, \dots, 1000$ m, $b = 50, 100, \dots, 1000$ m，以解析式 (7)、(8) 筛选。源距 51 点、两个误差维度各 5 点组成 1275 场景，逐一求交并取最大直径。
2. **局部加密。** 优先最小采样直径，同值时依次比较移动距离、 a, b ；在粗选点周围 ± 50 m 内以 5 m 步长加密。粗选 (800, 600) m 对应 134.976 m，加密得到 (800, 605) m 及 134.284 m。
3. **复核与还原。** 增加至 $301 \times 11 \times 11 = 36,421$ 场景，推荐点最大直径在所报精度下不变；按 $Q = S + au + bv$ 还原。再以 $\pm 1.005^\circ$ 复核既定 8 点及镜像，不重新搜索坐标。

每个场景的半平面数固定， N_Q 个候选、 K 个场景的评分复杂度为 $O(N_Q K)$ 。采样端点及评价次数见附录 E.3。

5.2.5 结果分析

以下列出 1.005° 包络下三个代表候选。锐交会角下界按 $\arcsin(c_{\min}/M)$ 计算，各点及其镜像均通过式 (7)、式 (8)；全部 8 点的精度与时间数据见附录 B.2。

表 2: 第二测点代表候选的解析条件与数值结果

局部偏移 $(a, b)/\text{m}$	交会角下界/ $^\circ$	采样最大直径/m	移动与检测/s
(500, 500)	25.767	236.082	146.421
(850, 450)	27.025	154.298	197.354
(800, 605)	36.258	135.026	205.602

推荐点满足 $c_{\min} = 590.875$ m、 $M = 999.078$ m 及两组解析条件。其他候选的 $M > 1000$ m 不等于失联，接收须按式 (7) 判断。

因此，以采样最坏定位直径为准，本文保留的第二测点推荐为

$$\begin{aligned} Q^\pm &= S + 800u \pm 605v \\ &= S + 800(\cos \theta, \sin \theta) \pm 605(-\sin \theta, \cos \theta). \end{aligned}$$

两镜像在完整扇形模型下等价；利用源域裁剪或后续任务后应另行比较。第二次测向后按第一问更新区域，仍以 20 m 覆盖判据决定清除。

图 6 展示固定候选库内的定位精度与移动代价。相对 (500, 500)，推荐点将采样最大直径降低 42.81%，但多花 59.18 s。表中计时为 $\ell/5 + 5$ ：追踪同一频道无需切频，尚未计入后续定位与清除。该比较说明第二问的精度推荐不能直接等同于第三问的最快清除动作。

候选的接收及交会保证来自式 (7)、式 (8)；134.284 m 和 135.026 m 分别是 1° 、 1.005° 条件下的有限场景最大值。加密后结果一致支持当前数值计算的稳定性，但不排除网格之间存在更差情形，也不证明候选搜索或 20° 阈值全局最优。第三问将继续使用这些几何条件，并把测量、移动和清除放入完整任务中评价。

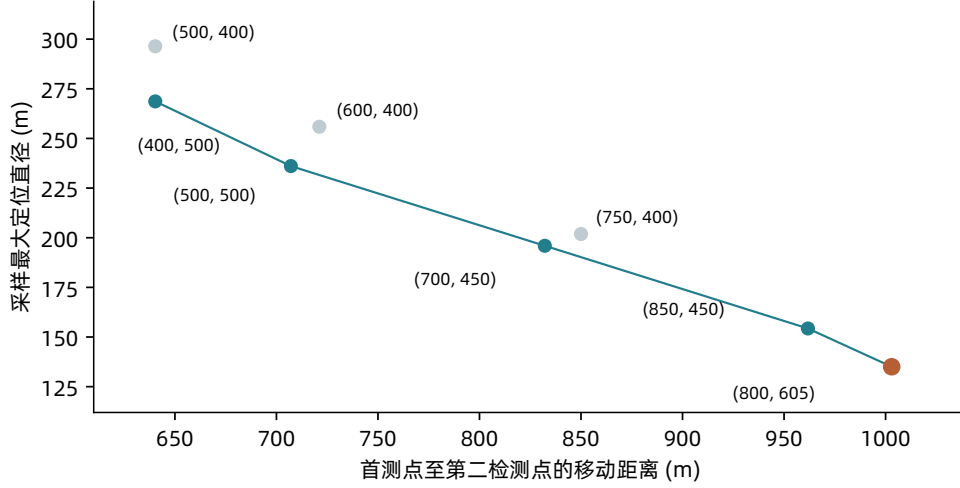


图 6: 固定候选库的精度与移动代价比较。展示 8 个上侧候选, 镜像具有相同指标; 蓝色为库内非支配点, 灰色为被支配点, 橙色为保留推荐点。纵轴为 36,421 个场景中的最大直径, 连线仅连接库内非支配点。

5.3 问题三: 全向干扰源的搜索与清除

第三问要求在源数、位置和接收半径未知的条件下完成全部清除, 并尽量缩短式 (2) 给出的总时间。为此, 将行动划分为两类任务: 搜索任务负责发现未知频道, 源服务任务负责将一个已知源定位并清除。以公开观测形成的位置区域作为决策依据, 在可靠测向与清除约束下安排任务顺序, 形成“区域更新—动作选择—任务执行—完成核验”的在线算法。

5.3.1 位置区域的建立与更新

在第 t 次任务决策时, 记机器人位置为 p_t , 已发现频道集合为 $\mathcal{D}_t$, 已成功清除频道集合为 $\mathcal{C}_t \subseteq \mathcal{D}_t$, 未完成搜索站的编号集合为 $\mathcal{S}_t$ 。搜索采用七个固定站, 其坐标及覆盖证明见第 5.3.4 节。每个已知未清频道 c 维护候选位置区域 P_c 、最新示向度 θ_c 及已检测位置记录; 每个未知频道保留各搜索站的无信号记录。发现和清除集合仅根据实际反馈更新, 隐藏源坐标不参与任务决策。

设频道 c 首次正常测向的位置为 S_c 、读数为 $\theta_c^{(0)}$ 。以 $\hat{B}_{32}(s, r)$ 表示圆盘 $B(s, r)$ 的 32 个外切支持半平面之交, 则初始位置区域及后续更新为

$$\begin{aligned} P_c^{(0)} &= \hat{B}_{32}(0, 1800) \cap \hat{B}_{32}(S_c, 1500) \cap W(S_c, \theta_c^{(0)}, \delta), \\ P_c^{(k+1)} &= P_c^{(k)} \cap W(s, \theta, \delta). \end{aligned}$$

其中 (s, θ) 为新获得的正常测向, 程序取 $\delta = 1.005^\circ$ 。1500 m 是首次接收给出的距离上界; 采用外切近似, 可保留圆周附近的合法位置。由于每个加入的约束均包含真实源 g_c , 有

$$g_c \in P_c^{(k+1)} \subseteq P_c^{(k)}.$$

因此, 位置区域随有效观测逐步缩小, 且始终保留真实源。后续无信号不用于裁剪这一多边形; 在保证接收的位置出现无信号时, 按模型或执行异常处理。

为从区域生成测点，沿最新示向方向建立正交基 $u_c = (\cos \theta_c, \sin \theta_c)^\top$ 、 $v_c = (-\sin \theta_c, \cos \theta_c)^\top$ 。对 $e \in \{u_c, v_c\}$ ，记 $\ell_{c,e} = \min_{x \in P_c} e^\top x$ 、 $h_{c,e} = \max_{x \in P_c} e^\top x$ ，则该方向下包围盒的中心及其覆盖半径为

$$q_c = \frac{1}{2} \sum_{e \in \{u_c, v_c\}} (\ell_{c,e} + h_{c,e})e, \quad R_c = R(P_c, q_c).$$

后文的区域中心均指 q_c ，即沿最新示向方向旋转的包围盒中心。由式 (5) 的顶点覆盖判据，对于任意候选测点 q ，

$$\begin{aligned} R(P_c, q) < 1000 \text{ m} &\implies \text{保证接收到该全向源,} \\ R(P_c, q) < 20 \text{ m} &\implies \text{保证在该点清除该源.} \end{aligned}$$

这两个条件分别把题目的最小接收半径和光学清除半径转化为可计算约束。程序逐顶点核验并预留数值余量；实际清除成功后才更新 C_t 。

5.3.2 测点选择与定位清除

(1) 局部测点选择。服务频道 c 时，先计算 q_c 与 R_c 。若 $R_c < 20 \text{ m}$ ，则前往 q_c 清除；否则，在本次服务的首轮局部测向中考察两个横向候选点

$$q_c^\pm = q_c \pm \sqrt{20R_c} (-\sin \theta_c, \cos \theta_c). \quad (10)$$

垂直于示向方向的位移用于形成视差，偏移尺度中的 20 表示清除半径 20 m。先筛选不满足接收约束的点，再从剩余候选中选取距当前位置最近者；若两点均不满足条件，则使用 q_c 。后续轮次在更新后的包围盒中心测向，每轮均先检查清除和接收条件。由于此前可能已有共享观测，服务首轮不一定对应该源的第二次测向。式 (10) 是固定的几何启发式，其效率由第 6.2 节的实验评价。

(2) 中心测量的收缩性质。若当前 $P \subseteq B(q, R)$ ，在 q 得到半宽 $\delta < 30^\circ$ 的正常方位后，新的区域记为 P^+ ，则

$$D(P^+) \leq \max\{R, 2R \sin \delta\} = R, \quad R(P^+, q^+) \leq \frac{R}{\sqrt{2}},$$

其中 q^+ 为新区域的包围盒中心。前一不等式来自半径 R 圆盘与前向楔形的交集：任意两点的径向距离在 $[0, R]$ 内，最远距离在径向端点组合中取得，最大值为 $\max\{R, 2R \sin \delta\}$ 。后一不等式利用包围盒两边长均不超过区域直径，其半对角线不超过 $R/\sqrt{2}$ 。因此，在实数几何模型和正常测向条件下，连续中心测量使覆盖半径按不超过 $1/\sqrt{2}$ 的因子收缩，有限次后可达到清除阈值。

(3) 有限回退。为处理同点重复测量和数值停滞，每个源服务至多进行 16 轮局部迭代。若迭代结束仍未清除，或下一测点已对该频道检测过，则沿最新示向方向划分当前包围盒，形成边长不超过 28 m 的网格，并依次前往格心尝试清除。每格内任一点距格心至多 $14\sqrt{2} < 20 \text{ m}$ ，故含真源的格必被覆盖。若测向返回 5 m 内的近场提示，则直接在当前位置清除。接收或清除反馈矛盾、网格耗尽仍未成功时，报告未完成。

图 7 展示三次测向的区域收缩：中心覆盖半径降至 10.66 m 后满足清除条件。该教学算例的坐标、读数及固定轴计算口径见附录 E.1。

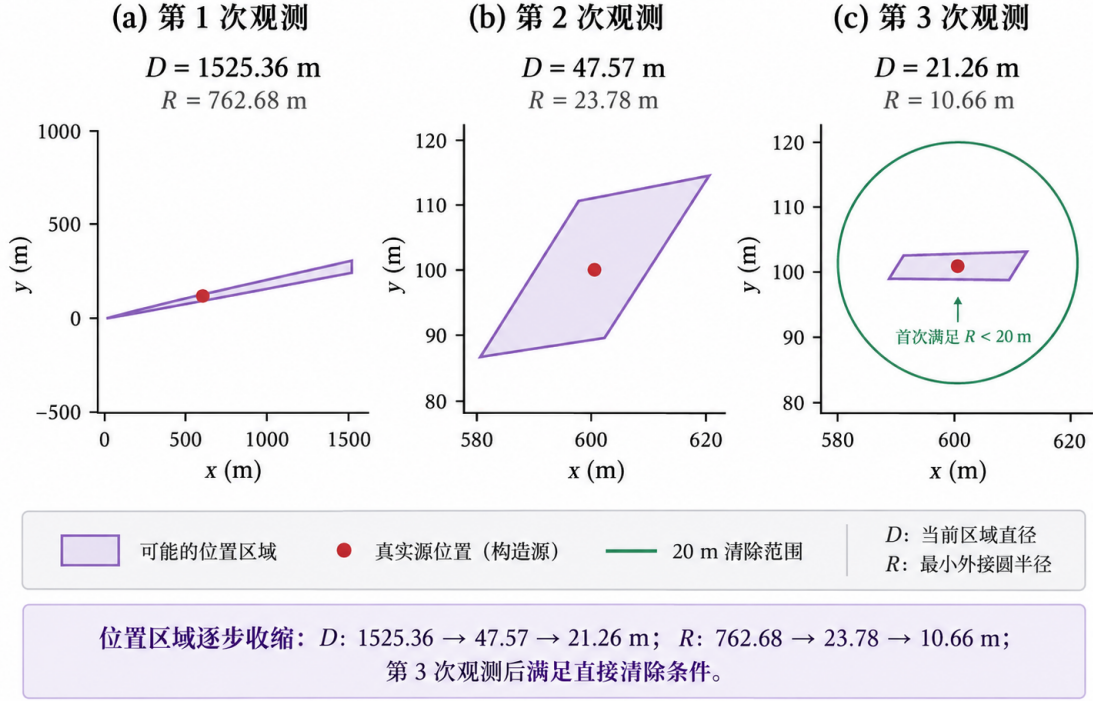


图 7: 位置区域收缩的几何算例。各子图分别放大当前区域, 坐标尺度不同; 红点为构造源, 绿色圆表示 20 m 清除范围。图中 R 的数值按固定坐标轴包围盒中心的覆盖半径计算, 图内“最小外接圆半径”应按此口径理解; 实际策略按最新示向方向旋转包围盒。图示并非实测轨迹, 不表示每个源均需三次测量。

5.3.3 观测共享与任务排序

(1) **共享观测的选择条件。**主目标的一次局部测向返回正常示向度后, 在当前位置 p 依次检查其他已知未清频道 j 。仅对尚未在 p 检测、当前 $R_j > 20$ m 且满足 $R(P_j, p) < 1000$ m 的频道, 进一步判断观测的收缩价值。

由 P_j 在 p 所见的真实方位范围向两侧扩展误差半宽 δ , 得到可能返回的示向度区间; 当 $p \in P_j$ 时, 保守取完整 360° 。将该区间完整分为 16 段, 记第 ℓ 段的中点为 m_ℓ 、半宽为 h_ℓ , 构造后验外包

$$\tilde{P}_{j\ell} = P_j \cap W(p, m_\ell, \delta + h_\ell).$$

对每个非空 $\tilde{P}_{j\ell}$, 沿 m_ℓ 方向计算包围盒中心 $\tilde{q}_{j\ell}$ 。仅在满足

$$\max_{\ell: \tilde{P}_{j\ell} \neq \emptyset} R(\tilde{P}_{j\ell}, \tilde{q}_{j\ell}) \leq \max(20, R_j/2)$$

时执行共享检测。对该段内任意实际读数 θ , 有 $W(p, \theta, \delta) \subseteq W(p, m_\ell, \delta + h_\ell)$, 因此判据覆盖全部可能读数。它保证每一种后验均存在满足阈值的覆盖圆, 但不保证按实际读数重新旋转的包围盒半径严格减半, 也不直接保证整局省时。

共享结果用于收缩其他源的区域, 近场反馈则可顺便清除该源。随后继续完成当前主目标的服务, 未补测或尚未清除的源留在任务集合中。普通搜索站只检测未知频道, 不重复扫描全部已知源。

(2) **任务访问顺序。**将未完成搜索站和已知未清源组成任务集合

$$\mathcal{J}_t = \{\text{scan}(k) : k \in \mathcal{S}_t\} \cup \{\text{serve}(c) : c \in \mathcal{D}_t \setminus \mathcal{C}_t\}.$$

搜索任务的代表点为站点坐标，源服务任务的代表点为当前 q_{co} 。记这些点为 $z_1, \dots, z_K$ ，访问顺序为排列 π ，则滚动排序采用的开放路径目标为

$$L_t(\pi) = \|p_t - z_{\pi_1}\| + \sum_{i=1}^{K-1} \|z_{\pi_i} - z_{\pi_{i+1}}\|.$$

借鉴旅行商问题的局部交换改进思想^[6]，本文按以下步骤近似求解开放路径问题：分别以各任务为首项，按最近邻规则生成完整路线；对每条路线用 2-opt 反转局部访问顺序，只接受严格缩短 L_t 的调整，直至不能继续改善；在所得路线中选择最短者，并执行其首项任务。由于任务结束后不要求返回起点，目标中不包含返程边。

完成整站必要扫描或当前源的定位清除后，再依据最新反馈重建 $\mathcal{J}_t$ 并排序，如图 8 所示。任务内部的观测更新位置区域，但不立即切换主目标。 L_t 仅计算任务代表点间的移动距离，不完整计入源服务内部行程、检测费用和提前发现第 16 个源的价值；因此该求解过程是整局时间目标的启发式近似。

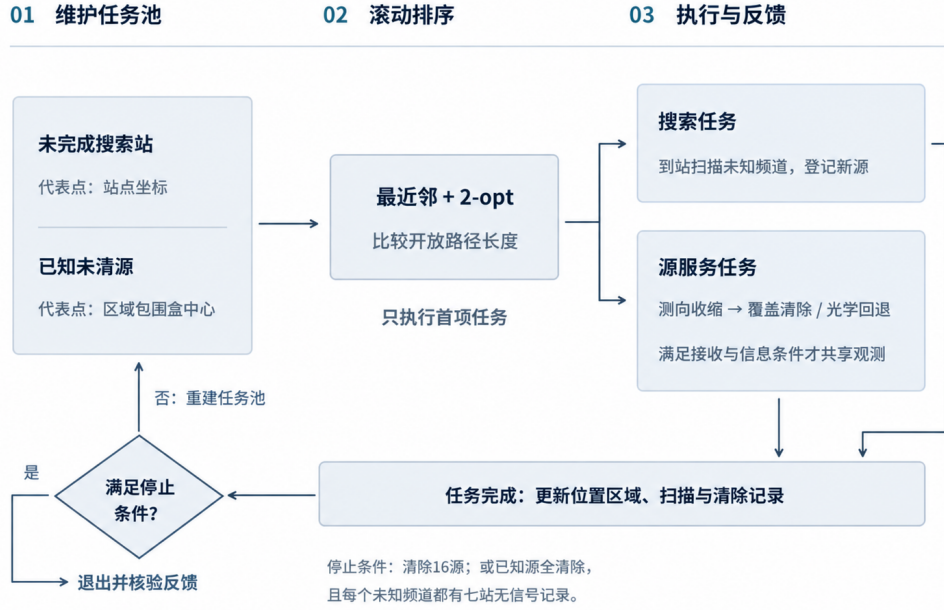


图 8: Q3 搜索与服务的调度流程。两类任务进入同一任务池，按代表点之间的开放路径长度排序，每次只执行首项任务。搜索与源服务为两种执行分支；共享观测须满足接收与信息条件，清除依据 20 m 覆盖条件或有限光学回退。任务完成后更新记录、核验停止条件，未满足时重建任务池并重排。

图 9 以三源教学场景示意交错访问，整局收益由第 6 章验证。

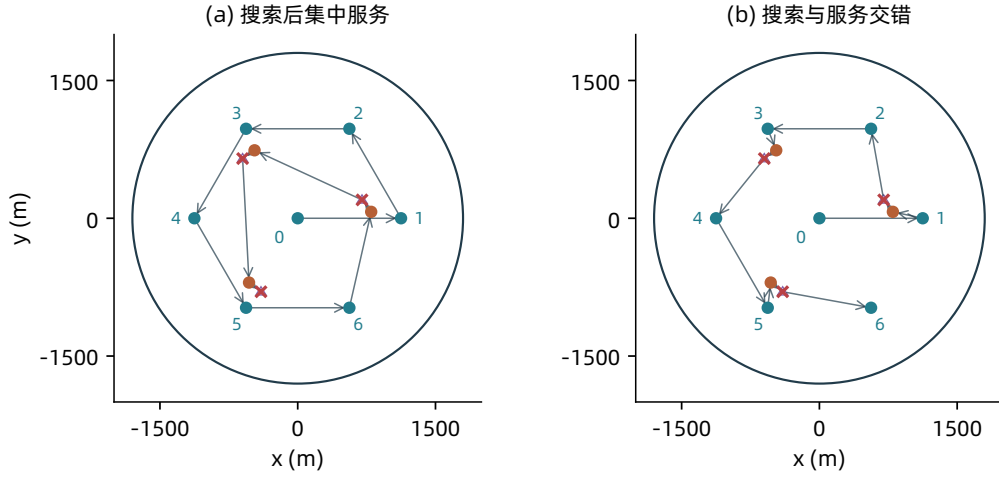


图 9: 三源教学场景中的任务顺序对比。蓝点为七个搜索站，橙点为后续测向点，红叉为构造源，紫色区域表示定位任务。左图先搜索后服务，右图交错执行；轨迹和区域均为流程示意，源数不符合正式场景要求，也不代表实验基线或性能结果。

5.3.4 搜索站布设与算法流程

(1) 七站发现覆盖。固定搜索站为原点和半径 1125 m 圆周上的六个等角站，其坐标可写为

$$s_0 = (0, 0), \quad s_{k+1} = 1125(\cos(k\pi/3), \sin(k\pi/3)), \quad k = 0, \dots, 5.$$

原点覆盖半径 900 m 的内圈。外圈任一点与最近圆周站的角距不超过 30° ；到该站的距离平方关于源的径向距离为凸函数，在 $[900, 1800]$ 上的最大值可由两端点控制。因此整个源域到最近站的距离满足

$$\max \left\{ 900, \max_{r \in \{900, 1800\}} \sqrt{r^2 + 1125^2 - 2r \cdot 1125 \cos 30^\circ} \right\} \approx 999.11 < 1000 \text{ m}. \quad (11)$$

每个全向源因而至少能在一站被接收。若一个未知频道在七站均无信号，就可排除圆域内存在该频道源的可能。1125 m 是在保持覆盖的同时缩短固定搜索行程的选择，与历史 1500 m 方案的比较见附录 B.3。

(2) 停止条件。记 $\mathcal{N}_c$ 为未知频道 c 获得无信号反馈的搜索站编号集合。任务仅在以下任一条件成立时完成：

$$|\mathcal{C}_t| = 16, \\ \text{或 } \mathcal{C}_t = \mathcal{D}_t, \quad \mathcal{N}_c = \{0, \dots, 6\} \quad \forall c \in \{1, \dots, 20\} \setminus \mathcal{D}_t.$$

第一种条件利用源数上限，第二种条件结合已知源清除记录和未知频道的覆盖排除。发现 16 个频道时可以取消剩余搜索，但仍需清除全部已知源；发现 10 个源或任务集合暂空均不足以退出。

(3) 联合算法步骤。

1. 初始化。从原点建立七站任务、频道记录及发现、清除集合。

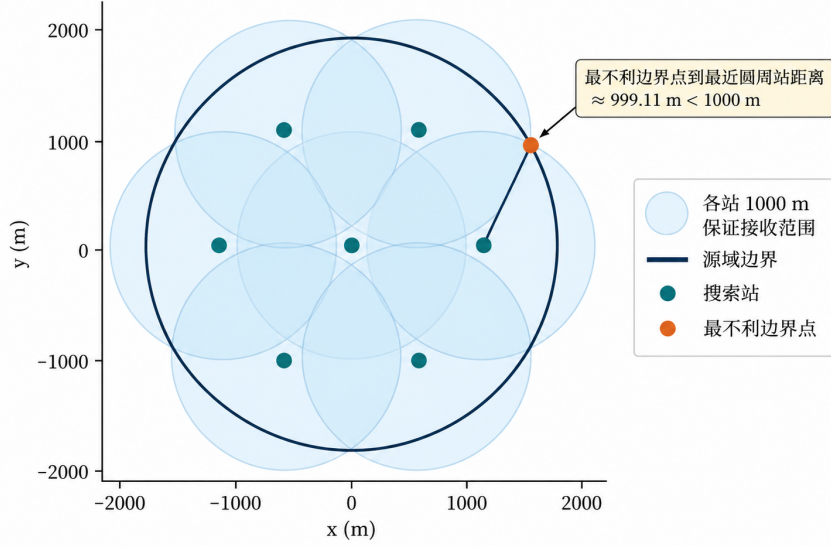


图 10: 七站对全向源的连续发现覆盖示意。浅蓝圆盘为各站 1000 m 的保证接收范围，深色圆为源域边界，橙点标示最不利边界位置。距离约 999.11 m 的数值由式 (11) 计算，示意坐标不用于量取距离；穿插源服务任务时仍保留各频道的实际扫描记录。

2. **停止核验。**满足前述条件则退出并检查反馈；否则重建任务。发现 16 频道后取消未知搜索。
3. **执行首任务。**搜索站只测未知频道，发现 16 源立即结束该站未知扫描；源服务按视差、中心测量及回退规则执行，正常测向后可插入合格共享观测。
4. **更新。**按反馈更新位置、清除及扫描记录，再回到步骤 2；响应矛盾、回退失败或超时报未完成。

七站保证发现，区域更新保留真源，有限定位与光学覆盖保证清除。每个外层任务完成一站或清除一源，至多 7 项搜索与 16 项服务；2-opt 在有限排列中严格缩短路线并终止。有限完成依赖题面模型、合法反馈与准确几何，不等于任意时限内完成。

5.4 问题四：混合源的搜索定位与清除

第四问中，全向源与定向源并存，源的类型和发射方向均未知。沿用第三问的频道状态与任务表示，对所有源采用兼容定向发射的位置区域更新规则；通过近距测站凸包保证未知源能够被发现，并以交会测向和有限光学覆盖清除已知源。搜索与源服务共同排序，总时间仍按式 (2) 计算。

5.4.1 混合源的位置区域模型

沿用第三问的发现集合 $\mathcal{D}_t$ 、清除集合 $\mathcal{C}_t$ 及未完成站集 $\mathcal{S}_t$ 。各频道维护位置区域、首测示向度及有效观测；正常方位或近场反馈确认发现，成功清除反馈才更新 $\mathcal{C}_t$ 。

设首次正常测向的位置为 S_c 、读数为 $\theta_c^{(0)}$ ，定义首测坐标基

$$u_c = (\cos \theta_c^{(0)}, \sin \theta_c^{(0)})^T, \quad v_c = (-\sin \theta_c^{(0)}, \cos \theta_c^{(0)})^T.$$

首次接收给出 1500 m 的源距上界。以 $\hat{B}_{64}(0, 1800)$ 表示源域的 64 边外切近似，构造首测扇区的外包矩形及初始位置区域：

$$\mathcal{R}_c = \{S_c + \xi u_c + \eta v_c : 0 \leq \xi \leq 1500, |\eta| \leq 1500 \sin \delta\},$$

$$P_c^{(0)} = \widehat{B}_{64}(0, 1800) \cap \mathcal{R}_c \cap W(S_c, \theta_c^{(0)}, \delta).$$

程序取 $\delta = 1.005^\circ$ ，并在边界处保留数值外扩余量。后续检测返回正常方位时继续取交，返回无信号时保留区域：

$$P_c^+ = \begin{cases} P_c \cap W(p, \theta, \delta), & \text{得到正常方位,} \\ P_c, & \text{无信号.} \end{cases}$$

合法观测因而维持 $g_c \in P_c^+ \subseteq P_c$ 。近场反馈表示源距不超过 5 m 且当前可接收，直接在当前位置尝试光学清除并核验成功反馈。

图 11 说明无信号分支为何不能删去附近圆盘：测点距源仅 350 m，却位于发射背面。算法不预先识别全向或定向类型，也不估计发射轴；全部频道使用上述更新方式，后续发现与清除保证同时适用于两类源。

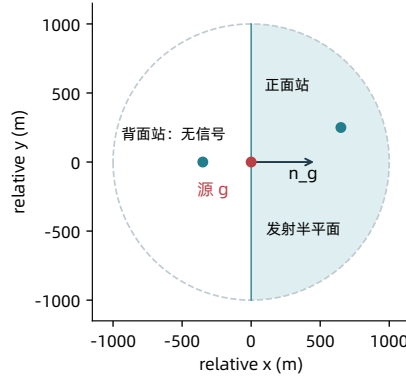


图 11: 定向源的无信号反例。测站位于源的发射背面，即使距离小于 1000 m 也无法接收。无信号时保留位置区域，光学清除仍只受 20 m 距离条件限制。

5.4.2 搜索站布设与连续覆盖

(1) 任意发射方向下的发现条件。设固定站集为 $\mathcal{V}$ ，与未完成站编号集合 $\mathcal{S}_t$ 区分。根据最小接收半径，要求对连续源域内任意位置 g 和任意发射方向单位向量 n ，均有

$$\forall g \in \Omega, \forall n \in \mathbb{S}^1, \exists s \in \mathcal{V} : \|s - g\| < 1000, \quad n^\top(s - g) \geq 0.$$

严格距离不等号用于构造接收裕量，发射半平面则按题设包含边界。该条件保证定向源能够被发现，也保证可见范围更大的全向源能够被发现。

近距凸包引理。若对源位置 g ，存在一组测站 $W_g \subseteq \mathcal{V}$ 满足

$$g \in \text{conv}(W_g), \quad \max_{w \in W_g} \|w - g\| < 1000, \quad (12)$$

则任意发射方向均至少可被一站接收。设 $g = \sum_i \lambda_i w_i$ ，其中 $\lambda_i \geq 0$ 、 $\sum_i \lambda_i = 1$ 。若全部站均不可见，则 $n^\top(w_i - g) < 0$ ；加权求和得到

$$0 = n^T \left(\sum_i \lambda_i w_i - g \right) = \sum_i \lambda_i n^T (w_i - g) < 0,$$

矛盾。因此至少一个近距站位于发射正面或边界。 W_g 仅用于证明，策略无须知道真实源的位置。

将目标域划分为三角网 $\mathcal{T}$ ，若满足

$$\Omega \subseteq \bigcup_{\tau \in \mathcal{T}} \tau, \quad \max_{\tau \in \mathcal{T}} \max_{a, b \in V(\tau)} \|a - b\| < 1000,$$

则每个源所在三角形的三个顶点均可作为见证站。三角形内任一点到一个顶点的距离不超过最长边，故连续发现要求转化为三角网覆盖和有限边长核验。

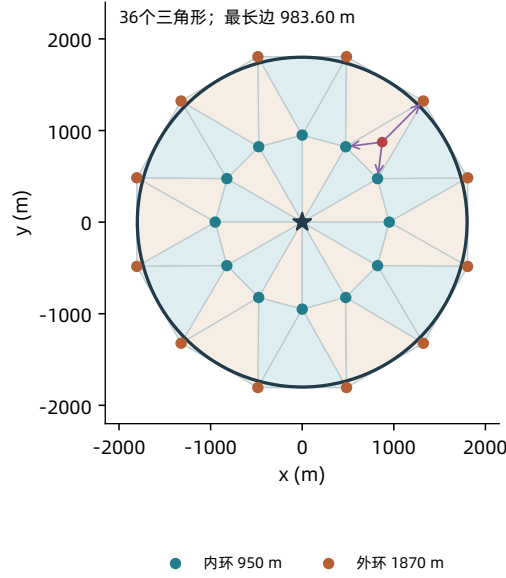


图 12: 25 站双环搜索构造。中心站与内、外环分别位于 0、950 和 1870 m 处，两环错开 15° ，连接形成 36 个三角形。最长边小于 1000 m，外环正多边形包围目标圆域，从而对任意源位置和发射方向提供发现保证。

(2) 双环构造。取中心站、内外两个正 12 边形环，半径分别为 950、1870 m，错开 15° ：

$$\begin{aligned} s_0 &= (0, 0), \\ a_k &= 950(\cos(k\pi/6), \sin(k\pi/6)), \\ b_k &= 1870(\cos(k\pi/6 + \pi/12), \sin(k\pi/6 + \pi/12)), \quad k = 0, \dots, 11. \end{aligned} \tag{13}$$

下标按模 12 连接 (s_0, a_k, a_{k+1}) 、 (a_k, a_{k+1}, b_k) 和 (a_k, b_{k-1}, b_k) ，得到 36 个三角形。最长边 983.60 m 小于 1000 m，外环内切半径 $1870 \cos 15^\circ = 1806.28$ m 大于 1800 m，因此三角网覆盖源域并满足近距凸包条件。图 13 分别示意局部可见性和全局包围。

对等站数正双环构造，外环须满足 $r_o \cos(\pi/m) \geq 1800$ 及 $2r_o \sin(\pi/m) < 1000$ ，推出每环至少 12 站。该下界只适用于此构造族，25 站不是一般布局的最少站数证明；半径选择与边长核验见附录 E.4。

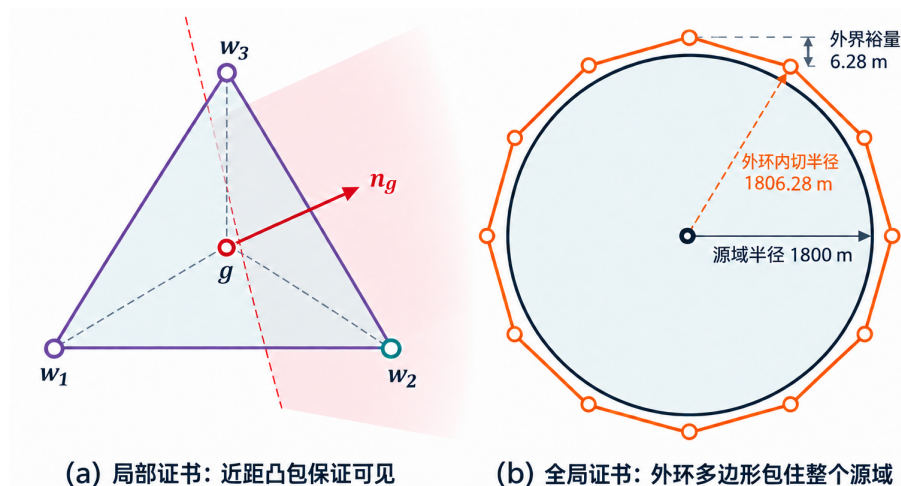


图 13: 定向源发现保证的局部与全局几何示意。(a) 源位于近距站的凸包内, 任意发射方向至少有一个站可见; (b) 外环正十二边形包围半径 1800 m 的源域, 内切半径为 1806.28 m。图不按比例绘制; 内切半径指圆心到边的垂距, 顶点到圆心的外环半径为 1870 m。

(1) **局部中心与补测。** 设位置区域 P_c 的顶点为 $x_1, \dots, x_v$, 取最长顶点对的中点及其覆盖半径:

若 $\rho_c < 20$ m, 前往 m_c 即可保证清除; 程序使用 $\rho_c < 20 - 10^{-5}$ m 以保留余量。此处对全部顶点核验覆盖, 不能将半径直接取为 $D(P_c)/2$ 。

$$q_c^\pm = m_c \pm 300v_c.$$

(2) **光学网格及有限清除。**沿首测基 u_c, v_c 取 P_c 的外包矩形, 投影端点为 $\ell_e = \min_{x \in P_c} e^\top x$, $h_e = \max_{x \in P_c} e^\top x$ 。各方向划分为 $n_e = \max(1, \lceil (h_e - \ell_e)/28 \rceil)$ 等份, 使格边 $d_u, d_v \leq 28$ m; 从近端开始按蛇形访问格心, 成功即停止 (图 14)。格心公式见附录 E.5。

真源始终在区域内, 故必有一格覆盖它, 且光学清除不受方向限制。初始矩形至多为 $1500 \times 52.62 \text{ m}$, 网格上界为 $\lceil 1500/28 \rceil \lceil 52.62/28 \rceil = 108$ 格; 有效测向可缩小范围。该数为初始访问上界, 实际可提前清除, 失败尝试的移动与 3 s 光学费用均计入总时间。

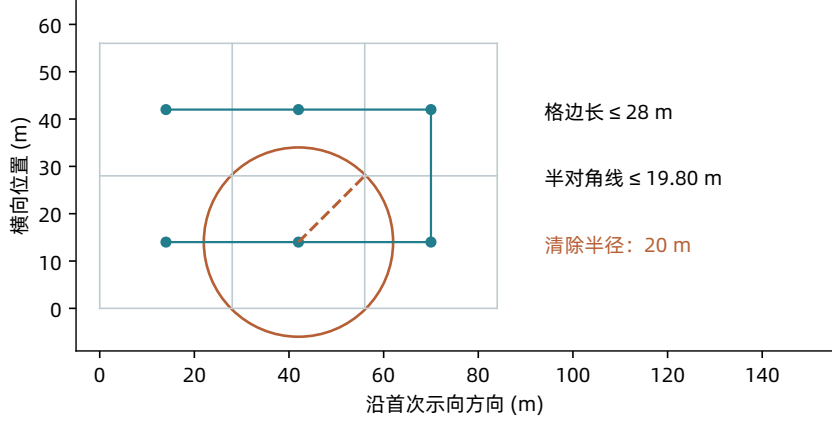


图 14: 有限光学网格。每格两边不超过 28 m, 因此格心的 20 m 清除圆覆盖整个格。图示六格局部区域及蛇形访问顺序, 实际格数由当前位置区域决定, 清除成功后停止访问。

5.4.4 搜索与清除的滚动调度

(1) **任务代表点与访问顺序。**沿用 5.3.3 的搜索—源服务任务集合及开放路径目标 $L_t(\pi)$ 。搜索任务的代表点为固定站坐标, 源任务的代表点改用位置区域的面积重心

$$z_c = \frac{1}{\text{area}(P_c)} \int_{P_c} x \, dx;$$

面积退化时取 m_{c_0} 。 z_c 只用于估计任务之间的路程; 执行源服务时, 补测和清除仍按上一节的 m_c 及覆盖判据确定, 不要求先访问 z_{c_0} 。

从当前位置出发, 以最近邻规则生成一条开放路线, 再作至多 50 轮 2-opt 调整, 每轮选择使路径缩短最多的片段反转; 无进一步缩短时提前结束。仅执行所得路线的首项完整任务, 完成整站扫描或该源清除后再更新并排序。该规则近似优化代表点间的行程, 未完整计入源服务内部移动、检测费用和提前发现第 16 源的价值, 其整局效果由 6.3 节评价。

(2) **必要频道与观测筛选。**对尚未完成的搜索站, 所有未知频道均需检测; 已清除频道不再检测。对于已知未清频道 c , 若 ρ_c 已满足直接清除阈值, 保留其清除任务而跳过本站测向。否则, 以 p 指向 m_c 的方位为参考, 将 P_c 各顶点相对于 p 的方位差归一到 $[-\pi, \pi)$, 用其最大值与最小值之差定义顶点方位跨度 $\omega_c(p)$ 。

以 20 m 为理想清除阈值, 本站需测频道可写为

$$\begin{aligned} \mathcal{Q}_t(p) = & (\{1, \dots, 20\} \setminus \mathcal{D}_t) \\ & \cup \{c \in \mathcal{D}_t \setminus \mathcal{C}_t : \rho_c \geq 20, \omega_c(p) \geq 3^\circ\}. \end{aligned}$$

程序在清除阈值中采用前述数值余量, 并优先检测当前频道, 其余按编号顺序进行。正常方位更新相应区域, 近场反馈可当场清除; 无信号仅保存记录。3° 规则用于减少低信息量检测, 被跳过的已知源始终保留为源服务任务。第四问在搜索站利用这些观测更新多个频道的区域, 局部补测则只服务当前目标。

5.4.5 完整算法与求解结果

(1) **停止条件与算法流程。**将 25 个固定站编号为 0—24, 记 $\mathcal{N}_c$ 为仍未知频道 c 取得无信号反馈的站编号集合。任务完成条件为

$$|C_t| = 16,$$

$$\text{或 } C_t = D_t, \quad \mathcal{N}_c = \{0, \dots, 24\} \quad \forall c \in \{1, \dots, 20\} \setminus D_t.$$

前者利用源数上限，后者同时核验已知源清除与未知频道排除。单个无信号反馈不足以删除附近圆盘，但一个未知频道在具有连续发现保证的全部站点均无信号，则可排除该频道源存在。

算法循环如下：

1. 从原点建立 25 站任务及空的观测、发现和清除记录。
2. 检查停止条件；否则按 5.4.4 排序并执行首项完整任务。已发现 16 频道时取消未知搜索。
3. 搜索任务扫描必要频道，源服务依次检查直接清除、两侧补测及光学网格；依据反馈更新记录后回到步骤 2。数量达到 16 时，在整站结束后的下一轮规划中取消搜索。
4. 区域为空、保证清除失败、网格耗尽或接口异常、超时，均报未完成，不能以任务池为空替代停止核验。

连续覆盖保证发现，区域保真与有限光学覆盖保证清除。外层至多完成 25 项搜索和 16 项源服务，路线调整与格心访问也有限；完成保证依赖题面模型、合法反馈与准确几何，实际执行还受数值与时限约束。

算法输出任务访问序列、频道清除记录、时间分解及停止依据。构造核验表见附录 E.4，模块接口见附录 A.3；25 站布局、重心排序与观测筛选共同构成推荐方案，其整体收益由 6.3 节评价，正式结果见 6.4 节。

6 模型分析与评价

本章结合自建配对实验、正式测试记录和几何证明，从任务完成情况、耗时表现及适用条件三个方面评价模型。

6.1 实验设计与评价方法

实验与独立单位。性能比较来自冻结的自建完整流程记录，同布局、同误差条件下配对；生成器不代表官方源分布。候选在开发后冻结源码、配置及规则，再生成新布局验证。Q3 冻结基线为历史第十九轮组合；Q4 历史比较以 31 站初始方案为基线，动态比较以成熟 25 站方案为基线，两个阶段的改善不能相加。

表 3: 自建实验组、独立布局与用途

证据组	独立布局	每布局条件	用途
Q2 候选复核	不适用	36,421 个几何场景/候选	采样定位直径
Q3 最终验证	120	4 误差场 ×2 舍入环境	联合方案与消融
Q4 历史验证	96+96, 互不重叠	1 误差场; 两批各 1 种环境	31 站、25 站及删减对照
Q4 动态验证	120	4 误差场 ×2 舍入环境	25 站与 4 个候选

统计口径。主要效应为 $100(1 - \bar{T}_{\text{candidate}}/\bar{T}_{\text{base}})$ ，正值为省时；图 16、20 以时间变化表示，负值为省时。均值、分位数及极值按运行等权计算，区间以独立布局为重采样块，保留块内条件：Q3 按 6 类分层重采样 20,000 次；Q4 历史两批各 10,000 次，动态验证按 6 类分层 10,000 次^[7]。条件重放不增加独立样本数，区间只反映自建生成器下的采样不确定性。分位数、图形口径与布局类别见附录 E.6。

选择与核验。后续候选事先要求完整率不降、平均降时至少 5%、P95 不恶化；最终验证不再调参。失败及超时预设计入 $\max(T, 360000)$ s，本批未触发，慢例全部保留。实现检查覆盖退化几何、响应重放、清除计数与停止条件，并以通用求解器交叉复核 Q2；规模见附录 B.6，检查不替代形式证明。正式测试另见 6.4 节，补充开发不并入原验证。

6.2 问题三：实验结果与机制分析

6.2.1 Q3 的试错：集合变紧并不会自动省时

开发中，单纯沿集合中心逼近比冻结基线慢 11.44%，加入视差和共享仍无平均优势；共同排序后组合才降时 12.83%。单方位光学覆盖、接收边界测距均增加耗时，共享还须逐频道检查接收条件。这说明局部定位改进需与任务顺序联合评价；开发及单例诊断见附录 B.4。

6.2.2 Q3 的锁定验证：平均更快，但保留少量明显慢例

120 个新布局、两种环境中，联合方案均完成全部运行。主模型平均省时 10.27%，95% 区间 [9.14%,11.35%]；舍入压力下省时 10.07%，区间 [8.95%,11.15%]。各行舍同一 120 布局的 4 次条件运行，共 480 次；完整耗时如下。

表 4: Q3 锁定验证的完整流程结果

误差环境	方案	均值/s	中位数/s	P95/s	本批最大/s	完成局数
有界读数	冻结基线	3323.52	3631.26	4517.29	5111.40	480/480
有界读数	联合方案	2982.18	3234.21	3823.72	4217.81	480/480
舍入压力	冻结基线	3319.49	3616.01	4505.79	5086.03	480/480
舍入压力	联合方案	2985.19	3234.30	3827.73	4217.91	480/480

图 15 概括总时间分布，图 16 保留六类布局的全部配对结果；平均改善并不意味着逐例占优。

主模型 435 次更快、45 次更慢，舍入压力下 433 次更快、47 次更慢。最差为 16 源聚集布局：基线 1063.06 s、联合方案 2058.36 s，慢 93.63%。原点可接收 15 源，联合方案却在发现最后一源前访问更多站，增加 791.31 s 移动、185 s 检测与 37 s 切频。该例揭示距离排序可能错失第 16 源带来的提前停止机会；逐动作诊断见附录 E.7，图 16 按布局合并后最高点不等于单次退化 93.63%。

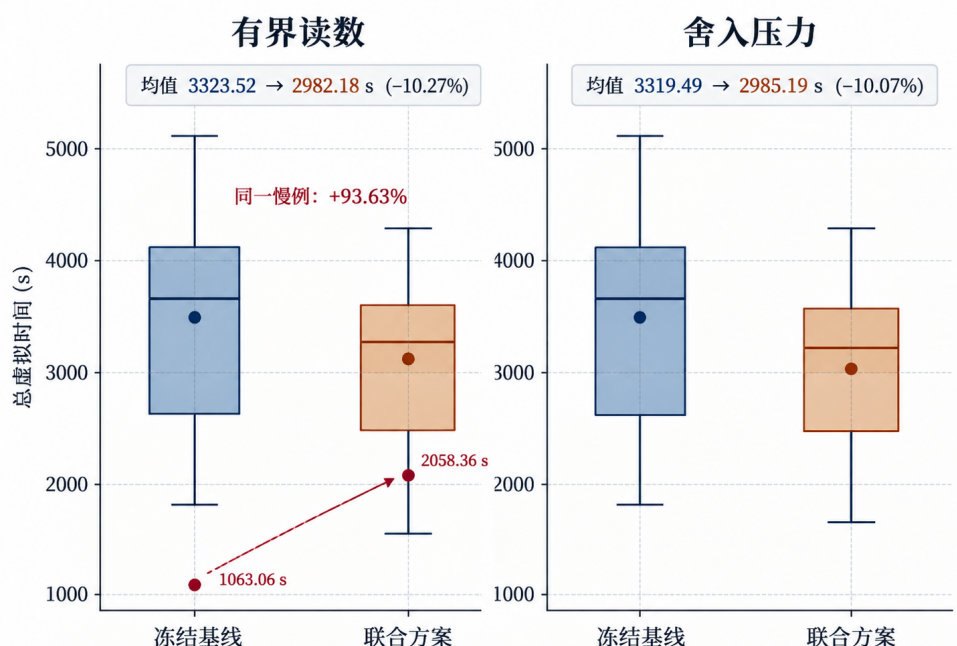


图 15: Q3 两种误差环境的总时间分布比较示意。每组 480 次运行，对应 120 个独立布局；左图红线标出同一慢例从 1063.06 s 增至 2058.36 s，退化 93.63%。图中部分统计点位为示意，精确均值和分位数以正文表格及源数据为准；箱体不表示效应置信区间。

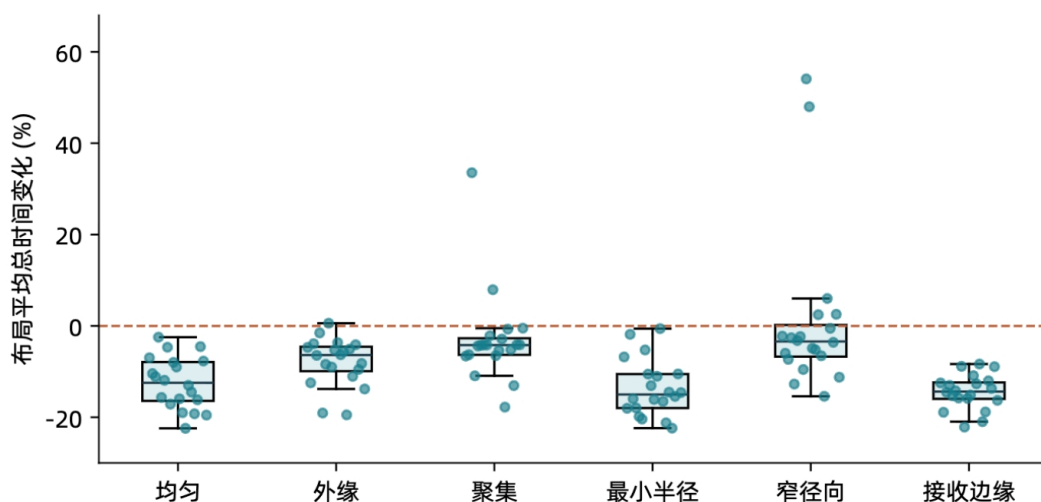


图 16: Q3 主模型的布局配对结果。纵轴为联合方案相对基线的总时间变化，负值表示更快。每点合并同一布局的 4 种误差，箱体为各类布局的四分位区间，中线为中位数；全部点叠加显示，不删除慢例。图中的点数 120 是布局数，不是 480 次条件重放。

6.2.3 Q3 为何更快：省下的移动支付了额外观测

图 17 按实际动作分解净收益：每局移动省 383.25 s，检测及切频多用 63.76、8.14 s，光学省 29.98 s，清除附加费用不变，净省 341.33 s。额外观测换来了更少行走。

联合方案 960 次运行无失败光学尝试、无网格回退；回退仍保留为有限清除保障。主模型逐局每源平均时间由 269.71 降至 242.21 s，舍入压力下由 269.36 降至 242.38 s。

图 18 以共同父方案逐项消融：删除目标原地再观测省 25.00 s；删除首次视差、完整行程规划或信息门控分别慢 243.72、37.20、27.00 s。“删除门控”仍共享观测，仅取消

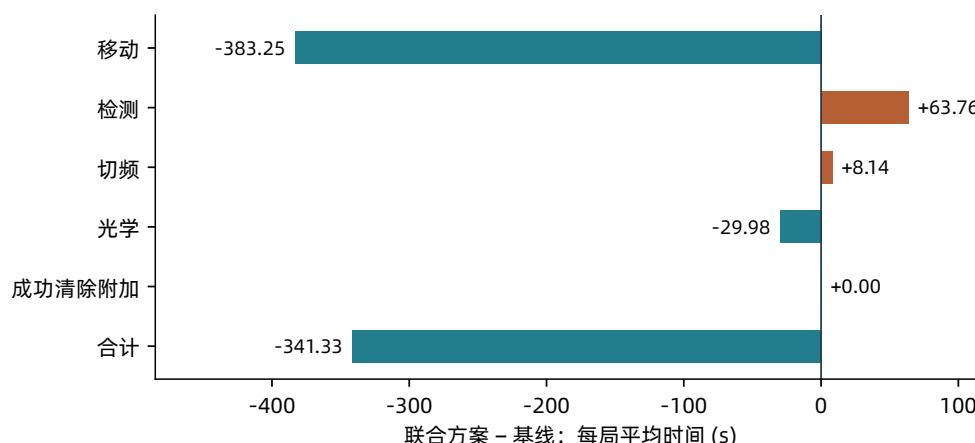


图 17: Q3 主模型的动作代价分解。数值为联合方案减去基线的每局平均时间，负值为节省。合计是前五项之和；不能再叠加搜索、定位等阶段耗时，否则会重复计费。

收缩价值筛选。各差值不可相加，父方案也不等于历史冻结基线；完整开关矩阵及 P95 见附录 E.8。

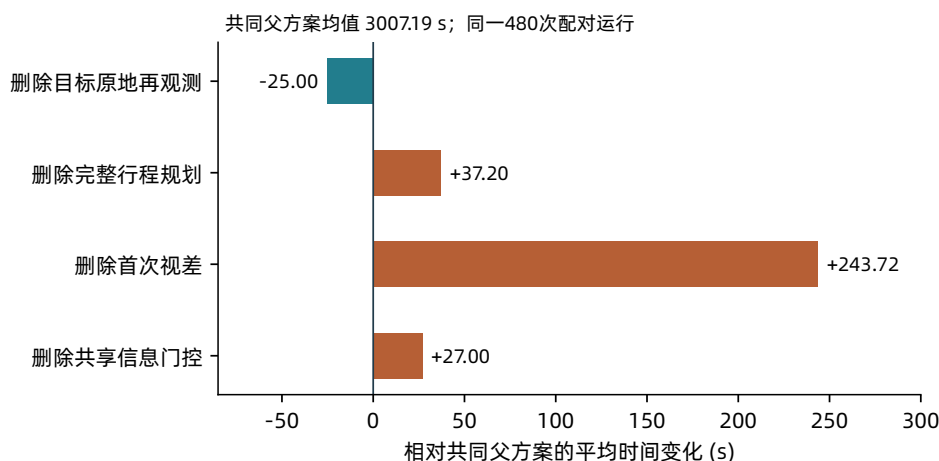


图 18: Q3 最终主模型的共同父方案消融。每行是在同一父方案上删除一项机制后的均值变化，使用同一 480 次配对运行；负值支持删除，正值支持保留。图为配对差的描述性均值，不单独宣称各行统计显著。

6.3 问题四：实验结果与方案比较

6.3.1 Q4 为何保留 25 站：覆盖成立只是第一道门槛

开发中的具体 21 站构造在 25 m 粗网格上无漏点，细化后却出现近站最大方向空隙 182.68°，可漏掉 180° 定向发射，说明粗采样不能替代连续证明。22 站构造虽通过连续证书，同批验证仍比 25 站慢 5.24%、3.39%；站数少不保证实际扫描少，案例见附录 E.9。

图 19 比较 31 站初始基线与 25 站完整方案。两批各 96 个独立布局，25 站平均时间为 6640.86、6694.69 s，分别省 41.83%、41.24%，配对 95% 区间为 [40.35%,43.24%]、[39.70%,42.79%]。各批 96 次均更快，所有比较方案均完整清除。

25 站的 P95 为 8694.62、9598.33 s，低于基线 13465.02、14367.42 s；删除重心排序后两批均慢超过 11%，删除低信息量观测筛选后慢约 4.6%。因此约 41% 收益属于布局、排序和观测筛选的整体，不能全归于减少 6 站。中位数、极值与代价细分见附录 E.9。

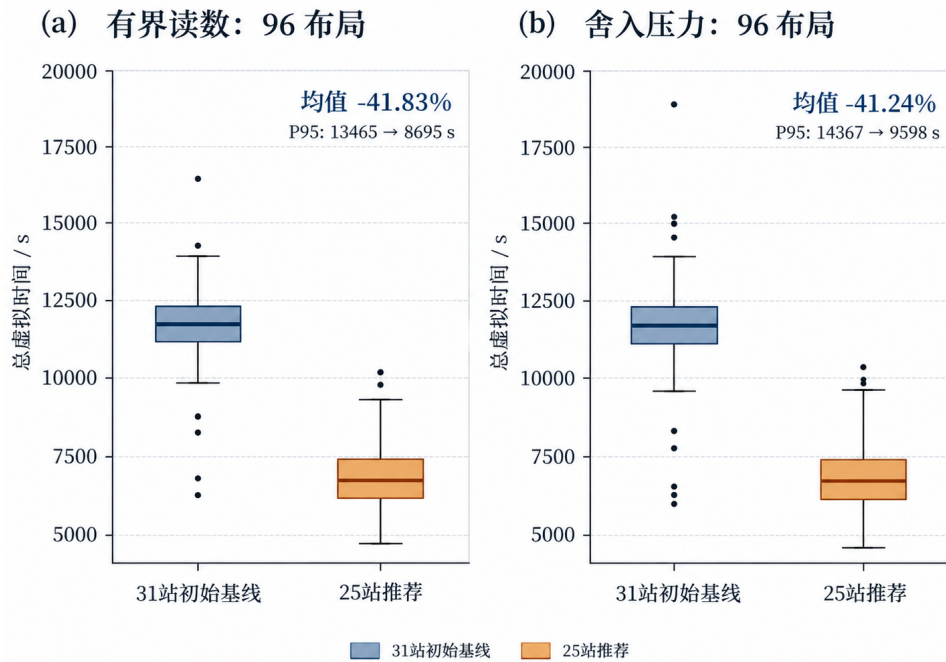


图 19: Q4 两批历史自建验证的时间分布比较示意。两批各 96 个互不重叠布局, 每批内部 31 站与 25 站方案配对; 图中标注均值降时及 P95 变化。箱体和点位仅概括分布形态, 精确中位数、分位数和极值以正文及源数据为准; 箱体不表示均值置信区间。

6.3.2 Q4 后续试错: 动态覆盖安全, 却没有足够的整局收益

进一步尝试以已到达位置替站, 或移动内环站缩短局部路线; 每次修改须独立重建三角网并通过连续覆盖核验, 预算不足则保留原站。组合开发省时 2.33%, 未达 5% 门槛, 随后在新布局上锁定验证。

五方案共 4800 次运行全部清除; 每方案为同一 120 布局 $\times 8$ 条件。动态组合平均 6719.67 s, 相对 25 站的 6744.38 s 省 0.366%, 95% 区间 $[-0.183\%, 0.921\%]$ 跨零, P95 仅降 0.394% (图 20)。组合 706 次更快、254 次更慢; 移动省 37.00 s, 但额外检测、光学及切频抵消部分收益, 净省 24.71 s。五方案完整统计见附录 E.10。

最差单例慢 33.69%: 移动首个内环站 103.88 m, 局部预测路程缩短 42.06 m, 实际扫描却从 7 站增至 19 站, 第 16 源发现推迟 2717.83 s。该例未发生动态替站, 不能归因于该操作。逐动作记录支持发现顺序变化的解释, 但无反事实实验分配单步因果损失。历史自适应定位虽省 1.931%, 仍未达 5% 门槛且最大值更高。因此保留原 25 站推荐, 动态覆盖仅作为可行但收益不足的扩展。

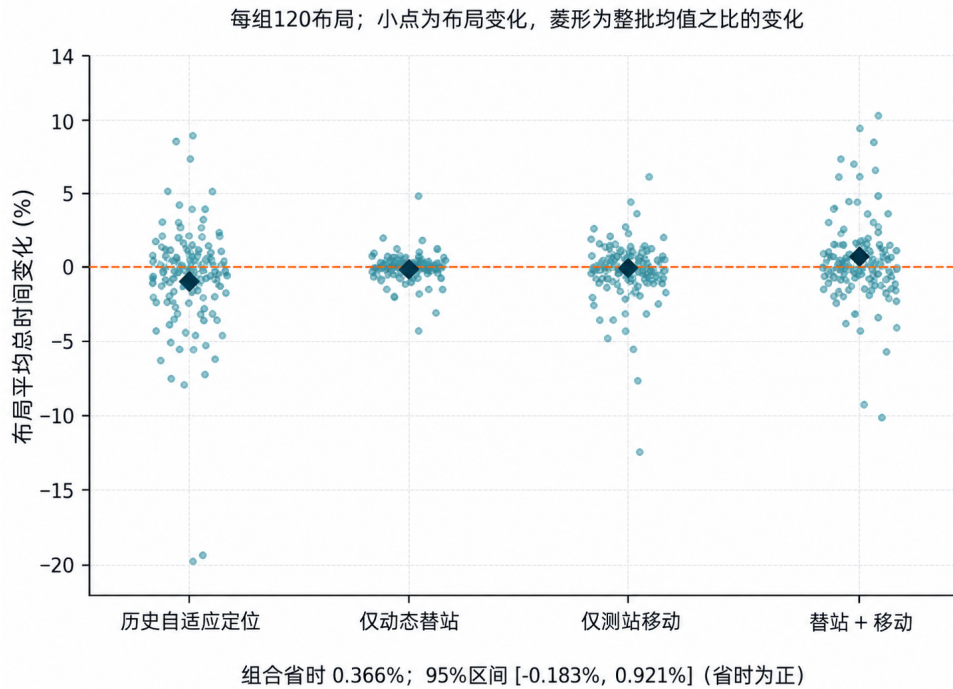


图 20: Q4 最新锁定验证的布局配对分布示意。每组对应 120 个布局，每个布局合并 8 种误差与舍入条件；纵轴为总时间变化，正值表示变慢，下方以省时为正报告效应区间。图中部分点位与核验统计存在偏差，组合菱形对应时间变化 -0.366% ，而非正值；精确配对结果以正文及源数据为准。

6.4 正式测试结果

2026 年 9 月 13 日，两问各完成三次正式测试。案例编码来自正式行为日志，清除数逐条核验成功响应；平均定位清除时间为退出时总虚拟时间除以成功清除数。原名日志与版本对应见附录 C。

第三问正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
5ER6-BK3Y-KHDQ-MUQT	10	295.56	0.434
VUK9-G6FH-ACSC-KFYG	16	243.03	0.682
3BEW-YK5X-AE2M-SJND	12	254.64	0.588

第四问正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
JHQ9-ZVTT-CGAR-GVTX	10	575.57	1.182
SXAN-9UWV-2X8Z-GZ58	15	388.30	1.691
CEBG-3EHE-MGUQ-H96W	11	578.52	1.146

程序运行时间为模拟器入场至主动退出的实时时间戳之差，与虚拟动作耗时分开。两问累计清除 38、36 源，三次每源平均时间的算术平均为 264.41、514.13 s，六次均满足停止判据并主动退出。Q3 无失败光学尝试，Q4 第二、三次有 5、6 次未命中，费用已计入。逐次总时间见附录 E.11。

Q3 配置及策略哈希对应联合方案；Q4 配置与 25 站布设对应推荐，未保存运行代码哈希，版本关联限于附录 C。客户端未获源总数，故不另算官方清除率。每问仅 3 例且无同案例基线，正式结果支持实际可运行性，不能用于推断自建实验约 10% 和 41% 的相对降时。

过程证据。图 21 上排来自 Q4 历史自建对照，首次全发现所需扫描站数中位数由 20、20.5 降至 15、18；这是读取真值后的事后指标，不替代停止核验。中排为 Q3 正式 38 源：31 源满足覆盖圆判据，7 源由近场反馈清除。下排的相邻清除点间耗时含搜索、定位、移动与光学动作；清除终点与退出时刻不同，后续核验时间见附录 E.11。

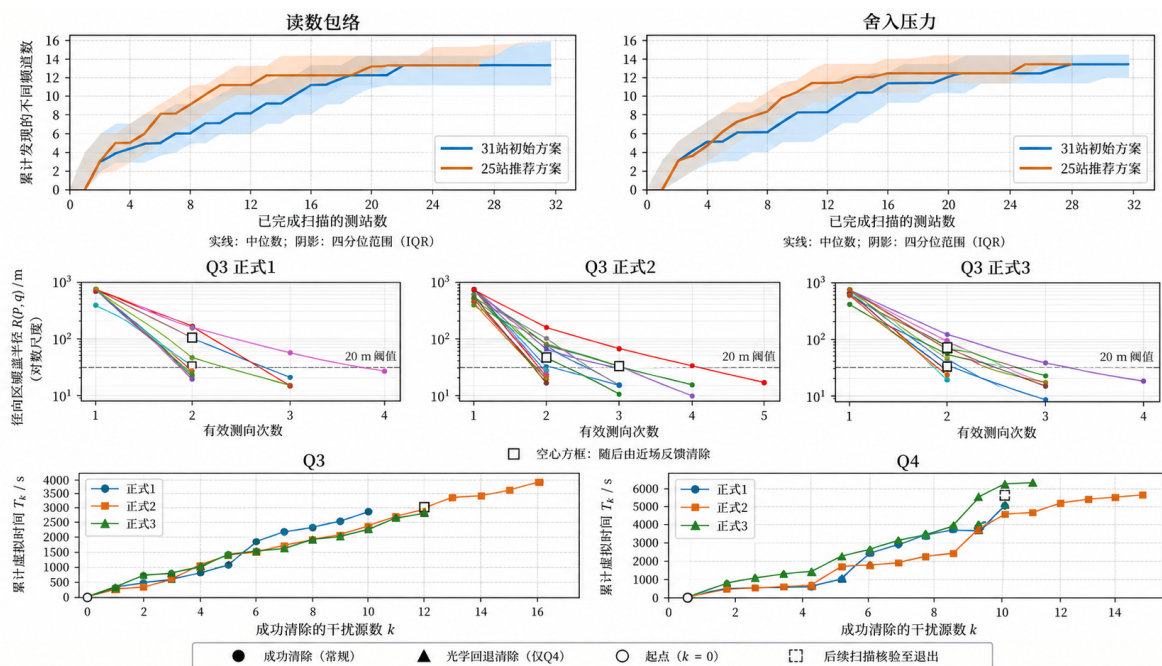


图 21: 发现、定位与清除过程的示意汇总。上排为 Q4 两批历史自建对照，每批 96 个配对布局；实线表示中位数，阴影表示四分位范围，端值延伸不代表继续扫描，25 站方案的实际预算仍为 25 站。中排为 Q3 三次正式测试的区域收敛，空心方框表示随后通过近场反馈清除。下排为两问各三次正式测试的清除与耗时关系，三角形表示光学回退清除。原图的阈值线位置及部分点位与核验记录有偏差，故仅用于过程示意；20 m 判据、清除终点和退出时间以正文及源数据为准。

6.5 模型优点与理论保证

位置集合将有界误差、接收和清除条件统一为可计算约束；七站覆盖与近距三角网分别提供全向、定向发现保证，有限光学网格补足失联后的清除。任务排序与观测共享在这些条件内改善效率。几何结论、按规则执行的条件性完成保证、配对实验的平均收益分属不同层次，不能相互替代。

6.6 模型局限与适用边界

证明依赖静态源、有界误差、半径下界及半平面发射；移动、遮挡或更窄发射扇区需重建覆盖。浮点外包与余量未经完整形式验证，旧审计器曾接受退化三角证书，补充核验后原 25 站构造仍有效，说明证明与检查器须分别验证。效率结论限于冻结自建比较；正式案例少、无配对基线，Q4 还缺运行代码哈希，不能据此推广相对收益。

6.7 改进方向

慢例共同指向发现顺序：找到第 16 源可取消剩余未知搜索，代表点距离目标尚未充分计入这种价值。后续需结合发现收益、源服务费用和结束位置排序，并在新布局上同时考察均值与慢例。Q3 补充开发的四种候选均未达选择条件（附录 B.5），故保留当前方案；自建源频率也不应直接作为官方发现概率。

未来可借鉴空间一致性信道模型的网格双线性插值思想^[8]，在保持测向误差有界及同点固定的条件下，构造不同相关尺度的误差场，检验定位与调度策略的稳定性。

7 结论

本文以测向楔形交建立位置区域，由顶点直径与覆盖半径区分定位精度和清除保证。第二问给出鲁棒接收、交会约束及 $(800, \pm 605)$ m 推荐点；第三问以七站覆盖、观测共享和滚动排序完成全向搜索；第四问以 25 站近距凸包和有限光学覆盖处理定向失联。

自建配对实验中，Q3、Q4 相对各自冻结基线平均省时约 10%、41%；正式测试各 3 次均满足停止判据并主动退出，累计清除 38、36 源。慢例及未采用的动态方案表明，局部精度、站数和代理路线长度均不能单独决定整局效率，后续应围绕发现顺序改进调度。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于选题与研究思路讨论、论文草稿整理及代码初步草拟。核心建模与分析由参赛队主导，AI 输出作为辅助参考，由参赛队负责取舍、修改与核验，详细使用情况见支撑材料。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题：无线电干扰源的快速自动定位与清除 [Z]. 2026.
- [2] 全国大学生数学建模竞赛组委会. 模拟器使用说明：2026 年 B 题附件 1[Z]. 2026.
- [3] 全国大学生数学建模竞赛组委会. 模拟器通信接口说明及编程指南：2026 年 B 题附件 2[Z]. 2026.
- [4] MILANESE M, VICINO A. Optimal estimation theory for dynamic systems with set membership uncertainty: An overview[J]. Automatica, 1991, 27(6): 997–1009. DOI: 10.1016/0005-1098(91)90134-N.
- [5] BOYD S, VANDENBERGHE L. Convex optimization[M]. Cambridge: Cambridge University Press, 2004.
- [6] CROES G A. A method for solving traveling-salesman problems[J]. Operations Research, 1958, 6(6): 791–812. DOI: 10.1287/opre.6.6.791.
- [7] EFRON B. Bootstrap methods: Another look at the jackknife[J]. The Annals of Statistics, 1979, 7(1): 1–26. DOI: 10.1214/aos/1176344552.
- [8] ADEMAJ F, SCHWARZ S, BERISHA T, et al. A spatial consistency model for geometry-based stochastic channels[J]. IEEE Access, 2019, 7: 183414–183427. DOI: 10.1109/ACCESS.2019.2958154.

附录

支撑材料文件清单

表内路径以支撑材料根目录为起点，列出各问的主要入口、策略和配置。运行时应保留完整目录结构及其调用的几何、路线和通信模块。

文件名称	文件内容
question1/solve.py	第一问测向区域求交、直径计算与圆覆盖判定
question2/strategy.py	第二问候选条件检查、网格搜索与场景评分
question2/round1_tradeoff.json	1.005° 误差包络下的候选点复核数据
question3/run_innovation_http.py	第三问联合方案的运行入口
question3/innovation/strategy.py	第三问集合定位、共享观测及任务调度
question3/innovation/best_candidate.json	第三问联合方案的固定配置
question4/run_http.py	第四问推荐方案的运行入口
question4/strategy.py	第四问搜索、定位清除及任务调度
question4/configs/recommended.json	第四问 25 站方案的固定配置
question3/requirements.txt; question4/requirements.txt	程序所需的 NumPy、SciPy 依赖

第一、二问的检查程序分别为question1/test_solve.py 与question2/test_strategy.py。第三、四问的原始正式日志另列于附录 C；完整实验动作记录及冻结源码按随文支撑材料保存。

附录内容索引

附录编号	内容
A	程序说明与核心算法流程
B	补充数据、开发比较与实现检查
C	正式测试日志及版本对应
D	AI 辅助使用说明
E	展开推导、完整统计与运行诊断

附录 A 程序说明与核心算法

A.1 第一、二问求解程序

第一问。 question1/solve.py 接收由检测点坐标 x 、 y 及示向度bearing_deg 组成的数据。函数bearing_halfplanes 将误差扇区转为半平面约束，solve_halfplanes 依次判断可行性和有界性，再枚举顶点并计算直径及直径圆覆盖结果。输出包括区域状态、顶点坐标、直径端点和覆盖判断。附录 B.1 给出正文三角形反例的输入；程序默认演示采用另一组边长 20 m 的三角形，复核正文时应使用附录数据。

第二问。 question2/strategy.py 在以首次示向为横轴的局部坐标系中求解。函数candidate 检查接收与交会条件，diameters 计算各场景的定位区域直径，search 完成网格搜索与局部加密，to_world 将局部偏移转换为全局测点坐标。默认入口使用 $\pm 1^\circ$ 误差，结果写入question2/results.json；附录 B.2 的 $\pm 1.005^\circ$ 候选表对应question2/round1_tradeoff.json。

A.2 第三、四问运行流程

第三问以question3.run_innovation_http 为入口，读取question3/innovation/best_candidate.json；第四问以question4.run_http 为入口，默认读取question4/configs/recommended.json。两问共用公开通信接口，主流程概括如下，完整实现见文件清单。

算法 A.1 第三、四问搜索定位与清除主流程（伪代码）

```
1 建立搜索站、频道记录与待清任务
2 发送 /enter，读取初始状态
3 循环：
4     若成功清除数为 16，或已知源均清除且未知频道覆盖完成：
5         发送 /exit，核验退出反馈，结束
6     按当前位置和已有观测重排任务，选择首项
7     若为搜索任务：
8         扫描本站必要频道，建立或更新位置区域
9     若为源服务任务：
10        检查 20 m 清除条件，否则执行测向或有限光学覆盖
11        第三问正常测向后，可插入通过筛选的共享观测
12    按实际反馈更新位置区域、扫描记录及成功清除集合
13    若反馈违反保证条件或接口状态无法确认：报告异常
```

两问的观测处理不同。第三问搜索站扫描未知频道，源服务期间可共享检测其他已知频道；第四问在搜索站对已知频道进行清除或观测筛选，局部定位只服务当前目标。第三问保证接收的补测若返回无信号，则报告异常；第四问补测允许无信号，并保留已有位置区域，必要时转入有限光学覆盖。

/measure 参数指定检测位置和频道，模拟器据此计入移动、切频及检测费用；/clear 仅在收到成功反馈后增加已清除源计数，有限网格中的未命中则继续遍历。发现 16 个频道可取消剩余未知搜索，但仍需清除全部已知源。

A.3 模块功能与计算量

表 A.1 给出主要模块的输入、输出及适用条件。各几何保证以前述物理模型、合法反馈和准确集合运算为前提。

表 A.1 算法模块的输入、输出与适用条件

模块	输入	输出	适用条件或限制
测向区域	位置、读数、误差界	楔形交集或异常状态	约束保留真实源，交集可能无界
第二测点	首测点与候选源域	安全候选测点	接收和交会的充分条件
全向搜索	源域、半径下界	七站及扫描记录	需完成必要频道扫描
混合源搜索	源域、半平面发射模型	25 站与三角网	需满足连续覆盖条件
区域更新	有效示向及已有区域	位置外包多边形	无信号须按源模型处理
光学清除	位置外包、20 m 阈值	清除点或有限网格	需实际执行并核验成功
在线调度	搜索任务与源服务任务	下一项任务	以行程为代理目标的启发式排序

对 n 次角度观测，边界交点枚举及约束筛选为 $O(n^3)$ 。若当前区域有 v 个顶点，加入一个半平面或计算指定点的覆盖半径为 $O(v)$ ，枚举顶点对直径为 $O(v^2)$ 。对 K 个任务，最近邻建路为 $O(K^2)$ ，一轮 2-opt 枚举 $O(K^2)$ 个候选区段；第三问逐候选重算路径长度，单轮为 $O(K^3)$ ，第四问采用边差评价，单轮为 $O(K^2)$ 。多起点和重复改进还会增加计算量。有限光学网格的执行费用取决于区域大小及遍历距离，有限性只给出终止依据。

附录 B 补充数据与实验结果

B.1 等边三角形反例的测向数据

表 B.1 给出第 5.1.3 节反例的三组测向输入，误差半角为 1° 。

表 B.1 等边三角形定位区域的测向输入

检测点	横坐标/m	纵坐标/m	示向度/ $^\circ$
S_1	-1000	0	1
S_2	539	$-500\sqrt{3}$	121
S_3	519.5	$519.5\sqrt{3}$	241

三条扇区下边界分别沿边长 39 m 的等边三角形三边，第三顶点相对相应边的偏角为 1.89746° ，小于扇区宽度 2° ，因此上边界不再截去三角形。取三角形中心为真实源，三个测向误差的绝对值均约为 0.36731° ，测点距源均约 1019.562 m；取接收半径 1200 m 即可满足题面条件。

B.2 第二检测点候选的复核结果

表 B.2 采用 1.005° 保守误差包络，每个候选均复核 36,421 个场景，镜像偏移的指标相同。费用仅包括从首测点移动到候选点并检测一次，不含后续定位与清除。

表 B.2 第二检测点候选的精度与时间比较

偏移 (a, b)/m	移动距离/m	移动与检测/s	采样最大直径/m	库内非支配
(400,500)	640.312	133.062	268.653	是
(500,400)	640.312	133.062	296.407	否
(500,500)	707.107	146.421	236.082	是
(600,400)	721.110	149.222	255.889	否
(700,450)	832.166	171.433	195.946	是
(750,400)	850.000	175.000	201.861	否
(850,450)	961.769	197.354	154.298	是
(800,605)	1003.008	205.602	135.026	是

表中保留三位小数用于对应原始记录；“采样最大直径”为离散场景的最大值，不代表连续空间的全局最优值或毫米级精度保证。

B.3 全向搜索站的环半径比较

在固定沿六边形、不穿插源服务的纯搜索路线中，从原点出发访问六个环站而不返回原点，行程为 6ρ 。表 B.3 比较两种半径；覆盖距离均小于保证接收下限 1000 m。

表 B.3 七站布局的覆盖与纯搜索费用

环半径 ρ /m	最坏覆盖距离/m	纯搜索行程/m	纯移动时间/s
1500	901.92	9000	1800
1125	999.11	6750	1350

1125 m 方案的选择依据是缩短搜索行程。表中的 2250 m 及 450 s 差值仅对应上述固定路线，不能作为联合调度的实际净收益，也不构成环半径的连续全局最优证明。

B.4 开发方案比较与共享观测诊断

表 B.4 保留开发阶段的方案取舍。第三问各方案使用相同 52 个开发案例，第四问各方案使用相同 24 个布局、每布局 4 种误差。正值表示平均总时间增加，负值表示减少；两问使用各自的比较基准，不并入第 6 章的锁定验证统计。

表 B.4 开发阶段的策略比较

问题	候选方案	平均总时间变化
第三问	仅沿位置集合中心逼近	+11.44%
第三问	中心逼近加首次视差	+5.52%
第三问	视差、原地再观测与共享观测组合	+0.99%
第三问	上述组合加入联合排序	-12.83%
第三问	单方位光学条带覆盖	+92.18%
第三问	沿接收边界往返测距	+525.62%
第四问	先搜索后清除	+12.98%
第四问	两站发现窗口	+17.47%
第四问	分区承诺	+13.95%
第四问	动态替站与测站移动组合	-2.328%

第三问的各行描述不同组合，不能解释为单个组件的独立贡献。第四问动态组合的开发改善未达到事前 5% 的替换门槛，后续锁定验证未据此新增机制或调参。

共享观测的一个聚集案例另列于表 B.5。仅收紧主目标条件后，多出的 908.68 s 全部来自移动；同时修正各共享频道的接收条件后，该例恢复收益。该案例只说明主目标与共享频道需要分别检查接收条件。

表 B.5 共享观测条件的单案例诊断

条件设置	可用共享缓存数	总时间/s
原条件	15	903.16
仅收紧主目标条件	0	1811.84
同时修正共享频道条件	—	735.55

注：“—”表示本表未列该项统计，不表示缓存数为零。

B.5 第三问补充调度比较

在保持原定位、共享观测和停止规则的条件下，用 42 个新布局比较当前联合方案与四种候选。布局覆盖六类场景及 10—16 个源，每布局使用四种固定误差和两种舍入设置；每方案 336 次，五方案共 1680 次，均完整清除。本批属于开发比较，不并入原 120 布局的最终验证。

表 B.6 相对当前联合方案的平均总时间变化

调度改动	有界读数	舍入压力
按未知区域覆盖收益重排搜索站	+1.54%	+1.34%
按估计发现与服务收益选择下一任务	+16.66%	+16.47%
计入服务费用，只规划前两项任务	+7.26%	+7.04%
将服务费用与结束位置估计纳入完整路线	-0.24%	-0.64%

正值表示变慢，负值表示省时。最后一种候选虽略微降低均值，但主模型 P95 由 4102.99 s 增至 4105.32 s，最大值由 4211.04 s 增至 4327.16 s。按事前“均值下降且 P95、最大值均不增加”的条件，本轮没有合格候选，故未进入后续独立验证，也未替换原算法。

B.6 几何计算与覆盖实现检查

第一、二问原有检查分别为 11 项和 8 项，覆盖空集、无界、退化、角度跨零及圆覆盖反例。第二问另有 144 个场景与通用半平面求解器交叉比较，直径最大差约 1.74×10^{-12} m。第三、四问还检查同点误差固定、公开响应重放、不同源清除计数、实际扫描记录与停止条件。

覆盖检查曾发现旧审计器接受重合原点形成的退化三角证书，随后补入真实三角网、外包边界和实际扫描记录核验。原 25 站构造仍然有效。以上检查支持程序与记录的一致性，不替代实数几何证明或浮点程序的形式验证。

附录 C 正式测试日志与版本对应

C.1 原始日志清单

第三、四问的原名日志分别位于 question3/results/jlog_question3/ 和 question4/results/jlog_question4/。表 C.1 按正式序号列出六份文件，原始文件名保持不变。

表 C.1 六次正式测试的原始日志

测试	原始日志文件名
第三问第 1 次	formal-p3-1-5ER6-BK3Y-KHDQ-MUQT.jlog
第三问第 2 次	formal-p3-2-VUK9-G6FH-ACSC-KFYG.jlog
第三问第 3 次	formal-p3-3-3BEW-YK5X-AE2M-SJND.jlog
第四问第 1 次	formal-p4-1-JHQ9-ZVTT-CGAR-GVTX.jlog
第四问第 2 次	formal-p4-2-SXAN-9UWV-2X8Z-GZ58.jlog
第四问第 3 次	formal-p4-3-CEBG-3EHE-MGUQ-H96W.jlog

日志明文头标记为 formal_behavior_log，其中题号、正式序号和案例编码与表中一致；行为正文加密，本次核验未解密。动作记录未含案例编码，按题号、正式顺序及唯一相邻时间建立对应，见表 C.2。

表 C.2 正式日志与运行记录的对应关系

正式测试	对应运行目录	日志生成滞后/ms
第三问第 1 次	windows_q3_final_001	22
第三问第 2 次	windows_q3_final_002	28
第三问第 3 次	windows_q3_final_004	13
第四问第 1 次	windows_q4_final_001	9
第四问第 2 次	windows_q4_final_002	6
第四问第 3 次	windows_q4_final_003	5

各运行目录位于相应的question3/results/ 或question4/results/ 内。滞后为原始日志生成时间减去对应退出响应时间。第三问windows_q3_final_003 仅记录入场连接中断，无成功入场、检测或清除响应；第四问日志目录中的第三问日志副本不重复计数。结果文件中沿用的official-rehearsal 标签不作为正式身份依据。

C.2 配置与源码版本

第三问三次配置均与联合方案一致，策略源码哈希与冻结验证锁相符；17 项运行源码哈希在三次间一致，其中 13 项与现有源码逐字节一致，其余 4 项仅有 Windows 换行符差异。

第四问三次配置均与 25 站推荐配置一致，动作记录中的站点及停止证书与该配置相符，但正式运行未保存代码哈希快照，因此仅建立配置与行为层面的对应。

附录 D AI 辅助使用说明

本次中文论文修订使用 AI 辅助整理既有材料、调整文字与标题、编写绘图程序，以及开展数值和排版核查。正式测试表依据已有原始日志及动作记录整理，未生成或替换测试数据。

历史 AI 工具型号、具体使用环节及参赛成员的最终人工审阅情况，仍需作者依据实际使用记录补全。

附录 E 展开推导与补充诊断

E.1 定位几何算例

采用图 2 的两测点算例验证上述计算。令构造源为 $G = (600, 180)$ m, 中心线误差取零, 读数由 $\theta_i = \arg(G - S_i)$ 生成, 输入为

$$\begin{aligned} S_1 &= (0, 0), & \theta_1 &\approx 16.699244^\circ, \\ S_2 &= (850, -450), & \theta_2 &\approx 111.644435^\circ. \end{aligned}$$

按上述步骤得到四个顶点, 单位为 m:

$$\begin{aligned} v_1 &\approx (592.768, 166.611), & v_2 &\approx (615.299, 172.944), \\ v_3 &\approx (607.425, 193.845), & v_4 &\approx (584.464, 186.518). \end{aligned}$$

最远点对为 v_2, v_4 , 计算得 $D(P) \approx 33.691$ m。上述角度与坐标仅作显示舍入, 计算使用未舍入读数。

图 7 以真实源 $(600, 100)$ m 和三个测点 $(0, 0)$ 、 $(350, -250)$ 、 $(500, 100)$ m 说明区域收缩。取 $\pm 1^\circ$ 包络、零中心线误差, 初始外包为 $[-1500, 1500]^2$; 三次取交后的直径为 1525.36、47.57、21.26 m, 固定坐标轴包围盒的中心覆盖半径为 762.68、23.78、10.66 m。第三次的覆盖半径小于 20 m, 才满足直接清除条件。

E.2 保证接收域的分段推导

为得到不依赖首测位置的显式策略, 先不使用 Ω 裁剪, 并闭合近端点, 构造完整外包扇形

$$\mathcal{F}_0 = \{S + r(\cos \alpha u + \sin \alpha v) : 5 \leq r \leq 1500, |\alpha| \leq \delta\}.$$

由于 $\mathcal{F}_1 \subseteq \mathcal{F}_0$, 对 $\mathcal{F}_0$ 保证接收的点也适用于真实候选集合。记移动距离 $\ell = \|Q - S\| = \sqrt{a^2 + b^2}$, 并令 $t(\alpha) = a \cos \alpha + b \sin \alpha$ 。展开距离平方可得

$$d_2^2 = \ell^2 + r^2 - 2rt(\alpha).$$

下面分别处理近、远两段源距, 将连续约束化为有限个不等式。

近段 $5 \leq r \leq 1000$ 。此时式 (6) 要求 $d_2 \leq 1000$ 。对固定 α , d_2^2 关于 r 的二阶导数为 2, 因而最大值在区间端点取得。检查 $r = 5$ 与 $r = 1000$, 分别得到

$$\ell^2 + 25 - 10t(\alpha) \leq 10^6, \quad \ell^2 \leq 2000t(\alpha).$$

远段 $1000 \leq r \leq 1500$ 。此时式 (6) 要求 $d_2 \leq r$, 即 $\ell^2 \leq 2rt(\alpha)$ 。近段的 $r = 1000$ 条件已迫使 $t(\alpha) \geq 0$, 故右端随 r 增大而不减, 远段不再增加更严格的约束。

取最不利方向。在 $r = 1000, \alpha = 0$ 时, 有 $\ell^2 \leq 2000a$, 所以必有 $a \geq 0$ 。对这一范围, $\cos \alpha \geq \cos \delta$ 且 $b \sin \alpha \geq -|b| \sin \delta$, 因此

$$\min_{|\alpha| \leq \delta} t(\alpha) = a \cos \delta - |b| \sin \delta =: h.$$

等号在偏向不利一侧的角度端点取得。将 $t(\alpha)$ 替换为 h ，完整扇形的保证接收条件最终为

$$a \geq 0, \quad \ell^2 - 10h + 25 \leq 10^6, \quad \ell^2 \leq 2000h.$$

式 (7) 对 $\mathcal{F}_0$ 是充要条件，相对利用源域裁剪后的 $\mathcal{F}_1$ 则可能偏保守。它也解释了前向位移的必要性：若 $a = 0, b \neq 0$ ，则 $h < 0$ ，无法满足最后一个不等式。检测点不额外受源位置圆域限制。

E.3 第二问的采样与评价次数

原始搜索源距在 $[5.001, 1500]$ m 等距取 51 点，角偏差与第二误差各取 5 点；解析约束仍包含 5 m 端点。复核增加至 301、11、11 点。

实际粗网格共有 98 个候选通过筛选，优选点为 (800, 600) m，其采样最大直径为 134.976 m。以此为心的细网格范围为 $a \in [750, 850]$ 、 $b \in [550, 650]$ m，共 237 次可行评价，得到 (800, 605) m 和 134.284 m。粗细两阶段合计 335 次评价，其中包含 5 个重复点。对该优选点，36,421 场景复核与原 1275 场景的最大直径在所报精度下一致。

随后针对两位小数舍入，将角度约束和场景中的 α, ε_2 同时扩为 $\pm 1.005^\circ$ ，对既定 8 个上侧偏移及其镜像进行复核。这一步没有重新搜索最优坐标；它检验已有候选在较宽误差包络下的条件与定位结果。

E.4 双环半径选择与核验

(2) **双环构造及参数选择**。采用中心站与两个等站数正环，内、外环半径分别为 r_i, r_o ，每环 m 站，两环相位差为 π/m 。外环正多边形须包围半径 1800 m 的源域，且外环相邻边作为三角网边应小于 1000 m，因此

$$r_o \cos \frac{\pi}{m} \geq 1800, \quad 2r_o \sin \frac{\pi}{m} < 1000.$$

消去 r_o 可得

$$3600 \tan \frac{\pi}{m} < 1000 \implies m > \frac{\pi}{\arctan(5/18)} \approx 11.5949.$$

故在本文采用的等站数正双环构造中，每环至少 12 站。取 $m = 12$ 时，外环半径满足

$$r_o \in \left[\frac{1800}{\cos 15^\circ}, \frac{500}{\sin 15^\circ} \right) \approx [1863.50, 1931.85) \text{ m}.$$

选取 $r_o = 1870$ m。内环需同时满足中心连边、内环相邻边和跨环边约束：

$$\begin{aligned} r_i &< 1000, & 2r_i \sin 15^\circ &< 1000, \\ r_i^2 + 1870^2 - 2r_i \cdot 1870 \cos 15^\circ &< 1000^2. \end{aligned}$$

该组不等式给出的可行内环半径下限约为 931.21 m、上限为 1000 m，选取 $r_i = 950$ m。由此得到固定 25 站：

$$\begin{aligned}
s_0 &= (0, 0), \\
a_k &= 950(\cos(k\pi/6), \sin(k\pi/6)), \\
b_k &= 1870(\cos(k\pi/6 + \pi/12), \sin(k\pi/6 + \pi/12)), \quad k = 0, \dots, 11.
\end{aligned}$$

按下标模 12 连接 (s_0, a_k, a_{k+1}) 、 (a_k, a_{k+1}, b_k) 和 (a_k, b_{k-1}, b_k) ，得到 36 个三角形。其最长边为 983.60 m；外环内切圆半径为 $1870 \cos 15^\circ = 1806.28$ m，大于 1800 m，故满足全域覆盖及近距凸包条件，如图 12 所示。12 站的下界仅针对上述构造族；25 站及两个半径是经核验的可行选择，不构成一般连续发现方案的最少站数或最短行程证明。

(2) **构造结果与运行输出。**对推荐站集及局部清除规则进行几何核验，结果如下。

表 E.1 第四问站点构造与光学覆盖核验

核验量	结果
搜索站数、覆盖三角形数	25、36
内环半径、外环半径	950 m、1870 m
中心至内环边长	950.00 m
内环相邻边长	491.76 m
外环相邻边长	967.98 m
跨环边长（最长边）	983.60 m
外环内切圆半径	1806.28 m
初始单源光学网格数上界	108

E.5 光学格心坐标

(2) **光学网格及有限清除。**沿首测方向 u_c, v_c 建立当前区域的外包矩形。对 $e \in \{u_c, v_c\}$ ，定义全局坐标投影的端点 $\ell_e = \min_{x \in P_c} e^\top x$ 、 $h_e = \max_{x \in P_c} e^\top x$ ，得到区间 $[\ell_u, h_u]$ 、 $[\ell_v, h_v]$ 。网格划分数取

$$n_u = \max\left(1, \left\lceil \frac{h_u - \ell_u}{28} \right\rceil\right), \quad n_v = \max\left(1, \left\lceil \frac{h_v - \ell_v}{28} \right\rceil\right).$$

分别等分两区间，格宽 $d_u = (h_u - \ell_u)/n_u$ 、 $d_v = (h_v - \ell_v)/n_v$ 均不超过 28 m。第 (i, j) 格的清除点为

$$p_{ij} = [\ell_u + (i + \frac{1}{2})d_u]u_c + [\ell_v + (j + \frac{1}{2})d_v]v_c, \quad 0 \leq i < n_u, \quad 0 \leq j < n_v.$$

E.6 实验布局与统计补充

Q3 的冻结基线是历史第十九轮保留的组合，联合方案使用集合定位、视差、跨频道观测筛选和任务行程规划。候选选择先使用开发材料和 60 个结构开发布局，随后冻结源码、配置、生成器及选择规则，再生成 120 个新布局。六类布局为均匀、外缘、聚集、最小接收半径、窄径向和接收边缘，各 20 个。每个布局配 4 种固定空间误差场及 2 种舍入环境，所以每方案共有 960 次运行，但独立布局仍只有 120 个。

Q4 保留两个不同的问题。第一个问题是成熟 25 站方案是否优于 31 站初始基线，在两批互不重叠、各 96 个新布局中比较；每批含 6 类布局，每类 16 个，每布局配一个预

定误差场，两个批次分别使用有界读数和舍入压力模型。第二个问题是成熟方案之后的动态替站与测站移动是否值得加入，在另 120 个新布局中进行 5 臂锁定验证，每布局配 4 种误差场和 2 种舍入环境，每臂 960 次运行。后者的参照已是成熟 25 站方案，不能把两个阶段的改善比例相加。

“布局”指固定的一组源位置、半径、频道以及适用时的方向；重复施加误差场不产生新的独立布局。采用自助法估计统计量的不确定性^[7]；本文以独立布局为重采样单位，重复有放回抽取布局块，并保留块内全部误差条件。同一物理布局上的算法使用相同误差条件，按配对方式比较。主要效应定义为 $100(1 - \bar{T}_{\text{candidate}}/\bar{T}_{\text{base}})$ ，正值表示省时；图 16、图 20 的纵轴使用相反符号的时间变化，正值表示变慢，图注逐一注明。完整清除数、均值、中位数、95% 分位数（P95，即本批约 95% 运行耗时不超过的值）和本批最大值同时保留，避免均值掩盖失败和长尾。表中时间指标按该行全部运行等权计算，分位数采用线性插值；主要效应区间按布局重采样。箱线图以箱体表示第 25% 至 75% 分位数，中线表示中位数，须延伸到 1.5 倍四分位距范围内的最远观测，范围外样本仍保留。

Q3 按 6 类分层、以布局为块进行 20,000 次配对自助法重采样；同一布局的 4 种误差条件一起重采样。Q4 历史两批分别以布局为单位作 10,000 次配对自助法重采样；最新验证以布局为块、6 类分层作 10,000 次重采样，同一布局的 8 次条件重放保持在一块中。区间只反映所用自建生成器中的采样不确定性，不代表未知官方场景的性能保证。

E.7 第三问慢例诊断

按单次运行统计，主模型 435 次更快、45 次更慢；舍入压力环境 433 次更快、47 次更慢。最差相对退化出现在一个 16 源聚集布局的空间哈希误差条件中：基线 1063.06 s，联合方案 2058.36 s，慢 93.63%。事后核对该布局的接收条件可知，原点已能接收到 15 个源，只需再发现最后一个频道，就可取消剩余未知搜索。基线较早完成了全部发现，而联合方案在此之前访问了更多搜索站。该例中，联合方案比基线多用 791.31 s 移动时间，检测次数由 39 次增至 76 次，增加 185 s 检测和 37 s 切频费用；16 次清除均成功，主要损失发生在行走与发现过程。图 16 先合并误差条件，所以其中的最高点不等于这个单次运行的 93.63%。

这个慢例说明，按任务代表点缩短路线，可能错过提前结束搜索的机会。机器人事先不知道最后一个源的位置，当前排序也未充分估计不同搜索站带来的发现机会。因此，平均省时不能解释为每局都更快。原验证结果与冻结方案均予保留；后续调度开发使用另一批新布局，未针对该慢例调参。

E.8 第三问完整消融矩阵

各方案的机制开关列于下表。“删除信息门控”仍保留共享观测，只是不再筛选共享观测的收缩价值。冻结基线使用另一套历史机制，不能把它简单表示为这些开关全部关闭。

表 E.2 Q3 共同父方案的机制矩阵

方案	共享	视差	行程	门控	目标再观测	均值/s	P95/s
共同父方案	有	有	有	有	有	3007.19	3822.14
最终联合	有	有	有	有	无	2982.18	3823.72
删除首次视差	有	无	有	有	有	3250.91	4111.08
删除完整行程	有	有	无	有	有	3044.39	4085.55
删除信息门控	有	有	有	无	有	3034.19	3839.33

同一 120 布局、每布局 4 误差条件，每行 480 次自建运行；开关只对共同父方案的组件定义有效。

E.9 第四问历史构造与耗时细分

Q4 的第一轮关键试错不是调度，而是发现证书。一个具体 21 站布局在 25 m 粗网格上未发现漏点，细化后却在 (898.68, -241.26) m 处发现：1000 m 内近站之间的最大方向空隙达到 182.68°，足以容纳一个避开全部近站的 180° 发射半平面。该反例否定了这个具体构造，并说明粗网格“未发现失败”不能代替连续证明；它没有证明所有 21 站布局都不可行。

随后, 22 站布局通过了连续凸包覆盖证书，但站数更少也没有稳定转化为整局更快。在最终保留的调度规则下，同批验证中 22 站方案分别比 25 站推荐慢 5.24% 和 3.39%。一个保留案例中，25 站方案只扫描 12 站便清除 16 源，22 站方案却需要扫描 16 站，总时间增加 67.12%。减少可选站数会改变发现顺序，并不保证减少实际扫描数。

两批中 31 站、25 站及所比较删减方案均完整清除。25 站推荐的 P95 分别为 8694.62 s 和 9598.33 s，低于初始基线的 13465.02 s 和 14367.42 s。读数包络批次移动耗时由 9163.35 s 降至 4588.99 s；删除重心排序后，两批总时间均增加超过 11%，删除低信息量观测筛选后均增加约 4.6%。结果支持将连续发现、定位清除和任务访问顺序作为一个整体，而不是分别追求最少站数或最少检测数。

有界读数批次中，基线/推荐的中位数为 11389.99/6367.34 s，本批最大值为 16463.37/9931.57 s；推荐方案 96 次更快、0 次更慢；舍入压力批次中，基线/推荐的中位数为 11445.89/6418.62 s，本批最大值为 18821.00/10396.77 s；推荐方案 96 次更快、0 次更慢。

E.10 动态覆盖完整比较与慢例

最终五方案自建矩阵共 4800 次运行均完成清除。下表各方案均使用同一 120 布局、每布局 8 种条件，共 960 次运行；相对省时按整批均值之比计算。

表 E.3 Q4 最新锁定验证的完整流程结果

方案	均值/s	P95/s	本批最大/s	相对 25 站推荐省时/%
25 站推荐	6744.38	9497.47	11986.29	0.000
历史自适应定位	6614.15	9417.57	12592.28	1.931
仅动态替站	6734.42	9497.47	11986.29	0.148
仅测站移动	6740.98	9460.06	11928.43	0.050
替站与移动组合	6719.67	9460.06	11928.43	0.366

该表时间保留两位小数，省时比例及其区间保留三位，以呈现不足 1% 的小效应。

图 20 概括布局配对变化, 定量判断采用核验后的统计值。组合的平均省时为 0.366%，95% 区间为 [-0.183%, 0.921%]，跨过零；P95 仅下降 0.394%。其移动平均少 37.00 s，却多出 6.86 s 检测、3.97 s 光学和 1.45 s 切频，净省 24.71 s。历史自适应定位在表中均值较低，但既未达到 5% 降时门槛，最大值也更高，不能事后按最低样本均值更换赢家。

按全部 960 次条件运行统计，25 站推荐与动态组合的中位数分别为 6377.21 s 和 6369.66 s；组合 706 次更快、254 次更慢、0 次相同。这些重放共享 120 个物理布局，不能当作 960 个独立样本。

最差反例解释了为什么局部路径改善不能保证整局收益。在一个均匀布局的正误差条件中，动态组合由 4228.69 s 变为 5653.40 s，慢 33.69%，两者均清除 16 源。候选移动首个内环站 103.88 m，局部预测路线缩短 42.06 m，但实际扫描站数从 7 增至 19，检测次数从 119 增至 242，第 16 个频道的首次发现从 2093.29 s 推迟到 4811.12 s。该例没有发生动态替站，因此不能把退化归因于未执行的操作。

逐动作记录支持“测站移动改变发现与服务顺序，推迟数量上限停止”的解释；由于没有逐项反事实撤回实验，不能为某一次动作分配独立因果损失。平均结果和反例共同导向同一个取舍：保留有连续证书和稳定收益的 25 站推荐方案，将动态覆盖视为可行但收益尚不足的扩展，而不是论文必须包装成成功的新方法。

E.11 正式运行的总时间与过程计量

Q3 三次总虚拟时间依次为 2955.60、3888.46、3055.66 s，Q4 依次为 5755.71、5824.43、6363.74 s；两问累计清除 38、36 源，逐次每源平均时间的算术平均为 264.41、514.13 s。六次均满足停止判据并主动退出。Q3 无失败光学尝试，Q4 第二、三次分别有 5、6 次未命中，费用均计入总时间。

Q3 的 38 条正式源轨迹中，31 源满足覆盖圆判据，7 源通过近场反馈清除。相邻清除点之间包含搜索、补测、移动及光学耗时，不能单独归为清除动作。Q3 正式 3 及 Q4 正式 1、2 在最后清除后，分别继续核验 140.85、463.79、123.77 s，因此清除终点与任务退出分开计量。

附录 F 支撑材料索引与完整源程序

支撑包中的evidence.tar.xz 保存较大的原始逐例结果、配置和冻结文件；运行restore_evidence.py 恢复目录，并按SUPPORT_MANIFEST.json 逐文件校验。六份正式.jlog 以原名、原始字节直接保存在第三、四问日志目录。全部完整 Python 源程序如下，相同内容的冻结副本只打印一次；对应路径全部列出。诊断文本中的本机用户目录已替换，原件和副本哈希均保留；正式日志未作改动。

支撑材料文件列表

顶层文件：README.md、AI 工具使用详情.pdf、SUPPORT_MANIFEST.json、SOURCE_MAP.json、restore_evidence.py、evidence.tar.xz。

B 题/B 题.pdf

manuscript/story_draft/figures/fig01_overview_optimized.png
manuscript/story_draft/figures/fig02_diameter.pdf
manuscript/story_draft/figures/fig03_q2_failures.pdf
manuscript/story_draft/figures/fig04_q2_region_optimized.png
manuscript/story_draft/figures/fig05_q2_tradeoff.pdf
manuscript/story_draft/figures/fig06_q3_algorithm_optimized.png
manuscript/story_draft/figures/fig07_q3_cover_optimized.png
manuscript/story_draft/figures/fig08_no_signal.pdf
manuscript/story_draft/figures/fig09_q4_cover.pdf
manuscript/story_draft/figures/fig10_optical.pdf
manuscript/story_draft/figures/fig11_q3_distribution_optimized.jpg
manuscript/story_draft/figures/fig12_q3_cost.pdf
manuscript/story_draft/figures/fig13_q3_ablation.pdf
manuscript/story_draft/figures/fig14_q4_historical_optimized.png
manuscript/story_draft/figures/fig15_q4_latest_optimized.png
manuscript/story_draft/figures/fig16_bearing_intersection.pdf
manuscript/story_draft/figures/fig17_shrink_optimized.png
manuscript/story_draft/figures/fig18_route_example.pdf
manuscript/story_draft/figures/fig19_q3_time_distribution_optimized.png
manuscript/story_draft/figures/fig20_q4_coverage_certificate.png
manuscript/story_draft/figures/fig21_discovery_localization_clearance.png
manuscript/story_draft/中文论文初稿.md
manuscript/story_draft/中文论文初稿.tex
question3/innovation/frozen/B 题/B 题.pdf
question3/innovation/validation_overview.png
question3/innovation/worst_case_paths.png
question3/results/jlog_question3/formal-p3-1-5ER6-BK3Y-KHDQ-MUQT.jlog
question3/results/jlog_question3/formal-p3-2-VUK9-G6FH-ACSC-KFYG.jlog

question3/results/jlog_question3/formal-p3-3-3BEW-YK5X-AE2M-SJND.jlog
question4/results/jlog_question4/formal-p4-1-JHQ9-ZVTT-CGAR-GVTX.jlog
question4/results/jlog_question4/formal-p4-2-SXAN-9UWV-2X8Z-GZ58.jlog
question4/results/jlog_question4/formal-p4-3-CEBG-3EHE-MGUQ-H96W.jlog
排版生成文件: manuscript/build/submission_inventory.tex、manuscript/build
/submission_code.tex 及manuscript/build/submission_code/ 中的编号源码副本。
压缩证据内部逐文件路径、大小和哈希见SUPPORT_MANIFEST.json。

程序 1

对应文件: manuscript/analysis/make_figures.py

```
1 """Reproduce the three manuscript figures from analytic constraints and frozen data.
2 Backend: Python/matplotlib. No invented observations or excluded validation runs.
3 """
4 from pathlib import Path
5 import sys, json, csv, math
6 import numpy as np
7 import matplotlib as mpl
8 mpl.use('Agg')
9 import matplotlib.pyplot as plt
10 from matplotlib.colors import ListedColormap
11 from matplotlib.patches import Circle
12 ROOT = Path(__file__).resolve().parents[2]
13 sys.path.insert(0, str(ROOT))
14 from question4.geometry import radial_mesh
15 mpl.rcParams.update({'font.family': 'sans-serif', 'font.sans-serif': ['Alibaba PuHuiTi', 'DejaVu Sans'],
16 'font.size': 9, 'axes.labelsize': 9, 'axes.titlesize': 10, 'axes.spines.top': False,
17 'axes.spines.right': False, 'pdf.fonttype': 42, 'svg.fonttype': 'none', 'axes.unicode_minus': False,
18 'legend.frameon': False, 'axes.linewidth': 0.7})
19 OUT=ROOT/'manuscript/figures'; OUT.mkdir(exist_ok=True)
20 TEAL='#227D8D'; GRAY='#C6D0D5'; ORANGE='#B86535'
21
22 def save(fig, name):
23     fig.canvas.draw()
24     # All exports are single scientific panels; alignment is not applicable.
25     (OUT/(name+'.alignment.json')).write_text(json.dumps({'status': 'NOT APPLICABLE', 'panels': 1}))
26     fig.savefig(OUT/(name+'.pdf'), metadata={'Creator': 'Python/matplotlib'}, bbox_inches=None)
27     fig.savefig(OUT/(name+'.svg'), metadata={'Creator': 'Python/matplotlib'}, bbox_inches=None)
28     fig.savefig(OUT/(name+'.png'), dpi=300)
29     plt.close(fig)
30
31 # Analytic region represented on a regular display grid; this is not a proof by sampling.
32 a, b = np.meshgrid(np.linspace(-40, 1100, 571), np.linspace(-1050, 1050, 701))
33 delta = np.deg2rad(1.005); h = a * np.cos(delta) - abs(b) * np.sin(delta); q2 = a * a + b * b
34 reception = (a >= 0) & (q2 - 10 * h + 25 <= 1000 ** 2) & (q2 <= 2000 * h)
35 cross = abs(b) * np.cos(delta) - a * np.sin(delta)
36 dmax = np.sqrt(np.maximum(q2 + 25 - 10 * h, q2 + 1500 ** 2 - 3000 * h))
37 safe = reception & (cross > 5) & (cross >= dmax * np.sin(np.deg2rad(20)))
38 z = np.where(safe, 2, np.where(reception, 1, 0))
39 fig, ax = plt.subplots(figsize=(5.35, 3.9)); fig.subplots_adjust(left=.15, right=.95, bottom=.15, top=.9)
40 ax.pcolormesh(a, b, z, cmap=ListedColormap(['white', '#E1E7EA', TEAL]), shading='auto', rasterized=True)
41 ax.axhline(0, color='#8A959B', linewidth=.65); ax.axvline(0, color='#8A959B', linewidth=.65)
42 ax.plot([0, 1500], [0, 1500 * np.tan(delta)], color=ORANGE, lw=1)
43 ax.plot([0, 1500], [0, -1500 * np.tan(delta)], color=ORANGE, lw=1)
44 ax.scatter([800, 800], [605, -605], c='#152F40', s=22, zorder=5)
45 ax.text(805, 700, '(800, 605)', fontsize=8)
46 ax.text(805, -765, '(800, -605)', fontsize=8)
47 ax.text(180, 795, '保证接收域', color='#44535F', fontsize=8)
48 ax.text(480, 410, '候选域', color='white', fontsize=9)
49 ax.set(xlim=(-40, 1100), ylim=(-1050, 1050), xlabel='沿首次定向方向的位移 a (m)', ylabel='横向位移 b (m)')
50 ax.set_xticks([0, 250, 500, 750, 1000]); ax.set_yticks([-1000, -500, 0, 500, 1000])
51 ax.set_aspect('equal', adjustable='box'); save(fig, 'q2_candidate')
52
53 # Layout-level pairing; all four error conditions are retained within every layout.
54 rows = list(csv.DictReader((ROOT/'question3/innovation/final_paired.csv').open()))
55 rows = [r for r in rows if r['rounding'] == 'bounded']
```

```

56 groups={}
57 for r in rows:
58     parts=r['case_id'].split('_');seed=next(x for x in parts if x.isdigit())
59     family=r['case_id'][len('final_'):r['case_id'].index('_'+seed)]
60     groups.setdefault((family,seed),[]).append(r)
61 assert len(groups)==120 and all(len(v)==4 for v in groups.values())
62 fams=['uniform','outer','cluster','min_radius','radial','reception_edge']
63 labels=['均匀','外缘','聚集','最小半径','窄径向','接收边缘']
64 source=[]
65 fig,ax=plt.subplots(figsize=(6.3,3.1));fig.subplots_adjust(left=.13,right=.98,bottom=.19,top=.93)
66 for i,fam in enumerate(fams):
67     vals=[]
68     for (f,s),rr in sorted(groups.items()):
69         if f!=fam:continue
70         base=np.mean([float(r['baseline']) for r in rr]);cand=np.mean([float(r['candidate']) for r in rr])
71         change=100*(cand/base-1);vals.append(change)
72         source.append({'family':f,'seed':s,'error_conditions':4,'base_s':base,'candidate_s':cand,'layout_mean_change_」
        ↳ pct':change})
73 # Deterministic positions only; no random thinning or case selection.
74 jitter=np.linspace(-.2,.2,len(vals))
75 ax.scatter(i+jitter,vals,s=19,color=TEAL,alpha=.78,edgecolors='white',linewidth=.3)
76 ax.plot([i-.25,i+.25],[np.median(vals)]*2,color='#182D3A',lw=1.5)
77 ax.axhline(0,color=ORANGE,ls='--',lw=.9)
78 ax.set_xticks(range(6),labels);ax.set_ylabel('布局平均总时间相对变化 (%)')
79 ax.set_ylim(-30,65);ax.set_xlim(-.55,5.55)
80 ax.grid(axis='y',color='#E5E9EC',lw=.5);ax.set_axisbelow(True)
81 ax.text(.01,1.015,'120 个布局, 每点合并 4 种误差, 横线为类别中位数',transform=ax.transAxes,fontsize=8)
82 save(fig,'q3_performance')
83 with (OUT/'q3_layout_source.csv').open('w') as f:
84     w=csv.DictWriter(f,fieldnames=list(source[0]));w.writeheader();w.writerows(source)
85
86 # Exact station coordinates and triangulation used by the retained Q4 strategy.
87 pts,tri=radial_mesh();maxedge=max(np.linalg.norm(pts[t][:,None]-pts[t][None,:],axis=2).max() for t in tri)
88 assert len(pts)==25 and len(tri)==36 and maxedge<1000
89 fig,ax=plt.subplots(figsize=(4.5,3.75));fig.subplots_adjust(left=.17,right=.96,bottom=.16,top=.96)
90 ax.triplot(pts[:,0],pts[:,1],tri,color='#AABBC2',lw=.7,zorder=1)
91 ax.add_patch(Circle((0,0),1800,fill=False,edgecolor=ORANGE,ls='--',lw=1.2,zorder=3))
92 ax.scatter(pts[1:13,0],pts[1:13,1],s=19,color=TEAL,zorder=4)
93 ax.scatter(pts[13:,0],pts[13:,1],s=24,marker='s',color='#223D4A',zorder=4)
94 ax.scatter([0],[0],s=27,marker='D',color=ORANGE,zorder=5)
95 ax.set(xlim=(-2100,2100),ylim=(-2100,2100),xlabel='x (m)',ylabel='y (m)')
96 ax.set_xticks([-1800,-900,0,900,1800]);ax.set_yticks([-1800,-900,0,900,1800]);ax.set_aspect('equal')
97 save(fig,'q4_coverage')
98 (OUT/'geometry_source.json').write_text(json.dumps({'q2_error_deg':1.005,'q2_min_angle_deg':20,'q4_stations':pts.tolist
    ↳ st(),'q4_triangles':tri.tolist(),'q4_max_edge_m':maxedge},indent=2))
99 print('Generated 3 figures; q3 layout change range',min(s['layout_mean_change_pct'] for s in
    ↳ source),max(s['layout_mean_change_pct'] for s in source))

```

程序 2

对应文件: manuscript/analysis/q12_verify.py

```

1 """Reproduce the manuscript's Q1/Q2 geometric claims without changing source data.
2
3 Run from the project root: python manuscript/analysis/q12_verify.py
4 All design scenarios are constructed examples, not official simulator trials.
5 """
6 from __future__ import annotations
7
8 import hashlib
9 import importlib.util
10 import json
11 import subprocess
12 import sys
13 from pathlib import Path
14
15 import numpy as np
16
17 ROOT = Path(__file__).resolve().parents[2]
18
19

```



```

20 def import_file(name, path):
21     spec = importlib.util.spec_from_file_location(name, path)
22     module = importlib.util.module_from_spec(spec)
23     spec.loader.exec_module(module)
24     return module
25
26
27 q1 = import_file("q12_first", ROOT / "question1/solve.py")
28 q2 = import_file("q12_second", ROOT / "question2/strategy.py")
29
30
31 def grid(error_deg, nr=301, na=11, ne=11):
32     delta = np.deg2rad(error_deg)
33     r, a, e = np.meshgrid(
34         np.linspace(5.001, 1500, nr),
35         np.linspace(-delta, delta, na),
36         np.linspace(-delta, delta, ne),
37         indexing="ij",
38     )
39     return np.column_stack(((r * np.cos(a)).ravel(), (r * np.sin(a)).ravel()), e.ravel())
40
41
42 def certificate(q, error_deg):
43     a, b = q
44     delta = np.deg2rad(error_deg)
45     norm2 = a * a + b * b
46     h = a * np.cos(delta) - abs(b) * np.sin(delta)
47     cross_lower = abs(b) * np.cos(delta) - a * np.sin(delta)
48     max_distance = np.sqrt(max(norm2 + 25 - 10 * h, norm2 + 1500**2 - 3000 * h))
49     return {
50         "near_endpoint_reception_margin_m2": float(1000**2 - (norm2 + 25 - 10 * h)),
51         "range_1000_reception_margin_m2": float(2000 * h - norm2),
52         "distance_lower_m": float(cross_lower),
53         "distance_upper_m": float(max_distance),
54         "acute_intersection_angle_lower_deg": float(np.rad2deg(np.arcsin(cross_lower / max_distance))),
55     }
56
57
58 def main():
59     source_paths = [
60         "question1/solve.py", "question1/test_solve.py", "question1/example_result.json",
61         "question1/第一问_建模与求解.md", "question2/strategy.py", "question2/test_strategy.py",
62         "question2/results.json", "question2/round1_tradeoff.json", "question2/第二问_建模与求解.md",
63         "question2/第二问_复核补充.md", "question2/第二问_第一轮权衡分析.md", "B题/B题.pdf",
64         "B题/附件/附件1.docx", "B题/附件/附件2.docx",
65     ]
66     report = {
67         "scope": "Q1 analytical geometry and Q2 constructed design scenarios; not official trials",
68         "source_sha256": {p: hashlib.sha256((ROOT / p).read_bytes()).hexdigest() for p in source_paths},
69         "unit_tests": [],
70     }
71     for directory in ("question1", "question2"):
72         run = subprocess.run(
73             [sys.executable, "-m", "unittest", "discover", "-s", directory, "-v"],
74             cwd=ROOT, capture_output=True, text=True, check=True,
75         )
76         report["unit_tests"].append({"directory": directory, "exit_code": run.returncode,
77                                     "output": run.stdout + run.stderr})
78     triangle = q1.solve(q1.triangle_measurements())
79     expected = json.loads((ROOT / "question1/example_result.json").read_text())["result"]
80     assert abs(triangle["diameter"] - expected["diameter"]) < 1e-9
81     assert triangle["diameter_circle_covers"] is False
82     report["q1_triangle"] = triangle
83     report["q1_triangle"]["minimum_enclosing_diameter_m"] = float(40 / np.sqrt(3))
84
85     original = json.loads((ROOT / "question2/results.json").read_text())
86     best, count = q2.search()
87     assert count == original["candidates_evaluated"]
88     assert np.allclose(best[2:], original["recommendation"], rtol=0, atol=1e-9)
89     assert abs(best[0] - original["sampled_search_worst"]) < 1e-9
90     report["q2_legacy_search_ldeg"] = {"best": list(best), "candidates_evaluated": count}
91
92     fixed = json.loads((ROOT / "question2/round1_tradeoff.json").read_text())

```

```

93     source, second_error = grid(1.005)
94     radius = np.linalg.norm(source, axis=1)
95     rows = []
96     max_discrepancy = 0.0
97     cross_checks = 0
98     sparse_source, sparse_error = grid(1.005, 3, 3, 3)
99     for old in fixed["rows"]:
100         q = np.asarray(old["offset"], dtype=float)
101         assert q2.candidate(q, error_deg=1.005)
102         assert q2.candidate(q * [1, -1], error_deg=1.005)
103         diameter = q2.diameters(q, source, second_error, error_deg=1.005)
104         worst = int(diameter.argmax())
105         discrepancy = abs(float(diameter[worst]) - old["dense_sample_worst_D"])
106         assert discrepancy < 1e-8
107         distance = np.linalg.norm(source - q, axis=1)
108         sine = np.abs(source[:, 0] * q[1] - source[:, 1] * q[0]) / (radius * distance)
109         assert np.all(distance <= np.maximum(1000, radius) + 1e-8)
110         assert np.all(distance > 5)
111         assert np.all(sine >= np.sin(np.deg2rad(20)) - 1e-12)
112         rows.append({"old": old, "recomputed_worst_D_m": float(diameter[worst]),
113                     "worst_source_m": source[worst].tolist(),
114                     "worst_second_error_deg": float(np.rad2deg(second_error[worst])),
115                     "certificate": certificate(q, 1.005)})
116     for mirror in (1, -1):
117         candidate = q * [1, mirror]
118         expected_d = q2.diameters(candidate, sparse_source, sparse_error, error_deg=1.005)
119         for i in range(0, len(sparse_source), 3):
120             angle = np.rad2deg(np.arctan2(*(sparse_source[i] - candidate)[:2])) + sparse_error[i]
121             general = q1.solve([
122                 {"x": 0, "y": 0, "bearing_deg": 0},
123                 {"x": float(candidate[0]), "y": float(candidate[1]), "bearing_deg": float(angle)},
124             ], error_deg=1.005)
125             assert general["status"] == "bounded"
126             difference = abs(general["diameter"] - expected_d[i])
127             assert difference < 1e-6
128             max_discrepancy = max(max_discrepancy, difference)
129             cross_checks += 1
130 # Recompute the finite-library dominance relation from the two reported metrics.
131 for row in rows:
132     dominated = any(
133         other["move_m"] <= row["move_m"] + 1e-9
134         and other["recomputed_worst_D_m"] <= row["recomputed_worst_D_m"] + 1e-9
135         and (other["move_m"] < row["move_m"] - 1e-9
136             or other["recomputed_worst_D_m"] < row["recomputed_worst_D_m"] - 1e-9)
137         for other in rows
138     )
139     assert row["pareto_within_fixed_library"] == (not dominated)
140 report["q2_fixed_library_1_005deg"] = {
141     "dense_samples_per_candidate": len(source), "rows": rows,
142     "general_solver_cross_checks": cross_checks,
143     "largest_general_solver_difference_m": max_discrepancy,
144     "continuous_worst_case_certified": False,
145     "continuous_global_optimum_certified": False,
146 }
147 report["status"] = "PASS"
148 output = Path(__file__).with_name("q12_verification.json")
149 output.write_text(json.dumps(report, ensure_ascii=False, indent=2) + "\n")
150 print(json.dumps({"status": report["status"], "output": str(output),
151                 "dense_samples_per_candidate": len(source),
152                 "general_solver_cross_checks": cross_checks,
153                 "largest_general_solver_difference_m": max_discrepancy}, ensure_ascii=False))
154
155
156 if __name__ == "__main__":
157     main()

```

程序 3

对应文件: manuscript/analysis/q3_verify.py

```

1 """Read-only reconstruction of Q3 manuscript evidence; no simulator is started.
2

```

```

3 Run from the repository root: python manuscript/analysis/q3_verify.py
4 Only manuscript/analysis/q3_* files are written. Official records are read via
5 SQLite mode=ro; encrypted logs are inspected only for their public envelopes.
6 """
7 from __future__ import annotations
8 import csv
9 import hashlib
10 import json
11 import math
12 import sqlite3
13 from collections import Counter
14 from datetime import datetime, timezone
15 from pathlib import Path
16 import numpy as np
17
18 ROOT = Path(__file__).resolve().parents[2]
19 OUT = ROOT / "manuscript/analysis"
20 INNOV = ROOT / "question3/innovation"
21
22
23 def read(path):
24     return json.loads(path.read_text(encoding="utf-8"))
25
26
27 def sha(path):
28     return hashlib.sha256(path.read_bytes()).hexdigest()
29
30
31 def save_csv(name, rows):
32     with (OUT / name).open("w", encoding="utf-8", newline="") as stream:
33         writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
34         writer.writeheader()
35         writer.writerows(rows)
36
37
38 def block_bootstrap(rows, selected, fixtures):
39     by = {v: {r["case_id"]: r for r in rows if r["variant"] == v}
40           for v in ("BASE", selected)}
41     ids_by_seed = {}
42     for f in fixtures:
43         ids_by_seed.setdefault(str(f["seed"]), []).append(f["id"])
44     seeds = sorted(ids_by_seed)
45     family = {str(f["seed"]): f["family"] for f in fixtures}
46     assert len(seeds) == 120 and all(len(x) == 4 for x in ids_by_seed.values())
47     data = {v: np.array([[by[v][k]["penalized_virtual_s"]
48                           for k in sorted(ids_by_seed[s])] for s in seeds]) for v in by}
49     rng = np.random.default_rng(20260912)
50     indices = []
51     for f in sorted(set(family.values())):
52         ids = np.array([i for i, s in enumerate(seeds) if family[s] == f])
53         assert len(ids) == 20
54         indices.append(rng.choice(ids, size=(20000, len(ids)), replace=True))
55     idx = np.concatenate(indices, axis=1)
56     means = {v: x.mean(axis=1)[idx].mean(axis=1) for v, x in data.items()}
57     change = 100 * (means[selected] / means["BASE"] - 1)
58     p95 = {v: np.percentile(x[idx].reshape(20000, -1), 95, axis=1)
59            for v, x in data.items()}
60     return {"layouts": len(seeds), "resamples": 20000,
61            "mean_change_pct_ci95": np.percentile(change, [2.5, 97.5]).tolist(),
62            "p95_change_seconds_ci95": np.percentile(p95[selected] - p95["BASE"], [2.5, 97.5]).tolist()}
63
64
65 def main():
66     evidence = read(INNOV / "FINAL_EVIDENCE.json")
67     lock = read(INNOV / "validation_lock.json")
68     generation = read(INNOV / "validation_generation.json")
69     selected = lock["selected"]
70     assert selected == evidence["selected"] == "GATED_TOUR_MULTI_PARALLAX"
71     assert read(INNOV / "best_candidate.json") == lock["configs"][selected]
72     assert generation["after_lock_sha256"] == sha(INNOV / "validation_lock.json")
73     assert generation["fixtures_sha256"] == sha(INNOV / "final_validation_cases.json")
74     assert datetime.fromisoformat(lock["created_utc"]) < datetime.fromisoformat(generation["generated_utc"])
75     for p, h in lock["code_sha256"].items():

```

```

76     assert sha(INNOV / p) == h, p
77     manifest = read(INNOV / "EXPORT_MANIFEST.json")
78     for p, entry in manifest["files"].items():
79         assert sha(INNOV / p) == entry["sha256"], p
80     fixtures = read(INNOV / "final_validation_cases.json")
81     dev = read(INNOV / "structured_development.json")
82     assert {f["seed"] for f in fixtures}.isdisjoint({f["seed"] for f in dev})
83     final, costs, families, pair_rows = [], [], [], []
84     verification = {"selected": selected, "source_lock_verified": True,
85                    "export_files_verified": len(manifest["files"]),
86                    "selection_before_final_generation": True,
87                    "final_layouts": len({f["seed"] for f in fixtures}),
88                    "final_case_error_pairs_per_rounding": len(fixtures), "bootstrap": {}}
89     for rounding in ("bounded", "pre_round_stress"):
90         folder = INNOV / "final_validation" / rounding
91         rows = read(folder / "summary.json")["rows"]
92         assert len(rows) == 3840
93         configs = read(folder / "configs.json")
94         assert all(config == lock["configs"][name] for name, config in configs.items())
95         by = {v: {r["case_id"]: r for r in rows if r["variant"] == v}
96              for v in configs}
97         assert all(set(x) == {f["id"] for f in fixtures} for x in by.values())
98         for variant, runs in by.items():
99             times = np.array([r["penalized_virtual_s"] for r in runs.values()])
100             raw = np.array([r["metrics"]["virtual_time_s"] for r in runs.values()])
101             assert all(r["success"] and r["status"] == "complete" and
102                       r["total"] == r["cleared"] and r["audit"] == "pass" and
103                       not r["timeout"] and not r["false_complete"] for r in runs.values())
104             assert np.array_equal(times, raw)
105             source = evidence["final"][rounding]["aggregates"][variant]
106             stats = {"mean": float(times.mean()), "p95": float(np.percentile(times, 95)),
107                     "maximum": float(times.max()),
108                     "mean_per_cleared": float(np.mean([r["metrics"]["virtual_time_s"] / r["cleared"] for r in
109                                                         ↪ runs.values()]))}
110             for key, val in stats.items():
111                 assert math.isclose(val, source[key], abs_tol=1e-8), (rounding, variant, key)
112             final.append({"rounding": rounding, "variant": variant, "runs": len(runs), "complete": len(runs),
113                           ↪ **stats})
114             if variant in ("BASE", selected):
115                 for component in ("movement_s", "switch_s", "detection_s", "optical_s", "laser_s"):
116                     value = float(np.mean([r["metrics"]["components"][component] for r in runs.values()]))
117                     assert math.isclose(value, source["movement_detection_etc"][component], abs_tol=1e-8)
118                     costs.append({"rounding": rounding, "variant": variant, "component": component, "mean_s": value})
119                 for f in sorted({f["family"] for f in fixtures}):
120                     ids = {x["id"] for x in fixtures if x["family"] == f}
121                     values = [runs[k]["penalized_virtual_s"] for k in sorted(ids)]
122                     families.append({"rounding": rounding, "variant": variant, "family": f,
123                                     "layouts": 20, "runs": len(values), "mean_s": float(np.mean(values)),
124                                     "p95_s": float(np.percentile(values, 95))})
125             for case_id, b in by["BASE"].items():
126                 c = by[selected][case_id]
127                 pair_rows.append({"rounding": rounding, "case_id": case_id,
128                                   "baseline_s": b["penalized_virtual_s"], "candidate_s": c["penalized_virtual_s"],
129                                   "change_s": c["penalized_virtual_s"] - b["penalized_virtual_s"],
130                                   "change_pct": 100 * (c["penalized_virtual_s"] / b["penalized_virtual_s"] - 1)})
131             boot = block_bootstrap(rows, selected, fixtures)
132             for key, val in boot.items():
133                 assert np.allclose(val, evidence["final"][rounding]["bootstrap"][key], atol=1e-10)
134             verification["bootstrap"][rounding] = boot
135             save_csv("q3_final_statistics.csv", final)
136             save_csv("q3_cost_components.csv", costs)
137             save_csv("q3_family_statistics.csv", families)
138             save_csv("q3_final_pairs.csv", pair_rows)
139             rounds = []
140             for name, item in evidence["rounds"].items():
141                 for rounding, batch in item["batches"].items():
142                     rounds.append({"round": name, "candidate": item["variant"], "parent": item["parent"],
143                                   "rounding": rounding, "stage": "development",
144                                   "baseline_mean_s": batch["baseline"]["mean"],
145                                   "candidate_mean_s": batch["candidate"]["mean"],
146                                   "vs_baseline_pct": batch["candidate"]["mean_change_pct"],
147                                   "vs_parent_pct": batch["direct_parent_mean_change_pct"]})
148             save_csv("q3_mechanism_development.csv", rounds)

```

```

147 # Read official practice summaries without exporting team identity or tickets.
148 official = ROOT / "JammersSimulatorData"
149 snapshot = read(OUT / 'q3_official_snapshot.json')
150 if (official / 'practice-statistics-queue.sqlite3').exists():
151     conn = sqlite3.connect(f"file:{official / 'practice-statistics-queue.sqlite3'}?mode=ro", uri=True)
152     conn.row_factory = sqlite3.Row
153     records = [dict(r) for r in conn.execute("SELECT
        ↳ practice_run_no,case_code,problem_no,entered,end_reason,cleared_jammer_count,jammer_count,virtual_time_us,
        ↳ ,measure_accepted_count,channel_switch_count,clear_failure_count,state,created_at_ms FROM
        ↳ practice_statistics_tasks ORDER BY created_at_ms")]
154     conn.close()
155 else:
156     records = snapshot['records']
157 official_rows = []
158 for r in records:
159     base = official / "behavior-logs" / f"practice-p{r['problem_no']}-{r['practice_run_no']}-{r['case_code']}"
160     result_path = base.with_suffix(".result.json")
161     result = read(result_path)
162     assert result["practice_run_no"] == r["practice_run_no"] and result["jammer_count"] == r["jammer_count"]
163     log_path = base.with_suffix(".jlog")
164     log_hash = sha(log_path) if log_path.exists() else snapshot["log_projection"][log_path.name]["sha256"]
165     assert result["package_sha256"] == log_hash
166     for ext, kind in ((".jlog", "practice_behavior_log"), (".psum", "practice_summary")):
167         log_path = base.with_suffix(ext)
168         if log_path.exists():
169             data = log_path.read_bytes()
170             n = int.from_bytes(data[10:14], "big")
171             header = json.loads(data[14:14+n])
172         else:
173             header = snapshot["log_projection"][log_path.name]["public_header"]
174             assert header["package_type"] == kind and header["problem_no"] == 3
175             assert header["formal_index"] is None and header["practice_run_no"] == r["practice_run_no"]
176             assert header["case_code"] == r["case_code"]
177         official_rows.append({"case_code": r["case_code"], "problem_no": r["problem_no"],
178             "started_utc": result["window_started_at_utc"], "ended_utc": result["ended_at_utc"],
179             "entered": r["entered"], "end_reason": r["end_reason"],
180             "cleared": r["cleared_jammer_count"], "total": r["jammer_count"],
181             "virtual_time_s": r["virtual_time_us"] / 1e6,
182             "mean_per_cleared_s": r["virtual_time_us"] / 1e6 / r["cleared_jammer_count"],
183             "measures": r["measure_accepted_count"], "switches": r["channel_switch_count"],
184             "failed_clears": r["clear_failure_count"], "local_queue_state": r["state"],
185             "algorithm_configuration": "unresolved",
186             "result_sha256": sha(result_path), "jlog_sha256": result["package_sha256"]})
187 save_csv("q3_official_practice.csv", official_rows)
188 if (official / 'formal-statistics-queue.sqlite3').exists():
189     conn = sqlite3.connect(f"file:{official / 'formal-statistics-queue.sqlite3'}?mode=ro", uri=True)
190     formal_count = conn.execute("SELECT COUNT(*) FROM statistics_tasks").fetchone()[0]
191     conn.close()
192     conn = sqlite3.connect(f"file:{official / 'upload-queue.sqlite3'}?mode=ro", uri=True)
193     upload_count = conn.execute("SELECT COUNT(*) FROM upload_tasks").fetchone()[0]
194     conn.close()
195 else:
196     formal_count = snapshot['formal_local_rows']
197     upload_count = snapshot['upload_local_rows']
198 values = np.array([r["virtual_time_s"] for r in official_rows])
199 practice_summary = {"runs": len(official_rows), "unique_case_codes": len({r["case_code"] for r in official_rows}),
200     "complete_by_official_counts": sum(r["entered"] == 1 and r["cleared"] == r["total"] for r in
        ↳ official_rows),
201     "queue_states": dict(Counter(r["local_queue_state"] for r in official_rows)),
202     "end_reasons": dict(Counter(r["end_reason"] for r in official_rows)),
203     "mean_s": float(values.mean()), "p95_s": float(np.percentile(values, 95)),
204     "minimum_s": float(values.min()), "maximum_s": float(values.max()),
205     "total_cleared": sum(r["cleared"] for r in official_rows),
206     "mean_per_cleared_s": float(np.mean([r["mean_per_cleared_s"] for r in official_rows])),
207     "formal_local_rows": formal_count, "upload_local_rows": upload_count,
208     "algorithm_attribution": "not established; encrypted logs do not expose algorithm
        ↳ configuration",
209     "scope": "local official-simulator records; no live backend or formal score verification"}
210 practice_summary["source_mode"] = "original_local_records" if (official /
        ↳ "practice-statistics-queue.sqlite3").exists() else "anonymous_snapshot_with_original_hashes"
211 verification["official_practice"] = practice_summary
212 verification["environment"] = {"numpy": np.__version__}
213 verification["input_sha256"] = {str(p.relative_to(ROOT)): sha(p) for p in

```

```

214             [INNOV / "FINAL_EVIDENCE.json", INNOV / "validation_lock.json",
215              INNOV / "final_validation/bounded/summary.json",
216              INNOV / "final_validation/pre_round_stress/summary.json"]}
217     (OUT / "q3_verification.json").write_text(json.dumps(verification, ensure_ascii=False, indent=2) + "\n",
↪     encoding="utf-8")
218     print(json.dumps(verification, ensure_ascii=False, indent=2))
219
220
221 if __name__ == "__main__":
222     main()

```

程序 4

对应文件: manuscript/analysis/q4_verify.py

```

1 """Read-only verification of existing Q4 evidence; never runs a strategy/API.
2
3 Run from project root: python manuscript/analysis/q4_verify.py
4 Outputs derived tables with original input SHA256 hashes.
5 """
6 from pathlib import Path
7 import collections
8 import csv
9 import hashlib
10 import json
11 import math
12 import numpy as np
13
14 ROOT = Path(__file__).resolve().parents[2]
15 OUT = Path(__file__).resolve().parent
16 RESET = ROOT / 'paper_evidence/q4_mechanism_reset_20260912'
17 inputs = {}
18
19
20 def read(path):
21     path = Path(path)
22     raw = path.read_bytes()
23     inputs[str(path.relative_to(ROOT))] = hashlib.sha256(raw).hexdigest()
24     return json.loads(raw)
25
26
27 def summarize(rows):
28     times = np.array([r['metrics']['virtual_time_s'] for r in rows])
29     counts = np.array([r['cleared'] for r in rows])
30     return {
31         'runs': len(rows),
32         'complete': sum(r['status'] == 'complete' and r['cleared'] == r['total'] and r['audit']['passed'] for r in
↪     rows),
33         'mean_s': float(times.mean()),
34         'p95_s': float(np.quantile(times, .95)),
35         'max_s': float(times.max()),
36         'mean_of_per_game_time_per_cleared_source_s': float(np.mean(times / counts)),
37         'pooled_time_per_source_s': float(times.sum() / counts.sum()),
38         'mean_clearance_ratio': float(np.mean(counts / np.array([r['total'] for r in rows]))),
39         'mean_source_count': float(counts.mean()),
40     }
41
42
43 latest = read(RESET / 'final_validation/summary.json')
44 published = read(RESET / 'FINAL_STATISTICS.json')['configurations']
45 latest_rows = latest['rows']
46 assert len(latest_rows) == 4800
47 assert len({r['config'], r['case_id']} for r in latest_rows) == 4800
48 assert len({r['layout_sha256'] for r in latest_rows}) == 120
49 stats = {}
50 stop_counts = {}
51 for name in sorted(published):
52     rows = [r for r in latest_rows if r['config'] == name]
53     assert len(rows) == 960
54     assert all(not r['timeout'] and not r['false_complete'] and r['success'] for r in rows)
55     stops = collections.Counter()

```

```

56     for row in rows:
57         stored = read(RESET / 'final_validation' / name / row['case_id'] / 'result.json')
58         assert stored['summary'] == row
59         assert stored['strategy']['metrics'] == row['metrics']
60         assert len(stored['strategy']['cleared']) == row['cleared']
61         assert row['penalized_virtual_s'] == row['metrics']['virtual_time_s']
62         stops[row['audit']['additional']['stop']['kind']] += 1
63     stats[name] = summarize(rows)
64     for key in ('mean_s', 'p95_s', 'max_s'):
65         assert abs(stats[name][key] - published[name][key]) < 1e-8
66     stop_counts[name] = dict(stops)
67
68 historical = {}
69 review = read(ROOT / 'question4/results/round9_review.json')
70 historical_rows = []
71 for batch in ('round9_main', 'round9_rounding'):
72     summary = read(ROOT / 'question4/results' / batch / 'summary.json')
73     historical[batch] = {}
74     for name in ('BASE', 'FINAL', 'COMPACT22', 'NO_CENTROID', 'NO_INFO'):
75         rows = [r for r in summary['rows'] if r['config'] == name]
76         assert len(rows) == 96
77         for row in rows:
78             raw = read(ROOT / 'question4/results' / batch / name / row['case_id'] / 'result.json')
79             assert raw['metrics'] == row['metrics']
80             historical[batch][name] = summarize(rows)
81             for key in ('mean_s', 'p95_s'):
82                 assert abs(historical[batch][name][key] - review[batch]['vs_baseline'][name][key]) < 1e-8
83             historical_rows.extend(dict(batch=batch, **r) for r in rows)
84
85 # Explicit geometric construction, independent of the legacy certificate auditor.
86 inner = 950.0
87 outer = 1870.0
88 edges = {
89     'origin_inner': inner,
90     'inner_inner': 2 * inner * math.sin(math.pi / 12),
91     'outer_outer': 2 * outer * math.sin(math.pi / 12),
92     'adjacent_inner_outer': math.sqrt(inner**2 + outer**2 - 2 * inner * outer * math.cos(math.pi / 12)),
93 }
94 assert max(edges.values()) < 1000
95 apothem = outer * math.cos(math.pi / 12)
96 assert apothem > 1800
97
98 fields = ['batch', 'config', 'case_id', 'layout_id', 'family', 'error_mode', 'rounding',
99           'virtual_time_s', 'cleared', 'total', 'time_per_source_s']
100 with (OUT / 'q4_source_data.csv').open('w', newline='', encoding='utf-8') as stream:
101     writer = csv.DictWriter(stream, fieldnames=fields)
102     writer.writeheader()
103     for row in historical_rows + [dict(batch='reset_final_validation', **r) for r in latest_rows]:
104         record = {k: row.get(k, '') for k in fields}
105         record['virtual_time_s'] = row['metrics']['virtual_time_s']
106         record['time_per_source_s'] = row['metrics']['virtual_time_s'] / row['cleared']
107         writer.writerow(record)
108
109 result = {
110     'scope': 'Existing result JSON and summary consistency; no official or self-built strategy runs.',
111     'latest': stats,
112     'historical': historical,
113     'latest_stop_counts': stop_counts,
114     'geometry_25_stations': {'edge_lengths_m': edges, 'outer_apothem_m': apothem,
115                             'triangle_count_explicit_construction': 36},
116     'per_source_aggregation': 'Mean of each game virtual_time_s/cleared; pooled ratio separately labelled.',
117     'inputs_verified': len(inputs),
118     'input_sha256': inputs,
119 }
120 (OUT / 'q4_verification.json').write_text(json.dumps(result, indent=2, ensure_ascii=False) + '\n')
121 print(json.dumps({k: result[k] for k in ('latest', 'historical', 'latest_stop_counts', 'geometry_25_stations',
    ↪ 'inputs_verified')}, ensure_ascii=False, indent=2))

```

程序 5

对应文件: manuscript/package_sources.py


```

1 """Build a size-bounded, reproducible support archive without altering originals.
2 Run from the project root: python manuscript/package_sources.py
3 Rebuild after the manuscript and AI detail PDF have been compiled.
4 """
5 from pathlib import Path
6 import io, json, hashlib, tarfile, zipfile, shutil, re, csv
7 ROOT=Path(__file__).resolve().parent.parent
8 M=ROOT/'manuscript'; STAGE=M/'build/support'; STAGE.mkdir(parents=True,exist_ok=True)
9 selected=set()
10 def add(p):
11     p=Path(p)
12     if not p.is_absolute():p=ROOT/p
13     if p.is_file() and '__pycache__' not in p.parts and p.suffix!='.pyc':selected.add(p.relative_to(ROOT))
14 def glob(pattern):
15     for p in ROOT.glob(pattern):add(p)
16 # Original problem files are needed by the source-hash verification.
17 for p in ROOT.glob('B 题/**/*.py'):
18     add(p)
19 # Complete live runtime and figure/analysis programs.
20 for q in ('question1','question2','question3','question4'):
21     glob(q+'/*.py');glob(q+'/requirements*.txt');glob(q+'/*.md')
22     glob(q+'/configs/**/*.json')
23     glob('question3/tests/*.py');glob('question3/docs/*.md')
24     glob('question3/innovation/*.py');glob('question4/innovation/*.py')
25     glob('question4/certificates/*.json');glob('question4/docs/*.md')
26 for p in ('question1/example_result.json','question2/results.json','question2/round1_tradeoff.json') :add(p)
27 # The complete Q3 export is required by its frozen provenance checker.
28 e=json.loads((ROOT/'question3/innovation/EXPORT_MANIFEST.json').read_text())
29 for p in e['files']:add(Path('question3/innovation')/p)
30 add('question3/innovation/EXPORT_MANIFEST.json')
31 # Original Q4 result bodies used by the independent consistency check.
32 e=json.loads((M/'analysis/q4_verification.json').read_text())
33 for p in e['input_sha256']:add(p)
34 # Source, locks, summaries and decision records of mechanism exploration.
35 reset='paper_evidence/q4_mechanism_reset_20260912'
36 for pat in ('*.py','*.json','*.md','validation_lock/*','reporting/*.py','reporting/WORST_CASE.*','source_review/*.json',
37     ↪ n','qa/*.py','qa/*.json','qa/*.md','final_validation/*.json','round/**/*.md','round/**/*.json','round*/development/*.json',
38     ↪ 'round*/development/code_snapshot/**/*.py','final_validation/code_snapshot/**/*.py','frozen/**/*.py'):
39     glob(reset+'/'+pat)
40 for pat in ('*.json','*.md','results/*comparison*.json','results/*statistics*.json','results/*evidence*.json','result'
41     ↪ s/*selection*.json','locks/**/*.json','frozen_baseline/**/*.py'):
42     glob('question4/innovation/'+pat)
43     glob(reset+'**/selection_snapshot.txt')
44     glob('question4/innovation/**/selection_snapshot.txt')
45 for pat in ('results*/summary.json','results*/fixtures.json','results*/configs.json','results*/lock.json','result'
46     ↪ s/*review*.json','results/negative21*.json'):
47     glob('question4/'+pat)
48 # Historical Q3 numerical reports and code snapshots underlying selected comparisons.
49 for prefix in ('round1_','round13_','round14_','round17_','round18_','round19_'):
50     for p in (ROOT/'question3/results').glob(prefix+'*'):
51         if p.is_dir():
52             for fn in ('summary.json','comparison.json','configs.json','paired_cases.csv'):add(p/fn)
53             for f in p.glob('code_snapshot/**/*.py'):add(f)
54         elif p.suffix=='.json':add(p)
55 # Representative Q4 failure case includes complete public actions and events.
56 for variant in ('BASE25','COVER_EXCHANGE_ELASTIC'):
57     for name in ('actions.jsonl','strategy.jsonl','result.json'):
58         add(f'{reset}/final_validation/{variant}/uniform_4c920b65f2c8599c__positive__bounded/{name}')
59 # Keep official practice original result sidecars only; encrypted envelopes contain team identity.
60 glob('JammersSimulatorData/behavior-logs/*.result.json')
61 # Manuscript source and derived analyses, excluding transient build files and private local paths.
62 glob('manuscript/sections/*.tex');add('manuscript/main.tex');add('manuscript/ai_details.tex')
63 glob('manuscript/analysis/*');glob('manuscript/figures/*')
64 for p in ('README.md','requirements.txt','build.sh','restore_evidence.py'):add('manuscript/'+p)
65 add('manuscript/package_sources.py')
66 for p in M.glob('audit/*.md'):add(p)
67 for p in M.glob('audit/reviews/*.md'):add(p)
68 # Single archive compression shares repeated result keys across files.
69 records=[];py_unique={};changed=[]
70 regular={};evidence={}
71 for rel in sorted(selected):
72     data=(ROOT/rel).read_bytes();sha=hashlib.sha256(data).hexdigest()
73 # Anonymous diagnostic/lock metadata copies; original hash chains still refer to originals.

```

```

70 # Executable code, algorithm configurations and numerical result values remain unchanged.
71 if (rel.parts[0]=='manuscript' or str(rel).startswith(reset+'/'+')) and rel.suffix in ('.md','.json','.txt'):
72     raw=data.decode('utf-8');clean=raw.replace(str(ROOT),'PROJECT_ROOT').replace(str(Path.home())+'/', 'USER_HOME/')
73     if clean!=raw:data=clean.encode();changed.append(str(rel))
74     output_sha=hashlib.sha256(data).hexdigest()
75     records.append({'path':str(rel),'bytes':len(data),'original_sha256':sha,'sha256':output_sha,'copy_anonymized':sha!=o
    ↳ utput_sha})
76 if rel.suffix=='.py':py_unique.setdefault(output_sha,[]).append(str(rel))
77 if rel.parts[0]=='manuscript' or (rel.suffix=='.py' and 'frozen' not in str(rel) and 'code_snapshot' not in
    ↳ rel.parts) or ('configs' in rel.parts and len(rel.parts)<5) or rel.name.startswith('requirements'):
78     regular[str(rel)]=data
79 else:evidence[str(rel)]=data
80 # Delete only this builder's previous support staging directory.
81 shutil.rmtree(STAGE);STAGE.mkdir(parents=True)
82 for rel,data in regular.items():
83     p=STAGE/rel;p.parent.mkdir(parents=True,exist_ok=True);p.write_bytes(data)
84 archive=STAGE/'evidence.tar.gz'
85 with tarfile.open(archive,'w:gz',compresslevel=9) as tar:
86     for rel,data in evidence.items():
87         info=tarfile.TarInfo(rel);info.size=len(data);info.mtime=0;info.mode=0o644;tar.addfile(info,io.BytesIO(data))
88 manifest={'version':1,'files':records,'regular_files':len(regular),'archived_files':len(evidence),'anonymized_copies'
    ↳ :changed,
89 'anonymized_lock_note':'Absolute local paths in Q4 lock/command metadata are projected to PROJECT_ROOT/USER_HOME.
    ↳ Existing lock-chain hashes refer to original files, not the projected bytes. Consult original_sha256; byte-level
    ↳ original lock verification requires the local originals.',
90 'excluded':'Original credential databases, encrypted practice envelopes with team identity, complete unused action
    ↳ histories; formal logs not available.'}
91 (STAGE/'SUPPORT_MANIFEST.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2))
92 # Copy the restore script to the top-level for a one-command extraction.
93 shutil.copy2(M/'restore_evidence.py',STAGE/'restore_evidence.py')
94 # Exact complete source bodies, once per content hash; duplicate locations have a map.
95 code_dir=M/'build/code';code_dir.mkdir(exist_ok=True)
96 code_tex=[];code_map=[]
97 for i,(sha,paths) in enumerate(sorted(py_unique.items(),key=lambda x:x[1][0]),1):
98     representative=paths[0];data=(ROOT/representative).read_bytes();out=code_dir/f'{i:03}.py';out.write_bytes(data)
99     code_tex += [f'\\subsection*{{{程序 {i}: \\texttt{{{Path(representative).name.replace("_",r"\\_")}}}}}'],
100     '\\noindent 文件路径: \\path{'+representative+'}\\par',
101     f'\\VerbatimInput[fontsize=\\fontsize{{{7.3}}}{{{8.5}}}\\selectfont,breaklines=true,breakanywhere=true,tabs=4,number
    ↳ s=left,numbersep=4pt,xleftmargin=12pt]{{{build/code/{i:03}.py}}}]'
102     code_map.append({'program':i,'sha256':sha,'paths':paths})
103 (M/'build/code_appendix.tex').write_text('\\n'.join(code_tex))
104 (M/'build/source_map.json').write_text(json.dumps(code_map,ensure_ascii=False,indent=2))
105 # Include all original filenames via a compact full list in the appendix, grouped by directories.
106 dirs={}
107 for name in regular:
108     p=Path(name);dirs.setdefault(str(p.parent),[]).append(p.name)
109 tex=[f'支撑包保存 {len(records)} 个来源文件, 其中 {len(py_unique)} 份内容不同的完整Python源程序。
    ↳ 相同源码的多个冻结路径不重复印刷, 映射见\\path{{{manuscript/build/source_map.json}}}。',
110 '较大的逐例结果以\\path{evidence.tar.gz}保存, 解压支撑ZIP后运行\\texttt{python
    ↳ restore\\_evidence.py}恢复原目录并校验哈希。',
111 '演练原始加密包含身份字段, 保留于本地项目; 支撑包只含脱敏统计投影、原结果旁文件及原件SHA256。它可重算统计,
    ↳ 但不替代原件身份或签名校验。',
112 '部分问题四历史命令与锁元数据的本机路径已脱敏, 原件及副本哈希均列入清单; 旧锁链哈希仍指向本地原件,
    ↳ 不宣称脱敏副本与原锁字节相同。',
113 '顶层文件为\\path{evidence.tar.gz}、\\path{SUPPORT_MANIFEST.json}、\\path{restore_evidence.py}及AI工具使用详情PDF。
    ↳ 以下按目录列出直接文件; 压缩证据的完整内部成员由\\path{SUPPORT_MANIFEST.json}列出。']
114 # Directory-level list is readable; full path list delivered in machine-readable manifest.
115 for directory,names in sorted(dirs.items()):
116     if len(directory.split('/'))>5 and ('final_validation' in directory or 'results' in directory):continue
117     tex.append('\\par\\noindent\\path{'+directory+'}/: '+str(len(names))+ '个文件。\\par')
118     tex.append('\\begin{group}\\footnotesize '+ '\\allowbreak; '.join('\\path{'+n+'}' for n in names)+'\\par\\endgroup')
119 (M/'build/support_inventory.tex').write_text('\\n'.join(tex))
120 for fn in ('code_appendix.tex','support_inventory.tex','source_map.json'):
121     dest=STAGE/'manuscript/build'/fn;dest.parent.mkdir(parents=True,exist_ok=True);shutil.copy2(M/'build'/fn,dest)
122 shutil.copypath(code_dir,STAGE/'manuscript/build/code',dirs_exist_ok=True)
123 # Final human documents are added when available.
124 for fn in ('AI 工具使用详情.pdf',):
125     p=M/'delivery'/fn
126     if p.exists():shutil.copy2(p,STAGE/fn)
127 zip_path=M/'delivery/支撑材料.zip';zip_path.parent.mkdir(exist_ok=True)
128 with zipfile.ZipFile(zip_path,'w',zipfile.ZIP_DEFLATED,compresslevel=9) as z:
129     for p in sorted(STAGE.rglob('*')):
130         if p.is_file():z.write(p,str(p.relative_to(STAGE)))

```

```

131 summary={'source_files':len(records),'unique_python_sources':len(py_unique),'python_lines':sum(len((ROOT/v[0]).read_text()
    ↳ ext().splitlines()) for v in
    ↳ py_unique.values()),'evidence_gzip_bytes':archive.stat().st_size,'support_zip_bytes':zip_path.stat().st_size,'with_
    ↳ hin_20MB':zip_path.stat().st_size<=20_000_000}
132 (M/'build/package_result.json').write_text(json.dumps(summary,indent=2));print(json.dumps(summary))

```

程序 6

对应文件: manuscript/package_submission.py

```

1 """Package the current paper, complete source appendix and necessary evidence.
2 Run with Python 3.13: python manuscript/package_submission.py
3 No simulator is started. ZIP and PDF must each remain below 20,000,000 bytes.
4 """
5 from pathlib import Path
6 import ast, hashlib, io, json, re, shutil, tarfile, zipfile
7 from PIL import Image, PngImagePlugin
8 ROOT = Path(__file__).resolve().parents[1]
9 M = ROOT / 'manuscript'
10 D = M / 'story_draft'
11 BUILD = M / 'build'
12 STAGE = BUILD / 'submission_support'
13 # Reuse the historical evidence selection, not its obsolete manuscript or output.
14 namespace = {'__file__': str(M / 'package_sources.py')}
15 selection = (M / 'package_sources.py').read_text().split('# Single archive compression')[0]
16 exec(compile(selection, 'package_sources_selection', 'exec'), namespace)
17 selected = {p for p in namespace['selected'] if p.parts[0] not in ('manuscript', 'JammersSimulatorData')}
18 def add(path):
19     p = Path(path)
20     if not p.is_absolute(): p = ROOT / p
21     if p.is_file() and '__pycache__' not in p.parts and p.suffix != '.pyc':
22         selected.add(p.relative_to(ROOT))
23 # Exact image assets referenced by the current TeX, never an old illustration set.
24 tex = (D / '中文论文初稿.tex').read_text()
25 for name in re.findall(r'\\includegraphics\\[\\^\\]*\\{\\{\\{\\}\\}\\}\\}', tex): add(D / name)
26 for pattern in ('analysis/*.py', 'analysis/source_data/*', 'figures/*.csv', 'figures/*.json',
    ↳ 'plot_data/20260913_process/*'):
27     for p in D.glob(pattern): add(p)
28 for name in ('中文论文初稿.md', '中文论文初稿.tex'): add(D / name)
29 for name in ('requirements.txt', 'package_sources.py', 'package_submission.py', 'ai_details.tex'): add(M / name)
30 for p in (M / 'analysis').glob('*'): add(p)
31 for q, indices in ((3, (1,2,4)), (4, (1,2,3))):
32     for p in (ROOT / f'question{q}/results/jlog_question{q}').glob(f'formal-p{q}-*.jlog'): add(p)
33     for i in indices:
34         for p in (ROOT / f'question{q}/results/windows_q{q}_final_{i:03d}').rglob('*'): add(p)
35 # Supplementary dispatch development discussed in Appendix B.5.
36 for p in (ROOT/'question3/dispatch_refinement').rglob('*'): add(p)
37 for p in (ROOT/'question3/results/dispatch_20260913').rglob('*'):
38     if p.name in ('summary.json','comparison.json','configs.json','fixtures.json','lock.json') or p.suffix == '.py':
    ↳ add(p)
39 # Scripts/data for process figures use these compact inputs and the preserved formal runs.
40 for script in ('export_q3_convergence.py','export_q4_discovery.py','export_formal_process.py'):
41     add(D/'analysis'/script)
42 # Unused whole-project manifests index excluded historical traces, not paper inputs.
43 for name in ('paper_evidence/q4_mechanism_reset_20260912/ARTIFACT_SHA256.json',
    ↳ 'question4/innovation/frozen_manifest.json'):
44     selected.discard(Path(name))
45 STAGE.mkdir(parents=True, exist_ok=True)
46 shutil.rmtree(STAGE)
47 STAGE.mkdir()
48 records=[]; sources={}; regular={}; evidence={}
49 for rel in sorted(selected):
50     data=(ROOT/rel).read_bytes()
51     original=hashlib.sha256(data).hexdigest()
52     if rel.suffix == '.png':
53         # Lossless encoding only: preserve pixels, dimensions and ancillary metadata.
54         im=Image.open(io.BytesIO(data)); metadata=PngImagePlugin.PngInfo(); offset=8
55         while offset < len(data):
56             size=int.from_bytes(data[offset:offset+4], 'big'); kind=data[offset+4:offset+8]
57             if kind not in (b'IHDR',b'IDAT',b'IEND'):
58                 metadata.add(kind,data[offset+8:offset+8+size])

```

```

59         offset+=size+12
60         output=io.BytesIO();im.save(output,format='PNG',optimize=True,compress_level=9,pnginfo=metadata)
61         optimized=output.getvalue()
62         assert Image.open(io.BytesIO(optimized)).tobytes()==im.tobytes()
63         if len(optimized)<len(data): data=optimized
64         # Diagnostic copies may contain local usernames; retain both hashes.
65         if rel.suffix in ('.py','.md','.json','.jsonl','.txt','.tex','.csv','.sh'):
66             try:
67                 text=data.decode('utf-8')
68                 text=text.replace(str(ROOT), 'PROJECT_ROOT').replace(str(Path.home())+'/', 'USER_HOME/')
69                 # Windows profile components are diagnostic paths, not model inputs.
70                 text=re.sub(r'(?:i)([A-Z]:[\\\/]+Users[\\\/]+)[^\\\/s"\']+', r'\\USER', text)
71                 data=text.encode('utf-8')
72             except UnicodeDecodeError: pass
73             sha=hashlib.sha256(data).hexdigest()
74             records.append(dict(path=str(rel),bytes=len(data),sha256=sha,original_sha256=original,copy_anonymized=sha!=original)
↳ al and rel.suffix!='.png',lossless_png=sha!=original and
↳ rel.suffix!='.png'))
75         if rel.suffix=='.py':
76             ast.parse(data.decode('utf-8-sig'),filename=str(rel))
77             sources.setdefault(sha,dict(data=data,paths=[]))['paths'].append(str(rel))
78         is_regular = rel.suffix in ('.png','.jpg','.pdf','.jlog') or str(rel) in
↳ ('manuscript/story_draft/中文论文初稿.tex','manuscript/story_draft/中文论文初稿.md')
79         (regular if is_regular else evidence)[str(rel)]=data
80 for name,data in regular.items():
81     p=STAGE/name;p.parent.mkdir(parents=True,exist_ok=True);p.write_bytes(data)
82 with tarfile.open(STAGE/'evidence.tar.xz','w:xz',preset=9) as tar:
83     for name,data in evidence.items():
84         info=tarfile.TarInfo(name);info.size=len(data);info.mtime=0;info.mode=0o644
85         tar.addfile(info,io.BytesIO(data))
86 restore=''from pathlib import Path
87 import hashlib,json,tarfile
88 root=Path(__file__).resolve().parent
89 with tarfile.open(root/'evidence.tar.xz','r:xz') as tar:
90     for item in tar:
91         dst=(root/item.name).resolve()
92         if not dst.is_relative_to(root) or not item.isfile(): raise ValueError('Unsafe member')
93         data=tar.extractfile(item).read()
94         if dst.exists() and dst.read_bytes()!=data: raise ValueError('Existing file differs: '+item.name)
95         dst.parent.mkdir(parents=True,exist_ok=True);dst.write_bytes(data)
96 for row in json.loads((root/'SUPPORT_MANIFEST.json').read_text())['files']:
97     assert hashlib.sha256((root/row['path']).read_bytes()).hexdigest()==row['sha256'],row['path']
98 print('All support hashes verified.')
99 '''
100 (STAGE/'restore_evidence.py').write_text(restore)
101 # Full code bodies appear once, with every duplicate frozen path explicitly mapped.
102 code_dir=BUILD/'submission_code';code_dir.mkdir(exist_ok=True)
103 code_tex=[];code_map=[]
104 for i,(sha,entry) in enumerate(sorted(sources.items(),key=lambda x:x[1]['paths'][0]),1):
105     name=f'{i:03d}.py';(code_dir/name).write_bytes(entry['data'])
106     code_tex += [r'\subsection*{程序 '+str(i)+'}',r'\noindent 对应文件: '+'; '.join(r'\path{'+p+'}' for p in
↳ entry['paths'])+r'\par',
107                 r'\VerbatimInput[fontsize=\fontsize{7.3}{8.5}\selectfont,breaklines=true,breakanywhere=true,tabsize=]
↳ 4,numbers=left,numbersep=4pt,xleftmargin=12pt]{../build/submission_code/'+name+'}']
108     code_map.append(dict(program=i,sha256=sha,paths=entry['paths']))
109 (BUILD/'submission_code.tex').write_text('\n'.join(code_tex))
110 # Print all direct file names. The single evidence archive has a full internal manifest.
111 inventory=[r'支撑包中的\path{evidence.tar.xz}保存较大的原始逐例结果、配置和冻结文件;
↳ 运行\path{restore_evidence.py}恢复目录,并按\path{SUPPORT_MANIFEST.json}逐文件校验。六份正式\path{.jlog}以原名、
↳ 原始字节直接保存在第三、四间日志目录。','r'全部完整Python源程序如下,相同内容的冻结副本只打印一次;对应路径全部列出。
↳ 诊断文本中的本机用户目录已替换,原件和副本哈希均保留;正式日志未作改动。
↳ ',r'\subsection*{支撑材料文件列表}','r'顶层文件: \path{README.md}、\path{AI工具使用详情.pdf}、
↳ \path{SUPPORT_MANIFEST.json}、\path{SOURCE_MAP.json}、\path{restore_evidence.py}、\path{evidence.tar.xz}。']
112 for name in sorted(regular): inventory.append(r'\par\noindent\path{'+name+'}')
113 inventory.append(r'\par\noindent 排版生成文件: \path{manuscript/build/submission_inventory.tex}、
↳ \path{manuscript/build/submission_code.tex}及\path{manuscript/build/submission_code/}中的编号源码副本。
↳ 压缩证据内部逐文件路径、大小和哈希见\path{SUPPORT_MANIFEST.json}。')
114 inventory_text='\n'.join(inventory)
115 # URL verbatim mode disables CJK switching; use ordinary text for Chinese names.
116 def cjk_path(match):
117     name=match.group(1)
118     if name.isascii(): return match.group(0)
119     escaped=name.replace('_',r'\_').replace('%',r'\%').replace('&',r'\&')

```

```

120     return r'\texttt{' + escaped + '}'
121 inventory_text=re.sub(r'\\path\{([^\}]+)\}', cjk_path, inventory_text)
122 (BUILD/'submission_inventory.tex').write_text(inventory_text)
123 for name in ('submission_code.tex', 'submission_inventory.tex'):
124     dst=STAGE/'manuscript/build'/name;dst.parent.mkdir(parents=True, exist_ok=True);shutil.copy2(BUILD/name, dst)
125 shutil.copytree(code_dir, STAGE/'manuscript/build/submission_code')
126 (STAGE/'SOURCE_MAP.json').write_text(json.dumps(code_map, ensure_ascii=False, indent=2))
127 shutil.copy2(M/'delivery/AI 工具使用详情.pdf', STAGE/'AI 工具使用详情.pdf')
128 (STAGE/'README.md').write_text(''# 支撑材料使用说明
129
130 对应论文：从两次测向到完整清除：有界误差下无线电干扰源的鲁棒定位与在线调度。
131
132 1. 先运行 `python restore_evidence.py`，恢复压缩证据并逐文件校验。
133 2. 安装 `python -m pip install -r manuscript/requirements.txt`。Python 3.13; NumPy 2.4、SciPy 1.17。
134 3. 基本检查：`python -m unittest discover -s question1 -p 'test*.py'`; `python -m unittest discover -s question2 -p
    ↳ 'test*.py'`。
135 4. 正式过程复算：`python manuscript/story_draft/analysis/export_formal_process.py`。仅读取已有日志，不启动测试。
136 5. 论文源文件是 `manuscript/story_draft/中文论文初稿.tex`，在所在目录用Tectonic或XeLaTeX编译；需Noto Serif/Sans CJK SC、
    ↳ Times New Roman和DejaVu Sans Mono字体。不要用早期的build_document.py覆盖现有TeX。
137
138 完整Python程序在论文附录F逐份收录；SOURCE_MAP.json映射重复冻结路径。较大历史证据以evidence.tar.xz保存；标准Python可解压。
    ↳ 保留正文最终逐例数值。必要对照、源程序与配置，不声称包含全部未采用研发方案的逐动作历史。第三、
    ↳ 四问六份正式日志原名和原字节保留，不修改其头信息或加密体；自动核验未解密官方行为体。
    ↳ 官方原件与旁路动作记录的联系按题号、顺序和时间戳核对。
139
140 诊断文本的本机绝对目录可能已替换为PROJECT_ROOT、USER_HOME或USER；清单保留原始及交付哈希，旧源码锁若指向原始字节，
    ↳ 不应把投影副本当作原件验证。数值未替换。不存在的本机工作目录需要按解压路径配置。
141
142 AI 用途与人工主导分工见 AI 工具使用详情.pdf；历史型号及逐项人工审查记录仍需参赛队据实补全。
143 '''
144 # Include generated files in the verification manifest, avoiding recursive self-hashes.
145 known={r['path'] for r in records}
146 for p in sorted(STAGE.rglob('*')):
147     name=str(p.relative_to(STAGE))
148     if p.is_file() and name not in known and name!='evidence.tar.xz':
149         data=p.read_bytes();sha=hashlib.sha256(data).hexdigest()
150         records.append(dict(path=name, bytes=len(data), sha256=sha, original_sha256=sha, copy_anonymized=False))
151 (STAGE/'SUPPORT_MANIFEST.json').write_text(json.dumps(dict(files=records, archive='evidence.tar.xz', formal_logs_unchan
    ↳ ged=True), ensure_ascii=False, indent=2))
152 zip_path=M/'delivery/支撑材料.zip'
153 with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED, compresslevel=9) as z:
154     for p in sorted(STAGE.rglob('*')):
155         if p.is_file(): z.write(p, str(p.relative_to(STAGE)))
156 result=dict(files=len(records), unique_sources=len(sources), source_lines=sum(len(x['data'].splitlines()) for x in
    ↳ sources.values()), zip_bytes=zip_path.stat().st_size, within_20MB=zip_path.stat().st_size<=20_000_000)
157 (BUILD/'submission_package_result.json').write_text(json.dumps(result, indent=2))
158 print(json.dumps(result))
159 if not result['within_20MB']: raise SystemExit('Support ZIP exceeds 20 MB')

```

程序 7

对应文件：manuscript/story_draft/analysis/build_document.py

```

1 """Convert the editable Chinese Markdown manuscript into a typeset LaTeX source."""
2 from pathlib import Path
3 import re, subprocess, shutil
4
5 ROOT=Path(__file__).resolve().parents[1]
6 pandoc=shutil.which('pandoc') or str(Path.home()/'./local/bin/pandoc')
7 body=subprocess.check_output([pandoc, str(ROOT/'中文论文初稿.md'), '-f', 'markdown+tex_math_dollars', '-t', 'latex', '--syn
    ↳ tax-highlighting=none'], text=True)
8 body=re.sub(r'\\pandocbounded\{\\includegraphics\[.*?\]\{figures\[^\}+\}\.png\}\\',
9     lambda m: r'\includegraphics[width=0.98\linewidth]{'+m[1]+'}.pdf', body)
10 body=body.replace(r'\def\LTcaptype{none}', r'\def\LTcaptype{table}')
11 body=body.replace(r'\begin{figure}', r'\begin{figure}[htbp]')
12 captions=iter(['核心集合与几何符号', '算法模块的输入、输出与限制', '自建实验组、
    ↳ 独立布局与用途', 'Q3锁定验证的完整流程结果', 'Q3共同父方案的机制矩阵', 'Q4最新锁定验证的完整流程结果', '构造等边三角形'
    ↳ 定位区域的测向数据', 'Q2固定候选库的完整复核结果', '主要结论的性质及证据边界'])
13 def convert_table(m):
14     chunk=m[0].replace(r'\begin{longtable}[]', r'\begin{tabular}').replace(r'\end{longtable}', r'\bottomrule'+'\n'+r'\e
    ↳ nd{tabular}')

```

```

15     chunk=chunk.replace(r'\endhead','').replace(r'\bottomrule\noalign{'+'\n'+r'\endlastfoot','')
16     return
    ↪ r'\begin{table}[H]+'+\n+r'\centering\small'+'\n'+r'\caption{'+next(captions)+'}\n'+chunk+'+\n'+r'\end{table}'
17 body=re.sub(r'\\begin{\longtable\}.*?\\end{\longtable\}',convert_table,body,flags=re.S)
18 # Keep the abstract on its own first page. Numbering is already explicit in the draft.
19 body=re.sub(r'\\section\{1\s+ 引言：问题重述与总体思路\}',lambda m:r'\clearpage'+'\n'+m[0],body)
20 preamble=r'''\documentclass[UTF8,fontset=none,zihao=-4,a4paper]{ctexart}
21 \usepackage[margin=25mm,footskip=11mm]{geometry}
22 \setCJKmainfont{Noto Serif CJK SC}[BoldFont=Noto Serif CJK SC Bold,ItalicFont=Noto Serif CJK SC]
23 \setCJKsansfont{Noto Sans CJK SC}
24 \setmainfont{Times New Roman}
25 \usepackage{fancyvrb}
26 \usepackage[section]{placeins}
27 \usepackage{amsmath,amssymb,booktabs,longtable,array,calc,graphicx,float,caption,xurl}
28 \usepackage[unicode,hidelinks,pdfaauthor={},pdftitle={从两次测向到完整清除：中文论文初稿}]{hyperref}
29 \setcounter{secnumdepth}{0}
30 \setlength{\parskip}{3pt}
31 \setlength{\emergencystretch}{3em}
32 \linespread{1.12}
33 \renewcommand{\arraystretch}{1.2}
34 \captionsetup{font=small,labelfont=bf,skip=6pt}
35 \ctexset{section={format=\large\bfseries,before=13pt,after=8pt},subsection={format=\normalsize\bfseries,before=
    ↪ eskip=9pt,after=5pt}}
36 \providecommand{\tightlist}{\setlength{\itemsep}{0pt}\setlength{\parskip}{0pt}}
37 \begin{document}
38 \begin{center}
39 {\zihao{3}\bfseries 从两次测向到完整清除：\\有界误差下无线电干扰源的鲁棒定位与在线调度}\par
40 \vspace{5pt}{\small 中文论文初稿}
41 \end{center}
42 ''
43 (ROOT/'中文论文初稿.tex').write_text(preamble+body+'\\n\\end{document}\\n')
44 print('Wrote self-contained LaTeX source; 19 vector figure references.')
```

程序 8

对应文件：manuscript/story_draft/analysis/check_artifacts.py

```

1 """Validate figure/source inventory, published aggregates, and rendered figure geometry."""
2 from pathlib import Path
3 import csv,json,re,sys,subprocess
4 import numpy as np
5 ROOT=Path(__file__).resolve().parents[1]
6 DATA=ROOT/'analysis/source_data'
7 SKILL=Path.home()/'codex/skills/nature-figure/scripts'
8 md=(ROOT/'中文论文初稿.md').read_text()
9 imgs=re.findall(r'!\\[[\\^\\]]+\\)\\((figures/[\\^\\]]+\\)\\)',md)
10 assert len(imgs)==19
11 for i,(_,p) in enumerate(imgs,1):
12     assert (ROOT/p).is_file()
13     assert f'图 {i}' in md
14 q3=list(csv.DictReader((DATA/'q3_final_pairs.csv').open()))
15 q3_stats=list(csv.DictReader((DATA/'q3_final_statistics.csv').open()))
16 checks=[]
17 for env in ['bounded','pre_round_stress']:
18     rr=[r for r in q3 if r['rounding']==env];assert len(rr)==480
19     for col,variant in [('baseline_s','BASE'),('candidate_s','GATED_TOUR_MULTI_PARALLAX')]:
20         vals=np.array([float(r[col]) for r in rr]);ref=next(r for r in q3_stats if r['rounding']==env and
    ↪ r['variant']==variant)
21         for key,calc in [(('mean',np.mean(vals)),('p95',np.quantile(vals,.95)),('maximum',np.max(vals)))]:
22             assert abs(calc-float(ref[key]))<1e-7
23         checks.append(dict(question='Q3',environment=env,variant=variant,runs=len(vals),mean=float(vals.mean())))
24 q4=list(csv.DictReader((DATA/'q4_source_data.csv').open()))
25 ref4=json.loads((DATA/'q4_verification.json').read_text())
26 for batch in sorted(set(r['batch'] for r in q4)):
27     for cf in sorted(set(r['config'] for r in q4 if r['batch']==batch)):
28         rr=[r for r in q4 if r['batch']==batch and r['config']==cf]
29         vals=np.array([float(r['virtual_time_s']) for r in rr]);ref=ref4['latest'][cf] if
    ↪ batch=='reset_final_validation' else ref4['historical'][batch][cf]
30         assert len(vals)==ref['runs'] and all(r['cleared']==r['total'] for r in rr)
31         for key,calc in [(('mean_s',np.mean(vals)),('p95_s',np.quantile(vals,.95)),('max_s',np.max(vals)))]:assert
    ↪ abs(calc-ref[key])<1e-7
```

```

32         checks.append(dict(question='Q4',batch=batch,config=cf,runs=len(vals),mean=float(vals.mean())))
33 out=ROOT/'qa'
34 (out/'numeric_checks.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2))
35 r=subprocess.run([sys.executable,str(SKILL/'validate_figure.py'),str(ROOT/'analysis/make_figures.py'),'--json'],capture_
    ↪ re_output=True,text=True)
36 (out/'source_check.json').write_text(r.stdout)
37 results=[]
38 for f in sorted((ROOT/'figures').glob('fig*.pdf')):
39     r=subprocess.run([sys.executable,str(SKILL/'audit_pdf_text.py'),str(f),'--json'],capture_output=True,text=True)
40     (out/f'{f.stem}.text.json').write_text(r.stdout);assert r.returncode==0
41     r=subprocess.run([sys.executable,str(SKILL/'audit_figure_collisions.py'),str(f),'--json-out',str(out/f'{f.stem}.c
    ↪ ollision.json')],capture_output=True,text=True)
42     results.append(dict(figure=f.stem,collision_exit_code=r.returncode))
43     if r.returncode:print(f.stem,r.stdout)
44 (out/'figure_check_summary.json').write_text(json.dumps(results,indent=2))
45 assert all(r['collision_exit_code']==0 for r in results)
46 print('PASS: 19 figures, every image explained, 19 aggregate groups reproduced, figure text and collision checks
    ↪ passed.')

```

程序 9

对应文件: manuscript/story_draft/analysis/export_formal_process.py

```

1 """Export plotting data from the six existing formal runs; never call the simulator.
2
3 Run from any directory: python manuscript/story_draft/analysis/export_formal_process.py
4 Original action logs stay local. Only compact derived data and provenance are exported.
5 """
6 from __future__ import annotations
7
8 import csv
9 import hashlib
10 import json
11 from collections import Counter
12 from datetime import datetime
13 from pathlib import Path
14
15 ROOT = Path(__file__).resolve().parents[3]
16 OUT = ROOT / "manuscript/story_draft/plot_data/20260913_process"
17 RUNS = [(3, 1, 1), (3, 2, 2), (3, 3, 4), (4, 1, 1), (4, 2, 2), (4, 3, 3)]
18
19
20 def read_jsonl(path):
21     return [json.loads(line) for line in path.read_text().splitlines() if line.strip()]
22
23
24 def write_csv(name, rows):
25     with (OUT / name).open("w", newline="", encoding="utf-8-sig") as stream:
26         writer = csv.DictWriter(stream, fieldnames=list(rows[0]), lineterminator="\n")
27         writer.writeheader()
28         writer.writerows(rows)
29
30
31 def seconds(value):
32     # The simulator responses expose virtual time to six decimal places.
33     return f"{value:.6f}"
34
35
36 def provenance(path):
37     return {"path": str(path.relative_to(ROOT)), "sha256": hashlib.sha256(path.read_bytes()).hexdigest()}
38
39
40 def main():
41     OUT.mkdir(parents=True, exist_ok=True)
42     points, summaries, events, sources = [], [], [], []
43     wide = [{"k_cleared": k} for k in range(17)]
44     for problem, formal_index, directory_index in RUNS:
45         run_dir = ROOT / f"question{problem}/results/windows_q{problem}_final_{directory_index:03d}"
46         jlogs = list((ROOT /
    ↪ f"question{problem}/results/jlog_question{problem}").glob(f"formal-p{problem}-{formal_index}-*.jlog"))
47         assert len(jlogs) == 1

```

```

48     jlog = jlogs[0]
49     raw = jlog.read_bytes()
50     # Only inspect the public JSON header. Do not decrypt the behavior body.
51     header_size = int.from_bytes(raw[10:14], "big")
52     header = json.loads(raw[14:14 + header_size])
53     assert header["package_type"] == "formal_behavior_log"
54     assert (header["problem_no"], header["formal_index"]) == (problem, formal_index)
55     case_code = header["case_code"]
56     series = f"Q{problem}-正式 {formal_index}"
57     common = {"problem": f"Q{problem}", "formal_index": formal_index,
58              "case_code": case_code, "series_id": series}
59     actions = read_jsonl(run_dir / "actions.jsonl")
60     strategy = read_jsonl(run_dir / "strategy.jsonl")
61     result = json.loads((run_dir / "result.json").read_text())
62     requests, accepted_ids = {}, set()
63     cleared, discovered = set(), set()
64     counts = Counter()
65     previous_time = 0.0
66     enter_time = enter_real = exit_real = None
67     fallback_channel = None
68     local_points = []
69     fallback_episodes = 0
70     no_signal_known = 0
71     for line_number, record in enumerate(actions, 1):
72         if record["type"] == "request":
73             requests[record["payload"]["request_id"]] = record
74             continue
75         assert record["type"] == "response", (run_dir, line_number, record["type"])
76         response = record["response"]
77         assert record["http_status"] == 200 and response["accepted"] is True
78         request_id = record["request_id"]
79         # Count each accepted request only once, including if a log is retried.
80         if request_id in accepted_ids:
81             continue
82         accepted_ids.add(request_id)
83         request = requests[request_id]
84         path, phase = record["path"], request["phase"]
85         assert request["path"] == path
86         payload = request["payload"]
87         channel = payload.get("channel", "")
88         virtual_time = float(response["virtual_time_s"])
89         assert virtual_time >= previous_time
90         before = len(cleared)
91         event_common = {**common, "k_cleared_before": before, "channel": channel,
92                          "phase": phase, "channel_previously_discovered": int(channel in discovered),
93                          "response_line": line_number}
94
95     def event(kind, when, timing_basis, after=None):
96         events.append({**event_common, "event": kind, "virtual_time_s": seconds(when),
97                       "k_cleared_after": len(cleared) if after is None else after,
98                       "timing_basis": timing_basis})
99
100    if path == "/enter":
101        enter_time = virtual_time
102        enter_real = response["real_timestamp_ms"]
103        assert enter_time == 0.0
104        local_points.append({**common, "k_cleared": 0, "virtual_time_s": seconds(0),
105                          "event": "enter", "channel": "", "phase": "control",
106                          "response_line": line_number})
107    elif path == "/measure":
108        counts["measure"] += 1
109        kind = response["measure_result"]
110        counts[kind] += 1
111        if kind in ("direction", "near"):
112            discovered.add(channel)
113        else:
114            assert kind == "no_signal"
115            if problem == 4:
116                event("no_signal", virtual_time, "response_end")
117                no_signal_known += int(channel in discovered)
118            fallback_channel = None
119    elif path == "/clear":
120        counts["clear"] += 1

```



```

121         if problem == 4 and phase == "optical_fallback" and fallback_channel != channel:
122             fallback_episodes += 1
123             # Beginning of the first fallback action, before its movement/optical costs.
124             event("optical_fallback_start", previous_time, "preceding_response_end")
125             fallback_channel = channel if phase == "optical_fallback" else None
126             kind = response["clear_result"]
127             if kind == "success":
128                 assert channel not in cleared
129                 assert channel in discovered
130                 cleared.add(channel)
131                 counts["clear_success"] += 1
132                 local_points.append({"common": "k_cleared": len(cleared),
133                                     "virtual_time_s": seconds(virtual_time), "event": "clear_success",
134                                     "channel": channel, "phase": phase, "response_line": line_number})
135             if problem == 4 and phase == "optical_fallback":
136                 event("optical_fallback_success", virtual_time, "response_end")
137                 fallback_channel = None
138             else:
139                 assert kind == "no_target_in_range"
140                 counts["clear_failure"] += 1
141                 if problem == 4:
142                     event("clear_no_target_in_range", virtual_time, "response_end")
143             elif path == "/exit":
144                 exit_real = response["real_timestamp_ms"]
145                 assert response["exit_reason"] == "user_exit"
146             else:
147                 raise AssertionError(path)
148             previous_time = virtual_time
149             assert enter_time is not None and exit_real is not None
150             assert result["status"] == "complete"
151             assert set(result["cleared_channels"] if problem == 3 else result["cleared"]) == cleared
152             metrics = result["metrics"]
153             assert abs(metrics["virtual_time_s"] - previous_time) < 1e-6
154             for key in ("measure", "clear", "clear_success", "clear_failure"):
155                 assert metrics["counts"].get(key, 0) == counts[key]
156             if problem == 4:
157                 assert sum(r["event"] == "optical_cover" for r in strategy) == fallback_episodes
158                 created_ms = datetime.fromisoformat(header["created_at_utc"].replace("Z", "+00:00")).timestamp() * 1000
159                 header_gap = round(created_ms - exit_real)
160                 assert 0 <= header_gap < 100
161                 last_clear = float(local_points[-1]["virtual_time_s"])
162                 summaries.append({"common": "run_directory": str(run_dir.relative_to(ROOT)),
163                                   "n_cleared": len(cleared), "last_clear_virtual_time_s": seconds(last_clear),
164                                   "exit_virtual_time_s": seconds(previous_time),
165                                   "post_last_clear_virtual_time_s": seconds(previous_time - last_clear),
166                                   "measure_count": counts["measure"], "no_signal_count": counts["no_signal"],
167                                   "known_channel_no_signal_count": no_signal_known if problem == 4 else "",
168                                   "clear_failure_count": counts["clear_failure"],
169                                   "optical_fallback_episodes": fallback_episodes if problem == 4 else "",
170                                   "formal_header_after_exit_ms": header_gap})
171             points.extend(local_points)
172             lookup = {p["k_cleared"]: p["virtual_time_s"] for p in local_points}
173             for row in wide:
174                 row[f"Q{problem}_formal_{formal_index}_T_s"] = lookup.get(row["k_cleared"], "")
175             sources.append({"common": "inputs": [provenance(run_dir / name) for name in
176                                                  ("actions.jsonl", "strategy.jsonl", "result.json", "config.json")] + [provenance(jlog)],
177                             "validation": "accepted request/response pairing; unique clear channels; final result
178                                     ↳ metrics; public formal header and exit timing"})
179             assert len(points) == 80 # 74 successful clears and six entry points.
180             write_csv("formal_clear_trajectories.csv", points)
181             write_csv("formal_clear_times_wide.csv", wide)
182             write_csv("formal_run_summary.csv", summaries)
183             write_csv("formal_q4_events.csv", events)
184             manifest = {"time_definition": "Simulator response.virtual_time_s, measured from successful /enter; never wall
185                               ↳ time.",
186                         "curve_definition": "Entry (0,0) and cumulative virtual time at each unique successful /clear
187                               ↳ response.",
188                         "exit_definition": "The exit time can exceed the last clear time; it is not another successful
189                               ↳ clear.",
190                         "event_definition": "No-signal and clear events use response end time. Fallback starts use the
191                               ↳ preceding response time, before the first fallback action.",
192                         "formal_directory_correction": "Q3 formal test 3 is windows_q3_final_004. Directory 003 failed before
193                               ↳ entry."

```

```

188         "sources": sources, "outputs": {"clear_rows_including_origins": len(points), "q4_event_rows":
        ↪ len(events)}}
189     (OUT / "formal_data_provenance.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2) + "\n")
190     print(json.dumps({"runs": len(summaries), "curve_rows": len(points), "q4_events": len(events),
191         "last_clear_to_exit_s": {r["series_id"]: r["post_last_clear_virtual_time_s"] for r in
        ↪ summaries}}, ensure_ascii=False))
192
193
194 if __name__ == "__main__":
195     main()

```

程序 10

对应文件: manuscript/story_draft/analysis/export_q3_convergence.py

```

1  """Export Q3 plotting data; read existing logs only, never run an arena session.
2
3  Run from the repository root:
4      python manuscript/story_draft/analysis/export_q3_convergence.py
5  Use --geometry-only when the local-only official logs are unavailable.
6  """
7  from __future__ import annotations
8
9  import argparse
10 import csv
11 import hashlib
12 import json
13 import math
14 import sys
15 from collections import Counter, defaultdict
16 from pathlib import Path
17
18 import numpy as np
19
20 ROOT = Path(__file__).resolve().parents[3]
21 sys.path.insert(0, str(ROOT))
22 from question3.innovation.geometry import SLACK, bbox, clip, initial, wedge
23
24 RUNS = (
25     ("windows_q3_final_001", "5ER6-BK3Y-KHDQ-MUQT", 1),
26     ("windows_q3_final_002", "VUK9-G6FH-ACSC-KFYG", 2),
27     ("windows_q3_final_004", "3BEW-YK5X-AE2M-SJND", 3),
28 )
29 OUT = ROOT / "manuscript/story_draft/plot_data/20260913_process"
30
31
32 def sha(path):
33     return hashlib.sha256(path.read_bytes()).hexdigest()
34
35
36 def write_csv(path, rows):
37     with path.open("w", encoding="utf-8-sig", newline="") as stream:
38         writer = csv.DictWriter(stream, fieldnames=list(rows[0]), lineterminator="\n")
39         writer.writeheader()
40         writer.writerows(rows)
41
42
43 def write_json(path, data):
44     path.write_text(json.dumps(data, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
45
46
47 def diameter(poly):
48     return float(np.linalg.norm(poly[:, None, :] - poly[None, :, :], axis=2).max())
49
50
51 def geometric_example(out):
52     # The manuscript's teaching example has fixed Cartesian bbox axes, +/-1 deg,
53     # and a square prior. It deliberately differs from the official algorithm.
54     source = np.array([600., 100.])
55     poly = np.array([[-1500., -1500.], [1500., -1500.],
56         [1500., 1500.], [-1500., 1500.]])

```

```

57 rows, vertices = [], []
58 for count, point in enumerate(((0., 0.), (350., -250.), (500., 100.)), 1):
59     point = np.asarray(point)
60     bearing = math.degrees(math.atan2(*(source - point)[:1])) % 360
61     # Exact geometric boundary, no numerical expansion of the wedge.
62     for sign in (-1, 1):
63         angle = math.radians(bearing + sign)
64         normal = np.array([-sign * math.sin(angle), sign * math.cos(angle)])
65         poly = clip(poly, normal, float(normal @ point))
66     centre = (poly.min(axis=0) + poly.max(axis=0)) / 2
67     radius = float(np.linalg.norm(poly - centre, axis=1).max())
68     rows.append(dict(example_id="fig17_geometric_recalculation", bearing_count=count,
69                     covering_radius_m=radius, region_diameter_m=diameter(poly),
70                     clear_threshold_m=20., centre_x_m=float(centre[0]),
71                     centre_y_m=float(centre[1]), measure_x_m=float(point[0]),
72                     measure_y_m=float(point[1]), bearing_deg=bearing,
73                     bearing_error_halfwidth_deg=1.,
74                     source_x_m=float(source[0]), source_y_m=float(source[1]),
75                     vertex_count=len(poly), direct_clear_geometrically_certified=radius < 20))
76     vertices.append(dict(bearing_count=count, vertices_m=poly.tolist()))
77 expected = [(762.68, 1525.36), (23.78, 47.57), (10.66, 21.26)]
78 for row, (radius, diam) in zip(rows, expected):
79     assert round(row["covering_radius_m"], 2) == radius
80     assert round(row["region_diameter_m"], 2) == diam
81 write_csv(out / "q3_convergence_geometric_example.csv", rows)
82 write_json(out / "q3_convergence_geometric_vertices.json", vertices)
83 return dict(
84     data_kind=" 按论文明确参数复算的几何算例；不是正式测试或平均结果",
85     source="manuscript/story_draft/中文论文初稿.md, 5.3.2, fig17_shrink",
86     original_figure_generator_found=False,
87     precision="float64 复算结果按 Python 最短往返表示保存，不补造原绘图程序的末位数值",
88     match=" 半径、直径保留两位小数与现论文完全一致",
89     initial_region="[-1500,1500]^2", centre_definition=" 固定全局坐标轴包围盒中心",
90     error_halfwidth_deg=1., centreline_error_deg=0., numeric_wedge_expansion_m=0.,
91     independent_layouts=1, formal_test=False,
92 )
93
94
95 def accepted_actions(path):
96     requests, seen, accepted = {}, set(), []
97     for line, raw in enumerate(path.read_text(encoding="utf-8").splitlines(), 1):
98         record = json.loads(raw)
99         if record["type"] == "request":
100             requests[record["payload"]["request_id"]] = record
101         elif record["type"] == "response" and record.get("response", {}).get("accepted"):
102             ident = record["request_id"]
103             if ident in seen:
104                 continue # A retried idempotent action is one observation.
105             seen.add(ident)
106             request = requests[ident]
107             assert request["path"] == record["path"]
108             accepted.append(dict(request=request, response=record["response"],
109                                path=record["path"], line=line))
110     return accepted
111
112
113 def official_run(run_id, case_code, formal_index):
114     folder = R00T / "question3/results" / run_id
115     lock = json.loads((folder / "code_sha256.json").read_text())
116     config = json.loads((folder / "config.json").read_text())
117     hashes = {}
118     for name in ("innovation/geometry.py", "innovation/strategy.py"):
119         actual = sha(R00T / "question3" / name)
120         expected = lock[name.replace("/", "\\")]
121         if actual != expected:
122             raise RuntimeError(f"{run_id}: source hash mismatch: {name}")
123         hashes[name] = actual
124     if config["mechanism"] != "gated_tour_multi_parallax":
125         raise RuntimeError(f"{run_id}: this exporter is scoped to the recorded recommended policy")
126     events = [(line, json.loads(raw)) for line, raw in enumerate(
127         (folder / "strategy.jsonl").read_text(encoding="utf-8").splitlines(), 1)]
128     actions = accepted_actions(folder / "actions.jsonl")
129     directions = [a for a in actions if a["response"].get("measure_result") == "direction"]

```

```

130 updates = [(line, e) for line, e in events if e["type"] == "feasible_update"]
131 assert len(directions) == len(updates)
132 error = config["baseline_config"]["error_deg"]
133 polygons, counts, bearings, rows = {}, Counter(), {}, []
134 max_vertex_difference = 0.
135 for action, (line, event) in zip(directions, updates):
136     payload, response = action["request"]["payload"], action["response"]
137     channel, point = payload["channel"], [payload["position"][axis] for axis in ("x", "y")]
138     angle = response["svd_deg"]
139     assert channel == event["channel"] and angle == event["bearing"]
140     np.testing.assert_array_equal(point, event["point"])
141     rebuilt = (initial(point, angle, error) if channel not in polygons
142               else wedge(polygons[channel], point, angle, error))
143     recorded = np.asarray(event["vertices"], dtype=float)
144     np.testing.assert_allclose(rebuilt, recorded, atol=1e-8, rtol=0)
145     max_vertex_difference = max(max_vertex_difference, float(np.abs(rebuilt - recorded).max()))
146     # Export metrics of the ORIGINAL logged vertices, not the rebuilt array.
147     centre, certified_radius, *_ = bbox(recorded, angle)
148     radius = float(np.linalg.norm(recorded - centre, axis=1).max())
149     counts[channel] += 1
150     polygons[channel], bearings[channel] = recorded, angle
151     rows.append(dict(
152         run_id=run_id, formal_index=formal_index, case_code=case_code,
153         source_curve_id=f"Q3-{formal_index}-ch{channel:02d}", channel=channel,
154         bearing_count=counts[channel], covering_radius_m=radius,
155         algorithm_radius_with_slack_m=certified_radius,
156         region_diameter_m=diameter(recorded), clear_threshold_m=20.,
157         centre_x_m=float(centre[0]), centre_y_m=float(centre[1]),
158         measure_x_m=point[0], measure_y_m=point[1], bearing_deg=angle,
159         virtual_time_s=response["virtual_time_s"], phase=action["request"]["phase"],
160         vertex_count=len(recorded), action_response_line=action["line"],
161         strategy_update_line=line,
162     ))
163 # Cross-check all certificates against the same latest recorded region.
164 current, last_angle, clear_certs, checked = {}, {}, {}, Counter()
165 max_certificate_difference = 0.
166 for _, event in events:
167     if event["type"] == "feasible_update":
168         current[event["channel"]] = np.asarray(event["vertices"])
169         last_angle[event["channel"]] = event["bearing"]
170     elif event["type"] in ("certified_measure", "certified_clear"):
171         channel = event["channel"]
172         np.testing.assert_array_equal(event["vertices"], current[channel])
173         expected = float(np.linalg.norm(current[channel] - event["point"], axis=1).max() + SLACK)
174         difference = abs(expected - event["radius"])
175         max_certificate_difference = max(max_certificate_difference, difference)
176         assert difference <= 1e-8
177         if event["type"] == "certified_clear":
178             centre, radius, *_ = bbox(current[channel], last_angle[channel])
179             np.testing.assert_allclose(event["point"], centre, atol=1e-8, rtol=0)
180             assert radius <= 20 - 1e-5
181             clear_certs[channel] = event
182             checked[event["type"]] += 1
183 by_channel = defaultdict(list)
184 for row in rows:
185     by_channel[row["channel"]].append(row)
186 clears = [a for a in actions if a["response"].get("clear_result") == "success"]
187 summary = []
188 assert len({a["request"]["payload"]["channel"] for a in clears}) == len(clears)
189 for action in clears:
190     payload = action["request"]["payload"]
191     channel = payload["channel"]
192     curve = by_channel[channel]
193     near = action["request"]["phase"] == "near"
194     if not near:
195         assert channel in clear_certs
196     summary.append(dict(
197         run_id=run_id, formal_index=formal_index, case_code=case_code,
198         source_curve_id=f"Q3-{formal_index}-ch{channel:02d}", channel=channel,
199         valid_direction_count=len(curve),
200         first_radius_m=curve[0]["covering_radius_m"] if curve else "",
201         last_radius_m=curve[-1]["covering_radius_m"] if curve else "",
202         clear_method="near_feedback" if near else "certified_region_circle",

```

```

203         successful_clear_virtual_time_s=action["response"]["virtual_time_s"],
204         clear_x_m=payload["position"]["x"], clear_y_m=payload["position"]["y"],
205         action_response_line=action["line"],
206     ))
207 result = json.loads((folder / "result.json").read_text())
208 assert sorted(r["channel"] for r in summary) == result["cleared_channels"]
209 summary.sort(key=lambda r: r["channel"])
210 return rows, summary, dict(
211     run_id=run_id, case_code=case_code, formal_index=formal_index,
212     normal_bearing_states=len(rows), source_curves=len(by_channel), cleared_sources=len(clears),
213     clear_methods=dict(Counter(r["clear_method"] for r in summary)),
214     source_code_sha256=hashes,
215     input_sha256={name: sha(folder / name) for name in (
216         "actions.jsonl", "strategy.jsonl", "config.json", "code_sha256.json", "result.json")},
217     error_halfwidth_deg=error, max_rebuilt_vertex_error_m=max_vertex_difference,
218     checked_certificates=dict(checked), max_certificate_radius_error_m=max_certificate_difference,
219 )
220
221
222 def main():
223     parser = argparse.ArgumentParser(description=__doc__)
224     parser.add_argument("--out-dir", type=Path, default=OUT)
225     parser.add_argument("--geometry-only", action="store_true")
226     args = parser.parse_args()
227     args.out_dir.mkdir(parents=True, exist_ok=True)
228     metadata = dict(
229         export_script="manuscript/story_draft/analysis/export_q3_convergence.py",
230         geometric_example=geometric_example(args.out_dir),
231         units=dict(distance="m", time="simulator virtual seconds", angle="degrees"),
232         csv_encoding="UTF-8 with BOM", floating_point_export="Python float repr, no decimal rounding",
233     )
234     if not args.geometry_only:
235         all_rows, all_summary, audits = [], [], []
236         for run in RUNS:
237             rows, summary, audit = official_run(*run)
238             all_rows.extend(rows)
239             all_summary.extend(summary)
240             audits.append(audit)
241         write_csv(args.out_dir / "q3_convergence_official.csv", all_rows)
242         write_csv(args.out_dir / "q3_convergence_official_source_summary.csv", all_summary)
243         metadata["official"] = dict(
244             provenance=" 从正式策略日志 feasible_update.vertices 计算指标; 另用动作响应和哈希一致源码独立重建核验",
245             independent_official_runs=3, new_runs_performed=0,
246             excluded_run="windows_q3_final_003: enter connection interruption, no accepted arena entry",
247             radius_definition="q 为最新示向方向旋转包围盒中心; covering_radius_m=max_v||v-q||",
248             algorithm_radius_definition="algorithm_radius_with_slack_m=covering_radius_m+1e-7",
249             count_definition=" 每个频道的 direction 正常测向次数; 不计 no_signal 或 near",
250             interpretation=" 每条线是一个源的真实运行轨迹; 三个正式测试共 38 源, 不是 38 独立布局",
251             terminal_note="near 后直接清除不产生新的多边形; 7 条此类轨迹不额外添加 R=5 或 R=0 点",
252             region_nesting_note=" 区域嵌套不自动保证旋转包围盒中心的覆盖半径严格单调, 不平滑或修正数据",
253             raw_logs_local_only=True, runs=audits,
254         )
255     write_json(args.out_dir / "q3_convergence_metadata.json", metadata)
256     print(json.dumps(metadata, ensure_ascii=False, indent=2))
257
258
259 if __name__ == "__main__":
260     main()

```

程序 11

对应文件: manuscript/story_draft/analysis/export_q4_discovery.py

```

1 #!/usr/bin/env python3
2 """Recover Q4 station-discovery curves with the frozen historical SELF code.
3
4 No HTTP transport, official testing, plotting, or new independent validation.
5 Raw action logs live only in worker memory; only derived plot data are exported.
6 """
7 from __future__ import annotations
8

```

```

9 import argparse
10 import collections
11 import csv
12 import hashlib
13 import json
14 import math
15 from pathlib import Path
16 import subprocess
17 import sys
18 import tempfile
19 import types
20
21 ROOT = Path(__file__).resolve().parents[3]
22 DEFAULT_OUTPUT = ROOT / "manuscript/story_draft/plot_data/20260913_process"
23 BATCHES = ("round9_main", "round9_rounding")
24 ARMS = {"BASE": "baseline31", "FINAL": "recommended25"}
25
26
27 def read_json(path):
28     return json.loads(Path(path).read_text(encoding="utf-8"))
29
30
31 def digest(path):
32     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
33
34
35 def write_csv(path, rows):
36     assert rows
37     with Path(path).open("w", encoding="utf-8-sig", newline="") as stream:
38         writer = csv.DictWriter(stream, fieldnames=list(rows[0]), lineterminator="\n")
39         writer.writeheader()
40         writer.writerows(rows)
41
42
43 def assert_metrics(actual, expected, context):
44     """Require integer microsecond clocks/counts exactly; tolerate float sums."""
45     assert actual["virtual_time_s"] == expected["virtual_time_s"], context
46     assert actual["counts"] == expected["counts"], context
47     for group in ("components", "phases"):
48         assert actual[group].keys() == expected[group].keys(), context
49         for key, value in actual[group].items():
50             assert math.isclose(value, expected[group][key], rel_tol=1e-12, abs_tol=1e-8), (
51                 context, group, key, value, expected[group][key]
52             )
53
54
55 def worker(batch, output):
56     folder = ROOT / "question4/results" / batch
57     lock = read_json(folder / "lock.json")
58     snapshot = folder / "code_snapshot"
59     for relative, expected in lock["code_sha256"].items():
60         assert digest(snapshot / relative) == expected, (batch, relative, "snapshot changed")
61
62     # A separate interpreter per batch prevents modules from another batch or
63     # the current repository from replacing a historically locked dependency.
64     for name in ("question3", "question4"):
65         package = types.ModuleType(name)
66         package.__path__ = [str(snapshot / name)]
67         sys.modules[name] = package
68     from question3.interface import Client
69     from question4.audit import audit
70     from question4.simulator import Simulator
71     from question4.strategy import Strategy
72
73     fixtures = read_json(folder / "fixtures.json")
74     historical = {
75         (r["case_id"], r["config"]): r for r in read_json(folder / "summary.json")["rows"]
76     }
77     runs = []
78     station_rows = []
79     for case in fixtures:
80         for config, arm in ARMS.items():
81             records = []

```

```

82     env = Simulator(case, rounding=lock["rounding"])
83     client = Client(env, "SELF", records.append)
84     strategy = Strategy(client, lock["configs"][config])
85     observations = []
86
87     class CompletedStations(set):
88         def add(self, station_id):
89             assert station_id not in self
90             super().add(station_id)
91             # Called by the original Strategy.run immediately after
92             # its complete channel loop, including near/clear actions.
93             discovered = set(strategy.known) | strategy.cleared
94             observations.append({
95                 "completed_stations": len(self),
96                 "station_id": station_id,
97                 "discovered_channels": len(discovered),
98                 "cleared_channels": len(strategy.cleared),
99                 "virtual_time_s": client.virtual,
100             })
101
102     strategy.done = CompletedStations()
103     result = strategy.run()
104     checked = audit(case, records, result)
105     truth = env.evaluation()
106     previous = historical[(case["id"], config)]
107     context = (batch, case["id"], config)
108     assert result["status"] == previous["status"] == "complete", context
109     assert checked["passed"] and previous["audit"]["passed"], context
110     assert truth["cleared"] == previous["cleared"] == previous["total"] == truth["total"], context
111     assert checked["accepted_actions"] == previous["audit"]["accepted_actions"], context
112     assert_metrics(client.metrics(), previous["metrics"], context)
113     assert observations[-1]["discovered_channels"] == truth["total"], context
114     assert all(a["discovered_channels"] <= b["discovered_channels"] for a, b in zip(observations,
115                                         ↪ observations[1:))), context
116     budget = len(strategy.stations)
117     assert budget == (31 if config == "BASE" else 25), context
118     common = dict(batch=batch, rounding=lock["rounding"], layout_id=case["id"],
119                 family=case["family"], error_mode=case["error_mode"],
120                 source_count=truth["total"], arm=arm, original_config=config,
121                 station_budget=budget, conditions_per_layout_per_arm=1)
122     completed = len(observations)
123     first_all = next(o["completed_stations"] for o in observations
124                     if o["discovered_channels"] == truth["total"])
125     runs.append(common | dict(actual_completed_stations=completed,
126                             first_all_discovered_station=first_all,
127                             first_16_discovered_station=first_all if truth["total"] == 16 else "",
128                             stopped_before_station_budget=int(completed < budget),
129                             stop_certificate=result["stop_certificate"]["kind"],
130                             final_cleared=truth["cleared"], final_virtual_time_s=client.virtual,
131                             historical_metrics_match=1, replay_physics_audit_passed=1))
132     initial = dict(completed_stations=0, station_id="", discovered_channels=0,
133                   cleared_channels=0, virtual_time_s=0.0)
134     for observation in [initial] + observations:
135         station_rows.append(common | observation | dict(
136             discovered_fraction=observation["discovered_channels"] / truth["total"]))
137     Path(output).write_text(json.dumps(dict(runs=runs, station_rows=station_rows), ensure_ascii=False),
138                             ↪ encoding="utf-8")
139     print(f"{batch}: {len(fixture)} paired layouts, {len(runs)} replayed runs match historical metrics", flush=True)
140
141 def export(output):
142     import numpy as np
143     output.mkdir(parents=True, exist_ok=True)
144     all_runs, observed = [], []
145     with tempfile.TemporaryDirectory(prefix="cumcm_q4_discovery_") as temp:
146         for batch in BATCHES:
147             temporary = Path(temp) / f"{batch}.json"
148             subprocess.run([sys.executable, str(Path(__file__).resolve()), "--worker", batch,
149                             ↪ "--worker-output", str(temporary)], check=True, cwd=R00T)
150             data = read_json(temporary)
151             all_runs.extend(data["runs"])
152             observed.extend(data["station_rows"])

```

```

153 # One physical fixture and one prespecified error condition per batch/arm.
154 # No condition averaging is needed; the run is the layout-level observation.
155 keyed = collections.defaultdict(list)
156 for row in observed:
157     keyed[(row["batch"], row["layout_id"], row["arm"])].append(row)
158 layout_curves = []
159 for run in all_runs:
160     curve = keyed[(run["batch"], run["layout_id"], run["arm"])]
161     for n in range(32):
162         endpoint = curve[min(n, len(curve) - 1)]
163         carried = n > run["actual_completed_stations"]
164         layout_curves.append({k: run[k] for k in (
165             "batch", "rounding", "layout_id", "family", "error_mode", "source_count", "arm",
166             "station_budget", "conditions_per_layout_per_arm", "actual_completed_stations")
167             | dict(station_index=n, discovered_channels=endpoint["discovered_channels"],
168                 discovered_fraction=endpoint["discovered_fraction"],
169                 value_kind="post_stop_carry_forward" if carried else "observed_scan_endpoint" if n else
170                 ↪ "initial",
171                 beyond_station_budget=int(n > run["station_budget"])))
172
173 grouped = collections.defaultdict(list)
174 for row in layout_curves:
175     for stratum in ("all_10_to_16", f"n_sources_{row['source_count']}"):
176         grouped[(row["batch"], row["arm"], stratum, row["station_index"])].append(row)
177 summaries = []
178 for (batch, arm, stratum, n), rows in sorted(grouped.items()):
179     count_q = np.quantile([r["discovered_channels"] for r in rows], [0.25, 0.5, 0.75], method="linear")
180     fraction_q = np.quantile([r["discovered_fraction"] for r in rows], [0.25, 0.5, 0.75], method="linear")
181     summaries.append(dict(batch=batch, arm=arm, source_stratum=stratum, station_index=n,
182         n_independent_layouts=len(rows), n_observed_scan_endpoints=sum(r["value_kind"] !=
183         ↪ "post_stop_carry_forward" for r in rows),
184         n_carried_forward=sum(r["value_kind"] == "post_stop_carry_forward" for r in rows),
185         count_q1=float(count_q[0]), count_median=float(count_q[1]), count_q3=float(count_q[2]),
186         fraction_q1=float(fraction_q[0]), fraction_median=float(fraction_q[1]), fraction_q3=float(fraction_q[2])))
187
188 run_lookup = {(r["batch"], r["layout_id"], r["arm"]): r for r in all_runs}
189 pairs = []
190 for run in all_runs:
191     if run["arm"] != "baseline31":
192         continue
193     matched = run_lookup[(run["batch"], run["layout_id"], "recommended25")]
194     pairs.append({k: run[k] for k in ("batch", "rounding", "layout_id", "family", "error_mode", "source_count")}
195         | dict(baseline31_first_all=run["first_all_discovered_station"],
196             recommended25_first_all=matched["first_all_discovered_station"],
197             paired_difference_recommended_minus_baseline=matched["first_all_discovered_station"]-run["first_all_discovered_station"],
198             ↪ "first_all_discovered_station",
199             baseline31_actual_completed=run["actual_completed_stations"],
200             recommended25_actual_completed=matched["actual_completed_stations"]]))
201
202 write_csv(output / "q4_discovery_runs.csv", all_runs)
203 write_csv(output / "q4_discovery_observed_stations.csv", observed)
204 write_csv(output / "q4_discovery_layout_curves.csv", layout_curves)
205 write_csv(output / "q4_discovery_summary.csv", summaries)
206 write_csv(output / "q4_discovery_paired_first_all.csv", pairs)
207 sources = {}
208 for batch in BATCHES:
209     folder = ROOT / "question4/results" / batch
210     fixture = read_json(folder / "fixtures.json")
211     sources[batch] = dict(
212         source_directory=str(folder.relative_to(ROOT)),
213         source_sha256={name: digest(folder / name) for name in ("fixtures.json", "summary.json", "lock.json")},
214         independent_layouts=len(fixture),
215         source_count_distribution=dict(sorted(collections.Counter(len(f["sources"]) for f in fixture).items())),
216         error_mode_distribution=dict(collections.Counter(f["error_mode"] for f in fixture)),
217         original_environment=read_json(folder / "summary.json").get("environment"),
218     )
219 manifest = dict(
220     provenance="offline reconstruction with individually SHA256-verified frozen historical source; not new
    ↪ validation or official testing",
221     original_action_logs_available=False, batches=sources,
222     original_arms=ARMS, independent_layouts_total=192, paired_runs_total=384,
223     matched_historical_runs=sum(r["historical_metrics_match"] for r in all_runs),

```



```

221     station_endpoint_definition="after original Strategy.run completes its station channel loop and calls
↳ done.add(i)",
222     discovered_definition="cardinality of known union cleared; includes earlier discoveries after clearing",
223     extension_definition="hold last discovery count after actual final scan through display index31; no
↳ unperformed measurements inferred",
224     quartile_method="numpy.quantile(method='linear'); distribution quartiles, not confidence intervals",
225     output_rows={"runs":len(all_runs), "observed_stations":len(observed), "layout_curves":len(layout_curves),
↳ "summary":len(summaries), "paired_first_all":len(pairs)},
226     output_sha256={p.name:digest(p) for p in sorted(output.glob("q4_discovery_*.csv"))},
227 )
228 (output / "q4_discovery_provenance.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2)+"\n",
↳ encoding="utf-8")
229 print(json.dumps(manifest["output_rows"], ensure_ascii=False), flush=True)
230
231
232 def main():
233     parser = argparse.ArgumentParser(description=__doc__)
234     parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
235     parser.add_argument("--worker", choices=BATCHES, help=argparse.SUPPRESS)
236     parser.add_argument("--worker-output", help=argparse.SUPPRESS)
237     args = parser.parse_args()
238     if args.worker:
239         worker(args.worker, args.worker_output)
240     else:
241         export(args.output)
242
243
244 if __name__ == "__main__":
245     main()

```

程序 12

对应文件: manuscript/story_draft/analysis/make_figures.py

```

1  """Draw the narrative manuscript figures from existing observations and analytic geometry.
2
3  Schematic coordinates illustrate logic; they are never presented as observed trajectories.
4  Statistical panels retain all paired cases in their stated batch. No simulations run here.
5  """
6  from pathlib import Path
7  import csv, json, math, hashlib, re, os
8  os.environ.setdefault('MPLCONFIGDIR', '/tmp/mcm-draft-mpl')
9  import numpy as np
10 import matplotlib as mpl
11 mpl.use('Agg')
12 import matplotlib.pyplot as plt
13 from matplotlib.patches import Circle, Polygon, Rectangle, Wedge, FancyBboxPatch
14 from matplotlib.colors import ListedColormap
15
16 ROOT = Path(__file__).resolve().parents[1]
17 DATA = Path(__file__).resolve().parent / 'source_data'
18 OUT = ROOT / 'figures'
19 QA = ROOT / 'qa'
20 TEAL, DARK, ORANGE, GRAY = '#227D8D', '#213B4A', '#B65F35', '#BECBD0'
21 mpl.rcParams.update({'font.family':'sans-serif', 'font.sans-serif':['Alibaba PuHuiTi', 'DejaVu Sans'],
22     'font.size':10, 'axes.labelsize':10, 'axes.titlesize':10, 'axes.spines.top':False,
23     'axes.spines.right':False, 'pdf.fonttype':42, 'svg.fonttype':'none',
24     'axes.unicode_minus':False, 'legend.frameon':False, 'axes.linewidth':.7})
25 manifest = []
26
27 def canvas(height=3.7, left=.14, bottom=.18):
28     fig, ax = plt.subplots(figsize=(6.8,height))
29     fig.subplots_adjust(left=left,right=.96,bottom=bottom,top=.93)
30     return fig,ax
31
32 def save(fig, name, claim, kind, source):
33     fig.canvas.draw()
34     (QA/(name+'.alignment.json')).write_text(json.dumps({'status':'NOT APPLICABLE', 'panels':1, 'reason':'One
↳ scientific drawing area.'}))
35     fig.savefig(OUT/(name+'.pdf'))
36     fig.savefig(OUT/(name+'.svg'))

```

```

37     fig.savefig(OUT/(name+'.png'),dpi=300)
38     manifest.append(dict(id=name,claim=claim,kind=kind,source=source))
39     plt.close(fig)
40
41 def box(ax,x,y,w,h,text,color=TEAL,fs=10):
42     ax.add_patch(FancyBboxPatch((x,y),w,h,boxstyle='round,pad=0.015,rounding_size=0.025',
43         facecolor='#F2F6F7',edgecolor=color,lw=1))
44     ax.text(x+w/2,y+h/2,text,ha='center',va='center',fontsize=fs,color=DARK)
45
46 def arrow(ax,p,q):
47     ax.annotate('',xy=q,xytext=p,arrowprops=dict(arrowstyle='->',color=DARK,lw=1.15))
48
49 # 1. The question chain and the shared observation/action loop.
50 fig,ax=canvas(3.2,.04,.05);ax.set(xlim=(0,10),ylim=(0,4));ax.axis('off')
51 for x,tst in [(0.15,'Q2: 两次测向\n约束接收与交会几何'),(3.5,'Q3: 整局时间\n联合观测、移动与清除'),(6.85,'Q4:
    ↳ 定向失联\n重建发现与清除保证')]:
52     box(ax,x,2.45,3,.95,tst)
53     arrow(ax,(3.2,2.92),(3.45,2.92));arrow(ax,(6.55,2.92),(6.8,2.92))
54 for x,tst in [(1.15,'公开反馈'),(2.7,'更新可行集合'),(5.25,'检查行动证书'),(7.8,'执行并重排')]:
55     box(ax,x,.6,2.05,.8,tst,GRAY)
56 for x in [2.25,4.8,7.35]:arrow(ax,(x,1),(x+.4,1))
57 ax.text(5.19,'共同底线: 不删去真实位置; 有依据才宣布任务完成',ha='center',color=DARK)
58 arrow(ax,(8.8,.55),(8.8,.2));ax.plot([1.15,8.8],[.2,.2],color=DARK,lw=1)
59 arrow(ax,(1.15,.2),(1.15,.55))
60 save(fig,'fig01_overview','三个问题逐步改变优化对象和信息条件','整体流程示意','model definitions')
61
62 # 2. An exact, measurement-compatible counterexample from the verified Q1 record.
63 fig,ax=canvas(3.4);t=np.array([[0,0],[20,0],[10,10*np.sqrt(3)]])
64 ax.add_patch(Polygon(t,facecolor='#DFEEF0',edgecolor=TEAL,lw=1.5))
65 ax.add_patch(Circle((10,0),10,fill=False,color=ORANGE,ls='--',lw=1.5))
66 ax.add_patch(Circle((10,10/np.sqrt(3)),20/np.sqrt(3),fill=False,color=DARK,lw=1))
67 ax.scatter(t[:,0],t[:,1],color=TEAL,s=28,zorder=5)
68 ax.text(23,15,'区域直径 D = 20 m',fontsize=10)
69 ax.text(23,10,'半径 D/2 的圆漏掉顶点',color=ORANGE)
70 ax.text(23,5,'最小覆盖半径 = 11.55 m',color=DARK)
71 ax.set(xlim=(-4,52),ylim=(-12,21),xlabel='x (m)',ylabel='y (m)');ax.set_aspect('equal')
72 save(fig,'fig02_diameter','区域直径的一半不是一般覆盖半径','严格几何反例','q12_verification.json: q1_triangle')
73
74 # 3. Concrete Q2 failures, in the same local coordinate system.
75 fig,ax=canvas(3.1);S=np.array([0,0]);G=np.array([1500,0]);side=np.array([0,500]);front=np.array([500,0])
76 ax.plot([0,1500],[0,0],color=TEAL,lw=2)
77 ax.plot([0,1500],[500,0],color=ORANGE,lw=1.5,ls='--')
78 ax.scatter([0,0,500,1500],[0,500,0,0],s=[30,35,35,45],c=[DARK,ORANGE,TEAL,DARK],zorder=4)
79 ax.text(60,525,'横移点 (0,500): 距离源 1581.14 m, 失联',color=ORANGE)
80 ax.text(500,-170,'前移点 (500,0): 中心线重合, 交会退化',ha='center',color=TEAL)
81 ax.text(1470,100,'源 (1500,0)',ha='right');ax.text(20,100,'首测点 S')
82 ax.text(900,410,'此例接收半径为 1500 m',ha='center',fontsize=9)
83 ax.set(xlim=(-80,1650),ylim=(-250,720),xlabel='沿首次示向方向 (m)',ylabel='横向位置 (m)')
84 save(fig,'fig03_q2_failures','横移可能失联, 沿示向前进可能退化','构造反例','Q2 documented counterexamples')
85
86 # 4. Exact inequalities visualized on a display grid, not a grid proof.
87 a,b=np.meshgrid(np.linspace(-40,1100,571),np.linspace(-1050,1050,701))
88 d=np.deg2rad(1.005);h=a*np.cos(d)-abs(b)*np.sin(d);q2=a*a+b*b
89 rec=(a>=0)&(q2-10*h+25<=1e6)&(q2<=2000*h)
90 c=abs(b)*np.cos(d)-a*np.sin(d)
91 M=np.sqrt(np.maximum(q2+25-10*h,q2+1500**2-3000*h))
92 safe=rec&(c>5)&(c>=M*np.sin(np.deg2rad(20)))
93 fig,ax=canvas(4.1)
94 ax.pcolormesh(a,b,np.where(safe,2,np.where(rec,1,0)),cmap=ListedColormap(['white','#DFE5E8',TEAL]),shading='auto',rasterized=True)
95 ax.axhline(0,color=GRAY,lw=.8)
96 ax.scatter([800,800],[605,-605],c=DARK,s=28,zorder=5)
97 ax.text(825,900,'(800,605)',fontsize=9);ax.text(825,-980,'(800,-605)',fontsize=9)
98 ax.text(480,120,'保证接收域',fontsize=9,ha='center');ax.text(530,500,'安全候选域',color='white',fontsize=9,ha='center')
99 ax.set(xlim=(-40,1100),ylim=(-1050,1050),xlabel='前向位移 a (m)',ylabel='横向位移 b (m)');ax.set_aspect('equal')
100 save(fig,'fig04_q2_region','两侧候选兼顾接收与非退化交会','解析条件显示','Q2 reception and safe-region inequalities')
101
102 # 5. Keep all eight archived candidates; mirror points have identical metrics.
103 q12=json.loads((DATA/'q12_verification.json').read_text())
104 rows=q12['q2_fixed_library_1_005deg']['rows']
105 fig,ax=canvas(3.5)
106 nd=sorted([r for r in rows if r['pareto_within_fixed_library']],key=lambda r:r['move_and_measure_s'])
107 ax.plot([r['move_and_measure_s'] for r in nd],[r['dense_sample_worst_D'] for r in nd],color=TEAL,lw=1)

```

```

108 offsets={(400,500):(-8,-23),(500,400):(8,4),(500,500):(-54,-16),(600,400):(8,5),(700,450):(-60,-18),(750,400):(7,5),(
    ↳ 800,605):(-73,-15),(850,450):(-76,6)}
109 for r in rows:
110     xy=(r['move_and_measure_s'],r['dense_sample_worst_D']);key=tuple(r['offset'])
111     ax.scatter(*xy,color=TEAL if r['pareto_within_fixed_library'] else GRAY,s=32,zorder=3)
112     ax.annotate(str(key),xy,xytext=offsets[key],textcoords='offset points',fontsize=8)
113 ax.set(xlim=(126,214),ylim=(112,319),xlabel='移动与第二次检测时间 (s)',ylabel='采样最大定位直径 (m)')
114 ax.set_axisbelow(True)
115 save(fig,'fig05_q2_tradeoff','更小的定位直径需要支付移动代价','有限场景数值结果','q12_verification.json: all eight
    ↳ fixed-library offsets')
116
117 # 6. Algorithmic decision schematic, no claimed sample trajectory.
118 fig,ax=canvas(4.1,.03,.04);ax.set(xlim=(0,10),ylim=(0,6));ax.axis('off')
119 box(ax,.2,4.8,2.8,.8,'保留各频道可行集合')
120 box(ax,3.6,4.8,2.8,.8,'搜索站 + 已知源任务')
121 box(ax,7.4,4.8,2.8,.8,'滚动规划,只执行首项')
122 arrow(ax,(3.05,5.2),(3.55,5.2));arrow(ax,(6.45,5.2),(6.95,5.2))
123 box(ax,.2,2.9,2.8,.85,'整个集合可被 20 m 圆覆盖?')
124 box(ax,3.6,2.9,2.8,.85,'首次横向视差\n 后续中心测量',fs=9)
125 box(ax,7.2,2.9,2.8,.85,'筛选其他频道\n 仅有足够价值时观测',fs=9)
126 arrow(ax,(8.4,4.75),(8.4,4.1));ax.plot([1.6,8.4],[4.1,4.1],color=DARK,lw=1)
127 arrow(ax,(1.6,4.1),(1.6,3.8));arrow(ax,(3.05,3.33),(3.55,3.33))
128 ax.text(3.3,3.58,'否',ha='center',fontsize=9)
129 arrow(ax,(6.45,3.33),(6.95,3.33))
130 box(ax,.2,.8,2.8,.9,'清除;记录成功频道')
131 box(ax,3.6,.8,2.8,.9,'依据真实反馈更新\n 完成任务后重排',fs=9)
132 box(ax,7.2,2.8,.9,'仅满足覆盖或计数证书\n 且无待清源时退出',fs=9)
133 arrow(ax,(1.6,2.85),(1.6,1.75));ax.text(1.85,2.25,'是',fontsize=9)
134 arrow(ax,(8.4,2.85),(8.4,2.25));arrow(ax,(5,2.25),(5,1.75))
135 arrow(ax,(3.05,1.25),(3.55,1.25));arrow(ax,(6.45,1.25),(6.95,1.25))
136 save(fig,'fig06_q3_algorithm','定位动作与任务调度共享同一个状态','算法示意图','Q3 implementation logic; finite
    ↳ fallback described in text')
137
138 # 7. Omnidirectional seven-station discovery coverage.
139 ang=np.arange(6)*np.pi/3
140 seven=np.vstack([[0,0],[1125*np.cos(ang),1125*np.sin(ang)]])
141 fig,ax=canvas(4)
142 for s in seven:ax.add_patch(Circle(s,1000,fill=False,color=GRAY,lw=.8))
143 ax.add_patch(Circle((0,0),1800,fill=False,color=ORANGE,lw=1.5))
144 ax.scatter(seven[:,0],seven[:,1],color=TEAL,s=26,zorder=4)
145 g=1800*np.array([np.cos(np.pi/6),np.sin(np.pi/6)])
146 ax.plot([seven[1,0],g[0]],[seven[1,1],g[1]],color=DARK,lw=1.3)
147 ax.scatter(*g,color=ORANGE,s=30,zorder=5)
148 ax.text(-2050,2250,'全域最近站距离约 999.11 m < 1000 m',fontsize=9)
149 ax.set(xlim=(-2250,2250),ylim=(-2250,2480),xlabel='x (m)',ylabel='y (m)');ax.set_aspect('equal')
150 save(fig,'fig07_q3_cover','七站圆盘覆盖支撑全向源的未知频道停止判据','解析构造','Q3 seven-station geometry')
151
152 # 8. Directional no-signal failure: source at origin, outward half-plane to the right.
153 fig,ax=canvas(3.5)
154 ax.add_patch(Wedge((0,0),1000,-90,90,facecolor='#DDEEFF',edgecolor=TEAL,lw=1.1))
155 ax.add_patch(Circle((0,0),1000,fill=False,color=GRAY,ls='--'))
156 ax.axvline(0,color=TEAL,lw=.8)
157 ax.scatter([0,-350,650],[0,0,250],color=[DARK,ORANGE,TEAL],s=35,zorder=4)
158 ax.text(-750,170,'背面站',color=ORANGE)
159 ax.text(.5,1.035,'背面站距离 350 m 仍无信号;正面站可接收',transform=ax.transAxes,ha='center',fontsize=9)
160 ax.text(100,-550,'发射半平面',color=TEAL,fontsize=9)
161 ax.text(220,400,'正面站',color=TEAL)
162 ax.text(40,-110,'源');ax.set(xlim=(-1150,1150),ylim=(-1080,1100),xlabel='相对源的 x (m)',ylabel='相对源的 y
    ↳ (m)');ax.set_aspect('equal')
163 save(fig,'fig08_no_signal','定向无信号不能排除近处仍有源','构造反例示意','directional reception model, R=1000 m')
164
165 # 9. Exact 25-station construction, without a dependency on the optimizer.
166 a0=np.arange(12)*np.pi/6
167 pts=np.vstack([[0,0],[950*np.cos(a0),950*np.sin(a0)],[1870*np.cos(a0+np.pi/12),1870*np.sin(a0+np.pi/12)]])
168 tri=[]
169 for k in range(12):
170     tri.extend([[0,1+k,1+(k+1)%12],[1+k,1+(k+1)%12,13+k],[1+k,13+(k-1)%12,13+k]])
171 tri=np.array(tri);maxedge=max(np.linalg.norm(pts[t][:,None]-pts[t][None,:],axis=2).max() for t in tri)
172 assert maxedge<1000 and 1870*np.cos(np.pi/12)>1800
173 fig,ax=canvas(4)
174 ax.triplot(pts[:,0],pts[:,1],tri,color=GRAY,lw=.8)
175 ax.add_patch(Circle((0,0),1800,fill=False,color=ORANGE,ls='--',lw=1.25))
176 ax.scatter(pts[:,0],pts[:,1],color=TEAL,s=23,zorder=4)

```

```

177 ax.add_patch(Polygon(pts[tri[4]],facecolor='#DFFEF0',edgecolor=TEAL,lw=1.6))
178 ax.text(-2050,2110,'36 个三角形; 最长边 983.60 m; 外包内切半径 1806.28 m',fontsize=8)
179 ax.set(xlim=(-2150,2150),ylim=(-2050,2350),xlabel='x (m)',ylabel='y (m)');ax.set_aspect('equal')
180 save(fig,'fig09_q4_cover','近距三角形顶点覆盖任意发射半平面','连续覆盖解析构造','25-station analytic coordinates and
    ↳ triangles')
181
182 # 10. Local portion of finite optical covering; units and scale are exact.
183 fig,ax=canvas(3.2)
184 for x in [0,28,56,84]:ax.plot([x,x],[0,56],color=GRAY,lw=.8)
185 for y in [0,28,56]:ax.plot([0,84],[y,y],color=GRAY,lw=.8)
186 centers=np.array([[14,14],[42,14],[70,14],[70,42],[42,42],[14,42]])
187 ax.plot(centers[:,0],centers[:,1],color=TEAL,lw=1.1)
188 ax.scatter(centers[:,0],centers[:,1],color=TEAL,s=20)
189 ax.add_patch(Circle((42,14),20,fill=False,color=ORANGE,lw=1.3))
190 ax.plot([42,56],[14,28],color=ORANGE,ls='--')
191 ax.text(94,40,'格边长  $\leq 28$  m')
192 ax.text(94,26,'半对角线  $\leq 19.80$  m')
193 ax.text(94,12,'清除半径: 20 m',color=ORANGE)
194 ax.set(xlim=(-5,155),ylim=(-9,65),xlabel='沿首次示向方向 (m)',ylabel='横向位置 (m)');ax.set_aspect('equal')
195 save(fig,'fig10_optical','光学有限覆盖不依赖是否还能接收无线信号','算法几何示意','28 m grid and 20 m clearance
    ↳ radius; local excerpt')
196
197 # 11. Layout-paired Q3 outcomes: every one of the 120 layouts is present.
198 pairs=list(csv.DictReader((DATA/'q3_final_pairs.csv').open()))
199 groups={}
200 for r in pairs:
201     if r['rounding']!='bounded':continue
202     m=re.fullmatch(r'final_(.)_(\d+)_(.)',r['case_id'])
203     groups.setdefault((m[1],m[2]),[]).append(r)
204 assert len(groups)==120 and all(len(v)==4 for v in groups.values())
205 fams=['uniform','outer','cluster','min_radius','radial','reception_edge']
206 labels=['均匀','外缘','聚集','最小半径','窄径向','接收边缘']
207 fig,ax=canvas(3.5)
208 layout_data=[]
209 for i,f in enumerate(fams):
210     vals=[]
211     for (fam,seed),rr in sorted(groups.items()):
212         if fam!=f:continue
213         base=np.mean([float(r['baseline_s']) for r in rr]);cand=np.mean([float(r['candidate_s']) for r in rr])
214         val=100*(cand/base-1);vals.append(val)
215         layout_data.append(dict(family=f,seed=seed,base_s=base,candidate_s=cand,change_pct=val))
216     ax.scatter(i+np.linspace(-.20,.20,len(vals)),vals,s=22,color=TEAL,alpha=.8)
217     ax.plot([i-.25,i+.25],[np.median(vals)*2,color=DARK,lw=1.6)
218 ax.axhline(0,color=ORANGE,ls='--',lw=1)
219 ax.set(xticks=range(6),xticklabels=labels,ylabel='布局平均总时间变化 (%)',ylim=(-30,68),xlim=(-.5,5.5))
220 ax.text(.01,1.025,'120 个布局; 每点合并 4 种误差; 短线为组内中位数',transform=ax.transAxes,fontsize=9)
221 save(fig,'fig11_q3_distribution','平均收益不意味着每个布局都变快','配对统计图','q3_final_pairs.csv: all 480 bounded
    ↳ runs, aggregated to 120 layouts')
222
223 # 12. Exact action-cost mean differences.
224 cost=list(csv.DictReader((DATA/'q3_cost_components.csv').open()))
225 comp=['movement_s','detection_s','switch_s','optical_s','laser_s'];cn=['移动','检测','切频','光学','成功清除附加']
226 dif=[]
227 for k in comp:
228     dct={r['variant']:float(r['mean_s']) for r in cost if r['rounding']=='bounded' and r['component']==k}
229     dif.append(dct['GATED_TOUR_MULTI_PARALLAX']-dct['BASE'])
230 fig,ax=canvas(3.1,.18)
231 y=np.arange(6);vals=dif+[sum(dif)]
232 ax.barh(y,vals,color=[TEAL if v<0 else ORANGE for v in vals],height=.55)
233 ax.axvline(0,color=DARK,lw=.8)
234 for yi,v in zip(y,vals):ax.text(v+(-7 if v<0 else 7),yi,f'{v:+.2f}',ha='right' if v<0 else
    ↳ 'left',va='center',fontsize=9)
235 ax.set(yticks=y,yticklabels=cn+['合计'],xlabel='联合方案 - 基线: 每局平均时间 (s)',xlim=(-485,120));ax.invert_yaxis()
236 save(fig,'fig12_q3_cost','更多检测换取更少移动, 净节省341.33秒','动作代价分解','q3_cost_components.csv: bounded, 480
    ↳ paired runs per arm')
237
238 # 13. Equal-layout final ablations share one parent; do not add their deltas.
239 stats=list(csv.DictReader((DATA/'q3_final_statistics.csv').open()))
240 lookup={r['variant']:float(r['mean']) for r in stats if r['rounding']=='bounded'}
241 parent='GATED_TOUR_SHARED_PARALLAX'
242 variants=['GATED_TOUR_MULTI_PARALLAX','GATED_NEAREST_SHARED_PARALLAX','GATED_TOUR_SHARED','TOUR_SHARED_PARALLAX']
243 names=['删除目标原地再观测','删除完整行程规划','删除首次视差','删除共享信息门控']
244 delta=[lookup[v]-lookup[parent] for v in variants]

```

```

245 fig,ax=canvas(3.0,.30)
246 ax.barh(range(4),delta,color=[TEAL if x<0 else ORANGE for x in delta],height=.5)
247 for i,v in enumerate(delta):ax.text(v+(-5 if v<0 else 5),i,f'{v:+.2f}',va='center',ha='right' if v<0 else
    ↳ 'left',fontSize=9)
248 ax.axvline(0,color=DARK,lw=.8)
249 ax.set(yticks=range(4),yticklabels=names,xlabel='相对共同父方案的平均时间变化 (s)',xlim=(-85,300));ax.invert_yaxis()
250 ax.text(.01,1.035,'共同父方案均值 3007.19 s; 同一 480 次配对运行',transform=ax.transAxes,fontSize=9)
251 save(fig,'fig13_q3_ablation','视差、
    ↳ 任务行程与观测筛选的贡献必须按共同父方案解释','配对消融均值图','q3_final_statistics.csv: bounded, four
    ↳ prespecified ablations')
252
253 # 14. Historical Q4 retained-scheme comparisons on both separate validation batches.
254 q4=list(csv.DictReader((DATA/'q4_source_data.csv').open()))
255 fig,ax=canvas(3.5,.24)
256 configs=['BASE','FINAL','COMPACT22','NO_CENTROID','NO_INFO']
257 names=['31 站初始基线','25 站推荐','改为 22 站','移除重心排序','移除信息删减']
258 historical=[]
259 for j,batch in enumerate(['round9_main','round9_rounding']):
260     values=[]
261     for cf in configs:
262         rr=[r for r in q4 if r['batch']==batch and r['config']==cf];assert len(rr)==96
263         v=float(np.mean([float(r['virtual_time_s']) for r in rr]));values.append(v)
264         historical.append(dict(batch=batch,config=cf,runs=96,mean_s=v))
265     ax.plot(values,np.arange(5)+(j-.5)*.18,marker='o' if j==0 else 's',lw=0,color=TEAL if j==0 else ORANGE,
266         label='读数包络: 96 布局' if j==0 else '舍入压力: 另 96 布局',markersize=5)
267 ax.set(yticks=range(5),yticklabels=names,xlabel='整局虚拟时间均值 (s)',xlim=(0,13500));ax.invert_yaxis()
268 ax.legend(loc='lower right',fontSize=8);ax.set_axisbelow(True)
269 save(fig,'fig14_q4_historical','25站布局与调度组合的收益,
    ↳ 不能仅归因于站数减少','两批自建验证均值图','q4_source_data.csv: all 960 historical rows; 96 layouts per batch per
    ↳ arm')
270
271 # 15. Latest candidate paired layout differences; full distribution plus primary CI.
272 latest=[r for r in q4 if r['batch']=='reset_final_validation']
273 cfigs=['HIST_ADAPT','COVER_EXCHANGE','COVER_ELASTIC','COVER_EXCHANGE_ELASTIC']
274 base_by={r['case_id']:float(r['virtual_time_s']) for r in latest if r['config']=='BASE25'}
275 assert len(base_by)==960
276 fig,ax=canvas(3.7,.14,.23)
277 latest_layout=[]
278 for i,cf in enumerate(cfigs):
279     gg={}
280     for r in latest:
281         if r['config']==cf:gg.setdefault(r['layout_id'],[]).append(r)
282     assert len(gg)==120 and all(len(v)==8 for v in gg.values())
283     vv=[]
284     for layout,rr in sorted(gg.items()):
285         b0=np.mean([base_by[r['case_id']] for r in rr]);v=np.mean([float(r['virtual_time_s']) for r in
            ↳ rr]);chg=100*(v/b0-1);vv.append(chg)
286         latest_layout.append(dict(config=cf,layout=layout,change_pct=chg))
287 ax.scatter(i+np.linspace(-.23,.23,len(vv)),vv,s=9,color=TEAL,alpha=.52)
288 overall=100*(np.mean([float(r['virtual_time_s']) for r in latest if
    ↳ r['config']==cf])/np.mean(list(base_by.values()))-1)
289 ax.scatter([i],[overall],marker='D',s=33,color=DARK,zorder=5)
290 ax.axhline(0,color=ORANGE,ls='--',lw=1)
291 ax.set(xticks=range(4),xticklabels=['历史自适应定位','仅动态替站','仅测站移动','替站+移动'],ylabel='布局平均总时间变
    ↳ 化
    ↳ (%)',xlim=(-.6,3.6))
292 ax.text(.01,1.035,'每组 120 布局; 小点为布局变化, 菱形为整批均值之比的变化',transform=ax.transAxes,fontSize=8)
293 ax.text(.5,-.26,'组合省时0.366%; 95%区间 [-0.183%, 0.921%] (省时为正)
    ↳ ',transform=ax.transAxes,ha='center',fontSize=9,color=DARK)
294 save(fig,'fig15_q4_latest','动态覆盖可行, 但未证明足以替换成熟方案的整局收益','配对统计与负结果图','q4_source_data.csv:
    ↳ all latest 4800 runs; primary CI from frozen statistics')
295
296 def writcsv(name,rr):
297     with (OUT/name).open('w') as f:
298         w=csv.DictWriter(f,fieldnames=list(rr[0]));w.writeheader();w.writerows(rr)
299 writcsv('q3_layout_source.csv',layout_data);writcsv('q4_historical_source.csv',historical);writcsv('q4_latest_layo
    ↳ ut_source.csv',latest_layout)
300 (OUT/'figure_manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2))
301 (OUT/'geometry_source.json').write_text(json.dumps({'stations':pts.tolist(),'triangles':tri.tolist(),'max_edge_m':max
    ↳ edge},indent=2))
302 inputs=['q12_verification.json','q3_final_pairs.csv','q3_cost_components.csv','q3_final_statistics.csv','q4_source_da
    ↳ ta.csv']

```

```

303 (QA/'source_hashes.json').write_text(json.dumps({s:hashlib.sha256((DATA/s).read_bytes()).hexdigest() for s in
    ↪ inputs},indent=2))
304 print('Generated',len(manifest),'figures; Q3 cost delta',sum(dif),'s; 25-station max edge',maxedge)

```

程序 13

对应文件: manuscript/story_draft/analysis/redesign_flow_figures.py

```

1 """Redraw manuscript Figures 1 and 8 as explanatory vector diagrams.
2
3 Contract: Figure 1 explains the Q1-Q4 dependency chain; Figure 8 explains
4 task-level receding-horizon scheduling and the two valid stopping conditions.
5 All small geometric drawings are conceptual, without coordinate or scale claims.
6 No experimental observations, strategy code, or quantitative results are changed.
7 Python/matplotlib produces both the authoritative figures and review comparisons.
8 """
9
10 from pathlib import Path
11 import argparse
12 import hashlib
13 import json
14 import os
15 import sys
16
17 os.environ.setdefault("MPLCONFIGDIR", "/tmp/mcm-flow-redesign-mpl")
18 import matplotlib as mpl
19 mpl.use("Agg")
20 import matplotlib.pyplot as plt
21 from matplotlib import font_manager
22 from matplotlib.patches import Circle, Polygon, Rectangle, FancyBboxPatch
23 import numpy as np
24
25 ROOT = Path(__file__).resolve().parents[1]
26 OUT = ROOT / "figures"
27 REVIEW = ROOT / "figure_review" / "batch01"
28 AUDIT = REVIEW / "audit"
29 SKILL_SCRIPTS = Path.home() / ".codex/skills/nature-figure/scripts"
30 sys.path.insert(0, str(SKILL_SCRIPTS))
31 from audit_panel_alignment import require_matplotlib_panel_alignment
32
33 NAVY = "#233D50"
34 TEAL = "#237D8C"
35 ORANGE = "#B96438"
36 PURPLE = "#8265A3"
37 MUTED = "#617580"
38 LINE = "#C7D4DA"
39 PALE = "#EFF5F7"
40 WARM = "#FBF2EB"
41 WHITE = "#FFFFFF"
42
43 # The installed PuHuiTi face is regular-only. Convert the needed Noto bold
44 # outlines to TrueType for reliable editable embedding through matplotlib and TeX.
45 # CFF outlines with pdf.fonttype=42 can extract as text yet disappear in the PDF.
46 from fontTools.ttLib import TTFont
47 from fontTools.fontBuilder import FontBuilder
48 from fontTools.pens.cu2quPen import Cu2QuPen
49 from fontTools.pens.ttGlyphPen import TTGlyphPen
50 from fontTools import subset
51 font_chars = Path(__file__).read_text() + " 改前改后图 0123456789"
52 font_key = hashlib.sha256(font_chars.encode()).hexdigest()[:12]
53 bold_path = Path(os.environ["MPLCONFIGDIR"]) / f"NotoSansCJKsc-Bold-{font_key}.ttf"
54 if not bold_path.exists():
55     bold_path.parent.mkdir(parents=True, exist_ok=True)
56     source_font = TTFont("/usr/share/fonts/opentype/noto/NotoSansCJK-Bold.ttc", fontNumber=2)
57     subsetter = subset.Subsetter()
58     subsetter.populate(text=font_chars)
59     subsetter.subset(source_font)
60     glyph_set = source_font.getGlyphSet()
61     glyph_order = source_font.getGlyphOrder()
62     glyphs = {}
63     for name in glyph_order:

```

```

64     pen = TTGlyphPen(None)
65     glyph_set[name].draw(Cu2QuPen(pen, max_err=1.0, reverse_direction=True))
66     glyphs[name] = pen.glyph()
67     builder = FontBuilder(source_font["head"].unitsPerEm, isTTF=True)
68     builder.setupGlyphOrder(glyph_order)
69     builder.setupCharacterMap(source_font.getBestCmap())
70     builder.setupGlyph(glyphs)
71     builder.setupHorizontalMetrics(source_font["hmtx"].metrics)
72     builder.setupHorizontalHeader(ascent=source_font["hea"].ascent,
73                                 descent=source_font["hea"].descent)
74     builder.setupNameTable({"familyName": "MCM Figure CJK", "styleName": "Bold",
75                             "uniqueFontIdentifier": "MCMFigureCJK-Bold",
76                             "fullName": "MCM Figure CJK Bold",
77                             "psName": "MCMFigureCJK-Bold"})
78     builder.setupOS2(usWeightClass=700, sTypoAscender=source_font["OS/2"].sTypoAscender,
79                     sTypoDescender=source_font["OS/2"].sTypoDescender,
80                     usWinAscent=source_font["OS/2"].usWinAscent,
81                     usWinDescent=source_font["OS/2"].usWinDescent)
82     builder.setupPost()
83     builder.setupMaxp()
84     builder.save(bold_path)
85
86     mpl.rcParams.update({
87         "font.family": "sans-serif",
88         "font.sans-serif": ["Alibaba PuHuiTi", "DejaVu Sans"],
89         "font.size": 9,
90         "text.color": NAVY,
91         "axes.spines.top": False,
92         "axes.spines.right": False,
93         "pdf.fonttype": 42,
94         "svg.fonttype": "none",
95         "axes.unicode_minus": False,
96         "savefig.facecolor": WHITE,
97     })
98
99
100 def text(ax, x, y, value, size=9, color=NAVY, weight="normal", ha="left"):
101     kwargs = {"fontproperties": font_manager.FontProperties(
102         fname=bold_path, family="Noto Sans CJK SC")} if weight == "bold" else {}
103     return ax.text(x, y, value, fontsize=size, color=color, weight=weight,
104                   ha=ha, va="center", linespacing=1.65, zorder=8, **kwargs)
105
106
107 def panel(ax, x, y, w, h, face=PALE, edge="none", radius=1.2):
108     patch = FancyBboxPatch((x, y), w, h,
109                            boxstyle=f"round,pad=0,rounding_size={radius}",
110                            facecolor=face, edgecolor=edge, linewidth=.8, zorder=0)
111     ax.add_patch(patch)
112     return patch
113
114
115 def arrow(ax, points, color=NAVY, width=1.15, dashed=False):
116     """Orthogonal paths keep the last segment's arrowhead away from labels."""
117     if len(points) > 2:
118         xy = np.asarray(points[:-1])
119         ax.plot(xy[:, 0], xy[:, 1], color=color, linewidth=width,
120               linestyle="--" if dashed else "-", zorder=3,
121               solid_joinstyle="round", solid_capstyle="round")
122     ax.annotate("", xy=points[-1], xytext=points[-2], zorder=3,
123               arrowprops=dict(arrowstyle="->", color=color, lw=width,
124                               mutation_scale=9, shrinkA=0, shrinkB=0,
125                               linestyle="--" if dashed else "-"))
126
127
128 def canvas(height):
129     fig = plt.figure(figsize=(7.086614173, height / 25.4))
130     ax = fig.add_axes([0, 0, 1, 1])
131     ax.set(xlim=(0, 180), ylim=(height, 0))
132     ax.axis("off")
133     return fig, ax
134
135
136 def save(fig, stem, axes=None, ids=None):

```

```

137     AUDIT.mkdir(parents=True, exist_ok=True)
138     report = require_matplotlib_panel_alignment(
139         fig, axes=axes, panel_ids=ids,
140         row_groups=[ids] if ids else None,
141         json_out=AUDIT / f"{stem}.alignment.json", strict=True,
142     )
143     fig.savefig(OUT / f"{stem}.pdf")
144     fig.savefig(OUT / f"{stem}.svg")
145     svg_path = OUT / f"{stem}.svg"
146     svg_path.write_text("\n".join(line.rstrip() for line in svg_path.read_text().splitlines()) + "\n")
147     fig.savefig(OUT / f"{stem}.png", dpi=300)
148     plt.close(fig)
149     print(stem, report["verdict"])
150
151
152 def mini_geometry(ax, index):
153     """Conceptual objects only; station ratios preserve the stated constructions."""
154     if index == 0:
155         pts = np.array([[5, 35], [10, 20], [26, 15], [32, 31], [19, 42]])
156         ax.add_patch(Circle((19, 29), 16, fill=False, edgecolor=TEAL, lw=.9))
157         ax.add_patch(Polygon(pts, facecolor="#EBE5F1", edgecolor=PURPLE, lw=1.15))
158         distance = np.linalg.norm(pts[:, None] - pts[None, :], axis=2)
159         i, j = np.unravel_index(np.argmax(distance), distance.shape)
160         ends = pts[[i, j]]
161         ax.plot(ends[:, 0], ends[:, 1], color=NAVY, lw=1.45)
162         ax.scatter(ends[:, 0], ends[:, 1], s=12, color=NAVY, zorder=4)
163         assert np.max(np.linalg.norm(pts - [19, 29], axis=1)) <= 16
164         # A radius and a diameter are different objects, even in the overview.
165         ax.plot([19, 35], [29, 29], color=TEAL, lw=1.0)
166         ax.scatter([19], [29], s=9, color=TEAL, zorder=4)
167     elif index == 1:
168         ax.add_patch(Polygon([[4, 37], [33, 28], [33, 33]],
169                             facecolor="#EBE5F1", edgecolor=PURPLE, lw=1))
170         ax.add_patch(Polygon([[12, 31], [19, 17], [27, 22], [29, 28]],
171                             facecolor="#E0EEF0", edgecolor="none"))
172         ax.add_patch(Polygon([[12, 39], [19, 46], [27, 41], [29, 34]],
173                             facecolor="#E0EEF0", edgecolor="none"))
174         ax.plot([4, 23, 31], [37, 23, 30.5], color=TEAL, lw=1.1)
175         ax.scatter([4], [37], color=NAVY, s=20, zorder=4)
176         ax.scatter([23], [23], color=ORANGE, s=27, zorder=4)
177         ax.scatter([31], [30.5], color=PURPLE, s=15, zorder=4)
178     elif index == 2:
179         ctr = np.array([19, 30])
180         ang = np.arange(6) * np.pi / 3
181         ring = ctr + 9.4 * np.c_[np.cos(ang), np.sin(ang)]
182         ax.add_patch(Circle(ctr, 15, fill=False, edgecolor=LINE, lw=1.1))
183         for p in np.vstack([ctr, ring]):
184             ax.add_patch(Circle(p, 8.35, facecolor="#E7F0F2", edgecolor=LINE,
185                                lw=.45, alpha=.38))
186         route = np.vstack([ctr, ring[[0, 1, 2, 3]]])
187         ax.plot(route[:, 0], route[:, 1], color=TEAL, lw=1.25)
188         ax.scatter(ring[:, 0], ring[:, 1], s=14, color=TEAL, zorder=4)
189         ax.scatter(*ctr, s=16, color=NAVY, zorder=4)
190         arrow(ax, [tuple(route[1]), tuple(route[2])], color=ORANGE)
191     else:
192         ctr = np.array([19, 30])
193         ang = np.arange(12) * np.pi / 6
194         inner = ctr + (950 / 1870 * 15.6) * np.c_[np.cos(ang), np.sin(ang)]
195         outer = ctr + 15.6 * np.c_[np.cos(ang + np.pi / 12), np.sin(ang + np.pi / 12)]
196         ax.add_patch(Circle(ctr, 1800 / 1870 * 15.6, fill=False,
197                            edgecolor=LINE, lw=1.0))
198         for k in range(12):
199             p = np.vstack([ctr, inner[k], inner[(k + 1) % 12], outer[k], inner[k]])
200             ax.plot(p[:, 0], p[:, 1], color=LINE, lw=.5)
201             p = np.vstack([inner[k], outer[(k - 1) % 12], outer[k]])
202             ax.plot(p[:, 0], p[:, 1], color=LINE, lw=.5)
203         ax.add_patch(Polygon([inner[0], inner[1], outer[0]], facecolor="#F5DFCB",
204                             edgecolor=ORANGE, lw=1.1, zorder=3))
205         ax.scatter(inner[:, 0], inner[:, 1], s=8, color=TEAL, zorder=4)
206         ax.scatter(outer[:, 0], outer[:, 1], s=8, color=ORANGE, zorder=4)
207         ax.scatter(*ctr, s=14, color=NAVY, marker="*", zorder=4)
208
209

```



```

210 def overview():
211     fig, base = canvas(109)
212     text(base, 6, 7, " 从局部定位到完整清除", size=12, weight="bold")
213     text(base, 174, 7, " 问题递进 · 方法衔接", size=8, color=MUTED, ha="right")
214     columns = []
215     labels = [
216         ("Q1", " 区域与覆盖", " 测向取交定位区域", " 区分直径与覆盖半径"),
217         ("Q2", " 鲁棒第二点", " 接收约束与交会几何", " 权衡定位精度与路程"),
218         ("Q3", " 全向多源搜索", " 七站发现 · 联合调度", " 兼顾共享观测与清除"),
219         ("Q4", " 定向混合场景", " 双环发现 · 光学回退", " 重建发现与清除保证"),
220     ]
221     for i, (qid, title, method, purpose) in enumerate(labels):
222         x = 6 + i * 43.5
223         ax = fig.add_axes([x / 180, (109 - 76) / 109, 37.5 / 180, 60 / 109])
224         ax.set(xlim=(0, 37.5), ylim=(60, 0))
225         ax.axis("off")
226         columns.append(ax)
227         text(ax, 0, 3, qid, 9, TEAL, "bold")
228         text(ax, 37.5, 3, title, 10, NAVY, "bold", "right")
229         ax.plot([0, 37.5], [8, 8], color=LINE, lw=.8)
230         mini_geometry(ax, i)
231         text(ax, 18.75, 51, method, 8.5, NAVY, "bold", "center")
232         text(ax, 18.75, 58, purpose, 8, MUTED, ha="center")
233         if i < 3:
234             arrow(base, [(x + 38.8, 46), (x + 42.2, 46)], color=MUTED, width=.9)
235     panel(base, 6, 82, 168, 20, face=PALE)
236     text(base, 10, 87.5, "Q3-Q4 在线执行", 8, TEAL, "bold")
237     steps = [" 观测反馈", " 更新位置区域", " 选择合法动作", " 执行当前任务"]
238     centers = [61, 93, 126, 158]
239     for i, (x, value) in enumerate(zip(centers, steps)):
240         text(base, x, 87.5, value, 8.4, NAVY, ha="center")
241         if i < 3:
242             arrow(base, [(x + 13.5, 87.5), (centers[i + 1] - 13.5, 87.5)],
243                     color=TEAL, width=.9)
244     arrow(base, [(158, 91.5), (158, 97), (61, 97), (61, 91.5)],
245            color=TEAL, width=.8)
246     # The feedback label is outside the return stroke, never overprinted on it.
247     text(base, 10, 97, " 任务完成后重排", 8, MUTED)
248     save(fig, "fig01_overview", columns, ["Q1", "Q2", "Q3", "Q4"])
249
250
251 def algorithm():
252     fig, ax = canvas(122)
253     for x, num, label in [(7, "01", " 维护任务池"), (65, "02", " 滚动排序"),
254                          (121, "03", " 执行与反馈")]:
255         text(ax, x, 8, num, 10, TEAL, "bold")
256         text(ax, x + 8, 8, label, 11, NAVY, "bold")
257     ax.plot([7, 173], [14, 14], color=LINE, lw=.8)
258
259     # One pool holds both task types. Top-level scheduling is task based.
260     panel(ax, 7, 24, 44, 49)
261     text(ax, 12, 33, " 未完成搜索站", 9.5, NAVY, "bold")
262     text(ax, 12, 41, " 代表点: 站点坐标", 8, MUTED)
263     ax.plot([12, 46], [48, 48], color=LINE, lw=.7)
264     text(ax, 12, 56, " 已知未清源", 9.5, NAVY, "bold")
265     text(ax, 12, 64, " 代表点: 区域包围盒中心", 8, MUTED)
266     arrow(ax, [(52, 48.5), (63, 48.5)], color=NAVY)
267
268     panel(ax, 65, 34, 42, 24, face=NAVY)
269     text(ax, 86, 41.5, " 最近邻 + 2-opt", 10, WHITE, "bold", "center")
270     text(ax, 86, 51, " 比较开放路径长度", 8.5, WHITE, ha="center")
271     text(ax, 86, 65.5, " 只执行首项任务", 9.5, ORANGE, "bold", "center")
272
273     # The branches are alternatives, not parallel execution.
274     arrow(ax, [(108.5, 46), (115, 46), (115, 32), (120, 32)])
275     arrow(ax, [(115, 46), (115, 63), (120, 63)])
276     panel(ax, 122, 21, 51, 22, face=PALE)
277     text(ax, 126, 27.5, " 搜索任务", 9.5, TEAL, "bold")
278     text(ax, 126, 35.5, " 到站扫描未知频道, 登记新源", 8)
279     panel(ax, 122, 49, 51, 31, face=WARM)
280     text(ax, 126, 55.5, " 源服务任务", 9.5, ORANGE, "bold")
281     text(ax, 126, 64, " 测向收缩 → 覆盖清除 / 光学回退", 7.8)
282     text(ax, 126, 72.5, " 满足接收与信息条件才共享观测", 7.8)

```

```

283
284 # Both complete tasks merge into one record-update step.
285 arrow(ax, [(174, 32), (177, 32), (177, 88), (163, 88), (163, 91)], width=.9)
286 arrow(ax, [(147.5, 81), (147.5, 91)], width=.9)
287 panel(ax, 64, 92, 109, 12, face=PALE, edge=LINE)
288 text(ax, 118.5, 98, " 任务完成: 更新位置区域、扫描与清除记录", 9, NAVY, "bold", "center")
289 arrow(ax, [(63, 98), (48, 98)], width=1)
290
291 decision = Polygon([(31, 87), (47, 98), (31, 109), (15, 98)],
292                    facecolor=WHITE, edgecolor=TEAL, linewidth=1.1)
293 ax.add_patch(decision)
294 text(ax, 31, 98, " 满足停止\n 条件? ", 8.5, NAVY, "bold", "center")
295 arrow(ax, [(31, 86), (31, 74)], color=TEAL)
296 text(ax, 34, 80, " 否: 重建任务池", 8, TEAL)
297 arrow(ax, [(14, 98), (7, 98), (7, 114), (18, 114)], color=TEAL, width=.9)
298 text(ax, 11, 94, " 是", 8, TEAL, ha="center")
299 text(ax, 20, 114, " 退出并核验反馈", 8.5, TEAL, "bold")
300 text(ax, 66, 113, " 停止条件: 清除 16 源; 或已知源全清除, ", 7.8, MUTED)
301 text(ax, 66, 118, " 且每个未知频道都有七站无信号记录.", 7.8, MUTED)
302 save(fig, "fig06_q3_algorithm")
303
304
305 def comparison(stem, number):
306     """Review sheet made from the unchanged old PDF and final vector export."""
307     import pymupdf
308     fig, base = canvas(242)
309     text(base, 6, 8, f" 图 {number} · 改前 / 改后", 12, NAVY, "bold")
310     regions = [(f" 改前", REVIEW / "before" / f"{stem}.pdf", 20),
311               (f" 改后", OUT / f"{stem}.pdf", 132)]
312     for title, path, top in regions:
313         text(base, 6, top, title, 9, TEAL, "bold")
314         doc = pymupdf.open(path)
315         pix = doc[0].get_pixmap(matrix=pymupdf.Matrix(2, 2), alpha=False)
316         arr = np.frombuffer(pix.samples, dtype=np.uint8).reshape(pix.height, pix.width, 3)
317         imax = fig.add_axes([5 / 180, (242 - top - 107) / 242, 170 / 180, 101 / 242])
318         imax.imshow(arr)
319         imax.axis("off")
320         doc.close()
321     fig.savefig(REVIEW / f"figure{number:02d}_comparison.pdf")
322     fig.savefig(REVIEW / f"figure{number:02d}_comparison.png", dpi=300)
323     plt.close(fig)
324
325
326 def main():
327     parser = argparse.ArgumentParser(description=__doc__)
328     parser.add_argument("--comparisons", action="store_true", help="Also export review sheets.")
329     args = parser.parse_args()
330     overview()
331     algorithm()
332     if args.comparisons:
333         comparison("fig01_overview", 1)
334         comparison("fig06_q3_algorithm", 8)
335
336
337 if __name__ == "__main__":
338     main()

```

程序 14

对应文件: manuscript/story_draft/analysis/revision_figures.py

```

1 """Revision: source-grounded geometry, conceptual routes, and complete distributions."""
2 exec((__import__('pathlib').Path(__file__).with_name('make_figures.py')).read_text())
3 import sys
4 sys.path.insert(0, 'USER_HOME/.codex/skills/nature-figure/scripts')
5 from audit_panel_alignment import require_matplotlib_panel_alignment
6 mpl.rcParams.update({'font.family': 'sans-serif', 'font.sans-serif': ['Alibaba PuHuiTi', 'DejaVu
   ↳ Sans'], 'pdf.fonttype': 42, 'svg.fonttype': 'none'})
7 PURPLE='#8564A9'; RED='#B94145'; GREEN='#3E8864'; LIGHT='#DFEEF0'
8 oldsave=save
9 def save(fig,name,claim,kind,source):

```

```

10     if len(fig.axes)>1:
11         require_matplotlib_panel_alignment(fig,json_out=QA/(name+'.alignment.json'),require_panel_labels=False)
12         fig.savefig(OUT/(name+'.pdf'))
13         fig.savefig(OUT/(name+'.svg'))
14         fig.savefig(OUT/(name+'.png'),dpi=300)
15         plt.close(fig)
16     else:oldsave(fig,name,claim,kind,source)
17 def multi(n,height=3.0):
18     f,aa=plt.subplots(1,n,figsize=(6.8,height));f.subplots_adjust(left=.145,right=.98,bottom=.20,top=.85,wspace=.42)
19     return f,aa
20
21 def clip(poly,normal,bound):
22     result=[]
23     for p,q in zip(poly,np.roll(poly,-1,axis=0)):
24         vp=normal@p-bound;vq=normal@q-bound
25         if vp<=1e-9:result.append(p)
26         if (vp<0 and vq>0) or (vp>0 and vq<0):result.append(p+(q-p)*vp/(vp-vq))
27     return np.array(result)
28 def observe(poly,s,g,delta=1):
29     theta=np.arctan2(*(g-s)[::-1]);d=np.deg2rad(delta)
30     lo=np.array([np.cos(theta-d),np.sin(theta-d)]);hi=np.array([np.cos(theta+d),np.sin(theta+d)])
31     for norm in [np.array([lo[1],-lo[0]]),np.array([-hi[1],hi[0]])]:poly=clip(poly,norm,norm@s)
32     return poly
33
34 # Three explicit layers and an observation feedback loop.
35 f,ax=canvas(3.9,.025,.04);ax.set(xlim=(0,10),ylim=(0,6));ax.axis('off')
36 for y,label in
37     ↳ [(4.8,'定位层'),(2.8,'搜索层'),(.8,'调度层')]:ax.text(.1,y+.35,label,va='center',weight='bold',fontsize=9)
38 for x,txt in [(1.35,'测向读数'),(3.5,'误差扇区'),(5.65,'候选区域'),(7.8,'第二点选择')]:box(ax,x,4.8,1.85,.75,txt,fs=9)
39 for x,txt in [(1.35,'多站覆盖\n发现未知源'),(4.5,'重复观测\n收缩位置区域'),(7.65,'覆盖检查\n光学清除')]:box(ax,x,2.6,2
39     ↳ ,.95,txt,fs=9)
40 arrow(ax,(3.4,3.05),(4.45,3.05));arrow(ax,(6.55,3.05),(7.6,3.05))
41 arrow(ax,(8.7,4.75),(8.7,4.15));arrow(ax,(8.7,4.15),(5.5,4.15));arrow(ax,(5.5,4.15),(5.5,3.6))
42 box(ax,1.35,.45,3.7,1.05,'搜索与清除任务进入同一任务池\n 仅执行首项,收到反馈后重排',fs=9)
43 box(ax,6.05,.45,3.6,1.05,'已知源清完 + 覆盖或数量证书\n 满足才退出,否则继续',fs=9)
44 arrow(ax,(5.1,.98),(6,.98));arrow(ax,(3.2,1.55),(3.2,2.2));arrow(ax,(3.2,2.2),(2.35,2.2));arrow(ax,(2.35,2.2),(2.35,2
44     ↳ .55))
45 save(f,'fig01_overview','定位、搜索和调度形成反馈闭环','整体流程图','model definitions')
46
47 # Diameter below40 is insufficient for a20m clearance disk.
48 f,ax=canvas(3.3);D=39.;t=np.array([[0,0],[D,0],[D/2,D*np.sqrt(3)/2]])
49 ax.add_patch(Polygon(t,facecolor='#E8E4F1',edgecolor=PURPLE,lw=1.5))
50 ax.plot([0,D],[0,0],color=DARK,lw=2)
51 ax.add_patch(Circle((D/2,0),D/2,fill=False,color=ORANGE,ls='--',lw=1.4))
52 ax.add_patch(Circle((D/2,D/(2*np.sqrt(3))),D/np.sqrt(3),fill=False,color=TEAL,lw=1.3))
53 ax.scatter(*t[2],color=RED,s=30,zorder=5)
54 for y,txt,c in [(28,'D = 39 m < 40 m',DARK),(16,'D/2 = 19.50 m: 漏顶点',ORANGE),(4,'最小覆盖半径 = 22.52
54     ↳ m',TEAL),(-8,'大于 20 m 清除半径',RED)]:ax.text(47,y,txt,color=c,fontsize=9)
55 ax.set(xlim=(-5,105),ylim=(-23,40),xlabel='x (m)',ylabel='y (m)',aspect='equal')
56 save(f,'fig02_diameter','直径小于40m仍不能保证一次20m清除','合法测向几何反例','D=39 equilateral construction, Appendix
56     ↳ A.1')
57
58 # Bearing intersection, exact polygons, with a zoom rather than exaggerated angles.
59 g=np.array([600.,180.]);sites=np.array([0.,0.],np.array([850.,-450.]])
60 p=np.array([[-1800.,-1800.],[1800.,-1800.],[1800.,1800.],[-1800.,1800.]])
61 for s in sites:p=observe(p,s,g)
62 dist=np.linalg.norm(p[:,None]-p[None,:],axis=2);i,j=np.unravel_index(np.argmax(dist),dist.shape)
63 f,aa=multi(2,3.5)
64 for ax in aa:
65     for k,s in enumerate(sites):
66         theta=np.arctan2(*(g-s)[::-1]);color=TEAL if k==0 else ORANGE
67         for delta in [-1,0,1]:
68             end=s+1600*np.array([np.cos(theta+np.deg2rad(delta)),np.sin(theta+np.deg2rad(delta))])
69             ax.plot([s[0],end[0]],[s[1],end[1]],color=color,lw=.8,ls='--' if delta else '-')
70             ax.add_patch(Polygon(p,facecolor='#E8E4F1',edgecolor=PURPLE,lw=1.2,zorder=4))
71             ax.plot(p[[i,j],0],p[[i,j],1],color=DARK,lw=1.7,zorder=5)
72             ax.set(xlabel='x (m)',ylabel='y (m)',aspect='equal')
73 aa[0].scatter(np.array(sites)[:0],np.array(sites)[1],c=[TEAL,ORANGE],s=25)
74 aa[0].text(0,65,'S1',color=TEAL);aa[0].text(800,-535,'S2',color=ORANGE)
75 aa[0].set(xlim=(-100,950),ylim=(-600,450),title='(a) 两条 ±1° 测向扇区')
76 aa[1].set(xlim=(570,630),ylim=(150,210),title='(b) 交集与最远顶点对')
77 aa[1].text(571,153,f'D(P) = {dist[i,j]:.2f} m',fontsize=9)

```

```

78 for ax in aa:
79     bounds=[(*ax.get_xlim(),),(*ax.get_ylim(),)]
80     for line in ax.lines:
81         xx,yy=line.get_data();p0=np.array([xx[0],yy[0]]);p1=np.array([xx[-1],yy[-1]]);d0=p1-p0;lo=0.;hi=1.
82         for dim,(lower,upper) in enumerate(bounds):
83             if abs(d0[dim])<1e-12:continue
84             t0=(lower-p0[dim])/d0[dim];t1=(upper-p0[dim])/d0[dim];lo=max(lo,min(t0,t1));hi=min(hi,max(t0,t1))
85             if lo<=hi:
86                 zz=np.array([p0+lo*d0,p0+hi*d0]);line.set_data(zz[:,0],zz[:,1])
87             else:line.set_data([],[])
88
89 save(f,'fig16_bearing_intersection','测向误差通过取交转为可计算区域','精确几何示意','two sites (0,0),(850,-450),
    ↪ source(600,180), zero errors, ±1°')
90
91 # Q2 selection: same scale in three panels, actual narrow angular wedge.
92 f,aa=multi(3,3.25)
93 for ax in aa:ax.set(xlim=(-100,1650),ylim=(-1100,1100),xlabel='a (m)',aspect='equal');ax.tick_params(labelsize=8)
94 aa[0].set_ylabel('b (m)')
95 ang=np.deg2rad(1.005);w=np.array([[0,0],[1500*np.cos(ang),-1500*np.sin(ang)],[1500,0],[1500*np.cos(ang),1500*np.sin(a
    ↪ ng]])
96 aa[0].add_patch(Polygon(w,facecolor='#E8E4F1',edgecolor='PURPLE',lw=1.4));aa[0].scatter(0,0,s=22,color=TEAL)
97 for end,label in [((600,0),'u'),((0,600),'v')]:arrow(aa[0],(0,0),end);aa[0].text(end[0]+50,end[1]+150,label)
98 aa[0].text(420,-430,'5 < r ≤ 1500 m',fontsize=8);aa[0].set_title('(a) 首测候选扇区',fontsize=9)
99 for ax in aa[1:]:
100     ax.pcolormesh(a,b,np.where(rec,1,0),cmap=ListedColormap(['white',LIGHT]),shading='auto',rasterized=True)
101     ax.axhline(0,color=GRAY,lw=.6)
102 aa[1].plot(w[[0,1,2,3,0],0],w[[0,1,2,3,0],1],color='PURPLE',lw=1)
103 aa[1].text(280,540,'保证接收',fontsize=8);aa[1].text(850,-1000,'域外无保证',fontsize=8);aa[1].set_title('(b)
    ↪ 对全部候选均接收',fontsize=9)
104 aa[2].contourf(a,b,safe,levels=[.5,1.5],colors=['#F3DCA3'])
105 aa[2].scatter([800,800],[605,-605],c=ORANGE,s=23,zorder=5)
106 aa[2].text(80,1000,'Q = (800, ±605)',fontsize=8);aa[2].set_title('(c) 排除近距与退化',fontsize=9)
107 save(f,'fig04_q2_region','从源候选集合到接收域再到安全候选','算法三联图','analytic inequalities (7),(8); near5m
    ↪ exclusion in final condition')
108
109 # Distance on Pareto x-axis.
110 f,ax=canvas(3.5)
111 nd=sorted([r for r in rows if r['pareto_within_fixed_library']],key=lambda r:r['move_m'])
112 ax.plot([r['move_m'] for r in nd],[r['dense_sample_worst_D'] for r in nd],color=TEAL,lw=1)
113 for r in rows:
114     xy=(r['move_m'],r['dense_sample_worst_D']);key=tuple(r['offset'])
115     ax.scatter(*xy,color=ORANGE if key==(800,605) else TEAL if r['pareto_within_fixed_library'] else GRAY,s=50 if
        ↪ key==(800,605) else 30,zorder=3)
116     ax.annotate(str(key),xy,xytext=offsets[key],textcoords='offset points',fontsize=8)
117 ax.set(xlim=(610,1040),ylim=(112,319),xlabel='首测点至第二检测点的移动距离 (m)',ylabel='采样最大定位直径 (m)')
118 save(f,'fig05_q2_tradeoff','移动更多并不总能取得更好精度','固定候选库统计','q12_verification.json: all8 offsets')
119
120 # Monotone set shrink, computed toy rather than invented quantities.
121 g=np.array([600.,100.]);ss=[np.array([0.,0.]),np.array([350.,-250.]),np.array([500.,100.])]
122 poly=np.array([[-1500.,-1500.],[1500.,-1500.],[1500.,1500.],[-1500.,1500.]])
123 pp=[];metrics=[]
124 for s in ss:
125     poly=observe(poly,s,g);pp.append(poly.copy());center=(poly.min(axis=0)+poly.max(axis=0))/2
126     metrics.append(dict(diameter=float(np.linalg.norm(poly[:,None]-poly[None,:],axis=2).max()),radius=float(np.linalg)
        ↪ .norm(poly-center,axis=1).max()),center=center.tolist())
127 assert metrics[-1]['radius']<20
128 f,aa=multi(3,3.2)
129 for k,ax in enumerate(aa):
130     poly=pp[k];c=np.array(metrics[k]['center']);rad=metrics[k]['radius']
131     ax.add_patch(Polygon(poly,facecolor='#E8E4F1',edgecolor='PURPLE',lw=1.2));ax.scatter(*g,c=RED,s=18,zorder=4)
132     if k==2:ax.add_patch(Circle(c,20,fill=False,color=GREEN,lw=1.2))
133     span=max(np.ptp(poly,axis=0).max()*1.18,48)
134     ax.set(xlim=(c[0]-span/2,c[0]+span/2),ylim=(c[1]-span/2,c[1]+span/2),aspect='equal',xlabel='x
        ↪ (m)',title=f'({chr(97+k)}) 第 {k+1} 次观测',ylabel='y (m)' if k==0 else '')
135     ax.tick_params(labelsize=8);ax.text(.5,1.01,f'D = {metrics[k]["diameter"]:.2f}
        ↪ m',transform=ax.transAxes,ha='center',fontsize=8)
136     ax.set_title(f'({chr(97+k)}) 第 {k+1} 次观测',fontsize=9,pad=26)
137 save(f,'fig17_shrink','连续有效测向取交保留真源并收缩集合','计算示意图','zero-error toy; initial box ±1500m,
    ↪ source(600,100), three listed sites')
138 (QA/'revision_geometry.json').write_text(json.dumps({'shrink':metrics,'bearing_diameter':float(dist[i,j]),indent=2))
139
140 # Three-source teaching example, NOT actual baseline or simulator trace.
141 f,aa=multi(2,3.7)

```

```

142 sources=np.array([[700.,200.],[-600.,650.],[-400.,-800.]])
143 second=sources+np.array([[100,-130],[130,90],[-130,100]])
144 search=[seven[k] for k in [0,1,2,3,4,5,6]]
145 late=search+[x for pair in zip(second,sources) for x in pair]
146 early=[seven[0],seven[1],second[0],sources[0],seven[2],seven[3],second[1],sources[1],seven[4],seven[5],second[2],sour
    ↪ ces[2],seven[6]]
147 for k,(ax,route) in enumerate(zip(aa,[late,early])):
148     ax.add_patch(Circle((0,0),1800,fill=False,color=DARK,lw=1))
149     ax.scatter(seven[:,0],seven[:,1],color=TEAL,s=22,zorder=4)
150     ax.scatter(second[:,0],second[:,1],color=ORANGE,s=22,zorder=5)
151     ax.scatter(sources[:,0],sources[:,1],color=RED,marker='x',s=25,zorder=5)
152     for p0,p1 in zip(route[:-1],route[1:]):
153         ax.annotate('',xy=p1,xytext=p0,arrowprops=dict(arrowstyle='->',lw=.7,color=DARK,alpha=.7))
154     for s in sources:
155         ax.add_patch(Polygon(s+np.array([[-55,-35],[55,-35],[0,55]]),facecolor='#E8E4F1',edgecolor=PURPLE,lw=.6))
156     for idx,s in enumerate(seven):
157         shift=np.array([-200.,-200.]) if idx==0 else 200*s/np.linalg.norm(s)
158         ax.text(*(s+shift),str(idx),fontsize=8,color=TEAL,ha='center',va='center')
159     ax.set(xlim=(-2000,2000),ylim=(-2000,2000),aspect='equal',xlabel='x (m)',ylabel='y (m)' if k==0 else
    ↪ '',title=['(a) 搜索后集中服务','(b) 搜索与服务交错'][k],xticks=[-1500,0,1500],yticks=[-1500,0,1500])
160 save(f,'fig18_route_example','交错调度改变服务时机与路线结构','三源教学示意,非实测','hand-constructed route order; not
    ↪ the frozen comparison baseline')
161
162 # Q3 distributions with all observed runs and medians; retain worst pair explicitly.
163 f,aa=multi(2,3,6)
164 rr=[r for r in pairs if r['rounding']=='bounded']
165 for ax,env,label in zip(aa,['bounded'],'pre_round_stress',['有界读数','舍入压力']):
166     rr=[r for r in pairs if r['rounding']==env];assert len(rr)==480
167     vals=[[float(r[key]) for r in rr] for key in ['baseline_s','candidate_s']]
168     bp=ax.boxplot(vals,tick_labels=['冻结基线','联合方案'],patch_artist=True,widths=.45,showfliers=True,flierprops=di
    ↪ ct(marker='.',markersize=3))
169     for b0,color in zip(bp['boxes'],[GRAY,TEAL]):b0.set(facecolor=color,alpha=.6)
170     ax.set(ylim=(800,5350),ylabel='总虚拟时间 (s)' if ax==aa[0] else '',title=label)
171     if ax==aa[0]:
172         worst=max(rr,key=lambda r:float(r['change_pct']));ys=[float(worst['baseline_s']),float(worst['candidate_s'])]
173         ax.plot([1,2],ys,color=RED,marker='o',lw=1)
174         ax.text(1.5,4900,'同一慢例: +93.63%',color=RED,ha='center',fontsize=8)
175 save(f,'fig19_q3_time_distribution','均值下降不排除单次严重退化','全部运行箱线图','q3_final_pairs.csv:480 runs per
    ↪ environment per arm')
176
177 # Add boxes to all six family distributions.
178 f,ax=canvas(3,5)
179 family_order=['uniform','outer','cluster','min_radius','radial','reception_edge']
180 values=[[r['change_pct'] for r in layout_data if r['family']==fam] for fam in family_order]
181 ax.boxplot(values,positions=range(6),widths=.45,showfliers=False,patch_artist=True,boxprops=dict(facecolor=LIGHT),med
    ↪ ianprops=dict(color=DARK))
182 for idx,vals in
    ↪ enumerate(values):ax.scatter(idx+np.linspace(-.18,.18,len(vals)),vals,s=14,color=TEAL,alpha=.65,zorder=3)
183 ax.axhline(0,color=ORANGE,ls='--',lw=1);ax.set(xticks=range(6),xticklabels=['均匀','外缘','聚集','最小半径','窄径向',
    ↪ '接收边缘'],ylabel='布局平均总时间变化
    ↪ (%)',ylim=(-30,68),xlim=(-.5,5.5))
184 save(f,'fig11_q3_distribution','保留全部布局与组内四分位分布','配对箱线与散点','120 layouts,4 bounded error conditions
    ↪ each')
185
186 # Q4 historical distributions, keep original ablation means in text.
187 f,aa=multi(2,3,6)
188 for ax,batch,label in zip(aa,['round9_main','round9_rounding'],['有界读数: 96 布局','舍入压力: 96 布局']):
189     vals=[[float(r['virtual_time_s']) for r in q4 if r['batch']==batch and r['config']==cf] for cf in
    ↪ ['BASE','FINAL']]
190     assert all(len(v)==96 for v in vals)
191     bp=ax.boxplot(vals,tick_labels=['31站初始基线','25站推荐'],patch_artist=True,widths=.45,flierprops=dict(marker='.',
    ↪ markersize=3))
192     for patch,color in zip(bp['boxes'],[GRAY,TEAL]):patch.set(facecolor=color,alpha=.6)
193     ax.set(title=label,ylim=(3000,20000),ylabel='总虚拟时间 (s)' if ax==aa[0] else '')
194     ax.tick_params(axis='x',labelsize=8)
195 save(f,'fig14_q4_historical','25站整套方案的时间分布改善','历史验证箱线图','q4_source_data.csv:2 batches96,BASE versus
    ↪ FINAL')
196
197 # Improve Q4 source direction annotation and 25 station color semantics.
198 f,ax=canvas(3,5)
199 ax.add_patch(Wedge((0,0),1000,-90,90,facecolor=LIGHT,edgecolor='none'));arc=np.linspace(-np.pi,np.pi,361);ax.plot(100
    ↪ 0*np.cos(arc),1000*np.sin(arc),color=GRAY,ls='--',lw=1);ax.plot([0,0],[-1000,1000],color=TEAL,lw=.8)
200 ax.scatter([0,-350,650],[0,0,250],color=[RED,TEAL,TEAL],s=30,zorder=4)

```

```

201 arrow(ax,(0,0),(480,0));ax.text(270,-120,'n_g',fontsize=10)
202 ax.text(-940,170,'背面站: 无信号',fontsize=9);ax.text(100,520,'正面站',fontsize=9)
203 ax.text(110,-520,'发射半平面',fontsize=9);ax.text(-390,-240,'源 g',color=RED)
204 ax.set(xlim=(-1150,1150),ylim=(-1150,1150),aspect='equal',xlabel='relative x (m)',ylabel='relative y (m)')
205 save(f,'fig08_no_signal','距离近仍可能背向失联','定向反例示意','R1000m, backwardsite350m fromsource')
206 f,ax=canvas(4.6,bottom=.27)
207 for k,tt in enumerate(tri):ax.add_patch(Polygon(pts[tt],facecolor=LIGHT if k%2 else '#F6EEE4',edgecolor=GRAY,lw=.7))
208 ax.add_patch(Circle((0,0),1800,fill=False,color=DARK,lw=1.7))
209 ax.scatter(pts[1:13,0],pts[1:13,1],color=TEAL,s=24,zorder=4,label='内环 950 m')
210 ax.scatter(pts[13:,0],pts[13:,1],color=ORANGE,s=24,zorder=4,label='外环 1870 m')
211 ax.scatter(0,0,marker='*',s=95,color=DARK,zorder=5)
212 tt=tri[4];center=pts[tt].mean(axis=0);ax.scatter(*center,color=RED,s=20,zorder=5)
213 for vertex in pts[tt]:ax.annotate(' ',xy=vertex,xytext=center,arrowprops=dict(arrowstyle='->',lw=.9,color=PURPLE))
214 ax.text(-2000,2170,'36 个三角形; 最长边 983.60 m',fontsize=9)
215 ax.set(xlim=(-2150,2150),ylim=(-2200,2400),aspect='equal',xlabel='x (m)',ylabel='y (m)');ax.legend(loc='lower
    ↪ center',bbox_to_anchor=(.5,-.38),ncol=2,fontsize=8)
216 save(f,'fig09_q4_cover','任意位置由近距三角形顶点包围','连续覆盖构造','25 exactstations and36triangles')
217 print('Revision figures complete; shrink metrics:',metrics)

```

程序 15

对应文件: manuscript/story_draft/analysis/second_pass_figures.py

```

1 """Targeted display repairs; no new experiments and no observations excluded."""
2 from pathlib import Path
3 exec(Path(__file__).with_name('make_figures.py').read_text().split('# 1. The question')[0])
4 import sys
5 sys.path.insert(0,str(Path.home()/'..codex/skills/nature-figure/scripts'))
6 from audit_panel_alignment import require_matplotlib_panel_alignment
7 mpl.rcParams.update({'font.family':'sans-serif','font.sans-serif':['Alibaba PuHuiTi','DejaVu
    ↪ Sans'],'pdf.fonttype':42,'svg.fonttype':'none'})
8 # Only changed statistical figure: retain the 18821s observation previously above the axis limit.
9 q4=list(csv.DictReader((DATA/'q4_source_data.csv').open()))
10 f,aa=plt.subplots(1,2,figsize=(6.8,3.6));f.subplots_adjust(left=.145,right=.98,bottom=.20,top=.85,wspace=.42)
11 for ax,batch,label in zip(aa,['round9_main','round9_rounding'],['有界读数: 96 布局','舍入压力: 96 布局']):
12     vals=[[float(r['virtual_time_s']) for r in q4 if r['batch']==batch and r['config']==cf] for cf in
        ↪ ['BASE','FINAL']]
13     assert all(len(v)==96 for v in vals)
14     assert all(3000<min(v) and max(v)<20000 for v in vals)
15     bp=ax.boxplot(vals,tick_labels=['31站初始基线','25站推荐'],patch_artist=True,widths=.45,flierprops=dict(marker='.',
        ↪ 'markersize=3))
16     for patch,color in zip(bp['boxes'],[GRAY,TEAL]):patch.set(facecolor=color,alpha=.6)
17     ax.set(title=label,ylim=(3000,20000),ylabel='总虚拟时间 (s)' if ax==aa[0] else '')
18     ax.tick_params(axis='x',labelsize=8)
19 require_matplotlib_panel_alignment(f,json_out=QA/'fig14_q4_historical.alignment.json')
20 f.savefig(OUT/'fig14_q4_historical.pdf');f.savefig(OUT/'fig14_q4_historical.svg');f.savefig(OUT/'fig14_q4_historical.
    ↪ png',dpi=300);plt.close(f)
21 # Disc fill explicitly shows what covers the domain.
22 angs=np.arange(6)*np.pi/3;seven=np.vstack([[0,0],1125*np.c_[np.cos(angs),np.sin(angs)]])
23 f,ax=canvas(4)
24 for station in seven:ax.add_patch(Circle(station,1000,facecolor='#DFFEF0',edgecolor=GRAY,lw=.8,alpha=.48))
25 ax.add_patch(Circle((0,0),1800,fill=False,color=DARK,lw=1.5))
26 ax.scatter(seven[:,0],seven[:,1],color=TEAL,s=26,zorder=4)
27 g=1800*np.array([np.cos(np.pi/6),np.sin(np.pi/6)])
28 ax.plot([seven[1,0],g[0]],[seven[1,1],g[1]],color=DARK,lw=1.3);ax.scatter(*g,color=ORANGE,s=30,zorder=5)
29 ax.text(-2050,2250,'全域最远近站距离约 999.11 m < 1000 m',fontsize=9)
30 ax.set(xlim=(-2250,2250),ylim=(-2250,2480),xlabel='x (m)',ylabel='y (m)',aspect='equal')
31 save(f,'fig07_q3_cover','连续七圆盘覆盖目标域','解析几何构造','seven fixed stations at radius1125m, guarantee
    ↪ radius1000m')
32 # Task flow, preserving implementation hierarchy: station task and source service task.
33 f,ax=canvas(4.2,.025,.035);ax.set(xlim=(0,10),ylim=(0,6.2));ax.axis('off')
34 box(ax,.2,5.0,2.0,.7,'到达搜索站',fs=9);box(ax,2.7,5.0,2.0,.7,'扫描必要频道',fs=9);box(ax,5.2,5.0,2.0,.7,'发现新源?
    ↪ ',fs=9);box(ax,7.7,5.0,2.0,.7,'建立源服务任务',fs=9)
35 for x in [2.25,4.75,7.25]:arrow(ax,(x,5.35),(x+.4,5.35))
36 ax.text(7.48,5.67,'是',fontsize=8,ha='center')
37 box(ax,.2,2.85,2.8,1.05,'任务池与停止检查\n 未访问搜索站 / 待清源',fs=8.5)
38 box(ax,3.6,2.85,2.8,1.05,'最近邻与 2-opt 重排\n 只执行首项任务',fs=9)
39 box(ax,7.0,2.85,2.7,1.05,'源服务: 依据当前区域\n 测向 / 直接清除 / 回退',fs=8.5)
40 arrow(ax,(3.05,3.38),(3.55,3.38));arrow(ax,(6.45,3.38),(6.95,3.38))
41 arrow(ax,(8.7,4.95),(8.7,4.45));arrow(ax,(8.7,4.45),(1.6,4.45));arrow(ax,(1.6,4.45),(1.6,3.95))
42 arrow(ax,(6.2,4.95),(6.2,4.5));ax.text(6.35,4.69,'否: 记录后重排',fontsize=8)

```

```

43 arrow(ax,(4.3,3.95),(4.3,4.1));arrow(ax,(4.3,4.1),(1.2,4.1));arrow(ax,(1.2,4.1),(1.2,4.95))
44 box(ax,7.0,.6,2.7,1.05,'满足信息价值才共享检测\n 按 20 m 覆盖方案执行清除',fs=8.5)
45 arrow(ax,(8.35,2.8),(8.35,1.7))
46 box(ax,3.6,.6,2.8,1.05,'更新集合、扫描与清除记录\n 检查停止证书',fs=8.5)
47 arrow(ax,(6.95,1.13),(6.45,1.13))
48 box(ax,.2,.6,2.8,1.05,'证书成立: 退出\n 否则: 保留未完成任务',fs=8.5)
49 arrow(ax,(3.55,1.13),(3.05,1.13));arrow(ax,(1.6,1.7),(1.6,2.8))
50 save(f,'fig06_q3_algorithm','搜索反馈与源服务共同改变下一任务','算法流程图','documented Q3 task hierarchy, not a real
    ↳ trajectory')
51 print('Updated Q3 flow/coverage and Q4 full-range distribution; all 384 observations included')

```

程序 16

对应文件: paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/__init__.py

程序 17

对应文件: paper_evidence/q4_mechanism_reset_20260912/audit.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py; paper_evidence/q4_mechanism_reset_20260912/round04

_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py;paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py;paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py;paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py;paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py;paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py;paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/audit.py

```

1 """Independent post-run physical replay, time accounting, and stop-certificate audit."""
2 import math
3 import hashlib
4 import json
5 from functools import lru_cache
6 from pathlib import Path
7 from collections import Counter
8 import numpy as np
9 from question4.geometry import segment_distance
10
11 @lru_cache(maxsize=4)
12 def check_cover(contents):
13     data=json.loads(contents);assert not data['failed_cells']
14     points=np.array(data['stations']);leaves={r['key']:r for r in data['leaves']}
15     assert len(leaves)==len(data['leaves'])
16     nodes=set()
17     for key in leaves:
18         assert set(key)<=set('0123')
19         nodes.update(key[:i] for i in range(len(key)))
20     assert not (nodes & leaves.keys())
21     for key in nodes:assert all(key+str(i) in nodes or key+str(i) in leaves for i in range(4))
22     assert '' in nodes or '' in leaves
23     for key,row in leaves.items():
24         x0,y0,x1,y1=-1800.,-1800.,1800.,1800.
25         for digit in key:
26             xm=(x0+x1)/2;ym=(y0+y1)/2
27             x0,y0,x1,y1=((x0,y0,xm,ym),(xm,y0,x1,ym),(x0,ym,xm,y1),(xm,ym,x1,y1))[int(digit)]
28         if row.get('outside'):
29             dx=max(x0,0,-x1);dy=max(y0,0,-y1)
30             assert dx*dx+dy*dy>1800**2
31             continue
32         corners=np.array([(x0,y0),(x1,y0),(x1,y1),(x0,y1)])
33         hull=points[row['witnesses']];assert len(hull)>=3
34         assert len(set(row['witnesses']))==len(hull)
35         q=np.roll(hull,-1,axis=0)
36         assert np.sum(hull[:,0]*q[:,1]-hull[:,1]*q[:,0])>1e-6
37         assert np.linalg.norm(hull[:,None]-corners[None,:],axis=2).max(<)<1000
38         for i in range(len(hull)):
39             a=hull[i];edge=hull[(i+1)%len(hull)]-a
40             cross=edge[0]*(corners[:,1]-a[1])-edge[1]*(corners[:,0]-a[0])
41             assert cross.min(>)= -1e-7
42     return points
43
44 def audit(case,records,result):
45     sources={s['channel']:s for s in case['sources']};cleared=set();measured={}
46     requests={};executed=set();p=(0.,0.);channel=1;vt=0.;n=0;exited=False
47     for row in records:
48         if row['type']=='request':
49             requests[row['payload']]['request_id']=row
50         if row['type']!='response' or row['request_id'] in executed:
51             continue

```



```

52 req=requests[row['request_id']]; b=req['payload'];path=req['path'];r=row['response']
53 assert row['http_status']==200 and r['accepted'] is True
54 executed.add(row['request_id'])
55 n+=1
56 if path=='/exit':exited=True
57 if path in ('/measure','/clear'):
58     q=(b['position']['x'],b['position']['y']);ch=b['channel']
59     vt+=math.dist(p,q)/5
60     s=sources.get(ch);d=math.inf if s is None or ch in cleared else math.dist(q,(s['x'],s['y']))
61     if path=='/measure':
62         vt+=5+(ch!=channel);channel=ch
63         axis=None if s is None else s.get('direction_deg')
64         angle=0 if s is None else math.degrees(math.atan2(q[1]-s['y'],q[0]-s['x']))
65         visible=axis is None or abs((angle-axis+180)%360-180)<=90+1e-9 or d<1e-10
66         expected='no_signal' if s is None or ch in cleared or d>s['radius'] or not visible else 'near' if
        ↪ d<=5 else 'direction'
67         assert r['measure_result']==expected,(expected,r)
68         if expected=='direction':
69             true=math.degrees(math.atan2(s['y']-q[1],s['x']-q[0]))
70             assert abs((r['svd_deg']-true+180)%360-180)<=1.0050001
71         measured.setdefault(ch,[]).append(q)
72     else:
73         expected='success' if d<=20 else 'no_target_in_range'
74         assert r['clear_result']==expected
75         vt+=5 if d<=20 else 3
76         if d<=20:cleared.add(ch)
77     p=q
78     assert abs(vt-r['virtual_time_s'])<n*1e-6+1e-5
79 assert exited and result['status']=='complete'
80 assert cleared==set(sources), 'False completion against hidden truth'
81 assert cleared==set(result['cleared'])
82 cert=result['stop_certificate']
83 if cert['kind']=='count16':
84     assert len(cleared)==16
85 else:
86     points=np.array(cert['stations']);triangles=cert['triangles']; edges=Counter()
87     if cert['kind']=='dynamic_cell_halfplane_cover':
88         assert np.array_equal(points,check_cover(json.dumps(cert['dynamic_cover'],sort_keys=True)))
89     if cert['kind']=='cell_halfplane_cover':
90         contents=(Path(__file__).with_name('certificates')/'compact22.json').read_bytes()
91         assert hashlib.sha256(contents).hexdigest()==cert['cover_sha256']
92         assert np.array_equal(points,check_cover(contents))
93     contains_origin=False
94     assert len({tuple(sorted(t)) for t in triangles})==len(triangles)
95     for tri in triangles:
96         v=points[tri]
97         assert np.linalg.norm(v[:,None]-v[None,:],axis=2).max()<1000
98         signs=[np.linalg.det(np.stack([v[(i+1)%3]-v[i],-v[i]])) for i in range(3)]
99         contains_origin |= min(signs)>=-1e-6 or max(signs)<=1e-6
100         for i in range(3):edges[tuple(sorted((tri[i],tri[(i+1)%3])))]+=1
101     assert contains_origin or cert['kind'] in ('cell_halfplane_cover','dynamic_cell_halfplane_cover')
102     for (a,b),count in edges.items():
103         assert count<=2
104         if count==1:assert segment_distance(np.zeros(2),points[a],points[b])>=1800-1e-5
105     for ch in set(range(1,21))-cleared:
106         actual=set(measured.get(ch,[]))
107         assert all(tuple(p) in actual for p in points),'Unscanned absent channel'
108     return dict(passed=True,accepted_actions=n,cleared=len(cleared),virtual_time_s=vt)

```

程序 18

对应文件: paper_evidence/q4_mechanism_reset_20260912/checks.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py; paper_ev

idence/q4_mechanism_reset_20260912/round03_window/development/code_s
 napshot/paper_evidence/q4_mechanism_reset_20260912/checks.py; paper_
 evidence/q4_mechanism_reset_20260912/round04_search_first/developmen
 t/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py;
 paper_evidence/q4_mechanism_reset_20260912/round05_sector/developmen
 t/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py;
 paper_evidence/q4_mechanism_reset_20260912/round06_exchange/developmen
 t/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/developme
 nt/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py;
 paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/develop
 ment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/devel
 opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py;
 paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel
 opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/checks.py

```

1  """Post-policy Q4 audit additions: continuous cover and fixed-point readings.
2
3  Only the evaluator calls this module, after the policy has returned. No source
4  coordinates are passed back to the policy. This supplements the historical
5  physical auditor; it does not mutate records, results, or old evidence.
6  """
7  import hashlib
8  import json
9  import math
10 from functools import lru_cache
11 from pathlib import Path
12
13 import numpy as np
14 from scipy.spatial import ConvexHull, Delaunay
15
16
17 def _require(condition, message):
18     if not condition:
19         raise AssertionError(message)
20
21
22 def _point(value):
23     p = tuple(float(x) for x in value)
24     _require(len(p) == 2 and all(math.isfinite(x) for x in p), 'Invalid point')
25     # Python numeric tuple equality identifies signed zero and int/float forms.
26     return p
27
28
29 def _origin_segment_distance(a, b):
30     edge = b-a
31     length2 = float(edge @ edge)
32     _require(length2 > 0, 'Zero-length coverage edge')
33     t = min(1., max(0., float(-a @ edge)/length2))
34     return float(np.linalg.norm(a+t*edge))
35
36
37 def _triangle_intersects_domain(vertices):
38     edges = np.roll(vertices, -1, axis=0)-vertices
39     signed = edges[:, 0]*(-vertices[:, 1])-edges[:, 1]*(-vertices[:, 0])
40     inside = np.all(signed >= -1e-8) or np.all(signed <= 1e-8)
41     return inside or min(_origin_segment_distance(vertices[i], vertices[(i+1) % 3])
42                          for i in range(3)) <= 1800.+1e-7
43
44
45 @lru_cache(maxsize=64)
46 def _triangular_cover(serialized):

```

```

47     """Rebuild a real planar mesh instead of trusting abstract edge counts.
48
49     The full convex hull contains the target disk. Delaunay triangles partition
50     that hull; each triangle touching the disk must have diameter below 1000 m.
51     Triangles entirely outside the source disk may be larger (the historical
52     31-station lattice has six such exterior fill triangles).
53     """
54     cert = json.loads(serialized)
55     points = np.asarray(cert['stations'], dtype=float)
56     _require(points.ndim == 2 and points.shape[1] == 2 and len(points) >= 3,
57             'Malformed station array')
58     _require(np.isfinite(points).all(), 'Nonfinite station')
59     _require(len({tuple(p) for p in points}) == len(points),
60             'Duplicate/coincident coverage stations')
61     hull = ConvexHull(points)
62     # scipy hull equations point outward, so each inequality must contain
63     # the origin strictly; the nearest point on every polygon side must lie
64     # beyond the source disk. Both tests are necessary.
65     _require(np.max(hull.equations[:, -1]) < 0., 'Coverage hull misses origin')
66     boundary = points[hull.vertices]
67     min_boundary = min(_origin_segment_distance(boundary[i], boundary[(i+1) % len(boundary)])
68                       for i in range(len(boundary)))
69     _require(min_boundary >= 1800.-1e-7, 'Coverage hull does not contain source disk')
70     rebuilt = Delaunay(points)
71     rebuilt_keys = {tuple(sorted(map(int, tri))) for tri in rebuilt.simplices}
72     claimed = cert.get('triangles')
73     _require(isinstance(claimed, list) and len(claimed) > 0, 'Missing claimed triangles')
74     claimed_keys = set()
75     for tri in claimed:
76         _require(isinstance(tri, list) and len(tri) == 3 and
77                 all(isinstance(i, int) and not isinstance(i, bool) and 0 <= i < len(points)
78                     for i in tri), 'Malformed triangle indices')
79         _require(len(set(tri)) == 3, 'Repeated vertex in coverage triangle')
80         key = tuple(sorted(tri))
81         _require(key not in claimed_keys, 'Duplicate coverage triangle')
82         claimed_keys.add(key)
83         vertices = points[tri]
84         cross = np.cross(vertices[1]-vertices[0], vertices[2]-vertices[0])
85         _require(abs(float(cross)) > 1e-8, 'Degenerate coverage triangle')
86         _require(key in rebuilt_keys, 'Claimed topology differs from rebuilt planar Delaunay mesh')
87     relevant = 0
88     max_diameter = 0.
89     for tri in rebuilt.simplices:
90         vertices = points[tri]
91         if not _triangle_intersects_domain(vertices):
92             continue
93         relevant += 1
94         _require(tuple(sorted(map(int, tri))) in claimed_keys,
95                 'Claimed coverage omits a domain-intersecting triangle')
96         diameter = float(np.linalg.norm(vertices[:, None]-vertices[None, :], axis=2).max())
97         max_diameter = max(max_diameter, diameter)
98         _require(diameter < 1000., 'Domain-intersecting triangle exceeds guaranteed range')
99     _require(relevant > 0, 'No source-domain coverage triangles')
100     return dict(kind='rebuilt_planar_delaunay', stations=len(points),
101               relevant_triangles=relevant, full_hull_triangles=len(rebuilt.simplices),
102               min_hull_edge_distance_m=min_boundary,
103               max_relevant_triangle_diameter_m=max_diameter)
104
105
106 def check_additional(case, records, result, events):
107     """Return audit counts or raise for unsupported completion evidence.
108
109     `case` and `events` are accepted for the experiment runner's uniform audit
110     interface. This check needs only public action records and the stop result.
111     The historical physical/region auditor separately checks source truth.
112     """
113     del case, events
114     requests, executed = {}, set()
115     measured, readings, cleared = {}, {}, set()
116     repeat_directions = 0
117     entered = exited = False
118     for row in records:
119         if row.get('type') == 'request':

```

```

120         request_id = row['payload']['request_id']
121         if request_id in requests:
122             old = requests[request_id]
123             _require(old['path'] == row['path'] and old['payload'] == row['payload'],
124                     'Request ID reused for a different action')
125             requests[request_id] = row
126             continue
127         if row.get('type') != 'response':
128             continue
129         response = row['response']
130         if row.get('http_status') != 200 or response.get('accepted') is not True:
131             continue
132         request_id = row['request_id']
133         _require(request_id in requests, 'Accepted response without request evidence')
134         if request_id in executed:
135             continue
136         executed.add(request_id)
137         req = requests[request_id]
138         path, body = req['path'], req['payload']
139         _require(not exited, 'New accepted action after exit')
140         if path == '/enter':
141             _require(not entered, 'Second accepted enter')
142             entered = True
143         elif path == '/exit':
144             _require(entered, 'Exit before enter')
145             exited = True
146         elif path in ('/measure', '/clear'):
147             _require(entered, 'Accepted action before enter')
148             p = _point((body['position']['x'], body['position']['y']))
149             channel = int(body['channel'])
150             if path == '/clear':
151                 if response['clear_result'] == 'success':
152                     _require(channel not in cleared, 'Same source cleared twice')
153                     cleared.add(channel)
154                     continue
155             measured.setdefault(channel, set()).add(p)
156             if response['measure_result'] == 'direction':
157                 reading = float(response['svd_deg'])
158                 _require(math.isfinite(reading), 'Nonfinite bearing')
159                 key = (channel, p)
160                 if key in readings:
161                     difference = abs((reading-readings[key]+180.) % 360.-180.)
162                     _require(difference <= 1e-12,
163                             f'Same-point bearing changed: channel={channel}, point={p}')
164                     repeat_directions += 1
165                 readings[key] = reading
166             else:
167                 raise AssertionError('Unknown accepted action path')
168         _require(entered and exited and result.get('status') == 'complete',
169                 'No accepted enter/exit and complete result')
170         _require(set(result['cleared']) == cleared and len(result['cleared']) == len(cleared),
171                 'Cleared list differs from accepted successful actions')
172         cert = result['stop_certificate']
173         kind = cert.get('kind')
174         if kind == 'count16':
175             _require(len(cleared) == 16, 'Count16 stop without 16 successful distinct clears')
176             coverage = dict(kind='count16', cleared=16)
177             scanned_pairs = 0
178         else:
179             _require(kind in ('triangular_halfplane_cover', 'cell_halfplane_cover',
180                             'dynamic_cell_halfplane_cover'),
181                     'Unknown stop certificate kind')
182             points = [_point(p) for p in cert['stations']]
183             if kind == 'triangular_halfplane_cover':
184                 coverage = _triangular_cover(json.dumps(
185                     dict(stations=cert['stations'], triangles=cert['triangles']),
186                     sort_keys=True, separators=(',', ':'), allow_nan=False)).copy()
187             else:
188                 import question4.audit as legacy
189                 if kind == 'dynamic_cell_halfplane_cover':
190                     contents = json.dumps(cert['dynamic_cover'], sort_keys=True,
191                                           separators=(',', ':'), allow_nan=False).encode()
192                 _require(len(set(points)) == len(points), 'Duplicate dynamic stations')

```

```

193         else:
194             contents = (Path(legacy.__file__).with_name('certificates')/'compact22.json').read_bytes()
195             _require(hashlib.sha256(contents).hexdigest() == cert['cover_sha256'],
196                     'Compact continuous cover hash mismatch')
197             expected = legacy.check_cover(contents)
198             _require(np.array_equal(np.asarray(points), expected), 'Compact station list mismatch')
199             coverage = dict(kind='independent_legacy_continuous_cell_cover', stations=len(points),
200                             dynamic=kind == 'dynamic_cell_halfplane_cover',
201                             cover_sha256=hashlib.sha256(contents).hexdigest())
202         scanned_pairs = 0
203         for channel in set(range(1, 21))-cleared:
204             observed = measured.get(channel, set())
205             _require(all(p in observed for p in points),
206                     f'Coverage station omitted for uncleared channel {channel}')
207             scanned_pairs += len(points)
208     return dict(passed=True, stop=coverage,
209                 repeated_direction_observations_checked=repeat_directions,
210                 unique_direction_points=len(readings),
211                 actual_unresolved_channel_station_pairs_checked=scanned_pairs,
212                 accepted_actions_checked=len(executed))

```

程序 19

对应文件: paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage.py

```

1 """Experimental Q4 coverage policies. Strategies use public feedback only.
2
3 No fixtures, simulator state, true source count, direction or radius are inputs.
4 The continuous cell certificate is deterministic and based on public geometry.
5 """
6 import math
7 from functools import lru_cache

```

```

8 import numpy as np
9 from scipy.spatial import ConvexHull, Delaunay, QhullError
10 from scipy.optimize import minimize
11 from question4.strategy import Strategy as Baseline
12 from question4.geometry import estimate
13 from question4.routing import tour
14
15 BASELINE_CONFIG = dict(scheduling='route', mesh_kind='radial', scan_known='certified',
16                        centroid_route=True, known_count_stop=True, scan_information=True)
17 CANDIDATES = {
18     'COVER_EXCHANGE': {'mechanism': 'exchange', 'certificate': 'cells'},
19     'COVER_EXCHANGE_TRI': {'mechanism': 'exchange', 'certificate': 'triangles'},
20     'COVER_ELASTIC': {'mechanism': 'elastic'},
21     'COVER_SEARCH_FIRST': {'mechanism': 'search_first'},
22     'COVER_SECTOR': {'mechanism': 'sector'},
23     'COVER_EXCHANGE_ELASTIC': {'mechanism': 'exchange_elastic', 'certificate': 'cells'},
24 }
25
26
27 def triangle_certificate(points):
28     """Sufficient continuous cover: Delaunay mesh + enclosing convex hull."""
29     p = np.asarray(points, float)
30     try:
31         hull = ConvexHull(p)
32         if np.max(hull.equations[:, 2] + 1800) > -1e-6:
33             return None
34         triangles = Delaunay(p).simplices
35     except QhullError:
36         return None
37     v = p[triangles]
38     if np.linalg.norm(v[:, :, None, :] - v[:, None, :, :], axis=3).max() >= 1000-1e-6:
39         return None
40     return triangles
41
42
43 @lru_cache(maxsize=16384)
44 def _cell(box):
45     x0, y0, x1, y1 = box
46     corners = np.array([(x0,y0),(x1,y0),(x1,y1),(x0,y1)])
47     dx, dy = max(x0,0,-x1), max(y0,0,-y1)
48     return corners, dx*dx+dy*dy > 1800**2
49
50
51 def cell_certificate(points, depth_limit=10):
52     """Finite sufficient cover; returns None if proof budget is exhausted.
53
54     Every retained box is inside the convex hull of stations that are within
55     1000 m of all its corners. Convexity proves the property throughout the box.
56     Rejection says only this proof attempt failed, not that the cover is false.
57     """
58     p = np.asarray(points, float)
59     # The convex hull is a necessary condition and a cheap early rejection.
60     try:
61         if np.max(ConvexHull(p).equations[:,2]+1800) > -1e-6:
62             return None
63     except QhullError:
64         return None
65     leaves=[]; stack=[('',(-1800.,-1800.,1800.,1800.))]
66     while stack:
67         key, box=stack.pop(); corners, outside=_cell(box)
68         if outside:
69             leaves.append({'key':key,'outside':True});continue
70         ids=np.flatnonzero(np.linalg.norm(p[:,None]-corners[None,:],axis=2).max(axis=1)<1000-1e-6)
71         if len(ids)>=3:
72             try:
73                 hull=ConvexHull(p[ids]); h=hull.equations
74                 if np.max(corners@h[:, :2].T+h[:,2])<=1e-8:
75                     leaves.append({'key':key,'witnesses':ids[hull.vertices].tolist()});continue
76             except QhullError:
77                 pass
78         if len(key)==depth_limit:
79             return None
80     x0,y0,x1,y1=box;xm=(x0+x1)/2;ym=(y0+y1)/2

```

```

81     children=((x0,y0,xm,ym),(xm,y0,x1,ym),(x0,ym,xm,y1),(xm,ym,x1,y1))
82     stack.extend((key+str(i), child) for i,child in enumerate(children))
83     return {'stations':p.tolist(),'leaves':leaves,'failed_cells':[]}
84
85
86 class Strategy(Baseline):
87     def __init__(self, client, config, log=lambda row:None):
88         allowed={'mechanism','certificate','baseline_config'}
89         if set(config)-allowed:
90             raise ValueError('Unknown coverage config: '+str(set(config)-allowed))
91         self.options={'mechanism':'exchange','certificate':'cells'}|config
92         if self.options['mechanism'] not in {'exchange','elastic','search_first','sector','exchange_elastic'}:
93             raise ValueError('Unknown coverage mechanism')
94         if self.options['certificate'] not in {'cells','triangles'}:
95             raise ValueError('Unknown certificate')
96         super().__init__(client, config.get('baseline_config',BASELINE_CONFIG),log)
97         if self.cfg['mesh_kind']!='radial':
98             raise ValueError('Coverage experiment requires the frozen 25-station parent')
99         self.stations=self.stations.copy();self.triangles=self.triangles.copy()
100        self._in_exchange=False;self._elastic_prepared=set();self._cell_cover=None
101        self._exchange_checks=set()
102
103    def _scan(self,i,p):
104        channels=[ch for ch in range(1,21) if ch not in self.cleared]
105        if self.c.channel in channels:
106            channels.remove(self.c.channel);channels.insert(0,self.c.channel)
107        for ch in channels:
108            if ch in self.known:
109                radius=estimate(self.known[ch]['poly'])[1]
110                if self.cfg['scan_known']=='none' or (self.cfg['scan_known']=='certified' and radius<20-1e-5):
111                    continue
112                if self.cfg['scan_information']:
113                    poly=self.known[ch]['poly'];center,_=estimate(poly);delta=poly-p
114                    ref=math.atan2(center[1]-p[1],center[0]-p[0])
115                    angles=(np.arctan2(delta[:1],delta[:0])-ref+math.pi)%(2*math.pi)-math.pi
116                    if np.ptp(angles)<math.radians(3):continue
117                self.observe(p,ch,'search');self.seen.add((i,ch))
118            self.done.add(i)
119
120    def _exchange_here(self):
121        if self._in_exchange or len(self.cleared)+len(self.known)==16:
122            return
123        p=np.asarray(self.c.position,float)
124        remaining=sorted((i for i in range(len(self.stations)) if i not in self.done),
125                        key=lambda i:(math.dist(p,self.stations[i]),i))
126        # Spatial locality is a fixed proof-computation budget, not a claim of
127        # invisible-source absence. The original pending station is the fallback.
128        for i in remaining[:3]:
129            if math.dist(p,self.stations[i])>300:continue
130            key=(i,tuple(p),tuple(self.stations.ravel()))
131            if key in self._exchange_checks:continue
132            self._exchange_checks.add(key)
133            proposal=self.stations.copy();proposal[i]=p
134            triangles=triangle_certificate(proposal);cover=None
135            if triangles is None and self.options['certificate']=='cells':
136                cover=cell_certificate(proposal)
137            if triangles is None and cover is None:continue
138            old=self.stations[i].copy();self.stations[i]=p
139            self.triangles=triangles if triangles is not None else np.empty((0,3),dtype=int)
140            self._cell_cover=cover
141            self.log({'event':'coverage_exchange','station':i,'old':old.tolist(),'point':p.tolist(),
142                    'certificate':'cells' if cover is not None else 'triangles',
143                    'avoided_station_offset_m':math.dist(old,p)})
144            self._in_exchange=True
145            try:self._scan(i,p)
146            finally:self._in_exchange=False
147            return
148
149    def clear(self,p,ch,phase):
150        ok=super().clear(p,ch,phase)
151        if ok and phase!='near' and self.options['mechanism'] in {'exchange','exchange_elastic'}:
152            self._exchange_here()
153        return ok

```

```

154
155 def _elastic(self,i):
156     """Move an unvisited interior station within a certified mesh cell.
157
158     The objective accounts for the incoming leg and one public nearest task.
159     Feasibility preserves old oriented faces and all incident edge bounds;
160     an independent Delaunay+outer hull check is still required before commit.
161     """
162     if i in self.done or i==0 or i>=13:return
163     p=self.stations[i].copy();start=np.asarray(self.c.position)
164     other=[self.stations[j] for j in range(len(self.stations)) if j not in self.done and j!=i]
165     other += [self.target(data) for data in self.known.values()]
166     if not other:return
167     nxt=min(other,key=lambda q:math.dist(p,q))
168     triangles=triangle_certificate(self.stations)
169     if triangles is None:return
170     incident=[t for t in triangles if i in t]
171     neighbors=sorted([int(j) for t in incident for j in t if j!=i])
172     def cost(q):return float(np.linalg.norm(q-start)+np.linalg.norm(q-nxt))
173     def feasible(q):
174         vals=[(1000-1e-4)**2-float(np.sum((q-self.stations[j])**2)) for j in neighbors]
175         for tri in incident:
176             old=self.stations[tri];v=old.copy();v[list(tri).index(i)]=q
177             sign=np.sign(np.cross(old[1]-old[0],old[2]-old[0]))
178             vals.append(float(sign*np.cross(v[1]-v[0],v[2]-v[0]))-1e-4)
179         return np.array(vals)
180     opt=minimize(cost,p,method='SLSQP',constraints={'type':'ineq','fun':feasible},
181                 options={'maxiter':35,'ftol':1e-5})
182     if not opt.success or cost(opt.x)>=cost(p)-1e-3:return
183     proposal=self.stations.copy();proposal[i]=opt.x;cert=triangle_certificate(proposal)
184     if cert is None:return
185     self.stations[i]=opt.x;self.triangles=cert;self._cell_cover=None
186     self.log({'event':'coverage_elastic','station':i,'old':p.tolist(),'point':opt.x.tolist(),
187             'local_path_saving_m':cost(p)-cost(opt.x)})
188
189 def observe(self,p,ch,phase):
190     if phase=='search' and self.options['mechanism'] in {'elastic','exchange_elastic'} and not self._in_exchange:
191         # Baseline p is a view of stations[i], hence the one committed move
192         # also updates all channels in that same original scanning batch.
193         ids=np.flatnonzero(np.linalg.norm(self.stations-p,axis=1)<1e-8)
194         if len(ids)==1 and int(ids[0]) not in self._elastic_prepared:
195             i=int(ids[0]);self._elastic_prepared.add(i);self._elastic(i)
196     return super().observe(p,ch,phase)
197
198 def _finish(self):
199     certified=len(self.cleared)==16 or (len(self.done)==len(self.stations) and not self.known)
200     if not certified:raise RuntimeError('Missing stop certificate')
201     self.c.exit()
202     return dict(status='complete',cleared=sorted(self.cleared),metrics=self.c.metrics(),
203               stop_certificate=dict(kind='count16' if len(self.cleared)==16 else 'triangular_halfplane_cover',
204               cover_sha256=None,spacing=self.cfg['spacing'],mesh_kind='radial',
205               stations=self.stations.tolist(),triangles=self.triangles.tolist(),
206               completed_stations=sorted(self.done),scanned_pairs=len(self.seen)))
207
208 def _survey_run(self,sector=False):
209     self.c.enter();active_sector=0
210     # Four fixed 90-degree sectors define committed contiguous work areas.
211     # This is scheduling only; all 25 original witness stations remain.
212     def region(p):return int(((math.atan2(p[1],p[0])+2*math.pi)%(2*math.pi))/(math.pi/2))
213     while len(self.done)<len(self.stations) or self.known:
214         if len(self.cleared)==16:break
215         remaining=[i for i in range(len(self.stations)) if i not in self.done]
216         if len(self.cleared)+len(self.known)==16:remaining=[]
217         if sector and remaining:
218             local=[i for i in remaining if region(self.stations[i])==active_sector]
219             known=[ch for ch,data in self.known.items() if region(self.target(data))==active_sector]
220             if not local and not known:
221                 active_sector=(active_sector+1)%4;continue
222         else:local=remaining;known=list(self.known)
223         # Search-first delays every source until the witness sweep ends.
224         tasks=[('station',i,self.stations[i]) for i in local]
225         if sector or not remaining:
226             tasks += [('source',ch,self.target(self.known[ch])) for ch in known]

```



```

227         if not tasks:
228             # Sources whose estimate crossed a completed sector are handled
229             # once all stations finish; no source is dropped from the queue.
230             if self.known:tasks=[('source',ch,self.target(data)) for ch,data in self.known.items()]
231             else:break
232             order=tour(self.c.position,[t[2] for t in tasks]);kind,key,p=tasks[order[0]]
233             if kind=='source':self.localize(key)
234             else:self._scan(key,p)
235         return self._finish()
236
237     def run(self):
238         mechanism=self.options['mechanism']
239         result=self._survey_run(mechanism=='sector') if mechanism in {'search_first','sector'} else super().run()
240         if result['stop_certificate']['kind']!='count16' and self._cell_cover is not None:
241             result['stop_certificate']['kind']='dynamic_cell_halfplane_cover'
242             result['stop_certificate']['dynamic_cover']=self._cell_cover
243         return result
244
245
246 def run(client,config=None,log=lambda row:None):
247     return Strategy(client,config or {},log).run()

```

程序 20

对应文件: paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/coverage_smoke.py

```

1 """SELF-only bounded mechanism smoke and continuous-certificate regression checks."""
2 import copy
3 import json
4 from pathlib import Path
5 import numpy as np
6 from paper_evidence.q4_mechanism_reset_20260912.coverage import run,CANDIDATES,triangle_certificate,cell_certificate

```

```

7 from question4.geometry import radial_mesh
8 from question4.cases import fixtures
9 from question4.simulator import Simulator
10 from question3.interface import Client
11 from question4.audit import audit,check_cover
12
13 ROOT=Path(__file__).parent
14
15 def main():
16     p,_=radial_mesh();assert triangle_certificate(p) is not None
17     assert cell_certificate(p) is not None
18     for i in (0,1,13):
19         assert triangle_certificate(np.delete(p,i,axis=0)) is None
20         assert cell_certificate(np.delete(p,i,axis=0)) is None
21     assert triangle_certificate(np.array([[1800,0],[1800,0],[1800,0]])) is None
22     proof=cell_certificate(p);check_cover(json.dumps(proof))
23     bad=copy.deepcopy(proof);bad['leaves'].pop()
24     try:check_cover(json.dumps(bad))
25     except AssertionError:pass
26     else:raise AssertionError('Missing cell accepted')
27     rows=[]
28     for case in fixtures(918200,6):
29         for name,config in CANDIDATES.items():
30             records=[];events=[];env=Simulator(case);client=Client(env,'SELF',records.append)
31             result=run(client,config,events.append);truth=env.evaluation()
32             assert truth['total']==truth['cleared']
33             cert=result['stop_certificate']
34             if cert['kind']=='dynamic_cell_halfplane_cover':
35                 # Root evaluator additionally performs full physical replay.
36                 assert np.array_equal(check_cover(json.dumps(cert['dynamic_cover'])),cert['stations'])
37                 requests={r['payload']['request_id']:r for r in records if r['type']=='request'}
38                 measured={ch:set() for ch in range(1,21)}
39                 for r in records:
40                     if r['type']=='response' and r['path']=='/measure':
41                         b=requests[r['request_id']]['payload'];measured[b['channel']].add((b['position']['x'],b['position']['y']))
42                 for ch in set(range(1,21))-set(result['cleared']):
43                     assert all(tuple(q) in measured[ch] for q in cert['stations'])
44             else:audit(case,records,result)
45             row=dict(case=case['id'],name=name,time_s=client.virtual,cleared=truth['cleared'],
46                     certificate=cert['kind'],exchanges=sum(e['event']=='coverage_exchange' for e in events),
47                     elastic=sum(e['event']=='coverage_elastic' for e in events),status='complete')
48             rows.append(row);print(row,flush=True)
49     (ROOT/'coverage_smoke.json').write_text(json.dumps({'geometry_checks':'passed','rows':rows},indent=2))
50
51 if __name__=='__main__':main()

```

程序 21

对应文件: paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/

code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/c
 ode_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py; pa
 per_evidence/q4_mechanism_reset_20260912/round08_optical_cells/developme
 nt/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/devel
 opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/experime
 nt.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region
 /development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/ex
 periment.py

```

1  """Frozen, paired Q4 SELF experiments; never invokes an HTTP/official environment.
2
3  From the project root::
4
5      python -m question4.innovation.experiment --configs path/to/configs.json \
6          --output question4/innovation/results/dev --seed 940000 --count 24 \
7          --error-modes spatial_hash,positive,negative,smooth \
8          --roundings bounded,pre_round_stress --workers 4 --stage development
9
10     python -m question4.innovation.experiment --configs path/to/selected.json \
11         --selection path/to/selection.json --output path/to/locked_dev \
12         --seed 950000 --count 6 --stage selection_lock
13
14     python -m question4.innovation.experiment --configs path/to/selected.json \
15         --selection path/to/selection.json --output path/to/validation \
16         --seed 960000 --count 96 --lock path/to/locked_dev/lock.json \
17         --stage independent_validation --workers 4
18
19  Reproduction: replace --seed/--count with --fixtures path/to/fixtures.json and
20  use --stage reproduction. Fixtures are always labelled non-independent.
21  --count counts physical layouts, not layout x error-law x rounding runs.
22  Source/configuration/selection locks and code snapshots precede case generation.
23  """
24  import argparse
25  import copy
26  import hashlib
27  import json
28  import platform
29  import re
30  import shutil
31  import sys
32  import time
33  import traceback
34  from collections import Counter
35  from concurrent.futures import ProcessPoolExecutor, as_completed
36  from pathlib import Path
37
38  import numpy as np
39  import scipy
40
41  from question3.interface import Client
42  from .audit import audit
43  from question4.cases import make_case
44  from question4.simulator import Simulator
45  from question4.innovation.analysis import analyze
46
47
48  PROJECT = Path(__file__).resolve().parents[2]
49  INNOVATION = PROJECT / 'question4' / 'innovation'
50  ROOT = Path(__file__).resolve().parent
51  FAMILIES = ('uniform', 'outward', 'inward', 'tangent', 'cluster', 'edge')
52  ERROR_MODES = ('spatial_hash', 'positive', 'negative', 'smooth', 'zero')
53  ROUNDINGS = ('bounded', 'pre_round_stress')
54  FAILURE_PENALTY_S = 360000.
55
56

```

```

57 class PublicClient:
58     """Policy capability: public actions, observations and accounting only."""
59     __slots__ = ('__client',)
60
61     def __init__(self, client):
62         self.__client = client
63
64     @property
65     def position(self):
66         return self.__client.position
67
68     @property
69     def channel(self):
70         return self.__client.channel
71
72     def enter(self):
73         return self.__client.enter()
74
75     def measure(self, *args, **kwargs):
76         return self.__client.measure(*args, **kwargs)
77
78     def clear(self, *args, **kwargs):
79         return self.__client.clear(*args, **kwargs)
80
81     def exit(self):
82         return self.__client.exit()
83
84     def metrics(self):
85         return self.__client.metrics()
86
87
88 def canonical_sha256(value):
89     return hashlib.sha256(json.dumps(value, sort_keys=True, ensure_ascii=False,
90                                     separators=(',', ':'), allow_nan=False).encode()).hexdigest()
91
92
93 def write_json(path, data):
94     path = Path(path)
95     path.parent.mkdir(parents=True, exist_ok=True)
96     temporary = path.with_name(path.name + '.writing')
97     temporary.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False) + '\n', encoding='utf-8')
98     temporary.replace(path)
99
100
101 def source_hashes():
102     paths = set((PROJECT / 'question4').glob('*.py'))
103     paths.update((PROJECT / 'question4' / 'certificates').glob('*.json'))
104     paths.update(INNOVATION.glob('*.py'))
105     paths.update(ROOT.glob('*.py'))
106     paths.update((ROOT / 'frozen').rglob('*.py'))
107     paths.update((ROOT / 'frozen').rglob('*.json'))
108     paths.add(PROJECT / 'question3' / 'interface.py')
109     return {str(path.relative_to(PROJECT)): hashlib.sha256(path.read_bytes()).hexdigest()
110             for path in sorted(paths)}
111
112
113 def _csv(value, allowed):
114     result = [part.strip() for part in value.split(',') if part.strip()]
115     if not result or len(result) != len(set(result)) or any(part not in allowed for part in result):
116         raise argparse.ArgumentTypeError(f'Expected distinct comma-separated values from {"", ".join(allowed)}')
117     return result
118
119
120 def _layout_identity(case):
121     # Source parameters, not seed or error law, define a physical layout block.
122     sources = sorted(case['sources'], key=lambda source: source['channel'])
123     return canonical_sha256(sources)
124
125
126 def _load_layouts(args):
127     if args.fixtures:
128         loaded = json.loads(Path(args.fixtures).read_text(encoding='utf-8'))
129         if isinstance(loaded, dict):

```

```

130         loaded = loaded.get('layouts', loaded.get('cases'))
131     if not isinstance(loaded, list) or not loaded:
132         raise ValueError('Fixtures must contain a nonempty list of case/layout objects')
133     seen = set()
134     layouts = []
135     for case in loaded:
136         identity = _layout_identity(case)
137         if identity not in seen:
138             case = copy.deepcopy(case)
139             case['layout_sha256'] = identity
140             case['layout_id'] = f'{case.get("family", "fixture")}_{identity[:16]}'
141             layouts.append(case)
142             seen.add(identity)
143     return layouts
144     return [make_case(args.seed + index, FAMILIES[index % len(FAMILIES)]) for index in range(args.count)]
145
146
147 def _expand_cases(layouts, error_modes, roundings):
148     cases = []
149     seen = set()
150     for layout in layouts:
151         digest = _layout_identity(layout)
152         if digest in seen:
153             raise ValueError('Duplicate physical layout in generated dataset')
154         seen.add(digest)
155         layout_id = f'{layout.get("family", "fixture")}_{digest[:16]}'
156         for error_mode in error_modes:
157             for rounding in roundings:
158                 case = copy.deepcopy(layout)
159                 case.update(layout_id=layout_id, layout_sha256=digest, error_mode=error_mode,
160                             rounding=rounding, id=f'{layout_id}_{error_mode}_{rounding}')
161                 cases.append(case)
162     return cases
163
164
165 def _run_case(task):
166     case, configs, output, stage, reference = task
167     from paper_evidence.q4_mechanism_reset_20260912.strategy import run
168
169     rows = []
170     names = list(configs)
171     offset = int(case['layout_sha256'][:8], 16) % len(names)
172     names = names[offset:] + names[:offset]
173     for name in names:
174         folder = Path(output) / name / case['id']
175         folder.mkdir(parents=True, exist_ok=False)
176         records, events = [], []
177         env = Simulator(case, rounding=case['rounding'])
178         client = Client(env, 'SELF', records.append)
179         started = time.perf_counter()
180         exception_trace = None
181         try:
182             result = run(PublicClient(client), copy.deepcopy(configs[name]), events.append)
183             if not isinstance(result, dict):
184                 raise TypeError('Strategy must return a dictionary')
185         except Exception as exc:
186             exception_trace = traceback.format_exc()
187             result = dict(status='incomplete', failure=f'{type(exc).__name__}: {exc}', metrics=client.metrics())
188         elapsed = time.perf_counter() - started
189         # This is the first evaluator truth access. Neither case nor env goes to run().
190         truth = env.evaluation()
191         try:
192             checked = audit(case, records, result)
193         except Exception as exc:
194             checked = dict(passed=False, failure=f'{type(exc).__name__}: {exc}', traceback=traceback.format_exc())
195         try:
196             from question4.innovation.audit import audit_regions
197             checked['regions'] = audit_regions(case, events)
198             from .checks import check_additional
199             checked['additional'] = check_additional(case, records, result, events)
200             from .optical_audit import check_cells
201             checked['optical_cells'] = check_cells(case, events)
202             checked['passed'] = bool(checked.get('passed')) and bool(checked['regions'].get('passed'))

```

```

203     except Exception as exc:
204         checked['regions'] = dict(passed=False, failure=f'{type(exc).__name__}: {exc}',
205                                   traceback=traceback.format_exc())
206         checked['passed'] = False
207     status = result.get('status', 'incomplete')
208     false_complete = status == 'complete' and truth['cleared'] != truth['total']
209     success = status == 'complete' and truth['cleared'] == truth['total'] and bool(checked.get('passed'))
210     virtual_s = max(float(truth['virtual_time_s']), float(client.metrics()['virtual_time_s']))
211     row = dict(case_id=case['id'], layout_id=case['layout_id'], layout_sha256=case['layout_sha256'],
212               seed=case.get('seed'), family=case.get('family', 'fixture'),
213               error_mode=case['error_mode'], rounding=case['rounding'], stage=stage,
214               config=name, reference=reference, status=status, success=success,
215               false_complete=false_complete, cleared=truth['cleared'], total=truth['total'],
216               metrics=client.metrics(), elapsed_s=elapsed, audit=checked,
217               failure=result.get('failure'), exception_trace=exception_trace,
218               timeout=bool(result.get('failure') and any(s in result['failure'].lower() for s in ('deadline',
219               ↪ 'timeout', 'expired'))),
219               penalized_virtual_s=virtual_s if success else max(virtual_s, FAILURE_PENALTY_S),
220               event_counts=dict(Counter(event.get('event'), event.get('type', 'untyped')) for event in events)))
221     write_json(folder / 'result.json', dict(summary=row, strategy=result, evaluation=truth))
222     write_json(folder / 'evaluator_hidden_case.json', case)
223     for filename, items in (('actions.jsonl', records), ('strategy.jsonl', events)):
224         with (folder / filename).open('x', encoding='utf-8') as handle:
225             for item in items:
226                 handle.write(json.dumps(item, ensure_ascii=False, allow_nan=False) + '\n')
227     rows.append(row)
228     print(f'RUN {case["id"]} {name} virtual_s={virtual_s:.3f} penalized_s={row["penalized_virtual_s"]:.3f} '
229           f'complete={truth["cleared"]} / {truth["total"]} audit={bool(checked.get("passed"))} '
230           ↪ wall_s={elapsed:.3f}', flush=True)
231
232
233 def main():
234     parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
235     parser.add_argument('--configs', required=True, help='JSON object mapping configuration names to dictionaries')
236     parser.add_argument('--output', required=True, help='New output directory; existing outputs are refused')
237     parser.add_argument('--seed', type=int, default=940000)
238     parser.add_argument('--count', type=int, default=24, help='Independent physical layouts, cycling six families')
239     parser.add_argument('--error-modes', default='spatial_hash,positive,negative,smooth')
240     parser.add_argument('--roundings', default='bounded,pre_round_stress')
241     parser.add_argument('--workers', type=int, default=1)
242     parser.add_argument('--fixtures', help='Previously generated fixtures: reproduction, never independent '
243     ↪ 'validation')
244     parser.add_argument('--stage', default='development')
245     parser.add_argument('--lock', help='Earlier lock.json or its experiment directory; verifies immutable '
246     ↪ 'source/config/selection')
247     parser.add_argument('--selection', help='Decision/selection JSON or text to persist and hash before generation')
248     parser.add_argument('--reference', help='Defaults to BASE if present, otherwise first configuration')
249     args = parser.parse_args()
250     if not __debug__:
251         parser.error('Python optimization disables assertions in the physical auditor; run without -O')
252     if args.count < 1 or args.workers < 1:
253         parser.error('--count and --workers must be positive')
254     try:
255         error_modes = _csv(args.error_modes, ERROR_MODES)
256         roundings = _csv(args.roundings, ROUNDINGS)
257     except argparse.ArgumentTypeError as exc:
258         parser.error(str(exc))
259     output = Path(args.output).resolve()
260     if output.exists():
261         parser.error(f'Output already exists: {output}')
262     configs = json.loads(Path(args.configs).read_text(encoding='utf-8'))
263     if not isinstance(configs, dict) or not configs or any(not isinstance(value, dict) for value in configs.values()):
264         parser.error('Configuration JSON must be a nonempty object mapping names to dictionaries')
265     if any(not re.fullmatch(r'[A-Za-z0-9][A-Za-z0-9_-]*', name) for name in configs):
266         parser.error('Configuration names must be safe directory names: letters, numbers, _, ., -')
267     reference = args.reference or ('BASE' if 'BASE' in configs else next(iter(configs)))
268     if reference not in configs:
269         parser.error('Reference configuration is absent from --configs')
270     is_validation = 'validation' in args.stage.lower() and not any(
271         word in args.stage.lower() for word in ('reproduction', 'not_independent', 'not_new'))
272     if is_validation and not args.lock:
273         parser.error('Independent validation requires --lock from an earlier frozen selection')

```

```

272 if is_validation and args.fixtures:
273     parser.error('Existing fixtures are reproduction; use --stage reproduction_not_independent')
274 hashes = source_hashes()
275 selection_bytes = (Path(args.selection).read_bytes() if args.selection else
276                    json.dumps(dict(configurations=list(configs), reference=reference,
277                                   declaration='Configuration set frozen before layout generation'),
278                               sort_keys=True).encode())
279 selection_hash = hashlib.sha256(selection_bytes).hexdigest()
280 previous = None
281 lock_path = None
282 if args.lock:
283     lock_path = Path(args.lock).resolve()
284     if lock_path.is_dir():
285         lock_path /= 'lock.json'
286     previous = json.loads(lock_path.read_text(encoding='utf-8'))
287     if is_validation and (args.seed != previous['generator_seed'] or args.count != previous['generator_count'] or
288 ↪ error_modes != previous['error_modes'] or roundings != previous['roundings']):
289         parser.error('Validation plan differs from pre-generation lock')
290     if previous['code_sha256'] != hashes:
291         changed = sorted(key for key in set(previous['code_sha256']) | set(hashes)
292                          if previous['code_sha256'].get(key) != hashes.get(key))
293         parser.error('Source changed after lock: ' + ', '.join(changed))
294     if any(previous['configs'].get(name) != config for name, config in configs.items()):
295         parser.error('Configuration changed after lock')
296     if args.selection and previous['selection_sha256'] != selection_hash:
297         parser.error('Selection decision changed after lock')
298     if not args.selection:
299         selection_bytes = (lock_path.parent / 'selection_snapshot.txt').read_bytes()
300         selection_hash = hashlib.sha256(selection_bytes).hexdigest()
301         if selection_hash != previous['selection_sha256']:
302             parser.error('Previous selection snapshot does not match its lock')
303     if previous.get('reference') != reference:
304         parser.error('Reference configuration changed after lock')
305 lock = dict(schema_version=1, stage=args.stage, created_unix=time.time(),
306            code_sha256=hashes, code_bundle_sha256=canonical_sha256(hashes),
307            configs=configs, configs_sha256=canonical_sha256(configs),
308            selection_sha256=selection_hash, reference=reference,
309            prior_lock=str(lock_path) if lock_path else None,
310            prior_lock_sha256=hashlib.sha256(lock_path.read_bytes()).hexdigest() if lock_path else None,
311            generator_seed=None if args.fixtures else args.seed,
312            generator_count=None if args.fixtures else args.count,
313            families=list(FAMILIES), error_modes=error_modes, roundings=roundings,
314            fixture_source=str(Path(args.fixtures).resolve()) if args.fixtures else None,
315            statistical_unit='physical layout', failure_penalty_s=FAILURE_PENALTY_S)
316 output.mkdir(parents=True, exist_ok=False)
317 write_json(output / 'configs.json', configs)
318 (output / 'selection_snapshot.txt').write_bytes(selection_bytes)
319 write_json(output / 'lock.json', lock)
320 for name, digest in hashes.items():
321     destination = output / 'code_snapshot' / name
322     destination.parent.mkdir(parents=True, exist_ok=True)
323     shutil.copy2(PROJECT / name, destination)
324     if hashlib.sha256(destination.read_bytes()).hexdigest() != digest:
325         raise RuntimeError('Source changed during snapshot: ' + name)
326 write_json(output / 'snapshot_complete.json', dict(completed_unix=time.time(),
327 ↪ code_bundle_sha256=lock['code_bundle_sha256']))
328 # No case generation or fixture parsing occurs before the persisted lock/snapshot.
329 layouts = _load_layouts(args)
330 cases = _expand_cases(layouts, error_modes, roundings)
331 identities = {case['layout_sha256'] for case in cases}
332 if is_validation and lock_path:
333     old_fixtures = lock_path.parent / 'fixtures.json'
334     if old_fixtures.exists():
335         old_cases = json.loads(old_fixtures.read_text(encoding='utf-8'))
336         overlap = identities & {_layout_identity(case) for case in old_cases}
337         if overlap:
338             raise RuntimeError(f'Independent validation reuses {len(overlap)} layouts from its lock batch')
339 write_json(output / 'fixtures.json', cases)
340 provenance = dict(stage=args.stage, independent_new_layouts=not bool(args.fixtures),
341                  independent_validation=bool(is_validation and not args.fixtures),
342                  reuse_label='reproduction_not_independent' if args.fixtures else 'new_generated_layouts',
343                  fixture_source=lock['fixture_source'], layouts=len(identities), scenarios=len(cases),
344                  algorithm_runs=len(cases) * len(configs), error_modes=error_modes, roundings=roundings,

```

```

343         family_layout_counts=dict(Counter(layout.get('family', 'fixture') for layout in layouts)),
344         layout_sha256=sorted(identities), fixtures_sha256=canonical_sha256(cases),
345         generated_after_snapshot_unix=time.time(), official_runs=0, http_runs=0,
346         lock_sha256=hashlib.sha256((output / 'lock.json').read_bytes()).hexdigest())
347 write_json(output / 'metadata.json', provenance)
348 print(f'START stage={args.stage} layouts={len(identities)} scenarios={len(cases)} '
349       f'configs={len(configs)} runs={provenance["algorithm_runs"]} workers={args.workers}', flush=True)
350 rows = []
351 started = time.perf_counter()
352 environment = dict(python=sys.version, numpy=np.__version__, scipy=scipy.__version__,
353 ↪ platform=platform.platform())
354
355 def persist(completed, invalid_reason=None):
356     ordered = sorted(rows, key=lambda row: (row['case_id'], list(configs).index(row['config'])))
357     write_json(output / 'summary.json', dict(stage=args.stage, provenance=provenance, environment=environment,
358     elapsed_s=time.perf_counter() - started,
359     completed_scenarios=completed, expected_scenarios=len(cases),
360     batch_invalid_reason=invalid_reason,
361     rows=ordered))
362     print(f'PROGRESS scenarios={completed}/{len(cases)} runs={len(rows)}/{provenance["algorithm_runs"]} '
363           f'failed={sum(not row["success"] for row in rows)} elapsed_s={time.perf_counter() - started:.1f}',
364           ↪ flush=True)
365
366 tasks = [(case, configs, str(output), args.stage, reference) for case in cases]
367 pool, pending, completed, active_case = None, {}, 0, None
368 try:
369     if args.workers == 1:
370         for task in tasks:
371             active_case = task[0]['id']
372             rows.extend(_run_case(task))
373             completed += 1
374             persist(completed)
375     else:
376         pool = ProcessPoolExecutor(max_workers=args.workers)
377         pending = {pool.submit(_run_case, task): task[0]['id'] for task in tasks}
378         for future in as_completed(pending):
379             active_case = pending[future]
380             rows.extend(future.result())
381             completed += 1
382             persist(completed)
383     active_case = None
384     stable_sources = source_hashes() == hashes
385     write_json(output / 'integrity.json', dict(source_unchanged_during_run=stable_sources,
386     finished_unix=time.time()))
387
388     if not stable_sources:
389         raise RuntimeError('Experimental source changed during execution; batch is not a locked result')
390     report = analyze(rows, reference=reference)
391     report['provenance'] = provenance
392     for name, stats in report['configurations'].items():
393         stats['timeouts'] = sum(bool(r.get('timeout')) for r in rows if r['config']==name)
394     write_json(output / 'comparison.json', report)
395     print(json.dumps({name: {key: value[key] for key in ('complete', 'runs', 'mean_s', 'p95_s', 'max_s',
396     'gain_ratio_of_means_pct')}}
397           for name, value in report['configurations'].items()), indent=2), flush=True)
398 except BaseException as exc:
399     failure = dict(batch_valid=False, kind='framework_failure', failure=f'{type(exc).__name__}: {exc}',
400     traceback=traceback.format_exc(), active_case_id=active_case,
401     completed_scenarios=completed, expected_scenarios=len(cases),
402     retained_summary_runs=len(rows), expected_runs=provenance['algorithm_runs'],
403     failed_unix=time.time(),
404     interpretation='Batch invalid; absent runs are not fabricated policy performance
405     ↪ observations.')
406     write_json(output / 'batch_failure.json', failure)
407     persist(completed, invalid_reason=failure['failure'])
408     for future in pending:
409         future.cancel()
410     print('BATCH INVALID: ' + failure['failure'], flush=True)
411     raise
412 finally:
413     if pool is not None:
414         pool.shutdown(wait=True, cancel_futures=True)
415 if any(not row['success'] for row in rows):
416     raise SystemExit(1)

```



```
413
414
415 if __name__ == '__main__':
416     main()
```

程序 22

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question3/interface.py;

paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question3/interface.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question3/interface.py; question3/innovation/frozen/question3/interface.py; question3/interface.py; question3/results/dispatch_20260913/development/code_snapshot/question3/interface.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/interface.py; question3/results/round13_adverse_main/code_snapshot/question3/interface.py; question3/results/round13_adverse_rounding/code_snapshot/question3/interface.py; question3/results/round13_dev_main/code_snapshot/question3/interface.py; question3/results/round13_dev_rounding/code_snapshot/question3/interface.py; question3/results/round13_holdout_main/code_snapshot/question3/interface.py; question3/results/round13_holdout_rounding/code_snapshot/question3/interface.py; question3/results/round13_outer_main/code_snapshot/question3/interface.py; question3/results/round13_outer_rounding/code_snapshot/question3/interface.py; question3/results/round14_adverse_main/code_snapshot/question3/interface.py; question3/results/round14_adverse_rounding/code_snapshot/question3/interface.py; question3/results/round14_dev_main/code_snapshot/question3/interface.py; question3/results/round14_dev_rounding/code_snapshot/question3/interface.py; question3/results/round14_holdout_main/code_snapshot/question3/interface.py; question3/results/round14_holdout_rounding/code_snapshot/question3/interface.py; question3/results/round14_initial_dev_main/code_snapshot/question3/interface.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/interface.py; question3/results/round14_outer_main/code_snapshot/question3/interface.py; question3/results/round14_outer_rounding/code_snapshot/question3/interface.py; question3/results/round17_adverse_main/code_snapshot/question3/interface.py; question3/results/round17_adverse_rounding/code_snapshot/question3/interface.py; question3/results/round17_dev_main/code_snapshot/question3/interface.py; question3/results/round17_dev_rounding/code_snapshot/question3/interface.py; question3/results/round18_adverse_main/code_snapshot/question3/interface.py; question3/results/round18_adverse_rounding/code_snapshot/question3/interface.py; question3/results/round18_dev_main/code_snapshot/question3/interface.py; question3/results/round18_dev_rounding/code_snapshot/question3/interface.py; question3/results/round18_holdout_main/code_snapshot/question3/interface.py; question3/results/round18_holdout_rounding/code_snapshot/question3/interface.py; question3/results/round18_outer_main/code_snapshot/question3/interface.py; question3/results/round18_outer_rounding/code_snapshot/question3/interface.py; question3/results/round19_adverse_main/code_snapshot/question3/interface.py; question3/results/round19_adverse_rounding/code_snapshot/question3/interface.py; question3/results/round19_dev_main/code_snapshot/question3/interface.py; question3/results/ro

und19_dev_rounding/code_snapshot/question3/interface.py; question3/results/round19_holdout_main/code_snapshot/question3/interface.py; question3/results/round19_holdout_rounding/code_snapshot/question3/interface.py; question3/results/round19_outer_main/code_snapshot/question3/interface.py; question3/results/round19_outer_rounding/code_snapshot/question3/interface.py; question3/results/round1_main/code_snapshot/question3/interface.py; question3/results/round1_rounding/code_snapshot/question3/interface.py; question4/innovation/frozen_baseline/question3/interface.py

```

1 """Sequential HTTP adapter, idempotent retries, accepted-only accounting.
2 This client cannot select official rehearsal vs formal mode: the UI does that.
3 """
4 import json
5 import math
6 import time
7 import uuid
8 from pathlib import Path
9 from urllib.request import Request, urlopen
10 from urllib.error import HTTPError, URLError
11
12 class ProtocolError(RuntimeError):pass
13 class TransportUncertain(RuntimeError):pass
14
15 class JsonlLog:
16     def __init__(self,path):
17         self.path=Path(path);self.path.parent.mkdir(parents=True,exist_ok=True)
18         self.file=self.path.open('w',encoding='utf-8')
19     def __call__(self,row):
20         self.file.write(json.dumps(row,ensure_ascii=False,allow_nan=False)+'\n');self.file.flush()
21     def close(self):self.file.close()
22
23 class HttpTransport:
24     def __init__(self,base_url):self.base_url=base_url.rstrip('/')
25     def request(self,path,payload,timeout):
26         req=Request(self.base_url+path,data=json.dumps(payload,allow_nan=False).encode('utf-8'),
27                     headers={'Content-Type':'application/json'},method='POST')
28         try:
29             with urlopen(req,timeout=timeout) as r:return r.status,json.load(r)
30         except HTTPError as e:
31             try:body=json.load(e)
32             except (ValueError,UnicodeError):body={}
33             return e.code,body
34
35 class Client:
36     def __init__(self,transport,robot_id,log=lambda data:None,reserve_s=5.,retries=2):
37         self.transport=transport;self.robot_id=robot_id;self.log=log;self.reserve=reserve_s;self.retries=retries
38         self.position=(0.,0.);self.channel=1;self.virtual=0.;self.deadline=None;self.max_virtual=360000
39         self.session=uuid.uuid4().hex;self.sequence=0;self.entered=False;self.closed=False;self.poisoned=False
40         self.phases=dict(movement_s=0.,switch_s=0.,detection_s=0.,optical_s=0.,laser_s=0.)
41         self.phases={};self.counts=dict(measure=0,clear=0,clear_success=0,clear_failure=0,retries=0)
42     def _call(self,path,position=None,channel=None,phase='control'):
43         if self.closed or self.poisoned:raise ProtocolError('Client closed or previous action uncertain')
44         if path!='enter' and not self.entered:raise ProtocolError('Not entered')
45         now=time.monotonic()
46         if self.deadline is not None and now>=self.deadline-(0 if path=='/exit' else self.reserve):
47             raise ProtocolError('Real deadline guard reached; completion not certified')
48         self.sequence+=1
49         body=dict(arena_id='default',robot_id=self.robot_id,request_id=f'{self.session}-{self.sequence}')
50         move=0.;switch=0.
51         if position is not None:
52             x,y=map(float,position)
53             if not all(math.isfinite(z) and abs(z)<=2000000 for z in (x,y)):
54                 raise ValueError('Invalid position')
55             if not isinstance(channel,int) or isinstance(channel,bool) or not 1<=channel<=20:raise
56                 ValueError('Invalid channel')
57             body.update(position=dict(x=x,y=y),channel=channel)
58             move=math.dist(self.position,(x,y))/5
59             switch=float(path=='/measure' and self.channel!=channel)

```

```

59         if self.virtual+move+switch+5>=self.max_virtual:
60             raise ProtocolError('Virtual deadline guard reached')
61     start=time.monotonic()
62     for attempt in range(self.retries+1):
63         self.log(dict(type='request',path=path,payload=body,attempt=attempt,phase=phase))
64         try:
65             timeout=5. if self.deadline is None else max(.001,min(5.,self.deadline-time.monotonic()))
66             status,r=self.transport.request(path,body,timeout)
67             if not isinstance(r,dict):raise ValueError('Response is not an object')
68             self.log(dict(type='response',path=path,request_id=body['request_id'],http_status=status,response=r,a
↵ ttempt=attempt))
69             break
70         except (OSError,URLError,ValueError,UnicodeError) as exc:
71             self.log(dict(type='transport_error',request_id=body['request_id'],error=str(exc),attempt=attempt))
72             if attempt==self.retries or (self.deadline is not None and time.monotonic()>=self.deadline):
73                 self.poisoned=True;raise TransportUncertain('Same-ID retries exhausted; action state unknown')
↵ from exc
74             self.counts['retries']+=1
75     if status!=200 or r.get('accepted') is not True:
76         self.poisoned=True
77         raise ProtocolError(f'HTTP {status}, accepted={r.get("accepted")}; clock retained at {self.virtual}')
78     vt=r.get('virtual_time_s')
79     if isinstance(vt,bool) or not isinstance(vt,(int,float)) or not math.isfinite(vt) or vt<self.virtual:
80         self.poisoned=True;raise ProtocolError('Invalid virtual clock')
81     delta=dict(movement_s=move,switch_s=switch,detection_s=0.,optical_s=0.,laser_s=0.)
82     if path=='/enter':
83         remaining=r.get('remaining_real_duration_s')
84         if not isinstance(remaining,(int,float)) or isinstance(remaining,bool) or not 0<=remaining<=1200:
85             raise ProtocolError('Missing/invalid remaining_real_duration_s')
86         self.deadline=start+remaining;self.max_virtual=float(r['max_virtual_duration_s']);self.entered=True
87     elif path=='/measure':
88         kind=r.get('measure_result')
89         if kind not in ('direction','near','no_signal'):raise ProtocolError('Invalid measure response')
90         if kind=='direction' and (not isinstance(r.get('svd_deg'),(int,float)) or not 0<=r['svd_deg']<360):
91             raise ProtocolError('Invalid bearing')
92         delta['detection_s']=5.;self.channel=channel;self.counts['measure']+=1
93     elif path=='/clear':
94         kind=r.get('clear_result')
95         if kind not in ('success','no_target_in_range'):raise ProtocolError('Invalid clear response')
96         delta['optical_s']=3.;delta['laser_s']=2.*(kind=='success');self.counts['clear']+=1
97         self.counts['clear_success' if kind=='success' else 'clear_failure']+=1
98     elif path=='/exit':
99         if r.get('exit_reason')!='user_exit':raise ProtocolError('Invalid exit response')
100         self.closed=True
101         if position is not None:self.position=(x,y)
102         for k,value in delta.items():self.parts[k]+=value
103         self.phases[phase]=self.phases.get(phase,0)+sum(delta.values())
104         self.virtual=float(vt)
105         if abs(sum(self.parts.values())-self.virtual)>max(1e-4,self.sequence*1e-6):
106             self.poisoned=True;raise ProtocolError('Virtual-time accounting mismatch')
107         return r
108     def enter(self):return self._call('/enter')
109     def measure(self,p,c,phase='measure'):return self._call('/measure',p,c,phase)
110     def clear(self,p,c,phase='clear'):return self._call('/clear',p,c,phase)
111     def exit(self):return self._call('/exit')
112     def metrics(self):return dict(virtual_time_s=self.virtual,components=self.parts.copy(),phases=self.phases.copy(),
↵ counts=self.counts.copy())

```

程序 23

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py; paper_evidence/q4_

mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/
 question4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round02
 _backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20
 260912/frozen/question4/__init__.py; paper_evidence/q4_mechanism_reset_20
 260912/round02_backbone/development/code_snapshot/question4/__init__.py;
 paper_evidence/q4_mechanism_reset_20260912/round03_window/development/
 code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/questi
 on4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round03_win
 dow/development/code_snapshot/question4/__init__.py; paper_evidence/q4_
 mechanism_reset_20260912/round04_search_first/development/code_snapshot/
 paper_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py;
 paper_evidence/q4_mechanism_reset_20260912/round04_search_first/developm
 ent/code_snapshot/question4/__init__.py; paper_evidence/q4_mechanism_re
 set_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_
 mechanism_reset_20260912/frozen/question4/__init__.py; paper_evidence/q
 4_mechanism_reset_20260912/round05_sector/development/code_snapshot/ques
 tion4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round06_e
 xchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_2026
 0912/frozen/question4/__init__.py; paper_evidence/q4_mechanism_reset_20
 260912/round06_exchange/development/code_snapshot/question4/__init__.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development
 /code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/questio
 n4/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round07_elast
 ic/development/code_snapshot/question4/__init__.py; paper_evidence/q4_me
 chanism_reset_20260912/round08_optical_cells/development/code_snapshot/p
 aper_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py;
 paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/devel
 opment/code_snapshot/question4/__init__.py; paper_evidence/q4_mechanis
 m_reset_20260912/round09_clear_discovery/development/code_snapshot/pap
 er_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/dev
 elopment/code_snapshot/question4/__init__.py; paper_evidence/q4_mechan
 ism_reset_20260912/round10_negative_region/development/code_snapshot/p
 aper_evidence/q4_mechanism_reset_20260912/frozen/question4/__init__.py;
 paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/d
 evelopment/code_snapshot/question4/__init__.py; question4/__init__.py;
 question4/innovation/frozen_baseline/question4/__init__.py

1 """Mixed directional source search with public-information certificates."""

程序 24

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code
 _snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/au
 dit.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_s

napshot/question4/audit.py; paper_evidence/q4_mechanism_reset_20260912/fro
 zen/question4/audit.py; paper_evidence/q4_mechanism_reset_20260912/round01
 _action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_rese
 t_20260912/frozen/question4/audit.py; paper_evidence/q4_mechanism_reset_20
 260912/round01_action_tasks/development/code_snapshot/question4/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round02_backbone/developmen
 t/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/qu
 estion4/audit.py; paper_evidence/q4_mechanism_reset_20260912/round02_b
 ackbone/development/code_snapshot/question4/audit.py; paper_evidence/q
 4_mechanism_reset_20260912/round03_window/development/code_snapshot/
 paper_evidence/q4_mechanism_reset_20260912/frozen/question4/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round03_window/development/
 code_snapshot/question4/audit.py; paper_evidence/q4_mechanism_reset_2026
 0912/round04_search_first/development/code_snapshot/paper_evidence/q4_
 mechanism_reset_20260912/frozen/question4/audit.py; paper_evidence/q4_me
 chanism_reset_20260912/round04_search_first/development/code_snapshot/qu
 estion4/audit.py; paper_evidence/q4_mechanism_reset_20260912/round05_sec
 tor/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912
 /frozen/question4/audit.py; paper_evidence/q4_mechanism_reset_20260912/r
 ound05_sector/development/code_snapshot/question4/audit.py; paper_eviden
 ce/q4_mechanism_reset_20260912/round06_exchange/development/code_snapsho
 t/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/
 code_snapshot/question4/audit.py; paper_evidence/q4_mechanism_reset_202609
 12/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism
 _reset_20260912/frozen/question4/audit.py; paper_evidence/q4_mechanism_re
 set_20260912/round07_elastic/development/code_snapshot/question4/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/develop
 ment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/que
 stion4/audit.py; paper_evidence/q4_mechanism_reset_20260912/round08_optic
 al_cells/development/code_snapshot/question4/audit.py; paper_evidence/q4_
 mechanism_reset_20260912/round09_clear_discovery/development/code_snapsh
 ot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/d
 evelopment/code_snapshot/question4/audit.py; paper_evidence/q4_mechan
 ism_reset_20260912/round10_negative_region/development/code_snapshot
 /paper_evidence/q4_mechanism_reset_20260912/frozen/question4/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel
 opment/code_snapshot/question4/audit.py; question4/audit.py; question4/in
 novation/frozen_baseline/question4/audit.py

```

1 """Independent post-run physical replay, time accounting, and stop-certificate audit."""
2 import math
3 import hashlib
4 import json
5 from functools import lru_cache

```

```

6 from pathlib import Path
7 from collections import Counter
8 import numpy as np
9 from .geometry import segment_distance
10
11 @lru_cache(maxsize=4)
12 def check_cover(contents):
13     data=json.loads(contents);assert not data['failed_cells']
14     points=np.array(data['stations']);leaves={r['key']:r for r in data['leaves']}
15     assert len(leaves)==len(data['leaves'])
16     nodes=set()
17     for key in leaves:
18         assert set(key)<=set('0123')
19         nodes.update(key[:i] for i in range(len(key)))
20     assert not (nodes & leaves.keys())
21     for key in nodes:assert all(key+str(i) in nodes or key+str(i) in leaves for i in range(4))
22     assert '' in nodes or '' in leaves
23     for key,row in leaves.items():
24         x0,y0,x1,y1=-1800.,-1800.,1800.,1800.
25         for digit in key:
26             xm=(x0+x1)/2;ym=(y0+y1)/2
27             x0,y0,x1,y1=((x0,y0,xm,ym),(xm,y0,x1,ym),(x0,ym,xm,y1),(xm,ym,x1,y1))[int(digit)]
28         if row.get('outside'):
29             dx=max(x0,0,-x1);dy=max(y0,0,-y1)
30             assert dx*dx+dy*dy>1800**2
31             continue
32         corners=np.array([(x0,y0),(x1,y0),(x1,y1),(x0,y1)])
33         hull=points[row['witnesses']];assert len(hull)>=3
34         assert len(set(row['witnesses']))==len(hull)
35         q=np.roll(hull,-1,axis=0)
36         assert np.sum(hull[:,0]*q[:,1]-hull[:,1]*q[:,0])>1e-6
37         assert np.linalg.norm(hull[:,None]-corners[None,:],axis=2).max()<1000
38         for i in range(len(hull)):
39             a=hull[i];edge=hull[(i+1)%len(hull)]-a
40             cross=edge[0]*(corners[:,1]-a[1])-edge[1]*(corners[:,0]-a[0])
41             assert cross.min()>=-1e-7
42     return points
43
44 def audit(case,records,result):
45     sources={s['channel']:s for s in case['sources']};cleared=set();measured={}
46     requests={};executed=set();p=(0.,0.);channel=1;vt=0.;n=0;exited=False
47     for row in records:
48         if row['type']=='request':
49             requests[row['payload']]['request_id']=row
50         if row['type']!='response' or row['request_id'] in executed:
51             continue
52         req=requests[row['request_id']];b=req['payload'];path=req['path'];r=row['response']
53         assert row['http_status']==200 and r['accepted'] is True
54         executed.add(row['request_id'])
55         n+=1
56         if path=='/exit':exited=True
57         if path in ('/measure','/clear'):
58             q=(b['position']['x'],b['position']['y']);ch=b['channel']
59             vt+=math.dist(p,q)/5
60             s=sources.get(ch);d=math.inf if s is None or ch in cleared else math.dist(q,(s['x'],s['y']))
61             if path=='/measure':
62                 vt+=5+(ch!=channel);channel=ch
63                 axis=None if s is None else s.get('direction_deg')
64                 angle=0 if s is None else math.degrees(math.atan2(q[1]-s['y'],q[0]-s['x']))
65                 visible=axis is None or abs((angle-axis+180)%360-180)<=90+1e-9 or d<1e-10
66                 expected='no_signal' if s is None or ch in cleared or d>s['radius'] or not visible else 'near' if
67                 ↪ d<=5 else 'direction'
68                 assert r['measure_result']==expected,(expected,r)
69                 if expected=='direction':
70                     true=math.degrees(math.atan2(s['y']-q[1],s['x']-q[0]))
71                     assert abs((r['svd_deg']-true+180)%360-180)<=1.0050001
72                 measured.setdefault(ch,[]).append(q)
73             else:
74                 expected='success' if d<=20 else 'no_target_in_range'
75                 assert r['clear_result']==expected
76                 vt+=5 if d<=20 else 3
77                 if d<=20:cleared.add(ch)
78     p=q

```

```

78     assert abs(vt-r['virtual_time_s'])<n*1e-6+1e-5
79     assert exited and result['status']=='complete'
80     assert cleared==set(sources), 'False completion against hidden truth'
81     assert cleared==set(result['cleared'])
82     cert=result['stop_certificate']
83     if cert['kind']=='count16':
84         assert len(cleared)==16
85     else:
86         points=np.array(cert['stations']);triangles=cert['triangles']; edges=Counter()
87         if cert['kind']=='cell_halfplane_cover':
88             contents=(Path(__file__).with_name('certificates')/'compact22.json').read_bytes()
89             assert hashlib.sha256(contents).hexdigest()==cert['cover_sha256']
90             assert np.array_equal(points,check_cover(contents))
91         contains_origin=False
92         assert len({tuple(sorted(t)) for t in triangles})==len(triangles)
93         for tri in triangles:
94             v=points[tri]
95             assert np.linalg.norm(v[:,None]-v[None,:],axis=2).max()<1000
96             signs=[np.linalg.det(np.stack([v[(i+1)%3]-v[i],-v[i]])) for i in range(3)]
97             contains_origin |= min(signs)>=-1e-6 or max(signs)<=1e-6
98             for i in range(3):edges[tuple(sorted((tri[i],tri[(i+1)%3])))]+=1
99         assert contains_origin or cert['kind']=='cell_halfplane_cover'
100        for (a,b),count in edges.items():
101            assert count<=2
102            if count==1:assert segment_distance(np.zeros(2),points[a],points[b])>=1800-1e-5
103        for ch in set(range(1,21))-cleared:
104            actual=set(measured.get(ch,[]))
105            assert all(tuple(p) in actual for p in points),'Unscanned absent channel'
106    return dict(passed=True,accepted_actions=n,cleared=len(cleared),virtual_time_s=vt)

```

程序 25

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/build_cover.py;

paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/
code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question
4/build_cover.py; paper_evidence/q4_mechanism_reset_20260912/round06_exc
hange/development/code_snapshot/question4/build_cover.py; paper_evidence
/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/p
aper_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py;
paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development
/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechanism_
reset_20260912/round08_optical_cells/development/code_snapshot/paper_
evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py;
paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/devel
opment/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechan
ism_reset_20260912/round09_clear_discovery/development/code_snapshot/pap
er_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py;
paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/devel
opment/code_snapshot/question4/build_cover.py; paper_evidence/q4_mechan
ism_reset_20260912/round10_negative_region/development/code_snapshot/pap
er_evidence/q4_mechanism_reset_20260912/frozen/question4/build_cover.py;
paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel
opment/code_snapshot/question4/build_cover.py; question4/build_cover.py;
question4/innovation/frozen_baseline/question4/build_cover.py

```

1  """Offline public-domain halfplane-cover certificate. No case inputs."""
2  import argparse
3  import json
4  import math
5  from pathlib import Path
6  import numpy as np
7  from scipy.spatial import ConvexHull, QhullError
8
9  def generate(inner=7, outer=14, ri=995., ro=1850., depth_limit=12):
10     points=np.array([(0.,0.)]+[(r*math.cos(2*math.pi*k/n),r*math.sin(2*math.pi*k/n))
11         for n,r in ((inner,ri),(outer,ro)) for k in range(n)])
12     leaves=[];failures=[];stack=[('',[-1800.,-1800.,1800.,1800.])]
13     while stack:
14         key,box=stack.pop();x0,y0,x1,y1=box
15         dx=min(abs(x0),abs(x1)) if x0*x1>0 else 0
16         dy=min(abs(y0),abs(y1)) if y0*y1>0 else 0
17         if dx*dx+dy*dy>1800**2:
18             leaves.append(dict(key=key,outside=True));continue
19         v=np.array([(x0,y0),(x1,y0),(x1,y1),(x0,y1)])
20         ids=np.where(np.linalg.norm(points[:,None]-v[None,:],axis=2).max(axis=1)<1000-1e-6)[0]
21         if len(ids)>=3:
22             try:
23                 hull=ConvexHull(points[ids]);h=hull.equations
24                 if np.max(v@h[:,:2].T+h[:,:2])<1e-8:
25                     leaves.append(dict(key=key,witnesses=ids[hull.vertices].tolist()));continue
26             except QhullError:pass
27         if len(key)==depth_limit:
28             failures.append(dict(key=key,box=box));continue
29         xm=(x0+x1)/2;ym=(y0+y1)/2
30         for i,b in enumerate(((x0,y0,xm,ym),(xm,y0,x1,ym),(x0,ym,xm,y1),(xm,ym,x1,y1))):
31             stack.append((key+str(i),b))
32     return dict(stations=points.tolist(),leaves=leaves,failed_cells=failures,
33         description='Each leaf lies inside convex hull of stations within 1000m of EVERY leaf point.')
34
35 def main():
36     p=argparse.ArgumentParser();p.add_argument('--output',required=True);p.add_argument('--negative21',action='store_
↵ true');args=p.parse_args()

```

```

37     data=generate(8,12,990.,1870.) if args.negative21 else generate()
38     Path(args.output).parent.mkdir(parents=True,exist_ok=True)
39     Path(args.output).write_text(json.dumps(data,separators=(',',':')))
40     print(dict(stations=len(data['stations']),leaves=len(data['leaves']),failed_cells=len(data['failed_cells'])))
41
42 if __name__=='__main__':main()

```

程序 26

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/d

development/code_snapshot/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/cases.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/cases.py; question4/cases.py; question4/innovation/frozen_baseline/question4/cases.py

```

1 """Evaluator-owned fixtures. Never imported by strategy or geometry."""
2 import math
3 import random
4
5 def make_case(seed, family='uniform', mode='spatial_hash'):
6     rng=random.Random(seed)
7     n=rng.randint(10,16); channels=rng.sample(range(1,21),n)
8     directional_count=rng.choice([1,n//2,n-1])
9     sources=[]
10    for i,ch in enumerate(channels):
11        a=rng.uniform(0,2*math.pi)
12        if family in ('outward','inward','tangent','edge'):
13            r=rng.uniform(1660,1800)
14        elif family=='cluster':
15            a=rng.choice([.3,2.5,4.7])+rng.uniform(-.12,.12);r=rng.uniform(900,1400)
16        else:
17            r=1800*math.sqrt(rng.random())
18        axis=rng.uniform(0,360)
19        if family=='outward':axis=math.degrees(a)
20        if family=='inward':axis=math.degrees(a)+180
21        if family=='tangent':axis=math.degrees(a)+rng.choice([-90,90])
22        # All benchmark fixtures contain both types, including extremes 1 and n-1 directional.
23        directional=(i < directional_count)
24        sources.append(dict(channel=ch,x=r*math.cos(a),y=r*math.sin(a),
25                            radius=1000 if family in ('outward','tangent') else rng.uniform(1000,1500),
26                            direction_deg=axis%360 if directional else None))
27    return dict(id=f'{family}_{seed}_{mode}',seed=seed,error_mode=mode,sources=sources,family=family)
28
29 def fixtures(seed,count):
30     families=['uniform','outward','inward','tangent','cluster','edge']
31     return [make_case(seed+i,families[i%len(families)]) for i in range(count)]

```

程序 27

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/experiment.py;

paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/experiment.py; question4/experiment.py; question4/innovation/frozen_baseline/question4/experiment.py

```

1 """Locked paired experiment runner; fixture truth is never passed to strategy."""
2 import argparse
3 import hashlib
4 import json
5 import platform
6 import shutil
7 import time
8 import sys
9 from pathlib import Path
10 import numpy as np
11 import scipy
12 from question3.interface import Client
13 from .simulator import Simulator
14 from .strategy import run
15 from .cases import fixtures
16 from .audit import audit
17
18 def write(path,data):
19     Path(path).write_text(json.dumps(data,ensure_ascii=False,indent=2),encoding='utf-8')
20
21 def source_hashes():
22     paths=list(Path('question4').glob('*.py'))+list(Path('question4/certificates').glob('*.json'))+[Path('question3/interface.py')]
23     return {str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted(paths)}
```

```

24
25 def main():
26     p=argparse.ArgumentParser();p.add_argument('--output',required=True);p.add_argument('--configs',required=True)
27     p.add_argument('--seed',type=int,default=41000);p.add_argument('--count',type=int,default=24)
28     p.add_argument('--fixtures');p.add_argument('--lock');p.add_argument('--stage',default='development')
29     p.add_argument('--rounding',default='bounded',choices=['bounded','pre_round_stress'])
30     p.add_argument('--error-modes',default='spatial_hash',help='Comma-separated spatial error laws, crossed across
    ↪ families')
31     args=p.parse_args();out=Path(args.output);out.mkdir(parents=True,exist_ok=False)
32     configs=json.loads(Path(args.configs).read_text());hashes=source_hashes()
33     lock=dict(code_sha256=hashes,configs=configs,created_unix=time.time(),stage=args.stage,
34             generator_seed=args.seed,generator_count=args.count,rounding=args.rounding)
35     lock['error_modes']=args.error_modes.split(',')
36     if args.lock:
37         old=json.loads(Path(args.lock).read_text())
38         assert old['code_sha256']==hashes,'Code changed after lock'
39         assert all(old['configs'].get(k)==v for k,v in configs.items()),'Configuration changed after lock'
40     write(out/'lock.json',lock)
41     for name in hashes:
42         dest=out/'code_snapshot'/name;dest.parent.mkdir(parents=True,exist_ok=True);shutil.copy2(name,dest)
43     # Generate new cases ONLY after code/config lock is persisted.
44     cases=json.loads(Path(args.fixtures).read_text()) if args.fixtures else fixtures(args.seed,args.count)
45     if not args.fixtures:
46         for i,case in enumerate(cases):
47             case['error_mode']=lock['error_modes'][(i//6)%len(lock['error_modes'])]
48             case['id']=case['id'].rsplit('_',2)[0]+'_'+case['error_mode'] if case['error_mode']!='spatial_hash' else
    ↪ case['id']
49     write(out/'fixtures.json',cases)
50     rows=[];started=time.perf_counter()
51     for case in cases:
52         for name,config in configs.items():
53             folder=out/name/case['id'];folder.mkdir(parents=True)
54             records=[];events=[];env=Simulator(case,rounding=args.rounding)
55             client=Client(env,'SELF',records.append);t=time.perf_counter()
56             try:
57                 result=run(client,config,events.append)
58                 checked=audit(case,records,result)
59             except Exception as exc:
60                 result=dict(status='incomplete',failure=f'{type(exc).__name__}:
    ↪ {exc}',metrics=client.metrics());checked=dict(passed=False)
61             elapsed=time.perf_counter()-t;truth=env.evaluation()
62             row=dict(case_id=case['id'],family=case['family'],config=name,status=result['status'],
63                     cleared=truth['cleared'],total=truth['total'],metrics=client.metrics(),elapsed_s=elapsed,audit=c
    ↪
    ↪ checked,
64                     failure=result.get('failure'))
65             rows.append(row);write(folder/'result.json',result)
66             for filename,items in [(('actions.jsonl',records),('strategy.jsonl',events))]:
67                 (folder/filename).write_text('').join(json.dumps(x,ensure_ascii=False)+'\n' for x in items))
68             print(case['id'],' '.join(f'{r['config']}: {r['metrics']['virtual_time_s']:.1f}/{r['status']}' for r in
    ↪ rows[-len(configs):]),flush=True)
69             write(out/'summary.json',dict(stage=args.stage,rows=rows,platform=platform.platform(),
70             environment=dict(python=sys.version,numpy=np.__version__,scipy=scipy.__version__),elapsed_s=time.perf_cou
    ↪
    ↪ nter()-started))
71     reference=next(iter(configs));report={}
72     base={r['case_id']:r for r in rows if r['config']==reference}
73     for name in configs:
74         selected=[r for r in rows if r['config']==name];values=np.array([r['metrics']['virtual_time_s'] for r in
    ↪ selected])
75         ref=np.array([base[r['case_id']]['metrics']['virtual_time_s'] for r in selected]);gain=(ref-values)/ref*100
76         report[name]=dict(mean_s=float(values.mean()),p95_s=float(np.percentile(values,95)),
77                             gain_ratio_of_means_pct=float((1-values.mean()/ref.mean())*100),
78                             mean_paired_gain_pct=float(gain.mean()),worst_gain_pct=float(gain.min()),
79                             complete=sum(r['status']=='complete' and r['cleared']==r['total'] and r['audit']['passed']
    ↪
    ↪ for r in selected),
80                             cases=len(selected),components={k:float(np.mean([r['metrics']['components'][k] for r in
    ↪ selected])) for k in selected[0]['metrics']['components']},
81                             failures=[r['case_id'] for r in selected if r['status']!='complete'])
82     write(out/'comparison.json',report);print(json.dumps(report,indent=2),flush=True)
83     if any(r['status']!='complete' or not r['audit']['passed'] for r in rows):raise SystemExit(1)
84
85 if __name__=='__main__':main()

```

程序 28

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/d

development/code_snapshot/question4/geometry.py; question4/geometry.py;
question4/innovation/frozen_baseline/question4/geometry.py

```

1  """Public geometry. No truth, emitter orientation, seed, or environment access."""
2  import math
3  import json
4  import hashlib
5  from pathlib import Path
6  from functools import lru_cache
7  import numpy as np
8  from scipy.spatial import Delaunay
9
10 EPS = 1e-7
11
12 def clip(poly, normal, bound):
13     p = np.asarray(poly, float)
14     if not len(p):
15         return p
16     d = p @ np.asarray(normal) - bound
17     q = []
18     for i in range(len(p)):
19         j = (i + 1) % len(p)
20         if d[i] <= 0:
21             q.append(p[i])
22         if (d[i] <= 0) != (d[j] <= 0):
23             q.append(p[i] + (p[j] - p[i]) * d[i] / (d[i] - d[j]))
24     return np.array(q).reshape(-1, 2)
25
26 def bearing_clip(poly, p, angle, error=1.005):
27     for theta, sign in ((angle-error, 1), (angle+error, -1)):
28         a = math.radians(theta)
29         n = sign*np.array([math.sin(a), -math.cos(a)])
30         poly = clip(poly, n, np.dot(n, p)+EPS)
31     if not len(poly):
32         raise RuntimeError('Empty bearing region: inconsistent readings')
33     return poly
34
35 def first_region(p, angle, error=1.005):
36     # Outer rectangle of the radius-1500 bearing sector, then outer domain polygon.
37     a = math.radians(angle)
38     u = np.array([math.cos(a), math.sin(a)])
39     v = np.array([-u[1], u[0]])
40     w = 1500*math.sin(math.radians(error))+EPS
41     poly = np.array([np.array(p)+x*u+y*v for x,y in ((0,-w),(1500+EPS,-w),(1500+EPS,w),(0,w))])
42     poly = bearing_clip(poly, p, angle, error)
43     for t in np.arange(64)*2*math.pi/64:
44         poly = clip(poly, [math.cos(t),math.sin(t)], 1800+EPS)
45     if not len(poly):
46         raise RuntimeError('Bearing outside public domain')
47     return poly
48
49 def bbox(poly, angle):
50     a = math.radians(angle)
51     rot = np.array([math.cos(a),-math.sin(a)], [math.sin(a),math.cos(a)])
52     local = np.asarray(poly)@rot
53     lo, hi = local.min(axis=0)-EPS, local.max(axis=0)+EPS
54     return lo, hi, rot
55
56 def optical_cover(poly, angle, start):
57     lo, hi, rot = bbox(poly, angle)
58     sizes = np.maximum(1,np.ceil((hi-lo)/28).astype(int))
59     points = []
60     for j in range(sizes[1]):
61         for i in (range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))):
62             points.append((lo+(np.array([i+.5,j+.5])/sizes)*(hi-lo))@rot.T)
63     if math.dist(start,points[-1]) < math.dist(start,points[0]):
64         points.reverse()
65     return points, float(np.linalg.norm((hi-lo)/sizes)/2)
66
67 def segment_distance(p, a, b):
68     v = b-a
69     t = np.clip(np.dot(p-a,v)/np.dot(v,v),0,1)

```



```
t_20260912/round02_backbone/development/code_snapshot/paper_evidence/
q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/
code_snapshot/question4/innovation/__init__.py; paper_evidence/q4_mechan
ism_reset_20260912/round03_window/development/code_snapshot/paper_eviden
ce/q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round03_window/development/co
de_snapshot/question4/innovation/__init__.py; paper_evidence/q4_mechanism_
reset_20260912/round04_search_first/development/code_snapshot/paper_evid
ence/q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round04_search_first/develo
pment/code_snapshot/question4/innovation/__init__.py; paper_evidence/q4_mech
anism_reset_20260912/round05_sector/development/code_snapshot/paper_evid
ence/q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/co
de_snapshot/question4/innovation/__init__.py; paper_evidence/q4_mechanis
m_reset_20260912/round06_exchange/development/code_snapshot/paper_eviden
ce/q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/
code_snapshot/question4/innovation/__init__.py; paper_evidence/q4_mechan
ism_reset_20260912/round07_elastic/development/code_snapshot/paper_evide
nce/q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/c
ode_snapshot/question4/innovation/__init__.py; paper_evidence/q4_mechanism_
reset_20260912/round08_optical_cells/development/code_snapshot/paper_evi
dence/q4_mechanism_reset_20260912/frozen/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/devel
opment/code_snapshot/question4/innovation/__init__.py; paper_evidence/q
4_mechanism_reset_20260912/round09_clear_discovery/development/code_snap
shot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovat
ion/__init__.py; paper_evidence/q4_mechanism_reset_20260912/round09_cle
ar_discovery/development/code_snapshot/question4/innovation/__init__.py;
paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel
opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/q
uestion4/innovation/__init__.py; paper_evidence/q4_mechanism_reset_202609
12/round10_negative_region/development/code_snapshot/question4/innovatio
n/__init__.py; question4/innovation/__init__.py
```

```
1 """Directional-source mechanism experiments; historical recommendation remains frozen."""
```

程序 30

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/
code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/ques
tion4/innovation/analysis.py; paper_evidence/q4_mechanism_reset_2026

0912/final_validation/code_snapshot/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovati
on/analysis.py; paper_evidence/q4_mechanism_reset_20260912/round01_act
ion_tasks/development/code_snapshot/paper_evidence/q4_mechanism_rese
t_20260912/frozen/question4/innovation/analysis.py; paper_evidence/q4_
mechanism_reset_20260912/round01_action_tasks/development/code_snap
shot/question4/innovation/analysis.py; paper_evidence/q4_mechanism_rese
t_20260912/round02_backbone/development/code_snapshot/paper_evidence
/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/
code_snapshot/question4/innovation/analysis.py; paper_evidence/q4_mechan
ism_reset_20260912/round03_window/development/code_snapshot/paper_eviden
ce/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round03_window/development/co
de_snapshot/question4/innovation/analysis.py; paper_evidence/q4_mechanism_
reset_20260912/round04_search_first/development/code_snapshot/paper_evid
ence/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round04_search_first/developm
ent/code_snapshot/question4/innovation/analysis.py; paper_evidence/q4_mech
anism_reset_20260912/round05_sector/development/code_snapshot/paper_evid
ence/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/co
de_snapshot/question4/innovation/analysis.py; paper_evidence/q4_mechanis
m_reset_20260912/round06_exchange/development/code_snapshot/paper_eviden
ce/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/
code_snapshot/question4/innovation/analysis.py; paper_evidence/q4_mechan
ism_reset_20260912/round07_elastic/development/code_snapshot/paper_evide
nce/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/c
ode_snapshot/question4/innovation/analysis.py; paper_evidence/q4_mechanism_
reset_20260912/round08_optical_cells/development/code_snapshot/paper_evi
dence/q4_mechanism_reset_20260912/frozen/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/devel
opment/code_snapshot/question4/innovation/analysis.py; paper_evidence/q
4_mechanism_reset_20260912/round09_clear_discovery/development/code_snap
shot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovat
ion/analysis.py; paper_evidence/q4_mechanism_reset_20260912/round09_cle
ar_discovery/development/code_snapshot/question4/innovation/analysis.py;
paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel
opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/q
uestion4/innovation/analysis.py; paper_evidence/q4_mechanism_reset_202609
12/round10_negative_region/development/code_snapshot/question4/innovatio

n/analysis.py; question4/innovation/analysis.py

```

1 """Layout-blocked statistics for Q4 SELF experiments; no policy or simulator calls.
2
3 Examples (all paths are relative to the project root)::
4
5     python -m question4.innovation.analysis --input question4/innovation/results/dev \
6         --output question4/innovation/results/dev/statistics.json --bootstrap 5000
7
8 Each physical layout is one resampling block. Error laws and serialization models
9 on that layout stay together. Family-stratified bootstrap preserves the number of
10 layouts in each of the six generator families. Positive paired gain means faster.
11 """
12 import argparse
13 import hashlib
14 import json
15 from collections import defaultdict
16 from pathlib import Path
17
18 import numpy as np
19
20
21 def _sha256_json(value):
22     return hashlib.sha256(json.dumps(value, sort_keys=True, ensure_ascii=False,
23                                     separators=(',', ':'), allow_nan=False).encode()).hexdigest()
24
25
26 def validate_batch(source, summary):
27     """Reject partial/invalid batches before producing formal statistics.
28
29     Historical experiments are checked against their own immutable source
30     snapshots. The current working-tree policy may legitimately have changed.
31     """
32     folder = Path(source).parent
33
34     def require(condition, message):
35         if not condition:
36             raise ValueError('Batch incomplete/invalid: ' + message)
37
38     require(not (folder / 'batch_failure.json').exists(), 'batch_failure.json records a framework failure')
39     required = ('lock.json', 'configs.json', 'fixtures.json', 'integrity.json', 'snapshot_complete.json')
40     for filename in required:
41         require((folder / filename).is_file(), 'missing ' + filename)
42     read = lambda filename: json.loads((folder / filename).read_text(encoding='utf-8'))
43     lock, configs, fixtures = read('lock.json'), read('configs.json'), read('fixtures.json')
44     integrity, snapshot = read('integrity.json'), read('snapshot_complete.json')
45     require(integrity.get('source_unchanged_during_run') is True, 'source integrity did not pass')
46     require(bool(configs) and configs == lock.get('configs'), 'configuration set differs from lock')
47     require(_sha256_json(configs) == lock.get('configs_sha256'), 'configuration hash differs from lock')
48     hashes = lock.get('code_sha256', {})
49     require(bool(hashes) and _sha256_json(hashes) == lock.get('code_bundle_sha256'), 'invalid source bundle hash')
50     require(snapshot.get('code_bundle_sha256') == lock['code_bundle_sha256'], 'source snapshot bundle differs from
    ↳ lock')
51     for name, digest in hashes.items():
52         path = folder / 'code_snapshot' / name
53         require(path.is_file() and hashlib.sha256(path.read_bytes()).hexdigest() == digest,
54                 'source snapshot missing or changed: ' + name)
55     selection = folder / 'selection_snapshot.txt'
56     require(selection.is_file() and hashlib.sha256(selection.read_bytes()).hexdigest() ==
    ↳ lock.get('selection_sha256'),
57             'selection snapshot missing or changed')
58     require(isinstance(fixtures, list) and bool(fixtures), 'empty or malformed fixtures')
59     cases = {case['id']: case for case in fixtures}
60     require(len(cases) == len(fixtures), 'duplicate fixture case IDs')
61     provenance = summary.get('provenance', {})
62     require(provenance.get('fixtures_sha256') == _sha256_json(fixtures), 'fixtures changed after generation')
63     require(provenance.get('lock_sha256') == hashlib.sha256((folder / 'lock.json').read_bytes()).hexdigest(),
64             'lock changed after generation')
65     require(summary.get('completed_scenarios') == len(cases) == summary.get('expected_scenarios'),
66             'not every expected scenario finished')
67     require(provenance.get('scenarios') == len(cases), 'scenario count differs from provenance')
68     expected = {(name, case_id) for name in configs for case_id in cases}
69     require(provenance.get('algorithm_runs') == len(expected), 'run count differs from locked matrix')

```

```

70 rows = summary.get('rows', [])
71 observed = [(row['config'], row['case_id']) for row in rows]
72 require(len(observed) == len(set(observed)), 'duplicate configuration/case rows')
73 missing, extra = expected - set(observed), set(observed) - expected
74 require(not missing and not extra, f'configuration/case matrix has {len(missing)} missing and {len(extra)} extra
↪ rows')
75 layouts = defaultdict(set)
76 for case in fixtures:
77     digest = _sha256_json(sorted(case['sources'], key=lambda source: source['channel']))
78     require(case.get('layout_sha256') == digest, 'fixture layout hash mismatch: ' + case['id'])
79     layouts[digest].add((case['error_mode'], case['rounding']))
80 axes = {(error, rounding) for error in lock['error_modes'] for rounding in lock['roundings']}
81 require(all(combinations == axes for combinations in layouts.values()), 'incomplete error/rounding cross within a
↪ layout')
82 require(len(fixtures) == len(layouts) * len(axes), 'duplicated layout/error/rounding scenario')
83 require(provenance.get('layouts') == len(layouts), 'layout count differs from provenance')
84 for row in rows:
85     case = cases[row['case_id']]
86     for field in ('layout_id', 'layout_sha256', 'family', 'seed', 'error_mode', 'rounding'):
87         require(row.get(field) == case.get(field), f'row/fixture {field} differs: {row["case_id"]}')
88 return dict(passed=True, expected_scenarios=len(cases), expected_runs=len(expected),
89             layouts=len(layouts), source_files_checked=len(hashes),
90             source_basis='batch code_snapshot, not current working tree')
91
92
93 def _mean_mapping(rows, section):
94     keys = sorted({key for row in rows for key in row.get('metrics', {}).get(section, {})})
95     return {key: float(np.mean([row.get('metrics', {}).get(section, {}).get(key, 0.)
96                                for row in rows])) for key in keys}
97
98
99 def _summary(rows):
100 values = np.asarray([row['penalized_virtual_s'] for row in rows], dtype=float)
101 raw = np.asarray([row['metrics']['virtual_time_s'] for row in rows], dtype=float)
102 worst = rows[int(np.argmax(values))]
103 return dict(runs=len(rows), layouts=len({row['layout_id'] for row in rows}),
104             complete=sum(bool(row['success']) for row in rows),
105             failed=sum(not row['success'] for row in rows),
106             false_complete=sum(bool(row.get('false_complete')) for row in rows),
107             mean_s=float(values.mean()), p95_s=float(np.percentile(values, 95)),
108             max_s=float(values.max()), raw_mean_s=float(raw.mean()),
109             raw_p95_s=float(np.percentile(raw, 95)), raw_max_s=float(raw.max()),
110             worst_case=worst['case_id'],
111             components=_mean_mapping(rows, 'components'),
112             phases=_mean_mapping(rows, 'phases'),
113             counts=_mean_mapping(rows, 'counts'),
114             wall_mean_s=float(np.mean([row.get('elapsed_s', 0.) for row in rows])),
115             wall_p95_s=float(np.percentile([row.get('elapsed_s', 0.) for row in rows], 95)),
116             failures=[dict(case_id=row['case_id'], status=row['status'],
117                            failure=row.get('failure'), audit=row.get('audit'))
118                       for row in rows if not row['success']]
119
120
121 def _paired(rows, baseline, worst_count=10):
122 pairs = [(baseline[row['case_id']], row) for row in rows if row['case_id'] in baseline]
123 if not pairs:
124     return dict(paired_runs=0)
125 ref = np.asarray([left['penalized_virtual_s'] for left, _ in pairs], dtype=float)
126 val = np.asarray([right['penalized_virtual_s'] for _, right in pairs], dtype=float)
127 gains = (ref - val) / ref * 100
128 blocks = defaultdict(list)
129 for left, right in pairs:
130     blocks[right['layout_id']].append((left['penalized_virtual_s'], right['penalized_virtual_s']))
131 layout_gains = [100 * (1 - np.mean([v for _, v in block]) / np.mean([r for r, _ in block]))
132                 for block in blocks.values()]
133 worst = sorted(zip(pairs, gains), key=lambda item: item[1][:worst_count])
134 return dict(paired_runs=len(pairs), paired_layouts=len(blocks),
135             gain_ratio_of_means_pct=float((1 - val.mean() / ref.mean()) * 100),
136             mean_paired_gain_pct=float(gains.mean()),
137             mean_layout_gain_pct=float(np.mean(layout_gains)),
138             mean_paired_saving_s=float((ref - val).mean()),
139             paired_gain_p05_pct=float(np.percentile(gains, 5)),
140             worst_gain_pct=float(gains.min()),

```

```

141         wins=int((val < ref).sum()), ties=int((val == ref).sum()),
142         losses=int((val > ref).sum()),
143         worst_paired_cases=[dict(case_id=right['case_id'], layout_id=right['layout_id'],
144                                 family=right['family'], error_mode=right['error_mode'],
145                                 rounding=right['rounding'], baseline_s=left['penalized_virtual_s'],
146                                 candidate_s=right['penalized_virtual_s'],
147                                 gain_pct=float(gain),
148                                 regression_s=right['penalized_virtual_s'] - left['penalized_virtual_s'])
149         for ((left, right), gain) in worst])
150
151
152 def _bootstrap(rows, baseline, samples, seed):
153     blocks = defaultdict(list)
154     families = {}
155     for row in rows:
156         ref = baseline.get(row['case_id'])
157         if ref is None:
158             continue
159         blocks[row['layout_id']].append((ref['penalized_virtual_s'], row['penalized_virtual_s']))
160         families[row['layout_id']] = row['family']
161     if len(blocks) < 2:
162         return dict(samples=0, unavailable_reason='At least two independent layout blocks are required.')
163     strata = defaultdict(list)
164     for layout_id, pairs in blocks.items():
165         # Use sums and run count to support explicit fixture subsets without treating
166         # individual error laws or roundings as independent observations.
167         pair = np.asarray(pairs, dtype=float)
168         strata[families[layout_id]].append([pair[:, 0].sum(), pair[:, 1].sum(), len(pair)])
169     rng = np.random.default_rng(seed)
170     total = np.zeros((samples, 3), dtype=float)
171     for family in sorted(strata):
172         values = np.asarray(strata[family], dtype=float)
173         indices = rng.integers(0, len(values), size=(samples, len(values)))
174         total += values[indices].sum(axis=1)
175     gain = 100 * (1 - total[:, 1] / total[:, 0])
176     saving = (total[:, 0] - total[:, 1]) / total[:, 2]
177     return dict(method='paired layout-block bootstrap stratified by generator family',
178               samples=samples, seed=seed, layouts=len(blocks),
179               family_layout_counts={key: len(value) for key, value in sorted(strata.items())},
180               gain_ratio_of_means_pct_ci95=[float(x) for x in np.percentile(gain, [2.5, 97.5])],
181               mean_paired_saving_s_ci95=[float(x) for x in np.percentile(saving, [2.5, 97.5])],
182               bootstrap_fraction_gain_above_zero=float(np.mean(gain > 0)),
183               bootstrap_fraction_gain_above_one_pct=float(np.mean(gain > 1)))
184
185
186 def analyze(rows, reference=None, bootstrap=0, seed=20260912, worst_count=10):
187     if not rows:
188         return dict(reference=reference, configurations={}, groups={}, independent_layouts=0)
189     names = list(dict.fromkeys(row['config'] for row in rows))
190     reference = reference or ('BASE' if 'BASE' in names else names[0])
191     if reference not in names:
192         raise ValueError(f'Reference configuration absent: {reference}')
193     seen = set()
194     for row in rows:
195         key = (row['config'], row['case_id'])
196         if key in seen:
197             raise ValueError(f'Duplicate configuration/case observation: {key}')
198         seen.add(key)
199     baseline = {row['case_id']: row for row in rows if row['config'] == reference}
200     configs = {}
201     groups = {}
202     for name in names:
203         selected = [row for row in rows if row['config'] == name]
204         result = _summary(selected)
205         result.update(_paired(selected, baseline, worst_count))
206         result['all_complete_and_audited'] = result['failed'] == 0
207         if bootstrap:
208             result['bootstrap'] = _bootstrap(selected, baseline, bootstrap, seed)
209         configs[name] = result
210     for dimension in ('family', 'error_mode', 'rounding'):
211         groups[dimension] = {}
212         for value in sorted({row[dimension] for row in rows}):
213             subset = [row for row in rows if row[dimension] == value]

```

```

214         groups[dimension][value] = {}
215         for name in names:
216             selected = [row for row in subset if row['config'] == name]
217             if selected:
218                 result = _summary(selected)
219                 result.update(_paired(selected, baseline, min(3, worst_count)))
220                 groups[dimension][value][name] = result
221     return dict(reference=reference, independent_layouts=len({row['layout_id'] for row in rows}),
222                 statistical_unit='physical layout; all error laws and roundings stay in the same block',
223                 primary_metric='whole-game virtual seconds; unsuccessful runs use max(raw seconds, 360000)',
224                 configurations=configs, groups=groups)
225
226
227 def main():
228     parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
229     parser.add_argument('--input', required=True, help='Experiment directory or summary.json')
230     parser.add_argument('--output', required=True, help='New JSON statistics file; existing files are refused')
231     parser.add_argument('--reference', help='Defaults to BASE if present, otherwise first configuration')
232     parser.add_argument('--bootstrap', type=int, default=5000, choices=[0, 5000, 10000])
233     parser.add_argument('--seed', type=int, default=20260912, help='Bootstrap RNG seed, not layout seed')
234     parser.add_argument('--worst-count', type=int, default=10)
235     args = parser.parse_args()
236     source = Path(args.input)
237     if source.is_dir():
238         source /= 'summary.json'
239     output = Path(args.output)
240     if output.exists():
241         parser.error(f'Output already exists: {output}')
242     try:
243         summary = json.loads(source.read_text(encoding='utf-8'))
244         integrity = validate_batch(source, summary)
245     except (ValueError, KeyError, TypeError, OSError) as exc:
246         parser.error(str(exc) if 'Batch incomplete/invalid:' in str(exc)
247                     else 'Batch incomplete/invalid: ' + str(exc))
248     report = analyze(summary['rows'], args.reference, args.bootstrap, args.seed, args.worst_count)
249     report['batch_validation'] = integrity
250     report['source_summary'] = str(source.resolve())
251     report['provenance'] = summary.get('provenance', {})
252     output.parent.mkdir(parents=True, exist_ok=True)
253     with output.open('x', encoding='utf-8') as handle:
254         json.dump(report, handle, ensure_ascii=False, indent=2, allow_nan=False)
255         handle.write('\n')
256     print(json.dumps({name: {key: value[key] for key in ('complete', 'runs', 'mean_s', 'p95_s', 'max_s',
257                                                         'gain_ratio_of_means_pct')}}
258              for name, value in report['configurations'].items(), indent=2, flush=True)
259
260
261 if __name__ == '__main__':
262     main()

```

程序 31

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/r

ound02_backbone/development/code_snapshot/question4/innovation/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round03_window/development/
 code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/questi
 on4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/r
 ound03_window/development/code_snapshot/question4/innovation/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round04_search_first/developm
 ent/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/ques
 tion4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/rou
 nd04_search_first/development/code_snapshot/question4/innovation/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/
 code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/questi
 on4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/r
 ound05_sector/development/code_snapshot/question4/innovation/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round06_exchange/developmen
 t/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/ques
 tion4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/r
 ound06_exchange/development/code_snapshot/question4/innovation/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/c
 ode_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4
 /innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/round07_e
 lastic/development/code_snapshot/question4/innovation/audit.py; paper_evid
 ence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_
 snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/inn
 ovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/round08_optic
 al_cells/development/code_snapshot/question4/innovation/audit.py; paper_ev
 idence/q4_mechanism_reset_20260912/round09_clear_discovery/development/c
 ode_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4
 /innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/round09
 _clear_discovery/development/code_snapshot/question4/innovation/audit.py;
 paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel
 opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/q
 uestion4/innovation/audit.py; paper_evidence/q4_mechanism_reset_20260912/r
 ound10_negative_region/development/code_snapshot/question4/innovation/au
 dit.py; question4/innovation/audit.py

```

1 """Evaluator-only checks of logged conservative regions and optical evidence.
2
3 This module is called after policy execution. It imports no policy geometry and
4 does not lend the policy any source coordinates, directions or radii.
5 """
6 import math
7 import numpy as np
8
9
10 TOLERANCE_M = 2e-5
11
12
13 def _vertices(row):
14     vertices = np.asarray(row['vertices'], dtype=float)
15     assert vertices.ndim == 2 and vertices.shape[1] == 2 and len(vertices), 'Malformed region'

```

```

16     assert np.isfinite(vertices).all(), 'Nonfinite region'
17     return vertices
18
19
20 def _inside(vertices, point):
21     """Independent convex support-line test, including degenerate polygons."""
22     point = np.asarray(point, dtype=float)
23     assert point.shape == (2,) and np.isfinite(point).all(), 'Malformed point'
24     low, high = vertices.min(axis=0), vertices.max(axis=0)
25     if np.any(point < low-TOLERANCE_M) or np.any(point > high+TOLERANCE_M):
26         return False
27     edges = np.roll(vertices, -1, axis=0)-vertices
28     lengths = np.linalg.norm(edges, axis=1)
29     active = lengths > 1e-12
30     if not active.any():
31         return bool(np.linalg.norm(point-vertices[0]) <= TOLERANCE_M)
32     offsets = point-vertices
33     signed = (edges[:, 0]*offsets[:, 1]-edges[:, 1]*offsets[:, 0])[active]/lengths[active]
34     return bool(np.all(signed >= -TOLERANCE_M) or np.all(signed <= TOLERANCE_M))
35
36
37 def _check_optical_rectangle(row, vertices, source):
38     """Reconstruct the rectangular grid from public angle, independently.
39
40     Historical pair-mode events omit the angle. Those only support a region
41     containment audit and are explicitly counted separately in the result.
42     """
43     radius = float(row['cover_radius'])
44     assert math.isfinite(radius) and 0 <= radius < 20, 'Invalid optical cell radius'
45     count = row['points']
46     assert isinstance(count, int) and not isinstance(count, bool) and count > 0, 'Invalid grid count'
47     if 'angle' not in row:
48         return False
49     theta = math.radians(float(row['angle']))
50     assert math.isfinite(theta), 'Nonfinite rectangle angle'
51     axes = np.array([[math.cos(theta), -math.sin(theta)], [math.sin(theta), math.cos(theta)]])
52     local = vertices @ axes
53     low, high = local.min(axis=0)-1e-7, local.max(axis=0)+1e-7
54     source_local = np.asarray(source) @ axes
55     assert np.all(source_local >= low-TOLERANCE_M) and np.all(source_local <= high+TOLERANCE_M), 'Source outside
↳ optical rectangle'
56     shape = np.maximum(1, np.ceil((high-low)/28).astype(int))
57     assert int(np.prod(shape)) == count, 'Optical grid count mismatch'
58     cell_width = (high-low)/shape
59     expected_radius = float(np.linalg.norm(cell_width)/2)
60     assert abs(expected_radius-radius) <= TOLERANCE_M, 'Optical cell radius mismatch'
61     cell_index = np.clip(np.floor((source_local-low)/cell_width).astype(int), 0, shape-1)
62     nearest = low+(cell_index+.5)*cell_width
63     assert np.linalg.norm(source_local-nearest) <= radius+TOLERANCE_M, 'Grid misses true source'
64     return True
65
66
67 def audit_regions(case, events):
68     """Return audit counts or raise AssertionError for invalid logged evidence."""
69     sources = {int(s['channel']): np.array([s['x'], s['y']], dtype=float) for s in case['sources']}
70     source_models = {int(s['channel']): s for s in case['sources']}
71     counts = dict(passed=True, checked_regions=0, checked_clear_circles=0,
72                  checked_optical_covers=0, optical_region_only=0,
73                  checked_no_signal_unchanged=0, checked_directional_shadows=0)
74     previous = {}
75     positive_observations, negative_observations = {}, {}
76     for row in events:
77         kind = row.get('event')
78         if kind == 'observe':
79             observed = positive_observations if row['kind'] in ('near', 'direction') else negative_observations
80             observed.setdefault(int(row['channel']), set()).add(tuple(row['point']))
81             if kind not in ('feasible_region', 'certified_clear', 'optical_cover', 'directional_shadow_skip'):
82                 continue
83             channel = int(row['channel'])
84             assert channel in sources, 'Region for nonexistent source'
85             vertices = _vertices(row)
86             source = sources[channel]
87             assert _inside(vertices, source), f'True source excluded: channel {channel}, event {kind}'

```



```

88     counts['checked_regions'] += 1
89     if kind == 'feasible_region':
90         if row.get('kind') == 'no_signal' and channel in previous:
91             assert np.array_equal(vertices, previous[channel]), 'no_signal changed feasible region'
92             counts['checked_no_signal_unchanged'] += 1
93             previous[channel] = vertices.copy()
94         elif kind == 'certified_clear':
95             point = np.asarray(row['point'], dtype=float)
96             radius = float(row['radius'])
97             assert point.shape == (2,) and np.isfinite(point).all(), 'Malformed clear point'
98             assert math.isfinite(radius) and 0 <= radius < 20, 'Invalid clear radius'
99             assert np.linalg.norm(vertices-point, axis=1).max() <= radius+TOLERANCE_M, 'Clear circle excludes a
↳ region vertex'
100             assert np.linalg.norm(source-point) <= 20+TOLERANCE_M, 'Clear circle misses true source'
101             counts['checked_clear_circles'] += 1
102         elif kind == 'optical_cover':
103             if _check_optical_rectangle(row, vertices, source):
104                 counts['checked_optical_covers'] += 1
105             else:
106                 counts['optical_region_only'] += 1
107         else:
108             # Evaluator truth is used only here, after execution. Neither the
109             # cone construction nor its implementation is reused by this audit.
110             model = source_models[channel]
111             assert model.get('direction_deg') is not None, 'Omnidirectional source assigned a shadow'
112             theta = math.radians(model['direction_deg'])
113             normal = np.array([math.cos(theta), math.sin(theta)])
114             query = np.asarray(row['point'], dtype=float)
115             assert query.shape == (2,) and np.isfinite(query).all(), 'Invalid shadow point'
116             assert float(normal @ (query-source)) < -1e-10, 'Shadow skip lies in true emitting halfplane'
117             proof = row['certificate']
118             n = np.asarray(proof['negative_point'], dtype=float)
119             positive = np.asarray(proof['positive_points'], dtype=float)
120             coefficients = np.asarray(proof['coefficients'], dtype=float)
121             assert n.shape == (2,) and positive.shape == (2, 2) and coefficients.shape == (2,), 'Malformed shadow
↳ witness'
122             assert np.isfinite(n).all() and np.isfinite(positive).all() and np.isfinite(coefficients).all(),
↳ 'Nonfinite shadow witness'
123             assert tuple(n) in negative_observations.get(channel, set()), 'Shadow negative point was not observed'
124             assert all(tuple(a) in positive_observations.get(channel, set()) for a in positive), 'Shadow positive
↳ point was not observed'
125             assert np.linalg.norm(vertices-n, axis=1).max() < 1000., 'Shadow negative point lacks range guarantee'
126             assert float(normal @ (n-source)) < -1e-10, 'Witness negativity is not directional'
127             assert np.all((positive-source) @ normal >= -1e-10), 'Witness positive point is behind source'
128             assert np.linalg.norm(positive-source, axis=1).max() <= model['radius'], 'Witness positive point is
↳ beyond range'
129             assert np.all(coefficients > 0), 'Shadow coefficients are not strictly positive'
130             reconstructed = n+coefficients @ (n-positive)
131             assert np.linalg.norm(query-reconstructed) <= TOLERANCE_M, 'Shadow combination mismatch'
132             counts['checked_directional_shadows'] += 1
133     return counts

```

程序 32

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q

4_mechanism_reset_20260912/frozen/question4/innovation/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/
 code_snapshot/question4/innovation/experiment.py; paper_evidence/q4_mechan
 ism_reset_20260912/round03_window/development/code_snapshot/paper_eviden
 ce/q4_mechanism_reset_20260912/frozen/question4/innovation/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round03_window/development/
 code_snapshot/question4/innovation/experiment.py; paper_evidence/q4_me
 chanism_reset_20260912/round04_search_first/development/code_snapshot/
 paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation
 /experiment.py; paper_evidence/q4_mechanism_reset_20260912/round04_sea
 rch_first/development/code_snapshot/question4/innovation/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/co
 de_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/
 innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round
 05_sector/development/code_snapshot/question4/innovation/experiment.py; pa
 per_evidence/q4_mechanism_reset_20260912/round06_exchange/development/co
 de_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/
 innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round
 06_exchange/development/code_snapshot/question4/innovation/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/c
 ode_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4
 /innovation/experiment.py; paper_evidence/q4_mechanism_reset_20260912/rou
 nd07_elastic/development/code_snapshot/question4/innovation/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/devel
 opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/froz
 en/question4/innovation/experiment.py; paper_evidence/q4_mechanism_re
 set_20260912/round08_optical_cells/development/code_snapshot/questio
 n4/innovation/experiment.py; paper_evidence/q4_mechanism_reset_202609
 12/round09_clear_discovery/development/code_snapshot/paper_evidence/
 q4_mechanism_reset_20260912/frozen/question4/innovation/experiment.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/devel
 opment/code_snapshot/question4/innovation/experiment.py; paper_evidence/q
 4_mechanism_reset_20260912/round10_negative_region/development/code_snap
 shot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovat
 ion/experiment.py; paper_evidence/q4_mechanism_reset_20260912/round10_neg
 ative_region/development/code_snapshot/question4/innovation/experiment.py;
 question4/innovation/experiment.py

```

1 """Frozen, paired Q4 SELF experiments; never invokes an HTTP/official environment.
2
3 From the project root::
4
5     python -m question4.innovation.experiment --configs path/to/configs.json \
6         --output question4/innovation/results/dev --seed 940000 --count 24 \
7         --error-modes spatial_hash,positive,negative,smooth \
8         --roundings bounded,pre_round_stress --workers 4 --stage development
9
10     python -m question4.innovation.experiment --configs path/to/selected.json \

```

```

11     --selection path/to/selection.json --output path/to/locked_dev \
12     --seed 950000 --count 6 --stage selection_lock
13
14     python -m question4.innovation.experiment --configs path/to/selected.json \
15     --selection path/to/selection.json --output path/to/validation \
16     --seed 960000 --count 96 --lock path/to/locked_dev/lock.json \
17     --stage independent_validation --workers 4
18
19 Reproduction: replace --seed/--count with --fixtures path/to/fixtures.json and
20 use --stage reproduction. Fixtures are always labelled non-independent.
21 --count counts physical layouts, not layout x error-law x rounding runs.
22 Source/configuration/selection locks and code snapshots precede case generation.
23 """
24 import argparse
25 import copy
26 import hashlib
27 import json
28 import platform
29 import re
30 import shutil
31 import sys
32 import time
33 import traceback
34 from collections import Counter
35 from concurrent.futures import ProcessPoolExecutor, as_completed
36 from pathlib import Path
37
38 import numpy as np
39 import scipy
40
41 from question3.interface import Client
42 from question4.audit import audit
43 from question4.cases import make_case
44 from question4.simulator import Simulator
45 from .analysis import analyze
46
47
48 PROJECT = Path(__file__).resolve().parents[2]
49 INNOVATION = PROJECT / 'question4' / 'innovation'
50 FAMILIES = ('uniform', 'outward', 'inward', 'tangent', 'cluster', 'edge')
51 ERROR_MODES = ('spatial_hash', 'positive', 'negative', 'smooth', 'zero')
52 ROUNDINGS = ('bounded', 'pre_round_stress')
53 FAILURE_PENALTY_S = 360000.
54
55
56 class PublicClient:
57     """Policy capability: public actions, observations and accounting only."""
58     __slots__ = ('__client',)
59
60     def __init__(self, client):
61         self.__client = client
62
63     @property
64     def position(self):
65         return self.__client.position
66
67     @property
68     def channel(self):
69         return self.__client.channel
70
71     def enter(self):
72         return self.__client.enter()
73
74     def measure(self, *args, **kwargs):
75         return self.__client.measure(*args, **kwargs)
76
77     def clear(self, *args, **kwargs):
78         return self.__client.clear(*args, **kwargs)
79
80     def exit(self):
81         return self.__client.exit()
82
83     def metrics(self):

```

```

84         return self.__client.metrics()
85
86
87 def canonical_sha256(value):
88     return hashlib.sha256(json.dumps(value, sort_keys=True, ensure_ascii=False,
89                                     separators=(',', ':'), allow_nan=False).encode()).hexdigest()
89
90
91
92 def write_json(path, data):
93     path = Path(path)
94     path.parent.mkdir(parents=True, exist_ok=True)
95     temporary = path.with_name(path.name + '.writing')
96     temporary.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False) + '\n', encoding='utf-8')
97     temporary.replace(path)
98
99
100 def source_hashes():
101     paths = set((PROJECT / 'question4').glob('*.py'))
102     paths.update((PROJECT / 'question4' / 'certificates').glob('*.json'))
103     paths.update(INNOVATION.glob('*.py'))
104     paths.update(INNOVATION.glob('baseline*.json'))
105     paths.update(INNOVATION.glob('frozen_manifest.json'))
106     frozen = INNOVATION / 'frozen_baseline'
107     paths.update(frozen.rglob('*.py'))
108     paths.update(frozen.rglob('*.json'))
109     paths.add(PROJECT / 'question3' / 'interface.py')
110     return {str(path.relative_to(PROJECT)): hashlib.sha256(path.read_bytes()).hexdigest()
111             for path in sorted(paths)}
112
113
114 def _csv(value, allowed):
115     result = [part.strip() for part in value.split(',') if part.strip()]
116     if not result or len(result) != len(set(result)) or any(part not in allowed for part in result):
117         raise argparse.ArgumentTypeError(f'Expected distinct comma-separated values from {", ".join(allowed)}')
118     return result
119
120
121 def _layout_identity(case):
122     # Source parameters, not seed or error law, define a physical layout block.
123     sources = sorted(case['sources'], key=lambda source: source['channel'])
124     return canonical_sha256(sources)
125
126
127 def _load_layouts(args):
128     if args.fixtures:
129         loaded = json.loads(Path(args.fixtures).read_text(encoding='utf-8'))
130         if isinstance(loaded, dict):
131             loaded = loaded.get('layouts', loaded.get('cases'))
132         if not isinstance(loaded, list) or not loaded:
133             raise ValueError('Fixtures must contain a nonempty list of case/layout objects')
134         seen = set()
135         layouts = []
136         for case in loaded:
137             identity = _layout_identity(case)
138             if identity not in seen:
139                 case = copy.deepcopy(case)
140                 case['layout_sha256'] = identity
141                 case['layout_id'] = f'{case.get("family", "fixture")}_{identity[:16]}'
142                 layouts.append(case)
143                 seen.add(identity)
144         return layouts
145     return [make_case(args.seed + index, FAMILIES[index % len(FAMILIES)]) for index in range(args.count)]
146
147
148 def _expand_cases(layouts, error_modes, roundings):
149     cases = []
150     seen = set()
151     for layout in layouts:
152         digest = _layout_identity(layout)
153         if digest in seen:
154             raise ValueError('Duplicate physical layout in generated dataset')
155         seen.add(digest)
156         layout_id = f'{layout.get("family", "fixture")}_{digest[:16]}'

```

```

157         for error_mode in error_modes:
158             for rounding in roundings:
159                 case = copy.deepcopy(layout)
160                 case.update(layout_id=layout_id, layout_sha256=digest, error_mode=error_mode,
161                             rounding=rounding, id=f'{layout_id}__{error_mode}__{rounding}')
162                 cases.append(case)
163     return cases
164
165
166 def _run_case(task):
167     case, configs, output, stage, reference = task
168     from question4.innovation.strategy import run
169
170     rows = []
171     names = list(configs)
172     offset = int(case['layout_sha256'][:8], 16) % len(names)
173     names = names[offset:] + names[:offset]
174     for name in names:
175         folder = Path(output) / name / case['id']
176         folder.mkdir(parents=True, exist_ok=False)
177         records, events = [], []
178         env = Simulator(case, rounding=case['rounding'])
179         client = Client(env, 'SELF', records.append)
180         started = time.perf_counter()
181         exception_trace = None
182         try:
183             result = run(PublicClient(client), copy.deepcopy(configs[name]), events.append)
184             if not isinstance(result, dict):
185                 raise TypeError('Strategy must return a dictionary')
186         except Exception as exc:
187             exception_trace = traceback.format_exc()
188             result = dict(status='incomplete', failure=f'{type(exc).__name__}: {exc}', metrics=client.metrics())
189         elapsed = time.perf_counter() - started
190         # This is the first evaluator truth access. Neither case nor env goes to run().
191         truth = env.evaluation()
192         try:
193             checked = audit(case, records, result)
194         except Exception as exc:
195             checked = dict(passed=False, failure=f'{type(exc).__name__}: {exc}', traceback=traceback.format_exc())
196         try:
197             from .audit import audit_regions
198             checked['regions'] = audit_regions(case, events)
199             checked['passed'] = bool(checked.get('passed')) and bool(checked['regions'].get('passed'))
200         except Exception as exc:
201             checked['regions'] = dict(passed=False, failure=f'{type(exc).__name__}: {exc}',
202                                     traceback=traceback.format_exc())
203             checked['passed'] = False
204         status = result.get('status', 'incomplete')
205         false_complete = status == 'complete' and truth['cleared'] != truth['total']
206         success = status == 'complete' and truth['cleared'] == truth['total'] and bool(checked.get('passed'))
207         virtual_s = max(float(truth['virtual_time_s']), float(client.metrics()['virtual_time_s']))
208         row = dict(case_id=case['id'], layout_id=case['layout_id'], layout_sha256=case['layout_sha256'],
209                  seed=case.get('seed'), family=case.get('family', 'fixture'),
210                  error_mode=case['error_mode'], rounding=case['rounding'], stage=stage,
211                  config=name, reference=reference, status=status, success=success,
212                  false_complete=false_complete, cleared=truth['cleared'], total=truth['total'],
213                  metrics=client.metrics(), elapsed_s=elapsed, audit=checked,
214                  failure=result.get('failure'), exception_trace=exception_trace,
215                  penalized_virtual_s=virtual_s if success else max(virtual_s, FAILURE_PENALTY_S),
216                  event_counts=dict(Counter(event.get('event', event.get('type', 'untyped')) for event in events)))
217         write_json(folder / 'result.json', dict(summary=row, strategy=result, evaluation=truth))
218         write_json(folder / 'evaluator_hidden_case.json', case)
219         for filename, items in (('actions.jsonl', records), ('strategy.jsonl', events)):
220             with (folder / filename).open('x', encoding='utf-8') as handle:
221                 for item in items:
222                     handle.write(json.dumps(item, ensure_ascii=False, allow_nan=False) + '\n')
223         rows.append(row)
224         print(f'RUN {case["id"]} {name} virtual_s={virtual_s:.3f} penalized_s={row["penalized_virtual_s"]:.3f} '
225               f'complete={truth["cleared"]}/{truth["total"]} audit={bool(checked.get("passed"))} '
226               f'↪ wall_s={elapsed:.3f}', flush=True)
227     return rows
228

```

```

229 def main():
230     parser = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
231     parser.add_argument('--configs', required=True, help='JSON object mapping configuration names to dictionaries')
232     parser.add_argument('--output', required=True, help='New output directory; existing outputs are refused')
233     parser.add_argument('--seed', type=int, default=940000)
234     parser.add_argument('--count', type=int, default=24, help='Independent physical layouts, cycling six families')
235     parser.add_argument('--error-modes', default='spatial_hash,positive,negative,smooth')
236     parser.add_argument('--roundings', default='bounded,pre_round_stress')
237     parser.add_argument('--workers', type=int, default=1)
238     parser.add_argument('--fixtures', help='Previously generated fixtures: reproduction, never independent
↳ validation')
239     parser.add_argument('--stage', default='development')
240     parser.add_argument('--lock', help='Earlier lock.json or its experiment directory; verifies immutable
↳ source/config/selection')
241     parser.add_argument('--selection', help='Decision/selection JSON or text to persist and hash before generation')
242     parser.add_argument('--reference', help='Defaults to BASE if present, otherwise first configuration')
243     args = parser.parse_args()
244     if not __debug__:
245         parser.error('Python optimization disables assertions in the physical auditor; run without -O')
246     if args.count < 1 or args.workers < 1:
247         parser.error('--count and --workers must be positive')
248     try:
249         error_modes = _csv(args.error_modes, ERROR_MODES)
250         roundings = _csv(args.roundings, ROUNDINGS)
251     except argparse.ArgumentTypeError as exc:
252         parser.error(str(exc))
253     output = Path(args.output).resolve()
254     if output.exists():
255         parser.error(f'Output already exists: {output}')
256     configs = json.loads(Path(args.configs).read_text(encoding='utf-8'))
257     if not isinstance(configs, dict) or not configs or any(not isinstance(value, dict) for value in configs.values()):
258         parser.error('Configuration JSON must be a nonempty object mapping names to dictionaries')
259     if any(not re.fullmatch(r'[A-Za-z0-9][A-Za-z0-9_-]*', name) for name in configs):
260         parser.error('Configuration names must be safe directory names: letters, numbers, _, ., -')
261     reference = args.reference or ('BASE' if 'BASE' in configs else next(iter(configs)))
262     if reference not in configs:
263         parser.error('Reference configuration is absent from --configs')
264     is_validation = 'validation' in args.stage.lower() and not any(
265         word in args.stage.lower() for word in ('reproduction', 'not_independent', 'not_new'))
266     if is_validation and not args.lock:
267         parser.error('Independent validation requires --lock from an earlier frozen selection')
268     if is_validation and args.fixtures:
269         parser.error('Existing fixtures are reproduction; use --stage reproduction_not_independent')
270     hashes = source_hashes()
271     selection_bytes = (Path(args.selection).read_bytes() if args.selection else
272                       json.dumps(dict(configurations=list(configs), reference=reference,
273                                     declaration='Configuration set frozen before layout generation'),
274                                 sort_keys=True).encode())
275     selection_hash = hashlib.sha256(selection_bytes).hexdigest()
276     previous = None
277     lock_path = None
278     if args.lock:
279         lock_path = Path(args.lock).resolve()
280         if lock_path.is_dir():
281             lock_path /= 'lock.json'
282         previous = json.loads(lock_path.read_text(encoding='utf-8'))
283         if previous['code_sha256'] != hashes:
284             changed = sorted(key for key in set(previous['code_sha256']) | set(hashes)
285                             if previous['code_sha256'].get(key) != hashes.get(key))
286             parser.error('Source changed after lock: ' + ', '.join(changed))
287         if any(previous['configs'].get(name) != config for name, config in configs.items()):
288             parser.error('Configuration changed after lock')
289         if args.selection and previous['selection_sha256'] != selection_hash:
290             parser.error('Selection decision changed after lock')
291         if not args.selection:
292             selection_bytes = (lock_path.parent / 'selection_snapshot.txt').read_bytes()
293             selection_hash = hashlib.sha256(selection_bytes).hexdigest()
294             if selection_hash != previous['selection_sha256']:
295                 parser.error('Previous selection snapshot does not match its lock')
296         if previous.get('reference') != reference:
297             parser.error('Reference configuration changed after lock')
298     lock = dict(schema_version=1, stage=args.stage, created_unix=time.time(),
299               code_sha256=hashes, code_bundle_sha256=canonical_sha256(hashes),

```

```

300         configs=configs, configs_sha256=canonical_sha256(configs),
301         selection_sha256=selection_hash, reference=reference,
302         prior_lock=str(lock_path) if lock_path else None,
303         prior_lock_sha256=hashlib.sha256(lock_path.read_bytes()).hexdigest() if lock_path else None,
304         generator_seed=None if args.fixtures else args.seed,
305         generator_count=None if args.fixtures else args.count,
306         families=list(FAMILIES), error_modes=error_modes, roundings=roundings,
307         fixture_source=str(Path(args.fixtures).resolve()) if args.fixtures else None,
308         statistical_unit='physical layout', failure_penalty_s=FAILURE_PENALTY_S)
309 output.mkdir(parents=True, exist_ok=False)
310 write_json(output / 'configs.json', configs)
311 (output / 'selection_snapshot.txt').write_bytes(selection_bytes)
312 write_json(output / 'lock.json', lock)
313 for name, digest in hashes.items():
314     destination = output / 'code_snapshot' / name
315     destination.parent.mkdir(parents=True, exist_ok=True)
316     shutil.copy2(PROJECT / name, destination)
317     if hashlib.sha256(destination.read_bytes()).hexdigest() != digest:
318         raise RuntimeError('Source changed during snapshot: ' + name)
319 write_json(output / 'snapshot_complete.json', dict(completed_unix=time.time(),
↵ code_bundle_sha256=lock['code_bundle_sha256']))
320 # No case generation or fixture parsing occurs before the persisted lock/snapshot.
321 layouts = _load_layouts(args)
322 cases = _expand_cases(layouts, error_modes, roundings)
323 identities = {case['layout_sha256'] for case in cases}
324 if is_validation and lock_path:
325     old_fixtures = lock_path.parent / 'fixtures.json'
326     if old_fixtures.exists():
327         old_cases = json.loads(old_fixtures.read_text(encoding='utf-8'))
328         overlap = identities & {_layout_identity(case) for case in old_cases}
329         if overlap:
330             raise RuntimeError(f'Independent validation reuses {len(overlap)} layouts from its lock batch')
331 write_json(output / 'fixtures.json', cases)
332 provenance = dict(stage=args.stage, independent_new_layouts=not bool(args.fixtures),
333                 independent_validation=bool(is_validation and not args.fixtures),
334                 reuse_label='reproduction_not_independent' if args.fixtures else 'new_generated_layouts',
335                 fixture_source=lock['fixture_source'], layouts=len(identities), scenarios=len(cases),
336                 algorithm_runs=len(cases) * len(configs), error_modes=error_modes, roundings=roundings,
337                 family_layout_counts=dict(Counter(layout.get('family', 'fixture') for layout in layouts)),
338                 layout_sha256=sorted(identities), fixtures_sha256=canonical_sha256(cases),
339                 generated_after_snapshot_unix=time.time(), official_runs=0, http_runs=0,
340                 lock_sha256=hashlib.sha256((output / 'lock.json').read_bytes()).hexdigest())
341 write_json(output / 'metadata.json', provenance)
342 print(f'START stage={args.stage} layouts={len(identities)} scenarios={len(cases)} '
343       f'configs={len(configs)} runs={provenance["algorithm_runs"]} workers={args.workers}', flush=True)
344 rows = []
345 started = time.perf_counter()
346 environment = dict(python=sys.version, numpy=np.__version__, scipy=scipy.__version__,
↵ platform=platform.platform())
347
348 def persist(completed, invalid_reason=None):
349     ordered = sorted(rows, key=lambda row: (row['case_id'], list(configs).index(row['config'])))
350     write_json(output / 'summary.json', dict(stage=args.stage, provenance=provenance, environment=environment,
351                                     elapsed_s=time.perf_counter() - started,
352                                     completed_scenarios=completed, expected_scenarios=len(cases),
353                                     batch_invalid_reason=invalid_reason,
354                                     rows=ordered))
355     print(f'PROGRESS scenarios={completed}/{len(cases)} runs={len(rows)}/{provenance["algorithm_runs"]} '
356           f'failed={sum(not row["success"] for row in rows)} elapsed_s={time.perf_counter() - started:.1f}',
↵ flush=True)
357
358 tasks = [(case, configs, str(output), args.stage, reference) for case in cases]
359 pool, pending, completed, active_case = None, {}, 0, None
360 try:
361     if args.workers == 1:
362         for task in tasks:
363             active_case = task[0]['id']
364             rows.extend(_run_case(task))
365             completed += 1
366             persist(completed)
367     else:
368         pool = ProcessPoolExecutor(max_workers=args.workers)
369         pending = {pool.submit(_run_case, task): task[0]['id'] for task in tasks}

```

```

370         for future in as_completed(pending):
371             active_case = pending[future]
372             rows.extend(future.result())
373             completed += 1
374             persist(completed)
375         active_case = None
376         stable_sources = source_hashes() == hashes
377         write_json(output / 'integrity.json', dict(source_unchanged_during_run=stable_sources,
378                                                    finished_unix=time.time()))
379         if not stable_sources:
380             raise RuntimeError('Experimental source changed during execution; batch is not a locked result')
381         report = analyze(rows, reference=reference)
382         report['provenance'] = provenance
383         write_json(output / 'comparison.json', report)
384         print(json.dumps({name: {key: value[key] for key in ('complete', 'runs', 'mean_s', 'p95_s', 'max_s',
385                                                             'gain_ratio_of_means_pct')}}
386                   for name, value in report['configurations'].items()), indent=2), flush=True)
387     except BaseException as exc:
388         failure = dict(batch_valid=False, kind='framework_failure', failure=f'{type(exc).__name__}: {exc}',
389                       traceback=traceback.format_exc(), active_case_id=active_case,
390                       completed_scenarios=completed, expected_scenarios=len(cases),
391                       retained_summary_runs=len(rows), expected_runs=provenance['algorithm_runs'],
392                       failed_unix=time.time(),
393                       interpretation='Batch invalid; absent runs are not fabricated policy performance
394                                   ↳ observations.')
395         write_json(output / 'batch_failure.json', failure)
396         persist(completed, invalid_reason=failure['failure'])
397         for future in pending:
398             future.cancel()
399         print('BATCH INVALID: ' + failure['failure'], flush=True)
400         raise
401     finally:
402         if pool is not None:
403             pool.shutdown(wait=True, cancel_futures=True)
404     if any(not row['success'] for row in rows):
405         raise SystemExit(1)
406
407 if __name__ == '__main__':
408     main()

```

程序 33

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/co

de_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/geometry.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/innovation/geometry.py; question4/innovation/geometry.py

```

1 """Geometry for directional experiments, using only public positive bearings.
2
3 Distance screens and information gates do NOT certify directional reception.
4 """
5 import math
6 import numpy as np
7 from question4.geometry import bearing_clip, estimate, EPS
8
9
10 def contains(poly, p):
11     edges = np.roll(poly, -1, axis=0) - poly
12     delta = np.asarray(p) - poly
13     cross = edges[:, 0]*delta[:, 1] - edges[:, 1]*delta[:, 0]
14     return bool(np.all(cross >= -EPS) or np.all(cross <= EPS))
15
16
17 def conditional_worthwhile(poly, point, fraction=.5, bins=16):
18     """All possible *positive* output intervals must halve enclosing radius.
19
20     Widening each interval by its half-width covers every serialized bearing.
21     A no_signal outcome is explicitly excluded and conveys no spatial update.
22     This is a contraction gate, not a probability or net time guarantee.

```

```

23     """
24     _, old = estimate(poly)
25     if old < 20-1e-5:
26         return False
27     point = np.asarray(point)
28     if contains(poly, point):
29         low, high = 0., 360.
30     else:
31         ref = math.degrees(math.atan2(*(poly.mean(axis=0)-point)[::-1]))
32         raw = np.degrees(np.arctan2((poly-point)[:, 1], (poly-point)[:, 0]))
33         angles = ref+(raw-ref+180) % 360-180
34         low, high = float(angles.min()-1.005), float(angles.max()+1.005)
35     width = (high-low)/bins
36     worst = 0.
37     for k in range(bins):
38         try:
39             posterior = bearing_clip(poly, point, low+(k+.5)*width, 1.005+width/2)
40         except RuntimeError:
41             continue
42         worst = max(worst, estimate(posterior)[1])
43     return worst < max(20-1e-5, old*fraction)-1e-5
44
45
46 def proposals(poly, angle, current, mode):
47     center, radius = estimate(poly)
48     if mode == 'center':
49         return [center]
50     a = math.radians(angle)
51     side = np.array([-math.sin(a), math.cos(a)])
52     offset = math.sqrt(radius*20.)
53     points = [center-offset*side, center+offset*side]
54     return sorted(points, key=lambda p: math.dist(current, p))
55
56
57 def optical_order(points, current, mode='original'):
58     """Permute an unchanged finite covering grid; never discard a point."""
59     n = len(points)
60     if mode == 'original':
61         return list(range(n))
62     if mode != 'nearest_branch':
63         raise ValueError('Unknown optical order')
64     k = min(range(n), key=lambda i: (math.dist(current, points[i]), i))
65     orders = [list(range(k, -1, -1))+list(range(k+1, n)),
66              list(range(k, n))+list(range(k-1, -1, -1))]
67     def distance(order):
68         return math.dist(current, points[order[0]])+sum(
69             math.dist(points[a], points[b]) for a, b in zip(order, order[1:]))
70     return min(orders, key=distance)

```

程序 34

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development

pment/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism

m_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/lock.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/lock.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/innovation/lock.py; question4/innovation/lock.py

```

1 """Freeze selection and complete independent design without generating a layout."""
2 import argparse
3 import hashlib
4 import json
5 import time
6 from pathlib import Path
7 from .experiment import source_hashes, canonical_sha256, write_json, FAMILIES
8
9
10 def main():
11     parser = argparse.ArgumentParser(description=__doc__)
12     for name in ('configs', 'selection', 'output'):
13         parser.add_argument('--'+name, required=True)
14     parser.add_argument('--seed', type=int, required=True)
15     parser.add_argument('--count', type=int, required=True)
16     parser.add_argument('--error-modes', default='spatial_hash,positive,negative,smooth')
17     parser.add_argument('--roundings', default='bounded,pre_round_stress')
18     parser.add_argument('--evidence', nargs='+', required=True)
19     args = parser.parse_args()
20     out = Path(args.output)
21     out.mkdir(parents=True, exist_ok=False)
22     configs = json.loads(Path(args.configs).read_text())
23     selection = Path(args.selection).read_bytes()
24     hashes = source_hashes()
25     evidence = {}
26     for folder in args.evidence:
27         for name in ('lock.json', 'summary.json', 'comparison.json', 'integrity.json'):
28             path = Path(folder)/name
29             evidence[str(path)] = hashlib.sha256(path.read_bytes()).hexdigest()
30     lock = dict(schema_version=1, stage='selection_lock_without_generation', created_unix=time.time(),
31                configs=configs, configs_sha256=canonical_sha256(configs),
32                code_sha256=hashes, code_bundle_sha256=canonical_sha256(hashes),
33                reference='BASE', selection_sha256=hashlib.sha256(selection).hexdigest(),
34                generator_seed=args.seed, generator_count=args.count, families=list(FAMILIES),
35                error_modes=args.error_modes.split(','), roundings=args.roundings.split(','),
36                failure_penalty_s=360000., evidence_sha256=evidence)
37     write_json(out/'lock.json', lock)
38     (out/'selection_snapshot.txt').write_bytes(selection)
39     write_json(out/'configs.json', configs)
40     print(json.dumps(dict(output=str(out), code_bundle_sha256=lock['code_bundle_sha256'],
41                          layouts_generated=0, design_layouts=args.count)))
42
43
44 if __name__ == '__main__':
45     main()

```

程序 35

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/innovation/reception.py;

paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/reception.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/innovation/reception.py; question4/innovation/reception.py

```

1  """Conservative directional shadow certificates from public observations only.
2
3  A negative observation n is known to be on the source's back side only when
4  the entire current source region lies strictly within its minimum 1000 m
5  reception radius. For positive observations a1, a2, a strict cone interior
6  point q = n + l1*(n-a1) + l2*(n-a2), l1,l2 > 0, is also on that back side:
7  h(q) = (l1+l2)*h(n)-l1*h(a1)-l2*h(a2) < 0.
8
9  The caller supplies previously observed positions. A distance check alone
10 never supplies either the negative direction fact or a positive observation.
11 """
12 from itertools import combinations
13 import math
14 import numpy as np
15
16
17 def shadow_certificate(poly, positive_points, negative_points, q):
18     """Return a two-positive-point shadow witness, or None when uncertain.
19
20     Collinear, near-collinear, cone-boundary and numerically ill-conditioned
21     cases are deliberately left unproved. The certificate is JSON-serializable.
22     """
23     try:
24         poly = np.asarray(poly, dtype=float)
25         positive = np.asarray(positive_points, dtype=float).reshape(-1, 2)
26         negative = np.asarray(negative_points, dtype=float).reshape(-1, 2)
27         query = np.asarray(q, dtype=float)
28     except (TypeError, ValueError):
29         return None
30     if (poly.ndim != 2 or poly.shape[1] != 2 or not len(poly)
31         or query.shape != (2,) or len(positive) < 2 or not len(negative)
32         or not all(np.isfinite(array).all() for array in (poly, positive, negative, query))):
33         return None
34     for n in negative:
35         # Convexity of the disk makes the all-vertex check sufficient. Strict
36         # margin avoids certifying a no_signal caused by an uncertain radius.
37         if np.linalg.norm(poly-n, axis=1).max() >= 1000.-1e-5:
38             continue
39         target = query-n
40         target_norm = float(np.linalg.norm(target))
41         if target_norm < 1e-5:
42             continue
43         for a1, a2 in combinations(positive, 2):
44             u, v = n-a1, n-a2
45             unorm, vnorm = float(np.linalg.norm(u)), float(np.linalg.norm(v))
46             if min(unorm, vnorm) < 1e-5:
47                 continue
48             determinant = float(u[0]*v[1]-u[1]*v[0])
49             if abs(determinant) <= 1e-7*unorm*vnorm:
50                 continue
51             l1 = float((target[0]*v[1]-target[1]*v[0])/determinant)
52             l2 = float((u[0]*target[1]-u[1]*target[0])/determinant)
53             if not all(math.isfinite(value) and 1e-7 < value < 1e6 for value in (l1, l2)):
54                 continue
55             # Also require an angular margin from each cone boundary. Merely
56             # positive rounded coefficients can be unreliable near a ray.
57             cross_u = abs(float(u[0]*target[1]-u[1]*target[0]))
58             cross_v = abs(float(v[0]*target[1]-v[1]*target[0]))
59             if min(cross_u/(unorm*target_norm), cross_v/(vnorm*target_norm)) <= 1e-7:
60                 continue
61             residual = np.linalg.norm(target-l1*u-l2*v)
62             if residual > 1e-8*max(1., target_norm):
63                 continue
64             return dict(negative_point=n.tolist(), positive_points=[a1.tolist(), a2.tolist()],

```

```
65         coefficients=[l1, l2])
66     return None
```

程序 36

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovat

ion/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/innovation/strategy.py; question4/innovation/strategy.py

```

1 """Experimental mechanisms with a frozen Q4 public-feedback policy as parent.
2
3 No truth, files, random seeds, source types, radii or orientations are accessed.
4 The 25-station discovery and finite optical fallback certificates are retained.
5 """
6 import math
7 import numpy as np
8 from question4.strategy import Strategy as Baseline, run as baseline_run
9 from question4.geometry import estimate, optical_cover
10 from .geometry import proposals, conditional_worthwhile, contains, optical_order
11
12 BASELINE_CONFIG = dict(scheduling='route', mesh_kind='radial', scan_known='certified',
13                        centroid_route=True, known_count_stop=True, scan_information=True)
14 DEFAULT = dict(mechanism='pair', sharing='off', adaptive_steps=8,
15               share_after_clear=True, adaptive_gate=False, optical_order='original',
16               shadow_skip=False, blind_clear=False)
17
18
19 class Strategy(Baseline):
20     def __init__(self, client, config, log=lambda row: None):
21         extras = {k: v for k, v in config.items() if k != 'baseline_config'}
22         if set(extras)-set(DEFAULT):
23             raise ValueError('Unknown innovation config: '+str(set(extras)-set(DEFAULT)))
24         self.options = DEFAULT | extras
25         if self.options['mechanism'] not in ('pair', 'center', 'parallax'):
26             raise ValueError('Unknown localization mechanism')
27         if self.options['sharing'] not in ('off', 'range', 'gated'):
28             raise ValueError('Unknown sharing mechanism')
29         if self.options['optical_order'] not in ('original', 'nearest_branch'):
30             raise ValueError('Unknown optical order')
31         for key in ('share_after_clear', 'adaptive_gate', 'shadow_skip', 'blind_clear'):
32             if not isinstance(self.options[key], bool):
33                 raise ValueError('Invalid boolean '+key)
34         if not isinstance(self.options['adaptive_steps'], int) or not 1 <= self.options['adaptive_steps'] <= 16:
35             raise ValueError('Invalid finite step bound')
36         super().__init__(client, config.get('baseline_config', BASELINE_CONFIG), log)
37         self.visited = {}
38         self.in_shared = False
39
40     def observe(self, p, ch, phase):
41         before = tuple(self.c.position)
42         old = self.known[ch]['poly'].copy() if ch in self.known else None
43         kind = super().observe(p, ch, phase)
44         self.visited.setdefault(ch, set()).add(tuple(map(float, p)))
45         if kind == 'no_signal' and old is not None:
46             assert np.array_equal(old, self.known[ch]['poly'])
47         if ch in self.known:
48             self.log(dict(event='feasible_region', channel=int(ch), kind=kind,
49                          vertices=self.known[ch]['poly'].tolist(), point=list(p)))
50         if phase != 'search' and not self.in_shared and math.dist(before, self.c.position) > 1e-6:
51             self.share(ch)
52         return kind
53
54     def clear(self, p, ch, phase):
55         before = tuple(self.c.position)
56         ok = super().clear(p, ch, phase)
57         if (phase != 'near' and self.options['share_after_clear'] and not self.in_shared
58             and math.dist(before, self.c.position) > 1e-6):
59             self.share(ch)
60         return ok
61

```



```

62 def share(self, active):
63     mode = self.options['sharing']
64     if mode == 'off':
65         return
66     self._in_shared = True
67     try:
68         p = np.asarray(self.c.position)
69         for ch in sorted(set(self.known)-{active}):
70             if tuple(p) in self.visited.get(ch, set()):
71                 continue
72             poly = self.known[ch]['poly']
73             _, radius = estimate(poly)
74             # Q3's distance condition is used ONLY as a cost screen. No
75             # assumption that the source faces this point follows from it.
76             if radius < 20-1e-5 or np.linalg.norm(poly-p, axis=1).max() >= 1000-1e-5:
77                 continue
78             if mode == 'gated' and not conditional_worthwhile(poly, p):
79                 self.log(dict(event='conditional_gate_skip', channel=int(ch), point=p.tolist()))
80                 continue
81             if self.options['shadow_skip']:
82                 from .reception import shadow_certificate
83                 certificate = shadow_certificate(poly, [a for a, _ in self.readings[ch]],
84                                                 self.missed.get(ch, []), p)
85                 if certificate is not None:
86                     self.log(dict(event='directional_shadow_skip', channel=int(ch), point=p.tolist(),
87                                 vertices=poly.tolist(), certificate=certificate))
88                     continue
89                 self.log(dict(event='shared_attempt', channel=int(ch), point=p.tolist(),
90                             reception_guaranteed=False, conditional_gate=mode == 'gated'))
91                 self.observe(p, ch, 'shared_localization')
92     finally:
93         self._in_shared = False
94
95 def optical_fallback(self, ch):
96     data = self.known[ch]
97     poly = data['poly']; center, radius = estimate(poly)
98     if radius < 20-1e-5:
99         self.log(dict(event='certified_clear', channel=int(ch), point=center.tolist(),
100                     radius=radius, vertices=poly.tolist()))
101         if not self.clear(center, ch, 'certified_clear'):
102             raise RuntimeError('Certified clear failed')
103         return
104     points, radius = optical_cover(poly, data['angle'], self.c.position)
105     self.log(dict(event='optical_cover', channel=int(ch), vertices=poly.tolist(),
106                 cover_radius=radius, points=len(points), angle=data['angle']))
107     order = optical_order(points, self.c.position, self.options['optical_order'])
108     self.log(dict(event='optical_route', channel=int(ch), order=order,
109                 start=list(self.c.position), method=self.options['optical_order']))
110     for j, index in enumerate(order):
111         p = points[index]
112         self.log(dict(event='optical_step', channel=int(ch), entry=j == 0,
113                     point=list(p), movement_s=math.dist(self.c.position, p)/5))
114         if self.clear(p, ch, 'optical_fallback'):
115             return
116     raise RuntimeError('Exhausted optical certificate without success')
117
118 def localize(self, ch):
119     mode = self.options['mechanism']
120     if mode == 'pair':
121         if self.options['optical_order'] == 'original':
122             return super().localize(ch)
123         # Preserve the fixed-pair measurement policy; only optical ordering
124         # differs from the frozen parent. Baseline refinement is not used.
125         if self.cfg['refinement'] or self.cfg['opportunistic_clear']:
126             raise ValueError('Optical-order ablation requires unmodified pair parent')
127         data = self.known[ch]; center, radius = estimate(data['poly'])
128         if self.cfg['localization'] == 'pair' and radius >= 20-1e-5 and len(self.readings[ch]) < 2:
129             a = math.radians(data['angle']); side = np.array([-math.sin(a), math.cos(a)])
130             center = self.target(data) if self.cfg['centroid_probe'] else center
131             points = sorted([center+self.cfg['probe_width']*side, center-self.cfg['probe_width']*side],
132                             key=lambda p: math.dist(self.c.position, p))
133             for p in points:
134                 self.observe(p, ch, 'localize')

```

```

135         if ch in self.cleared:
136             return
137         if len(self.readings[ch]) >= 2:
138             break
139         return self.optical_fallback(ch)
140     for step in range(self.options['adaptive_steps']):
141         data = self.known[ch]; poly = data['poly']
142         center, radius = estimate(poly)
143         if radius < 20*1e-5:
144             break
145         points = proposals(poly, data['angle'], self.c.position, mode)
146         found = False
147         for p in points:
148             if tuple(p) in self.visited.get(ch, set()):
149                 continue
150             if self.options['adaptive_gate'] and not conditional_worthwhile(poly, p):
151                 continue
152             self.log(dict(event='adaptive_attempt', channel=int(ch), step=step,
153                          point=p.tolist(), radius=radius, reception_guaranteed=False))
154             kind = self.observe(p, ch, 'adaptive_localization')
155             if ch in self.cleared:
156                 return
157             if kind == 'direction':
158                 found = True
159                 break
160             if self.options['blind_clear'] and contains(self.known[ch]['poly'], p):
161                 # Radio has already paid for this movement. A 3-second
162                 # optical attempt can succeed on the blind side, even near.
163                 self.log(dict(event='blind_optical_attempt', channel=int(ch), point=list(p)))
164                 if self.clear(p, ch, 'blind_optical_attempt'):
165                     return
166             if not found or estimate(self.known[ch]['poly'])[1] > radius*.9:
167                 break
168         self.optical_fallback(ch)
169
170
171 def run(client, config=None, log=lambda row: None):
172     config = config or {}
173     if config.get('mechanism') == 'baseline':
174         if set(config)-{'mechanism', 'baseline_config'}:
175             raise ValueError('Baseline must not contain experimental options')
176         return baseline_run(client, config.get('baseline_config', BASELINE_CONFIG), log)
177     return Strategy(client, config, log).run()

```

程序 37

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/t

est_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round03_w
indow/development/code_snapshot/question4/innovation/test_mechanisms.py;
paper_evidence/q4_mechanism_reset_20260912/round04_search_first/developm
ent/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/ques
tion4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_
20260912/round04_search_first/development/code_snapshot/question4/innova
tion/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/rou
nd05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_
20260912/frozen/question4/innovation/test_mechanisms.py; paper_evidence
/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/qu
estion4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_rese
t_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_
mechanism_reset_20260912/frozen/question4/innovation/test_mechanisms.py;
paper_evidence/q4_mechanism_reset_20260912/round06_exchange/developmen
t/code_snapshot/question4/innovation/test_mechanisms.py; paper_evidence
/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/p
aper_evidence/q4_mechanism_reset_20260912/frozen/question4/innovation/te
st_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round07_ela
stic/development/code_snapshot/question4/innovation/test_mechanisms.py;
paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/devel
opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen
/question4/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_re
set_20260912/round08_optical_cells/development/code_snapshot/question4
/innovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_202609
12/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_m
echanism_reset_20260912/frozen/question4/innovation/test_mechanisms.py;
paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/devel
opment/code_snapshot/question4/innovation/test_mechanisms.py; paper_eviden
ce/q4_mechanism_reset_20260912/round10_negative_region/development/code_
snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/inn
ovation/test_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/rou
nd10_negative_region/development/code_snapshot/question4/innovation/test
_mechanisms.py; question4/innovation/test_mechanisms.py

```

1 """Self-only adversarial tests for the directional mechanism migration."""
2 import copy
3 import math
4 import unittest
5 import numpy as np
6 from question3.interface import Client
7 from question4.audit import audit as audit_actions
8 from question4.geometry import first_region, bearing_clip, estimate
9 from question4.simulator import Simulator
10 from .audit import audit_regions, _inside
11 from .geometry import proposals, conditional_worthwhile
12 from .strategy import Strategy, run
13 from .reception import shadow_certificate
14
15
16 def source(channel=1, x=100., y=0., direction=180., radius=1000.):
17     return dict(channel=channel, x=x, y=y, radius=radius, direction_deg=direction)

```

```

18
19
20 def setup(sources, config=None, error_mode='zero', rounding='bounded'):
21     case = dict(seed=6197, sources=sources, error_mode=error_mode)
22     records, events = [], []
23     client = Client(Simulator(case, rounding=rounding), 'SELF', records.append)
24     policy = Strategy(client, config or {}, events.append)
25     client.enter()
26     return case, client, policy, records, events
27
28
29 class DirectionalMechanismTests(unittest.TestCase):
30     def test_near_and_exact_direction_range_boundaries(self):
31         _, client, _, _ = setup([source(x=0., direction=0.)])
32         for point, expected in [((-4., 0.), 'no_signal'), ((4., 0.), 'near'),
33                                 ((0., 1000.), 'direction'), ((-1e-5, 999.), 'no_signal'),
34                                 ((1e-5, 999.), 'direction'), ((1000., 0.), 'direction'),
35                                 ((1000.0001, 0.), 'no_signal')]:
36             with self.subTest(point=point):
37                 self.assertEqual(client.measure(point, 1)['measure_result'], expected)
38             self.assertEqual(client.clear((-20., 0.), 1)['clear_result'], 'success')
39
40     def test_positive_station_convex_hull_reception(self):
41         rng = np.random.default_rng(2937)
42         for axis in (0., 37., 90., 179.99, 270.):
43             theta = math.radians(axis)
44             rotate = np.array([[math.cos(theta), -math.sin(theta)], [math.sin(theta), math.cos(theta)]])
45             positive = np.array([[0., 900.], [0., -900.], [900., 0.]]) @ rotate.T
46             _, client, _, _ = setup([source(x=0., direction=axis)])
47             for point in positive:
48                 self.assertNotEqual(client.measure(point, 1)['measure_result'], 'no_signal')
49             for weights in rng.dirichlet(np.ones(3), 30):
50                 point = weights @ positive
51                 self.assertNotEqual(client.measure(point, 1)['measure_result'], 'no_signal')
52             backside = np.array([-4., 0.]) @ rotate.T
53             self.assertEqual(client.measure(backside, 1)['measure_result'], 'no_signal')
54
55     def test_no_signal_preserves_region_and_finite_fallback(self):
56         for mode in ('center', 'parallax'):
57             case, client, policy, _, events = setup([source()], dict(mechanism=mode, adaptive_steps=16))
58             policy.observe((0., 0.), 1, 'search')
59             original = policy.known[1]['poly'].copy()
60             policy.localize(1)
61             attempts = [row for row in events if row.get('event') == 'adaptive_attempt']
62             negative = [row for row in events if row.get('event') == 'feasible_region' and row['kind'] == 'no_signal']
63             self.assertTrue(negative)
64             self.assertLessEqual(len(attempts), 2)
65             self.assertTrue(all(np.array_equal(original, row['vertices']) for row in negative))
66             self.assertIn(1, policy.cleared)
67             self.assertTrue(any(row.get('event') == 'optical_cover' for row in events))
68             self.assertLess(client.counts['clear'], 120)
69             self.assertTrue(audit_regions(case, events)['passed'])
70
71     def test_conditional_positive_gate_can_receive_no_signal(self):
72         case, client, policy, _, events = setup([source()], dict(sharing='gated'))
73         policy.observe((0., 0.), 1, 'search')
74         poly = policy.known[1]['poly'].copy()
75         point = proposals(poly, 0., client.position, 'parallax')[0]
76         self.assertTrue(conditional_worthwhile(poly, point))
77         self.assertLess(np.linalg.norm(poly-point, axis=1).max(), 1000)
78         client.measure(point, 20, 'test_positioning')
79         policy.share(20)
80         self.assertTrue(np.array_equal(poly, policy.known[1]['poly']))
81         self.assertEqual(events[-1]['kind'], 'no_signal')
82         attempt = next(row for row in events if row.get('event') == 'shared_attempt')
83         self.assertFalse(attempt['reception_guaranteed'])
84         self.assertTrue(attempt['conditional_gate'])
85         measured = client.counts['measure']
86         policy.share(20)
87         self.assertEqual(client.counts['measure'], measured, 'Fixed point/channel must not be retried')
88         self.assertEqual(audit_regions(case, events)['checked_no_signal_unchanged'], 1)
89
90     def test_shared_near_clears_multiple_pending_channels(self):

```

```

91     case, client, policy, _, events = setup(
92         [source(2, 200., direction=None), source(3, 202., direction=None)], dict(sharing='gated'))
93     policy.observe((-500., 0.), 2, 'search')
94     policy.observe((-500., 0.), 3, 'search')
95     client.measure((201., 0.), 1, 'test_positioning')
96     # Near outcomes remain possible when this conservative conditional gate
97     # rejects an expensive bearing: range sharing isolates safe mutation.
98     policy.options['sharing'] = 'range'
99     policy.share(1)
100    self.assertEqual(policy.cleared, {2, 3})
101    self.assertFalse(policy.known)
102    self.assertFalse(policy._in_shared)
103    self.assertEqual(client.counts['clear_success'], 2)
104    self.assertEqual(client.channel, 3, 'Clearing must not switch the measurement channel')
105    self.assertTrue(audit_regions(case, events)['passed'])
106
107    def test_extreme_bearing_error_keeps_true_source(self):
108        rng = np.random.default_rng(616027)
109        for index in range(240):
110            angle = rng.uniform(-math.pi, math.pi)
111            radius = (1800. if index % 3 == 0 else rng.uniform(0., 1800.))
112            truth = radius*np.array([math.cos(angle), math.sin(angle)])
113            poly = None
114            events = []
115            for k in range(3):
116                bearing = (359.999 if index % 5 == 0 and k == 0 else rng.uniform(0., 360.))
117                error = (-1.005 if (index+k) % 2 else 1.005)
118                distance = (1500. if index % 4 == 0 else rng.uniform(5.01, 1500.))
119                theta = math.radians(bearing)
120                point = truth-distance*np.array([math.cos(theta), math.sin(theta)])
121                observed = (bearing+error) % 360
122                poly = first_region(point, observed) if poly is None else bearing_clip(poly, point, observed)
123                events.append(dict(event='feasible_region', channel=1, kind='direction', vertices=poly.tolist()))
124            case = dict(sources=[source(x=float(truth[0]), y=float(truth[1]))])
125            self.assertEqual(audit_regions(case, events)['checked_regions'], 3)
126
127    def test_conditional_gate_covers_serialized_positive_outputs(self):
128        poly = first_region((0., 0.), 359.999)
129        point = proposals(poly, 359.999, (0., 0.), 'parallax')[0]
130        self.assertTrue(conditional_worthwhile(poly, point))
131        _, old = estimate(poly)
132        # Enumerate possible true positions along polygon chords, with both
133        # maximal errors. Compare posterior vertices to an independent circle.
134        for blend in np.linspace(0., 1., 11):
135            for vertex in poly:
136                truth = blend*vertex+(1-blend)*poly.mean(axis=0)
137                true_angle = math.degrees(math.atan2(*(truth-point)[: -1]))
138                for error in (-1.005, 1.005):
139                    posterior = bearing_clip(poly, point, (true_angle+error) % 360)
140                    self.assertTrue(_inside(posterior, truth))
141                    # The conditional gate promises an available covering
142                    # circle, not an exact half-radius for every estimator.
143                    center = posterior.mean(axis=0)
144                    self.assertLess(np.linalg.norm(posterior-center, axis=1).max(), old)
145
146    def test_independent_audit_rejects_tampered_region_and_circles(self):
147        case = dict(sources=[source(x=0., y=0.)])
148        vertices = [[-10., -10.], [10., -10.], [10., 10.], [-10., 10.]]
149        event = dict(event='certified_clear', channel=1, vertices=vertices, point=[0., 0.], radius=math.sqrt(200))
150        self.assertTrue(audit_regions(case, [event])['passed'])
151        for mutation in ({'point': [50., 0.], {'radius': 5.}, {'vertices': [[30., 0.], [40., 1.], [40., -1.]]}):
152            altered = copy.deepcopy(event); altered.update(mutation)
153            with self.assertRaises(AssertionError):
154                audit_regions(case, [altered])
155        optical = dict(event='optical_cover', channel=1, vertices=vertices, angle=0.,
156            points=1, cover_radius=math.sqrt(2)*(10.+1e-7))
157        self.assertEqual(audit_regions(case, [optical])['checked_optical_covers'], 1)
158        for mutation in ({'points': 2}, {'cover_radius': 20.}, {'cover_radius': 1.}):
159            altered = copy.deepcopy(optical); altered.update(mutation)
160            with self.assertRaises(AssertionError):
161                audit_regions(case, [altered])
162        old = dict(event='feasible_region', channel=1, vertices=vertices, kind='direction')

```

```

163         changed = dict(event='feasible_region', channel=1, vertices=(np.array(vertices)*.9).tolist(),
↪ kind='no_signal')
164     with self.assertRaises(AssertionError):
165         audit_regions(case, [old, changed])
166     self.assertFalse(_inside(np.array([[0., 0.], [1., 0.]]), (2., 0.)))
167
168     def test_complete_ten_and_sixteen_sources_late_channel_boundaries(self):
169         for count, error, rounding in [(10, 'positive', 'bounded'), (16, 'negative', 'pre_round_stress')]:
170             channels = list(range(1, count))+[20]
171             sources = []
172             for index, channel in enumerate(channels):
173                 theta = 2*math.pi*index/count
174                 axis = math.degrees(theta) if index % 2 == 0 else None
175                 sources.append(source(channel, 1800*math.cos(theta), 1800*math.sin(theta), axis))
176             case = dict(sources=sources, seed=67213+count, error_mode=error)
177             actions, events = [], []
178             client = Client(Simulator(case, rounding=rounding), 'SELF', actions.append)
179             result = run(client, dict(mechanism='parallax', sharing='gated'), events.append)
180             self.assertEqual(len(result['cleared']), count)
181             self.assertIn(20, result['cleared'])
182             self.assertTrue(audit_actions(case, actions, result)['passed'])
183             self.assertTrue(audit_regions(case, events)['passed'])
184             self.assertEqual(result['stop_certificate']['kind'], 'count16' if count == 16 else
↪ 'triangular_halfplane_cover')
185
186     def test_directional_shadow_random_physics_and_independent_audit(self):
187         rng = np.random.default_rng(749013)
188         for index in range(400):
189             truth = rng.uniform(-1100., 1100., 2)
190             angle = rng.uniform(0., 360.)
191             theta = math.radians(angle)
192             rotation = np.array([[math.cos(theta), -math.sin(theta)], [math.sin(theta), math.cos(theta)]])
193             extents = rng.uniform(1., 75., 2)
194             poly = truth+np.array([[-1, -1], [1, -1], [1, 1], [-1, 1]])*extents
195             n = truth+np.array([-rng.uniform(10., 650.), rng.uniform(-150., 150.)]) @ rotation.T
196             positives = truth+np.array([[rng.uniform(10., 300.), rng.uniform(100., 500.)],
197                                     [rng.uniform(10., 300.), -rng.uniform(100., 500.)]]) @ rotation.T
198             coefficients = rng.uniform(.01, 2., 2)
199             query = n+coefficients @ (n-positives)
200             proof = shadow_certificate(poly, positives, [n], query)
201             self.assertIsNotNone(proof)
202             model = source(x=float(truth[0]), y=float(truth[1]), direction=angle, radius=rng.uniform(1000., 1500.))
203             case, client, _, _ = setup([model])
204             events = []
205             for p in positives:
206                 response = client.measure(p, 1)
207                 self.assertNotEqual(response['measure_result'], 'no_signal')
208                 events.append(dict(event='observe', channel=1, point=p.tolist(), kind=response['measure_result']))
209             self.assertEqual(client.measure(n, 1)['measure_result'], 'no_signal')
210             events.append(dict(event='observe', channel=1, point=n.tolist(), kind='no_signal'))
211             self.assertEqual(client.measure(query, 1)['measure_result'], 'no_signal')
212             events.append(dict(event='directional_shadow_skip', channel=1, point=query.tolist(),
213                             vertices=poly.tolist(), certificate=proof))
214             self.assertEqual(audit_regions(case, events)['checked_directional_shadows'], 1)
215
216     def test_directional_shadow_rejects_range_cone_boundary_and_degeneracy(self):
217         poly = np.array([[-10., -10.], [10., -10.], [10., 10.], [-10., 10.]])
218         positive = np.array([[100., 100.], [100., -100.]])
219         n = np.array([-100., 0.])
220         self.assertIsNotNone(shadow_certificate(poly, positive, [n], (-300., 0.)))
221         for query in ((0., 0.), (100., 0.), (-100., 200.), n, n+(n-positives[0])):
222             self.assertIsNone(shadow_certificate(poly, positive, [n], query))
223         self.assertIsNone(shadow_certificate(poly, positive, [(-1000., 0.)], (-2000., 0.)))
224         self.assertIsNone(shadow_certificate(poly, positive, [(-989.999999, 0.)], (-2000., 0.)))
225         for flat in ([[100., 0.], [200., 0.], [100., 0.], [200., 1e-10]], [[100., 100.], [100., 100.]]):
226             self.assertIsNone(shadow_certificate(poly, flat, [n], (-300., 0.)))
227         self.assertIsNone(shadow_certificate(poly, positive[:1], [n], (-300., 0.)))
228         self.assertIsNone(shadow_certificate(poly, positive, [], (-300., 0.)))
229
230     def test_independent_shadow_audit_rejects_forward_skip_and_fake_witness(self):
231         poly = [[-10., -10.], [10., -10.], [10., 10.], [-10., 10.]]
232         positive = [[100., 100.], [100., -100.]]
233         n, query = [-100., 0.], [-300., 0.]

```

```

234     case = dict(sources=[source(x=0., direction=0.)])
235     history = [dict(event='observe', channel=1, point=p, kind='direction') for p in positive]
236     history.append(dict(event='observe', channel=1, point=n, kind='no_signal'))
237     event = dict(event='directional_shadow_skip', channel=1, vertices=poly, point=query,
238                 certificate=shadow_certificate(poly, positive, [n], query))
239     self.assertEqual(audit_regions(case, history+[event])['checked_directional_shadows'], 1)
240     for mutation in ({'point': [100., 0.]}, {'point': [-301., 0.]}):
241         changed = copy.deepcopy(event); changed.update(mutation)
242         with self.assertRaises(AssertionError):
243             audit_regions(case, history+[changed])
244     with self.assertRaises(AssertionError):
245         audit_regions(case, [event])
246     omni = dict(sources=[source(x=0., direction=None)])
247     with self.assertRaises(AssertionError):
248         audit_regions(omni, history+[event])
249
250
251 if __name__ == '__main__':
252     unittest.main()

```

程序 38

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/routing.py

stic/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/routing.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/routing.py; question4/innovation/frozen_baseline/question4/routing.py; question4/routing.py

```

1 """Deterministic free-end open tour, replanned after each public action batch."""
2 import numpy as np
3
4 def tour(start,points,source_weights=None,station_flags=None,relocate=False):
5     p=np.vstack([start,points]);d=np.linalg.norm(p[:,None]-p[None,:],axis=2)
6     remaining=set(range(1,len(p)));order=[0]
7     while remaining:
8         j=min(remaining,key=lambda j:(d[order[-1],j],j));order.append(j);remaining.remove(j)
9     # Reverse a segment, including the free endpoint. No return-to-origin penalty.
10    for _ in range(50):
11        weights=np.array([0.]+list(source_weights or [0.]*len(points)))[order]
12        flags=np.array([0.]+list(station_flags or [0.]*len(points)))[order]
13        sw=np.r_[0.,np.cumsum(weights)];sf=np.r_[0.,np.cumsum(flags)]
14        inv=np.r_[0.,np.cumsum(weights*sf[:-1])]
15        best=0.;move=None
16        for i in range(1,len(order)-1):
17            for j in range(i+1,len(order)):
18                delta=d[order[i-1],order[j]]-d[order[i-1],order[i]]
19                if j+1<len(order):delta+=d[order[i],order[j+1]]-d[order[j],order[j+1]]
20                # Clearing a known source saves future station detections.
21                inversion=(inv[j+1]-inv[i])-sf[i]*(sw[j+1]-sw[i])
22                delta+=5*((sf[j+1]-sf[i])*(sw[j+1]-sw[i])-2*inversion)
23                if delta<best-1e-8:best=delta;move=(i,j)
24        if move is None:break
25        i,j=move;order[i:j+1]=reversed(order[i:j+1])
26    if relocate:
27        def cost(o):
28            distance=sum(d[a,b] for a,b in zip(o,o[1:]))
29            weights0=[0.]+list(source_weights or [0.]*len(points))
30            flags0=[0.]+list(station_flags or [0.]*len(points))
31            seen=0.;penalty=0.
32            for i in o:
33                penalty+=weights0[i]*seen;seen+=flags0[i]
34            return distance+5*penalty
35        for _ in range(10):
36            best=cost(order);candidate=None
37            for i in range(1,len(order)):
38                short=order[:i]+order[i+1:]
39                for j in range(1,len(order)):
40                    if i==j:continue
41                    proposal=short[:j]+[order[i]]+short[j:];value=cost(proposal)
42                    if value<best-1e-8:best=value;candidate=proposal
43            if candidate is None:break
44            order=candidate
45    return [i-1 for i in order[1:]]

```


程序 39

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/run_http.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/devel

opment/code_snapshot/question4/run_http.py; question4/innovation/frozen_baseline/question4/run_http.py; question4/run_http.py

```
1 """Connect to self HTTP or an already-ready official Q4 rehearsal."""
2 import argparse
3 import json
4 import time
5 from pathlib import Path
6 from question3.interface import Client, HttpTransport, JsonLog
7 from .strategy import run
8
9 def main():
10     p=argparse.ArgumentParser()
11     p.add_argument('--mode',choices=['self-http','official-rehearsal'],default='self-http')
12     p.add_argument('--url');p.add_argument('--robot-id',default='SELF')
13     p.add_argument('--confirm-rehearsal-ui',action='store_true',help='Operator confirms Q4 rehearsal UI is ready')
14     p.add_argument('--output',required=True)
15     p.add_argument('--config',type=Path,default=Path(__file__).with_name('configs')/'recommended.json')
16     args=p.parse_args()
17     if args.mode=='official-rehearsal' and (not args.confirm_rehearsal_ui or args.robot_id=='SELF'):
18         p.error('Confirm official Q4 rehearsal UI and supply the logged-in team robot-id.')
19     config=json.loads(args.config.read_text(encoding='utf-8'))
20     out=Path(args.output);out.mkdir(parents=True,exist_ok=False)
21     (out/'config.json').write_text(json.dumps(config,indent=2),encoding='utf-8')
22     actions=JsonLog(out/'actions.jsonl');events=JsonLog(out/'strategy.jsonl')
23     url=args.url or ('http://127.0.0.1:2028' if args.mode=='self-http' else 'http://127.0.0.1:2026')
24     client=Client(HttpTransport(url),args.robot_id,actions)
25     start=time.perf_counter()
26     try:
27         result=run(client,config,events)
28     except Exception as exc:
29         result=dict(status='incomplete',failure=f'{type(exc).__name__}: {exc}',metrics=client.metrics())
30     finally:
31         actions.close();events.close()
32     result.update(mode=args.mode,elapsed_wall_s=time.perf_counter()-start,official_total=None,
33                 official_clearance_ratio=None,config=config)
34     (out/'result.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
35     print(json.dumps({k:v for k,v in result.items() if k!='stop_certificate'},ensure_ascii=False,indent=2))
36     if result['status']!='complete':raise SystemExit(1)
37
38 if __name__=='__main__':main()
```

程序 40

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/qu

estion4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/self_server.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/self_server.py; question4/innovation/frozen_baseline/question4/self_server.py; question4/self_server.py

```

1 """Local mixed-source HTTP server. Never starts the official application."""
2 import argparse
3 from question3.self_server import make_server
4 from .cases import make_case
5 from .simulator import Simulator
6
7 def main():
8     p=argparse.ArgumentParser();p.add_argument('--port',type=int,default=2028)
9     p.add_argument('--seed',type=int,default=0);p.add_argument('--family',default='uniform')
10    args=p.parse_args();server=make_server(Simulator(make_case(args.seed,args.family)),args.port)
11    print(f'SELF Q4 http://127.0.0.1:{server.server_port} robot_id=SELF',flush=True)
12    try:server.serve_forever()
13    except KeyboardInterrupt:pass
14    finally:server.server_close()
15
16 if __name__=='__main__':main()

```

程序 41

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/si

mulator.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/simulator.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/simulator.py; question4/innovation/frozen_baseline/question4/simulator.py; question4/simulator.py

1 ""SELF simulation of mixed omnidirectional/directional physics and valid-action protocol.
 2 Not official simulator. No login/window UI/encrypted official log implementation.

```

3 Hidden case data stays here, evaluator accesses it only outside strategy execution.
4 """
5 import copy
6 import hashlib
7 import json
8 import math
9 import time
10
11 class Simulator:
12     def __init__(self, case, robot_id='SELF', remaining_s=1200, rounding='bounded'):
13         self._sources=copy.deepcopy(case['sources']);self._seed=case.get('seed',0)
14         self._error_mode=case.get('error_mode','spatial_hash');self._rounding=rounding
15         self._robot_id=robot_id;self._cache={};self._entered=False;self._exited=False
16         self._position=(0.,0.);self._channel=1;self._virtual_us=0;self._remaining=remaining_s;self._start=None
17         self._cleared=set();self._components=dict(movement_s=0.,switch_s=0.,detection_s=0.,optical_s=0.,laser_s=0.)
18         if len({s['channel'] for s in self._sources})!=len(self._sources):raise ValueError('Duplicate channel')
19         for s in self._sources:
20             if not 1<=s['channel']<=20 or not 1000<=s['radius']<=1500 or math.hypot(s['x'],s['y'])>1800+1e-7:
21                 raise ValueError('Invalid self case')
22     def _response(self,accepted,**extra):
23         return dict(accepted=accepted,real_timestamp_ms=int(time.time()*1000),
24                     virtual_time_s=self._virtual_us/1e6 if accepted else 0,**extra)
25     def _bias(self,p,c):
26         x,y=p
27         # IEEE signed zero describes the same physical location.
28         if x==0:x=0.
29         if y==0:y=0.
30         if self._error_mode=='positive':return 1.
31         if self._error_mode=='negative':return -1.
32         if self._error_mode=='smooth':return math.sin(x/173+y/251+c+self._seed)
33         if self._error_mode=='zero':return 0.
34         if self._error_mode!='spatial_hash':raise ValueError('Unknown spatial error mode')
35         # Same location and channel => same bias, independent of call count/time.
36         data=f'{self._seed}:{c}:{float(x).hex()}:{float(y).hex()}'.encode()
37         value=int.from_bytes(hashlib.sha256(data).digest()[8:],'big')
38         return 2*value/(2**64-1)-1
39     def _reading(self,p,s):
40         true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
41         tick=round((true+self._bias(p,s['channel']))*100)
42         if self._rounding=='bounded':
43             # Honour the stated final +/-1deg bound and two-decimal serialization together.
44             tick=min(math.floor((true+1)*100),max(math.ceil((true-1)*100),tick))
45         elif self._rounding!='pre_round_stress':raise ValueError('Unknown rounding model')
46         return (tick%36000)/100
47     def request(self,path,payload,timeout=5.):
48         b=copy.deepcopy(payload)
49         if path not in ('/enter','/measure','/clear','/exit'):return 404,self._response(False)
50         required={'arena_id','robot_id','request_id'}|({'position','channel'} if path in ('/measure','/clear') else
51         ↪ set())
52         if not required<=b.keys():return 400,self._response(False)
53         if set(b)!=required or b['arena_id']!='default' or b['robot_id']!=self._robot_id:return
54         ↪ 200,self._response(False)
55         if not isinstance(b['request_id'],str) or not b['request_id']:return 400,self._response(False)
56         if 'position' in b:
57             p=b['position'];c=b['channel']
58             if not isinstance(p,dict) or set(p)!={'x','y'}:return 400,self._response(False)
59             if any(isinstance(p[k],bool) or not isinstance(p[k],(int,float)) or not math.isfinite(p[k]) or
60             ↪ abs(p[k])>2000000 for k in ('x','y')):
61                 return 400,self._response(False)
62             if isinstance(c,bool) or not isinstance(c,(int,float)) or not math.isfinite(c) or int(c)!=c or not
63             ↪ 1<=c<=20:
64                 return 400,self._response(False)
65         signature=(path,json.dumps(b,sort_keys=True,allow_nan=False))
66         if b['request_id'] in self._cache:
67             old,result=self._cache[b['request_id']]
68             return copy.deepcopy(result) if signature==old else (409,self._response(False))
69         if self._exited:raise ConnectionError('Self API closed')
70         if self._start is not None and (time.monotonic()-self._start>=self._remaining or
71         ↪ self._virtual_us>=360000000000):
72             self._exited=True;raise ConnectionError('Self deadline expired')
73         if (path=='/enter' and self._entered) or (path!='/enter' and not self._entered):return
74         ↪ 200,self._response(False)
75         if path=='/enter':

```

```

70         self._entered=True;self._start=time.monotonic()
71         response=self._response(True,max_virtual_duration_s=360000,max_real_duration_s=1200,
72                                 remaining_real_duration_s=self._remaining)
73     elif path=='/exit':
74         self._exited=True;response=self._response(True,exit_reason='user_exit')
75     else:
76         p=(float(b['position']['x']),float(b['position']['y']));c=int(b['channel'])
77         move=math.dist(self._position,p)/5;switch=float(path=='/measure' and c!=self._channel)
78         parts=dict(movement_s=move,switch_s=switch,detection_s=0.,optical_s=0.,laser_s=0.)
79         s=next((s for s in self._sources if s['channel']==c and c not in self._cleared),None)
80         distance=math.inf if s is None else math.hypot(s['x']-p[0],s['y']-p[1])
81         if path=='/measure':
82             parts['detection_s']=5.;self._channel=c
83             axis = None if s is None else s.get('direction_deg')
84             visible = axis is None or
85                 ↪ ((p[0]-s['x'])*math.cos(math.radians(axis))+(p[1]-s['y'])*math.sin(math.radians(axis)) >= -1e-10)
86             if s is None or distance>s['radius'] or not visible:result=dict(measure_result='no_signal')
87             elif distance<=5:result=dict(measure_result='near')
88             else:result=dict(measure_result='direction',svd_deg=self._reading(p,s))
89         else:
90             parts['optical_s']=3.
91             if distance<=20:
92                 self._cleared.add(c);parts['laser_s']=2.;result=dict(clear_result='success')
93             else:result=dict(clear_result='no_target_in_range')
94         for k,v in parts.items():self._components[k]+=v
95         self._virtual_us+=round(sum(parts.values())*1e6);self._position=p
96         response=self._response(True,**result)
97         result=(200,response);self._cache[b['request_id']]=(signature,copy.deepcopy(result));return result
98     def evaluation(self):
99         """Evaluator-only: never exposed by /enter, /measure, /clear, /exit."""
100         return dict(total=len(self._sources),cleared=len(self._cleared),
101                     remaining_channels=sorted(s['channel'] for s in self._sources if s['channel'] not in
102                     ↪ self._cleared),
103                     components=self._components.copy(),virtual_time_s=self._virtual_us/1e6)

```

程序 42

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_

mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/strategy.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/strategy.py; question4/innovation/frozen_baseline/question4/strategy.py; question4/strategy.py

```

1 """Certified directional search; accepts only a public action client and fixed config."""
2 import math
3 import numpy as np
4 from .geometry import mesh, radial_mesh, compact_mesh, cover_hash, first_region, bearing_clip, optical_cover,
   ↳ estimate, bbox
5 from .routing import tour
6
7 DEFAULT = dict(spacing=990., scheduling='immediate',
8               localization='pair', probe_width=300., opportunistic_clear=False,
9               refinement=False, joint_weight=1.,
10              mesh_kind='lattice', scan_known='all')
11 DEFAULT.update(centroid_route=False, detection_cost_route=False, known_count_stop=False)
12 DEFAULT.update(centroid_probe=False, relocate_route=False, scan_information=False)
13
14 class Strategy:
15     def __init__(self, client, config, log=lambda x:None):
16         if set(config)-set(DEFAULT):raise ValueError('Unknown configuration fields: '+str(set(config)-set(DEFAULT)))
17         self.c=client; self.cfg=DEFAULT|config; self.log=log
18         for key,allowed in dict(scheduling={'immediate','joint','route'},localization={'pair','optical'},
19                               mesh_kind={'lattice','radial','compact'},scan_known={'all','none','certified'}).items():
20             if self.cfg[key] not in allowed:raise ValueError('Invalid '+key)
21         if not math.isfinite(self.cfg['probe_width']) or not 10<=self.cfg['probe_width']<=1000:
22             raise ValueError('Invalid probe width')
23         for key,value in DEFAULT.items():
24             if isinstance(value,bool) and not isinstance(self.cfg[key],bool):raise ValueError('Invalid boolean '+key)
25         self.stations,self.triangles=(compact_mesh() if self.cfg['mesh_kind']=='compact' else
26                                     radial_mesh() if self.cfg['mesh_kind']=='radial' else mesh(self.cfg['spacing']))
27         self.cleared=set(); self.known={}; self.seen=set()
28         self.readings={}; self.missed={}; self.done=set()
29
30     def observe(self,p,ch,phase):
31         r=self.c.measure(p,int(ch),phase)
32         kind=r['measure_result']
33         if kind=='near':

```

```

34         if not self.clear(p,ch,'near'):
35             raise RuntimeError('Near clear failed')
36     elif kind=='direction':
37         a=r['svd_deg']
38         if ch in self.known:
39             self.known[ch]['poly']=bearing_clip(self.known[ch]['poly'],p,a)
40         else:
41             self.known[ch]=dict(poly=first_region(p,a),first=np.array(p),angle=a)
42         self.readings.setdefault(ch,[]).append((list(p),a))
43     else:
44         self.missed.setdefault(ch,[]).append(list(p))
45     self.log(dict(event='observe',channel=int(ch),point=list(p),kind=kind))
46     return kind
47
48 def clear(self,p,ch,phase):
49     ok=self.c.clear(p,int(ch),phase)['clear_result']=='success'
50     if ok:
51         self.cleared.add(ch); self.known.pop(ch,None)
52     return ok
53
54 def target(self,data):
55     poly=data['poly']
56     if not self.cfg['centroid_route']:return estimate(poly)[0]
57     q=np.roll(poly,-1,axis=0);cross=poly[:,0]*q[:,1]-q[:,0]*poly[:,1]
58     if abs(cross.sum())<1e-8:return estimate(poly)[0]
59     return ((poly+q)*cross[:,None]).sum(axis=0)/(3*cross.sum())
60
61 def localize(self,ch):
62     data=self.known[ch]; poly=data['poly']; center,radius=estimate(poly)
63     if radius<20-1e-5:
64         if not self.clear(center,ch,'certified_clear'):
65             raise RuntimeError('Certified clear failed')
66         return
67     if self.cfg['localization']=='pair' and len(self.readings[ch])<2:
68         a=math.radians(data['angle']); v=np.array([-math.sin(a),math.cos(a)])
69         probe_center=self.target(data) if self.cfg['centroid_probe'] else center
70         candidates=[probe_center+self.cfg['probe_width']*v,probe_center-self.cfg['probe_width']*v]
71         candidates.sort(key=lambda q:math.dist(self.c.position,q))
72         for q in candidates:
73             self.observe(q,ch,'localize')
74             if ch in self.cleared:
75                 return
76             if len(self.readings[ch])>=2:
77                 break
78     if ch not in self.known:
79         return
80     data=self.known[ch]; poly=data['poly']; center,radius=estimate(poly)
81     if self.cfg['refinement'] and radius>35:
82         # A third bearing is valuable only when the existing region is expensive to sweep.
83         lo,hi,rot=bbox(poly,data['angle'])
84         direction=rot[:,int(np.argmax(hi-lo))]
85         normal=np.array([-direction[1],direction[0]])
86         width=max(45.,min(180.,radius*.6))
87         proposals=[center+width*normal,center-width*normal]
88         proposals.sort(key=lambda q:math.dist(self.c.position,q))
89         for q in proposals:
90             kind=self.observe(q,ch,'refine')
91             if ch in self.cleared:return
92             if kind=='direction':break
93     poly=self.known[ch]['poly'];center,radius=estimate(poly)
94     if radius<20-1e-5:
95         if not self.clear(center,ch,'certified_clear'):
96             raise RuntimeError('Certified clear failed')
97         return
98     points,r=optical_cover(poly,data['angle'],self.c.position)
99     if self.cfg['opportunistic_clear']:
100         if self.clear(center,ch,'center_attempt'):return
101     self.log(dict(event='optical_cover',channel=int(ch),vertices=poly.tolist(),cover_radius=r,points=len(points)))
102     for q in points:
103         if self.clear(q,ch,'optical_fallback'):
104             return
105     raise RuntimeError('Exhausted optical certificate without success')
106

```



```

107 def run(self):
108     self.c.enter()
109     while len(self.done)<len(self.stations) or self.known:
110         if len(self.cleared)==16:
111             break
112         remaining=[i for i in range(len(self.stations)) if i not in self.done]
113         if self.cfg['known_count_stop'] and len(self.cleared)+len(self.known)==16:
114             remaining=[]
115         i=min(remaining,key=lambda j:(math.dist(self.c.position,self.stations[j]),j)) if remaining else None
116         route_choice=None
117         if self.cfg['scheduling']=='route':
118             tasks=[('station',j,self.stations[j]) for j in remaining]+[('source',ch,self.target(data)) for
↪ ch,data in self.known.items()]
119             weights=[6. if t[0]=='source' and (self.cfg['scan_known']!='certified' or
↪ estimate(self.known[t[1]]['poly'])[1]>=20-1e-5) else 0. for t in tasks]
120             flags=[float(t[0]=='station') for t in tasks]
121             order=tour(self.c.position,[t[2] for t in tasks],weights if self.cfg['detection_cost_route'] else
↪ None,flags,self.cfg['relocate_route'])
122             route_choice=tasks[order[0]]
123             if route_choice[0]=='source':
124                 self.localize(route_choice[1]);continue
125             i=route_choice[1]
126         if self.known:
127             ch=min(self.known,key=lambda c:math.dist(self.c.position,estimate(self.known[c]['poly'])[0]))
128             source_cost=math.dist(self.c.position,estimate(self.known[ch]['poly'])[0])/5
129             station_cost=math.inf if i is None else
↪ math.dist(self.c.position,self.stations[i])/5+(20-len(self.cleared))*6*self.cfg['joint_weight']
130             if self.cfg['scheduling']=='immediate' or (route_choice is None and source_cost<=station_cost):
131                 self.localize(ch)
132                 continue
133         if not remaining:
134             break
135         p=self.stations[i]
136         channels=[ch for ch in range(1,21) if ch not in self.cleared]
137         if self.c.channel in channels:
138             channels.remove(self.c.channel);channels.insert(0,self.c.channel)
139         for ch in channels:
140             if ch in self.known:
141                 radius=estimate(self.known[ch]['poly'])[1]
142                 if self.cfg['scan_known']=='none' or (self.cfg['scan_known']=='certified' and radius<20-1e-5):
143                     continue
144                 if self.cfg['scan_information']:
145                     poly=self.known[ch]['poly'];center,_=estimate(poly)
146                     delta=poly-p
147                     ref=math.atan2(center[1]-p[1],center[0]-p[0])
148                     angles=(np.arctan2(delta[:,1],delta[:,0])-ref+math.pi)%(2*math.pi)-math.pi
149                     if np.ptp(angles)<math.radians(3):continue
150                 self.observe(p,ch,'search')
151                 self.seen.add((i,ch))
152             self.done.add(i)
153         certified=(len(self.cleared)==16 or (len(self.done)==len(self.stations) and not self.known))
154         if not certified:
155             raise RuntimeError('Missing stop certificate')
156         self.c.exit()
157         return dict(status='complete',cleared=sorted(self.cleared),metrics=self.c.metrics(),
158             stop_certificate=dict(kind='count16' if len(self.cleared)==16 else 'cell_halfplane_cover' if
↪ self.cfg['mesh_kind']=='compact' else 'triangular_halfplane_cover',
159             cover_sha256=cover_hash() if self.cfg['mesh_kind']=='compact' else None,
160             spacing=self.cfg['spacing'],mesh_kind=self.cfg['mesh_kind'],stations=self.stations.tolist(),tj
↪ riangles=self.triangles.tolist(),
161             completed_stations=sorted(self.done),scanned_pairs=len(self.seen)))
162
163 def run(client,config=None,log=lambda x:None):
164     return Strategy(client,config or {},log).run()

```

程序 43

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/final_validation/co

de_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/frozen/question4/summarize.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/question4/summarize.py; question4/innovation/frozen_baseline/question4/summarize.py; question4/summarize.py

```
1 """Paper-ready paired statistics. Independent layouts, not duplicate noise replays, are units."""
2 import argparse
3 import json
```

```

4 from pathlib import Path
5 import numpy as np
6
7 def pair_stats(rows, candidate, reference):
8     ref={r['case_id']:r for r in rows if r['config']==reference}
9     chosen=[r for r in rows if r['config']==candidate]
10    x=np.array([ref[r['case_id']]['metrics']['virtual_time_s'] for r in chosen])
11    y=np.array([r['metrics']['virtual_time_s'] for r in chosen])
12    gains=100*(1-y/x);rng=np.random.default_rng(20260912)
13    ids=rng.integers(0,len(x),size=(10000,len(x)))
14    boot=100*(1-y[ids].mean(axis=1)/x[ids].mean(axis=1))
15    ordered=sorted(zip(gains,chosen),key=lambda t:t[0])
16    return dict(n=len(x),mean_s=float(y.mean()),reference_mean_s=float(x.mean()),
17               gain_pct=float(100*(1-y.mean()/x.mean())),bootstrap95_pct=np.percentile(boot,[2.5,97.5]).tolist(),
18               p95_s=float(np.percentile(y,95)),reference_p95_s=float(np.percentile(x,95)),
19               median_paired_gain_pct=float(np.median(gains)),wins=int(np.sum(gains>1e-7)),ties=int(np.sum(abs(gains)<=1e-7))
20    ↪ ),
21    worst=[dict(case_id=r['case_id'],gain_pct=float(g),candidate_s=r['metrics']['virtual_time_s'],
22               reference_s=ref[r['case_id']]['metrics']['virtual_time_s'],components=r['metrics']['components']) for g,r
23    ↪ in ordered[:5]],
24    by_family={f:dict(n=sum(r['family']==f for r in chosen),
25                      gain_pct=float(100*(1-np.mean([r['metrics']['virtual_time_s'] for r in chosen if r['family']==f])/
26    ↪ np.mean([ref[r['case_id']]['metrics']['virtual_time_s'] for r in chosen if
27    ↪ r['family']==f])))
28    for f in sorted({r['family'] for r in chosen})},
29    components={k:float(np.mean([r['metrics']['components'][k] for r in chosen])) for k in
30    ↪ chosen[0]['metrics']['components']},
31    complete=sum(r['status']=='complete' and r['audit']['passed'] and r['cleared']==r['total'] for r in chosen),
32    max_wall_s=max(r['elapsed_s'] for r in chosen))
33
34 def main():
35     p=argparse.ArgumentParser();p.add_argument('runs',nargs='+');p.add_argument('--candidate',default='FINAL');p.add_
36     ↪ argument('--output',required=True)
37     args=p.parse_args();report={}
38     for directory in args.runs:
39         data=json.loads((Path(directory)/'summary.json').read_text());rows=data['rows'];names=list(dict.fromkeys(r['c
40     ↪ onfig'] for r in
41     ↪ rows))
42         report[Path(directory).name]=dict(total_runs=len(rows),all_complete=all(r['status']=='complete' and
43     ↪ r['audit']['passed'] and r['cleared']==r['total'] for r in rows),
44         vs_baseline={name:pair_stats(rows,name,names[0]) for name in names},
45         vs_final={name:pair_stats(rows,name,args.candidate) for name in names})
46     Path(args.output).write_text(json.dumps(report,ensure_ascii=False,indent=2))
47     for batch,data in report.items():
48         print(batch)
49         for name,s in data['vs_final'].items():
50             print(name,round(s['mean_s'],2),round(s['gain_pct'],3),[round(x,3) for x in s['bootstrap95_pct']])
51
52 if __name__=='__main__':main()

```

程序 44

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/

development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/local_mechanisms.py

```

1 """New public-feedback mechanisms; independent from evaluator-owned fixtures."""
2 import math
3 from itertools import combinations
4 import numpy as np
5 from scipy.spatial import ConvexHull
6 from question4.strategy import Strategy as Baseline
7 from question4.geometry import estimate, clip, optical_cover
8
9 BASE = dict(scheduling='route', mesh_kind='radial', scan_known='certified',
10             centroid_route=True, known_count_stop=True, scan_information=True)
11 CANDIDATES = {
12     'CELL_OPTICAL': {'local_mechanism': 'cell_optical'},
13     'CLEAR_DISCOVERY': {'local_mechanism': 'clear_discovery'},
14     'NEGATIVE_REGION': {'local_mechanism': 'negative_region'},
15 }
16
17
18 def cell_cover(poly, angle, start):
19     """Partition the bounding rectangle into strips and variable-width cells.
20
21     Every retained polygon cell lies inside a rectangle of diagonal <40 m.
22     All nonempty clipped cells are retained. Disks around their rectangle
23     centers therefore cover the entire feasible region, including boundaries.
24     """
25     theta = math.radians(angle)
26     axes = np.array([[math.cos(theta), -math.sin(theta)],
27                     [math.sin(theta), math.cos(theta)]])
28     p = poly @ axes
29     low, high = p.min(0)-1e-7, p.max(0)+1e-7
30     width, height = high-low
31     options = []
32     for ny in range(max(1, math.ceil(height/39)), max(2, math.ceil(height/20))+1):
33         dy = height/ny
34         if dy >= 39.999:
35             continue
36         dx_limit = math.sqrt(39.999**2-dy**2)
37         nx = max(1, math.ceil(width/dx_limit))
38         options.append((nx*ny, ny, nx))
39     _, ny, nx = min(options)
40     dx, dy = (high-low)/np.array([nx, ny])
41     cells = []
42     for j in range(ny):
43         for i in (range(nx) if j%2==0 else reversed(range(nx))):
44             lo=low+np.array([i*dx,j*dy]); hi=lo+[dx,dy]
45             cell=p.copy()
46             for normal, bound in (([1,0],hi[0]),([-1,0],-lo[0]),([0,1],hi[1]),([0,-1],-lo[1])):
47                 cell=clip(cell,normal,bound)
48             if len(cell):
49                 cells.append(dict(vertices=(cell@axes.T).tolist(),
50                                     center=((lo+hi)/2)@axes.T.tolist(), index=[i,j]))
51     if math.dist(start,cells[-1]['center']) < math.dist(start,cells[0]['center']):
52         cells.reverse()

```

```

53     return dict(vertices=poly.tolist(), angle=angle, low=low.tolist(), high=high.tolist(),
54                 shape=[nx,ny], cells=cells, radius=float(math.hypot(dx,dy)/2))
55
56
57 def prune_directional(poly, positive, negative):
58     """Exclude strictly impossible source positions; convexify conservatively.
59
60     If n is directionally negative and a,b positive, source s cannot equal
61      $n+l_1*(n-a)+l_2*(n-b)$ ,  $l_1,l_2>0$ . The affine emission function would give  $h(s)<0$ .
62     Each complement halfplane is retained with a positive numerical margin.
63     """
64     proofs=[]
65     for n0 in negative[-8:]:
66         n=np.asarray(n0)
67         if np.linalg.norm(poly-n,axis=1).max()>=1000-1e-5:
68             continue
69         for a0,b0 in combinations(positive[-8:],2):
70             a,b=np.asarray(a0),np.asarray(b0)
71             matrix=np.column_stack([n-a,n-b])
72             if abs(np.linalg.det(matrix))<1e-5*np.linalg.norm(n-a)*np.linalg.norm(n-b):
73                 continue
74             inv=np.linalg.inv(matrix)
75             pieces=[clip(poly,normal,float(normal@n)+1e-6) for normal in inv]
76             keep=[x for x in pieces if len(x)]
77             if not keep:
78                 raise RuntimeError('Directional inference inconsistent with public observations')
79             vertices=np.vstack(keep)
80             if len(vertices)<3:
81                 continue
82             try:
83                 new=vertices[ConvexHull(vertices).vertices]
84             except Exception:
85                 continue
86             old_area=ConvexHull(poly).volume
87             if ConvexHull(new).volume < old_area-1e-5:
88                 proofs.append(dict(before=poly.tolist(),vertices=new.tolist(),negative_point=n.tolist(),
89                                   positive_points=[a.tolist(),b.tolist()]))
90             poly=new
91     return poly,proofs
92
93
94 class Strategy(Baseline):
95     def __init__(self,client,config,log=lambda row:None):
96         self.mode=config['local_mechanism']
97         if self.mode not in ('cell_optical','clear_discovery','negative_region'):
98             raise ValueError('Unknown local mechanism')
99         super().__init__(client,BASE,log)
100         self.in_discovery=False
101
102     def observe(self,p,ch,phase):
103         kind=super().observe(p,ch,phase)
104         if ch in self.known:
105             self.log(dict(event='feasible_region',channel=int(ch),kind=kind,
106                           vertices=self.known[ch]['poly'].tolist(),point=list(p)))
107             if self.mode=='negative_region':
108                 poly,proofs=prune_directional(self.known[ch]['poly'],
109                                                [q for q,_ in self.readings[ch]],self.missed.get(ch,[]))
110                 self.known[ch]['poly']=poly
111                 for proof in proofs:
112                     self.log(dict(event='directional_region_prune',channel=int(ch),**proof))
113                 if proofs:
114                     self.log(dict(event='feasible_region',channel=int(ch),kind='directional_inference',
115                                   vertices=poly.tolist(),point=list(p)))
116         return kind
117
118     def clear(self,p,ch,phase):
119         before=tuple(self.c.position)
120         ok=super().clear(p,ch,phase)
121         if (ok and self.mode=='clear_discovery' and not self.in_discovery
122             and math.dist(before,self.c.position)>1e-6 and len(self.known)+len(self.cleared)<16):
123             self.in_discovery=True
124             try:
125                 for other in range(1,21):

```

```

126         if other in self.known or other in self.cleared:
127             continue
128         self.observe(np.asarray(self.c.position), other, 'clear_discovery')
129         if len(self.known)+len(self.cleared)==16:
130             break
131     finally:
132         self.in_discovery=False
133     return ok
134
135     def localize(self, ch):
136         if self.mode!='cell_optical':
137             return super().localize(ch)
138         data=self.known[ch]; center, radius=estimate(data['poly'])
139         if radius>=20-1e-5 and len(self.readings[ch])<2:
140             a=math.radians(data['angle']); side=np.array([-math.sin(a), math.cos(a)])
141             probes=sorted([center+300*side, center-300*side], key=lambda q: math.dist(self.c.position, q))
142             for q in probes:
143                 self.observe(q, ch, 'localize')
144                 if ch in self.cleared: return
145                 if len(self.readings[ch])>=2: break
146         data=self.known[ch]; center, radius=estimate(data['poly'])
147         if radius<20-1e-5:
148             if not self.clear(center, ch, 'certified_clear'):
149                 raise RuntimeError('Certified clear failed')
150             return
151         proof=cell_cover(data['poly'], data['angle'], self.c.position)
152         self.log(dict(event='optical_cells', channel=int(ch), **proof))
153         for cell in proof['cells']:
154             if self.clear(cell['center'], ch, 'optical_fallback'): return
155         raise RuntimeError('Exhausted cell cover without successful clear')
156
157
158     def run(client, config, log=lambda row: None):
159         return Strategy(client, config, log).run()

```

程序 45

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/op

tical_audit.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/optical_audit.py

```

1 """Evaluator-only verification of the cell optical certificate."""
2 import math
3 import numpy as np
4 from question4.innovation.audit import _inside
5
6
7 def check_cells(case,events):
8     checked=0
9     for event in events:
10         if event.get('event')!='optical_cells':continue
11         p=np.array(event['vertices']);lo=np.array(event['low']);hi=np.array(event['high'])
12         theta=math.radians(event['angle'])
13         rot=np.array([[math.cos(theta),-math.sin(theta)],[math.sin(theta),math.cos(theta)]])
14         local=p@rot;nx,ny=event['shape'];step=(hi-lo)/[nx,ny]
15         assert 0<math.hypot(*step)/2<20
16         assert np.all(local>=lo-1e-7) and np.all(local<=hi+1e-7)
17         ids={tuple(c['index']) for c in event['cells']}
18         assert len(ids)==len(event['cells'])
19         # Independently determine cell/convex-polygon intersection with SAT.
20         for j in range(ny):
21             for i in range(nx):
22                 lower=lo+[i*step[0],j*step[1]];upper=lower+step
23                 corners=np.array([lower,[upper[0],lower[1]],upper,[lower[0],upper[1]]])
24                 edges=np.roll(local,-1,0)-local
25                 axes=[np.array([1.,0]),np.array([0.,1])]+[np.array([-e[1],e[0]]) for e in edges if
26                     ↪ np.linalg.norm(e)>1e-10]
27                 # A strictly positive overlap needs a retained cell; zero-area
28                 # shared boundaries are covered by their adjacent retained cell.
29                 overlap=all(min((local@a).max(),(corners@a).max())-max((local@a).min(),(corners@a).min()))>1e-8 for a
30                     ↪ in axes)
31                 if overlap:assert (i,j) in ids,'Omitted intersecting cell'
32             for cell in event['cells']:
33                 i,j=cell['index'];assert 0<=i<nx and 0<=j<ny
34                 center=(lo+(np.array([i,j])+.5)*step)@rot.T
35                 assert np.linalg.norm(center-cell['center'])<1e-6
36                 assert np.linalg.norm(np.asarray(cell['vertices'])-center,axis=1).max()<20
37                 s=next(s for s in case['sources'] if s['channel']==event['channel'])
38                 source=np.array([s['x'],s['y']]);assert _inside(p,source)
39                 assert min(np.linalg.norm(source-np.asarray(c['center'])) for c in event['cells'])<20
40             checked+=1
41     return {'passed':True,'checked_cell_covers':checked}

```

程序 46

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/

code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/c
 ode_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py; pa
 per_evidence/q4_mechanism_reset_20260912/round08_optical_cells/developme
 nt/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/devel
 opment/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduli
 ng.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region
 /development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/sc
 heduling.py; paper_evidence/q4_mechanism_reset_20260912/scheduling.py

```

1 """Independent SELF-only Q4 scheduling mechanisms using public feedback only.
2
3 The policy imports public strategy/geometry code, never cases, Simulator, files,
4 random seeds, orientations or source positions. Scheduling predictions are cost
5 heuristics; the inherited enclosing-region/finite-grid clearing and station
6 coverage certificates remain the only completeness mechanisms.
7 """
8 import math
9 import numpy as np
10
11 from question4.strategy import run as baseline_run
12 from question4.geometry import estimate, optical_cover
13 from question4.routing import tour
14 from question4.innovation.strategy import Strategy as Parent, BASELINE_CONFIG
15 from question4.innovation.geometry import proposals
16
17
18 CANDIDATES = {
19     'ACTUAL_TASK': {'schedule': 'actual_task', 'schedule_parent': 'baseline'},
20     'BACKBONE': {'schedule': 'backbone', 'schedule_parent': 'baseline'},
21     'WINDOW': {'schedule': 'window', 'schedule_parent': 'baseline'},
22     'BACKBONE_ADAPT': {'schedule': 'backbone', 'schedule_parent': 'adapt'},
23 }
24
25
26 class Strategy(Parent):
27     def __init__(self, client, config=None, log=lambda row: None):
28         config = config or {}
29         if set(config) - {'schedule', 'schedule_parent'}:
30             raise ValueError('Unknown scheduling configuration')
31         self.schedule = config.get('schedule', 'actual_task')
32         self.schedule_parent = config.get('schedule_parent', 'baseline')
33         if self.schedule not in ('baseline', 'actual_task', 'backbone', 'window'):
34             raise ValueError('Unknown schedule')
35         if self.schedule_parent not in ('baseline', 'adapt'):
36             raise ValueError('Unknown schedule_parent')
37         options = ({'mechanism': 'parallax', 'sharing': 'gated'}
38                   if self.schedule_parent == 'adapt'
39                   else {'mechanism': 'pair', 'sharing': 'off'})
40         super().__init__(client, options, log)
41         self.backbone = tour((0., 0.), self.stations.tolist())
42         self.window_queue = []
43         self.window_draining = False
44
45     def scan_worthwhile(self, ch, p):
46         """The original low-information rule, also used as a cost forecast."""
47         if ch in self.cleared:
48             return False
49         if ch not in self.known:
50             return True
51         poly = self.known[ch]['poly']
52         center, radius = estimate(poly)
53         if radius < 20*1e-5:
54             return False
55         delta = poly-p
56         ref = math.atan2(center[1]-p[1], center[0]-p[0])

```



```

57     angles = (np.arctan2(delta[:, 1], delta[:, 0]) - ref + math.pi) % (2 * math.pi) - math.pi
58     return np.ptp(angles) >= math.radians(3)
59
60 def scan(self, index):
61     p = self.stations[index]
62     channels = [ch for ch in range(1, 21) if ch not in self.cleared]
63     if self.c.channel in channels:
64         channels.remove(self.c.channel)
65         channels.insert(0, self.c.channel)
66     for ch in channels:
67         if self.scan_worthwhile(ch, p):
68             self.observe(p, ch, 'search')
69             self.seen.add((index, ch))
70     self.done.add(index)
71
72 def descriptor(self, ch):
73     """Possible first actual action points and a heuristic completion point.
74
75     A predicted exit is NOT a certificate. The actual parent localization
76     executes until certified clear/finite-cover success, then we replan.
77     """
78     data = self.known[ch]
79     center, radius = estimate(data['poly'])
80     endpoint = self.target(data)
81     if radius < 20 * 1e-5:
82         return [center], center
83     if self.schedule_parent == 'adapt':
84         candidates = proposals(data['poly'], data['angle'], self.c.position, 'parallax')
85         candidates = [p for p in candidates if tuple(p) not in self.visited.get(ch, set())]
86         if candidates:
87             return candidates, endpoint
88     elif len(self.readings[ch]) < 2:
89         a = math.radians(data['angle'])
90         normal = np.array([-math.sin(a), math.cos(a)])
91         return [center + self.cfg['probe_width'] * normal,
92               center - self.cfg['probe_width'] * normal], endpoint
93     points, _ = optical_cover(data['poly'], data['angle'], self.c.position)
94     # Original optical traversal chooses one of its two ends as entry.
95     return [np.asarray(points[0]), np.asarray(points[-1])], endpoint
96
97 def actual_task_choice(self, remaining):
98     """Open directed service tour, accounting for service entry and exit."""
99     tasks = [(('station', index) for index in remaining) + (('source', ch) for ch in sorted(self.known))]
100     entries, exits = [], []
101     for kind, key in tasks:
102         if kind == 'station':
103             p = self.stations[key]
104             entries.append([p]); exits.append(p)
105         else:
106             a, b = self.descriptor(key)
107             entries.append(a); exits.append(b)
108     entries = [np.asarray(a, dtype=float) for a in entries]
109     exits = np.asarray(exits, dtype=float)
110     # To predict the parent's entry choice, first select nearest entry from
111     # the predecessor, then pay the entry->predicted-exit internal travel.
112     transition = np.zeros((len(tasks)+1, len(tasks)), dtype=float)
113     for i, prev in enumerate([np.asarray(self.c.position), *exits]):
114         for j, points in enumerate(entries):
115             distances = np.linalg.norm(points - prev, axis=1)
116             k = int(np.argmin(distances))
117             transition[i, j] = distances[k] + np.linalg.norm(points[k] - exits[j])
118     order, unused, prev = [], set(range(len(tasks))), 0
119     while unused:
120         j = min(unused, key=lambda j: (transition[prev, j], j))
121         order.append(j); unused.remove(j); prev = j+1
122
123 def cost(path):
124     return float(transition[0, path[0]] + sum(transition[a+1, b] for a, b in zip(path, path[1:])))
125
126 # Directed edge costs require recalculating reversed internal edges.
127 # Fixed effort limit is shared by all cases; no case-specific tuning.
128 for _ in range(8):
129     best, changed = cost(order), None

```

```

130         for i in range(len(order)-1):
131             for j in range(i+1, len(order)):
132                 candidate = order[:i]+list(reversed(order[i:j+1]))+order[j+1:]
133                 value = cost(candidate)
134                 if value < best-1e-8:
135                     best, changed = value, candidate
136             if changed is None:
137                 break
138             order = changed
139         selected = tasks[order[0]]
140         self.log(dict(event='actual_task_plan', selected=selected,
141                     predicted_distance_m=cost(order),
142                     order=[tasks[j] for j in order], prediction_is_certificate=False))
143         return selected
144
145     def backbone_choice(self, remaining):
146         """Protect station discovery order and insert sources at minimum cost."""
147         route = [index for index in self.backbone if index in remaining]
148         if not route:
149             points = [self.target(self.known[ch]) for ch in sorted(self.known)]
150             channels = sorted(self.known)
151             return 'source', channels[tour(self.c.position, points)[0]]
152         sites = [np.asarray(self.c.position)]+[self.stations[index] for index in route]
153         best = None
154         for ch in sorted(self.known):
155             target = self.target(self.known[ch])
156             # Waiting for a station only costs a detection if the original
157             # public polygon-based rule would actually scan this channel.
158             waiting = 0.
159             alternatives = []
160             for slot in range(len(route)+1):
161                 prev = sites[slot]
162                 delta = math.dist(prev, target)
163                 if slot < len(route):
164                     nxt = sites[slot+1]
165                     delta += math.dist(target, nxt)-math.dist(prev, nxt)
166                 alternatives.append(delta/5+waiting)
167                 if slot < len(route) and self.scan_worthwhile(ch, sites[slot+1]):
168                     waiting += 6.
169             slot = min(range(len(alternatives)), key=lambda i: (alternatives[i], i))
170             self.log(dict(event='backbone_insertion', channel=ch, best_slot=slot,
171                         predicted_cost_s=alternatives[slot], immediate_cost_s=alternatives[0]))
172             if slot == 0:
173                 candidate = (alternatives[0], ch)
174                 if best is None or candidate < best:
175                     best = candidate
176         if best is not None:
177             return 'source', best[1]
178         return 'station', route[0]
179
180     def window_choice(self, remaining):
181         """Collect two scheduled bearings before draining a source batch."""
182         if self.window_draining and self.known:
183             channels = sorted(self.known)
184             order = tour(self.c.position, [self.target(self.known[ch]) for ch in channels])
185             return 'source', channels[order[0]]
186         if self.window_draining:
187             self.window_draining = False
188         if not self.window_queue:
189             order = tour(self.c.position, [self.stations[index] for index in remaining])
190             self.window_queue = [remaining[j] for j in order[:2]]
191             self.log(dict(event='paired_discovery_window', stations=self.window_queue.copy()))
192         index = self.window_queue.pop(0)
193         if not self.window_queue:
194             self.window_draining = True
195         return 'station', index
196
197     def run(self):
198         if self.schedule == 'baseline':
199             return super().run()
200         self.c.enter()
201         while len(self.done) < len(self.stations) or self.known:
202             if len(self.cleared) == 16:

```

```

203         break
204     remaining = [i for i in range(len(self.stations)) if i not in self.done]
205     if len(self.cleared)+len(self.known) == 16:
206         remaining = []
207     if not remaining:
208         if not self.known:
209             break
210         channels = sorted(self.known)
211         order = tour(self.c.position, [self.target(self.known[ch]) for ch in channels])
212         kind, key = 'source', channels[order[0]]
213     elif self.schedule == 'actual_task':
214         kind, key = self.actual_task_choice(remaining)
215     elif self.schedule == 'backbone':
216         kind, key = self.backbone_choice(remaining)
217     else:
218         kind, key = self.window_choice(remaining)
219     if kind == 'source':
220         self.localize(key)
221     else:
222         self.scan(key)
223     certified = len(self.cleared) == 16 or (len(self.done) == len(self.stations) and not self.known)
224     if not certified:
225         raise RuntimeError('Missing stop certificate')
226     self.c.exit()
227     return dict(status='complete', cleared=sorted(self.cleared), metrics=self.c.metrics(),
228               stop_certificate=dict(kind='count16' if len(self.cleared) == 16 else 'triangular_halfplane_cover',
229                                   cover_sha256=None, spacing=self.cfg['spacing'], mesh_kind=self.cfg['mesh_kind'],
230                                   stations=self.stations.tolist(), triangles=self.triangles.tolist(),
231                                   completed_stations=sorted(self.done), scanned_pairs=len(self.seen)))
232
233
234 def run(client, config=None, log=lambda row: None):
235     config = config or {}
236     if config == {'schedule': 'baseline', 'schedule_parent': 'baseline'}:
237         return baseline_run(client, BASELINE_CONFIG, log)
238     return Strategy(client, config, log).run()

```

程序 47

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/p

aper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py; paper_evidence/q4_mechanism_reset_20260912/scheduling_smoke.py

```

1 """SELF regression checks on historical seed range; not performance selection.
2
3 Run from repository root with python -m
4 paper_evidence.q4_mechanism_reset_20260912.scheduling_smoke. Assertions concern
5 completion, physical accounting and identity ablation, not candidate ranking.
6 """
7 import json
8 from pathlib import Path
9
10 from question3.interface import Client
11 from question4.audit import audit
12 from question4.cases import make_case
13 from question4.simulator import Simulator
14 from question4.strategy import run as baseline
15 from question4.innovation.experiment import PublicClient
16 from question4.innovation.strategy import BASELINE_CONFIG, run as adapt_run
17 from .scheduling import CANDIDATES, run
18
19
20 def execute(case, runner, config):
21     records = []
22     client = Client(Simulator(case), 'SELF', records.append)
23     result = runner(PublicClient(client), config)
24     checked = audit(case, records, result)
25     signature = []
26     for record in records:
27         if record['type'] == 'request':
28             payload = record['payload']
29             signature.append((record['path'], payload.get('position'), payload.get('channel'), record['phase']))
30     return result, signature, checked
31
32
33 def main():
34     rows = []
35     families = ('uniform', 'outward', 'inward', 'tangent', 'cluster', 'edge')
36     for index, family in enumerate(families):
37         case = make_case(71000+index, family, 'spatial_hash')
38         for name, config in CANDIDATES.items():
39             result, _, checked = execute(case, run, config)
40             assert checked['passed'] and result['status'] == 'complete'
41             rows.append(dict(case=case['id'], candidate=name, check='complete_physical_audit', passed=True))
42     case = make_case(71000, 'uniform', 'spatial_hash')
43     for parent, original, original_config in (
44         ('baseline', baseline, BASELINE_CONFIG),
45         ('adapt', adapt_run, {'mechanism': 'parallax', 'sharing': 'gated'}),
46     ):
47         a, signature_a, _ = execute(case, original, original_config)
48         b, signature_b, _ = execute(case, run, {'schedule': 'baseline', 'schedule_parent': parent})
49         assert a == b and signature_a == signature_b
50         rows.append(dict(check='parent_ablation_action_and_result_identity', parent=parent, passed=True))
51     output = Path(__file__).with_name('scheduling_smoke.json')
52     output.write_text(json.dumps(dict(stage='historical_self_regression_not_selection',
53         cases_note='Six historical seed-range layouts, one fixed spatial error field, bounded rounding.',
54         performance_used_for_selection=False, checks=rows), indent=2)+'\n')
55     print(json.dumps(dict(passed=len(rows), failed=0, output=str(output))))
56
57
58 if __name__ == '__main__':
59     main()

```

程序 48

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;

paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round03_window/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/strategy.py;
 paper_evidence/q4_mechanism_reset_20260912/strategy.py

```

1 """Dispatcher exposing only public actions to every compared policy."""
2 from .frozen.question4.strategy import run as frozen_run
3 from question4.innovation.strategy import run as historical_run, BASELINE_CONFIG
4
5
6 def run(client,config=None,log=lambda row:None):
7     config=dict(config or {})
8     family=config.pop('family')
9     if family=='baseline':
10         if config:raise ValueError('Frozen baseline has no experimental parameters')
11         return frozen_run(client,BASELINE_CONFIG,log)
12     if family=='historical':
13         if config:raise ValueError('Historical reference configuration is fixed')
14         return historical_run(client,dict(mechanism='parallax',sharing='gated'),log)
15     if family=='scheduling':
16         from .scheduling import run as method
17     elif family=='coverage':
18         from .coverage import run as method
19     elif family=='local':
20         from .local_mechanisms import run as method
21     else:raise ValueError('Unknown mechanism family')
22     return method(client,config,log)

```

程序 49

对应文件: paper_evidence/q4_mechanism_reset_20260912/final_validation/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
 paper_evidence/q4_mechanism_reset_20260912/round01_action_tasks/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
 paper_evidence/q4_mechanism_reset_20260912/round02_backbone/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
 paper_evidence/q4_mechanism_reset_20260912/round03_window/development

```
t/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round04_search_first/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round05_sector/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round06_exchange/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round07_elastic/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round08_optical_cells/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round09_clear_discovery/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/round10_negative_region/development/code_snapshot/paper_evidence/q4_mechanism_reset_20260912/suite.py;
paper_evidence/q4_mechanism_reset_20260912/suite.py
```

```
1 """Run the predeclared ten development rounds, retaining every attempted run."""
2 import json
3 import subprocess
4 import sys
5 from pathlib import Path
6
7 ROOT=Path(__file__).resolve().parent
8
9
10 def main():
11     progress=[]
12     for row in json.loads((ROOT/'round_index.json').read_text()):
13         folder=ROOT/row['round']
14         command=[sys.executable, '-m', 'paper_evidence.q4_mechanism_reset_20260912.experiment',
15                 '--configs', str(folder/'configs.json'), '--output', str(folder/'development'),
16                 '--seed', '291000', '--count', '24', '--error-modes', 'spatial_hash,positive,negative,smooth',
17                 '--roundings', 'bounded', '--workers', '4', '--stage', 'development', '--reference', 'BASE25']
18         (folder/'command.json').write_text(json.dumps(command, indent=2))
19         with (folder/'execution.log').open('x') as output:
20             result=subprocess.run(command, stdout=output, stderr=subprocess.STDOUT)
21         progress.append(dict(round=row['round'], exit_code=result.returncode))
22         (ROOT/'development_progress.json').write_text(json.dumps(progress, indent=2))
23         comparison=folder/'development/comparison.json'
24         if comparison.exists():
25             stats=json.loads(comparison.read_text())['configurations']
26             compact={name:{k:s[k] for k in ('complete', 'runs', 'mean_s', 'p95_s', 'gain_ratio_of_means_pct')}} for name, s
27                 ↪ in stats.items()}
28             print(json.dumps(dict(round=row['round'], results=compact)), flush=True)
29         else:
30             print(json.dumps(dict(round=row['round'], framework_failure=True, exit=result.returncode)), flush=True)
31             raise SystemExit(result.returncode or 1)
32
33 if __name__=='__main__':main()
```

程序 50

对应文件: paper_evidence/q4_mechanism_reset_20260912/qa/replay.py

```
1 """Replay public Q4 request/response logs without importing an evaluator.
2
3 python -m paper_evidence.q4_mechanism_reset_20260912.qa.replay \
4     --case-dir path/to/batch/CANDIDATE/case_id \
```

```

5  --config path/to/configs.json --config-name CANDIDATE \
6  --output path/to/new_replay_directory
7
8  --config also accepts an inline JSON policy object. Outputs must not exist.
9  The policy receives public action methods only, backed by recorded responses;
10 no Simulator, fixture, source table, hidden-case file, or true count is loaded.
11 """
12 import argparse
13 import ast
14 import copy
15 import hashlib
16 import importlib.util
17 import json
18 from pathlib import Path
19 import traceback
20
21 from question3.interface import Client
22
23 PROJECT = Path(__file__).resolve().parents[3]
24 ENTRY_MODULE = 'paper_evidence.q4_mechanism_reset_20260912.strategy'
25
26
27 def digest(path):
28     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
29
30
31 def write_new(path, value):
32     with Path(path).open('x', encoding='utf-8') as out:
33         json.dump(value, out, ensure_ascii=False, indent=2, allow_nan=False)
34         out.write('\n')
35
36
37 class PublicActions:
38     """Capability wrapper. No transport or evaluator attribute is exposed."""
39     __slots__ = ('__delegate',)
40
41     def __init__(self, delegate):
42         self.__delegate = delegate
43
44     @property
45     def position(self):
46         return self.__delegate.position
47
48     @property
49     def channel(self):
50         return self.__delegate.channel
51
52     def enter(self):
53         return self.__delegate.enter()
54
55     def measure(self, *args, **kwargs):
56         return self.__delegate.measure(*args, **kwargs)
57
58     def clear(self, *args, **kwargs):
59         return self.__delegate.clear(*args, **kwargs)
60
61     def exit(self):
62         return self.__delegate.exit()
63
64     def metrics(self):
65         return self.__delegate.metrics()
66
67
68 def signature(path, payload):
69     # Request identifiers are per-process idempotency tokens, not policy input.
70     return path, {k: v for k, v in payload.items() if k != 'request_id'}
71
72
73 class RecordedResponses:
74     """Strict sequential transport containing only a public transcript.
75
76     A response becomes available only after its preceding request matches
77     exactly, including all point coordinates and channel. Retry outcomes are

```

```

78     retained; an exhausted retry remains a replay failure rather than success.
79     """
80     __slots__ = ('__steps', '__cursor', '__id_map')
81
82     def __init__(self, rows):
83         steps = []
84         pending = None
85         for row in rows:
86             kind = row.get('type')
87             if kind == 'request':
88                 if pending is not None:
89                     raise ValueError('Transcript has a request with no outcome')
90                 pending = copy.deepcopy(row)
91             elif kind in ('response', 'transport_error'):
92                 if pending is None or row['request_id'] != pending['payload']['request_id']:
93                     raise ValueError('Transcript outcome does not match its preceding request')
94                 steps.append((pending, copy.deepcopy(row)))
95                 pending = None
96             if pending is not None or not steps:
97                 raise ValueError('Transcript is empty or ends with an unresolved request')
98         self.__steps = steps
99         self.__cursor = 0
100        self.__id_map = {}
101
102    @property
103    def progress(self):
104        return self.__cursor, len(self.__steps)
105
106    def request(self, path, payload, timeout=5.):
107        if self.__cursor >= len(self.__steps):
108            raise AssertionError('Policy emitted an extra action after the recorded transcript')
109        expected, outcome = self.__steps[self.__cursor]
110        action_index = self.__cursor
111        if signature(path, payload) != signature(expected['path'], expected['payload']):
112            raise AssertionError('Public action mismatch at index '+str(action_index)+' : '+
113                                json.dumps({'expected': signature(expected['path'], expected['payload']),
114                                              'actual': signature(path, payload)}, ensure_ascii=False))
115        old_id, new_id = expected['payload']['request_id'], payload['request_id']
116        if old_id in self.__id_map and self.__id_map[old_id] != new_id:
117            raise AssertionError('A replayed retry changed its request identifier')
118        if old_id not in self.__id_map and new_id in self.__id_map.values():
119            raise AssertionError('Distinct actions reused a replay request identifier')
120        self.__id_map[old_id] = new_id
121        self.__cursor += 1
122        if outcome['type'] == 'transport_error':
123            raise OSError(outcome.get('error', 'Recorded transport error'))
124        # Timestamp is retained, but never used to derive positions or truth.
125        return outcome['http_status'], copy.deepcopy(outcome['response'])
126
127
128    def _module_path(module, project):
129        path = project.joinpath(*module.split('.'))
130        if path.with_suffix('.py').is_file():
131            return path.with_suffix('.py')
132        if (path/'__init__.py').is_file():
133            return path/'__init__.py'
134        return None
135
136
137    def static_scan(entry=ENTRY_MODULE, project=PROJECT):
138        """AST dependency scan; useful evidence, explicitly not a sandbox proof."""
139        project = Path(project)
140        forbidden_module_parts = {'simulator', 'cases', 'experiment', 'self_server', 'run_http'}
141        forbidden_attributes = {'transport', 'Simulator', 'evaluation', '_sources', '_seed',
142                                '_error_mode', '_rounding', 'case', 'hidden', 'truth',
143                                'evaluator_hidden_case', 'direction_deg'}
144        forbidden_keys = {'sources', 'seed', 'direction_deg', 'evaluator_hidden_case',
145                           'hidden_case', '_sources', '_seed'}
146        dynamic_calls = {'eval', 'exec', '__import__', 'getattr', 'setattr', 'globals', 'locals', 'vars'}
147        seen = set(); queue = [entry]; modules = []; violations = []; reviews = []
148        while queue:
149            module = queue.pop()
150            if module in seen:

```



```

151         continue
152     seen.add(module)
153     path = _module_path(module, project)
154     if path is None:
155         continue
156     tree = ast.parse(path.read_text(encoding='utf-8'), filename=str(path))
157     imports = []
158     for node in ast.walk(tree):
159         targets = []
160         if isinstance(node, ast.Import):
161             targets = [x.name for x in node.names]
162         elif isinstance(node, ast.ImportFrom):
163             package = module if path.name == '__init__.py' else module.rsplit('.', 1)[0]
164             target = importlib.util.resolve_name('.*node.level+(node.module or '')', package) if node.level else
↳ (node.module or '')
165             targets = [target]
166         for target in targets:
167             imports.append(target)
168             if forbidden_module_parts.intersection(target.split('.')):
169                 violations.append({'module': module, 'line': node.lineno, 'kind': 'forbidden_import', 'value': target})
170             if _module_path(target, project) is not None:
171                 queue.append(target)
172         if isinstance(node, ast.Attribute) and node.attr in forbidden_attributes:
173             violations.append({'module': module, 'line': node.lineno, 'kind': 'forbidden_attribute', 'value': node.attr})
174         if isinstance(node, ast.Subscript) and isinstance(node.ctx, ast.Load):
175             key = node.slice.value if isinstance(node.slice, ast.Constant) else None
176             if isinstance(key, str) and key in forbidden_keys:
177                 violations.append({'module': module, 'line': node.lineno, 'kind': 'forbidden_key', 'value': key})
178         if isinstance(node, ast.Call):
179             name = node.func.id if isinstance(node.func, ast.Name) else node.func.attr if
↳ isinstance(node.func, ast.Attribute) else None
180             if name in dynamic_calls:
181                 reviews.append({'module': module, 'line': node.lineno, 'kind': 'dynamic_introspection_review', 'value':
↳ name})
182                 if name in {'open', 'read_text', 'read_bytes', 'load', 'loads'}:
183                     reviews.append({'module': module, 'line': node.lineno, 'kind': 'file_or_decoder_review', 'value': name})
184             modules.append({'module': module, 'path': str(path.relative_to(project)),
185                             'sha256': digest(path), 'imports': sorted(set(imports))})
186     return {'entry': entry, 'passed': not violations, 'modules': sorted(modules, key=lambda row: row['module']),
187            'violations': violations, 'manual_review_findings': reviews,
188            'scope': 'Recursively follows repository Python imports from policy dispatcher; external libraries are not
↳ analyzed.',
189            'limitation': 'AST checks are evidence of inspected code and do not prove absence of arbitrary dynamic
↳ behavior or constitute a security sandbox.'}
190
191
192 def load_config(value, name=None):
193     config = json.loads(value) if value.lstrip().startswith('{') else
↳ json.loads(Path(value).read_text(encoding='utf-8'))
194     if name is not None:
195         config = config[name]
196     if not isinstance(config, dict) or 'family' not in config:
197         raise ValueError('Expected one dispatcher policy configuration; select a matrix arm with --config-name')
198     return config
199
200
201 def replay(case_dir, config, output):
202     case_dir, output = Path(case_dir).resolve(), Path(output).resolve()
203     if output == case_dir or case_dir in output.parents:
204         raise ValueError('Replay output must be outside the source case directory')
205     output.mkdir(parents=True, exist_ok=False)
206     records = [json.loads(line) for line in (case_dir/'actions.jsonl').read_text(encoding='utf-8').splitlines() if
↳ line]
207     # The result is evaluator-owned comparison material and never reaches the
208     # public transport or policy. In particular, evaluator_hidden_case is unread.
209     document = json.loads((case_dir/'result.json').read_text(encoding='utf-8'))
210     expected = document.get('strategy', document)
211     transport = RecordedResponses(records)
212     robot_ids = {r['payload']['robot_id'] for r in records if r.get('type') == 'request'}
213     if len(robot_ids) != 1:
214         raise ValueError('A replay transcript must belong to one robot ID')
215     produced, events = [], []
216     client = Client(transport, robot_ids.pop(), produced.append)

```

```

217 public = PublicActions(client)
218 assert not any(hasattr(public, name) for name in ('transport', 'Simulator', 'hidden', 'case', 'truth', 'evaluation'))
219 scan = static_scan()
220 write_new(output/'static_scan.json', scan)
221 report = {'source_directory':str(case_dir), 'source_actions_sha256':digest(case_dir/'actions.jsonl'),
222          'source_result_sha256':digest(case_dir/'result.jsonl'), 'config':config,
223          'source_truth_or_simulator_provided':False, 'hidden_case_file_read':False,
224          'static_scan_passed':scan['passed'], 'passed':False}
225 try:
226     if not scan['passed']:
227         raise AssertionError('Policy dependency scan found forbidden evaluator access')
228     from paper_evidence.q4_mechanism_reset_20260912.strategy import run
229     result = run(public, copy.deepcopy(config), events.append)
230     cursor, total = transport.progress
231     assert cursor == total, 'Policy stopped before all recorded actions were consumed'
232     assert result['status'] == expected['status'], 'Completion status differs'
233     assert result.get('cleared') == expected.get('cleared'), 'Cleared channel list differs'
234     assert result['metrics'] == expected['metrics'], 'Full public time accounting differs'
235     assert result.get('stop_certificate') == expected.get('stop_certificate'), 'Stop certificate differs'
236     original_phases = [r.get('phase') for r in records if r.get('type') == 'request']
237     replay_phases = [r.get('phase') for r in produced if r.get('type') == 'request']
238     assert replay_phases == original_phases, 'Action phase attribution differs'
239     report.update(passed=True, actions=cursor, all_positions_channels_exact=True,
240                  cleared_equal=True, metrics_equal=True, stop_certificate_equal=True, phases_equal=True)
241     write_new(output/'result.json', result)
242 except Exception as exc:
243     report.update(failure=f'{type(exc).__name__}: {exc}', traceback=traceback.format_exc(),
244                  consumed_actions=transport.progress[0], recorded_actions=transport.progress[1])
245 for filename, rows in (('actions.jsonl', produced), ('strategy.jsonl', events)):
246     with (output/filename).open('x', encoding='utf-8') as handle:
247         for row in rows:
248             handle.write(json.dumps(row, ensure_ascii=False, allow_nan=False)+'\n')
249 write_new(output/'replay.json', report)
250 return report
251
252
253 def main():
254     parser=argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
255     parser.add_argument('--case-dir', required=True)
256     parser.add_argument('--config', required=True, help='Single policy JSON path or inline JSON; matrices require
257 ↵ --config-name')
258     parser.add_argument('--config-name')
259     parser.add_argument('--output', required=True, help='New output directory outside source case folder')
260     args=parser.parse_args()
261     report=replay(args.case_dir, load_config(args.config, args.config_name), args.output)
262     print(json.dumps(report, ensure_ascii=False, indent=2))
263     if not report['passed']:
264         raise SystemExit(1)
265
266 if __name__ == '__main__':
267     main()

```

程序 51

对应文件: paper_evidence/q4_mechanism_reset_20260912/qa/test_replay.py

```

1 """Meaningful regression checks for exact public-response replay boundaries."""
2 import copy
3 from pathlib import Path
4 import tempfile
5 import unittest
6 from paper_evidence.q4_mechanism_reset_20260912.qa.replay import RecordedResponses, static_scan
7
8 ROOT=Path(__file__).parent
9
10 def request(identifier='old'):
11     return {'type':'request', 'path': '/measure', 'payload':{'request_id':identifier, 'robot_id':'SELF',
12                  'arena_id':'default', 'position':{'x':12.5, 'y':-7.25}, 'channel':3}, 'attempt':0}
13
14 def response(identifier='old'):

```

```

15     return {'type': 'response', 'path': '/measure', 'request_id': identifier, 'http_status': 200,
16             'response': {'accepted': True, 'virtual_time_s': 8.5, 'real_timestamp_ms': 123, 'measure_result': 'no_signal'}}
17
18 class Tests(unittest.TestCase):
19     def test_exact_point_rejects_coordinate_change(self):
20         req=request();trans=RecordedResponses([req,response()]);body=copy.deepcopy(req['payload'])
21         body['position']['x']+=1e-8
22         with self.assertRaisesRegex(AssertionError,'Public action mismatch'):trans.request('/measure',body)
23         self.assertEqual(trans.progress,(0,1))
24
25     def test_request_id_can_change_but_retry_must_reuse(self):
26         req=request();retry=copy.deepcopy(req);retry['attempt']=1
27         error={'type': 'transport_error', 'request_id': 'old', 'error': 'recorded socket error'}
28         trans=RecordedResponses([req,error,retry,response()]);body=copy.deepcopy(req['payload']);body['request_id']=''
29         ↪ new'
29         with self.assertRaises(0SError):trans.request('/measure',body)
30         wrong=body|{'request_id': 'another'}
31         with self.assertRaisesRegex(AssertionError,'retry changed'):trans.request('/measure',wrong)
32         self.assertEqual(trans.request('/measure',body),(200,response()['response']))
33         self.assertEqual(trans.progress,(2,2))
34
35     def test_no_extra_actions(self):
36         req=request();trans=RecordedResponses([req,response()]);trans.request('/measure',req['payload'])
37         with self.assertRaisesRegex(AssertionError,'extra action'):trans.request('/measure',req['payload'])
38
39     def test_static_scan_catches_evaluator_import_and_hidden_access(self):
40         with tempfile.TemporaryDirectory(prefix='scan_test_',dir=ROOT) as temporary:
41             folder=Path(temporary);(folder/'policy.py').write_text('from question4.simulator import Simulator\ndef
42             ↪ f(c):\n return c.hidden["sources"]\n')
42             report=static_scan('policy',folder)
43             self.assertFalse(report['passed']);self.assertEqual(len(report['violations']),3)
44
45     def test_policy_dependency_scan_passes_with_public_geometry_io_review(self):
46         report=static_scan();self.assertTrue(report['passed']);self.assertGreater(len(report['modules']),10)
47         self.assertTrue(report['manual_review_findings'])
48         self.assertTrue(all('geometry' in r['module'] for r in report['manual_review_findings']))
49
50 if __name__=='__main__':unittest.main()

```

程序 52

对应文件: paper_evidence/q4_mechanism_reset_20260912/reporting/development.py

```

1 """Create traceable per-round conclusions from every unfiltered development run."""
2 import json
3 from pathlib import Path
4 from collections import Counter
5 ROOT=Path(__file__).resolve().parents[1]
6 index=json.loads((ROOT/'round_index.json').read_text())
7 rows=[]
8 for item in index:
9     folder=ROOT/item['round']; path=folder/'development/comparison.json'
10    if not path.exists():continue
11    comparison=json.loads(path.read_text()); stats=comparison['configurations']; base=stats['BASE25']
12    summary=json.loads((folder/'development/summary.json').read_text())
13    details=[]
14    lines=[f"# {item['round']} 开发结论",',',item['motivation'],'',
15    ↪ '|方案|完整|局数|失败|超时|均值秒|p95秒|最差秒|均值收益|', '|---|---|---|---|---|---|---|']
16    for name in item['configs']:
17        s=stats[name]; significant=s['gain_ratio_of_means_pct']>=5 and
18        ↪ s['complete']/s['runs']>=base['complete']/base['runs'] and s['p95_s']<=base['p95_s']
17        lines.append(f"|{name}|{s['complete']}/{s['runs']}|{s['failed']}|{s.get('timeouts',0)}|{s['mean_s']:.2f}|{s['p95_s']
18        ↪ |:2f}|{s['max_s']:.2f}|{s['gain_ratio_of_means_pct']:.3f}%|")
19    if name not in ('BASE25','HIST_ADAPT'):
20        events=Counter()
21        for r in summary['rows']:
22            if r['config']==name:events.update(r['event_counts'])
23            row=dict(round=item['round'],config=name,primary=name==item['primary'],significant=significant,**{k:s[k] for k in
24            ↪ ('gain_ratio_of_means_pct','complete','runs','failed','p95_s','mean_s','max_s')},events=dict(events))
25            rows.append(row)

```

```

24     details += [' ', f'{name}: {'达到开发筛选门槛, 需独立验证' if significant else '未达到 5% / 成功率 / P95 联合门槛;
    ↳ 不替换推荐'}. ',
25     f" 最差绝对案例 {s['worst_case']}; 最差配对退化 {s['worst_gain_pct']:.3f}%。 ",
26     '动作耗时分解 (秒/局): ' + json.dumps(s['components'], ensure_ascii=False),
27     '相对基线动作差 (秒/局): ' + json.dumps({k: round(v-base['components'].get(k,0),3) for k,v in
    ↳ s['components'].items(), ensure_ascii=False),
28     '机制事件总次数: ' + json.dumps(dict(events), ensure_ascii=False)]
29 lines += details
30 lines += [' ', '源码/configs、每局 actions.jsonl/strategy.jsonl/result.json 及失败堆栈均位于 development/。
    ↳ 上表没有剔除失败、慢例或无触发案例。组合/证书消融不单独计轮。', ' ', '证据等级:
    ↳ 完整性来自保守几何与有限覆盖/光学兜底; 时间收益或退化仅获本开发集实验支持。未证明整局最优,
    ↳ 未验证官方分布或官方运行。']
31 (folder/'REVIEW.md').write_text('\n'.join(lines)+'\n')
32 (ROOT/'development_evidence.json').write_text(json.dumps(rows, ensure_ascii=False, indent=2))
33 print(json.dumps(rows, ensure_ascii=False, indent=2))

```

程序 53

对应文件: paper_evidence/q4_mechanism_reset_20260912/reporting/finalize.py

```

1 """Analyze the frozen final matrix; never selects a different candidate."""
2 import json, hashlib
3 from pathlib import Path
4 import numpy as np
5 from question4.innovation.analysis import analyze, validate_batch
6 ROOT=Path(__file__).resolve().parents[1]
7 PROJECT=ROOT.parents[1]
8
9 def write(path, obj): path.write_text(json.dumps(obj, ensure_ascii=False, indent=2)+'\n')
10 def main():
11     summary=json.loads((ROOT/'final_validation/summary.json').read_text())
12     valid=validate_batch(ROOT/'final_validation/summary.json', summary)
13     report=analyze(summary['rows'], reference='BASE25', bootstrap=10000)
14     report['matrix_audit']=valid
15     for name, s in report['configurations'].items():
16         s['timeouts']=sum(bool(r.get('timeout')) for r in summary['rows'] if r['config']==name)
17     write(ROOT/'FINAL_STATISTICS.json', report)
18     final_ids={r['layout_sha256'] for r in summary['rows']}
19     all_old=set(); sets={}
20     for p in
    ↳ list((PROJECT/'question4/results').glob('*/*fixtures.json'))+list((PROJECT/'question4/innovation/results').glob('*
    ↳ */fixtures.json'))+list(ROOT.glob('round*/development/fixtures.json'))+[ROOT/'stress/results/fixtures.json']:
21         cases=json.loads(p.read_text())
22         ids={hashlib.sha256(json.dumps(sorted(c['sources'], key=lambda
    ↳ s:s['channel']), ensure_ascii=False, sort_keys=True, separators=(',', ':'), allow_nan=False).encode()).hexdigest()
    ↳ for c in cases}
23         sets[str(p.relative_to(PROJECT))]=len(ids&final_ids); all_old|=ids
24     assert not all_old&final_ids
25     frozen=json.loads((ROOT/'freeze_manifest.json').read_text())
26     changed=[p for p, h in frozen['files'].items() if hashlib.sha256((PROJECT/p).read_bytes()).hexdigest()!=h]
27     assert not changed
28     provenance=dict(final_physical_layouts=len(final_ids), prior_physical_layouts_compared=len(all_old), overlap_by_prior_
    ↳ batch=sets, overlap=0, original_files_changed=changed)
29     write(ROOT/'PROVENANCE_CHECK.json', provenance)
30     primary='COVER_EXCHANGE_ELASTIC'; stats=report['configurations']; base=stats['BASE25']; candidate=stats[primary]
31     dev=json.loads((ROOT/'development_evidence.json').read_text()); stress=json.loads((ROOT/'stress/results/comparison.js
    ↳ on').read_text())
32     significant=candidate['gain_ratio_of_means_pct']>=5 and
    ↳ candidate['complete']/candidate['runs']>=base['complete']/base['runs'] and candidate['p95_s']<=base['p95_s']
33     evidence=dict(selected=primary, significant_improvement=significant, threshold_gain_pct=5, consecutive_unsuccessful_dev
    ↳ elopment_rounds=10,
34     final_all_complete_and_audited=all(s['failed']==0 for s in
    ↳ stats.values()), final_algorithm_runs=len(summary['rows']),
35     development_algorithm_runs=3168, adversarial_algorithm_runs=640, official_runs=0, source_unchanged=True,
36     mean_gain_pct=candidate['gain_ratio_of_means_pct'], p95_gain_pct=(1-candidate['p95_s']/base['p95_s'])*100,
37     replace_recommendation=significant, provenance=provenance)
38     write(ROOT/'FINAL_EVIDENCE.json', evidence)
39     lines=['# 第四问算法机制重新探索: 最终报告', ' ',
40     f'结论: {'达到预设门槛' if significant else '没有找到满足预设门槛的显著更好方案'}. 锁定新候选 {primary} 在最终 120
    ↳ 个未见布局上平均总虚拟时间降低 **{candidate['gain_ratio_of_means_pct']:.3f}%**, P95 {'降低' if
    ↳ candidate['p95_s']<=base['p95_s'] else '增加'} **{abs(evidence['p95_gain_pct']:.3f}%**. 建议{'考虑替换' if
    ↳ significant else '保留原 25 站推荐, 不为小幅收益增加复杂度'}. ', ' ',

```

```

41 '共完成 10 个不同核心机制的研发轮次，另有2个组合和1个证书消融。每轮默认预配置，零外部参数扫描。
    ↳ 连续10轮均未达到平均降时≥5%、清除成功率不降低且P95不恶化的联合门槛，小幅收益没有重置计数。按要求停止新机制探索，
    ↳ 再对事前选定候选完成独立确认，最终结果没有用于改代码、调参或换候选。','',
42 '## 最终完整流程对照',' ',
43 '每臂120布局×4固定误差场×2舍入模型=960局。误差和舍入属于同一布局的重复压力条件，统计单位仍为120个物理布局。表中失败使用
    ↳ max(实际耗时,360000秒)，原始耗时也在JSON中保留。本次五臂全部成功，因此惩罚与原始时间相同。',' ',
44 '|方案|完整/局数|失败/超时|均值秒|P95秒|最差秒|相对原推荐均值收益|','|---|---|---|---|---|---|---|'
45 for name in ['BASE25','HIST_ADAPT','COVER_EXCHANGE','COVER_ELASTIC',primary]:
46 s=stats[name];lines.append(f'|{name}|{s['complete']}/{s['runs']}/{s['failed']}/{s['timeouts']}/{s['mean_s']:.2f}|{s}
    ↳ ['p95_s']:.2f}|{s['max_s']:.2f}|{s['gain_ratio_of_means_pct']:.3f}%|')
47 ci=candidate['bootstrap'][['gain_ratio_of_means_pct_ci95']]
48 lines += [' ','f"锁定候选均值收益的布局分块、家族分层配对 bootstrap 95%区间为 **[{ci[0]:.3f}%, {ci[1]:.3f}%]** (10000次)
    ↳ 。{candidate['wins']}局更快、{candidate['losses']}局更慢、{candidate['ties']}局相同。
    ↳ 该区间描述本自建生成器的布局采样变化，不能代表未知官方分布。",' ',
49 '## 创新机制和消融',' ',
50 '新候选把固定覆盖测站变成可以更新的覆盖见证。成功清除后，若实际到达点可替代一个未访站且全域证书仍成立，
    ↳ 就在此处完成该站扫描；访问内环站之前，在保持覆盖边与三角朝向约束下，优化其到当前点及下一个公共任务的局部路线。
    ↳ 两种修改都必须重新验证连续覆盖，失败则保留原站。原保守定位与有限光学清除兜底仍执行。',' ',
51 '这区别于调小站距或探测宽度：测站位置由当前已公开动作和任务动态决定，覆盖证书成为每次更新的可行性约束。
    ↳ 站点优化使用固定35步计算预算；这是在线算法计算，不是使用测试真值调参。',' ',
52 '最终五臂构成严格2×2消融：BASE25、仅替站、仅移动站、两者组合；HIST_ADAPT是已有历史候选，未作为本次发明。
    ↳ 贡献不可简单相加：站点移动会改变接收方向、清除顺序及随后抽到的固定误差场位置。',' ',
53 '| 动作组件 | 原推荐秒/局 | 锁定候选秒/局 | 候选 - 基线 |','|---|---|---|---|'
54 for k,v in base['components'].items():
55 c=candidate['components'].get(k,0);lines.append(f'|{k}|{v:.2f}|{c:.2f}|{c-v:.2f}|')
56 lines += [' ','阶段分解 (秒/局): ',' ', '| 阶段 | 原推荐 | 候选 |','|---|---|---|'
57 for k in sorted(set(base['phases'])|set(candidate['phases'])):
58 lines.append(f'|{k}|{base['phases'].get(k,0):.2f}|{candidate['phases'].get(k,0):.2f}|')
59 lines += [' ','动作组件与阶段是同一总耗时的两种分组，不能加重重复计费。',' ', '## 十轮结果与放弃原因',' ',
    ↳ '|轮|机制|开发均值收益|决定|','|---|---|---|---|'
60 for r in dev:
61 if r['primary']:lines.append(f'|{r['round'][:5]}|{r['config']}|{r['gain_ratio_of_means_pct']:.3f}%| 未达门槛 |')
62 lines += [' ','动作入口/出口建模未弥补预测误差；固定骨架、两站批处理、
    ↳ 全巡视后清除及分区承诺均失去了原在线调度的重要适应性。动态替站触发有限；移动测站获得温和收益，组合仍低于门槛。
    ↳ 减少光学单元并不保证早命中更早；清除后额外发现消耗了更多检测和切频；方向阴性位置收缩安全但触发/边际效应很小。
    ↳ 每轮详细组件差、最差配对案例、事件频次、配置和原始日志见相应 round*/REVIEW.md 和 development/。',' ',
63 '## 最差案例和可靠性',' ',
64 worst=candidate['worst_paired_cases'][0]
65 lines += [f"最差相对案例 {worst['case_id']}: 原推荐 {worst['baseline_s']:.2f} 秒，候选 {worst['candidate_s']:.2f} 秒，
    ↳ 变慢 {worst['regression_s']:.2f} 秒 ({abs(worst['gain_pct']):.2f}%)。两者均完整清除。候选最大绝对耗时案例为
    ↳ '{candidate['worst_case']}'。",' ',
66 '开发24布局×4误差，10轮合计3168次算法运行；对抗16布局×4误差×2舍入×5臂，640次；最终120新布局×4误差×2舍入×5臂，4800次，
    ↳ 合计8608次完整流程自建运行，全部清除并通过动作、计时与停止审计。重复基线运行不能计为独立布局；
    ↳ 几何单测及历史种子烟测另列，不混入独立验收样本。',' ',
67 '## 对抗复核与证据等级',' ',
68 '- 数学已证明（在所列几何前提下）：直径<1000的三角形包含源时，任意发射半平面至少包含一顶点；
    ↳ 四叉树每叶位于近距见证点的凸包内可推广到全连续圆域；有限光学覆盖及16个不同成功清除支持可靠停止。
    ↳ 阴性区域收缩的仿射半平面矛盾证明见局部机制审查。',' ',
69 '- 实验支持：完整动作费用一致、固定点重复方位一致、最终配对收益及分组结果；
    ↳ 公共响应重放和静态依赖复核证明所检查的执行不依赖隐藏布局对象。',' ',
70 '- 待验证：未知官方案例分布、官方计时实现、真实网络/Windows环境、所有浮点退化输入的机器级形式证明；
    ↳ 本次未执行官方演练或正式测试。',' ',
71 '- 旧审计器接受退化伪网格的漏洞已保存反例，本轮增加真实平面Delaunay重建及连续覆盖检查。
    ↳ 非入选光学CELL_OPTICAL的独立审计对极薄伪证书有阈值盲点；实际生成器检查通过，仍保留该审计局限，
    ↳ 不把全量通过当作形式验证。',' ',
72 '详见 SOURCE_AUDIT.md、qa/LOCAL_ADVERSARIAL.md、qa/FINAL_ADVERSARIAL_REVIEW.md 和 qa/ 公共重放记录。',' ',
73 '## 复现和替换建议',' ',
74 '原推荐、原题附件及已存在的question4源文件哈希全部与freeze_manifest.json一致。只新增paper_evidence本轮目录，
    ↳ 没有官方运行、云端推送或替换推荐。复现入口见 REPRODUCE.md；最终代码快照在final_validation/code_snapshot/，
    ↳ 锁文件在validation_lock/。',' ',
75 '本轮没有证据支持按 5% 门槛替换推荐。保留可复现候选作为论文中的约束下自适应覆盖探索与负结果，停止继续叠加机制。']
76 lines += [' ','## 终审补充',' ',
77 '本轮每种新机制采用相同的单配置外部调参预算，但成熟基线已有历史研发投入；不声称各方案累计调参投入完全相同。',' ',
78 f"本机每局策略计算均值从 {base['wall_mean_s']:.4f} 秒升至 {candidate['wall_mean_s']:.4f} 秒，P95从
    ↳ {base['wall_p95_s']:.4f} 秒升至 {candidate['wall_p95_s']:.4f} 秒。两者均远低于本机现实期限，
    ↳ 但候选引入了更多在线优化计算；这些不是官方网络/Windows实测时间。",' ',
79 '最差配对案例中，基线完成7站，候选完成19站，均清除16源后按计数停止。候选只触发测站移动，没有触发动态替站；
    ↳ 多走的搜索路线涉及11个无新频道发现的站，不能把该退化归因于替站触发。详见 reporting/WORST_CASE.md
    ↳ 的逐动作证据和推断边界。',' ',
80 '候选960局的停止证书包括714局真实Delaunay覆盖、70局动态连续单元覆盖、176局16源成功计数。六条最终公共日志
    ↳ （基线/候选各三条）无需隐藏布局均精确重放，见 qa/final_replay_summary.json。独立复核重新计算全部最终统计及区间，
    ↳ 与主报告相符。']
81 (ROOT/'FINAL_REPORT.md').write_text('\n'.join(lines)+'\n')

```

```

82 print(json.dumps({n:{k:s[k] for k in ('mean_s','p95_s','max_s','gain_ratio_of_means_pct','failed','timeouts')}} for
    ↪ n,s in stats.items()),ensure_ascii=False,indent=2))
83 if __name__=='__main__':main()

```

程序 54

对应文件: paper_evidence/q4_mechanism_reset_20260912/reporting/lock_final.py

```

1 """Freeze candidate + ablations and validation design before generating truth."""
2 import hashlib,json,time,shutil
3 from pathlib import Path
4 from paper_evidence.q4_mechanism_reset_20260912.experiment import source_hashes,canonical_sha256,write_json,FAMILIES
5 ROOT=Path(__file__).resolve().parents[1]
6 out=ROOT/'validation_lock'
7 if (out/'lock.json').exists():raise RuntimeError('Refuse replacing validation lock')
8 configs=json.loads((ROOT/'candidate_matrix.json').read_text()); selection=(ROOT/'SELECTION.md').read_bytes()
9 evidence={}
10 for p in list(ROOT.glob('round*/development/comparison.json'))+[ROOT/'stress/results/comparison.json',ROOT/'best_cand_
    ↪ idate.json']:
11     evidence[str(p)]=hashlib.sha256(p.read_bytes()).hexdigest()
12 hashes=source_hashes()
13 lock=dict(schema_version=1,stage='selection_lock_without_generation',created_unix=time.time(),
14     configs=configs,configs_sha256=canonical_sha256(configs),code_sha256=hashes,
15     code_bundle_sha256=canonical_sha256(hashes),reference='BASE25',
16     selection_sha256=hashlib.sha256(selection).hexdigest(),generator_seed=391000,generator_count=120,
17     families=list(FAMILIES),error_modes=['spatial_hash','positive','negative','smooth'],
18     roundings=['bounded','pre_round_stress'],failure_penalty_s=360000,evidence_sha256=evidence,
19     selected='COVER_EXCHANGE_ELASTIC',consecutive_unsuccessful_mechanisms=10)
20 write_json(out/'lock.json',lock);(out/'selection_snapshot.txt').write_bytes(selection)
21 write_json(out/'configs.json',configs)
22 print(json.dumps(dict(layouts_generated=0,selected=lock['selected'],code_bundle_sha256=lock['code_bundle_sha256'])))

```

程序 55

对应文件: paper_evidence/q4_mechanism_reset_20260912/reporting/seal.py

```

1 """Seal local deliverables without touching original recommendations or cloud."""
2 from pathlib import Path
3 import hashlib,json,shutil,zipfile
4 ROOT=Path(__file__).resolve().parents[1]
5 for p in sorted(ROOT.rglob('__pycache__'),key=lambda p:len(p.parts),reverse=True):
6     if p.is_dir():shutil.rmtree(p)
7 archive=ROOT/'candidate_source.zip'
8 with zipfile.ZipFile(archive,'w',zipfile.ZIP_DEFLATED) as z:
9     for p in sorted((ROOT/'final_validation/code_snapshot').rglob('*')):
10         if p.is_file():z.write(p,p.relative_to(ROOT/'final_validation/code_snapshot'))
11     for name in ['candidate_matrix.json','best_candidate.json','validation_lock/lock.json','validation_lock/selection_sn
        ↪ apshot.txt','README.md','REPRODUCE.md','FINAL_REPORT.md','FINAL_STATISTICS.json','FINAL_EVIDENCE.json']:
12         z.write(ROOT/name,Path('paper_evidence/q4_mechanism_reset_20260912')/name)
13     for p in sorted((ROOT/'reporting').glob('*.py')):
14         z.write(p,Path('paper_evidence/q4_mechanism_reset_20260912/reporting')/p.name)
15     for p in sorted((ROOT/'qa').glob('*.py')):
16         z.write(p,Path('paper_evidence/q4_mechanism_reset_20260912/qa')/p.name)
17     for name in ['SOURCE_AUDIT.md','COVERAGE_REVIEW.md','SCHEDULING_REVIEW.md','qa/FINAL_ADVERSARIAL_REVIEW.md','qa/LOCA
        ↪ L_ADVERSARIAL.md','qa/REPLAY.md','reporting/WORST_CASE.md']:
18         z.write(ROOT/name,Path('paper_evidence/q4_mechanism_reset_20260912')/name)
19     z.writestr('requirements.txt','numpy==2.4.4\nscipy==1.17.1\n')
20 with zipfile.ZipFile(archive) as z:
21     assert z.testzip() is None
22     package=dict(sha256=hashlib.sha256(archive.read_bytes()).hexdigest(),entries=len(z.namelist()),bytes=archive.stat().
        ↪ st_size,crc_passed=True,includes_raw_experiments=False)
23     (ROOT/'package_verification.json').write_text(json.dumps(package,indent=2)+'\n')
24     excluded={'ARTIFACT_SHA256.json','artifact_verification.json'}
25     files=[p for p in sorted(ROOT.rglob('*')) if p.is_file() and p.name not in excluded and '__pycache__' not in p.parts]
26     manifest={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in files}
27     (ROOT/'ARTIFACT_SHA256.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2)+'\n')

```

```

28 verification=dict(files=len(files),bytes=sum(p.stat().st_size for p in files),manifest_sha256=hashlib.sha256((ROOT/'A'
↳ RTIFACT_SHA256.json').read_bytes()).hexdigest(),all_artifacts_hashed=True,original_source_unchanged=json.loads((R
↳ OOT/'PROVENANCE_CHECK.json').read_text())['original_files_changed']==[],package=package)
29 (ROOT/'artifact_verification.json').write_text(json.dumps(verification,indent=2)+'\n')
30 print(json.dumps(verification))

```

程序 56

对应文件: question1/solve.py; question3/innovation/frozen/question1/solve.py; question3/results/dispatch_20260913/development/code_snapshot/question1/solve.py; question3/results/dispatch_20260913/smoke/code_snapshot/question1/solve.py; question3/results/round13_adverse_main/code_snapshot/question1/solve.py; question3/results/round13_adverse_rounding/code_snapshot/question1/solve.py; question3/results/round13_dev_main/code_snapshot/question1/solve.py; question3/results/round13_dev_rounding/code_snapshot/question1/solve.py; question3/results/round13_holdout_main/code_snapshot/question1/solve.py; question3/results/round13_holdout_rounding/code_snapshot/question1/solve.py; question3/results/round13_outer_main/code_snapshot/question1/solve.py; question3/results/round13_outer_rounding/code_snapshot/question1/solve.py; question3/results/round14_adverse_main/code_snapshot/question1/solve.py; question3/results/round14_adverse_rounding/code_snapshot/question1/solve.py; question3/results/round14_dev_main/code_snapshot/question1/solve.py; question3/results/round14_dev_rounding/code_snapshot/question1/solve.py; question3/results/round14_holdout_main/code_snapshot/question1/solve.py; question3/results/round14_holdout_rounding/code_snapshot/question1/solve.py; question3/results/round14_initial_dev_main/code_snapshot/question1/solve.py; question3/results/round14_initial_dev_rounding/code_snapshot/question1/solve.py; question3/results/round14_outer_main/code_snapshot/question1/solve.py; question3/results/round14_outer_rounding/code_snapshot/question1/solve.py; question3/results/round17_adverse_main/code_snapshot/question1/solve.py; question3/results/round17_adverse_rounding/code_snapshot/question1/solve.py; question3/results/round17_dev_main/code_snapshot/question1/solve.py; question3/results/round17_dev_rounding/code_snapshot/question1/solve.py; question3/results/round18_adverse_main/code_snapshot/question1/solve.py; question3/results/round18_adverse_rounding/code_snapshot/question1/solve.py; question3/results/round18_dev_main/code_snapshot/question1/solve.py; question3/results/round18_dev_rounding/code_snapshot/question1/solve.py; question3/results/round18_holdout_main/code_snapshot/question1/solve.py; question3/results/round18_holdout_rounding/code_snapshot/question1/solve.py; question3/results/round18_outer_main/code_snapshot/question1/solve.py; question3/results/round18_outer_rounding/code_snapshot/question1/solve.py; question3/results/round19_adverse_main/code_snapshot/question1/solve.py; question3/results/round19_adverse_rounding/code_snapshot/question1/solve.py; question3/results/round19_dev_main/code_snapshot/question1/solve.py; question3/results/round19_dev_rounding/code_snapshot/question1/solve.py;

question3/results/round19_holdout_main/code_snapshot/question1/solve.py;
question3/results/round19_holdout_rounding/code_snapshot/question1/solve.py;
question3/results/round19_outer_main/code_snapshot/question1/solve.py;
question3/results/round19_outer_rounding/code_snapshot/question1/solve.py;
question3/results/round1_main/code_snapshot/question1/solve.py; question3/
results/round1_rounding/code_snapshot/question1/solve.py

```

1 """Question 1: bearing-wedge intersection; units metres/degrees.
2 Run: python question1/solve.py [measurements.json]
3 JSON input: [{"x": 0, "y": 0, "bearing_deg": 45}, ...]
4 Requires numpy, scipy. No simulator calls.
5 """
6 import json
7 import sys
8 from itertools import combinations
9 from pathlib import Path
10 import numpy as np
11 from scipy.optimize import linprog
12
13
14 def bearing_halfplanes(measurements, error_deg=1.0):
15     if not 0 < error_deg < 90:
16         raise ValueError('error_deg must lie in (0, 90)')
17     A, b = [], []
18     for item in measurements:
19         x, y, theta = (float(item[k]) for k in ('x', 'y', 'bearing_deg'))
20         if not np.isfinite([x, y, theta]).all():
21             raise ValueError('Non-finite measurement')
22         lo, hi = np.deg2rad([(theta % 360)-error_deg, (theta % 360)+error_deg])
23         # cross(u_lo, p-s)>=0; cross(u_hi, p-s)<=0.
24         for normal in ([np.sin(lo), -np.cos(lo)], [-np.sin(hi), np.cos(hi)]):
25             A.append(normal)
26             b.append(np.dot(normal, [x, y]))
27     if not A:
28         raise ValueError('At least one bearing is required')
29     return np.array(A), np.array(b)
30
31
32 def solve_halfplanes(A, b, tol=1e-7):
33     A, b = np.asarray(A, float), np.asarray(b, float)
34     if A.ndim != 2 or A.shape[1] != 2 or b.shape != (len(A),) or not len(A):
35         raise ValueError('Expected nonempty A[m,2], b[m]')
36     if not np.isfinite(A).all() or not np.isfinite(b).all():
37         raise ValueError('Non-finite half-plane')
38     lengths = np.linalg.norm(A, axis=1)
39     if np.any(lengths == 0):
40         raise ValueError('Zero half-plane normal')
41     A, b = A / lengths[:, None], b / lengths
42     def lp(c):
43         return linprog(c, A_ub=A, b_ub=b, bounds=[(None, None)]*2,
44                        method='highs', options={'primal_feasibility_tolerance': 1e-9})
45     feasible = lp([0, 0])
46     if feasible.status == 2:
47         return {'status': 'empty', 'diameter': None, 'vertices': []}
48     if not feasible.success:
49         raise RuntimeError(feasible.message)
50     for c in ([1, 0], [-1, 0], [0, 1], [0, -1]):
51         result = lp(c)
52         if result.status == 3:
53             return {'status': 'unbounded', 'diameter': None, 'vertices': []}
54         if not result.success:
55             raise RuntimeError(result.message)
56     vertices = []
57     for i, j in combinations(range(len(A)), 2):
58         M = A[[i, j]]
59         if abs(np.linalg.det(M)) <= 1e-12:
60             continue
61         p = np.linalg.solve(M, b[[i, j]])
62         if np.all(A @ p <= b + tol) and not any(np.linalg.norm(p-q) <= tol for q in vertices):
63             vertices.append(p)

```



```

64     if not vertices:
65         raise RuntimeError('No vertices recovered: numerical degeneracy; use higher precision')
66     v = np.array(vertices)
67     order = np.argsort(np.arctan2(v[:, 1]-v[:, 1].mean(), v[:, 0]-v[:, 0].mean()))
68     v = v[order]
69     d2 = np.sum((v[:, None, :]-v[None, :, :])**2, axis=2)
70     i, j = np.unravel_index(np.argmax(d2), d2.shape)
71     D = float(np.sqrt(d2[i, j]))
72     center = (v[i]+v[j])/2
73     distances = np.linalg.norm(v-center, axis=1)
74     return {'status': 'bounded', 'vertices': v.tolist(), 'diameter': D,
75           'diameter_endpoints': [v[i].tolist(), v[j].tolist()],
76           'diameter_circle_center': center.tolist(),
77           'diameter_circle_covers': bool(np.max(distances) <= D/2+tol),
78           'max_distance_to_diameter_center': float(np.max(distances))}
79
80
81 def solve(measurements, error_deg=1.0):
82     return solve_halfplanes(*bearing_halfplanes(measurements, error_deg))
83
84
85 def triangle_measurements():
86     vertices = np.array([[0., 0.], [20., 0.], [10., 10*np.sqrt(3)]])
87     data = []
88     for i in range(3):
89         u = (vertices[(i+1) % 3]-vertices[i])/20
90         s = vertices[i]-1000*u
91         data.append({'x': float(s[0]), 'y': float(s[1]),
92                   'bearing_deg': float((np.rad2deg(np.arctan2(u[1], u[0]))+1) % 360)})
93     return data
94
95
96 if __name__ == '__main__':
97     data = json.loads(Path(sys.argv[1]).read_text()) if len(sys.argv)>1 else triangle_measurements()
98     print(json.dumps({'measurements': data, 'result': solve(data)}, ensure_ascii=False, indent=2))

```

程序 57

对应文件: question1/test_solve.py; question3/results/dispatch_20260913/development/code_snapshot/question1/test_solve.py; question3/results/dispatch_20260913/smoke/code_snapshot/question1/test_solve.py

```

1 import unittest
2 import numpy as np
3 from solve import solve, solve_halfplanes, bearing_halfplanes, triangle_measurements
4
5 class GeometryTests(unittest.TestCase):
6     def test_rectangle(self):
7         r = solve_halfplanes([[1,0],[-1,0],[0,1],[0,-1]], [3,0,4,0])
8         self.assertAlmostEqual(r['diameter'], 5)
9         self.assertTrue(r['diameter_circle_covers'])
10    def test_actual_wedge_triangle_counterexample(self):
11        r = solve(triangle_measurements())
12        self.assertEqual(len(r['vertices']), 3)
13        self.assertAlmostEqual(r['diameter'], 20, places=7)
14        self.assertFalse(r['diameter_circle_covers'])
15        self.assertAlmostEqual(r['max_distance_to_diameter_center'], 10*np.sqrt(3), places=7)
16    def test_empty(self):
17        self.assertEqual(solve_halfplanes([[1,0],[-1,0]], [0,-1])['status'], 'empty')
18    def test_unbounded_wedge(self):
19        self.assertEqual(solve([{'x':0,'y':0,'bearing_deg':0}])['status'], 'unbounded')
20    def test_segment(self):
21        r = solve_halfplanes([[1,0],[-1,0],[0,1],[0,-1]], [3,0,0,0])
22        self.assertAlmostEqual(r['diameter'], 3)
23    def test_point(self):
24        r = solve_halfplanes([[1,0],[-1,0],[0,1],[0,-1]], [0,0,0,0])
25        self.assertEqual(r['diameter'], 0)
26    def test_wraparound_and_backward_rejection(self):
27        A,b = bearing_halfplanes([{'x':0,'y':0,'bearing_deg':359.8}])
28        self.assertTrue(np.all(A @ [100,0] <= b+1e-8))

```

```

29     self.assertFalse(np.all(A @ [-100,0] <= b+1e-8))
30 def test_parallel_duplicates(self):
31     r=solve_halfplanes([[1,0],[-1,0],[0,1],[0,-1],[1,0]], [3,0,4,0,3])
32     self.assertAlmostEquals(r['diameter'], 5)
33 def test_negative_coordinates(self):
34     r=solve_halfplanes([[1,0],[-1,0],[0,1],[0,-1]], [-1,4,-2,6])
35     self.assertAlmostEquals(r['diameter'], 5)
36 def test_random_true_source_containment(self):
37     rng = np.random.default_rng(20260911)
38     for _ in range(50):
39         g=rng.uniform(-300,300,2)
40         measurements=[]
41         for angle in [0,120,240]:
42             a=np.deg2rad(angle+rng.uniform(-10,10))
43             s=g+600*np.array([np.cos(a),np.sin(a)])
44             theta=np.rad2deg(np.arctan2(*(g-s)[::-1]))+rng.uniform(-1,1)
45             measurements.append(dict(x=s[0], y=s[1], bearing_deg=theta%360))
46         A,b=bearing_halfplanes(measurements)
47         self.assertTrue(np.all(A@g <= b+1e-7))
48         r=solve(measurements)
49         self.assertEqual(r['status'],'bounded')
50         v=np.array(r['vertices'])
51         self.assertTrue(np.all(A@v.T <= b[:None]+1e-7))
52         # Independent projection lower bounds on diameter.
53         for a in np.linspace(0,2*np.pi,20):
54             projections=v@np.array([np.cos(a),np.sin(a)])
55             self.assertLessEqual(np.ptp(projections),r['diameter']+1e-7)
56 def test_invalid_input(self):
57     with self.assertRaises(ValueError): solve([])
58     with self.assertRaises(ValueError): solve([dict(x=float('nan'), y=0, bearing_deg=0)])
59
60 if __name__ == '__main__': unittest.main(verbosity=2)

```

程序 58

对应文件: question2/draw_region.py; question3/results/dispatch_20260913/development/code_snapshot/question2/draw_region.py; question3/results/dispatch_20260913/smoke/code_snapshot/question2/draw_region.py

```

1 """Write a standalone SVG scientific diagram; sampled fill, analytic constraints in report."""
2 from pathlib import Path
3 import numpy as np
4 from strategy import candidate, DELTA
5
6 def point(a,b):return (80+.36*a,430-.36*b)
7 parts=['<svg xmlns="http://www.w3.org/2000/svg" width="1060" height="850" viewBox="0 0 1060 850">',
8 '<rect width="1060" height="850" fill="white"/>',
9 '<style>text{font-family:Arial,sans-serif;fill:#183047}.title{font-size:24px;font-weight:600}.body{font-size:16px}.smj
↳ all{font-size:13px}</style>',
10 '<text x="50" y="40" class="title">Second detection point: guaranteed reception and geometry</text>',
11 '<text x="50" y="66" class="body">Local coordinates: first detection at S, first bearing along +a (metres)</text>']
12 for a in range(0,1051,10):
13     for b in range(-1050,1051,10):
14         h=a*np.cos(DELTA)-abs(b)*np.sin(DELTA)
15         reception=a*a+b*b-10*h+25<=1e6 and a*a+b*b<=2000*h
16         if reception:
17             x,y=point(a,b)
18             color='#24a896' if candidate([a,b]) else '#dde4ea'
19             parts.append(f'<rect x="{x-1.8:.2f}" y="{y-1.8:.2f}" width="3.6" height="3.6" fill="{color}"/>')
20 for r in [500,1000,1500]:
21     x,_=point(r,0)
22     parts.append(f'<line x1="{x}" y1="100" x2="{x}" y2="790" stroke="#ccd4dc" stroke-dasharray="3 5"/>')
23     parts.append(f'<text x="{x-16}" y="450" class="small">{r}</text>')
24 for b in [-1000,-500,500,1000]:
25     _,y=point(0,b)
26     parts.append(f'<text x="25" y="{y+5}" class="small">{b}</text>')
27 parts += ['<line x1="65" y1="430" x2="665" y2="430" stroke="#30475c"/>', '<line x1="80" y1="810" x2="80" y2="90"
↳ stroke="#30475c"/>',
28 '<text x="650" y="420" class="body">a</text>', '<text x="65" y="95" class="body">b</text>']
29 x1,y1=point(1500*np.cos(DELTA),1500*np.sin(DELTA));x2,y2=point(1500*np.cos(DELTA),-1500*np.sin(DELTA))

```

```

30 parts.append(f'<path d="M80 430 L{x1} {y1} L{x2} {y2} Z" fill="#f7b44b" fill-opacity=".8" stroke="#bd7519"/>')
31 for a,b,label in [(0,0,'S'),(800,605,'Q+ (800, 605)'),(800,-605,'Q- (800, -605)')]:
32     x,y=point(a,b)
33     if a:parts.append(f'<line x1="80" y1="430" x2="{x}" y2="{y}" stroke="#1e6575" stroke-dasharray="6 4"/>')
34     parts.append(f'<circle cx="{x}" cy="{y}" r="5" fill="#123a54"/><text x="{x+10}" y="{y-10}"
    ↪ class="body">{label}</text>')
35 for y,color,label in [(160,'#dde4ea','Guaranteed reception region'),(210,'#24a896','Safe candidates: angle >= 20
    ↪ deg'),(260,'#f7b44b','Initial +/-1 deg source sector')]:
36     parts.append(f'<rect x="700" y="{y-15}" width="20" height="20" fill="{color}"/><text x="730" y="{y}"
    ↪ class="body">{label}</text>')
37 for i,line in enumerate(['Full source sector: 5 < r <= 1500 m','Receiver radius: 1000 <= R <= 1500 m','Blue dots:
    ↪ sampled minimax design','','Candidate fill sampled every 10 m.','Analytic inequalities define the
    ↪ region.','Target-disc clipping is not used.']):
38     parts.append(f'<text x="700" y="{330+i*27}" class="small">{line.replace("<","&lt;")}</text>')
39 parts.append(f'<text x="50" y="830" class="small">Reflection gives equivalent choices for the full sector; actual
    ↪ target-disc clipping may break this symmetry.</text></svg>')
40 Path(__file__).with_name('candidate_region.svg').write_text('\n'.join(parts))

```

程序 59

对应文件: question2/strategy.py; question3/innovation/frozen/question2/strategy.py; question3/results/dispatch_20260913/development/code_snaps hot/question2/strategy.py; question3/results/dispatch_20260913/smoke/co de_snapshot/question2/strategy.py; question3/results/round13_adverse_ma in/code_snapshot/question2/strategy.py; question3/results/round13_adve rse_rounding/code_snapshot/question2/strategy.py; question3/results/ro und13_dev_main/code_snapshot/question2/strategy.py; question3/results/ round13_dev_rounding/code_snapshot/question2/strategy.py; question3/re sults/round13_holdout_main/code_snapshot/question2/strategy.py; question 3/results/round13_holdout_rounding/code_snapshot/question2/strategy.py; question3/results/round13_outer_main/code_snapshot/question2/strategy.py; question3/results/round13_outer_rounding/code_snapshot/question2/strateg y.py; question3/results/round14_adverse_main/code_snapshot/question2/str ategy.py; question3/results/round14_adverse_rounding/code_snapshot/quest ion2/strategy.py; question3/results/round14_dev_main/code_snapshot/quest ion2/strategy.py; question3/results/round14_dev_rounding/code_snapshot/q uestion2/strategy.py; question3/results/round14_holdout_main/code_snapsh ot/question2/strategy.py; question3/results/round14_holdout_rounding/cod e_snapshot/question2/strategy.py; question3/results/round14_initial_dev_ main/code_snapshot/question2/strategy.py; question3/results/round14_init ial_dev_rounding/code_snapshot/question2/strategy.py; question3/results/ round14_outer_main/code_snapshot/question2/strategy.py; question3/result s/round14_outer_rounding/code_snapshot/question2/strategy.py; question3/ results/round17_adverse_main/code_snapshot/question2/strategy.py; questi on3/results/round17_adverse_rounding/code_snapshot/question2/strategy.py; question3/results/round17_dev_main/code_snapshot/question2/strategy.py; question3/results/round17_dev_rounding/code_snapshot/question2/strateg y.py; question3/results/round18_adverse_main/code_snapshot/question2/s trategy.py; question3/results/round18_adverse_rounding/code_snapshot/que stion2/strategy.py; question3/results/round18_dev_main/code_snapshot/que stion2/strategy.py; question3/results/round18_dev_rounding/code_snapshot

/question2/strategy.py; question3/results/round18_holdout_main/code_snapshot/question2/strategy.py; question3/results/round18_holdout_rounding/code_snapshot/question2/strategy.py; question3/results/round18_outer_main/code_snapshot/question2/strategy.py; question3/results/round18_outer_rounding/code_snapshot/question2/strategy.py; question3/results/round19_adverse_main/code_snapshot/question2/strategy.py; question3/results/round19_adverse_rounding/code_snapshot/question2/strategy.py; question3/results/round19_dev_main/code_snapshot/question2/strategy.py; question3/results/round19_dev_rounding/code_snapshot/question2/strategy.py; question3/results/round19_holdout_main/code_snapshot/question2/strategy.py; question3/results/round19_holdout_rounding/code_snapshot/question2/strategy.py; question3/results/round19_outer_main/code_snapshot/question2/strategy.py; question3/results/round19_outer_rounding/code_snapshot/question2/strategy.py; question3/results/round1_main/code_snapshot/question2/strategy.py; question3/results/round1_rounding/code_snapshot/question2/strategy.py

```

1 """Conservative second-bearing design. Local x axis = first measured bearing.
2 Run python question2/strategy.py. Requires numpy; scipy only for tests via Q1.
3 Numerical minimax search is sampled, NOT a certificate of continuous optimality.
4 """
5 import json
6 from pathlib import Path
7 import numpy as np
8
9 DELTA = np.deg2rad(1.)
10
11 def candidate(q, min_angle_deg=20., error_deg=1.):
12     """Analytic sufficient certificate, not a sampled feasibility test.
13
14     error_deg also applies to reception geometry (not only intersection width).
15     Physical ranges are the problem's fixed 5/1000/1500 metres.
16     """
17     a,b = np.asarray(q, float)
18     if not np.isfinite([a,b,error_deg]).all() or not 0 < min_angle_deg < 90 or not 0 < error_deg < 45:
19         return False
20     if a < 0: return False
21     delta = np.deg2rad(error_deg)
22     h = a*np.cos(delta)-abs(b)*np.sin(delta)
23     norm2=a*a+b*b
24     reception = norm2-10*h+25 <= 1000**2+1e-8 and norm2 <= 2000*h+1e-8
25     cross_lower=abs(b)*np.cos(delta)-a*np.sin(delta)
26     max_distance=np.sqrt(max(norm2+25-10*h, norm2+1500**2-3000*h))
27     return bool(reception and cross_lower > 5 and cross_lower >= max_distance*np.sin(np.deg2rad(min_angle_deg)))
28
29
30 def to_world(q, first_position=(0,0), bearing_deg=0):
31     angle=np.deg2rad(bearing_deg)
32     rotation=np.array([[np.cos(angle), -np.sin(angle)], [np.sin(angle), np.cos(angle)]])
33     return np.asarray(first_position)+rotation@np.asarray(q)
34
35
36 def scenarios(nr=51, na=5, ne=5):
37     r,alpha,error=np.meshgrid(np.linspace(5.001,1500,nr),np.linspace(-DELTA,DELTA,na),
38                               np.linspace(-DELTA,DELTA,ne),indexing='ij')
39     g=np.column_stack([(r*np.cos(alpha)).ravel(),(r*np.sin(alpha)).ravel()])
40     return g,error.ravel()
41
42
43 def diameters(q,g,error,error_deg=1.):
44     """Exact four-half-plane vertex diameter for every scenario, in float arithmetic.
45     Requires geometry in candidate(), which guarantees bounded intersections.
46     """
47     delta=np.deg2rad(error_deg)

```

```

48     q=np.asarray(q,float)
49     theta=np.arctan2(g[:,1]-q[1],g[:,0]-q[0])+error
50     count=len(g)
51     A=np.empty((count,4,2))
52     A[:,0]=[np.sin(-delta),-np.cos(-delta)]
53     A[:,1]=[-np.sin(delta),np.cos(delta)]
54     A[:,2,0]=np.sin(theta-delta); A[:,2,1]=-np.cos(theta-delta)
55     A[:,3,0]=-np.sin(theta+delta); A[:,3,1]=np.cos(theta+delta)
56     rhs=np.zeros((count,4)); rhs[:,2]=A[:,2:]@q
57     points=[];valid=[]
58     for i in range(4):
59         for j in range(i+1,4):
60             det=A[:,i,0]*A[:,j,1]-A[:,i,1]*A[:,j,0]
61             safe=np.abs(det)>1e-12
62             den=np.where(safe,det,1)
63             p=np.column_stack([(rhs[:,i]*A[:,j,1]-A[:,i,1]*rhs[:,j])/den,
64                               (A[:,i,0]*rhs[:,j]-rhs[:,i]*A[:,j,0])/den])
65             ok=safe & np.all(np.einsum('nij,nj->ni',A,p)<=rhs+1e-7,axis=1)
66             points.append(p);valid.append(ok)
67     v=np.stack(points,axis=1);ok=np.stack(valid,axis=1)
68     if np.any(ok.sum(axis=1)<2): raise RuntimeError('Degenerate scenario outside solver assumptions')
69     max2=np.zeros(count)
70     for i in range(6):
71         for j in range(i+1,6):
72             d2=np.sum((v[:,i]-v[:,j])**2,axis=1)
73             max2=np.maximum(max2,np.where(ok[:,i]&ok[:,j],d2,0))
74     return np.sqrt(max2)
75
76
77 def search():
78     g,e=scenarios()
79     best=None;tested=0
80     # Reflection symmetry: search upper component, reflect for second recommendation.
81     for step,arange,brange in [(50,np.arange(0,1001,50),np.arange(50,1001,50)),]:
82         for a in arange:
83             for b in brange:
84                 if not candidate([a,b]):continue
85                 score=float(diameters([a,b],g,e).max());tested+=1
86                 row=(score,float(np.hypot(a,b)),float(a),float(b))
87                 if best is None or row<best:best=row
88     _,a0,b0=best
89     for a in np.arange(max(0,a0-50),a0+51,5):
90         for b in np.arange(max(5,b0-50),b0+51,5):
91             if not candidate([a,b]):continue
92             score=float(diameters([a,b],g,e).max());tested+=1
93             row=(score,float(np.hypot(a,b)),float(a),float(b))
94             if row<best:best=row
95     return best,tested
96
97
98 def main():
99     best,tested=search();score,distance,a,b=best
100     g,e=scenarios(301,11,11)
101     rows=[]
102     for name,q in [('recommended',[a,b]),('balanced',[500,500]),('forward_biased',[750,400])]:
103         if not candidate(q):continue
104         d=diameters(q,g,e);k=int(d.argmax())
105         rows.append(dict(name=name,local_point=q,move_distance=float(np.linalg.norm(q)),
106                         move_and_detection_seconds=float(np.linalg.norm(q)/5+5),
107                         sampled_worst_diameter=float(d[k]),worst_source=g[k].tolist(),
108                         worst_second_error_deg=float(np.rad2deg(e[k])))
109     result=dict(parameters=dict(error_deg=1,min_angle_deg=20,initial_range=[5,1500]),
110                search_scenarios=51*5*5,candidates_evaluated=tested,
111                validation_scenarios=len(g),recommendation=[a,b],mirror=[a,-b],
112                sampled_search_worst=score,comparisons=rows,
113                caveat='Sampled minimax on a conservative full-sector candidate region; not continuous global
114                        ↪ optimum.')
114     Path(__file__).with_name('results.json').write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n')
115     print(json.dumps(result,ensure_ascii=False,indent=2))
116
117 if __name__=='__main__':main()

```

程序 60

对应文件: question2/test_strategy.py; question3/results/dispatch_20260913/development/code_snapshot/question2/test_strategy.py; question3/results/dispatch_20260913/smoke/code_snapshot/question2/test_strategy.py

```
1 import sys
2 import unittest
3 from pathlib import Path
4 import numpy as np
5 sys.path.insert(0, str(Path(__file__).resolve().parents[1]/'question1'))
6 from solve import solve
7 from strategy import candidate, scenarios, diameters, to_world, DELTA
8
9 class StrategyTests(unittest.TestCase):
10     def test_recommended_and_reflection(self):
11         self.assertTrue(candidate([800, 605]))
12         self.assertTrue(candidate([800, -605]))
13     def test_collinear_and_sideways_rejected(self):
14         for q in [[0, 0], [700, 0], [0, 500], [-10, 100], [2000, 500], [float('nan'), 1]]:
15             self.assertFalse(candidate(q))
16     def test_continuous_conditions_against_dense_geometry(self):
17         rng = np.random.default_rng(29)
18         g, e = scenarios(121, 31, 1)
19         r = np.linalg.norm(g, axis=1)
20         qs = [[800, 605], [800, -605], [500, 500]]
21         qs += [q for q in rng.uniform([0, 0], [1000, 1000], (100, 2)) if candidate(q)]
22         self.assertGreater(len(qs), 10)
23         for q in qs:
24             d = np.linalg.norm(g - q, axis=1)
25             self.assertTrue(np.all(d <= np.maximum(1000, r) + 1e-7))
26             self.assertTrue(np.all(d > 5))
27             cross = np.abs(g[:, 0] * (g[:, 1] - q[1]) - g[:, 1] * (g[:, 0] - q[0]))
28             self.assertTrue(np.all(cross / (r * d) >= np.sin(np.deg2rad(20)) - 1e-10))
29     def test_vectorized_diameter_matches_question1(self):
30         g, e = scenarios(7, 3, 3)
31         q = [800, 605]
32         exact = diameters(q, g, e)
33         for k in range(0, len(g), 3):
34             theta = np.rad2deg(np.arctan2(g[k, 1] - q[1], g[k, 0] - q[0]) + e[k]) % 360
35             result = solve([dict(x=0, y=0, bearing_deg=0), dict(x=q[0], y=q[1], bearing_deg=theta)])
36             self.assertEqual(result['status'], 'bounded')
37             self.assertAlmostEqual(exact[k], result['diameter'], places=6)
38     def test_frame_invariance(self):
39         self.assertTrue(np.allclose(to_world([800, 605], [100, -200], 90), [-505, 600]))
40         g, e = scenarios(3, 3, 3); q = np.array([800, 605]); j = 7
41         theta = np.rad2deg(np.arctan2(g[j, 1] - q[1], g[j, 0] - q[0]) + e[j])
42         world = to_world(q, [123, -456], 359)
43         result = solve([dict(x=123, y=-456, bearing_deg=359), dict(x=world[0], y=world[1], bearing_deg=(theta+359)%360)])
44         self.assertAlmostEqual(diameters(q, g, e)[j], result['diameter'], places=6)
45     def test_reflected_objective(self):
46         g, e = scenarios(7, 3, 3)
47         self.assertTrue(np.allclose(diameters([800, 605], g, e), diameters([800, -605], g * [1, -1], -e)))
48     def test_no_guarantee_for_perpendicular_move(self):
49         g = np.array([1500., 0.]); q = np.array([0., 500.])
50         self.assertGreater(np.linalg.norm(g - q), 1500)
51     def test_collinear_measurements_unbounded(self):
52         # Collinear forward baseline has an unbounded overlap for a farther source.
53         r = solve([dict(x=0, y=0, bearing_deg=0), dict(x=500, y=0, bearing_deg=0)])
54         self.assertEqual(r['status'], 'unbounded')
55
56 if __name__ == '__main__': unittest.main(verbosity=2)
```

程序 61

对应文件: question3/__init__.py; question3/innovation/frozen/question3/__init__.py; question3/results/dispatch_20260913/development/code_snapshot/question3/__init__.py; question3/results/dispatch_20260913/smoke/co

[illegible]

```
1 """Omnidirectional baseline. Self simulation is not official rehearsal."""
```

程序 62

对应文件: question3/analyze_matrix.py; question3/innovation/frozen/question3/analyze_matrix.py; question3/results/dispatch_20260913/development/code_snapshot/question3/analyze_matrix.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/analyze_matrix.py; question3/results/round13_adverse_main/code_snapshot/question3/analyze_matrix.py; question3/results/round13_adverse_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round13_dev_main/code_snapshot/question3/analyze_matrix.py; question3/results/round13_dev_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round13_holdout_main/code_snapshot/question3/analyze_matrix.py; question3/results/round13_holdout_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round13_outer_main/code_snapshot/question3/analyze_matrix.py; question3/results/round13_outer_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round14_adverse_main/code_snapshot/question3/analyze_matrix.py; question3/results/round14_adverse_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round14_dev_main/code_snapshot/question3/analyze_matrix.py; question3/results/round14_dev_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round14_holdout_main/code_snapshot/question3/analyze_matrix.py; question3/results/round14_holdout_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round14_initial_dev_main/code_snapshot/question3/analyze_matrix.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round14_outer_main/code_snapshot/question3/analyze_matrix.py; question3/results/round14_outer_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round17_adverse_main/code_snapshot/question3/analyze_matrix.py; question3/results/round17_adverse_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round17_dev_main/code_snapshot/question3/analyze_matrix.py; question3/results/round17_dev_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round18_adverse_main/code_snapshot/question3/analyze_matrix.py; question3/results/round18_adverse_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round18_dev_main/code_snapshot/question3/analyze_matrix.py; question3/results/round18_dev_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round18_holdout_main/code_snapshot/question3/analyze_matrix.py; question3/results/round18_holdout_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round18_outer_main/code_snapshot/question3/analyze_matrix.py; question3/results/round18_outer_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round19_adverse_main/code_snapshot/question3/analyze_matrix.py; question3/results/round19_adverse_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round1

9_dev_main/code_snapshot/question3/analyze_matrix.py; question3/results/round19_dev_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round19_holdout_main/code_snapshot/question3/analyze_matrix.py; question3/results/round19_holdout_rounding/code_snapshot/question3/analyze_matrix.py; question3/results/round19_outer_main/code_snapshot/question3/analyze_matrix.py; question3/results/round19_outer_rounding/code_snapshot/question3/analyze_matrix.py

```

1 """Report every case, failure penalty, tails and factorial conditional effects."""
2 import argparse
3 import csv
4 import json
5 from pathlib import Path
6 import numpy as np
7 from .run_matrix import VARIANTS
8
9
10 def good(row): return row['status']=='complete' and row['clearance_ratio']==1 and row.get('audit')=='pass' and not
    ↪ row.get('false_complete')
11 def objective(row): return row['metrics']['virtual_time_s'] if good(row) else
    ↪ max(360000, row['metrics']['virtual_time_s'])
12
13 def main():
14     p=argparse.ArgumentParser();p.add_argument('folder');p.add_argument('--reference',default='baseline');args=p.parse_args()
    ↪ e_args();root=Path(args.folder)
15     data=json.loads((root/'summary.json').read_text());lookup={r['variant'],r['case_id']}:r for r in data['cases']}
16     ids=sorted({r['case_id'] for r in data['cases']});stats=[]
17     variants=list(data['configs'])
18     if args.reference not in variants:raise ValueError('Missing reference variant')
19     for name in variants:
20         rows=[lookup[name,i] for i in ids];base=[lookup[args.reference,i] for i in ids]
21         values=np.array([objective(r) for r in rows]);raw=np.array([r['metrics']['virtual_time_s'] for r in rows])
22         improvement=1-values/np.array([objective(r) for r in base]);wall=[r['elapsed_wall_s'] for r in rows]
23         worst=int(np.argmax(values));regress=int(np.argmin(improvement));per_clear=[r['average_virtual_per_cleared']
    ↪ for r in rows]
24         stats.append(dict(variant=name,runs=len(rows),complete=sum(good(r) for r in rows),
25                             failed=sum(not good(r) for r in rows),timeouts=sum(r['timeout'] for r in rows),
26                             false_complete=sum(r['false_complete'] for r in rows),
27                             mean_clearance_ratio=float(np.mean([r['clearance_ratio'] for r in rows])),
28                             mean_virtual_s=float(values.mean()),raw_all_case_mean_virtual_s=float(raw.mean()),
29                             p95_virtual_s=float(np.percentile(values,95)),cvar_worst10_s=float(np.sort(values)[-max(1,i)
    ↪ nt(np.ceil(.1*len(rows))))].mean()),
30                             max_virtual_s=float(values.max()),max_virtual_case=ids[worst],
31                             mean_case_average_per_clear_s=None if any(x is None for x in per_clear) else
    ↪ float(np.mean(per_clear)),
32                             pooled_virtual_per_clear_s=float(raw.sum()/sum(r['cleared'] for r in rows)) if
    ↪ sum(r['cleared'] for r in rows) else None,
33                             median_paired_improvement=float(np.median(improvement)),
34                             worst_case_improvement=float(improvement.min()),worst_regression_case=ids[regress],
35                             worse_than_baseline=int(np.sum(improvement < -1e-9)),wins=int(np.sum(improvement>1e-9)),
36                             wall_p95_s=float(np.percentile(wall,95)),wall_max_s=max(wall),
37                             mean_failed_clears=float(np.mean([r['metrics']['counts']['clear_failure'] for r in rows])),
38                             max_failed_clears=max(r['metrics']['counts']['clear_failure'] for r in rows),
39                             mean_components={k:float(np.mean([r['metrics']['components'][k] for r in rows])) for k in
    ↪ rows[0]['metrics']['components']})
40     effects=[]
41     for module in 'ABC':
42         for without in variants:
43             if module in without or without=='baseline' and False:continue
44             letters='' if without=='baseline' else without
45             with_name=''.join(sorted(letters+module))
46             if with_name not in variants:continue
47             before=np.array([objective(lookup[without,i]) for i in
    ↪ ids]);after=np.array([objective(lookup[with_name,i]) for i in ids])
48             change=1-after/before;k=int(np.argmin(change))
49             effects.append(dict(module=module,without=without,with_variant=with_name,mean_saved_s=float(np.mean(befor
    ↪ e-after)),
50                                     median_paired_improvement=float(np.median(change)),wins=int(np.sum(change>1e-9)),
51                                     losses=int(np.sum(change< -1e-9)),worst_change=float(change[k]),worst_case=ids[k]))

```

```

52 interactions=[]
53 mean={s['variant']:s['mean_virtual_s'] for s in stats}
54 for a,b in [('A','B'),('A','C'),('B','C')]:
55     if not all(k in mean for k in [a,b,a+b,'baseline']):continue
56     interactions.append(dict(pair=a+b,additive_interaction_s=mean[a+b]-mean[a]-mean[b]+mean['baseline'],
57                             definition='T_AB - T_A - T_B + T_baseline; negative = savings exceed additive
58                                     ↪ single-module savings'))
59 failures=[r for r in data['cases'] if not good(r)]
60 report=dict(reference=args.reference,kind=data['kind'],rounding=data['rounding'],stats=stats,effects=effects,inte
61             ↪ ractions=interactions,failures=failures,
62             notes=['All runs retained. Failure/timeout/noncertified runs ranked with >=360000s penalty.',
63                   'Percentiles include all cases; experiment kind identifies development versus held-out
64                   ↪ fixtures.'])
65 (root/'comparison.json').write_text(json.dumps(report,indent=2),encoding='utf-8')
66 with (root/'paired_cases.csv').open('w',newline='',encoding='utf-8') as f:
67     writer=csv.writer(f);writer.writerow(['case_id']+variants)
68     for i in ids:writer.writerow([i]+[objective(lookup[n,i]) for n in variants])
69 for s in stats:print(s['variant'],f'{s["complete"]}/{s["runs"]}',f'mean={s["mean_virtual_s"]:.2f}',f'p95={s["p95_
70             ↪ virtual_s"]:.2f}',
71             f'max={s["max_virtual_s"]:.2f}',f'median_gain={s["median_paired_improvement"]:.2%}',
72             f'worst_gain={s["worst_case_improvement"]:.2%}',s['worst_regression_case'])
73 print('EFFECTS',json.dumps(effects));print('INTERACTIONS',json.dumps(interactions))
74
75 if __name__=='__main__':main()

```

程序 63

对应文件: question3/audit.py; question3/innovation/frozen/question3/audit.py; question3/results/dispatch_20260913/development/code_snapshot/question3/audit.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/audit.py; question3/results/round19_adverse_main/code_snapshot/question3/audit.py; question3/results/round19_adverse_rounding/code_snapshot/question3/audit.py; question3/results/round19_dev_main/code_snapshot/question3/audit.py; question3/results/round19_dev_rounding/code_snapshot/question3/audit.py; question3/results/round19_holdout_main/code_snapshot/question3/audit.py; question3/results/round19_holdout_rounding/code_snapshot/question3/audit.py; question3/results/round19_outer_main/code_snapshot/question3/audit.py; question3/results/round19_outer_rounding/code_snapshot/question3/audit.py

```

1 """Post-run oracle audit: evaluator only, NEVER called by the strategy."""
2 import argparse
3 import json
4 import math
5 import statistics
6 from pathlib import Path
7 import numpy as np
8 from question2.strategy import candidate
9 from .geometry import stations,station_cover_bound
10
11
12 def audit_folder(folder):
13     case=json.loads((folder/'evaluator_hidden_case.json').read_text(encoding='utf-8'))
14     result=json.loads((folder/'result.json').read_text(encoding='utf-8'))
15     sources={s['channel']:s for s in case['sources']};requests={};cleared=set();negative={c:set() for c in
16             ↪ range(1,21)}
17     position=(0.,0.);channel=1;vt=0.;done=set();bearings={};measure_moves={};checks=0;outer_entry=None
18     for line in (folder/'actions.jsonl').read_text(encoding='utf-8').splitlines():
19         row=json.loads(line)
20         if row['type']=='request':requests[row['payload']['request_id']]=row
21         elif row['type']=='response' and row['response'].get('accepted') is True:
22             rid=row['request_id']
23             if rid in done:continue

```

```

23     done.add(rid);req=requests[rid];b=req['payload'];r=row['response'];path=req['path']
24     if path in ('/measure','/clear'):
25         p=(b['position']['x'],b['position']['y']);c=b['channel'];move=math.dist(position,p)
26         if path=='/measure' and req['phase']=='search' and math.hypot(*p)>1e-7 and outer_entry is None:
27             outer_entry=(position,p)
28         vt+=move/5;position=p
29         s=sources.get(c);distance=math.inf if s is None or c in cleared else
        ↪ math.hypot(s['x']-p[0],s['y']-p[1])
30         if path=='/measure':
31             measure_moves[p,c]=move
32             vt+=5*(c!=channel);channel=c
33             expected='no_signal' if s is None or c in cleared or distance>s['radius'] else ('near' if
        ↪ distance<=5 else 'direction')
34             assert r['measure_result']==expected,(folder,expected,r)
35             if expected=='no_signal' and req['phase']=='search':negative[c].add(p)
36             if expected=='direction':
37                 true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
38                 envelope=1. if result['summary']['rounding']=='bounded' else 1.005
39                 assert abs((r['svd_deg']-true+180)%360-180)<=envelope+1e-9
40                 key=(p,c)
41                 if key in bearings:assert bearings[key]==r['svd_deg']
42                 bearings[key]=r['svd_deg']
43         else:
44             vt+=3
45             if distance<=20:
46                 assert r['clear_result']=='success';cleared.add(c);vt+=2
47             else:assert r['clear_result']=='no_target_in_range'
48             checks+=1
49             assert abs(vt-r['virtual_time_s'])<=len(done)*1e-6+1e-4
50     strategy=result['strategy']
51     assert strategy['status']=='complete' and strategy['exit_confirmed']
52     assert set(strategy['cleared_channels'])==cleared
53     cover={tuple(s) for s in strategy['station_positions']}
54     assert len(cover)==7
55     radius=math.dist((0,0),strategy['station_positions'][1])
56     assert station_cover_bound(radius)<1000
57     phase=strategy.get('station_phase_rad',0.)
58     assert math.isfinite(phase)
59     assert np.allclose(strategy['station_positions'],stations(radius,phase),atol=1e-7,rtol=0)
60     certificate=strategy.get('stopping_certificate') or {}
61     if certificate.get('method')=='source_count_upper_bound':
62         # Count accepted SUCCESS feedback on distinct channels, never attempts or detections.
63         assert certificate['source_upper_bound']==16
64         assert len(certificate['successful_channels'])==16
65         assert len(cleared)==16 and set(certificate['successful_channels'])==cleared
66         assert set(certificate['inferred_absent_channels'])==set(range(1,21))-cleared
67         assert len(sources)<=16 # Evaluator checks the published model's applicability.
68     else:
69         for c in range(1,21):assert c in cleared or cover<=negative[c],(folder,c)
70     assert cleared==set(sources)
71     phase_events=[e for e in map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()) if
        ↪ e['type']=='ring_phase_selected']
72     assert len(phase_events)==(1 if 'station_phase_rad' in strategy else 0)
73     if phase_events:
74         e=phase_events[0];i=e['station_id'];assert 1<=i<=6 and e['visited_before']==[0]
75         assert e['phase_rad']==phase and outer_entry is not None
76         assert tuple(e['position'])==outer_entry[0] and tuple(e['new_station'])==outer_entry[1]
77         assert np.allclose(e['old_station'],stations(radius)[i],atol=1e-7,rtol=0)
78         assert tuple(strategy['station_positions'][i])==outer_entry[1]
79         shortest=abs(math.hypot(*outer_entry[0])-radius)
80         assert abs(math.dist(*outer_entry)-shortest)<1e-7
81         saved=(math.dist(outer_entry[0],e['old_station'])-math.dist(*outer_entry))/5
82         assert saved>=-1e-7 and abs(saved-e['local_movement_saved_s'])<1e-7
83     for region in strategy['regions']:
84         selection=region.get('selection',{})
85         if selection.get('mode') in ('domain_cost_proxy','domain_current','domain_shared'):
86             first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
87             theta=math.radians(bearings[first,c])%(2*math.pi);d=math.radians(selection['error_deg'])
88             assert selection['error_deg']>=1.005 and selection['range_prior']=='published_domain_1800'
89             assert 4*selection['error_deg']<selection['min_angle_deg']<90
90             rho=math.hypot(*first);assert rho<=1800
91             directions=[theta-d,theta+d];opposite=math.atan2(first[1],first[0])+math.pi
92             directions.extend(opposite+2*k*math.pi for k in range(-2,3) if theta-d<=opposite+2*k*math.pi<=theta+d)

```

```

93     projection=min(first[0]*math.cos(t)+first[1]*math.sin(t) for t in directions)
94     bound=min(1500.,-projection+math.sqrt(max(0.,projection**2+1800**2-rho**2)))
95     upper=selection['source_range_upper_m'];assert 5<upper<1500 and upper>=bound-1e-7
96     g=(sources[c]['x'],sources[c]['y']);assert math.dist(g,first)<=upper+1e-7
97     dx=second[0]-first[0];dy=second[1]-first[1]
98     a=dx*math.cos(theta)+dy*math.sin(theta);b=-dx*math.sin(theta)+dy*math.cos(theta)
99     assert np.allclose((a,b),selection['offset'],atol=1e-7,rtol=0) and a>=-1e-7
100    h=a*math.cos(d)-abs(b)*math.sin(d);n=a*a+b*b
101    radii=[5,upper]+([1000] if upper>1000 else [])
102    for radius in radii:assert n+radius**2-2*radius*h<=max(1000,radius)**2+1e-6
103    maximum=math.sqrt(max(n+25-10*h,n+upper**2-2*upper*h))
104    cross=abs(b)*math.cos(d)-a*math.sin(d)
105    assert cross>5 and cross>=maximum*math.sin(math.radians(selection['min_angle_deg']))-1e-7
106    assert (second,c) in bearings
107    if selection['mode'] in ('domain_current','domain_shared'):assert measure_moves[second,c]==0
108    if selection.get('mode') in ('shared_certified','opportunistic_current'):
109        first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
110        angle=math.radians(bearings[first,c]);dx=second[0]-first[0];dy=second[1]-first[1]
111        offset=(dx*math.cos(angle)+dy*math.sin(angle),-dx*math.sin(angle)+dy*math.cos(angle))
112        assert np.allclose(offset,selection['offset'],atol=1e-7,rtol=0)
113        assert candidate(offset,selection['min_angle_deg'],selection['error_deg'])
114        assert (second,c) in bearings
115        if selection['mode']=='opportunistic_current':assert measure_moves[second,c]==0
116    if 'vertices' not in region:continue
117    g=np.array([sources[region['channel']]['x'],sources[region['channel']]['y']]);v=np.array(region['vertices'])
118    edges=np.roll(v,-1,axis=0)-v;rel=g-v
119    cross=edges[:,0]*rel[:,1]-edges[:,1]*rel[:,0]
120    assert np.all(cross>=-1e-5) or np.all(cross<=1e-5)
121    assert region['covering_radius']<20
122    if 'single_circle_centre' in region:
123        centre=np.array(region['single_circle_centre'])
124        assert np.linalg.norm(v-centre,axis=1).max()<=20-1e-5+1e-9
125        assert np.linalg.norm(g-centre)<=20
126        assert region['planned_clear_calls']==1
127    # Independently recompute each omitted action's separating witness from accepted bearings.
128    from question1.solve import bearing_halfplanes
129    regions={r['channel']:r for r in strategy['regions']}
130    for line in (folder/'strategy.jsonl').read_text(encoding='utf-8').splitlines():
131        event=json.loads(line)
132        if event['type']!='certified_clear_skip':continue
133        c=event['channel'];region=regions[c];first=tuple(region['first_position']);second=tuple(region['second_positi
        ↪ on'])
134        if 'radius_prior' in event:
135            expected={'domain':((0.,0.),1800.),'first_received':(first,1500.),'second_received':(second,1500.)}
136            centre,radius=expected[event['radius_prior']]
137            assert tuple(event['centre'])==centre and event['containing_radius_m']==radius
138            if event['radius_prior']!='domain':assert (centre,c) in bearings
139            gap=math.dist(event['point'],centre)-radius
140            assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
141            assert abs(gap-event['separation_m'])<1e-8
142            g=(sources[c]['x'],sources[c]['y'])
143            assert math.dist(g,centre)<=radius+1e-7 and math.dist(g,event['point'])>20
144            continue
145        assert event['error_deg']>=(1. if result['summary']['rounding']=='bounded' else 1.005)
146        A,b=bearing_halfplanes([dict(x=p[0],y=p[1],bearing_deg=bearings[p,c]) for p in
        ↪ [first,second]],event['error_deg'])
147        p=np.asarray(event['point'])
148        if 'halfplane_weights' in event:
149            weights=np.asarray(event['halfplane_weights'])
150            assert weights.shape==(len(A),) and np.isfinite(weights).all() and np.all(weights>=0)
151            assert abs(float(weights.sum())-1)<1e-12
152            normal=weights@A;bound=float(weights@b);length=float(np.linalg.norm(normal))
153            assert length>=1e-8
154            gap=float((normal@p-bound)/length)
155            assert event['slack_m']>=1e-5/length
156        else:
157            k=event['halfplane_index'];gap=float((A[k]@p-b[k])/np.linalg.norm(A[k]))
158            assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
159            assert abs(gap-event['separation_m'])<1e-8
160            assert math.dist(p,(sources[c]['x'],sources[c]['y']))>20
161    return checks
162
163

```

```

164 def main():
165     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)
166     summary=json.loads((root/'summary.json').read_text(encoding='utf-8'));rows=summary['cases']
167     checks=sum(audit_folder(root/r['case_id']) for r in rows)
168     aggregate=dict(kind='SELF_POST_RUN_AUDIT',runs=len(rows),actions_checked=checks,
169                   all_stopping_certificates_valid=True,all_physics_and_time_checks_passed=True,
170                   mean_virtual_s=statistics.mean(r['metrics']['virtual_time_s'] for r in rows),
171                   mean_case_average_per_clear_s=statistics.mean(r['average_virtual_per_cleared'] for r in rows),
172                   pooled_virtual_per_clear_s=sum(r['metrics']['virtual_time_s'] for r in rows)/sum(r['cleared'] for
173                   ↪ r in rows),
174                   total_cleared_over_runs=sum(r['cleared'] for r in rows),
175                   worst_total_case=max(rows,key=lambda r:r['metrics']['virtual_time_s']),
176                   worst_per_clear_case=max(rows,key=lambda r:r['average_virtual_per_cleared']),
177                   most_failed_clear_case=max(rows,key=lambda r:r['metrics']['counts']['clear_failure']),
178                   component_means={k:statistics.mean(r['metrics']['components'][k] for r in rows) for k in
179                   ↪ rows[0]['metrics']['components']},
180                   min_wall_s=min(r['elapsed_wall_s'] for r in rows),max_wall_s=max(r['elapsed_wall_s'] for r in
181                   ↪ rows))
182     (root/'audit.json').write_text(json.dumps(aggregate,ensure_ascii=False,indent=2),encoding='utf-8')
183     print(json.dumps({k:v for k,v in aggregate.items() if not isinstance(v,dict)},ensure_ascii=False,indent=2))
184
185 if __name__=='__main__':main()

```

程序 64

对应文件: question3/batch_scheduling.py; question3/innovation/frozen/question3/batch_scheduling.py; question3/results/dispatch_20260913/development/code_snapshot/question3/batch_scheduling.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/batch_scheduling.py; question3/results/round17_adverse_main/code_snapshot/question3/batch_scheduling.py; question3/results/round17_adverse_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round17_dev_main/code_snapshot/question3/batch_scheduling.py; question3/results/round17_dev_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round18_adverse_main/code_snapshot/question3/batch_scheduling.py; question3/results/round18_adverse_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round18_dev_main/code_snapshot/question3/batch_scheduling.py; question3/results/round18_dev_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round18_holdout_main/code_snapshot/question3/batch_scheduling.py; question3/results/round18_holdout_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round18_outer_main/code_snapshot/question3/batch_scheduling.py; question3/results/round18_outer_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round19_adverse_main/code_snapshot/question3/batch_scheduling.py; question3/results/round19_adverse_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round19_dev_main/code_snapshot/question3/batch_scheduling.py; question3/results/round19_dev_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round19_holdout_main/code_snapshot/question3/batch_scheduling.py; question3/results/round19_holdout_rounding/code_snapshot/question3/batch_scheduling.py; question3/results/round19_outer_main/code_snapshot/question3/batch_scheduling.py; question3/results/round19_outer_rounding/code_snapshot/question3/batch_scheduling.py

```

1 """Batch boundary changes only scheduling, not evidence or stopping state."""
2
3
4 def gate(mode,pending,cached,jobs,remaining,consecutive):
5     if mode=='nearest_finite':return jobs,remaining,False,None
6     if mode=='known_batch' and pending:
7         return jobs,[],False,dict(mode=mode,batch_channels=sorted(pending),deferred_stations=list(remaining))
8     if mode=='cached_batch' and cached:
9         eligible=[j for j in jobs if j[1] in cached]
10        if not eligible:raise RuntimeError('Cached batch has no eligible task')
11        return eligible,[],False,dict(mode=mode,batch_channels=sorted(cached),deferred_stations=list(remaining))
12    return jobs,remaining,bool(remaining and consecutive>=2),None

```

程序 65

对应文件: question3/cached_routing.py; question3/innovation/frozen/question3/cached_routing.py; question3/results/dispatch_20260913/development/code_snapshot/question3/cached_routing.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/cached_routing.py; question3/results/round13_adverse_main/code_snapshot/question3/cached_routing.py; question3/results/round13_adverse_rounding/code_snapshot/question3/cached_routing.py; question3/results/round13_dev_main/code_snapshot/question3/cached_routing.py; question3/results/round13_dev_rounding/code_snapshot/question3/cached_routing.py; question3/results/round13_holdout_main/code_snapshot/question3/cached_routing.py; question3/results/round13_holdout_rounding/code_snapshot/question3/cached_routing.py; question3/results/round13_outer_main/code_snapshot/question3/cached_routing.py; question3/results/round13_outer_rounding/code_snapshot/question3/cached_routing.py; question3/results/round14_adverse_main/code_snapshot/question3/cached_routing.py; question3/results/round14_adverse_rounding/code_snapshot/question3/cached_routing.py; question3/results/round14_dev_main/code_snapshot/question3/cached_routing.py; question3/results/round14_dev_rounding/code_snapshot/question3/cached_routing.py; question3/results/round14_holdout_main/code_snapshot/question3/cached_routing.py; question3/results/round14_holdout_rounding/code_snapshot/question3/cached_routing.py; question3/results/round14_initial_dev_main/code_snapshot/question3/cached_routing.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/cached_routing.py; question3/results/round14_outer_main/code_snapshot/question3/cached_routing.py; question3/results/round14_outer_rounding/code_snapshot/question3/cached_routing.py; question3/results/round17_adverse_main/code_snapshot/question3/cached_routing.py; question3/results/round17_adverse_rounding/code_snapshot/question3/cached_routing.py; question3/results/round17_dev_main/code_snapshot/question3/cached_routing.py; question3/results/round17_dev_rounding/code_snapshot/question3/cached_routing.py; question3/results/round18_adverse_main/code_snapshot/question3/cached_routing.py; question3/results/round18_adverse_rounding/code_snapshot/question3/cached_routing.py; question3/results/round18_dev_main/code_snapshot/question3/cached_routing.py; question3/results/round18_dev_rounding/code_snapshot/question3/cached_routing.py; question3/results/round18_holdout_main/code_snapshot/question3/cached_routing.py;

question3/results/round18_holdout_rounding/code_snapshot/question3/cache
d_routing.py; question3/results/round18_outer_main/code_snapshot/question3
/cached_routing.py; question3/results/round18_outer_rounding/code_snapshot
/question3/cached_routing.py; question3/results/round19_adverse_main/code_
snapshot/question3/cached_routing.py; question3/results/round19_adverse_ro
unding/code_snapshot/question3/cached_routing.py; question3/results/round1
9_dev_main/code_snapshot/question3/cached_routing.py; question3/results/
round19_dev_rounding/code_snapshot/question3/cached_routing.py; question
3/results/round19_holdout_main/code_snapshot/question3/cached_routing.py;
question3/results/round19_holdout_rounding/code_snapshot/question3/cache
d_routing.py; question3/results/round19_outer_main/code_snapshot/question3
/cached_routing.py; question3/results/round19_outer_rounding/code_snapshot
/question3/cached_routing.py

```

1 """Insertion cost of fully localized cached jobs; no source-location samples.
2
3 The full fixed optical path bounds every successful prefix followed by the anchor
4 by the triangle inequality. This is a single-job bound, not a global route bound.
5 """
6 import math
7
8
9 def insertion_cost(current,points,anchor):
10     distance=math.dist(current,points[0])
11     distance+=sum(math.dist(a,b) for a,b in zip(points,points[1:]))
12     distance+=math.dist(points[-1],anchor)-math.dist(current,anchor)
13     return distance/5+3*len(points)+2
14
15
16 def reorder_cached(base,current,cover,remaining,jobs,cached):
17     task,target,info=base
18     if info.get('forced') or task!='localize' or target[1] not in cached or not remaining:
19         return base
20     anchor_id=min(remaining,key=lambda k:(math.dist(current,cover[k]),k))
21     anchor=cover[anchor_id]
22     costs={j[1]:insertion_cost(current,cached[j[1]][0],anchor) for j in jobs if j[1] in cached}
23     original=target[1];chosen=min(costs,key=lambda c:(costs[c],c))
24     # Keep the previous decision for numerical ties rather than introduce tie drift.
25     if costs[original]<=costs[chosen]+1e-8:chosen=original
26     target=next(j for j in jobs if j[1]==chosen)
27     return task,target,dict(info,cache_routing='station_insertion',anchor_station_id=anchor_id,
28                             insertion_costs_s=costs,original_channel=original,
29                             cache_order_changed=chosen!=original)

```

程序 66

对应文件: question3/cases.py; question3/innovation/frozen/question3/ca
ses.py; question3/results/dispatch_20260913/development/code_snapshot/
question3/cases.py; question3/results/dispatch_20260913/smoke/code_sna
pshot/question3/cases.py; question3/results/round13_adverse_main/code_
snapshot/question3/cases.py; question3/results/round13_adverse_roundi
ng/code_snapshot/question3/cases.py; question3/results/round13_dev_ma
in/code_snapshot/question3/cases.py; question3/results/round13_dev_ro
unding/code_snapshot/question3/cases.py; question3/results/round13_ho
ldout_main/code_snapshot/question3/cases.py; question3/results/round1
3_holdout_rounding/code_snapshot/question3/cases.py; question3/result

s/round13_outer_main/code_snapshot/question3/cases.py; question3/results/round13_outer_rounding/code_snapshot/question3/cases.py; question3/results/round14_adverse_main/code_snapshot/question3/cases.py; question3/results/round14_adverse_rounding/code_snapshot/question3/cases.py; question3/results/round14_dev_main/code_snapshot/question3/cases.py; question3/results/round14_dev_rounding/code_snapshot/question3/cases.py; question3/results/round14_holdout_main/code_snapshot/question3/cases.py; question3/results/round14_holdout_rounding/code_snapshot/question3/cases.py; question3/results/round14_initial_dev_main/code_snapshot/question3/cases.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/cases.py; question3/results/round14_outer_main/code_snapshot/question3/cases.py; question3/results/round14_outer_rounding/code_snapshot/question3/cases.py; question3/results/round17_adverse_main/code_snapshot/question3/cases.py; question3/results/round17_adverse_rounding/code_snapshot/question3/cases.py; question3/results/round17_dev_main/code_snapshot/question3/cases.py; question3/results/round17_dev_rounding/code_snapshot/question3/cases.py; question3/results/round18_adverse_main/code_snapshot/question3/cases.py; question3/results/round18_adverse_rounding/code_snapshot/question3/cases.py; question3/results/round18_dev_main/code_snapshot/question3/cases.py; question3/results/round18_dev_rounding/code_snapshot/question3/cases.py; question3/results/round18_holdout_main/code_snapshot/question3/cases.py; question3/results/round18_holdout_rounding/code_snapshot/question3/cases.py; question3/results/round18_outer_main/code_snapshot/question3/cases.py; question3/results/round18_outer_rounding/code_snapshot/question3/cases.py; question3/results/round19_adverse_main/code_snapshot/question3/cases.py; question3/results/round19_adverse_rounding/code_snapshot/question3/cases.py; question3/results/round19_dev_main/code_snapshot/question3/cases.py; question3/results/round19_dev_rounding/code_snapshot/question3/cases.py; question3/results/round19_holdout_main/code_snapshot/question3/cases.py; question3/results/round19_holdout_rounding/code_snapshot/question3/cases.py; question3/results/round19_outer_main/code_snapshot/question3/cases.py; question3/results/round19_outer_rounding/code_snapshot/question3/cases.py; question3/results/round1_main/code_snapshot/question3/cases.py; question3/results/round1_rounding/code_snapshot/question3/cases.py

```

1 """Self-case fixtures; never imported by strategy. Fixed development seeds only."""
2 import math
3 import random
4
5 def random_case(seed,error_mode='spatial_hash'):
6     rng=random.Random(seed);n=rng.randint(10,16);channels=rng.sample(range(1,21),n)
7     sources=[]
8     for c in channels:
9         r=1800*math.sqrt(rng.random());a=rng.uniform(0,2*math.pi)
10        sources.append(dict(channel=c,x=r*math.cos(a),y=r*math.sin(a),radius=rng.uniform(1000,1500)))
11    return dict(id=f'dev_{seed}_{error_mode}',seed=seed,error_mode=error_mode,sources=sources)
12
13 def boundary_case():
14     # 16 sources, minimum receiver radius, circle edge + station-sector bisectors.
15     return dict(id='boundary_minR_16',seed=77,error_mode='positive',sources=[

```



```

16         dict(channel=c,x=1800*math.cos((c-.5)*2*math.pi/16),y=1800*math.sin((c-.5)*2*math.pi/16),radius=1000.)
17     for c in range(1,17)]])
18
19 def late_source_case():
20     # First ten are easy to find at origin; 11th, channel20, is found at the last ring station.
21     sources=[dict(channel=c,x=20*c,y=0.,radius=1000.) for c in range(1,11)]
22     a=5*math.pi/3
23     sources.append(dict(channel=20,x=1800*math.cos(a),y=1800*math.sin(a),radius=1000.))
24     return dict(id='late_channel20_after_ten',seed=9,error_mode='negative',sources=sources)
25
26 def near_case():
27     case=random_case(888,'smooth');case['id']='near_at_origin';case['sources'][0].update(x=0.,y=0.,channel=case['sourc
↵ ces'][0]['channel'])
28     return case
29
30 def clustered_case():
31     sources=[dict(channel=c,x=1500.+c*.1,y=-.1*c,radius=1500.) for c in range(1,17)]
32     return dict(id='clustered_16',seed=12,error_mode='smooth',sources=sources)
33
34 def development_cases():
35     yield from [random_case(seed,mode) for mode in ['spatial_hash','smooth','positive','negative'] for seed in
↵ range(12)]
36     yield from [boundary_case(),late_source_case(),near_case(),clustered_case()]

```

程序 67

对应文件: question3/clear_skip.py; question3/innovation/frozen/question3/clear_skip.py; question3/results/dispatch_20260913/development/code_snapshot/question3/clear_skip.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/clear_skip.py; question3/results/round13_adverse_main/code_snapshot/question3/clear_skip.py; question3/results/round13_adverse_rounding/code_snapshot/question3/clear_skip.py; question3/results/round13_dev_main/code_snapshot/question3/clear_skip.py; question3/results/round13_dev_rounding/code_snapshot/question3/clear_skip.py; question3/results/round13_holdout_main/code_snapshot/question3/clear_skip.py; question3/results/round13_holdout_rounding/code_snapshot/question3/clear_skip.py; question3/results/round13_outer_main/code_snapshot/question3/clear_skip.py; question3/results/round13_outer_rounding/code_snapshot/question3/clear_skip.py; question3/results/round14_adverse_main/code_snapshot/question3/clear_skip.py; question3/results/round14_adverse_rounding/code_snapshot/question3/clear_skip.py; question3/results/round14_dev_main/code_snapshot/question3/clear_skip.py; question3/results/round14_dev_rounding/code_snapshot/question3/clear_skip.py; question3/results/round14_holdout_main/code_snapshot/question3/clear_skip.py; question3/results/round14_holdout_rounding/code_snapshot/question3/clear_skip.py; question3/results/round14_initial_dev_main/code_snapshot/question3/clear_skip.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/clear_skip.py; question3/results/round14_outer_main/code_snapshot/question3/clear_skip.py; question3/results/round14_outer_rounding/code_snapshot/question3/clear_skip.py; question3/results/round17_adverse_main/code_snapshot/question3/clear_skip.py; question3/results/round17_adverse_rounding/code_snapshot/question3/clear_skip.py; question3/results/round17_dev_main/code_snapshot/question3/clear_skip.py;

question3/results/round17_dev_rounding/code_snapshot/question3/clear_skip.py; question3/results/round18_adverse_main/code_snapshot/question3/clear_skip.py; question3/results/round18_adverse_rounding/code_snapshot/question3/clear_skip.py; question3/results/round18_dev_main/code_snapshot/question3/clear_skip.py; question3/results/round18_dev_rounding/code_snapshot/question3/clear_skip.py; question3/results/round18_holdout_main/code_snapshot/question3/clear_skip.py; question3/results/round18_holdout_rounding/code_snapshot/question3/clear_skip.py; question3/results/round18_outer_main/code_snapshot/question3/clear_skip.py; question3/results/round18_outer_rounding/code_snapshot/question3/clear_skip.py; question3/results/round19_adverse_main/code_snapshot/question3/clear_skip.py; question3/results/round19_adverse_rounding/code_snapshot/question3/clear_skip.py; question3/results/round19_dev_main/code_snapshot/question3/clear_skip.py; question3/results/round19_dev_rounding/code_snapshot/question3/clear_skip.py; question3/results/round19_holdout_main/code_snapshot/question3/clear_skip.py; question3/results/round19_holdout_rounding/code_snapshot/question3/clear_skip.py; question3/results/round19_outer_main/code_snapshot/question3/clear_skip.py; question3/results/round19_outer_rounding/code_snapshot/question3/clear_skip.py

```

1 """Conservative separating-halfplane proof that a radius-20 clear must fail.
2
3 If all feasible sources satisfy  $a \cdot G \leq b$  and  $(a \cdot p - b) / \|a\| > 20$ ,
4 the entire closed clear disk is outside that halfplane. Failure is then certain.
5 This is sufficient only: an inconclusive test keeps the original action.
6 """
7 import math
8 from itertools import combinations
9 import numpy as np
10 from question1.solve import bearing_halfplanes
11
12
13 def constraints(first, bearing, second, reading, error_deg):
14     return bearing_halfplanes([
15         dict(x=first[0], y=first[1], bearing_deg=bearing),
16         dict(x=second[0], y=second[1], bearing_deg=reading)], error_deg)
17
18
19 def failure_certificate(point, planes, combine=False):
20     if planes is None: return None
21     A, b = planes; p = np.asarray(point, dtype=float)
22     if not np.isfinite(p).all(): return None
23     norm = np.linalg.norm(A, axis=1)
24     gaps = (A @ p - b) / norm
25     # Outward slack, never a tolerance allowing uncertain overlap to be removed.
26     slack = 1e-5 + 64 * np.finfo(float).eps * (np.linalg.norm(p) + np.max(np.abs(b)) + 1)
27     k = int(np.argmax(gaps))
28     if math.isfinite(float(gaps[k])) and gaps[k] > 20 + slack:
29         return dict(halfplane_index=k, separation_m=float(gaps[k]), slack_m=float(slack))
30     if not combine: return None
31     # A pair's intersection proposes a normal. Only the final nonnegative
32     # combination of original inequalities is trusted as the separating proof.
33     for i, j in combinations(range(len(A)), 2):
34         matrix = A[[i, j]]
35         if abs(np.linalg.det(matrix)) < 1e-10: continue
36         vertex = np.linalg.solve(matrix, b[[i, j]])
37         weights = np.linalg.solve(matrix.T, p - vertex)
38         if not np.isfinite(weights).all() or np.any(weights < 0) or weights.sum() <= 0: continue
39         weights = weights / weights.sum()
40         normal = weights @ matrix; length = float(np.linalg.norm(normal))

```

```

41 if length<1e-8:continue
42 bound=float(weights@b[[i,j]])
43 separation=float((normal@p-bound)/length)
44 # Near cancellation amplifies arithmetic error; inflate slack accordingly.
45 margin=slack/length
46 if not math.isfinite(separation) or separation<=20+margin:continue
47 witness=np.zeros(len(A));witness[[i,j]]=weights
48 return dict(halfplane_weights=witness.tolist(),separation_m=separation,slack_m=margin)
49 return None
50
51
52 def radius_failure_certificate(point,first,second,mode):
53     """Published containing disks only; never infer the source's actual radius."""
54     circles=[]
55     if mode in ('domain','both'):circles.append(('domain',(0.,0.),1800.))
56     if mode in ('received','both'):
57         circles.extend([('first_received',first,1500.),('second_received',second,1500.)])
58     for name,centre,radius in circles:
59         distance=math.dist(point,centre)
60         slack=1e-5+64*np.finfo(float).eps*(math.hypot(*point)+math.hypot(*centre)+radius+1)
61         if math.isfinite(distance) and distance>radius+20+slack:
62             return dict(radius_prior=name,centre=list(centre),containing_radius_m=radius,
63                           separation_m=distance-radius,slack_m=slack)
64     return None

```

对应文件：question3/current_point.py; question3/innovation/frozen/question3/current_point.py; question3/results/dispatch_20260913/development/code_snapshot/question3/current_point.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/current_point.py; question3/results/round13_adverse_main/code_snapshot/question3/current_point.py; question3/results/round13_adverse_rounding/code_snapshot/question3/current_point.py; question3/results/round13_dev_main/code_snapshot/question3/current_point.py; question3/results/round13_dev_rounding/code_snapshot/question3/current_point.py; question3/results/round13_holdout_main/code_snapshot/question3/current_point.py; question3/results/round13_holdout_rounding/code_snapshot/question3/current_point.py; question3/results/round13_outer_main/code_snapshot/question3/current_point.py; question3/results/round13_outer_rounding/code_snapshot/question3/current_point.py; question3/results/round14_adverse_main/code_snapshot/question3/current_point.py; question3/results/round14_adverse_rounding/code_snapshot/question3/current_point.py; question3/results/round14_dev_main/code_snapshot/question3/current_point.py; question3/results/round14_dev_rounding/code_snapshot/question3/current_point.py; question3/results/round14_holdout_main/code_snapshot/question3/current_point.py; question3/results/round14_holdout_rounding/code_snapshot/question3/current_point.py; question3/results/round14_initial_dev_main/code_snapshot/question3/current_point.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/current_point.py; question3/results/round14_outer_main/code_snapshot/question3/current_point.py; question3/results/round14_outer_rounding/code_snapshot/question3/current_point.py; question3/results/round17_adverse_main/code_snapshot/question3/current_point.py; question3/results/round17_adverse_rounding/code_snapshot/question3/current_point.py; question3/results/round17 dev main/code snapshot/question3/current_point.py

stion3/current_point.py; question3/results/round17_dev_rounding/code_snapshot/question3/current_point.py; question3/results/round18_adverse_main/code_snapshot/question3/current_point.py; question3/results/round18_adverse_rounding/code_snapshot/question3/current_point.py; question3/results/round18_dev_main/code_snapshot/question3/current_point.py; question3/results/round18_dev_rounding/code_snapshot/question3/current_point.py; question3/results/round18_holdout_main/code_snapshot/question3/current_point.py; question3/results/round18_holdout_rounding/code_snapshot/question3/current_point.py; question3/results/round18_outer_main/code_snapshot/question3/current_point.py; question3/results/round18_outer_rounding/code_snapshot/question3/current_point.py; question3/results/round19_adverse_main/code_snapshot/question3/current_point.py; question3/results/round19_adverse_rounding/code_snapshot/question3/current_point.py; question3/results/round19_dev_main/code_snapshot/question3/current_point.py; question3/results/round19_dev_rounding/code_snapshot/question3/current_point.py; question3/results/round19_holdout_main/code_snapshot/question3/current_point.py; question3/results/round19_holdout_rounding/code_snapshot/question3/current_point.py; question3/results/round19_outer_main/code_snapshot/question3/current_point.py; question3/results/round19_outer_rounding/code_snapshot/question3/current_point.py

```

1 """Current-position opportunity: analytic eligibility, unchanged A cost proxy.
2 No source or error-field truth; the sampled diameter is a score, not a certificate.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np
7 from question2.strategy import candidate,diameters
8
9
10 def local_offset(first,bearing,current):
11     angle=math.radians(bearing);dx=current[0]-first[0];dy=current[1]-first[1]
12     return (dx*math.cos(angle)+dy*math.sin(angle),-dx*math.sin(angle)+dy*math.cos(angle))
13
14
15 @lru_cache(maxsize=2048)
16 def current_proxy(offset,error_deg,min_angle_deg):
17     if not candidate(offset,min_angle_deg,error_deg):return None
18     delta=math.radians(error_deg)
19     r,a,e=np.meshgrid(np.linspace(5.001,1500,31),np.linspace(-delta,delta,3),
20                       np.linspace(-delta,delta,3),indexing='ij')
21     g=np.column_stack([(r*np.cos(a)).ravel(),(r*np.sin(a)).ravel()])
22     diameter=float(diameters(offset,g,e.ravel(),error_deg).max())
23     x,y=offset;h=x*math.cos(delta)-abs(y)*math.sin(delta)
24     max_range=math.sqrt(max(x*x+y*y+25-10*h,x*x+y*y+1500**2-3000*h))
25     cells=math.ceil(diameter/28)*math.ceil(2*1500*math.sin(delta)/28)
26     continuation=max_range/5+8.6*cells
27     return dict(offset=offset,sampled_worst_diameter=diameter,max_source_distance=max_range,
28               proxy_cells=cells,continuation_proxy_s=continuation)

```

程序 69

对应文件: question3/dispatch_refinement/__init__.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/__init__.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/__init__.py

```
1 """Isolated dispatch candidates; frozen official candidate stays unchanged."""
```

程序 70

对应文件: question3/dispatch_refinement/analyze.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/analyze.py

```
1 """Describe every paired result and apply the predeclared replacement criteria."""
2 import argparse
3 import csv
4 import json
5 from collections import defaultdict
6 import numpy as np
7 from .experiment import OUT, ROUNDINGS, aggregate, dump
8
9
10 def paired_summary(rows, name, rounding):
11     block = [r for r in rows if r['rounding'] == rounding]
12     current = {r['case_id']: r for r in block if r['variant'] == 'CURRENT'}
13     pairs = []
14     for row in block:
15         if row['variant'] != name:
16             continue
17         base = current[row['case_id']]
18         a, b = base['penalized_virtual_s'], row['penalized_virtual_s']
19         pairs.append(dict(case_id=row['case_id'], seed=row['seed'], family=row['family'],
20                          source_count=row['source_count'], error_mode=row['error_mode'], rounding=rounding,
21                          variant=name, current_s=a, candidate_s=b, change_s=b-a, change_pct=100*(b/a-1),
22                          current_success=base['success'], candidate_success=row['success']))
23     by_layout = defaultdict(list)
24     for p in pairs:
25         by_layout[p['seed']].append((p['current_s'], p['candidate_s']))
26     means = np.array([np.mean(values, axis=0) for _, values in sorted(by_layout.items())])
27     rng = np.random.default_rng(20260913)
28     indices = rng.integers(len(means), size=(4000, len(means)))
29     draws = means[indices].mean(axis=1)
30     savings = 100 * (1 - draws[:, 1] / draws[:, 0])
31     return dict(faster=sum(p['change_s'] < -1e-6 for p in pairs),
32                slower=sum(p['change_s'] > 1e-6 for p in pairs),
33                worst_relative=max(pairs, key=lambda p: p['change_pct']),
34                saving_ci95_pct=np.percentile(savings, [2.5, 97.5]).tolist(),
35                independent_layouts=len(means)), pairs
36
37
38 def main():
39     parser = argparse.ArgumentParser(description=__doc__)
40     parser.add_argument('--stage', choices=('development', 'validation'), required=True)
41     args = parser.parse_args()
42     folder = OUT / args.stage
43     data = json.loads((folder / 'summary.json').read_text())
44     rows, stats = data['rows'], data['aggregates']
45     selected = json.loads((OUT / 'selection_lock.json').read_text())['selected']
46     report = dict(stage=args.stage, selected=selected, aggregates=stats, paired={}, breakdown={},
47                  audited_actions=sum(r.get('audited_actions', 0) for r in rows),
48                  failures=sum(not r['success'] for r in rows), official_runs=0, formal_runs=0)
49     pairs = []
50     for name in sorted({r['variant'] for r in rows} - {'CURRENT'}):
51         report['paired'][name] = {}
52         for rounding in ROUNDINGS:
53             summary, part = paired_summary(rows, name, rounding)
54             report['paired'][name][rounding] = summary
55             pairs.extend(part)
56     for key in ('family', 'source_count', 'error_mode'):
57         report['breakdown'][key] = {str(value): aggregate([r for r in rows if r[key] == value])
58                                     for value in sorted({r[key] for r in rows})}
59     report['replacement_passed'] = args.stage == 'validation' and selected is not None and all(
60         stats[r][selected]['failures'] == 0 and stats[r]['CURRENT']['failures'] == 0
61         and stats[r][selected]['saving_pct'] >= 5
62         and stats[r][selected]['p95'] <= stats[r]['CURRENT']['p95']
```

```

63         and stats[r][selected]['maximum'] <= stats[r]['CURRENT']['maximum']
64         and stats[r][selected]['mean_per_source'] < stats[r]['CURRENT']['mean_per_source']
65     for r in ROUNDINGS)
66     dump(folder / 'analysis.json', report)
67     with (folder / 'paired.csv').open('w', newline='', encoding='utf-8') as stream:
68         writer = csv.DictWriter(stream, fieldnames=list(pairs[0]))
69         writer.writeheader()
70         writer.writerows(pairs)
71     print(json.dumps(dict(stage=args.stage, selected=selected,
72         replacement_passed=report['replacement_passed'], failures=report['failures'],
73         audited_actions=report['audited_actions'], aggregates=stats), ensure_ascii=False, indent=2))
74
75
76 if __name__ == '__main__':
77     main()

```

程序 71

对应文件: question3/dispatch_refinement/discovery.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/discovery.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/discovery.py

```

1  """Public-information discovery priorities; never supplies a stopping certificate.
2
3  A fixed 100 m area grid estimates each unknown channel's unexcluded region.
4  Uniform area weights are a scheduling heuristic, not a source distribution claim.
5  All receive, clear and stop decisions remain in the unchanged policy controller.
6  """
7  from functools import lru_cache
8  import math
9
10 import numpy as np
11
12 from question3.innovation.geometry import bbox
13 from question3.innovation.routing import choose as choose_tour
14
15
16 @lru_cache(maxsize=8)
17 def _station_masks(stations):
18     axis = np.arange(-1750.0, 1800.0, 100.0)
19     xx, yy = np.meshgrid(axis, axis)
20     points = np.column_stack((xx.ravel(), yy.ravel()))
21     points = points[np.linalg.norm(points, axis=1) <= 1800.0]
22     masks = np.linalg.norm(points[None, :, :] - np.asarray(stations)[None, :, :], axis=2) <= 1000.0
23     masks.setflags(write=False)
24     return masks
25
26
27 def _coverage_gains(scan_ids, context):
28     known = set(context['first']) | set(context['cleared'])
29     unknown = sorted(set(range(1, 21)) - known)
30     stations = tuple(tuple(map(float, p)) for p in context['stations'])
31     masks = _station_masks(stations)
32     sums = {station: 0.0 for station in scan_ids}
33     unresolved = 0
34     for channel in unknown:
35         excluded_stations = sorted(context['negative'].get(channel, set()))
36         remaining = ~masks[excluded_stations].any(axis=0) if excluded_stations else np.ones(masks.shape[1],
37             dtype=bool)
38         count = int(remaining.sum())
39         if not count:
40             # A grid can miss a small real region: do not infer channel absence.
41             unresolved += 1
42             continue
43         for station in scan_ids:
44             sums[station] += float((masks[station] & remaining).sum()) / count
45     # Public upper count supplies an optimistic discovery-value scale, not a
46     # claimed estimate of how many sources actually remain.

```

```

46     remaining_upper = max(0, 16 - len(known))
47     scale = remaining_upper / len(unknown) if unknown else 0.0
48     return {station: value * scale for station, value in sums.items()}, len(unknown), unresolved
49
50
51 def choose(current, tasks, context):
52     """Return an existing (kind, id, point) task and JSON-compatible diagnostics.
53
54     coverage_order: preserve the original tour's search/service choice; only
55         reorder a selected search task using discovery value per estimated second.
56     coverage_priority: compare that discovery value with one known-source service
57         per estimated second. The service estimate uses its public region radius
58         as an additional travel scale and one measurement; it is not a bound.
59     """
60     if not tasks:
61         raise ValueError('Discovery dispatch requires at least one task')
62     variant = context['config'].get('dispatch_variant', 'coverage_order')
63     if variant not in ('coverage_order', 'coverage_priority'):
64         raise ValueError('Unsupported discovery variant: ' + str(variant))
65     original, route = choose_tour(current, tasks, True)
66     details = dict(variant=variant, original_task=[original[0], int(original[1])],
67                   original_route=route, grid_spacing_m=100.0,
68                   value_model='uniform unexcluded area, scaled by remaining source-count upper bound',
69                   scores=[])
70     scans = [task for task in tasks if task[0] == 'scan']
71     if not scans or (variant == 'coverage_order' and original[0] != 'scan'):
72         return original, details
73     gains, unknown_count, unresolved = _coverage_gains([task[1] for task in scans], context)
74     details.update(unknown_channels=unknown_count, unresolved_grid_channels=unresolved)
75     ranked = []
76     for task in tasks:
77         kind, ident, point = task
78         travel = math.dist(current, point) / 5.0
79         if kind == 'scan':
80             # All still unknown channels may need a five-second measurement and
81             # a one-second channel switch. Count-based early scan exit is ignored.
82             cost = travel + 6.0 * unknown_count
83             value = gains[ident]
84         elif kind == 'localize' and variant == 'coverage_priority':
85             radius = bbox(context['polys'][ident], context['bearings'][ident])[1]
86             cost = travel + 5.0
87             if radius > 20.0:
88                 cost += radius / 5.0 + 6.0
89             value = 1.0
90         else:
91             continue
92         score = value / max(cost, 1e-9)
93         details['scores'].append(dict(task=kind, target=int(ident), value=value,
94                                     estimated_seconds=cost, score=score))
95         ranked.append((-score, cost, kind, int(ident), task))
96     if not ranked or max(-row[0] for row in ranked) <= 0:
97         # An unresolved grid or zero proxy never removes any required task.
98         return original, details
99     return min(ranked, key=lambda row: row[:4])[-1], details

```

程序 72

对应文件: question3/dispatch_refinement/experiment.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/experiment.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/experiment.py

```

1 """Paired self experiments. Hidden cases are only used by the evaluator and audit."""
2 import argparse
3 from concurrent.futures import ProcessPoolExecutor, as_completed
4 from datetime import datetime, timezone
5 import hashlib
6 import json
7 from pathlib import Path

```

```

8 import shutil
9 import subprocess
10 import time
11
12 import numpy as np
13 from question3.audit import audit_folder
14 from question3.interface import Client, JsonlLog
15 from question3.run_innovation_http import PublicClient
16 from question3.simulator import Simulator
17 from question3.innovation.strategy import run as current_run
18 from .strategy import run as candidate_run
19 from .fixtures import generate
20
21 ROOT = Path(__file__).resolve().parents[2]
22 HERE = Path(__file__).resolve().parent
23 OUT = ROOT / 'question3/results/dispatch_20260913'
24 VARIANTS = ('CURRENT', 'COVERAGE_ORDER', 'COVERAGE_PRIORITY', 'SERVICE_PAIR', 'SERVICE_TOUR')
25 ROUNDINGS = ('bounded', 'pre_round_stress')
26
27
28 def dump(path, data):
29     path.parent.mkdir(parents=True, exist_ok=True)
30     path.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')
31
32
33 def hashes():
34     paths = []
35     for directory in ('question1', 'question2', 'question3', 'question3/innovation', 'question3/dispatch_refinement'):
36         paths += sorted((ROOT / directory).glob('*.py'))
37     paths += [ROOT / 'question3/innovation/best_candidate.json', HERE / 'PROTOCOL.md']
38     return {str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest() for p in paths}
39
40
41 def audit_geometry(folder):
42     case = json.loads((folder / 'evaluator_hidden_case.json').read_text())
43     truth = {s['channel']: np.array([s['x'], s['y']]) for s in case['sources']}
44     counts = {}
45     for line in (folder / 'strategy.jsonl').read_text().splitlines():
46         event = json.loads(line)
47         kind = event['type']
48         counts[kind] = counts.get(kind, 0) + 1
49         if kind not in ('feasible_update', 'certified_measure', 'certified_clear'):
50             continue
51         vertices, source = np.asarray(event['vertices']), truth[event['channel']]
52         assert len(vertices) and np.isfinite(vertices).all()
53         edges = np.roll(vertices, -1, axis=0) - vertices
54         delta = source - vertices
55         cross = edges[:, 0] * delta[:, 1] - edges[:, 1] * delta[:, 0]
56         assert np.all(cross >= -1e-4) or np.all(cross <= 1e-4), 'True source excluded'
57         if kind in ('certified_measure', 'certified_clear'):
58             radius = event['radius']
59             assert np.linalg.norm(vertices - event['point'], axis=1).max() <= radius + 1e-8
60             assert radius < (1000 if kind == 'certified_measure' else 20)
61     return counts
62
63
64 def one_run(job):
65     case, variant, config, rounding, stage = job
66     folder = OUT / stage / rounding / variant / case['id']
67     folder.mkdir(parents=True, exist_ok=False)
68     dump(folder / 'evaluator_hidden_case.json', case)
69     actions, events = JsonlLog(folder / 'actions.jsonl'), JsonlLog(folder / 'strategy.jsonl')
70     env = Simulator(case, rounding=rounding)
71     client = Client(env, 'SELF', actions, config['baseline_config']['real_time_reserve_s'])
72     start = time.perf_counter()
73     try:
74         result = (current_run if variant == 'CURRENT' else candidate_run)(PublicClient(client), config, events)
75     except Exception as exc:
76         result = dict(status='incomplete', failure=f'{type(exc).__name__}: {exc}',
77                       metrics=client.metrics(), exit_confirmed=False)
78     finally:
79         actions.close()
80         events.close()

```



```

81     elapsed = time.perf_counter() - start
82     truth = env.evaluation()
83     success = result['status'] == 'complete' and not truth['remaining_channels']
84     row = dict(case_id=case['id'], seed=case['seed'], family=case['family'],
85               source_count=truth['total'], error_mode=case['error_mode'],
86               variant=variant, rounding=rounding, stage=stage,
87               status=result['status'], failure=result.get('failure'),
88               success=success, false_complete=result['status'] == 'complete' and not success,
89               cleared=truth['cleared'], total=truth['total'],
90               elapsed_wall_s=elapsed, metrics=result['metrics'])
91     dump(folder / 'result.json', dict(summary=row, strategy=result))
92     try:
93         if success:
94             row['audited_actions'] = audit_folder(folder)
95             row['event_counts'] = audit_geometry(folder)
96             row['audit'] = 'pass'
97         else:
98             row['audit'] = 'incomplete retained'
99     except Exception as exc:
100         row.update(success=False, false_complete=True, audit=f'{type(exc).__name__}: {exc}')
101     row['penalized_virtual_s'] = truth['virtual_time_s'] if row['success'] else max(360000., truth['virtual_time_s'])
102     dump(folder / 'result.json', dict(summary=row, strategy=result))
103     return row
104
105
106 def aggregate(rows):
107     result = {}
108     for rounding in ROUNDINGS:
109         groups = {}
110         for name in sorted({r['variant'] for r in rows}):
111             block = [r for r in rows if r['variant'] == name and r['rounding'] == rounding]
112             if not block:
113                 continue
114             values = [r['penalized_virtual_s'] for r in block]
115             groups[name] = dict(runs=len(block), complete=sum(r['success'] for r in block),
116                               failures=sum(not r['success'] for r in block),
117                               false_complete=sum(r['false_complete'] for r in block),
118                               mean=float(np.mean(values)), p95=float(np.percentile(values, 95)),
119                               maximum=max(values),
120                               mean_per_source=float(np.mean([r['penalized_virtual_s'] / r['total'] for r in
121                               ↪ block])),
122                               wall_p95=float(np.percentile([r['elapsed_wall_s'] for r in block], 95)))
123         if 'CURRENT' in groups:
124             for value in groups.values():
125                 value['saving_pct'] = 100 * (1 - value['mean'] / groups['CURRENT']['mean'])
126         result[rounding] = groups
127     return result
128
129 def select(stats):
130     eligible = []
131     for name in VARIANTS[1:]:
132         if all(name in stats[rounding] and stats[rounding][name]['failures'] == 0
133               and stats[rounding][name]['mean'] < stats[rounding]['CURRENT']['mean']
134               and stats[rounding][name]['p95'] <= stats[rounding]['CURRENT']['p95']
135               and stats[rounding][name]['maximum'] <= stats[rounding]['CURRENT']['maximum']
136               for rounding in ROUNDINGS):
137             eligible.append((sum(stats[r][name]['mean'] / stats[r]['CURRENT']['mean'] for r in ROUNDINGS), name))
138     return min(eligible)[1] if eligible else None
139
140
141 def main():
142     parser = argparse.ArgumentParser(description=__doc__)
143     parser.add_argument('--stage', choices=('smoke', 'development', 'validation'), required=True)
144     parser.add_argument('--workers', type=int, default=4)
145     args = parser.parse_args()
146     out = OUT / args.stage
147     out.mkdir(parents=True, exist_ok=False)
148     config = json.loads((ROOT / 'question3/innovation/best_candidate.json').read_text())
149     configs = {name: dict(config, **({ if name == 'CURRENT' else {'dispatch_variant': name.lower()}})) for name in
150     ↪ VARIANTS}
151     if args.stage == 'validation':
152         lock = json.loads((OUT / 'selection_lock.json').read_text())

```

```

152     assert lock['source_sha256'] == hashes(), 'Source changed after development selection'
153     winner = lock['selected']
154     assert winner in VARIANTS[1:]
155     assert configs == lock['configs']
156     names = ('CURRENT', winner)
157     cases = generate(2026092300, 2, 'dispatch_validation')
158 else:
159     names = VARIANTS
160     cases = generate(2026091300 if args.stage == 'development' else 2026091200, 1, 'dispatch_' + args.stage)
161     if args.stage == 'smoke':
162         cases = [c for c in cases if c['seed'] == 2026091206 and c['error_mode'] == 'spatial_hash']
163 dump(out / 'fixtures.json', cases)
164 dump(out / 'configs.json', {name: configs[name] for name in names})
165 before = hashes()
166 for file in before:
167     dest = out / 'code_snapshot' / file
168     dest.parent.mkdir(parents=True, exist_ok=True)
169     shutil.copy2(ROOT / file, dest)
170 dump(out / 'metadata.json', dict(stage=args.stage, generated_at=datetime.now(timezone.utc).isoformat(),
171 git_head=subprocess.check_output(['git', 'rev-parse', 'HEAD'], cwd=ROOT, text=True).strip(),
172 independent_layouts=len([c['seed'] for c in cases]), source_sha256=before,
173 fixtures_sha256=hashlib.sha256((out / 'fixtures.json').read_bytes()).hexdigest(),
174 official_runs=0, formal_runs=0))
175 jobs = []
176 for i, case in enumerate(cases):
177     order = names[i % len(names):] + names[:i % len(names)]
178     for rounding in ROUNDINGS:
179         jobs.extend((case, name, configs[name], rounding, args.stage) for name in order)
180 rows = []
181 start = time.perf_counter()
182 with ProcessPoolExecutor(max_workers=args.workers) as pool:
183     pending = [pool.submit(one_run, job) for job in jobs]
184     for future in as_completed(pending):
185         row = future.result()
186         rows.append(row)
187         if not row['success']:
188             print('FAIL', row['variant'], row['case_id'], row['failure'], row['audit'], flush=True)
189         if len(rows) % 80 == 0 or len(rows) == len(jobs):
190             print(f'{len(rows)}/{len(jobs)} runs; elapsed={time.perf_counter()-start:.1f}s', flush=True)
191             dump(out / 'progress.json', dict(done=len(rows), total=len(jobs), failures=sum(not r['success'] for r
192 ↪ in rows)))
193 rows.sort(key=lambda r: (r['rounding'], r['case_id'], r['variant']))
194 stats = aggregate(rows)
195 assert hashes() == before, 'Source changed during experiment'
196 dump(out / 'summary.json', dict(rows=rows, aggregates=stats))
197 dump(out / 'comparison.json', stats)
198 if args.stage == 'development':
199     winner = select(stats)
200     dump(out / 'selection_lock.json', dict(selected=winner, configs=configs, source_sha256=before,
201 locked_at=datetime.now(timezone.utc).isoformat(), protocol='question3/dispatch_refinement/PROTOCOL.md'))
202     print('Selected:', winner, flush=True)
203 print(json.dumps(stats, ensure_ascii=False, indent=2), flush=True)
204
205 if __name__ == '__main__':
206     main()

```

程序 73

对应文件: question3/dispatch_refinement/fixtures.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/fixtures.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/fixtures.py

```

1 """Evaluator-only fresh layouts, crossed explicitly by family and source count."""
2 import math
3 import random
4
5 FAMILIES = ('uniform', 'outer', 'cluster', 'min_radius', 'radial', 'reception_edge')

```

```

6 MODES = ('spatial_hash', 'smooth', 'positive', 'negative')
7
8
9 def generate(start, repeats, prefix):
10     cases = []
11     index = 0
12     for repeat in range(repeats):
13         for family in FAMILIES:
14             for count in range(10, 17):
15                 seed = start + index
16                 rng = random.Random(seed)
17                 channels = rng.sample(range(1, 21), count)
18                 phase = rng.uniform(0, 2 * math.pi)
19                 cluster_r = rng.uniform(100, 1700)
20                 cx, cy = cluster_r * math.cos(phase), cluster_r * math.sin(phase)
21                 sources = []
22                 for j, channel in enumerate(channels):
23                     r = 1800 * math.sqrt(rng.random())
24                     angle = rng.uniform(0, 2 * math.pi)
25                     radius = rng.uniform(1000, 1500)
26                     if family == 'outer':
27                         r = rng.uniform(1600, 1800)
28                         if (count + repeat) % 2:
29                             angle = phase + rng.uniform(-.06, .06)
30                     elif family == 'cluster':
31                         x, y = cx + rng.uniform(-60, 60), cy + rng.uniform(-60, 60)
32                         r, angle = math.hypot(x, y), math.atan2(y, x)
33                     elif family == 'min_radius':
34                         radius = 1000.
35                     elif family == 'radial':
36                         angle, r = phase + rng.uniform(-.002, .002), rng.uniform(0, 1800)
37                         radius = 1000. if j % 2 else 1500.
38                     elif family == 'reception_edge':
39                         radius = 1000. if j % 2 else 1500.
40                         r = radius + (-1 if j % 3 else 1) * rng.uniform(.001, 1.) if j % 4 else 1800.
41                     r = min(1800., r)
42                     sources.append(dict(channel=channel, x=r * math.cos(angle), y=r * math.sin(angle), radius=radius))
43                 for mode in MODES:
44                     cases.append(dict(id=f'{prefix}_{family}_{seed}_{mode}', seed=seed,
45                                     error_mode=mode, family=family, sources=sources))
46                 index += 1
47     return cases

```

程序 74

对应文件: question3/dispatch_refinement/service.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/service.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/service.py

```

1 """Public-information dispatch with service and endpoint cost proxies.
2
3 SERVICE_PAIR plans two tasks, so it does not precommit to completing every
4 currently remaining scan. SERVICE_TOUR retains a full open tour as a control.
5 Both only choose the next existing task; the original policy still executes it.
6 The cost is a deterministic proxy, not a time bound or an expected value.
7 """
8 import math
9
10 from question3.innovation.geometry import bbox, parallax_point
11
12
13 def choose(current, tasks, context):
14     """Return an unchanged (kind, id, point) task and diagnostic details."""
15     if not tasks:
16         raise ValueError("Service dispatch requires at least one task")
17     variant = context.get("config", {}).get("dispatch_variant", "service_pair").lower()
18     if variant not in ("service_pair", "service_tour"):
19         raise ValueError(f"Unknown service dispatch variant: {variant}")

```

```

20 tasks = list(tasks)
21 known = set(context["first"]) | set(context["cleared"])
22 unknown = [c for c in range(1, 21) if c not in known]
23 models = []
24 for kind, target, point in tasks:
25     if kind == "scan":
26         models.append(dict(end=tuple(point), channels=unknown,
27                             action_s=5.0 * len(unknown), direct=tuple(point)))
28     elif kind == "localize":
29         centre, radius, *_ = bbox(context["polys"][target], context["bearings"][target])
30         clearable = radius <= 20.0 - 1e-5
31         # For an uncertain source, price one radio measurement and one
32         # optical+laser action. Further measurements are not predicted.
33         # The current centre is only a service-endpoint proxy.
34         models.append(dict(end=tuple(centre), channels=[] if clearable else [target],
35                             action_s=5.0 if clearable else 10.0,
36                             direct=tuple(centre) if clearable else None))
37     else:
38         raise ValueError(f"Unknown dispatch task kind: {kind}")
39
40 n = len(tasks)
41 # Row n represents the real current position; other rows represent the
42 # preceding task's estimated endpoint. Geometric work is done once.
43 positions = [m["end"] for m in models] + [tuple(current)]
44 travel = [[0.0] * n for _ in positions]
45 entries = [[None] * n for _ in positions]
46 for previous, position in enumerate(positions):
47     for j, model in enumerate(models):
48         point = model["direct"]
49         if point is None:
50             target = tasks[j][1]
51             point, _ = parallax_point(context["polys"][target],
52                                     context["bearings"][target], position)
53         point = tuple(float(x) for x in point)
54         entries[previous][j] = point
55         travel[previous][j] = (math.dist(position, point)
56                               + math.dist(point, model["end"])) / 5.0
57
58 def leg(previous, channel, j):
59     model = models[j]
60     channels = model["channels"]
61     switches = len(channels) - int(bool(channels) and channel == channels[0])
62     after_channel = channels[-1] if channels else channel
63     return travel[previous][j] + model["action_s"] + switches, after_channel
64
65 initial_channel = context.get("channel")
66
67 def cost(route):
68     total, previous, channel = 0.0, n, initial_channel
69     for j in route:
70         value, channel = leg(previous, channel, j)
71         total += value
72         previous = j
73     return total
74
75 candidates = []
76 for start in range(n):
77     route = [start]
78     left = set(range(n)) - {start}
79     _, channel = leg(n, initial_channel, start)
80     if variant == "service_pair":
81         if left:
82             following = min(left, key=lambda j: (leg(start, channel, j)[0], j))
83             route.append(following)
84         else:
85             while left:
86                 following = min(left, key=lambda j: (leg(route[-1], channel, j)[0], j))
87                 _, channel = leg(route[-1], channel, following)
88                 route.append(following)
89                 left.remove(following)
90             # Strictly improving two-opt, including the initial task. The
91             # directed service costs are recomputed for each reversed route.
92             while True:

```

```

93         old = cost(route)
94         improvement = None
95         for i in range(n - 1):
96             for j in range(i + 1, n):
97                 alternative = route[:i] + list(reversed(route[i:j + 1])) + route[j + 1:]
98                 value = cost(alternative)
99                 if value < old - 1e-7 and (improvement is None or
100                                         (value, alternative) < improvement):
101                     improvement = (value, alternative)
102             if improvement is None:
103                 break
104             route = improvement[1]
105             candidates.append((cost(route), route))
106         score, route = min(candidates)
107         selected = route[0]
108         immediate, _ = leg(n, initial_channel, selected)
109         return tasks[selected], dict(
110             method=variant, cost_kind="deterministic_service_proxy_not_bound",
111             score_s=score, immediate_proxy_s=immediate,
112             estimated_entry=list(entries[n][selected]),
113             estimated_endpoint=list(models[selected]["end"]),
114             unknown_channel_count=len(unknown), horizon_tasks=len(route),
115             tour=[dict(task=task[j][0], target=task[j][1], point=list(task[j][2])) for j in route],
116         )

```

程序 75

对应文件: question3/dispatch_refinement/strategy.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/strategy.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/strategy.py

```

1  """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2  import math
3  import numpy as np
4  from question3.geometry import stations, station_cover_bound
5  from question3.modules import choose_second
6  from question3.innovation.geometry import initial, wedge, bbox, enclosing, grid, parallax_point, clip, worthwhile
7  from question3.innovation.routing import choose as choose_route
8  from question3.innovation.coverage import covered, cover_radius
9  from .discovery import choose as choose_discovery
10 from .service import choose as choose_service
11
12 def run(client, config, event=lambda row: None):
13     cover=stations(1125)
14     assert station_cover_bound(1125)<1000
15     first={}; polys={}; bearings={}; measured={}; cleared=set(); visited=set()
16     negative={c:set() for c in range(1,21)}
17     negative_points={c:set() for c in range(1,21)}; absent=set()
18     failure=None; exit_confirmed=False; certificate=None
19     base=config['baseline_config']
20     mode=config.get('mechanism', 'homing')
21     continuous_cover=mode.startswith('cover_')
22     if continuous_cover: mode=mode[len('cover_'):]
23     information_gate=mode.startswith('gated_')
24     if information_gate: mode=mode[len('gated_'):]
25     route_mode='global' if mode.startswith('tour_') else ('nearest' if mode.startswith('nearest_') else None)
26     if route_mode: mode=mode.split('_', 1)[1]
27     refresh=mode in ('refresh', 'refresh_parallax', 'shared', 'shared_parallax')
28     parallax=mode in ('parallax', 'refresh_parallax', 'shared_parallax', 'multi_parallax')
29     sharing=mode in ('shared', 'shared_parallax', 'multi_parallax')
30     survey=mode=='survey'
31     def centre(c):
32         return bbox(polys[c], bearings[c])[2:]
33     def update(c, p, response):
34         kind=response['measure_result']
35         if kind=='near':
36             if client.clear(p, c, phase='near')['clear_result']!='success':
37                 raise RuntimeError('Near feedback contradicted by failed clear')

```

```

38         cleared.add(c)
39         return
40     if kind!='direction':
41         raise RuntimeError('Certified reception returned no_signal')
42     theta=response['svd_deg']
43     if c not in first:
44         first[c]=(tuple(p),theta)
45         polys[c]=initial(p,theta,base['error_deg'])
46         measured[c]=set()
47     else:
48         polys[c]=wedge(polys[c],p,theta,base['error_deg'])
49         measured[c].add(tuple(p));bearings[c]=theta
50         event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
51 def scan(k):
52     p=cover[k]
53     for c in range(1,21):
54         if not survey and len(set(first)|cleared)==16:
55             return
56         if c in cleared or c in absent or (c in first and not survey):
57             continue
58         if survey and len(set(first)|cleared)==16 and c not in first:continue
59         r=client.measure(p,c,phase='search')
60         if r['measure_result']=='no_signal':
61             negative[c].add(k);negative_points[c].add(tuple(p))
62         else:update(c,p,r)
63     visited.add(k)
64 def localize(c):
65     if mode=='reception_bracket':
66         s,theta=first[c];s=np.asarray(s)
67         t=math.radians(theta);axis=np.array([math.cos(t),math.sin(t)])
68         bounds=[]
69         for sign in (-1,1):
70             inside=0.;outside=3001.
71             while outside-inside>10:
72                 mid=(inside+outside)/2;p=s+sign*mid*axis
73                 r=client.measure(p,c,phase='reception_range')
74                 if r['measure_result']=='near':
75                     update(c,p,r);return
76                 if r['measure_result']=='no_signal':outside=mid
77                 else:inside=mid
78                 bounds.append(sorted([sign*inside,sign*outside]))
79             lower=(bounds[0][0]+bounds[1][0])/2
80             upper=(bounds[0][1]+bounds[1][1])/2
81             polys[c]=clip(polys[c],axis,float(axis@s+upper+1e-7))
82             polys[c]=clip(polys[c],-axis,float(-axis@s-lower+1e-7))
83             event(dict(type='range_bracket',channel=c,first=list(s),axis=axis.tolist(),lower=lower,upper=upper,vertic
↵ es=polys[c].tolist()))
84             for p in grid(polys[c],theta):
85                 if client.clear(p,c,phase='bracket_optical')['clear_result']=='success':
86                     cleared.add(c);return
87             raise RuntimeError('Reception bracket optical cover exhausted')
88     if mode=='strip':
89         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
90         for p in grid(polys[c],bearings[c]):
91             if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
92                 cleared.add(c);return
93         raise RuntimeError('Single-bearing finite cover exhausted')
94     if refresh and tuple(client.position) not in measured[c]:
95         radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
96         if radius<1000-1e-5:
97             event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↵ s[c].tolist()))
98             update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
99             if c in cleared:return
100     for step in range(16):
101         p,radius=centre(c)
102         if radius<=20-1e-5:
103             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↵ )
104             if client.clear(p,c,phase='optical')['clear_result']!='success':
105                 raise RuntimeError('Certified containing circle clear failed')
106             cleared.add(c);return
107     if parallax and step==0:

```

```

108         p, radius=parallax_point(polys[c], bearings[c], client.position)
109     if radius>=1000-1e-5:
110         raise RuntimeError('Adaptive measurement lacks reception certificate')
111     if tuple(p) in measured[c]:
112         break
113     event(dict(type='certified_measure', channel=c, point=p.tolist(), radius=radius, vertices=polys[c].tolist()))
114     update(c, p, client.measure(p, c, phase='adaptive_localization'))
115     if c in cleared: return
116     if sharing:
117         for other in sorted(set(first)-cleared-{c}):
118             if tuple(client.position) in measured[other]: continue
119             rr=float(np.linalg.norm(polys[other]-np.asarray(client.position), axis=1).max()+1e-7)
120             if rr>=1000-1e-5: continue
121             if information_gate and not
122                 ↪ worthwhile(polys[other], bearings[other], client.position, base['error_deg']):
123                 event(dict(type='information_gate_skip', channel=other)); continue
124                 event(dict(type='certified_measure', channel=other, point=list(client.position), radius=rr, vertices=
125                     ↪ polys[other].tolist()))
126                 update(other, client.position, client.measure(client.position, other, phase='multi_target_refinement'
127                     ↪ ))
128     event(dict(type='finite_optical_fallback', channel=c, vertices=polys[c].tolist()))
129     for p in grid(polys[c], bearings[c]):
130         if client.clear(p, c, phase='optical_fallback')['clear_result']=='success':
131             cleared.add(c); return
132     raise RuntimeError('Finite optical cover exhausted without success')
133 client.enter()
134 try:
135     while True:
136         if continuous_cover and len(set(first)|cleared)<16:
137             for c in sorted(set(range(1,21))-set(first)-cleared-absent):
138                 pts=negative_points[c]
139                 if covered(pts): absent.add(c); continue
140                 if tuple(client.position) in pts: continue
141                 augmented=pts|{tuple(client.position)}
142                 if not covered(augmented): continue
143                 event(dict(type='coverage_probe', channel=c, point=list(client.position), negative_positions=[list(p)
144                     ↪ ) for p in
145                     ↪ sorted(pts)], cover_radius=cover_radius(tuple(sorted(augmented))))
146                 r=client.measure(client.position, c, phase='coverage_completion')
147                 if r['measure_result']=='no_signal':
148                     pts.add(tuple(client.position)); absent.add(c)
149                 else: update(c, client.position, r)
150     known=set(first)|cleared
151     if len(known)>16: raise RuntimeError('Source count upper bound contradicted')
152     pending=sorted(set(first)-cleared)
153     remaining=[] if len(known)==16 and not survey else sorted(set(range(7))-visited)
154     if continuous_cover and known|absent==set(range(1,21)): remaining=[]
155     if not pending and not remaining: break
156     jobs=[]
157     for c in pending:
158         # R1 keeps the baseline's pre-schedule second-point proxy.
159         planning={**base, 'second_point_opportunity': 'off', 'second_range_prior': 'off'}
160         q,_=choose_second(*first[c], client.position, planning)
161         jobs.append((math.dist(client.position, q), 'localize', c))
162     jobs += [(math.dist(client.position, cover[k]), 'scan', k) for k in remaining]
163     if survey:
164         tasks=[('scan', k, cover[k]) for k in remaining] if remaining else
165             [('localize', c, centre(c)[0]) for c in pending]
166         (task, target, _), tour=choose_route(client.position, tasks, True)
167         event(dict(type='survey_route', route=tour))
168     elif route_mode:
169         tasks=[('localize', c, centre(c)[0]) for c in pending]
170         tasks += [('scan', k, cover[k]) for k in remaining]
171         dispatch = config.get('dispatch_variant', '').lower()
172         context = dict(polys=polys, bearings=bearings, negative=negative,
173             first=first, cleared=cleared, stations=cover, visited=visited,
174             channel=client.channel, config=config)
175         if dispatch.startswith('coverage_'):
176             (task, target, _), tour=choose_discovery(client.position, tasks, context)
177         elif dispatch.startswith('service_'):
178             (task, target, _), tour=choose_service(client.position, tasks, context)
179         else:
180             (task, target, _), tour=choose_route(client.position, tasks, route_mode=='global')

```

```

176         event(dict(type='belief_route',mode=dispatch or route_mode,route=tour))
177     else:
178         _,task,target=min(jobs)
179         event(dict(type='schedule',task=task,target=target))
180         if task=='scan':scan(target)
181         else:localize(target)
182     if len(cleared)==16:
183         certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
184             successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
185     elif continuous_cover and all(c in cleared or c in absent for c in range(1,21)):
186         certificate=dict(method='continuous_disk_union',negative_positions={str(c):[list(p) for p in
187             ↪ sorted(negative_points[c])] for c in absent})
188     elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
189         certificate=dict(method='seven_station_channel_cover')
190     else:raise RuntimeError('Incomplete stopping certificate')
191     if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
192     exit_confirmed=client.exit()['exit_reason']=='user_exit'
193 except (RuntimeError,ValueError,KeyError) as exc:
194     failure=f'{type(exc).__name__}: {exc}'
195 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
196     exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
197     absent_channels=sorted(c for c in negative if len(negative[c])==7),
198     negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
199     station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
200 event(dict(type='strategy_result',**result))
201 return result

```

程序 76

对应文件: question3/dispatch_refinement/test_discovery.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/test_discovery.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/test_discovery.py

```

1 """Small checks for the new ordering heuristic, without any simulator truth."""
2 import copy
3 import unittest
4
5 import numpy as np
6
7 from .discovery import choose
8 from question3.innovation.routing import choose as choose_tour
9
10
11 class DiscoveryTests(unittest.TestCase):
12     def context(self):
13         return dict(polys={}, bearings={}, first={}, cleared=set(),
14             stations=[(0., 0.), (1125., 0.), (0., 0.)], visited={0},
15             negative={c: {0} for c in range(1, 21)},
16             config={'dispatch_variant': 'coverage_order'})
17
18     def test_negative_record_changes_search_order_without_mutation(self):
19         context = self.context()
20         before = copy.deepcopy(context)
21         tasks = [('scan', 2, (0., 0.)), ('scan', 1, (1125., 0.))]
22         self.assertEqual(choose_tour((0., 0.), tasks, True)[0][1], 2)
23         selected, details = choose((0., 0.), tasks, context)
24         self.assertEqual(selected[1], 1)
25         scores = {row['target']: row for row in details['scores']}
26         self.assertEqual(scores[2]['value'], 0.)
27         self.assertGreater(scores[1]['value'], 0.)
28         self.assertEqual(context, before)
29
30     def test_order_variant_preserves_selected_service(self):
31         context = self.context()
32         context['first'][1] = ((0., 0.), 0.)
33         context['polys'][1] = np.array([(0., 0.), [1., 0.], [0., 1.]])
34         context['bearings'][1] = 0.
35         tasks = [('localize', 1, (0., 0.)), ('scan', 1, (1125., 0.))]

```



```

36         self.assertEqual(choose((0., 0.), tasks, context)[0][0], 'localize')
37
38     def test_count_upper_limit_gives_no_fabricated_discovery_reward(self):
39         context = self.context()
40         context['first'] = {c: ((0., 0.), 0.) for c in range(1, 17)}
41         tasks = [('scan', 2, (0., 0.)), ('scan', 1, (1125., 0.))]
42         selected, details = choose((0., 0.), tasks, context)
43         self.assertEqual(selected, choose_tour((0., 0.), tasks, True)[0])
44         self.assertTrue(all(row['value'] == 0 for row in details['scores']))
45
46
47 if __name__ == '__main__':
48     unittest.main()

```

程序 77

对应文件: question3/dispatch_refinement/test_service.py; question3/results/dispatch_20260913/development/code_snapshot/question3/dispatch_refinement/test_service.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/dispatch_refinement/test_service.py

```

1 """Small accounting and dispatch-contract checks; no simulator is launched."""
2 import unittest
3
4 import numpy as np
5
6 from .service import choose
7
8
9 class ServiceDispatchTests(unittest.TestCase):
10     def context(self, **extra):
11         result = dict(first={}, cleared=set(), polys={}, bearings={},
12                       channel=1, config={})
13         result.update(extra)
14         return result
15
16     def test_scan_charges_unknown_channels_and_real_initial_switch(self):
17         context = self.context(first={c: None for c in range(1, 16)}, channel=16)
18         _, detail = choose((0, 0), [("scan", 0, (0, 0))], context)
19         self.assertEqual(detail["immediate_proxy_s"], 29.0)
20         self.assertEqual(detail["unknown_channel_count"], 5)
21
22     def test_clear_preserves_channel_for_following_scan(self):
23         square = np.array([[-1., -1.], [1., -1.], [1., 1.], [-1., 1.]])
24         context = self.context(first={1: None}, polys={1: square}, bearings={1: 0.}, channel=2)
25         selected, detail = choose((0, 0), [("localize", 1, (0, 0)),
26                                           ("scan", 0, (0, 0))], context)
27         self.assertEqual(selected[0], "localize")
28         self.assertEqual(detail["immediate_proxy_s"], 5.0)
29         self.assertEqual(detail["score_s"], 5.0 + 95.0 + 18.0)
30
31     def test_pair_does_not_price_all_tail_stations(self):
32         tasks = [("scan", j, (float(j * 10), 0.)) for j in range(4)]
33         _, pair = choose((0., 0.), tasks, self.context())
34         _, tour = choose((0., 0.), tasks,
35                         self.context(config={"dispatch_variant": "SERVICE_TOUR"}))
36         self.assertEqual(pair["horizon_tasks"], 2)
37         self.assertEqual(tour["horizon_tasks"], 4)
38         self.assertLess(pair["score_s"], tour["score_s"])
39
40     def test_single_task_is_returned_unchanged(self):
41         task = ("scan", 2, (100., 0.))
42         selected, _ = choose((0., 0.), [task], self.context())
43         self.assertIs(selected, task)
44
45
46 if __name__ == "__main__":
47     unittest.main()

```

程序 78

对应文件: question3/domain_range.py; question3/innovation/frozen/question3/domain_range.py; question3/results/dispatch_20260913/development/code_snapshot/question3/domain_range.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/domain_range.py; question3/results/round18_adverse_main/code_snapshot/question3/domain_range.py; question3/results/round18_adverse_rounding/code_snapshot/question3/domain_range.py; question3/results/round18_dev_main/code_snapshot/question3/domain_range.py; question3/results/round18_dev_rounding/code_snapshot/question3/domain_range.py; question3/results/round18_holdout_main/code_snapshot/question3/domain_range.py; question3/results/round18_holdout_rounding/code_snapshot/question3/domain_range.py; question3/results/round18_outer_main/code_snapshot/question3/domain_range.py; question3/results/round18_outer_rounding/code_snapshot/question3/domain_range.py; question3/results/round19_adverse_main/code_snapshot/question3/domain_range.py; question3/results/round19_adverse_rounding/code_snapshot/question3/domain_range.py; question3/results/round19_dev_main/code_snapshot/question3/domain_range.py; question3/results/round19_dev_rounding/code_snapshot/question3/domain_range.py; question3/results/round19_holdout_main/code_snapshot/question3/domain_range.py; question3/results/round19_holdout_rounding/code_snapshot/question3/domain_range.py; question3/results/round19_outer_main/code_snapshot/question3/domain_range.py; question3/results/round19_outer_rounding/code_snapshot/question3/domain_range.py

```
1 """Published-domain range bound and sufficient second-bearing certificate.
2
3 No source truth. The 31x3x3 diameter proxy ranks candidates only; analytic
4 inequalities certify the continuous sector. Used after task selection only.
5 """
6 from functools import lru_cache
7 import math
8 import numpy as np
9 from question2.strategy import diameters,to_world
10 from .current_point import local_offset
11
12
13 def range_upper(first,bearing,error_deg):
14     rho=math.hypot(*first)
15     if not math.isfinite(rho+bearing+error_deg) or not 0<error_deg<45:
16         raise ValueError('Invalid first-bearing geometry')
17     if rho==0 or rho>1800:return 1500
18     delta=math.radians(error_deg)
19     beta=(math.atan2(first[1],first[0])-math.radians(bearing)+math.pi)%(2*math.pi)-math.pi
20     m=rho*(-1. if abs(beta)+delta>=math.pi else min(math.cos(beta-delta),math.cos(beta+delta)))
21     exact=-m+math.sqrt(max(0.,m*m+1800**2-rho*rho))
22     upper=min(1500,math.ceil(exact+1e-5))
23     return upper if upper>5 else 1500
24
25
26 def certified(offset,upper,error_deg,min_angle_deg):
27     a,b=offset
28     if not all(map(math.isfinite,[a,b,upper,error_deg,min_angle_deg])):return False
29     if a<0 or not 5<upper<=1500 or not 0<error_deg<45 or not 4*error_deg<min_angle_deg<90:return False
30     d=math.radians(error_deg);h=a*math.cos(d)-abs(b)*math.sin(d);n=a*a+b*b
31     d5=n+25-10*h;du=n+upper**2-2*upper*h
32     # R >= max(1000,r). On [1000,U], n-2rh <= 0 follows at r=1000.
33     receives=d5<=1000**2-1e-4 and (du<=1000**2-1e-4 if upper<=1000 else n<=2000*h-1e-4)
```

```

34     cross=abs(b)*math.cos(d)-a*math.sin(d)
35     return bool(receives and cross>5+1e-7 and cross>=math.sqrt(max(d5,du))*math.sin(math.radians(min_angle_deg))+1e-7)
36
37
38 @lru_cache(maxsize=4096)
39 def proxy(offset,upper,error_deg,min_angle_deg):
40     if not certified(offset,upper,error_deg,min_angle_deg):return None
41     d=math.radians(error_deg)
42     r,a=np.meshgrid(np.linspace(5.001,upper,31),np.linspace(-d,d,3),np.linspace(-d,d,3),indexing='ij')
43     g=np.column_stack([(r*np.cos(a)).ravel(),(r*np.sin(a)).ravel()])
44     diameter=float(diameters(offset,g,e.ravel(),error_deg).max())
45     x,y=offset;h=x*math.cos(d)-abs(y)*math.sin(d);n=x*x+y*y
46     maximum=math.sqrt(max(n+25-10*h,n+upper**2-2*upper*h))
47     cells=math.ceil(diameter/28)*math.ceil(2*upper*math.sin(d)/28)
48     return dict(offset=offset,sampled_worst_diameter=diameter,max_source_distance=maximum,
49               proxy_cells=cells,continuation_proxy_s=maximum/5+8.6*cells)
50
51
52 def choose(first,bearing,current,config,base_catalog,upper):
53     original={tuple(row['offset']) for row in base_catalog}
54     offsets=original|{(x*upper/1500,y*upper/1500) for x,y in original}
55     best=None
56     for offset in sorted(offsets):
57         row=proxy(offset,upper,config['error_deg'],config['min_angle_deg'])
58         if row is None:continue
59         q=tuple(to_world(offset,first,bearing));score=math.dist(current,q)/5+5+row['continuation_proxy_s']
60         if best is None or (score,offset)<best[0]:best=((score,offset),q,row)
61     if best is None:raise RuntimeError('No domain-certified candidate including original catalog')
62     metadata=dict(source_range_upper_m=upper,range_prior='published_domain_1800',
63                 error_deg=config['error_deg'],min_angle_deg=config['min_angle_deg'])
64     opportunity=config.get('second_point_opportunity','off')
65     if opportunity!='off':
66         row=proxy(local_offset(first,bearing,current),upper,config['error_deg'],config['min_angle_deg'])
67         if row is not None:
68             score=5+row['continuation_proxy_s']
69             if opportunity=='current_if_certified' or score<best[0][0]-1e-8:
70                 return tuple(current),dict(mode='domain_current',score_s=score,**row,**metadata,
71                     selection_rule=opportunity,catalog_score_s=best[0][0],catalog_point=best[1])
72     return best[1],dict(mode='domain_cost_proxy',score_s=best[0][0]**best[2]**metadata)

```

程序 79

对应文件: question3/geometry.py; question3/innovation/frozen/question3/geometry.py; question3/results/dispatch_20260913/development/code_snapshot/question3/geometry.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/geometry.py; question3/results/round14_adverse_main/code_snapshot/question3/geometry.py; question3/results/round14_adverse_rounding/code_snapshot/question3/geometry.py; question3/results/round14_dev_main/code_snapshot/question3/geometry.py; question3/results/round14_dev_rounding/code_snapshot/question3/geometry.py; question3/results/round14_holdout_main/code_snapshot/question3/geometry.py; question3/results/round14_holdout_rounding/code_snapshot/question3/geometry.py; question3/results/round14_initial_dev_main/code_snapshot/question3/geometry.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/geometry.py; question3/results/round14_outer_main/code_snapshot/question3/geometry.py; question3/results/round14_outer_rounding/code_snapshot/question3/geometry.py; question3/results/round17_adverse_main/code_snapshot/question3/geometry.py; question3/results/round17_adverse_rounding/code_snapshot/question3/geometry.py; question3/results/round17_dev_main/code_snapshot/question3/geometry.py; question3/results/round17_dev_rounding/code_snapshot/question3/

/geometry.py; question3/results/round18_adverse_main/code_snapshot/question3/geometry.py; question3/results/round18_adverse_rounding/code_snapshot/question3/geometry.py; question3/results/round18_dev_main/code_snapshot/question3/geometry.py; question3/results/round18_dev_rounding/code_snapshot/question3/geometry.py; question3/results/round18_holdout_main/code_snapshot/question3/geometry.py; question3/results/round18_holdout_rounding/code_snapshot/question3/geometry.py; question3/results/round18_outer_main/code_snapshot/question3/geometry.py; question3/results/round18_outer_rounding/code_snapshot/question3/geometry.py; question3/results/round19_adverse_main/code_snapshot/question3/geometry.py; question3/results/round19_adverse_rounding/code_snapshot/question3/geometry.py; question3/results/round19_dev_main/code_snapshot/question3/geometry.py; question3/results/round19_dev_rounding/code_snapshot/question3/geometry.py; question3/results/round19_holdout_main/code_snapshot/question3/geometry.py; question3/results/round19_holdout_rounding/code_snapshot/question3/geometry.py; question3/results/round19_outer_main/code_snapshot/question3/geometry.py; question3/results/round19_outer_rounding/code_snapshot/question3/geometry.py

```

1 """Public geometry only; reuse Q1 and Q2, no simulator imports."""
2 import math
3 import numpy as np
4 from question1.solve import solve
5 from question2.strategy import candidate, to_world
6
7
8 def stations(radius=1500., phase_rad=0.):
9     return [(0., 0.)] + [(radius*math.cos(phase_rad+k*math.pi/3), radius*math.sin(phase_rad+k*math.pi/3)) for k in
    ↪ range(6)]
10
11
12 def station_cover_bound(radius=1500.):
13     if not math.isfinite(radius) or radius<=0: raise ValueError('Invalid search ring radius')
14     # Centre covers radius <=900. Remaining annulus: nearest ring angle <=30deg.
15     return max(900., *(math.sqrt(r*r+radius**2-2*r*radius*math.cos(math.pi/6)) for r in [900,1800]))
16
17
18 def second_point(first, bearing, offset, error_deg, min_angle):
19     if not candidate(offset,min_angle,error_deg):
20         raise ValueError('Second offset lacks analytic reception/angle certificate')
21     if min_angle <= 4*error_deg:
22         raise ValueError('Angle margin insufficient for bounded two-wedge intersection')
23     return tuple(to_world(offset,first,bearing))
24
25
26 def optical_plan(first, bearing, second, reading, error_deg, cell_m, max_points):
27     """Cover an inflated rotated bounding box by <=28m cells. No hidden G.
28
29     Cells have diagonal/2 <20m. Testing every centre guarantees a hit if G is in P.
30     """
31     if not 0 < cell_m < 20*math.sqrt(2):
32         raise ValueError('Optical cells must fit strictly inside a radius-20 circle')
33     result=solve([dict(x=first[0],y=first[1],bearing_deg=bearing),
34                  dict(x=second[0],y=second[1],bearing_deg=reading)],error_deg)
35     if result['status']!='bounded':
36         raise RuntimeError('Invalid localization region: '+result['status'])
37     vertices=np.array(result['vertices'])
38     a=math.radians(bearing)
39     rot=np.array([[math.cos(a),-math.sin(a)],[math.sin(a),math.cos(a)]])
40     local=(vertices-np.array(first))@rot
41     lo=local.min(axis=0)-1e-5;hi=local.max(axis=0)+1e-5

```

```

42     sizes=np.maximum(1,np.ceil((hi-lo)/cell_m).astype(int))
43     n=int(np.prod(sizes))
44     if n>max_points:
45         raise RuntimeError(f'Optical plan exceeds configured guard: {n}')
46     points=[]
47     for j in range(sizes[1]):
48         cols=range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))
49         for i in cols:
50             p=lo+(np.array([i+.5,j+.5])/sizes)*(hi-lo)
51             points.append(tuple(np.array(first)+rot@p))
52     certificate=dict(diameter=result['diameter'],vertices=result['vertices'],
53                     bbox_widths=(hi-lo).tolist(),planned_clear_calls=n,
54                     covering_radius=float(np.linalg.norm((hi-lo)/sizes)/2))
55     return points,certificate

```

程序 80

对应文件: question3/innovation/__init__.py; question3/innovation/ablation/bounded/code_snapshot/__init__.py; question3/innovation/ablation/pre_round_stress/code_snapshot/__init__.py; question3/innovation/ablation_expanded/bounded/code_snapshot/__init__.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/__init__.py; question3/innovation/expanded_development/bounded/code_snapshot/__init__.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/__init__.py; question3/innovation/final_validation/bounded/code_snapshot/__init__.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/__init__.py; question3/innovation/package_smoke/bounded/code_snapshot/__init__.py; question3/innovation/round01_homing/bounded/code_snapshot/__init__.py; question3/innovation/round01_homing/bounded_full/code_snapshot/__init__.py; question3/innovation/round01_homing/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round02_parallax/bounded/code_snapshot/__init__.py; question3/innovation/round02_parallax/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round03_refresh/bounded/code_snapshot/__init__.py; question3/innovation/round03_refresh/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round04_shared/bounded/code_snapshot/__init__.py; question3/innovation/round04_shared/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round05_strip/bounded/code_snapshot/__init__.py; question3/innovation/round05_strip/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round06_belief_tour/bounded/code_snapshot/__init__.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round07_reception_bracket/bounded/code_snapshot/__init__.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round08_survey/bounded/code_snapshot/__init__.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round09_information/bounded/code_snapshot/__init__.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/__init__.py; question3/innovation/round10_continuous_cover/bounded/code_snapshot/__init__.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/__init__.py; question3/results/dispatch

```
atch_20260913/development/code_snapshot/question3/innovation/__init__.py;
question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/__init__.py
```

```
1 """Isolated Q3 mechanism experiments, not the recommended policy."""
```

程序 81

对应文件: question3/innovation/ablation/bounded/code_snapshot/bootstrap.py; question3/innovation/ablation/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/ablation_expanded/bounded/code_snapshot/bootstrap.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/expanded_development/bounded/code_snapshot/bootstrap.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/final_validation/bounded/code_snapshot/bootstrap.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round01_homing/bounded/code_snapshot/bootstrap.py; question3/innovation/round01_homing/bounded_full/code_snapshot/bootstrap.py; question3/innovation/round01_homing/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round02_parallax/bounded/code_snapshot/bootstrap.py; question3/innovation/round02_parallax/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round03_refresh/bounded/code_snapshot/bootstrap.py; question3/innovation/round03_refresh/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round04_shared/bounded/code_snapshot/bootstrap.py; question3/innovation/round04_shared/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round05_strip/bounded/code_snapshot/bootstrap.py; question3/innovation/round05_strip/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round06_belief_tour/bounded/code_snapshot/bootstrap.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round07_reception_bracket/bounded/code_snapshot/bootstrap.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round08_survey/bounded/code_snapshot/bootstrap.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round09_information/bounded/code_snapshot/bootstrap.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/bootstrap.py; question3/innovation/round10_continuous_cover/bounded/code_snapshot/bootstrap.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/bootstrap.py

```
1 """Freeze source materials and recommendation before any experimental changes."""
2 import hashlib
3 import json
4 import platform
5 import shutil
6 import subprocess
7 import sys
```

```

8 import zipfile
9 from pathlib import Path
10 from xml.etree import ElementTree as ET
11
12 R00T = Path(__file__).resolve().parent
13 REPO = R00T.parents[1]
14
15 def main():
16     frozen = R00T / 'frozen'
17     frozen.mkdir(exist_ok=False)
18     paths = list((REPO/'question3').glob('*.py'))
19     paths += [REPO/'question1/solve.py', REPO/'question2/strategy.py',
20              REPO/'question3/configs/recommended_omni.json',
21              REPO/'question3/releases/recommended_manifest.json']
22     paths += list((REPO/'question3/docs').glob('*'))
23     paths += list((REPO/'B 题').rglob('*.pdf')) + list((REPO/'B 题').rglob('*.docx'))
24     hashes = {}
25     for p in paths:
26         if not p.is_file():
27             continue
28         rel = p.relative_to(REPO)
29         dst = frozen/rel
30         dst.parent.mkdir(parents=True, exist_ok=True)
31         shutil.copy2(p, dst)
32         hashes[str(rel)] = hashlib.sha256(p.read_bytes()).hexdigest()
33     prior = json.loads((REPO/'question3/releases/recommended_manifest.json').read_text())
34     mismatch = [p for p,h in prior['code_sha256'].items() if hashes.get(p) != h]
35     assert not mismatch, mismatch
36     assert hashes['question3/configs/recommended_omni.json'] == prior['config_sha256']
37     (R00T/'freeze_manifest.json').write_text(json.dumps(dict(
38         sha256=hashes, recommendation_matches_manifest=True,
39         python=sys.version, platform=platform.platform(),
40         git_head=subprocess.check_output(['git', 'rev-parse', 'HEAD'], cwd=REPO, text=True).strip(),
41         initial_git_status=subprocess.check_output(['git', 'status', '--short'], cwd=REPO, text=True),
42         official_runs=0, formal_runs=0), ensure_ascii=False, indent=2))
43     sources = R00T/'source_review'
44     sources.mkdir()
45     subprocess.run(['pdftotext', '-layout', str(REPO/'B 题/B 题.pdf'), str(sources/'题面.txt')], check=True)
46     for p in (REPO/'B 题/附件').glob('*.docx'):
47         with zipfile.ZipFile(p) as z:
48             xml = ET.fromstring(z.read('word/document.xml'))
49             ns = {'w': 'http://schemas.openxmlformats.org/wordprocessingml/2006/main'}
50             paragraphs = [''.join(t.text or '' for t in para.findall('.//w:t', ns)) for para in xml.findall('.//w:p', ns)]
51             (sources/(p.stem+'.txt')).write_text('\n'.join(paragraphs))
52     print(json.dumps({'frozen_files': len(hashes), 'manifest_match': True}))
53
54 if __name__ == '__main__':
55     main()

```

程序 82

对应文件: question3/innovation/ablation/bounded/code_snapshot/coverage.py; question3/innovation/ablation/pre_round_stress/code_snapshot/coverage.py; question3/innovation/ablation_expanded/bounded/code_snapshot/coverage.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/coverage.py; question3/innovation/coverage.py; question3/innovation/expanded_development/bounded/code_snapshot/coverage.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/coverage.py; question3/innovation/final_validation/bounded/code_snapshot/coverage.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/coverage.py; question3/innovation/package_smoke/bounded/code_snapshot/coverage.py; question3/innovation/round02_parallax/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round03_refresh/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round04_shared/pre_round_st

ress/code_snapshot/coverage.py; question3/innovation/round05_strip/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/coverage.py; question3/innovation/round10_continuous_cover/bounded/code_snapshot/coverage.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/coverage.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/coverage.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/coverage.py

```

1 """Continuous equal-disk coverage from public no-signal observations only."""
2 import itertools
3 import math
4 from functools import lru_cache
5 import numpy as np
6
7 @lru_cache(maxsize=8192)
8 def cover_radius(points):
9     if not points: return math.inf
10    p=np.asarray(points);candidates=[np.array([0.,0.])]
11    for a in p:
12        norm=float(np.linalg.norm(a))
13        candidates.append(-1800*a/norm if norm else np.array([1800.,0.]))
14    for a,b in itertools.combinations(p,2):
15        n=b-a;norm=float(np.linalg.norm(n))
16        if norm<1e-8:continue
17        unit=n/norm;offset=float(((b@a-a@a)/(2*norm))
18        if abs(offset)<=1800:
19            tangent=np.array([-unit[1],unit[0]])*math.sqrt(max(0.,1800**2-offset**2))
20            candidates.extend([offset*unit+tangent,offset*unit-tangent])
21    for a,b,c in itertools.combinations(p,3):
22        A=2*np.stack([b-a,c-a])
23        if abs(float(np.linalg.det(A)))<1e-8:continue
24        v=np.linalg.solve(A,[b@a-a@a,c@c-a@a])
25        if np.linalg.norm(v)<=1800+1e-8:candidates.append(v)
26    v=np.asarray(candidates)
27    return float(np.sqrt(((v[:,None,:]-p[None,:,:])**2).sum(axis=2)).min(axis=1).max())
28
29 def covered(points):
30     pts=tuple(sorted(set(tuple(map(float,p)) for p in points)))
31     return cover_radius(pts)<1000-1e-4
32
33 def verify_cover_boxes(points):
34     """Independent conservative subdivision audit: no Voronoi formula reuse."""
35     pts=np.asarray(points);stack=[(0.,0.,1800.,0)];leaves=0;visited=0;deepest=0
36     while stack:
37         x,y,h,depth=stack.pop();visited+=1;deepest=max(deepest,depth)
38         if visited>2000000 or depth>30:raise AssertionError('Coverage interval audit unresolved')
39         nearest=np.maximum(np.abs([x,y])-h,0)
40         if nearest@nearest>1800**2:continue
41         corners=np.array([[x-h,y-h],[x+h,y-h],[x+h,y+h],[x-h,y+h]])
42         maximum=np.linalg.norm(corners[None,:,:]-pts[:,None,:],axis=2).max(axis=1)
43         if float(maximum.min())<=1000-1e-8:
44             leaves+=1;continue
45         for sx,sy in itertools.product((-1,1),repeat=2):
46             stack.append((x+sx*h/2,y+sy*h/2,h/2,depth+1))
47     return dict(covered_boxes=leaves,visited_boxes=visited,max_depth=deepest)

```


程序 83

对应文件: question3/innovation/ablation/bounded/code_snapshot/coverage_audit.py; question3/innovation/ablation/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/ablation_expanded/bounded/code_snapshot/coverage_audit.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/coverage_audit.py; question3/innovation/expanded_development/bounded/code_snapshot/coverage_audit.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/final_validation/bounded/code_snapshot/coverage_audit.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/package_smoke/bounded/code_snapshot/coverage_audit.py; question3/innovation/round02_parallax/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round03_refresh/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round04_shared/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round05_strip/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/coverage_audit.py; question3/innovation/round10_continuous_cover/bounded/code_snapshot/coverage_audit.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/coverage_audit.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/coverage_audit.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/coverage_audit.py

```
1 """Post-run physical and continuous stop audit, never imported by policy."""
2 import json
3 import math
4 from .coverage import verify_cover_boxes
5
6 def audit(folder):
7     data=json.loads((folder/'result.json').read_text())
8     case=json.loads((folder/'evaluator_hidden_case.json').read_text())
9     sources={s['channel']:s for s in case['sources']}
10    requests={};done=set();cleared=set();negatives={c:set() for c in range(1,21)};directions={}
11    position=(0.,0.);channel=1;virtual=0.;exited=False;actions=0
12    for row in map(json.loads,(folder/'actions.jsonl').read_text().splitlines()):
13        if row['type']=='request':requests[row['payload']]['request_id']=row
14        if row['type']!='response' or not row['response'].get('accepted'):continue
15        rid=row['request_id']
16        if rid in done:continue
17        done.add(rid);req=requests[rid];r=row['response'];b=req['payload'];path=req['path']
18        if path in ('/measure','/clear'):
19            p=(b['position']['x'],b['position']['y']);c=b['channel']
20            assert all(math.isfinite(v) and abs(v)<=2000000 for v in p)
21            virtual+=math.dist(position,p)/5;position=p
22            s=sources.get(c);d=math.inf if s is None or c in cleared else math.dist(p,(s['x'],s['y']))
23            if path=='/measure':
24                virtual+=5+(c!=channel);channel=c
25                expected='no_signal' if s is None or c in cleared or d>s['radius'] else ('near' if d<=5 else
                    ↳ 'direction')
```

```

26         assert r['measure_result']==expected
27         if expected=='no_signal':negatives[c].add(p)
28         if expected=='direction':
29             angle=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
30             error=1. if data['summary']['rounding']=='bounded' else 1.005
31             assert abs((r['svd_deg']-angle+180)%360-180)<=error+1e-8
32             if (p,c) in directions:assert directions[p,c]==r['svd_deg']
33             directions[p,c]=r['svd_deg']
34         else:
35             virtual+=3
36             assert (r['clear_result']=='success')== (d<=20)
37             if d<=20:cleared.add(c);virtual+=2
38             actions+=1
39             elif path=='/exit':exited=True
40             assert abs(virtual-r['virtual_time_s'])<len(done)*1e-6+1e-4
41         assert exited and cleared==set(sources)
42         result=data['strategy'];assert result['status']=='complete' and set(result['cleared_channels'])==cleared
43         cert=result['stopping_certificate'];checked={}
44         if cert['method']=='source_count_upper_bound':
45             assert len(cleared)==16
46         elif cert['method']=='continuous_disk_union':
47             for c in set(range(1,21))-cleared:
48                 pts=tuple(sorted(tuple(p) for p in cert['negative_positions'][str(c)]))
49                 assert set(pts)<=negatives[c]
50                 if pts not in checked:checked[pts]=verify_cover_boxes(pts)
51         else:
52             cover=set(map(tuple,result['station_positions']))
53             for c in set(range(1,21))-cleared:assert cover<=negatives[c]
54         return actions

```

程序 84

对应文件: question3/innovation/ablation/bounded/code_snapshot/experiment.py; question3/innovation/ablation/pre_round_stress/code_snapshot/experiment.py; question3/innovation/expanded_development/bounded/code_snapshot/experiment.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round02_parallax/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round03_refresh/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round04_shared/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round05_strip/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round10_continuous_cover/bounded/code_snapshot/experiment.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/experiment.py

```

1 """Evaluator-only self experiments. Strategy inputs exclude cases and evaluator state."""
2 import argparse
3 import hashlib
4 import json
5 import math
6 import shutil
7 import statistics
8 import sys
9 import time
10 from pathlib import Path
11 import numpy as np
12

```

```

13 ROOT=Path(__file__).resolve().parent
14 sys.path.insert(0,str(ROOT/'frozen'))
15 from question3.cases import development_cases
16 from question3.simulator import Simulator
17 from question3.interface import Client,JsonLlog
18 from question3.strategy import run as baseline_run
19 from question3.audit import audit_folder
20 from .strategy import run as candidate_run
21
22 class PublicClient:
23     """Narrow public observation/action capability; no transport/evaluation accessor."""
24     __slots__=('__client',)
25     def __init__(self,client):self.__client=client
26     @property
27     def position(self):return self.__client.position
28     @property
29     def channel(self):return self.__client.channel
30     def enter(self):return self.__client.enter()
31     def measure(self,*a,**kw):return self.__client.measure(*a,**kw)
32     def clear(self,*a,**kw):return self.__client.clear(*a,**kw)
33     def exit(self):return self.__client.exit()
34     def metrics(self):return self.__client.metrics()
35
36 def dump(path,data):
37     path.parent.mkdir(parents=True,exist_ok=True)
38     path.write_text(json.dumps(data,ensure_ascii=False,indent=2))
39
40 def audit_new(folder):
41     case=json.loads((folder/'evaluator_hidden_case.json').read_text())
42     truth={s['channel']:np.array([s['x'],s['y']]) for s in case['sources']}
43     events=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
44     counts={}
45     for e in events:
46         kind=e['type'];counts[kind]=counts.get(kind,0)+1
47         if kind not in ('feasible_update','certified_measure','certified_clear','range_bracket'):
48             continue
49         v=np.asarray(e['vertices']);g=truth[e['channel']]
50         assert len(v)>0 and np.isfinite(v).all()
51         edges=np.roll(v,-1,axis=0)-v;delta=g-v
52         cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
53         assert np.all(cross>=-1e-4) or np.all(cross<=1e-4),('source excluded',e)
54         if kind=='range_bracket':
55             z=(g-np.asarray(e['first']))@np.asarray(e['axis'])
56             assert e['lower']-1e-6<=z<=e['upper']+1e-6
57         if kind in ('certified_measure','certified_clear'):
58             p=np.asarray(e['point']);radius=e['radius']
59             assert np.linalg.norm(v-p,axis=1).max()<=radius+1e-8
60             assert radius<(1000 if kind=='certified_measure' else 20)
61     return counts
62
63 def aggregate(rows,reference='BASE'):
64     groups={n:[r for r in rows if r['variant']==n] for n in sorted([r['variant'] for r in rows])}
65     ref={r['case_id']:r for r in groups[reference]}
66     result={}
67     for n,rs in groups.items():
68         t=[r['penalized_virtual_s'] for r in rs]
69         deltas=[(r['penalized_virtual_s']/ref[r['case_id']]['penalized_virtual_s']-1,r['case_id']) for r in rs]
70         result[n]=dict(runs=len(rs),complete=sum(r['success'] for r in rs),
71             failed=sum(not r['success'] for r in rs),timeouts=sum(r['timeout'] for r in rs),
72             false_complete=sum(r['false_complete'] for r in rs),
73             mean=float(np.mean(t)),p95=float(np.percentile(t,95)),maximum=max(t),
74             worst_case=rs[int(np.argmax(t))]['case_id'],worst_paired_regression=max(deltas),
75             clearance_ratio_mean=float(np.mean([r['cleared']/r['total'] for r in rs])),
76             raw_mean=float(np.mean([r['metrics']['virtual_time_s'] for r in rs])),
77             movement_detection_etc={k:float(np.mean([r['metrics']['components'][k] for r in rs])) for k in
78                 ↪ rs[0]['metrics']['components']},
79             phase_means={k:float(np.mean([r['metrics']['phases'].get(k,0) for r in rs])) for k in
80                 ↪ sorted(set().union(*([r['metrics']['phases'] for r in rs])))},
81             mean_per_cleared=float(np.mean([r['metrics']['virtual_time_s']/max(1,r['cleared']) for r in rs])),
82             wall_p95=float(np.percentile([r['elapsed_wall_s'] for r in rs],95)),
83             clear_failure_mean=float(np.mean([r['metrics']['counts']['clear_failure'] for r in rs]))
84     for n,v in result.items():
85         v['mean_change_pct']=100*(v['mean']/result[reference]['mean']-1)

```

```

84         v['significant']=n!=reference and v['mean_change_pct']<=-5 and v['complete']>=result[reference]['complete']
85         ↪ and v['p95']<=result[reference]['p95']
86     return result
87
88 def main():
89     p=argparse.ArgumentParser()
90     p.add_argument('--round',required=True);p.add_argument('--variants',default='BASE,HOMING')
91     p.add_argument('--rounding',default='bounded',choices=['bounded','pre_round_stress'])
92     p.add_argument('--fixtures');p.add_argument('--limit',type=int)
93     p.add_argument('--stage',default='development');p.add_argument('--label')
94     args=p.parse_args()
95     rounddir=ROOT/args.round
96     out=rounddir/(args.label or args.rounding)
97     out.mkdir(parents=True,exist_ok=False)
98     baseline=json.loads((ROOT/'frozen/question3/configs/recommended_omni.json').read_text())
99     names=args.variants.split(',')
100     configs={n:(baseline if n=='BASE' else dict(baseline_config=baseline,mechanism=n.lower())) for n in names}
101     dump(out/'configs.json',configs)
102     manifest={str(x.relative_to(ROOT)):hashlib.sha256(x.read_bytes()).hexdigest() for x in ROOT.glob('*.py')}
103     dump(out/'code_sha256.json',manifest)
104     for file in ROOT.glob('*.py'):
105         dest=out/'code_snapshot'/file.name;dest.parent.mkdir(exist_ok=True);shutil.copy2(file,dest)
106     cases=json.loads(Path(args.fixtures).read_text()) if args.fixtures else list(development_cases())
107     if args.limit:cases=cases[:args.limit]
108     dump(out/'fixtures.json',cases)
109     dump(out/'metadata.json',dict(stage=args.stage,rounding=args.rounding,
110         distinct_layouts=len({json.dumps(c['sources'],sort_keys=True) for c in cases}),
111         case_sha256={c['id']:hashlib.sha256(json.dumps(c,sort_keys=True).encode()).hexdigest() for c in cases},
112         official_runs=0,formal_runs=0,numeric_parameter_search_trials=0))
113     rows=[]
114     for i,case in enumerate(cases):
115         order=names[i%len(names):]+names[:i%len(names)]
116         for name in order:
117             folder=out/name/case['id'];folder.mkdir(parents=True)
118             dump(folder/'evaluator_hidden_case.json',case)
119             log=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
120             env=Simulator(case,rounding=args.rounding)
121             client=Client(env,'SELF',log,5.)
122             start=time.perf_counter()
123             try:
124                 result=(baseline_run if name=='BASE' else candidate_run)(PublicClient(client),configs[name],events)
125             except Exception as exc:
126                 result=dict(status='incomplete',failure=f'{type(exc).__name__}:
127                 ↪ {exc}',metrics=client.metrics(),regions=[])
128             wall=time.perf_counter()-start
129             truth=env.evaluation() # First evaluator access, after the policy returned.
130             row=dict(variant=name,case_id=case['id'],stage=args.stage,rounding=args.rounding,
131                 status=result['status'],failure=result.get('failure'),elapsed_wall_s=wall,
132                 total=truth['total'],cleared=truth['cleared'],metrics=result['metrics'])
133             row['false_complete']=result['status']=='complete' and bool(truth['remaining_channels'])
134             row['timeout']=bool(result.get('failure') and any(x in result['failure'].lower() for x in
135                 ↪ ('deadline','timeout','budget')))
136             row['success']=result['status']=='complete' and not truth['remaining_channels']
137             dump(folder/'result.json',dict(summary=row,strategy=result))
138             log.close();events.close()
139             try:
140                 if row['success']:
141                     if name.startswith('COVER_'):
142                         from .coverage_audit import audit as coverage_audit
143                         row['audited_actions']=coverage_audit(folder)
144                     else:row['audited_actions']=audit_folder(folder)
145                     row['event_counts']=audit_new(folder)
146                     row['audit']='pass'
147                 else:row['audit']='incomplete retained'
148             except Exception as exc:
149                 row['audit']=f'FAIL {type(exc).__name__}: {exc}'
150                 row['success']=False;row['false_complete']=True
151                 row['penalized_virtual_s']=truth['virtual_time_s'] if row['success'] else
152                 ↪ max(360000,truth['virtual_time_s'])
153             dump(folder/'result.json',dict(summary=row,strategy=result))
154             rows.append(row)
155     dump(out/'summary.json',dict(rows=rows,aggregates=aggregate(rows)))

```

```

152         print(f'{i+1}/{len(cases)} {case["id"]} '+'', '.join(f'{r["variant"]}={r["penalized_virtual_s"]:.1f}'+('
        ↪ '+str(r['failure'])+' '+r['audit'] if not r['success'] else '') for r in rows[-len(names):]),flush=True)
153     stats=aggregate(rows);dump(out/'comparison.json',stats)
154     print(json.dumps({n:{k:v[k] for k in ('complete','runs','mean','mean_change_pct','p95','maximum','significant')}}
        ↪ for n,v in stats.items()),indent=2),flush=True)
155
156 if __name__=='__main__':main()

```

程序 85

对应文件: question3/innovation/ablation/bounded/code_snapshot/fixtures.py;
question3/innovation/ablation/pre_round_stress/code_snapshot/fixtures.py;
question3/innovation/ablation_expanded/bounded/code_snapshot/fixtures.py;
question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/expanded_development/bounded/code_snapshot/fixtures.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/final_validation/bounded/code_snapshot/fixtures.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/fixtures.py;
question3/innovation/package_smoke/bounded/code_snapshot/fixtures.py;
question3/innovation/round02_parallax/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round03_refresh/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round04_shared/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round05_strip/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/fixtures.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/fixtures.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/fixtures.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/fixtures.py

```

1 """Evaluator-only structured generators; policy never imports this module."""
2 import json
3 import math
4 import random
5 from pathlib import Path
6
7 FAMILIES=('uniform','outer','cluster','min_radius','radial','reception_edge')
8 MODES=('spatial_hash','smooth','positive','negative')
9
10 def generate(start,count,prefix):
11     cases=[]
12     for i,seed in enumerate(range(start,start+count)):
13         rng=random.Random(seed);family=FAMILIES[i%6]
14         n=rng.randint(10,16);channels=rng.sample(range(1,21),n)
15         phase=rng.uniform(0,2*math.pi)
16         cluster_r=rng.uniform(100,1700)
17         cluster=(cluster_r*math.cos(phase),cluster_r*math.sin(phase))
18         sources=[]
19         for j,c in enumerate(channels):
20             r=1800*math.sqrt(rng.random());a=rng.uniform(0,2*math.pi)
21             receiver=rng.uniform(1000,1500)
22             if family=='outer':

```

```

23         r=rng.uniform(1600,1800)
24         if i%12==1:a=phase+rng.uniform(-.06,.06)
25     elif family=='cluster':
26         x=cluster[0]+rng.uniform(-60,60);y=cluster[1]+rng.uniform(-60,60)
27         r=math.hypot(x,y);a=math.atan2(y,x)
28     elif family=='min_radius':receiver=1000.
29     elif family=='radial':
30         a=phase+rng.uniform(-.002,.002);r=rng.uniform(0,1800)
31         receiver=1000. if j%2 else 1500.
32     elif family=='reception_edge':
33         receiver=1000. if j%2 else 1500.
34         # Around origin reception threshold; remaining sources on domain boundary.
35         r=receiver+(-1 if j%3 else 1)*rng.uniform(.001,1.) if j%4 else 1800.
36     r=min(1800,r)
37     sources.append(dict(channel=c,x=r*math.cos(a),y=r*math.sin(a),radius=receiver))
38 for mode in MODES:
39     cases.append(dict(id=f'{prefix}_{family}_{seed}_{mode}',seed=seed,error_mode=mode,family=family,sources=sources))
40 return cases
41
42 if __name__=='__main__':
43     root=Path(__file__).resolve().parent
44     out=root/'structured_development.json'
45     if out.exists():raise SystemExit('Refusing to overwrite fixtures')
46     out.write_text(json.dumps(generate(900000,60,'structdev'),indent=2))

```

程序 86

对应文件: question3/innovation/ablation/bounded/code_snapshot/geometry.py;
question3/innovation/ablation/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/ablation_expanded/bounded/code_snapshot/geometry.py;
question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/expanded_development/bounded/code_snapshot/geometry.py;
question3/innovation/expanded_development/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/final_validation/bounded/code_snapshot/geometry.py;
question3/innovation/final_validation/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/geometry.py;
question3/innovation/package_smoke/bounded/code_snapshot/geometry.py;
question3/innovation/round02_parallax/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round03_refresh/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round04_shared/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round05_strip/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round08_survey/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round09_information/bounded/code_snapshot/geometry.py;
question3/innovation/round09_information/pre_round_stress/code_snapshot/geometry.py;
question3/innovation/round10_continuous_cover/bounded/code_snapshot/geometry.py;
question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/geometry.py;
question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/geometry.py;
question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/geometry.py

```

1 """Conservative convex sets from public bounded bearings. No environment access."""
2 import itertools
3 import math
4 import numpy as np
5
6 SLACK = 1e-7
7
8 def clip(poly, normal, bound):
9     normal = np.asarray(normal, float)
10    result = []
11    for a,b in zip(poly, np.roll(poly,-1,axis=0)):
12        da, db = float(normal@a-bound), float(normal@b-bound)
13        if da <= 0:
14            result.append(a)
15            if (da <= 0) != (db <= 0):
16                result.append(a + (b-a)*(da/(da-db)))
17    if not result:
18        raise RuntimeError('Empty conservative feasible polygon')
19    return np.asarray(result)
20
21 def disk(poly, centre, radius):
22     # Tangent halfplanes OUTSIDE the disk; never use an inscribed approximation.
23     centre = np.asarray(centre)
24     for k in range(32):
25         t = 2*math.pi*k/32
26         n = np.array([math.cos(t),math.sin(t)])
27         poly = clip(poly,n,float(n@centre+radius+SLACK))
28     return poly
29
30 def wedge(poly, point, bearing, error=1.005):
31     for sign in (-1,1):
32         t = math.radians(bearing+sign*error)
33         n = np.array([-sign*math.sin(t),sign*math.cos(t)])
34         poly = clip(poly,n,float(n@np.asarray(point)+SLACK))
35     return poly
36
37 def initial(point, bearing, error=1.005):
38     poly = np.array([[-1800.,-1800.],[1800.,-1800.],[1800.,1800.],[-1800.,1800.]])
39     poly = wedge(poly,point,bearing,error)
40     return disk(disk(poly,(0,0),1800),point,1500)
41
42 def bbox(poly, bearing):
43     t = math.radians(bearing)
44     rot = np.array([math.cos(t),-math.sin(t)],[math.sin(t),math.cos(t)])
45     local = poly@rot
46     lo,hi = local.min(axis=0)-SLACK,local.max(axis=0)+SLACK
47     centre = ((lo+hi)/2)@rot.T
48     radius = float(np.linalg.norm(poly-centre,axis=1).max()+SLACK)
49     return centre,radius,lo,hi,rot
50
51 def enclosing(poly):
52     # Small convex polygons: enumerate every support pair/triple, then verify all vertices.
53     candidates = [p for p in poly]
54     candidates += [(a+b)/2 for a,b in itertools.combinations(poly,2)]
55     for a,b,c in itertools.combinations(poly,3):
56         u,v = b-a,c-a
57         A=2*np.stack([u,v])
58         if abs(float(np.linalg.det(A))) < 1e-9:
59             continue
60         p=a+np.linalg.solve(A,[u@u,v@v])
61         if np.isfinite(p).all():
62             candidates.append(p)
63     best=min((float(np.linalg.norm(poly-p,axis=1).max()),tuple(p)) for p in candidates)
64     return np.array(best[1]),best[0]+SLACK
65
66 def grid(poly,bearing):
67     _,_,lo,hi,rot=bbox(poly,bearing)
68     sizes=np.maximum(1,np.ceil((hi-lo)/28).astype(int))
69     if int(np.prod(sizes))>2000:
70         raise RuntimeError('Optical finite fallback guard')
71     for j in range(sizes[1]):
72         for i in (range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))):
73             yield (lo+np.array([i+.5,j+.5])*(hi-lo)/sizes)@rot.T

```

```

74
75 def contains(poly,p,tol=1e-4):
76     e=np.roll(poly,-1,axis=0)-poly
77     v=np.asarray(p)-poly
78     cross=e[:,0]*v[:,1]-e[:,1]*v[:,0]
79     return bool(np.all(cross>=-tol) or np.all(cross<=tol))
80
81 def parallax_point(poly,bearing,current):
82     """Geometry-scaled parallax: balance cross-track detour and angular uncertainty.
83
84     Baseline scale derives from  $r \sin(\delta)$ , with 20m as the physical accuracy target.
85     No source-distance or error-mode oracle. Candidates require a full vertex reception check.
86     """
87     centre,radius,*_ =bbox(poly,bearing)
88     t=math.radians(bearing);side=np.array([-math.sin(t),math.cos(t)])
89     # sqrt(range * optical radius) gives a scale between optical accuracy and range.
90     offset=math.sqrt(radius*20)
91     candidates=[centre+sign*offset*side for sign in (-1,1)]
92     valid=[p for p in candidates if np.linalg.norm(poly-p,axis=1).max()<1000-1e-5]
93     if not valid:return centre,radius
94     p=min(valid,key=lambda p:math.dist(current,p))
95     return p,float(np.linalg.norm(poly-p,axis=1).max()+SLACK)
96
97 def worthwhile(poly,bearing,point,error=1.005):
98     """Conservative all-output contraction gate, NOT a sampled source probability."""
99     old=bbox(poly,bearing)[1]
100     if old<=20-1e-5:return False
101     point=np.asarray(point)
102     if contains(poly,point):
103         low,high=0.,360.
104     else:
105         ref=math.degrees(math.atan2(*(poly.mean(axis=0)-point)[::-1]))
106         angles=[ref+(math.degrees(math.atan2(*(v-point)[::-1]))-ref+180)%360-180 for v in poly]
107         low,high=min(angles)-error,max(angles)+error
108     width=(high-low)/16
109     worst=0.
110     for i in range(16):
111         mid=low+(i+.5)*width
112         try:
113             posterior=wedge(poly,point,mid,error+width/2)
114         except RuntimeError:
115             continue
116         worst=max(worst,bbox(posterior,mid)[1])
117     return worst<=max(20-1e-5,old/2)-1e-5

```

程序 87

对应文件: question3/innovation/ablation/bounded/code_snapshot/routing.py;
question3/innovation/ablation/pre_round_stress/code_snapshot/routing.py;
question3/innovation/ablation_expanded/bounded/code_snapshot/routing.py;
question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/
routing.py; question3/innovation/expanded_development/bounded/code_sna
pshot/routing.py; question3/innovation/expanded_development/pre_round_
stress/code_snapshot/routing.py; question3/innovation/final_validatio
n/bounded/code_snapshot/routing.py; question3/innovation/final_validat
ion/pre_round_stress/code_snapshot/routing.py; question3/innovation/p
ackage_smoke/bounded/code_snapshot/routing.py; question3/innovation/r
ound02_parallax/pre_round_stress/code_snapshot/routing.py; question3/
innovation/round03_refresh/pre_round_stress/code_snapshot/routing.py;
question3/innovation/round04_shared/pre_round_stress/code_snapshot/rou
ting.py; question3/innovation/round05_strip/pre_round_stress/code_sna
pshot/routing.py; question3/innovation/round06_belief_tour/bounded/cod

e_snapshot/routing.py; question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/routing.py; question3/innovation/round07_reception_bracket/bounded/code_snapshot/routing.py; question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/routing.py; question3/innovation/round08_survey/bounded/code_snapshot/routing.py; question3/innovation/round08_survey/pre_round_stress/code_snapshot/routing.py; question3/innovation/round09_information/bounded/code_snapshot/routing.py; question3/innovation/round09_information/pre_round_stress/code_snapshot/routing.py; question3/innovation/round10_continuous_cover/bounded/code_snapshot/routing.py; question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/routing.py; question3/innovation/routing.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/routing.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/routing.py

```

1 """Receding open tour through public feasible-set centres and required scans."""
2 import math
3
4 def choose(current,jobs,global_tour=True):
5     # job = (kind,id,point). Unknown true endpoints are never used.
6     if not global_tour:
7         return min(jobs,key=lambda j:(math.dist(current,j[2]),j[0],j[1])),None
8     n=len(jobs)
9     distance=[[math.dist(a[2],b[2]) for b in jobs] for a in jobs]
10    entry=[math.dist(current,j[2]) for j in jobs]
11    def length(route):
12        return entry[route[0]]+sum(distance[a][b] for a,b in zip(route,route[1:]))
13    candidates=[]
14    for start in range(n):
15        route=[start];left=set(range(n))-{start}
16        while left:
17            nxt=min(left,key=lambda k:(distance[route[-1]][k],k))
18            route.append(nxt);left.remove(nxt)
19        # Strict improvement, finite permutations; iteration is not a tuning budget.
20        while True:
21            cost=length(route);improvement=None
22            for i in range(n-1):
23                for j in range(i+1,n):
24                    alt=route[:i]+list(reversed(route[i:j+1]))+route[j+1:]
25                    value=length(alt)
26                    if value<cost-1e-7 and (improvement is None or (value,alt)<improvement):
27                        improvement=(value,alt)
28            if improvement is None:break
29            route=improvement[1]
30            candidates.append((length(route),route))
31    cost,route=min(candidates)
32    return jobs[route[0]],dict(length_m=cost,tour=[dict(task=jobs[k][0],target=jobs[k][1],point=list(jobs[k][2])) for
↵ k in route])

```

程序 88

对应文件: question3/innovation/ablation/bounded/code_snapshot/run_stress.py; question3/innovation/ablation/pre_round_stress/code_snapshot/run_stress.py; question3/innovation/ablation_expanded/bounded/code_snapshot/run_stress.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/run_stress.py; question3/innovation/expanded_development/bounded/code_snapshot/run_stress.py; question3/innovation/expanded_development/pre_round_stress/code_snapshot/run_stress.py;

question3/innovation/final_validation/bounded/code_snapshot/run_stress.py;
question3/innovation/final_validation/pre_round_stress/code_snapshot/r
un_stress.py; question3/innovation/round02_parallax/pre_round_stress/cod
e_snapshot/run_stress.py; question3/innovation/round03_refresh/pre_round
_stress/code_snapshot/run_stress.py; question3/innovation/round04_share
d/pre_round_stress/code_snapshot/run_stress.py; question3/innovation/rou
nd05_strip/pre_round_stress/code_snapshot/run_stress.py; question3/inno
vation/round06_belief_tour/pre_round_stress/code_snapshot/run_stress.py;
question3/innovation/round07_reception_bracket/pre_round_stress/code_sna
pshot/run_stress.py; question3/innovation/round08_survey/pre_round_stress
/code_snapshot/run_stress.py; question3/innovation/round09_information/pre
_round_stress/code_snapshot/run_stress.py; question3/innovation/round10_c
ontinuous_cover/pre_round_stress/code_snapshot/run_stress.py

```
1 """Run each already-defined mechanism on the identical rounding stress cases."""
2 import subprocess
3 import sys
4 from concurrent.futures import ThreadPoolExecutor
5 from pathlib import Path
6
7 ROOT=Path(__file__).resolve().parent
8 TASKS={
9     'round02_parallax': 'BASE,HOMING,PARALLAX',
10    'round03_refresh': 'BASE,REFRESH,REFRESH_PARALLAX',
11    'round04_shared': 'BASE,REFRESH,REFRESH_PARALLAX,SHARED,SHARED_PARALLAX',
12    'round05_strip': 'BASE,STRIP',
13    'round06_belief_tour': 'BASE,NEAREST_HOMING,TOUR_HOMING,NEAREST_SHARED_PARALLAX,TOUR_SHARED_PARALLAX',
14    'round07_reception_bracket': 'BASE,RECEPTION_BRACKET',
15    'round08_survey': 'BASE,SURVEY',
16    'round09_information': 'BASE,TOUR_SHARED_PARALLAX,GATED_TOUR_SHARED_PARALLAX',
17    'round10_continuous_cover': 'BASE,TOUR_SHARED_PARALLAX,COVER_TOUR_SHARED_PARALLAX',
18 }
19
20 def task(pair):
21     folder,variants=pair
22     with (ROOT/folder/'stress.log').open('w') as f:
23         cmd=[sys.executable,'-m','paper_evidence.q3_mechanism_reset_20260912.experiment',
24             '--round',folder,'--variants',variants,'--rounding','pre_round_stress']
25         subprocess.run(cmd,stdout=f,stderr=subprocess.STDOUT,check=True)
26     print(folder,'finished',flush=True)
27
28 if __name__=='__main__':
29     with ThreadPoolExecutor(max_workers=2) as pool:
30         list(pool.map(task,TASKS.items()))
```

程序 89

对应文件: question3/innovation/ablation/bounded/code_snapshot/strategy.py;
question3/innovation/ablation/pre_round_stress/code_snapshot/strategy.py;
question3/innovation/ablation_expanded/bounded/code_snapshot/strategy.py;
question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/st
rategy.py; question3/innovation/final_validation/bounded/code_snapshot/st
rategy.py; question3/innovation/final_validation/pre_round_stress/code_s
n
apshot/strategy.py; question3/innovation/package_smoke/bounded/code_sna
pshot/strategy.py; question3/innovation/strategy.py; question3/results/disp
atch_20260913/development/code_snapshot/question3/innovation/strategy.py;

question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/strategy.py

```

1 """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2 import math
3 import numpy as np
4 from question3.geometry import stations,station_cover_bound
5 from question3.modules import choose_second
6 from .geometry import initial,wedge,bbox,enclosing,grid,parallax_point,clip,worthwhile
7 from .routing import choose as choose_route
8 from .coverage import covered,cover_radius
9
10 def run(client,config,event=lambda row:None):
11     cover=stations(1125)
12     assert station_cover_bound(1125)<1000
13     first={};polys={};bearings={};measured={};cleared=set();visited=set()
14     negative={c:set() for c in range(1,21)}
15     negative_points={c:set() for c in range(1,21)};absent=set()
16     failure=None;exit_confirmed=False;certificate=None
17     base=config['baseline_config']
18     mode=config.get('mechanism','homing')
19     continuous_cover=mode.startswith('cover_')
20     if continuous_cover:mode=mode[len('cover_'):]
21     information_gate=mode.startswith('gated_')
22     if information_gate:mode=mode[len('gated_'):]
23     route_mode='global' if mode.startswith('tour_') else ('nearest' if mode.startswith('nearest_') else None)
24     if route_mode:mode=mode.split('_')[1]
25     refresh=mode in ('refresh','refresh_parallax','shared','shared_parallax')
26     parallax=mode in ('parallax','refresh_parallax','shared_parallax','multi_parallax')
27     sharing=mode in ('shared','shared_parallax','multi_parallax')
28     survey=mode=='survey'
29     def centre(c):
30         return bbox(polys[c],bearings[c])[2]
31     def update(c,p,response):
32         kind=response['measure_result']
33         if kind=='near':
34             if client.clear(p,c,phase='near')['clear_result']!='success':
35                 raise RuntimeError('Near feedback contradicted by failed clear')
36             cleared.add(c)
37             return
38         if kind!='direction':
39             raise RuntimeError('Certified reception returned no_signal')
40         theta=response['svd_deg']
41         if c not in first:
42             first[c]=(tuple(p),theta)
43             polys[c]=initial(p,theta,base['error_deg'])
44             measured[c]=set()
45         else:
46             polys[c]=wedge(polys[c],p,theta,base['error_deg'])
47             measured[c].add(tuple(p));bearings[c]=theta
48             event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
49     def scan(k):
50         p=cover[k]
51         for c in range(1,21):
52             if not survey and len(set(first)|cleared)==16:
53                 return
54             if c in cleared or c in absent or (c in first and not survey):
55                 continue
56             if survey and len(set(first)|cleared)==16 and c not in first:continue
57             r=client.measure(p,c,phase='search')
58             if r['measure_result']=='no_signal':
59                 negative[c].add(k);negative_points[c].add(tuple(p))
60             else:update(c,p,r)
61         visited.add(k)
62     def localize(c):
63         if mode=='reception_bracket':
64             s,theta=first[c];s=np.asarray(s)
65             t=math.radians(theta);axis=np.array([math.cos(t),math.sin(t)])
66             bounds=[]
67             for sign in (-1,1):
68                 inside=0.;outside=3001.
69                 while outside-inside>10:

```

```

70         mid=(inside+outside)/2;p=s+sign*mid*axis
71         r=client.measure(p,c,phase='reception_range')
72         if r['measure_result']=='near':
73             update(c,p,r);return
74         if r['measure_result']=='no_signal':outside=mid
75         else:inside=mid
76         bounds.append(sorted([sign*inside,sign*outside]))
77     lower=(bounds[0][0]+bounds[1][0])/2
78     upper=(bounds[0][1]+bounds[1][1])/2
79     polys[c]=clip(polys[c],axis,float(axis@upper+1e-7))
80     polys[c]=clip(polys[c],-axis,float(-axis@s-lower+1e-7))
81     event(dict(type='range_bracket',channel=c,first=list(s),axis=axis.tolist(),lower=lower,upper=upper,vertic
↵ es=polys[c].tolist()))
82     for p in grid(polys[c],theta):
83         if client.clear(p,c,phase='bracket_optical')['clear_result']=='success':
84             cleared.add(c);return
85     raise RuntimeError('Reception bracket optical cover exhausted')
86 if mode=='strip':
87     event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
88     for p in grid(polys[c],bearings[c]):
89         if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
90             cleared.add(c);return
91     raise RuntimeError('Single-bearing finite cover exhausted')
92 if refresh and tuple(client.position) not in measured[c]:
93     radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
94     if radius<1000-1e-5:
95         event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↵ s[c].tolist()))
96         update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
97         if c in cleared:return
98 for step in range(16):
99     p,radius=centre(c)
100     if radius<=20-1e-5:
101         event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↵ )
102         if client.clear(p,c,phase='optical')['clear_result']!='success':
103             raise RuntimeError('Certified containing circle clear failed')
104         cleared.add(c);return
105     if parallax and step==0:
106         p,radius=parallax_point(polys[c],bearings[c],client.position)
107     if radius>=1000-1e-5:
108         raise RuntimeError('Adaptive measurement lacks reception certificate')
109     if tuple(p) in measured[c]:
110         break
111     event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
112     update(c,p,client.measure(p,c,phase='adaptive_localization'))
113     if c in cleared:return
114     if sharing:
115         for other in sorted(set(first)-cleared-{c}):
116             if tuple(client.position) in measured[other]:continue
117             rr=float(np.linalg.norm(polys[other]-np.asarray(client.position),axis=1).max()+1e-7)
118             if rr>=1000-1e-5:continue
119             if information_gate and not
↵ worthwhile(polys[other],bearings[other],client.position,base['error_deg']):
120                 event(dict(type='information_gate_skip',channel=other));continue
121                 event(dict(type='certified_measure',channel=other,point=list(client.position),radius=rr,vertices=
↵ polys[other].tolist()))
122                 update(other,client.position,client.measure(client.position,other,phase='multi_target_refinement'
↵ ))
123     event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
124     for p in grid(polys[c],bearings[c]):
125         if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
126             cleared.add(c);return
127     raise RuntimeError('Finite optical cover exhausted without success')
128 client.enter()
129 try:
130     while True:
131         if continuous_cover and len(set(first)|cleared)<16:
132             for c in sorted(set(range(1,21))-set(first)-cleared-absent):
133                 pts=negative_points[c]
134                 if covered(pts):absent.add(c);continue
135                 if tuple(client.position) in pts:continue
136                 augmented=pts|{tuple(client.position)}

```

```

137         if not covered(augmented):continue
138         event(dict(type='coverage_probe',channel=c,point=list(client.position),negative_positions=[list(p)
↵         ) for p in
↵         sorted(pts)],cover_radius=cover_radius(tuple(sorted(augmented))))
139         r=client.measure(client.position,c,phase='coverage_completion')
140         if r['measure_result']=='no_signal':
141             pts.add(tuple(client.position));absent.add(c)
142         else:update(c,client.position,r)
143     known=set(first)|cleared
144     if len(known)>16:raise RuntimeError('Source count upper bound contradicted')
145     pending=sorted(set(first)-cleared)
146     remaining=[] if len(known)==16 and not survey else sorted(set(range(7))-visited)
147     if continuous_cover and known|absent==set(range(1,21)):remaining=[]
148     if not pending and not remaining:break
149     jobs=[]
150     for c in pending:
151         # R1 keeps the baseline's pre-schedule second-point proxy.
152         planning={**base,'second_point_opportunity':'off','second_range_prior':'off'}
153         q,_=choose_second(*first[c],client.position,planning)
154         jobs.append((math.dist(client.position,q),'localize',c))
155     jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]
156     if survey:
157         tasks=[('scan',k,cover[k]) for k in remaining] if remaining else
158             [('localize',c,centre(c)[0]) for c in pending]
159         (task,target,_),tour=choose_route(client.position,tasks,True)
160         event(dict(type='survey_route',route=tour))
161     elif route_mode:
162         tasks=[('localize',c,centre(c)[0]) for c in pending]
163         tasks += [('scan',k,cover[k]) for k in remaining]
164         (task,target,_),tour=choose_route(client.position,tasks,route_mode=='global')
165         event(dict(type='belief_route',mode=route_mode,route=tour))
166     else:
167         _,task,target=min(jobs)
168         event(dict(type='schedule',task=task,target=target))
169         if task=='scan':scan(target)
170         else:localize(target)
171     if len(cleared)==16:
172         certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
173             successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
174     elif continuous_cover and all(c in cleared or c in absent for c in range(1,21)):
175         certificate=dict(method='continuous_disk_union',negative_positions={str(c):[list(p) for p in
↵         sorted(negative_points[c])] for c in absent})
176     elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
177         certificate=dict(method='seven_station_channel_cover')
178     else:raise RuntimeError('Incomplete stopping certificate')
179     if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
180     exit_confirmed=client.exit()['exit_reason']=='user_exit'
181 except (RuntimeError,ValueError,KeyError) as exc:
182     failure=f'{type(exc).__name__}: {exc}'
183 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
184     exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
185     absent_channels=sorted(c for c in negative if len(negative[c])==7),
186     negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
187     station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
188 event(dict(type='strategy_result',**result))
189 return result

```

程序 90

对应文件: question3/innovation/ablation/bounded/code_snapshot/test_adversarial.py; question3/innovation/ablation/pre_round_stress/code_snapshot/test_adversarial.py; question3/innovation/ablation_expanded/bounded/code_snapshot/test_adversarial.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/test_adversarial.py; question3/innovation/final_validation/bounded/code_snapshot/test_adversarial.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/test_adversarial.py

```

1 """Self-only tests for nontrivial geometry, capability boundaries and safe failure."""
2 import ast
3 import json
4 import math
5 import random
6 import unittest
7 from pathlib import Path
8 import numpy as np
9 from .experiment import ROOT,PublicClient
10 from .geometry import initial,wedge,bbox,parallax_point,contains,worthwhile
11 from .coverage import covered,verify_cover_boxes
12 from .strategy import run
13 from question3.interface import Client
14 from question3.simulator import Simulator
15 from question3.cases import random_case,boundary_case,late_source_case,near_case,clustered_case
16 from question3.geometry import stations
17
18 BASE=json.loads((ROOT/'frozen/question3/configs/recommended_omni.json').read_text())
19 CONFIG=dict(baseline_config=BASE,mechanism='gated_tour_shared_parallax')
20
21 class Adversarial(unittest.TestCase):
22     def test_extreme_bounded_geometry(self):
23         rng=random.Random(611)
24         for _ in range(600):
25             theta=rng.uniform(0,360);r=rng.choice([5.0001,999.999,1000.,1499.999,1500.])
26             angle=math.radians(theta);s=np.array([0.,0.]);g=np.array([r*math.cos(angle),r*math.sin(angle)])
27             read=(theta+rng.choice([-1.005,1.005]))%360
28             poly=initial(s,read)
29             for step in range(12):
30                 self.assertTrue(contains(poly,g))
31                 q,R=bbox(poly,read)[:2]
32                 if R<20-1e-5:
33                     self.assertLess(math.dist(q,g),20);break
34                 if step==0:q,R=parallax_point(poly,read,s)
35                 self.assertLess(R,1000)
36                 actual=math.degrees(math.atan2(*(g-q)[::-1]))
37                 read=(actual+rng.choice([-1.005,1.005]))%360
38                 poly=wedge(poly,q,read)
39             else:self.fail('No contraction to terminal set in 12 steps')
40
41     def test_wrap_and_near_boundary(self):
42         for theta in [0.,.005,89.999,180.,270.,359.995]:
43             for sign in [-1,1]:
44                 s=np.array([1125.,0.]);g=np.array([1800.,0.])
45                 actual=math.degrees(math.atan2(*(g-s)[::-1]))
46                 p=initial(s,(actual+sign*1.005)%360)
47                 self.assertTrue(contains(p,g))
48             angle=math.radians(theta)
49             g=np.array([1500*math.cos(angle),1500*math.sin(angle)])
50             self.assertTrue(contains(initial([0,0],theta+1.005),g))
51
52     def test_information_gate_covers_outputs(self):
53         rng=random.Random(911)
54         for _ in range(80):
55             theta=rng.uniform(0,360);poly=initial([0,0],theta);q,R=parallax_point(poly,theta,[0,0])
56             if not worthwhile(poly,theta,q):continue
57             old=bbox(poly,theta)[1]
58             for g in list(poly)+[(a+b)/2 for a,b in zip(poly,np.roll(poly,-1,axis=0))]:
59                 bearing=math.degrees(math.atan2(*(g-q)[::-1]))
60                 for error in (-1.005,0,1.005):
61                     post=wedge(poly,q,bearing+error)
62                     # Audit a valid bound with the new reading orientation; numerical enclosures may rotate.
63                     self.assertLessEqual(bbox(post,bearing+error)[1],max(20,old/2)+1.)
64
65     def test_coverage_certificate_independent(self):
66         pts=stations(1125)
67         self.assertTrue(covered(pts))
68         self.assertGreater(verify_cover_boxes(pts)['covered_boxes'],0)
69         for missing in range(7):self.assertFalse(covered(pts[:missing]+pts[missing+1:]))
70         shifted=list(pts);shifted[-1]=(shifted[-1][0]+.01,shifted[-1][1])
71         self.assertTrue(covered(shifted));verify_cover_boxes(shifted)
72
73     def test_policy_has_no_oracle_or_file_access(self):

```

```

74     forbidden={'open','eval','exec','getattr','setattr','__import__'}
75     for name in ('strategy.py','geometry.py','routing.py'):
76         tree=ast.parse((ROOT/name).read_text())
77         for node in ast.walk(tree):
78             if isinstance(node,ast.Call) and
79                 ↳ isinstance(node.func,ast.Name):self.assertNotIn(node.func.id,forbidden)
80             if isinstance(node,(ast.Import,ast.ImportFrom)):
81                 names=[a.name for a in node.names] if isinstance(node,ast.Import) else [node.module or '']
82                 self.assertFalse(any(any(k in n for k in
83                     ↳ ['simulator','cases','fixtures','pathlib','os','random','experiment']) for n in names))
84             if isinstance(node,ast.Attribute):self.assertNotIn(node.attr,{'transport','_sources','_seed','_error_']
85                 ↳ mode','evaluation','__dict__'})
86             c=PublicClient(Client(Simulator(random_case(0)),'SELF'))
87             self.assertFalse(hasattr(c,'transport'));self.assertFalse(hasattr(c,'evaluation'))
88
89     def test_same_point_error_is_fixed(self):
90         for mode in ['positive','negative','smooth','spatial_hash']:
91             env=Simulator(random_case(0,mode));p=(100.125,-232.5)
92             self.assertEqual(env._bias(p,7),env._bias(p,7))
93             self.assertEqual(env._bias((0.,0.),7),env._bias((-0.,0.),7))
94
95     def test_complete_adversarial_families(self):
96         for case in [boundary_case(),late_source_case(),near_case(),clustered_case()]:
97             env=Simulator(case);result=run(PublicClient(Client(env,'SELF')),CONFIG)
98             self.assertEqual(result['status'],'complete',result['failure'])
99             self.assertEqual(env.evaluation()['remaining_channels'],[])
100             self.assertTrue(result['exit_confirmed'])
101
102     def test_deadline_never_reports_complete(self):
103         env=Simulator(random_case(0),remaining_s=0)
104         result=run(PublicClient(Client(env,'SELF')),CONFIG)
105         self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed'])
106
107     def test_rejected_clear_never_reports_complete(self):
108         class Rejected(Simulator):
109             def request(self,path,payload,timeout=5):
110                 if path=='/clear':return 200,dict(accepted=False,virtual_time_s=0)
111                 return super().request(path,payload,timeout)
112         env=Rejected(random_case(0));result=run(PublicClient(Client(env,'SELF')),CONFIG)
113         self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed'])
114
115     def test_certified_no_signal_never_reports_complete(self):
116         class Corrupted(Simulator):
117             def request(self,path,payload,timeout=5):
118                 status,response=super().request(path,payload,timeout)
119                 if path=='/measure':
120                     p=(payload['position']['x'],payload['position']['y'])
121                     if min(math.dist(p,s) for s in stations(1125))>1e-4:
122                         response['measure_result']='no_signal';response.pop('svd_deg',None)
123                 return status,response
124         env=Corrupted(random_case(0));result=run(PublicClient(Client(env,'SELF')),CONFIG)
125         self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed'])
126
127 if __name__=='__main__':unittest.main(verbosity=2)

```

程序 91

对应文件: question3/innovation/ablation_expanded/bounded/code_snapshot/experiment.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/experiment.py; question3/innovation/experiment.py; question3/innovation/final_validation/bounded/code_snapshot/experiment.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/experiment.py; question3/innovation/package_smoke/bounded/code_snapshot/experiment.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/experiment.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/experiment.py

```

1 """Evaluator-only self experiments. Strategy inputs exclude cases and evaluator state."""
2 import argparse
3 import hashlib
4 import json
5 import math
6 import shutil
7 import statistics
8 import sys
9 import time
10 from pathlib import Path
11 import numpy as np
12
13 ROOT=Path(__file__).resolve().parent
14 sys.path.insert(0,str(ROOT/'frozen'))
15 from question3.cases import development_cases
16 from question3.simulator import Simulator
17 from question3.interface import Client,JsonLog
18 from question3.strategy import run as baseline_run
19 from question3.audit import audit_folder
20 from .strategy import run as candidate_run
21
22 class PublicClient:
23     """Narrow public observation/action capability; no transport/evaluation accessor."""
24     __slots__=('__client',)
25     def __init__(self,client):self.__client=client
26     @property
27     def position(self):return self.__client.position
28     @property
29     def channel(self):return self.__client.channel
30     def enter(self):return self.__client.enter()
31     def measure(self,*a,**kw):return self.__client.measure(*a,**kw)
32     def clear(self,*a,**kw):return self.__client.clear(*a,**kw)
33     def exit(self):return self.__client.exit()
34     def metrics(self):return self.__client.metrics()
35
36 def dump(path,data):
37     path.parent.mkdir(parents=True,exist_ok=True)
38     path.write_text(json.dumps(data,ensure_ascii=False,indent=2))
39
40 def audit_new(folder):
41     case=json.loads((folder/'evaluator_hidden_case.json').read_text())
42     truth={s['channel']:np.array([s['x'],s['y']]) for s in case['sources']}
43     events=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
44     counts={}
45     for e in events:
46         kind=e['type'];counts[kind]=counts.get(kind,0)+1
47         if kind not in ('feasible_update','certified_measure','certified_clear','range_bracket'):
48             continue
49         v=np.asarray(e['vertices']);g=truth[e['channel']]
50         assert len(v)>0 and np.isfinite(v).all()
51         edges=np.roll(v,-1,axis=0)-v;delta=g-v
52         cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
53         assert np.all(cross>=-1e-4) or np.all(cross<=1e-4),('source excluded',e)
54         if kind=='range_bracket':
55             z=(g-np.asarray(e['first']))@np.asarray(e['axis'])
56             assert e['lower']-1e-6<=z<=e['upper']+1e-6
57         if kind in ('certified_measure','certified_clear'):
58             p=np.asarray(e['point']);radius=e['radius']
59             assert np.linalg.norm(v-p,axis=1).max()<=radius+1e-8
60             assert radius<(1000 if kind=='certified_measure' else 20)
61     return counts
62
63 def aggregate(rows,reference='BASE'):
64     groups={n:[r for r in rows if r['variant']==n] for n in sorted({r['variant'] for r in rows})}
65     ref={r['case_id']:r for r in groups[reference]}
66     result={}
67     for n,rs in groups.items():
68         t=[r['penalized_virtual_s'] for r in rs]
69         deltas=[(r['penalized_virtual_s']/ref[r['case_id']])['penalized_virtual_s']-1,r['case_id']) for r in rs]
70         result[n]=dict(runs=len(rs),complete=sum(r['success'] for r in rs),
71             failed=sum(not r['success'] for r in rs),timeouts=sum(r['timeout'] for r in rs),
72             false_complete=sum(r['false_complete'] for r in rs),
73             mean=float(np.mean(t)),p95=float(np.percentile(t,95)),maximum=max(t),

```



```

74     worst_case=rs[int(np.argmax(t))]['case_id'],worst_paired_regression=max(deltas),
75     clearance_ratio_mean=float(np.mean([r['cleared']/r['total'] for r in rs])),
76     raw_mean=float(np.mean([r['metrics']['virtual_time_s'] for r in rs])),
77     movement_detection_etc={k:float(np.mean([r['metrics']['components'][k] for r in rs])) for k in
    ↪ rs[0]['metrics']['components']},
78     phase_means={k:float(np.mean([r['metrics']['phases'].get(k,0) for r in rs])) for k in
    ↪ sorted(set().union([r['metrics']['phases'] for r in rs]))},
79     mean_per_cleared=float(np.mean([r['metrics']['virtual_time_s']/max(1,r['cleared']) for r in rs])),
80     wall_p95=float(np.percentile([r['elapsed_wall_s'] for r in rs],95)),
81     clear_failure_mean=float(np.mean([r['metrics']['counts']['clear_failure'] for r in rs]))
82 for n,v in result.items():
83     v['mean_change_pct']=100*(v['mean']/result[reference]['mean']-1)
84     v['significant']=n!=reference and v['mean_change_pct']<=-5 and v['complete']>=result[reference]['complete']
    ↪ and v['p95']<=result[reference]['p95']
85 return result
86
87 def main():
88     p=argparse.ArgumentParser()
89     p.add_argument('--round',required=True);p.add_argument('--variants',default='BASE,HOMING')
90     p.add_argument('--rounding',default='bounded',choices=['bounded','pre_round_stress'])
91     p.add_argument('--fixtures');p.add_argument('--limit',type=int)
92     p.add_argument('--stage',default='development');p.add_argument('--label')
93     p.add_argument('--lock')
94     args=p.parse_args()
95     rounddir=R00T/args.round
96     out=rounddir/(args.label or args.rounding)
97     out.mkdir(parents=True,exist_ok=False)
98     baseline=json.loads((R00T/'frozen/question3/configs/recommended_omni.json').read_text())
99     names=args.variants.split(',')
100    configs={n:(baseline if n=='BASE' else dict(baseline_config=baseline,mechanism=n.lower())) for n in names}
101    if args.stage=='final_validation' and not args.lock:
102        raise RuntimeError('Final validation requires a pre-generation source/config lock')
103    if args.lock:
104        lock=json.loads(Path(args.lock).read_text())
105        for file,digest in lock['code_sha256'].items():
106            if hashlib.sha256((R00T/file).read_bytes()).hexdigest()!=digest:
107                raise RuntimeError('Locked experimental source changed: '+file)
108        for file,digest in lock['baseline_sha256'].items():
109            if hashlib.sha256((R00T/'frozen'/file).read_bytes()).hexdigest()!=digest:
110                raise RuntimeError('Frozen baseline changed: '+file)
111        for name,cfg in configs.items():
112            if lock['configs'].get(name)!=cfg:raise RuntimeError('Locked config changed: '+name)
113    dump(out/'configs.json',configs)
114    manifest={str(x.relative_to(R00T)):hashlib.sha256(x.read_bytes()).hexdigest() for x in R00T.glob('*.*py')}
115    dump(out/'code_sha256.json',manifest)
116    for file in R00T.glob('*.*py'):
117        dest=out/'code_snapshot'/file.name;dest.parent.mkdir(exist_ok=True);shutil.copy2(file,dest)
118    cases=json.loads(Path(args.fixtures).read_text()) if args.fixtures else list(development_cases())
119    if args.limit:cases=cases[:args.limit]
120    dump(out/'fixtures.json',cases)
121    dump(out/'metadata.json',dict(stage=args.stage,rounding=args.rounding,
122        distinct_layouts=len({json.dumps(c['sources'],sort_keys=True) for c in cases}),
123        case_sha256={c['id']:hashlib.sha256(json.dumps(c,sort_keys=True).encode()).hexdigest() for c in cases},
124        official_runs=0,formal_runs=0,numeric_parameter_search_trials=0))
125    rows=[]
126    for i,case in enumerate(cases):
127        order=names[i%len(names):]+names[:i%len(names)]
128        for name in order:
129            folder=out/name/case['id'];folder.mkdir(parents=True)
130            dump(folder/'evaluator_hidden_case.json',case)
131            log=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
132            env=Simulator(case,rounding=args.rounding)
133            client=Client(env,'SELF',log,5.)
134            start=time.perf_counter()
135            try:
136                result=(baseline_run if name=='BASE' else candidate_run)(PublicClient(client),configs[name],events)
137            except Exception as exc:
138                result=dict(status='incomplete',failure=f'{type(exc).__name__}:
    ↪ {exc}',metrics=client.metrics(),regions=[])
139            wall=time.perf_counter()-start
140            truth=env.evaluation() # First evaluator access, after the policy returned.
141            row=dict(variant=name,case_id=case['id'],stage=args.stage,rounding=args.rounding,
142                status=result['status'],failure=result.get('failure'),elapsed_wall_s=wall,

```

```

143         total=truth['total'],cleared=truth['cleared'],metrics=result['metrics'])
144     row['false_complete']=result['status']=='complete' and bool(truth['remaining_channels'])
145     row['timeout']=bool(result.get('failure')) and any(x in result['failure'].lower() for x in
↳ ('deadline','timeout','budget'))
146     row['success']=result['status']=='complete' and not truth['remaining_channels']
147     dump(folder/'result.json',dict(summary=row,strategy=result))
148     log.close();events.close()
149     try:
150         if row['success']:
151             if name.startswith('COVER_'):
152                 from .coverage_audit import audit as coverage_audit
153                 row['audited_actions']=coverage_audit(folder)
154             else:row['audited_actions']=audit_folder(folder)
155             row['event_counts']=audit_new(folder)
156             row['audit']='pass'
157         else:row['audit']='incomplete retained'
158     except Exception as exc:
159         row['audit']=f'FAIL {type(exc).__name__}: {exc}'
160         row['success']=False;row['false_complete']=True
161         row['penalized_virtual_s']=truth['virtual_time_s'] if row['success'] else
↳ max(360000,truth['virtual_time_s'])
162         dump(folder/'result.json',dict(summary=row,strategy=result))
163         rows.append(row)
164     dump(out/'summary.json',dict(rows=rows,aggregates=aggregate(rows)))
165     print(f'{i+1}/{len(cases)} {case["id"]} '+' ', '.join(f'{r["variant"]]={r["penalized_virtual_s"]:.1f}'+('
↳ '+str(r['failure'])+' '+r['audit'] if not r['success'] else '') for r in rows[-len(names):]),flush=True)
166     stats=aggregate(rows);dump(out/'comparison.json',stats)
167     print(json.dumps({n:{k:v[k] for k in ('complete','runs','mean','mean_change_pct','p95','maximum','significant')}}
↳ for n,v in stats.items()),indent=2,flush=True)
168
169 if __name__=='__main__':main()

```

程序 92

对应文件: question3/innovation/ablation_expanded/bounded/code_snapshot/lock_validation.py; question3/innovation/ablation_expanded/pre_round_stress/code_snapshot/lock_validation.py; question3/innovation/final_validation/bounded/code_snapshot/lock_validation.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/lock_validation.py

```

1 """Select from development only, lock sources, THEN generate the final fixtures."""
2 import datetime
3 import hashlib
4 import json
5 from pathlib import Path
6 from .fixtures import generate
7
8 ROOT=Path(__file__).resolve().parent
9 NAMES=['NEAREST_SHARED_PARALLAX','TOUR_SHARED_PARALLAX','GATED_TOUR_MULTI_PARALLAX','GATED_TOUR_SHARED_PARALLAX','COV_
↳ ER_TOUR_SHARED_PARALLAX']
10 OLD={'NEAREST_SHARED_PARALLAX':'round06_belief_tour','TOUR_SHARED_PARALLAX':'round06_belief_tour',
11      'GATED_TOUR_SHARED_PARALLAX':'round09_information','COVER_TOUR_SHARED_PARALLAX':'round10_continuous_cover',
12      'GATED_TOUR_MULTI_PARALLAX':'ablation'}
13 ABLATIONS=['GATED_TOUR_SHARED','TOUR_REFRESH_PARALLAX','GATED_TOUR_MULTI_PARALLAX','GATED_NEAREST_SHARED_PARALLAX']
14
15 def main():
16     out=ROOT/'validation_lock.json'
17     if out.exists():raise RuntimeError('Refusing to replace existing final-validation lock')
18     scores={};eligible=[]
19     for name in NAMES:
20         batches=[]
21         for folder in [OLD[name],('ablation_expanded' if name=='GATED_TOUR_MULTI_PARALLAX' else
↳ 'expanded_development')]:
22             for rounding in ['bounded','pre_round_stress']:
23                 rows=json.loads((ROOT/folder/rounding/'comparison.json').read_text())
24                 batches.append(dict(batch=f'{folder}/{rounding}',reference=rows['BASE'],candidate=rows[name]))
25     score=sum(b['candidate']['mean']/b['reference']['mean'] for b in batches)/4
26     passed=all(b['candidate']['significant'] for b in batches)

```

```

27     scores[name]=dict(score=score,passed=passed,batches=batches)
28     if passed:eligible.append(name)
29     selected=min(eligible or NAMES,key=lambda n:(scores[n]['score'],NAMES.index(n)))
30     config=json.loads((ROOT/'frozen/question3/configs/recommended_omni.json').read_text())
31     variants=list(dict.fromkeys(['BASE',selected])+[n for n in NAMES+ABLATIONS if n!=selected]))
32     configs={n:(config if n=='BASE' else dict(baseline_config=config,mechanism=n.lower())) for n in variants}
33     runtime=['strategy.py','geometry.py','routing.py','coverage.py','coverage_audit.py','experiment.py','fixtures.py']
    ↪ ,['__init__.py']
34     frozen=json.loads((ROOT/'freeze_manifest.json').read_text()['sha256'])
35     lock=dict(created_utc=datetime.datetime.now(datetime.timezone.utc).isoformat(),selected=selected,
36               eligibility=bool(eligible),selection_scores=scores,configs=configs,
37               code_sha256={p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest() for p in runtime},
38               baseline_sha256=frozen,final_seed_start=910000,final_layout_count=120,
39               outcome_rule='Selected only: per rounding mean <= .95*BASE, success >= BASE, P95 <= BASE. No validation
    ↪ reselection.',
40               bootstrap='20000 paired stratified layout-block resamples; error modes never treated as independent layouts',
41               official_runs=0,formal_runs=0)
42     out.write_text(json.dumps(lock,ensure_ascii=False,indent=2))
43     (ROOT/'best_candidate.json').write_text(json.dumps(configs[selected],indent=2))
44     fixtures=ROOT/'final_validation_cases.json'
45     if fixtures.exists():raise RuntimeError('Final fixtures already exist; cannot claim unseen generation')
46     fixtures.write_text(json.dumps(generate(910000,120,'final'),indent=2))
47     (ROOT/'validation_generation.json').write_text(json.dumps(dict(
48         generated_utc=datetime.datetime.now(datetime.timezone.utc).isoformat(),
49         after_lock_sha256=hashlib.sha256(out.read_bytes()).hexdigest(),
50         fixtures_sha256=hashlib.sha256(fixtures.read_bytes()).hexdigest(),layouts=120,case_error_pairs=480),indent=2))
51     print(json.dumps(dict(selected=selected,scores={n:dict(score=s['score'],passed=s['passed']) for n,s in
    ↪ scores.items()}),indent=2))
52
53 if __name__=='__main__':main()

```

程序 93

对应文件: question3/innovation/analyze_final.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/analyze_final.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/analyze_final.py

```

1 """Post-lock evaluation only: never called by any decision policy."""
2 import csv
3 import hashlib
4 import json
5 from pathlib import Path
6 import numpy as np
7 from .experiment import ROOT,aggregate,dump
8
9 ROUNDS={
10 'round01_homing':('HOMING','BASE','中心直进虽然安全,但径向折返和旧任务评分增加移动;不采用单项。'),
11 'round02_parallax':('PARALLAX','HOMING','视差改善中心逼近,却未单独胜过冻结基线;作为组合因素保留。'),
12 'round03_refresh':('REFRESH_PARALLAX','PARALLAX','原地再观测单项仍失败;最终组合删除这一非必要步骤。'),
13 'round04_shared':('SHARED_PARALLAX','REFRESH_PARALLAX','跨频道持续收缩减少重复定位,但仍需与新任务路线配合。'),
14 'round05_strip':('STRIP','BASE','单方位有限光学条带覆盖很昂贵;完整放弃。'),
15 'round06_belief_tour':('TOUR_SHARED_PARALLAX','SHARED_PARALLAX','首次显著突破来自新信息集合与路线调度共同工作;
    ↪ 最近入口消融表明完整2-opt只贡献一部分。'),
16 'round07_reception_bracket':('RECEPTION_BRACKET','BASE','边界成员查询可测距,但长距离往返压倒信息收益;放弃。'),
17 'round08_survey':('SURVEY','BASE','固定多站信息网络增加搜索检测和聚集案例绕行;放弃。'),
18 'round09_information':('GATED_TOUR_SHARED_PARALLAX','TOUR_SHARED_PARALLAX','相对直接父组合收益不足1%,
    ↪ 不视作又一次5%突破;其有限观测价值证书与温和收益可留在候选。'),
19 'round10_continuous_cover':('COVER_TOUR_SHARED_PARALLAX','TOUR_SHARED_PARALLAX','开发轨迹完全一致,
    ↪ 连续覆盖证书未产生收益;最终候选不包含此机制。'),
20 }
21
22 def comparison(folder,rounding):
23     part='bounded_full' if folder=='round01_homing' and rounding=='bounded' else rounding
24     return json.loads((ROOT/folder/part/'comparison.json').read_text())
25
26 def bootstrap(rows,selected):
27     by={n:{r['case_id']:r for r in rows if r['variant']==n} for n in ['BASE',selected]}
28     keys=sorted(by['BASE']);layouts={}

```

```

29     for key in keys:
30         seed=key.split('_')[-2] if key.endswith(('smooth','positive','negative')) else key.split('_')[-3]
31         # Error mode spatial_hash has an underscore; source seed is reliable from the fixture id.
32         seed=next(word for word in key.split('_') if word.isdigit())
33         layouts.setdefault(seed,[]).append(key)
34     fixtures=json.loads((ROOT/'final_validation_cases.json').read_text())
35     family_by_seed={str(c['seed']):c['family'] for c in fixtures}
36     seeds=sorted(layouts);data={n:np.asarray([[by[n][k]['penalized_virtual_s'] for k in sorted(layouts[s])] for s in
    ↪ seeds]) for n in by}
37     assert all(len(layouts[s])==4 for s in seeds)
38     rng=np.random.default_rng(20260912);indices=[]
39     for family in sorted(set(family_by_seed.values())):
40         ids=np.array([i for i,s in enumerate(seeds) if family_by_seed[s]==family])
41         indices.append(rng.choice(ids,size=(20000,len(ids)),replace=True))
42     idx=np.concatenate(indices,axis=1)
43     means={n:data[n].mean(axis=1)[idx].mean(axis=1) for n in by}
44     change=100*(means[selected]/means['BASE']-1)
45     p95={n:np.percentile(data[n][idx].reshape(20000,-1),95,axis=1) for n in by}
46     return dict(layouts=len(seeds),resamples=20000,mean_change_pct_ci95=np.percentile(change,[2.5,97.5]).tolist(),
47             p95_change_seconds_ci95=np.percentile(p95[selected]-p95['BASE'],[2.5,97.5]).tolist(),
48             bootstrap_fraction_mean_improvement_ge5=float(np.mean(change<=-5)))
49
50 def main():
51     lock=json.loads((ROOT/'validation_lock.json').read_text());selected=lock['selected']
52     evidence=dict(selected=selected,final={},rounds={},scope_audit={})
53     for folder,(variant,parent,reason) in ROUNDS.items():
54         batches={}
55         for rounding in ['bounded','pre_round_stress']:
56             table=comparison(folder,rounding)
57             source=table
58             if parent not in table:
59                 parent_folder='round02_parallax' if parent=='PARALLAX' else 'round04_shared'
60                 source=comparison(parent_folder,rounding)
61             batches[rounding]=dict(candidate=table[variant],baseline=table['BASE'],parent=source[parent],
62                 direct_parent_mean_change_pct=100*(table[variant]['mean']/source[parent]['mean']-1))
63             evidence['rounds'][folder]=dict(variant=variant,parent=parent,reason=reason,batches=batches)
64             lines=[f'# {folder} 完整开发复核', '', reason, '', '| 舍入 | BASE均值 | 候选均值 | 对BASE变化 | 直接父方案变化
    ↪ | BASE P95 | 候选P95 | 候选最大 | 完整/总局 | 失败/超时 | ',
    ↪ '|---|---|---|---|---|---|---|---|---|---|']
65             for r,x in batches.items():
66                 a,b=x['baseline'],x['candidate']
67                 lines.append(f'| {r} | {a["mean"]:.2f} | {b["mean"]:.2f} | {b["mean_change_pct"]:.2f}% |
    ↪ | {x["direct_parent_mean_change_pct"]:.2f}% | {a["p95"]:.2f} | {b["p95"]:.2f} | {b["maximum"]:.2f} |
    ↪ | {b["complete"]}/{b["runs"]} | {b["failed"]}/{b["timeouts"]} | ')
68             lines += [' ', '以上单位为总虚拟秒。所有场景配对、不删除失败。完整配置、源码快照、
    ↪ 动作/反馈/策略日志与每种方案的耗时分解在对应舍入目录; R1主批使用bounded_full, bounded保留最初4例探针.', '',
    ↪ '原理与证明范围见 ../THEORY.md; 本表是开发证据, 不是最终验证。R9和R10继承父组合已超过基线的收益,
    ↪ 不能宣称新增机制再次带来≥5%改善。']
69             (ROOT/folder/'REVIEW.md').write_text('\n'.join(lines)+'\n')
70     all_paired=[]
71     for rounding in ['bounded','pre_round_stress']:
72         path=ROOT/'final_validation'/rounding
73         rows=json.loads((path/'summary.json').read_text())['rows']
74         assert len(rows)==480*8,('Incomplete final matrix',rounding,len(rows))
75         configs=json.loads((path/'configs.json').read_text())
76         for n,c in configs.items():assert c==lock['configs'][n]
77         snap=json.loads((path/'code_sha256.json').read_text())
78         for p,h in lock['code_sha256'].items():assert snap[p]==h
79         meta=json.loads((path/'metadata.json').read_text())
80         fixtures=json.loads((ROOT/'final_validation_cases.json').read_text())
81         for case in fixtures:assert
    ↪ meta['case_sha256'][case['id']]==hashlib.sha256(json.dumps(case,sort_keys=True).encode()).hexdigest()
82     by={n:{r['case_id']:r for r in rows if r['variant']==n} for n in configs}
83     assert all(len(rs)==480 for rs in by.values())
84     family={}
85     for f in sorted({c['family'] for c in fixtures}):
86         ids={c['id'] for c in fixtures if c['family']==f}
87         family[f]=aggregate([r for r in rows if r['case_id'] in ids])
88     pairs=[]
89     for case_id,b in by['BASE'].items():
90         c=by[selected][case_id]
91         pair=dict(rounding=rounding,case_id=case_id,baseline=b['penalized_virtual_s'],candidate=c['penalized_virt
    ↪ ual_s'],

```

```

92         change_s=c['penalized_virtual_s']-b['penalized_virtual_s'],
93         change_pct=100*(c['penalized_virtual_s']/b['penalized_virtual_s']-1),
94         baseline_success=b['success'],candidate_success=c['success'])
95     for k in b['metrics']['components']:pair[k+ '_difference']=c['metrics']['components'][k]-b['metrics']['com
↪ ponents'][k]
96     pairs.append(pair)
97     all_paired.extend(pairs)
98     worst=sorted(pairs,key=lambda p:p['change_pct'],reverse=True)[:10]
99     evidence['final'][rounding]=dict(agggregates=aggregate(rows),families=family,bootstrap=bootstrap(rows,selected
↪ ),
100     worst_regressions=worst,better=sum(p['change_s']<=-1e-6 for p in pairs),
101     worse=sum(p['change_s']>1e-6 for p in pairs),actions=sum(r.get('audited_actions',0) for r in rows),
102     winner_event_counts={k:sum(r.get('event_counts',{}).get(k,0) for r in by[selected].values())
103     for k in sorted(set().union(*(r.get('event_counts',{}) for r in by[selected].values()))))}
104     with (ROOT/'final_paired.csv').open('w') as f:
105         w=csv.DictWriter(f,fieldnames=list(all_paired[0]));w.writeheader();w.writerows(all_paired)
106     frozen=json.loads((ROOT/'freeze_manifest.json').read_text())['sha256']
107     repo=ROOT.parents[1]
108     assert all(hashlib.sha256((repo/p).read_bytes()).hexdigest()==h for p,h in frozen.items())
109     assert all(hashlib.sha256((ROOT/'frozen'/p).read_bytes()).hexdigest()==h for p,h in frozen.items())
110     parity=0
111     for old,new in (('round19_dev_main','bounded_full'),('round19_dev_rounding','pre_round_stress')):
112         original=json.loads((repo/'question3/results/old/summary.json').read_text())['cases']
113         reference={r['case_id']:r for r in original if r['variant']=='BOTH'}
114         fresh=json.loads((ROOT/'round01_homing/new/summary.json').read_text())['rows']
115         for r in fresh:
116             if r['variant']=='BASE':
117                 other=reference[r['case_id']]
118                 assert r['metrics']==other['metrics'] and r['cleared']==other['cleared'] and
↪                 r['status']==other['status']
119                 parity+=1
120     inventory=[]
121     for path in sorted(ROOT.glob('*/*/summary.json')):
122         if 'rows' not in (data:=json.loads(path.read_text())):continue
123         rows=data['rows']
124         inventory.append(dict(path=str(path.parent.relative_to(ROOT)),runs=len(rows),
125         failures=sum(not r['success'] for r in rows),timeouts=sum(r['timeout'] for r in rows),
126         false_completes=sum(r['false_complete'] for r in rows),actions=sum(r.get('audited_actions',0) for r in
↪ rows)))
127     evidence['scope_audit']=dict(frozen_and_recommendation_unchanged=True,old_baseline_exact_matches=parity,
128     total_matrix_runs=sum(r['runs'] for r in inventory),failures=sum(r['failures'] for r in inventory),
129     timeouts=sum(r['timeouts'] for r in inventory),false_completes=sum(r['false_completes'] for r in inventory),
130     audited_actions=sum(r['actions'] for r in inventory),inventory=inventory,official_runs=0,formal_runs=0)
131     evidence['replacement_gate_passed']=all(x['agggregates'][selected]['significant'] for x in
↪ evidence['final'].values())
132     dump(ROOT/'FINAL_EVIDENCE.json',evidence)
133     print(json.dumps({k:v for k,v in evidence.items() if k not in ['rounds','scope_audit','final']},indent=2))
134     for rounding,x in evidence['final'].items():
135         print(rounding,json.dumps(dict(base=x['agggregates']['BASE']['mean'],candidate=x['agggregates'][selected]['mean
↪ '],
136         change=x['agggregates'][selected]['mean_change_pct'],p95=x['agggregates'][selected]['p95'],bootstrap=x['bo
↪ tstrap'],indent=2))
137
138 if __name__=='__main__':main()

```

程序 94

对应文件: question3/innovation/deliver.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/deliver.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/deliver.py

```

1 """Build readable report and static scientific figures from locked experiment results."""
2 import json
3 import math
4 from pathlib import Path
5 import numpy as np
6 import matplotlib
7 matplotlib.use('Agg')

```

```

8 import matplotlib.pyplot as plt
9
10 ROOT=Path(__file__).resolve().parent
11
12 def main():
13     evidence=json.loads((ROOT/'FINAL_EVIDENCE.json').read_text());name=evidence['selected']
14     final=evidence['final'];main=final['bounded'];base=main['aggregates']['BASE'];best=main['aggregates'][name]
15     plt.rcParams.update({'font.family':'DejaVu
↳ Sans','font.size':10,'axes.spines.top':False,'axes.spines.right':False})
16     fig,axes=plt.subplots(1,3,figsize=(15,4.5),layout='constrained')
17     families=list(main['families']);y=np.arange(len(families));height=.35
18     axes[0].barh(y-height/2,[main['families'][f]['BASE']['mean'] for f in families],height,label='Frozen
↳ BOTH',color='#627d98')
19     axes[0].barh(y+height/2,[main['families'][f][name]['mean'] for f in families],height,label='Locked
↳ candidate',color='#008578')
20     axes[0].set_yticks(y,families);axes[0].set_xlabel('Mean total virtual time (s)');axes[0].legend(frameon=False)
21     labels=['Move','Switch','Measure','Optical','Laser'];keys=list(base['movement_detection_etc'])
22     change=[best['movement_detection_etc'][k]-base['movement_detection_etc'][k] for k in keys]
23     axes[1].bar(labels,change,color=['#008578' if v<0 else '#c27335' for v in
↳ change]);axes[1].axhline(0,color='#444',lw=.8)
24     axes[1].set_ylabel('Candidate - baseline (s per run)');axes[1].set_title('Complete cost accounting')
25     rows=json.loads((ROOT/'final_validation/bounded/summary.json').read_text()['rows'])
26     ref={r['case_id']:r['penalized_virtual_s'] for r in rows if r['variant']=='BASE'}
27     chosen=[r for r in rows if r['variant']==name]
28     x=np.array([ref[r['case_id']] for r in chosen]);y=np.array([r['penalized_virtual_s'] for r in chosen])
29     axes[2].scatter(x,y,s=12,alpha=.55,color='#008578');axes[2].plot([0,5500],[0,5500],color='#777',ls='--',lw=1)
30     axes[2].set_xlabel='Frozen BOTH (s)',ylabel='Locked candidate (s)',title='All 480 paired runs
↳ retained',xlim=(0,5500),ylim=(0,5500))
31     fig.suptitle('Unseen validation: 120 layouts, four fixed error fields | bounded rounding',fontsize=13)
32     fig.savefig(ROOT/'validation_overview.png',dpi=180);fig.savefig(ROOT/'validation_overview.svg');plt.close(fig)
33     case_id=main['worst_regressions'][0]['case_id']
34     fig,axes=plt.subplots(1,2,figsize=(11,5),layout='constrained')
35     for ax,variant,title in zip(axes,['BASE',name],['Frozen BOTH','Locked candidate']):
36         folder=ROOT/'final_validation/bounded'/variant/case_id
37         case=json.loads((folder/'evaluator_hidden_case.json').read_text())
38         requests={};path=[(0,0)];scans=[]
39         for row in map(json.loads,(folder/'actions.jsonl').read_text().splitlines()):
40             if row['type']=='request':requests[row['payload']]['request_id']=row
41             if row['type']=='response' and row['response'].get('accepted'):
42                 req=requests[row['request_id']];payload=req['payload']
43                 if 'position' in payload:
44                     p=(payload['position']['x'],payload['position']['y']);path.append(p)
45                     if req['phase']=='search':scans.append(p)
46                     p=np.asarray(path);ax.plot(p[:,0],p[:,1],lw=1,color='#008578',alpha=.8)
47                     ax.scatter([s['x'] for s in case['sources']], [s['y'] for s in
↳ case['sources']],c='#be5546',s=20,marker='x',label='True sources (evaluation only)')
48                     sc=np.asarray(scans);ax.scatter(sc[:,0],sc[:,1],s=30,marker='s',facecolors='none',edgecolors='#627d98',label='
↳ Search
↳ stops')
49                     ax.add_patch(plt.Circle((0,0),1800,fill=False,color='aaa',ls='--'))
50                     result=json.loads((folder/'result.json').read_text()['summary'])
51                     ax.set(aspect='equal',xlim=(-1900,1900),ylim=(-1900,1900),xlabel='x (m)',ylabel='y (m)',title=f'{title}:
↳ {result["metrics"] ["virtual_time_s"]:.2f} s')
52     axes[0].legend(loc='lower left',fontsize=8,frameon=False)
53     fig.suptitle(f'Worst relative regression retained: {case_id}\nHidden locations used only by this post-run
↳ plot',fontsize=11)
54     fig.savefig(ROOT/'worst_case_paths.png',dpi=180);plt.close(fig)
55
56 lines=['# 第三问全向源算法机制重探：最终报告','',
57     '''**结论：找到达到预设显著改进标准的候选，按均值、清除率和P95标准值得作为下一推荐版本。当前推荐BOTH未被覆盖。**
↳ 最终锁定 `GATED_TOUR_MULTI_PARALLAX`；两种舍入下平均完整流程时间分别降低10.27%和10.07%，均完整清除且P95下降。
↳ 它不是逐案例优势：最差单例仍慢93.63%，聚集/窄径向类别没有达到各自5%均值收益。','',
58     '本次没有官方演练、正式测试或云端推送。完成10种机制与有意义的组合/删除消融后，依据充分证据提前收束；
↳ **不是连续10轮失败**。R9不足1%的边际收益和R10零收益不当作新的5%突破，不用继承组合收益重置探索叙述。','',
59     '## 最终未见布局的完整流程对照','',
60     '源码、配置、选择规则和生成器锁定后，生成120个新布局，六类各20个；每布局四种固定误差场、两种舍入。每方案960局，
↳ 但独立布局仅120个。最终矩阵同时含8个已锁定方案，共7680局。','',
61     '| 模型 | 方案 | 均值/秒 | P95/秒 | 最大/秒 | 完整/总局 | 失败 | 超时 | 假完成 |',
62     '|---|---|---|---|---|---|---|---|---|')
63 for rounding,x in final.items():
64     for variant in ['BASE',name]:
65         a=x['aggregates'][variant]

```

```

66         lines.append(f' | {rounding} | {"BOTH基线" if variant=="BASE" else "锁定候选"} | {a["mean"]:.2f} |
        ↪ {a["p95"]:.2f} | {a["maximum"]:.2f} | {a["complete"]}/{a["runs"]} | {a["failed"]} | {a["timeouts"]} |
        ↪ {a["false_complete"]} | ')
67 lines+=['','120个布局按类别分层、以布局为块进行20000次配对bootstrap。均值相对变化的95%区间：主模型-11.35%至-9.14%，
↪ 舍入压力-11.15%至-8.95%。这些区间只描述自建生成器下的采样不确定性，不能保证官方分布的收益。
↪ P95差值95%区间分别为-789.26至-534.00秒、-794.48至-531.29秒。','',
68 '主模型435/480例更快、45例更慢；压力模型433/480例更快、47例更慢。失败惩罚预先固定为max(实际虚拟时间,360000秒)，
↪ 未删除任何失败或慢例；本次性能矩阵没有失败局。','',
69 '![最终验证概览](validation_overview.png)','',
70 '## 创新机制及为什么组合有效','',
71 '旧BOTH主要围绕“保证较大交角的第二点-固定光学覆盖-缓存第二次测量”组织行动。
↪ 新候选维护每个已知频道的完整保守位置集合：向不确定集合靠近，用新地点的有界方位收缩集合；
↪ 首个移动检测增加随不确定尺度变化的适度视差，避免中心直进的径向反复折返。每次实际移动后，
↪ 只对保证接收且所有可能输出均有足够收缩价值的其他频道共享测量。清除必须有整个候选集合落入20米圆的证书。','',
72 '调度把当前集合中心与剩余搜索站放入同一滚动开路，执行首项后根据真实反馈重新规划。R6的最近入口消融已取得大部分收益，
↪ 完整行程只是追加改善；不能把全部收益归功于2-opt。消融还表明，切换任务时额外原地再观测没有稳定贡献，
↪ 因此最终删去。连续覆盖停止虽已实现，但开发轨迹没有变化，最终也不包含它。七站/16个成功清除的可靠停止证书保留。
↪ ','',
73 '这是相对项目原基线的机制变化，不声称文献首创或全局最优。几何命题、
↪ 浮点实现和时间收益分级见[THEORY.md](THEORY.md)。','',
74 '## 耗时分解：主模型平均每局','',
75 '| 项目 | BOTH/秒 | 候选/秒 | 候选 -BOTH/秒 | ','|---|---:|---:|---:|')
76 chinese={'movement_s':'移动','switch_s':'切频','detection_s':'无线电检测','optical_s':'光学定位','laser_s':'激光清」
↪ 除'}
77 for k in base['movement_detection_etc']:
78     a=base['movement_detection_etc'][k];b=best['movement_detection_etc'][k]
79     lines.append(f' | {chinese[k]} | {a:.2f} | {b:.2f} | {b-a:+.2f} | ')
80 lines+=['','移动减少383.25秒，额外检测与切频增加71.90秒，光学减少29.98秒，净省341.33秒。所有动作完整计费；
↪ 候选最终960局失败光学尝试为0，有限网格回退触发0次。题面逐局“总时间/清除数”的平均值，主模型269.71-242.21秒，
↪ 压力模型269.36-242.38秒。阶段总时间另保存在FINAL_EVIDENCE.json的phase_means，阶段与动作分类不得重复相加。','',
81 '## 各类场景与最坏退化','',
82 '| 类别 (每类 20 布局 ×4 误差场) | BOTH 均值/秒 | 候选均值/秒 | 均值变化 | BOTH P95 | 候选 P95 | ',
83 '|---|---:|---:|---:|---:|')
84 for family,z in main['families'].items():
85     a,b=z['BASE'],z[name]
86     lines.append(f' | {family} | {a["mean"]:.2f} | {b["mean"]:.2f} | {b["mean_change_pct"]:+.2f}% | {a["p95"]:.2f}
↪ | {b["p95"]:.2f} | ')
87 lines+=['','最差相对退化`final_cluster_910014_spatial_hash`: 1063.06-2058.36秒，慢995.31秒/+93.63%。
↪ 增加移动791.31秒、检测185秒、切频37秒，虽节省18秒失败光学仍明显变慢。压力批同例仍慢93.63%，另有径向
↪ `910046_positive`: 1343.14-2576.79秒，慢91.85%。聚集区域快速发现16源后提前结束搜索是基线的强项；
↪ 新滚动路线使用集合中心代理，可能先扫更多站，使本可早结束的整局延长。','',
88 '这是模型选择和任务终止时机的局限，不是删除失败光学就能消除。最终验证后没有针对这些案例调参、改机制或换候选。
↪ 候选总体最大值虽更低，但单例相对遗憾仍大，故不能承诺逐场景替代。','',
89 '![最坏退化路径](worst_case_paths.png)','',
90 '## 十轮方法及负结果','',
91 '| 轮次 | 主对照候选 | 原开发均值相对 BOTH | 结论 | ','|---|---:|---:|---:|')
92 for folder,x in evidence['rounds'].items():
93     a=x['batches']['bounded']['candidate']
94     lines.append(f' | [{folder}]{(folder)/REVIEW.md} | {x["variant"]} | {a["mean_change_pct"]:+.2f}% |
↪ {x["reason"]} | ')
95 lines+=['','所有轮次都有事前动机、实现快照、配置、动作与策略原始日志、两种舍入对照和REVIEW.md。R1最初4例探针也保留。
↪ 完整负结果包括单方位光学清扫+92.18%和接收边界测距+525.62%，没有用局部效率替代完整流程结论。','',
96 '## 消融与选择可追溯性','',
97 '初始选择规则见SELECTION.md；开发消融揭示可删除原地再观测后，先记录SELECTION_AMENDMENT_1.md，
↪ 再补同一结构开发测试，最后锁定简化版。最终数据在validation_lock.json写入后才生成，
↪ 哈希与时间见validation_generation.json。最终验证后未改选；8个方案中最优均值也是事前选中的候选。','',
98 '| 最终主模型方案 | 均值/秒 | P95/秒 | 作用 | ','|---|---:|---:|---:|')
99 labels={name:'最终简化候选','GATED_TOUR_SHARED_PARALLAX':'加回被删除的目标原地再观测','TOUR_SHARED_PARALLAX':'完整」
↪ 父组合去信息门控','NEAREST_SHARED_PARALLAX':'完整父组合仅最近入口','GATED_NEAREST_SHARED_PARALLAX':'完整父组合」
↪ 有门控、
↪ 仅最近入口','GATED_TOUR_SHARED':'完整父组合去视差','TOUR_REFRESH_PARALLAX':'完整父组合去跨频道共享','BASE':'冻」
↪ 结BOTH'}
100 for variant,a in main['aggregates'].items():
101     lines.append(f' | {variant} | {a["mean"]:.2f} | {a["p95"]:.2f} | {labels[variant]} | ')
102 lines+=['','消融锚定“带目标再观测的完整父组合”，只有加/删目标再观测一项与最终简化候选是严格单因素对照；
↪ 不能把表中所有差值都当最终候选的独立可加贡献。门控比较TOUR_SHARED_PARALLAX与GATED_TOUR_SHARED_PARALLAX；
↪ 视差比较GATED_TOUR_SHARED与GATED_TOUR_SHARED_PARALLAX；
↪ 全行程比较GATED_NEAREST_SHARED_PARALLAX与GATED_TOUR_SHARED_PARALLAX。共享的严格开发对照在R4；
↪ 最终无共享臂同时没有共享门控，不能独立识别二者贡献。','',
103 '## 对抗复核与验证边界','',

```

```

104     f'性能矩阵共{evidence["scope_audit"]["total_matrix_runs"]}局，全部完整清除，失败/超时/假完成均0；
    ↳ 独立审计{evidence["scope_audit"]["audited_actions"]}条接受动作。
    ↳ 104条冻结BASE结果与原第十九轮BOTH逐项完全一致。120个最终布局与60个结构开发布局种子分离；
    ↳ 四种误差场以及两种舍入不当作独立布局数翻倍。每个候选无数值参数网格搜索，额外简化候选的预算修订已明示，
    ↳ 历史基线19轮累计研发预算无法重新配平。', '',
105     '选中配置通过10项对抗测试：600组极端有界误差几何、角度跨零、信息区间覆盖、独立阴性圆方格证明、
    ↳ 策略无真值/文件访问静态检查、同点误差固定、晚频道/边界/近点/聚集完整流程、现实截止、
    ↳ 拒绝清除和保证点错误no_signal的安全失败。公开响应重放3个困难案例，传给策略的环境不存在源坐标、
    ↳ 半径或误差类别对象；动作及时间完全复现。', '',
106     '另有2局实际本机自建HTTP成功并关闭服务器，端口由系统分配，未连接2026端口或官方程序。
    ↳ 首次自建HTTP在enter前被沙箱禁止创建socket，保留self_http_check.log；获准本机socket后重试成功，
    ↳ 见self_http_check_retry.log。这个基础设施失败不计作已进入场景的策略性能失败。', '',
107     '当前推荐配置、源码和87项冻结材料哈希均未改变。Windows、官方HTTP、官方场景/误差空间结构、
    ↳ 文献新颖性和普遍最坏时间未验证。浮点余量与600组边界测试不是形式化区间算术证明；
    ↳ 总体统计改进不推出所有场景改进。', '',
108     '## 替换建议及复现', '',
109     '**按本次预先约定的均值≥5%、清除率不降、P95不升标准，值得将锁定候选作为下一推荐方案；
    ↳ 保留BOTH回退及上述聚集/径向退化记录。** 若实际更重视逐案例不退化而非本次约定指标，
    ↳ 则证据尚不足以支持无条件替代。按照用户范围，工作区当前推荐文件维持原样，本次只提交独立候选。', '',
110     '入口文件：[候选配置](best_candidate.json)、[源码策略](strategy.py)、[数学依据](THEORY.md)、
    ↳ [全部机器可核对结论](FINAL_EVIDENCE.json)、[960条最终配对数据](final_paired.csv)、
    ↳ [验证锁](validation_lock.json)、[复现说明](REPRODUCE.md)。', '',
111     '停止依据是10类机制、开发与最终验证、关键移除消融和对抗反例已充分支持上述限定结论；
    ↳ 继续为小幅收益堆机制不符合本次任务。未宣称已经穷尽算法空间或取得全局最优。']
112     (ROOT/'FINAL_REPORT.md').write_text('\n'.join(lines)+'\n')
113     (ROOT/'README.md').write_text('# 第三问机制重探证据入口\n\n[最终报告](FINAL_REPORT.md) •
    ↳ [候选配置](best_candidate.json) • [复现说明](REPRODUCE.md) • [数学依据](THEORY.md)\n\n全部工作与推荐代码隔离。
    ↳ 原推荐BOTH冻结于frozen/，新实验在round01_homing至round10_continuous_cover，
    ↳ 结构开发与消融在expanded_development/、ablation/、ablation_expanded/，最终未见验证在final_validation/。
    ↳ 每个运行目录保存源码快照、配置、场景哈希、原始动作/策略日志及结果。\\n\\n禁止把同布局的多个误差场、
    ↳ 两种舍入或重复复现计作独立布局。官方演练、正式测试、云端推送均为0。\\n')
114
115 if __name__=='__main__':main()

```

程序 95

对应文件：question3/innovation/expanded_development/bounded/code_snapshot/strategy.py；question3/innovation/expanded_development/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round02_parallax/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round03_refresh/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round04_shared/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round05_strip/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round06_belief_tour/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round07_reception_bracket/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round08_survey/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round09_information/pre_round_stress/code_snapshot/strategy.py；question3/innovation/round10_continuous_cover/bounded/code_snapshot/strategy.py；question3/innovation/round10_continuous_cover/pre_round_stress/code_snapshot/strategy.py

```

1 """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2 import math
3 import numpy as np
4 from question3.geometry import stations, station_cover_bound
5 from question3.modules import choose_second
6 from .geometry import initial, wedge, bbox, enclosing, grid, parallax_point, clip, worthwhile
7 from .routing import choose as choose_route
8 from .coverage import covered, cover_radius
9
10 def run(client, config, event=lambda row: None):
11     cover=stations(1125)

```



```

12 assert station_cover_bound(1125)<1000
13 first={};polys={};bearings={};measured={};cleared=set();visited=set()
14 negative={c:set() for c in range(1,21)}
15 negative_points={c:set() for c in range(1,21)};absent=set()
16 failure=None;exit_confirmed=False;certificate=None
17 base=config['baseline_config']
18 mode=config.get('mechanism','homing')
19 continuous_cover=mode.startswith('cover_')
20 if continuous_cover:mode=mode[len('cover_'):]
21 information_gate=mode.startswith('gated_')
22 if information_gate:mode=mode[len('gated_'):]
23 route_mode='global' if mode.startswith('tour_') else ('nearest' if mode.startswith('nearest_') else None)
24 if route_mode:mode=mode.split('_',1)[1]
25 refresh=mode in ('refresh','refresh_parallax','shared','shared_parallax')
26 parallax=mode in ('parallax','refresh_parallax','shared_parallax')
27 sharing=mode in ('shared','shared_parallax')
28 survey=mode=='survey'
29 def centre(c):
30     return bbox(polys[c],bearings[c]):2]
31 def update(c,p,response):
32     kind=response['measure_result']
33     if kind=='near':
34         if client.clear(p,c,phase='near')['clear_result']!='success':
35             raise RuntimeError('Near feedback contradicted by failed clear')
36         cleared.add(c)
37         return
38     if kind!='direction':
39         raise RuntimeError('Certified reception returned no_signal')
40     theta=response['svd_deg']
41     if c not in first:
42         first[c]=(tuple(p),theta)
43         polys[c]=initial(p,theta,base['error_deg'])
44         measured[c]=set()
45     else:
46         polys[c]=wedge(polys[c],p,theta,base['error_deg'])
47     measured[c].add(tuple(p));bearings[c]=theta
48     event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
49 def scan(k):
50     p=cover[k]
51     for c in range(1,21):
52         if not survey and len(set(first)|cleared)==16:
53             return
54         if c in cleared or c in absent or (c in first and not survey):
55             continue
56         if survey and len(set(first)|cleared)==16 and c not in first:continue
57         r=client.measure(p,c,phase='search')
58         if r['measure_result']=='no_signal':
59             negative[c].add(k);negative_points[c].add(tuple(p))
60         else:update(c,p,r)
61     visited.add(k)
62 def localize(c):
63     if mode=='reception_bracket':
64         s,theta=first[c];s=np.asarray(s)
65         t=math.radians(theta);axis=np.array([math.cos(t),math.sin(t)])
66         bounds=[]
67         for sign in (-1,1):
68             inside=0.;outside=3001.
69             while outside-inside>10:
70                 mid=(inside+outside)/2;p=s+sign*mid*axis
71                 r=client.measure(p,c,phase='reception_range')
72                 if r['measure_result']=='near':
73                     update(c,p,r);return
74                 if r['measure_result']=='no_signal':outside=mid
75                 else:inside=mid
76             bounds.append(sorted([sign*inside,sign*outside]))
77         lower=(bounds[0][0]+bounds[1][0])/2
78         upper=(bounds[0][1]+bounds[1][1])/2
79         polys[c]=clip(polys[c],axis,float(axis@s+upper+1e-7))
80         polys[c]=clip(polys[c],-axis,float(-axis@s-lower+1e-7))
81         event(dict(type='range_bracket',channel=c,first=list(s),axis=axis.tolist(),lower=lower,upper=upper,vertices=polys[c].tolist()))
82         for p in grid(polys[c],theta):
83             if client.clear(p,c,phase='bracket_optical')['clear_result']=='success':

```

```

84         cleared.add(c);return
85     raise RuntimeError('Reception bracket optical cover exhausted')
86     if mode=='strip':
87         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
88         for p in grid(polys[c],bearings[c]):
89             if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
90                 cleared.add(c);return
91         raise RuntimeError('Single-bearing finite cover exhausted')
92     if refresh and tuple(client.position) not in measured[c]:
93         radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
94         if radius<1000-1e-5:
95             event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↵ s[c].tolist()))
96             update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
97             if c in cleared:return
98     for step in range(16):
99         p,radius=centre(c)
100         if radius<=20-1e-5:
101             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↵ )
102             if client.clear(p,c,phase='optical')['clear_result']!='success':
103                 raise RuntimeError('Certified containing circle clear failed')
104             cleared.add(c);return
105         if parallax and step==0:
106             p,radius=parallax_point(polys[c],bearings[c],client.position)
107         if radius>=1000-1e-5:
108             raise RuntimeError('Adaptive measurement lacks reception certificate')
109         if tuple(p) in measured[c]:
110             break
111         event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
112         update(c,p,client.measure(p,c,phase='adaptive_localization'))
113         if c in cleared:return
114         if sharing:
115             for other in sorted(set(first)-cleared-{c}):
116                 if tuple(client.position) in measured[other]:continue
117                 rr=float(np.linalg.norm(polys[other]-np.asarray(client.position),axis=1).max()+1e-7)
118                 if rr>=1000-1e-5:continue
119                 if information_gate and not
↵ worthwhile(polys[other],bearings[other],client.position,base['error_deg']):
120                     event(dict(type='information_gate_skip',channel=other));continue
121                     event(dict(type='certified_measure',channel=other,point=list(client.position),radius=rr,vertices=
↵ polys[other].tolist()))
122                     update(other,client.position,client.measure(client.position,other,phase='multi_target_refinement'
↵ ))
123         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
124         for p in grid(polys[c],bearings[c]):
125             if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
126                 cleared.add(c);return
127         raise RuntimeError('Finite optical cover exhausted without success')
128     client.enter()
129     try:
130         while True:
131             if continuous_cover and len(set(first)|cleared)<16:
132                 for c in sorted(set(range(1,21))-set(first)-cleared-absent):
133                     pts=negative_points[c]
134                     if covered(pts):absent.add(c);continue
135                     if tuple(client.position) in pts:continue
136                     augmented=pts|{tuple(client.position)}
137                     if not covered(augmented):continue
138                     event(dict(type='coverage_probe',channel=c,point=list(client.position),negative_positions=[list(p)
↵ ) for p in
↵ sorted(pts)],cover_radius=cover_radius(tuple(sorted(augmented))))
139                     r=client.measure(client.position,c,phase='coverage_completion')
140                     if r['measure_result']=='no_signal':
141                         pts.add(tuple(client.position));absent.add(c)
142                     else:update(c,client.position,r)
143             known=set(first)|cleared
144             if len(known)>16:raise RuntimeError('Source count upper bound contradicted')
145             pending=sorted(set(first)-cleared)
146             remaining=[] if len(known)==16 and not survey else sorted(set(range(7))-visited)
147             if continuous_cover and known|absent==set(range(1,21)):remaining=[]
148             if not pending and not remaining:break
149             jobs=[]

```

```

150         for c in pending:
151             # R1 keeps the baseline's pre-schedule second-point proxy.
152             planning={**base,'second_point_opportunity':'off','second_range_prior':'off'}
153             q,_=choose_second(*first[c],client.position,planning)
154             jobs.append((math.dist(client.position,q),'localize',c))
155         jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]
156         if survey:
157             tasks=([( 'scan',k,cover[k]) for k in remaining] if remaining else
158                    [( 'localize',c,centre(c)[0]) for c in pending])
159             (task,target,_),tour=choose_route(client.position,tasks,True)
160             event(dict(type='survey_route',route=tour))
161         elif route_mode:
162             tasks=([( 'localize',c,centre(c)[0]) for c in pending]
163                    [( 'scan',k,cover[k]) for k in remaining])
164             (task,target,_),tour=choose_route(client.position,tasks,route_mode=='global')
165             event(dict(type='belief_route',mode=route_mode,route=tour))
166         else:
167             _,task,target=min(jobs)
168             event(dict(type='schedule',task=task,target=target))
169             if task=='scan':scan(target)
170             else:localize(target)
171         if len(cleared)==16:
172             certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
173                             successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
174         elif continuous_cover and all(c in cleared or c in absent for c in range(1,21)):
175             certificate=dict(method='continuous_disk_union',negative_positions={str(c):[list(p) for p in
176                                     ↪ sorted(negative_points[c])] for c in absent})
177         elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
178             certificate=dict(method='seven_station_channel_cover')
179         else:raise RuntimeError('Incomplete stopping certificate')
180         if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
181         exit_confirmed=client.exit()['exit_reason']=='user_exit'
182     except (RuntimeError,ValueError,KeyError) as exc:
183         failure=f'{type(exc).__name__}: {exc}'
184     result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
185               exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
186               absent_channels=sorted(c for c in negative if len(negative[c])==7),
187               negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
188               station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
189     event(dict(type='strategy_result',**result))
190     return result

```

程序 96

对应文件: question3/innovation/final_validation/bounded/code_snapshot/replay_audit.py; question3/innovation/final_validation/pre_round_stress/code_snapshot/replay_audit.py; question3/innovation/package_smoke/bounded/code_snapshot/replay_audit.py

```

1 """Replay a policy with responses only: no simulated world or hidden case object."""
2 import json
3 import math
4 from pathlib import Path
5 from .experiment import ROOT,PublicClient
6 from .strategy import run
7 from question3.interface import Client
8
9 class TranscriptTransport:
10     def __init__(self,path):
11         requests={};self.steps=[];self.i=0;seen=set()
12         for row in map(json.loads,path.read_text().splitlines()):
13             if row['type']=='request':requests[row['payload']]['request_id']=row
14             if row['type']=='response' and row['response'].get('accepted'):
15                 rid=row['request_id']
16                 if rid not in seen:self.steps.append((requests[rid],row));seen.add(rid)
17     def request(self,path,payload,timeout=5):
18         req,response=self.steps[self.i];self.i+=1
19         assert path==req['path']
20         expected=req['payload']

```

```

21     for key in ('channel','position'):
22         if key in expected:
23             if key=='position':
24                 assert math.dist(list(payload[key].values()),list(expected[key].values()))<1e-7
25             else:assert payload[key]==expected[key]
26     return response['http_status'],dict(response['response'])
27
28 def main():
29     selected=json.loads((ROOT/'validation_lock.json').read_text())['selected']
30     cfg=json.loads((ROOT/'best_candidate.json').read_text())
31     # The ablation matrix contains both possible selected gate variants and adverse cases.
32     cases=['boundary_minR_16','late_channel20_after_ten','clustered_16']
33     rows=[]
34     for case in cases:
35         folder=ROOT/'ablation/bounded'/selected/case
36         transport=TranscriptTransport(folder/'actions.jsonl')
37         result=run(PublicClient(Client(transport,'SELF')),cfg)
38         original=json.loads((folder/'result.json').read_text())['strategy']
39         assert result['status']=='complete' and result['cleared_channels']==original['cleared_channels']
40         assert result['metrics']==original['metrics']
41         assert transport.i==len(transport.steps)
42         rows.append(dict(case=case,matched_actions=transport.i,virtual_time_s=result['metrics']['virtual_time_s'],wor_
↳ ld_object_present=False))
43     (ROOT/'public_transcript_replay.json').write_text(json.dumps(rows,indent=2))
44     print(json.dumps(rows,indent=2))
45
46 if __name__=='__main__':main()

```

程序 97

对应文件: question3/innovation/frozen/question3/lookahead.py; question3/lookahead.py; question3/results/dispatch_20260913/development/code_snaps hot/question3/lookahead.py; question3/results/dispatch_20260913/smoke/co de_snapshot/question3/lookahead.py; question3/results/round17_adverse_ma in/code_snapshot/question3/lookahead.py; question3/results/round17_advers e_rounding/code_snapshot/question3/lookahead.py; question3/results/roun d17_dev_main/code_snapshot/question3/lookahead.py; question3/results/ro und17_dev_rounding/code_snapshot/question3/lookahead.py; question3/resu lts/round18_adverse_main/code_snapshot/question3/lookahead.py; question 3/results/round18_adverse_rounding/code_snapshot/question3/lookahead.py; question3/results/round18_dev_main/code_snapshot/question3/lookahead.py; question3/results/round18_dev_rounding/code_snapshot/question3/lookahead .py; question3/results/round18_holdout_main/code_snapshot/question3/loo kahead.py; question3/results/round18_holdout_rounding/code_snapshot/que stion3/lookahead.py; question3/results/round18_outer_main/code_snapshot /question3/lookahead.py; question3/results/round18_outer_rounding/code_ snapshot/question3/lookahead.py; question3/results/round19_adverse_main /code_snapshot/question3/lookahead.py; question3/results/round19_advers e_rounding/code_snapshot/question3/lookahead.py; question3/results/roun d19_dev_main/code_snapshot/question3/lookahead.py; question3/results/ro und19_dev_rounding/code_snapshot/question3/lookahead.py; question3/resu lts/round19_holdout_main/code_snapshot/question3/lookahead.py; question 3/results/round19_holdout_rounding/code_snapshot/question3/lookahead.py; question3/results/round19_outer_main/code_snapshot/question3/lookahead.py; question3/results/round19_outer_rounding/code_snapshot/question3/lookahe

ad.py

```

1 """Round2: stable numerical ties and one-step terminal-region routing proxy.
2 No truth access. Scenario filtering affects scores only, never safety/stop proofs.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np
7 from .scan_route import remaining_open_path
8
9
10 def stable_min(items,score,identity,tolerance=0.):
11     if not items:return None
12     best=min(score(x) for x in items)
13     return min((x for x in items if score(x)<=best+tolerance),key=identity)
14
15
16 @lru_cache(maxsize=4096)
17 def source_region_samples(first,bearing,negative_positions,error_deg):
18     delta=math.radians(error_deg);angle=math.radians(bearing)
19     r,a=np.meshgrid(np.linspace(5.001,1500,31),angle+np.linspace(-delta,delta,3),indexing='ij')
20     points=np.asarray(first)+np.column_stack([(r*np.cos(a)).ravel(),(r*np.sin(a)).ravel()])
21     valid=np.linalg.norm(points,axis=1)<=1800+1e-7
22     for pos in negative_positions:valid &= np.linalg.norm(points-pos,axis=1)>1000-1e-7
23     filtered=points[valid]
24     fallback=len(filtered)==0
25     if fallback:filtered=points # Coarse samples are not an exact feasibility test.
26     return filtered,fallback
27
28
29 def decide(current,current_channel,cover,remaining,jobs,first,negative,unknown,config,forced=False):
30     """Jobs have (move_distance, channel, certified_q, A_selection).
31     End samples are geometric proxies, not a probability distribution or error bound.
32     """
33     options=config.get('routing',{});tolerance=options.get('tie_tolerance_m',0.)
34     mode=options.get('mode','nearest');weight=float(options.get('lookahead_weight',0.))
35     if mode not in ('nearest','region','point'):raise ValueError('Unknown routing mode')
36     if weight<0 or not math.isfinite(weight):raise ValueError('Invalid lookahead weight')
37     if mode=='nearest':
38         station=stable_min(remaining,lambda k:math.dist(current,cover[k]),lambda k:k,tolerance)
39         route_info={}
40         if config.get('scan_order','nearest')=='remaining_open_path':
41             path,cost=remaining_open_path(current,cover,remaining)
42             station=path[0] if path else None
43             route_info=dict(scan_route_mode='remaining_open_path',scan_route=list(path),scan_route_cost_m=cost)
44         job=stable_min(jobs,lambda j:j[0],lambda j:j[1],tolerance)
45         choose_scan=station is not None and (job is None or forced or
46         ⇨ math.dist(current,cover[station])<=job[0]+tolerance)
47         if choose_scan:return 'scan',station,dict(mode=mode,forced=forced,**route_info)
48         return 'localize',job,dict(mode=mode,forced=False,**route_info)
49     anchors=[('scan',k,cover[k]) for k in remaining]+[('localize',j[1],j[2]) for j in jobs]
50     candidates=[]
51     for k in remaining:
52         other=[p for kind,ident,p in anchors if not (kind=='scan' and ident==k)]
53         after=min((math.dist(cover[k],p) for p in other),default=0.)
54         scan_cost=5*len(unknown)+max(0,len(unknown)-1)+(bool(unknown) and unknown[0]!=current_channel)
55         score=math.dist(current,cover[k])/5+scan_cost+weight*after/5
56         candidates.append(dict(kind='scan',id=k,score_s=float(score),forced=forced))
57     if not forced:
58         for _,c,q,selection in jobs:
59             s,bearing=first[c]
60             points,fallback=source_region_samples(tuple(s),bearing,tuple(tuple(cover[k]) for k in
61             ⇨ sorted(negative[c])),config['error_deg'])
62             if mode=='point':points=points.mean(axis=0,keepdims=True)
63             other=np.array([p for kind,ident,p in anchors if not (kind=='localize' and ident==c)])
64             after=np.linalg.norm(points[:,None,:]-other[None,:,:],axis=2).min(axis=1) if len(other) else
65             ⇨ np.zeros(len(points))
66             transit=np.linalg.norm(points-np.asarray(q),axis=1)+weight*after
67             cells=selection.get('proxy_cells')
68             if cells is None:raise ValueError('Lookahead currently requires A cost metadata')
69             score=math.dist(current,q)/5+5+8.6*cells+float(transit.max())/5
70             candidates.append(dict(kind='localize',id=c,score_s=score,sample_count=len(points),sample_fallback=fallba
71             ⇨ ck))

```

```

68     chosen=stable_min(candidates,lambda x:x['score_s'],lambda x:(x['kind']!='scan',x['id']),tolerance/5)
69     if chosen is None:raise RuntimeError('No routing task available')
70     info=dict(mode=mode,weight=weight,forced=forced,chosen_score_s=chosen['score_s'],candidates=candidates)
71     if chosen['kind']=='scan':return 'scan',chosen['id'],info
72     return 'localize',next(j for j in jobs if j[1]==chosen['id']),info

```

程序 98

对应文件: question3/innovation/frozen/question3/make_holdout.py; question3/make_holdout.py; question3/results/dispatch_20260913/development/code_snapshot/question3/make_holdout.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/make_holdout.py; question3/results/round13_adverse_main/code_snapshot/question3/make_holdout.py; question3/results/round13_adverse_rounding/code_snapshot/question3/make_holdout.py; question3/results/round13_dev_main/code_snapshot/question3/make_holdout.py; question3/results/round13_dev_rounding/code_snapshot/question3/make_holdout.py; question3/results/round13_holdout_main/code_snapshot/question3/make_holdout.py; question3/results/round13_holdout_rounding/code_snapshot/question3/make_holdout.py; question3/results/round13_outer_main/code_snapshot/question3/make_holdout.py; question3/results/round13_outer_rounding/code_snapshot/question3/make_holdout.py; question3/results/round14_adverse_main/code_snapshot/question3/make_holdout.py; question3/results/round14_adverse_rounding/code_snapshot/question3/make_holdout.py; question3/results/round14_dev_main/code_snapshot/question3/make_holdout.py; question3/results/round14_dev_rounding/code_snapshot/question3/make_holdout.py; question3/results/round14_holdout_main/code_snapshot/question3/make_holdout.py; question3/results/round14_holdout_rounding/code_snapshot/question3/make_holdout.py; question3/results/round14_initial_dev_main/code_snapshot/question3/make_holdout.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/make_holdout.py; question3/results/round14_outer_main/code_snapshot/question3/make_holdout.py; question3/results/round14_outer_rounding/code_snapshot/question3/make_holdout.py; question3/results/round17_adverse_main/code_snapshot/question3/make_holdout.py; question3/results/round17_adverse_rounding/code_snapshot/question3/make_holdout.py; question3/results/round17_dev_main/code_snapshot/question3/make_holdout.py; question3/results/round17_dev_rounding/code_snapshot/question3/make_holdout.py; question3/results/round18_adverse_main/code_snapshot/question3/make_holdout.py; question3/results/round18_adverse_rounding/code_snapshot/question3/make_holdout.py; question3/results/round18_dev_main/code_snapshot/question3/make_holdout.py; question3/results/round18_dev_rounding/code_snapshot/question3/make_holdout.py; question3/results/round18_holdout_main/code_snapshot/question3/make_holdout.py; question3/results/round18_holdout_rounding/code_snapshot/question3/make_holdout.py; question3/results/round18_outer_main/code_snapshot/question3/make_holdout.py; question3/results/round18_outer_rounding/code_snapshot/question3/make_holdout.py; question3/results/round19_adverse_main/code_snapshot/question3/make_holdout.py; question3/results/ro

und19_adverse_rounding/code_snapshot/question3/make_holdout.py; question3/results/round19_dev_main/code_snapshot/question3/make_holdout.py; question3/results/round19_dev_rounding/code_snapshot/question3/make_holdout.py; question3/results/round19_holdout_main/code_snapshot/question3/make_holdout.py; question3/results/round19_holdout_rounding/code_snapshot/question3/make_holdout.py; question3/results/round19_outer_main/code_snapshot/question3/make_holdout.py; question3/results/round19_outer_rounding/code_snapshot/question3/make_holdout.py

```

1 """Generate reserved fixtures only after checking a written algorithm/configuration lock.
2 Evaluator-only fixture generator; strategy never imports this module.
3 """
4 import argparse
5 import hashlib
6 import json
7 from pathlib import Path
8 from .cases import random_case
9
10
11 def main():
12     p=argparse.ArgumentParser();p.add_argument('--lock',required=True);p.add_argument('--output',required=True)
13     args=p.parse_args();lock=json.loads(Path(args.lock).read_text());out=Path(args.output)
14     if out.exists():raise RuntimeError('Do not overwrite validation fixtures')
15     for path,digest in lock['code_sha256'].items():
16         if hashlib.sha256(Path(path).read_bytes()).hexdigest()!=digest:raise RuntimeError('Locked code changed:
17             ↳ '+path)
18     cases=[]
19     for seed in range(lock['seed_start'],lock['seed_stop_exclusive']):
20         for mode in lock['error_modes']:
21             case=random_case(seed,mode);case['id']=case['id'].replace('dev_', 'holdout_',1);cases.append(case)
22     out.parent.mkdir(parents=True,exist_ok=True);out.write_text(json.dumps(cases,indent=2),encoding='utf-8')
23     print('Generated',len(cases),'held-out configurations after lock;
24         ↳ SHA256',hashlib.sha256(out.read_bytes()).hexdigest())
25
26 if __name__=='__main__':main()

```

程序 99

对应文件: question3/innovation/frozen/question3/modules.py; question3/modules.py; question3/results/dispatch_20260913/development/code_snapshot/question3/modules.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/modules.py; question3/results/round18_adverse_main/code_snapshot/question3/modules.py; question3/results/round18_adverse_rounding/code_snapshot/question3/modules.py; question3/results/round18_dev_main/code_snapshot/question3/modules.py; question3/results/round18_dev_rounding/code_snapshot/question3/modules.py; question3/results/round18_holdout_main/code_snapshot/question3/modules.py; question3/results/round18_holdout_rounding/code_snapshot/question3/modules.py; question3/results/round18_outer_main/code_snapshot/question3/modules.py; question3/results/round18_outer_rounding/code_snapshot/question3/modules.py; question3/results/round19_adverse_main/code_snapshot/question3/modules.py; question3/results/round19_adverse_rounding/code_snapshot/question3/modules.py; question3/results/round19_dev_main/code_snapshot/question3/modules.py; question3/results/round19_dev_rounding/code_snapshot/question3/modules.py; question3/results/round19_holdout_main/code_snapshot/question3/modules.py;

question3/results/round19_holdout_rounding/code_snapshot/question3/modules.py; question3/results/round19_outer_main/code_snapshot/question3/modules.py; question3/results/round19_outer_rounding/code_snapshot/question3/modules.py

```

1 """Optional public-information modules A (local cost), B (route via strategy), C (evidence).
2 No simulator/case imports. All design constants fixed before round-one comparison.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np
7 from question2.strategy import candidate, diameters, to_world
8 from question1.solve import bearing_halfplanes
9 from .geometry import second_point
10 from .current_point import local_offset,current_proxy
11
12 OFFSETS=((400,500),(500,400),(500,500),(600,400),(700,450),(750,400),(800,605),(850,450))
13
14 @lru_cache(maxsize=8)
15 def design_catalog(error_deg=1.005, min_angle=20.):
16     delta=math.radians(error_deg)
17     r,alpha,error=np.meshgrid(np.linspace(5.001,1500,31),np.linspace(-delta,delta,3),
18                               np.linspace(-delta,delta,3),indexing='ij')
19     g=np.column_stack([(r*np.cos(alpha)).ravel(),(r*np.sin(alpha)).ravel()]);error=error.ravel()
20     catalog=[]
21     for a,b in OFFSETS:
22         if not candidate([a,b],min_angle,error_deg):continue
23         d=float(diameters([a,b],g,error,error_deg).max())
24         h=a*math.cos(delta)-b*math.sin(delta)
25         max_range=math.sqrt(max(a*a+b*b+25-10*h,a*a+b*b+1500**2-3000*h))
26         # Surrogate only: diagonal-based box count can overcount or undercount.
27         # Never used as a coverage/feasibility certificate or a probabilistic expectation.
28         cells=math.ceil(d/28)*math.ceil(2*1500*math.sin(delta)/28)
29         continuation=max_range/5+(3+28/5)*cells
30         for sign in [1,-1]:
31             catalog.append(dict(offset=(a,sign*b),sampled_worst_diameter=d,
32                                max_source_distance=max_range,proxy_cells=cells,continuation_proxy_s=continuation))
33     return tuple(catalog)
34
35
36 def choose_second(first,bearing,current,config):
37     if config.get('second_range_prior','off')=='domain':
38         from .domain_range import range_upper,choose
39         upper=range_upper(first,bearing,config['error_deg'])
40         if upper<1500:return
41         ↪ choose(first,bearing,current,config,design_catalog(config['error_deg'],config['min_angle_deg']),upper)
42     if not config.get('modules',{}).get('A',False):
43         q=second_point(first,bearing,config['second_offset'],config['error_deg'],config['min_angle_deg'])
44         return q,dict(mode='fixed',offset=config['second_offset'])
45     best=None
46     for row in design_catalog(config['error_deg'],config['min_angle_deg']):
47         q=tuple(to_world(row['offset'],first,bearing))
48         score=math.dist(current,q)/5+5+row['continuation_proxy_s']
49         key=(score,row['offset'])
50         if best is None or key<best[0]:best=(key,q,row)
51     if best is None:raise RuntimeError('No certified second-point candidates')
52     opportunity=config.get('second_point_opportunity','off')
53     if opportunity!='off':
54         offset=local_offset(first,bearing,current)
55         row=current_proxy(offset,config['error_deg'],config['min_angle_deg'])
56         if row is not None:
57             score=5+row['continuation_proxy_s']
58             if opportunity=='current_if_certified' or score<best[0][0]-1e-8:
59                 return tuple(current),dict(mode='opportunistic_current',score_s=score,**row,
60                                             min_angle_deg=config['min_angle_deg'],error_deg=config['error_deg'],
61                                             selection_rule=opportunity,catalog_score_s=best[0][0],catalog_point=best[1],
62                                             catalog_entry_distance_m=math.dist(current,best[1]),
63                                             catalog_sampled_diameter=best[2]['sampled_worst_diameter'])
64     return best[1],dict(mode='cost_proxy',score_s=best[0][0],**best[2])
65

```



```

66 class EvidenceCover:
67     """Finite covering debt. Fine cells are excluded only by whole-cell proofs.
68
69     Every fine cell has its original coarse-grid centre within 20m of all corners.
70     Failures remove cells fully inside failed-clear disks; they never infer absence
71     from a bare centre test. Point selection uses covered geometric area per cost,
72     not a probability of source presence. No fine-grid point estimate of truth.
73     """
74     def __init__(self, points, certificate, first, bearing, second, reading, negative_positions, error_deg, subdiv=4):
75         v=np.asarray(certificate['vertices']);origin=np.asarray(first)
76         angle=math.radians(bearing);rot=np.array([[math.cos(angle),-math.sin(angle)],[math.sin(angle),math.cos(angle)]]
77             ↪ ))
78         local=(v-origin)@rot;lo=local.min(axis=0)-1e-5;hi=local.max(axis=0)+1e-5
79         sizes=np.maximum(1,np.ceil((hi-lo)/28).astype(int));fine=sizes*subdiv
80         corners=[]
81         for j in range(fine[1]):
82             for i in range(fine[0]):
83                 cell=lo+(np.array([[i,j],[i+1,j],[i+1,j+1],[i,j+1]])/fine)*(hi-lo)
84                 corners.append(origin+cell@rot.T)
85         self.corners=np.array(corners);self.points=np.asarray(points);self.alive=np.ones(len(corners),bool)
86         self.untried=np.ones(len(points),bool);self.initial_count=len(corners);self.prior_dropped=0;self.failed_dropp
87             ↪ ed=0
88         A,b=bearing_halfplanes([dict(x=first[0],y=first[1],bearing_deg=bearing),
89             ↪ dict(x=second[0],y=second[1],bearing_deg=reading)],error_deg)
90         projections=np.einsum('kd,nvd->nkv',A,self.corners)
91         self.alive &= ~np.any(projections.min(axis=2)>b[None,:]+1e-7,axis=1)
92         centres=self.corners.mean(axis=1);radii=np.linalg.norm(self.corners-centres[:,None,:],axis=2).max(axis=1)
93         for centre,radius in [((0,0),1800),(first,1500),(second,1500)]:
94             self.alive &= np.linalg.norm(centres-centre,axis=1)-radii <= radius+1e-7
95         for p in negative_positions:
96             self.alive &= np.linalg.norm(self.corners-np.asarray(p),axis=2).max(axis=1)>1000-1e-7
97         self.prior_dropped=int(self.initial_count-self.alive.sum())
98         self.covers=np.linalg.norm(self.points[:,None,None,:]-self.corners[None,:, :, :],axis=3).max(axis=2)<=20-1e-7
99         if not np.all(self.covers.any(axis=0)):raise RuntimeError('Fine cells lack original optical coverage')
100         if not self.alive.any():raise RuntimeError('All source cells excluded by prior evidence')
101     def next_point(self,current):
102         if not self.alive.any():raise RuntimeError('All feasible cells removed without success')
103         gain=self.covers[:,self.alive].sum(axis=1)
104         cost=3+np.linalg.norm(self.points-np.asarray(current),axis=1)/5
105         scores=np.where(self.untried & (gain>0),gain/cost,-1)
106         k=int(np.argmax(scores))
107         if scores[k]<0:raise RuntimeError('Optical covering debt cannot be satisfied')
108         self.untried[k]=False
109         return k,tuple(self.points[k]),dict(remaining_cells=int(self.alive.sum()),fully_covered_cells=int(gain[k]))
110     def failed(self,k):
111         remove=self.alive & self.covers[k];self.failed_dropped+=int(remove.sum());self.alive[remove]=False
112     def metrics(self):return dict(fine_cells=self.initial_count,prior_excluded_cells=self.prior_dropped,
113             ↪ failed_clear_excluded_cells=self.failed_dropped,remaining_cells=int(self.alive.sum(
114             ↪ )))

```

程序 100

对应文件: question3/innovation/frozen/question3/optical_order.py; question3/optical_order.py; question3/results/dispatch_20260913/development/code_snapshot/question3/optical_order.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/optical_order.py; question3/results/round13_adverse_main/code_snapshot/question3/optical_order.py; question3/results/round13_adverse_rounding/code_snapshot/question3/optical_order.py; question3/results/round13_dev_main/code_snapshot/question3/optical_order.py; question3/results/round13_dev_rounding/code_snapshot/question3/optical_order.py; question3/results/round13_holdout_main/code_snapshot/question3/optical_order.py; question3/results/round13_holdout_rounding/code_snapshot/question3/optical_order.py; question3/results/round13_outer_main/code_snapshot/question3/optical_order.py; question3/results/round13_outer_rounding/code_snapshot/question3/optical_order.py

unding/code_snapshot/question3/optical_order.py; question3/results/round14_adverse_main/code_snapshot/question3/optical_order.py; question3/results/round14_adverse_rounding/code_snapshot/question3/optical_order.py; question3/results/round14_dev_main/code_snapshot/question3/optical_order.py; question3/results/round14_dev_rounding/code_snapshot/question3/optical_order.py; question3/results/round14_holdout_main/code_snapshot/question3/optical_order.py; question3/results/round14_holdout_rounding/code_snapshot/question3/optical_order.py; question3/results/round14_initial_dev_main/code_snapshot/question3/optical_order.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/optical_order.py; question3/results/round14_outer_main/code_snapshot/question3/optical_order.py; question3/results/round14_outer_rounding/code_snapshot/question3/optical_order.py; question3/results/round17_adverse_main/code_snapshot/question3/optical_order.py; question3/results/round17_adverse_rounding/code_snapshot/question3/optical_order.py; question3/results/round17_dev_main/code_snapshot/question3/optical_order.py; question3/results/round17_dev_rounding/code_snapshot/question3/optical_order.py; question3/results/round18_adverse_main/code_snapshot/question3/optical_order.py; question3/results/round18_adverse_rounding/code_snapshot/question3/optical_order.py; question3/results/round18_dev_main/code_snapshot/question3/optical_order.py; question3/results/round18_dev_rounding/code_snapshot/question3/optical_order.py; question3/results/round18_holdout_main/code_snapshot/question3/optical_order.py; question3/results/round18_holdout_rounding/code_snapshot/question3/optical_order.py; question3/results/round18_outer_main/code_snapshot/question3/optical_order.py; question3/results/round18_outer_rounding/code_snapshot/question3/optical_order.py; question3/results/round19_adverse_main/code_snapshot/question3/optical_order.py; question3/results/round19_adverse_rounding/code_snapshot/question3/optical_order.py; question3/results/round19_dev_main/code_snapshot/question3/optical_order.py; question3/results/round19_dev_rounding/code_snapshot/question3/optical_order.py; question3/results/round19_holdout_main/code_snapshot/question3/optical_order.py; question3/results/round19_holdout_rounding/code_snapshot/question3/optical_order.py; question3/results/round19_outer_main/code_snapshot/question3/optical_order.py; question3/results/round19_outer_rounding/code_snapshot/question3/optical_order.py

```

1 """Choose the nearer end of a fixed optical path, preserving its covering set."""
2 import math
3
4
5 def nearer_end(points,current):
6     if math.dist(current,points[-1])+1e-8<math.dist(current,points[0]):
7         return list(reversed(points))
8     return points

```

程序 101

对应文件: question3/innovation/frozen/question3/optical_shortcut.py; question3/optical_shortcut.py; question3/results/dispatch_20260913/development/code_snapshot/question3/optical_shortcut.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/optical_shortcut.py; question3/results/round13_adverse_main/code_snapshot/question3/optical_shortcut.py; question3/results/round13_adverse_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round13_dev_main/code_snapshot/question3/optical_shortcut.py; question3/results/round13_dev_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round13_holdout_main/code_snapshot/question3/optical_shortcut.py; question3/results/round13_holdout_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round13_outer_main/code_snapshot/question3/optical_shortcut.py; question3/results/round13_outer_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round14_adverse_main/code_snapshot/question3/optical_shortcut.py; question3/results/round14_adverse_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round14_dev_main/code_snapshot/question3/optical_shortcut.py; question3/results/round14_dev_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round14_holdout_main/code_snapshot/question3/optical_shortcut.py; question3/results/round14_holdout_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round14_initial_dev_main/code_snapshot/question3/optical_shortcut.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round14_outer_main/code_snapshot/question3/optical_shortcut.py; question3/results/round14_outer_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round17_adverse_main/code_snapshot/question3/optical_shortcut.py; question3/results/round17_adverse_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round17_dev_main/code_snapshot/question3/optical_shortcut.py; question3/results/round17_dev_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round18_adverse_main/code_snapshot/question3/optical_shortcut.py; question3/results/round18_adverse_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round18_dev_main/code_snapshot/question3/optical_shortcut.py; question3/results/round18_dev_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round18_holdout_main/code_snapshot/question3/optical_shortcut.py; question3/results/round18_holdout_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round18_outer_main/code_snapshot/question3/optical_shortcut.py; question3/results/round18_outer_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round19_adverse_main/code_snapshot/question3/optical_shortcut.py; question3/results/round19_adverse_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round19_dev_main/code_snapshot/question3/optical_shortcut.py;

n3/optical_shortcut.py; question3/results/round19_dev_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round19_holdout_main/code_snapshot/question3/optical_shortcut.py; question3/results/round19_holdout_rounding/code_snapshot/question3/optical_shortcut.py; question3/results/round19_outer_main/code_snapshot/question3/optical_shortcut.py; question3/results/round19_outer_rounding/code_snapshot/question3/optical_shortcut.py

```

1 """Optional D: one certified clear for a convex bearing-intersection polygon.
2 Enumerated centres are proposals; safety comes from rechecking EVERY vertex.
3 """
4 import itertools
5 import numpy as np
6
7
8 def single_circle(vertices,margin=1e-5):
9     v=np.asarray(vertices,dtype=float)
10    if v.ndim!=2 or v.shape[1]!=2 or not len(v) or not np.isfinite(v).all():
11        raise ValueError('Invalid convex polygon vertices')
12    centres=list(v)
13    centres.extend((a+b)/2 for a,b in itertools.combinations(v,2))
14    for a,b,c in itertools.combinations(v,3):
15        A=2*np.stack([b-a,c-a]);rhs=np.array([np.dot(b-a,b-a),np.dot(c-a,c-a)])
16        if abs(np.linalg.det(A))<=1e-10:continue
17        try:centres.append(a+np.linalg.solve(A,rhs))
18        except np.linalg.LinAlgError:continue
19    candidates=[(float(np.linalg.norm(v-p,axis=1).max()),tuple(map(float,p))) for p in centres if
    ↪ np.isfinite(p).all()]
20    radius,centre=min(candidates)
21    # A disk is convex: containing all vertices contains their entire convex hull.
22    return (centre,radius) if radius<=20*margin else None

```

程序 102

对应文件: question3/innovation/frozen/question3/order_routing.py; question3/order_routing.py; question3/results/dispatch_20260913/development/code_snapshot/question3/order_routing.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/order_routing.py; question3/results/round13_adverse_main/code_snapshot/question3/order_routing.py; question3/results/round13_adverse_rounding/code_snapshot/question3/order_routing.py; question3/results/round13_dev_main/code_snapshot/question3/order_routing.py; question3/results/round13_dev_rounding/code_snapshot/question3/order_routing.py; question3/results/round13_holdout_main/code_snapshot/question3/order_routing.py; question3/results/round13_holdout_rounding/code_snapshot/question3/order_routing.py; question3/results/round13_outer_main/code_snapshot/question3/order_routing.py; question3/results/round13_outer_rounding/code_snapshot/question3/order_routing.py; question3/results/round14_adverse_main/code_snapshot/question3/order_routing.py; question3/results/round14_adverse_rounding/code_snapshot/question3/order_routing.py; question3/results/round14_dev_main/code_snapshot/question3/order_routing.py; question3/results/round14_dev_rounding/code_snapshot/question3/order_routing.py; question3/results/round14_holdout_main/code_snapshot/question3/order_routing.py; question3/results/round14_holdout_rounding/code_snapshot/question3/order_routing.py; question3/results/round14_initial_dev_main/c

```

ode_snapshot/question3/order_routing.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/order_routing.py; question3/results/round14_outer_main/code_snapshot/question3/order_routing.py; question3/results/round14_outer_rounding/code_snapshot/question3/order_routing.py; question3/results/round17_adverse_main/code_snapshot/question3/order_routing.py; question3/results/round17_adverse_rounding/code_snapshot/question3/order_routing.py; question3/results/round17_dev_main/code_snapshot/question3/order_routing.py; question3/results/round17_dev_rounding/code_snapshot/question3/order_routing.py; question3/results/round18_adverse_main/code_snapshot/question3/order_routing.py; question3/results/round18_adverse_rounding/code_snapshot/question3/order_routing.py; question3/results/round18_dev_main/code_snapshot/question3/order_routing.py; question3/results/round18_dev_rounding/code_snapshot/question3/order_routing.py; question3/results/round18_holdout_main/code_snapshot/question3/order_routing.py; question3/results/round18_holdout_rounding/code_snapshot/question3/order_routing.py; question3/results/round18_outer_main/code_snapshot/question3/order_routing.py; question3/results/round18_outer_rounding/code_snapshot/question3/order_routing.py; question3/results/round19_adverse_main/code_snapshot/question3/order_routing.py; question3/results/round19_adverse_rounding/code_snapshot/question3/order_routing.py; question3/results/round19_dev_main/code_snapshot/question3/order_routing.py; question3/results/round19_dev_rounding/code_snapshot/question3/order_routing.py; question3/results/round19_holdout_main/code_snapshot/question3/order_routing.py; question3/results/round19_holdout_rounding/code_snapshot/question3/order_routing.py; question3/results/round19_outer_main/code_snapshot/question3/order_routing.py; question3/results/round19_outer_rounding/code_snapshot/question3/order_routing.py

```

```

1 """0: same station+job task set, paired regret of incremental movement.
2 Public geometric samples are ranking proxies, never feasibility/stop certificates.
3 """
4 import math
5 import numpy as np
6 from .lookahead import decide,stable_min,source_region_samples
7
8
9 def incremental_regrets(current,station,entry,terminals,anchor=None):
10     terminals=np.asarray(terminals)
11     # Difference = scan-first minus job-first; the same job's internal costs cancel.
12     difference=math.dist(current,station)+math.dist(station,entry)-math.dist(current,entry)
13     difference=difference-np.linalg.norm(terminals-np.asarray(station),axis=1)
14     if anchor is not None:
15         difference=difference+np.linalg.norm(terminals-np.asarray(anchor),axis=1)-math.dist(station,anchor)
16     difference=difference/5
17     return max(0.,float((-difference).max())),max(0.,float(difference.max())),difference
18
19
20 def decide_order(current,current_channel,cover,remaining,jobs,first,negative,unknown,config,cached,forced=False):
21     base=decide(current,current_channel,cover,remaining,jobs,first,negative,unknown,config,forced)
22     if forced or not remaining or not jobs:return base
23     k=stable_min(remaining,lambda k:math.dist(current,cover[k]),lambda k:k)
24     job=stable_min(jobs,lambda j:j[0],lambda j:j[1]);_c,q,selection=job
25     anchors=[cover[i] for i in remaining if i!=k]+[j[2] for j in jobs if j[1]!=c]
26     anchor=min(anchors,key=lambda p:math.dist(current,p)) if anchors else None
27     if c in cached:

```

```

28     # Actual possible clearing stops; the same fixed optical prefix cancels.
29     terminals=np.asarray(cached[c][0]);fallback=False;terminal_kind='cached_optical_points'
30     else:
31         s,bearing=first[c]
32         terminals,fallback=source_region_samples(tuple(s),bearing,tuple(tuple(cover[i]) for i in
33             ↪ sorted(negative[c])),config['error_deg'])
34         terminal_kind='first_bearing_samples'
35     regret_job,regret_scan,difference=incremental_regrets(current,cover[k],q,terminals,anchor)
36     # Respect the old decision on numeric ties, rather than add another tie policy.
37     task=('localize' if regret_job<regret_scan else 'scan') if abs(regret_job-regret_scan)>1e-8 else base[0]
38     target=job if task=='localize' else k
39     return task,target,dict(mode='same_task_order',forced=False,station_id_compared=k,channel_compared=c,
40         anchor=anchor,job_first_regret_s=regret_job,scan_first_regret_s=regret_scan,
41         scan_minus_job_range_s=[float(difference.min()),float(difference.max())],
42         sample_count=len(terminals),sample_fallback=fallback,terminal_kind=terminal_kind,
43         original_task=base[0],order_changed=task!=base[0])

```

程序 103

对应文件: question3/innovation/frozen/question3/ring_phase.py; question3/results/dispatch_20260913/development/code_snapshot/question3/ring_phase.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/ring_phase.py; question3/results/round14_adverse_main/code_snapshot/question3/ring_phase.py; question3/results/round14_adverse_rounding/code_snapshot/question3/ring_phase.py; question3/results/round14_dev_main/code_snapshot/question3/ring_phase.py; question3/results/round14_dev_rounding/code_snapshot/question3/ring_phase.py; question3/results/round14_holdout_main/code_snapshot/question3/ring_phase.py; question3/results/round14_holdout_rounding/code_snapshot/question3/ring_phase.py; question3/results/round14_initial_dev_main/code_snapshot/question3/ring_phase.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/ring_phase.py; question3/results/round14_outer_main/code_snapshot/question3/ring_phase.py; question3/results/round14_outer_rounding/code_snapshot/question3/ring_phase.py; question3/results/round17_adverse_main/code_snapshot/question3/ring_phase.py; question3/results/round17_adverse_rounding/code_snapshot/question3/ring_phase.py; question3/results/round17_dev_main/code_snapshot/question3/ring_phase.py; question3/results/round17_dev_rounding/code_snapshot/question3/ring_phase.py; question3/results/round18_adverse_main/code_snapshot/question3/ring_phase.py; question3/results/round18_adverse_rounding/code_snapshot/question3/ring_phase.py; question3/results/round18_dev_main/code_snapshot/question3/ring_phase.py; question3/results/round18_dev_rounding/code_snapshot/question3/ring_phase.py; question3/results/round18_holdout_main/code_snapshot/question3/ring_phase.py; question3/results/round18_holdout_rounding/code_snapshot/question3/ring_phase.py; question3/results/round18_outer_main/code_snapshot/question3/ring_phase.py; question3/results/round18_outer_rounding/code_snapshot/question3/ring_phase.py; question3/results/round19_adverse_main/code_snapshot/question3/ring_phase.py; question3/results/round19_adverse_rounding/code_snapshot/question3/ring_phase.py; question3/results/round19_dev_main/code_snapshot/question3/ring_phase.py;

question3/results/round19_dev_rounding/code_snapshot/question3/ring_phase.py; question3/results/round19_holdout_main/code_snapshot/question3/ring_phase.py; question3/results/round19_holdout_rounding/code_snapshot/question3/ring_phase.py; question3/results/round19_outer_main/code_snapshot/question3/ring_phase.py; question3/results/round19_outer_rounding/code_snapshot/question3/ring_phase.py; question3/ring_phase.py

```
1 """Once-only radial alignment AFTER selecting the first outer scan task."""
2 import math
3 from .geometry import stations
4
5
6 def align_first_entry(cover,station_id,position):
7     if not 1<=station_id<=6:raise ValueError('Need an outer station')
8     if math.hypot(*position)<=1e-9:return cover,0.
9     phase=math.atan2(position[1],position[0])-(station_id-1)*math.pi/3
10    radius=math.hypot(*cover[1])
11    return stations(radius,phase),phase
```

程序 104

对应文件: question3/innovation/frozen/question3/run_faults.py; question3/results/dispatch_20260913/development/code_snapshot/question3/run_faults.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/run_faults.py; question3/results/round13_adverse_main/code_snapshot/question3/run_faults.py; question3/results/round13_adverse_rounding/code_snapshot/question3/run_faults.py; question3/results/round13_dev_main/code_snapshot/question3/run_faults.py; question3/results/round13_dev_rounding/code_snapshot/question3/run_faults.py; question3/results/round13_holdout_main/code_snapshot/question3/run_faults.py; question3/results/round13_holdout_rounding/code_snapshot/question3/run_faults.py; question3/results/round13_outer_main/code_snapshot/question3/run_faults.py; question3/results/round13_outer_rounding/code_snapshot/question3/run_faults.py; question3/results/round14_adverse_main/code_snapshot/question3/run_faults.py; question3/results/round14_adverse_rounding/code_snapshot/question3/run_faults.py; question3/results/round14_dev_main/code_snapshot/question3/run_faults.py; question3/results/round14_dev_rounding/code_snapshot/question3/run_faults.py; question3/results/round14_holdout_main/code_snapshot/question3/run_faults.py; question3/results/round14_holdout_rounding/code_snapshot/question3/run_faults.py; question3/results/round14_initial_dev_main/code_snapshot/question3/run_faults.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/run_faults.py; question3/results/round14_outer_main/code_snapshot/question3/run_faults.py; question3/results/round14_outer_rounding/code_snapshot/question3/run_faults.py; question3/results/round17_adverse_main/code_snapshot/question3/run_faults.py; question3/results/round17_adverse_rounding/code_snapshot/question3/run_faults.py; question3/results/round17_dev_main/code_snapshot/question3/run_faults.py; question3/results/round17_dev_rounding/code_snapshot/question3/run_faults.py; question3/results/round18_adverse_main/code_snapshot/question3/run_faults.py; question

3/results/round18_adverse_rounding/code_snapshot/question3/run_faults.py;
question3/results/round18_dev_main/code_snapshot/question3/run_faults.py;
question3/results/round18_dev_rounding/code_snapshot/question3/run_faults.py;
question3/results/round18_holdout_main/code_snapshot/question3/run_faults.py;
question3/results/round18_holdout_rounding/code_snapshot/question3/run_faults.py;
question3/results/round18_outer_main/code_snapshot/question3/run_faults.py;
question3/results/round18_outer_rounding/code_snapshot/question3/run_faults.py;
question3/results/round19_adverse_main/code_snapshot/question3/run_faults.py;
question3/results/round19_adverse_rounding/code_snapshot/question3/run_faults.py;
question3/results/round19_dev_main/code_snapshot/question3/run_faults.py;
question3/results/round19_dev_rounding/code_snapshot/question3/run_faults.py;
question3/results/round19_holdout_main/code_snapshot/question3/run_faults.py;
question3/results/round19_holdout_rounding/code_snapshot/question3/run_faults.py;
question3/results/round19_outer_main/code_snapshot/question3/run_faults.py;
question3/results/round19_outer_rounding/code_snapshot/question3/run_faults.py;
question3/results/round1_main/code_snapshot/question3/run_faults.py;
question3/results/round1_rounding/code_snapshot/question3/run_faults.py;
question3/run_faults.py

```

1 """Save negative-path evidence. Faults deliberately violate physics/transport contracts;
2 these are NOT valid benchmark cases and are excluded from clearance performance rates.
3 """
4 import json
5 from pathlib import Path
6 from .cases import random_case
7 from .simulator import Simulator
8 from .interface import Client, JsonlLog
9 from .strategy import run
10
11
12 def main():
13     config=json.loads(Path('question3/configs/baseline.json').read_text(encoding='utf-8'))
14     root=Path('question3/results/fault_injection');root.mkdir(parents=True,exist_ok=True)
15     summaries=[]
16     for fault in ['lost_measure_response', 'second_point_no_signal', 'all_clear_feedback_failed', 'zero_real_budget']:
17         out=root/fault;out.mkdir(exist_ok=True)
18         env=Simulator(random_case(0),remaining_s=0 if fault=='zero_real_budget' else 1200)
19         log=JsonlLog(out/'actions.jsonl');events=JsonlLog(out/'strategy.jsonl');phase={'localizing':False};dropped=set()
20         ↪ t()
21         def event(row):
22             if row['type']=='coverage_complete':phase['localizing']=True
23             events(row)
24         class FaultTransport:
25             def request(self,path,payload,timeout):
26                 status,r=env.request(path,payload,timeout)
27                 if fault=='lost_measure_response' and path=='/measure' and not dropped:
28                     dropped.add(payload['request_id']);raise ConnectionError('Injected loss after accepted action')
29                 if fault=='second_point_no_signal' and phase['localizing'] and path=='/measure':
30                     r['measure_result']='no_signal';r.pop('svd_deg',None)
31                 if fault=='all_clear_feedback_failed' and path=='/clear' and r.get('clear_result')=='success':
32                     env._virtual_us-=2000000;env._components['laser_s']-=2;r['virtual_time_s']-=2
33                     r['clear_result']='no_target_in_range'
34                 return status,r
35         client=Client(FaultTransport(),'SELF',log,config['real_time_reserve_s'])
36         result=run(client,config,event)
37         expected='complete' if fault=='lost_measure_response' else 'incomplete'
38         assert result['status']==expected,(fault,result)
39         row=dict(kind='FAULT_INJECTION_NOT_VALID_CASE', fault=fault,expected=expected,result=result)
40         (out/'result.json').write_text(json.dumps(row,indent=2),encoding='utf-8')

```



```

40     summaries.append(dict(fault=fault,expected=expected,actual=result['status'],failure=result['failure']))
41     log.close();events.close()
42     (root/'summary.json').write_text(json.dumps(summaries,indent=2),encoding='utf-8')
43     print(json.dumps(summaries,indent=2))
44
45 if __name__=='__main__':main()

```

程序 105

对应文件: question3/innovation/frozen/question3/run_http.py; question3/results/dispatch_20260913/development/code_snapshot/question3/run_http.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/run_http.py; question3/results/round13_adverse_main/code_snapshot/question3/run_http.py; question3/results/round13_adverse_rounding/code_snapshot/question3/run_http.py; question3/results/round13_dev_main/code_snapshot/question3/run_http.py; question3/results/round13_dev_rounding/code_snapshot/question3/run_http.py; question3/results/round13_holdout_main/code_snapshot/question3/run_http.py; question3/results/round13_holdout_rounding/code_snapshot/question3/run_http.py; question3/results/round13_outer_main/code_snapshot/question3/run_http.py; question3/results/round13_outer_rounding/code_snapshot/question3/run_http.py; question3/results/round14_adverse_main/code_snapshot/question3/run_http.py; question3/results/round14_adverse_rounding/code_snapshot/question3/run_http.py; question3/results/round14_dev_main/code_snapshot/question3/run_http.py; question3/results/round14_dev_rounding/code_snapshot/question3/run_http.py; question3/results/round14_holdout_main/code_snapshot/question3/run_http.py; question3/results/round14_holdout_rounding/code_snapshot/question3/run_http.py; question3/results/round14_initial_dev_main/code_snapshot/question3/run_http.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/run_http.py; question3/results/round14_outer_main/code_snapshot/question3/run_http.py; question3/results/round14_outer_rounding/code_snapshot/question3/run_http.py; question3/results/round17_adverse_main/code_snapshot/question3/run_http.py; question3/results/round17_adverse_rounding/code_snapshot/question3/run_http.py; question3/results/round17_dev_main/code_snapshot/question3/run_http.py; question3/results/round17_dev_rounding/code_snapshot/question3/run_http.py; question3/results/round18_adverse_main/code_snapshot/question3/run_http.py; question3/results/round18_adverse_rounding/code_snapshot/question3/run_http.py; question3/results/round18_dev_main/code_snapshot/question3/run_http.py; question3/results/round18_dev_rounding/code_snapshot/question3/run_http.py; question3/results/round18_holdout_main/code_snapshot/question3/run_http.py; question3/results/round18_holdout_rounding/code_snapshot/question3/run_http.py; question3/results/round18_outer_main/code_snapshot/question3/run_http.py; question3/results/round18_outer_rounding/code_snapshot/question3/run_http.py; question3/results/round19_adverse_main/code_snapshot/question3/run_http.py; question3/results/round1

9_adverse_rounding/code_snapshot/question3/run_http.py; question3/results/round19_dev_main/code_snapshot/question3/run_http.py; question3/results/round19_dev_rounding/code_snapshot/question3/run_http.py; question3/results/round19_holdout_main/code_snapshot/question3/run_http.py; question3/results/round19_holdout_rounding/code_snapshot/question3/run_http.py; question3/results/round19_outer_main/code_snapshot/question3/run_http.py; question3/results/round19_outer_rounding/code_snapshot/question3/run_http.py; question3/run_http.py

```

1 """Run strategy against self HTTP or an already-open official Q3 REHEARSAL.
2 No formal-test mode; no simulator UI automation; no exe launch.
3 """
4 import argparse
5 import json
6 import time
7 from pathlib import Path
8 from .interface import Client,HttpTransport,JsonLog
9 from .strategy import run
10
11
12 def main():
13     p=argparse.ArgumentParser()
14     p.add_argument('--mode',choices=['self-http','official-rehearsal'],default='self-http')
15     p.add_argument('--url');p.add_argument('--robot-id',default='SELF')
16     p.add_argument('--confirm-rehearsal-ui',action='store_true',help='Human confirms Q3 rehearsal, NOT formal UI')
17     p.add_argument('--output',required=True)
18     p.add_argument('--config',type=Path,default=Path(__file__).with_name('configs')/'baseline.json',
19                   help='Strategy JSON; pass configs/round1/AB.json to run the approved AB candidate')
20     args=p.parse_args()
21     if args.mode=='official-rehearsal' and (not args.confirm_rehearsal_ui or args.robot_id=='SELF'):
22         p.error('Confirm official Q3 REHEARSAL UI and supply team robot-id. Formal mode is not implemented.')
23     url=args.url or ('http://127.0.0.1:2027' if args.mode=='self-http' else 'http://127.0.0.1:2026')
24     config=json.loads(args.config.read_text(encoding='utf-8'))
25     out=Path(args.output)
26     out.mkdir(parents=True,exist_ok=False)
27     (out/'config.json').write_text(json.dumps(config,ensure_ascii=False,indent=2),encoding='utf-8')
28     log=JsonLog(out/'actions.jsonl');events=JsonLog(out/'strategy.jsonl')
29     client=Client(HttpTransport(url),args.robot_id,log,config['real_time_reserve_s'])
30     start=time.perf_counter()
31     try:result=run(client,config,events)
32     except Exception as e:result=dict(status='incomplete',failure=f'{type(e).__name__}: {e}',metrics=client.metrics())
33     result.update(kind=args.mode,config=config,elapsed_wall_s=time.perf_counter()-start,
34                 official_total=None,clearance_ratio=None,
35                 note='Official count must be read from rehearsal UI after exit; no hidden count in HTTP API.')
36     (out/'result.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
37     log.close();events.close();print(json.dumps(result,ensure_ascii=False,indent=2))
38     if result['status']!='complete':raise SystemExit(1)
39
40 if __name__=='__main__':main()

```

程序 106

对应文件: question3/innovation/frozen/question3/run_matrix.py; question3/results/dispatch_20260913/development/code_snapshot/question3/run_matrix.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/run_matrix.py; question3/results/round13_adverse_main/code_snapshot/question3/run_matrix.py; question3/results/round13_adverse_rounding/code_snapshot/question3/run_matrix.py; question3/results/round13_dev_main/code_snapshot/question3/run_matrix.py; question3/results/round13_dev_rounding/code_snapshot/question3/run_matrix.py; question3/results/round13_holdout_main/code_snapshot/question3/run_matrix.py; question3/results/round13_holdout_ro

```

unding/code_snapshot/question3/run_matrix.py; question3/results/round13_
outer_main/code_snapshot/question3/run_matrix.py; question3/results/roun
d13_outer_rounding/code_snapshot/question3/run_matrix.py; question3/resu
lts/round14_adverse_main/code_snapshot/question3/run_matrix.py; question
3/results/round14_adverse_rounding/code_snapshot/question3/run_matrix.py;
question3/results/round14_dev_main/code_snapshot/question3/run_matrix.py;
question3/results/round14_dev_rounding/code_snapshot/question3/run_matri
x.py; question3/results/round14_holdout_main/code_snapshot/question3/run
_matrix.py; question3/results/round14_holdout_rounding/code_snapshot/que
stion3/run_matrix.py; question3/results/round14_initial_dev_main/code_sn
apshot/question3/run_matrix.py; question3/results/round14_initial_dev_ro
unding/code_snapshot/question3/run_matrix.py; question3/results/round14_
outer_main/code_snapshot/question3/run_matrix.py; question3/results/roun
d14_outer_rounding/code_snapshot/question3/run_matrix.py; question3/resu
lts/round17_adverse_main/code_snapshot/question3/run_matrix.py; question
3/results/round17_adverse_rounding/code_snapshot/question3/run_matrix.py;
question3/results/round17_dev_main/code_snapshot/question3/run_matrix.py;
question3/results/round17_dev_rounding/code_snapshot/question3/run_matri
x.py; question3/results/round18_adverse_main/code_snapshot/question3/run
_matrix.py; question3/results/round18_adverse_rounding/code_snapshot/que
stion3/run_matrix.py; question3/results/round18_dev_main/code_snapshot/q
uestion3/run_matrix.py; question3/results/round18_dev_rounding/code_snap
shot/question3/run_matrix.py; question3/results/round18_holdout_main/cod
e_snapshot/question3/run_matrix.py; question3/results/round18_holdout_ro
unding/code_snapshot/question3/run_matrix.py; question3/results/round18_
outer_main/code_snapshot/question3/run_matrix.py; question3/results/roun
d18_outer_rounding/code_snapshot/question3/run_matrix.py; question3/resu
lts/round19_adverse_main/code_snapshot/question3/run_matrix.py; question
3/results/round19_adverse_rounding/code_snapshot/question3/run_matrix.py;
question3/results/round19_dev_main/code_snapshot/question3/run_matrix.py;
question3/results/round19_dev_rounding/code_snapshot/question3/run_matri
x.py; question3/results/round19_holdout_main/code_snapshot/question3/run_m
atrix.py; question3/results/round19_holdout_rounding/code_snapshot/questio
n3/run_matrix.py; question3/results/round19_outer_main/code_snapshot/quest
ion3/run_matrix.py; question3/results/round19_outer_rounding/code_snapshot
/question3/run_matrix.py; question3/run_matrix.py

```

```

1 """First-round fixed factorial experiment. Saves EVERY run including failures.
2 Case truth accessible to evaluator only, after strategy. No holdout generation.
3 """
4 import argparse
5 import hashlib
6 import json
7 import platform
8 import sys
9 import time
10 from pathlib import Path
11 from .cases import development_cases
12 from .simulator import Simulator

```

```

13 from .interface import Client,JsonlLog
14 from .strategy import run
15 from .modules import design_catalog
16 from .audit import audit_folder
17
18 VARIANTS=['baseline','A','B','C','AB','AC','BC','ABC']
19
20 def main():
21     parser=argparse.ArgumentParser();parser.add_argument('--output',required=True)
22     parser.add_argument('--rounding',choices=['bounded','pre_round_stress'],default='bounded')
23     parser.add_argument('--limit',type=int)
24     parser.add_argument('--config-dir',default='question3/configs/round1')
25     parser.add_argument('--fixtures')
26     parser.add_argument('--stage',default='SELF_ROUND1_DEVELOPMENT_NOT_HOLDOUT')
27     parser.add_argument('--lock')
28     args=parser.parse_args();out=Path(args.output);out.mkdir(parents=True,exist_ok=False)
29     names=VARIANTS if args.config_dir=='question3/configs/round1' else sorted(p.stem for p in
↳ Path(args.config_dir).glob('*.json'))
30     configs={n:json.loads((Path(args.config_dir)/(n+'.json')).read_text()) for n in names}
31     if args.lock:
32         lock=json.loads(Path(args.lock).read_text())
33         for name,digest in lock['code_sha256'].items():
34             if hashlib.sha256(Path(name).read_bytes()).hexdigest()!=digest:raise RuntimeError('Locked source changed:
↳ '+name)
35             if configs!=lock['configs']:raise RuntimeError('Locked configurations changed')
36     if args.fixtures and not args.lock:raise RuntimeError('External validation fixtures require a pre-run
↳ configuration lock')
37     fixtures=json.loads(Path(args.fixtures).read_text()) if args.fixtures else list(development_cases());rows=[]
38     startup=time.perf_counter();design_catalog();catalog_startup=time.perf_counter()-startup
39     files=list(Path('question3').glob('*.py'))+[Path('question1/solve.py'),Path('question2/strategy.py')]
40     manifest={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in files}
41     snapshot=out/'code_snapshot';snapshot.mkdir()
42     for p in files:
43         dest=snapshot/p;dest.parent.mkdir(parents=True,exist_ok=True);dest.write_bytes(p.read_bytes())
44     fixtures_hash={c['id']:hashlib.sha256(json.dumps(c,sort_keys=True).encode()).hexdigest() for c in fixtures}
45     (out/'configs.json').write_text(json.dumps(configs,indent=2))
46     for i,case in enumerate(fixtures):
47         if args.limit is not None and i>=args.limit:break
48         # Rotate config order: no module consistently absorbs first-run overhead.
49         order=names[i%len(names):]+names[:i%len(names)]
50         for name in order:
51             cfg=configs[name];folder=out/name/case['id'];folder.mkdir(parents=True)
52             (folder/'evaluator_hidden_case.json').write_text(json.dumps(case,indent=2),encoding='utf-8')
53             log=JsonlLog(folder/'actions.jsonl');events=JsonlLog(folder/'strategy.jsonl')
54             env=Simulator(case,rounding=args.rounding);client=Client(env,'SELF',log,cfg['real_time_reserve_s'])
55             start=time.perf_counter()
56             try:result=run(client,cfg,events)
57             except Exception as exc:result=dict(status='incomplete',failure=f'{type(exc).__name__}:
↳ {exc}',metrics=client.metrics(),regions=[])
58             wall=time.perf_counter()-start;truth=env.evaluation()
59             row=dict(variant=name,case_id=case['id'],kind=args.stage,rounding=args.rounding,
60                     status=result['status'],failure=result.get('failure'),elapsed_wall_s=wall,
61                     total=truth['total'],cleared=truth['cleared'],remaining_channels=truth['remaining_channels'],
62                     clearance_ratio=truth['cleared']/truth['total'],
63                     average_virtual_per_cleared=truth['virtual_time_s']/truth['cleared'] if truth['cleared'] else
↳ None,
64                     metrics=result['metrics'])
65             row['false_complete']=result['status']=='complete' and bool(truth['remaining_channels'])
66             row['timeout']=bool(result.get('failure') and any(s in result['failure'].lower() for s in
↳ ('deadline','timeout','budget')))
67             row['penalized_virtual_s']=truth['virtual_time_s'] if result['status']=='complete' and not
↳ truth['remaining_channels'] else max(360000,truth['virtual_time_s'])
68             (folder/'result.json').write_text(json.dumps(dict(summary=row,strategy=result),ensure_ascii=False,indent=)
↳ 2),encoding='utf-8')
69             log.close();events.close()
70             if row['status']=='complete':
71                 try:row['audited_actions']=audit_folder(folder);row['audit']='pass'
72                 except Exception as exc:row['audit']='FAIL: '+str(exc);row['false_complete']=True
73             else:row['audit']='incomplete run retained; no complete certificate asserted'
74             rows.append(row)
75             if row['status']!='complete' or
↳ row['false_complete']:print('FAIL',name,case['id'],row['failure'],row['audit'],flush=True)

```

```

76     print(f'{i+1}/{len(fixture)} {case["id"]}: '+,
↪     '.join(f'{"variant"}={r["metrics"]["virtual_time_s"]:.0f}' for r in rows[-len(names):]),flush=True)
77     summary=dict(kind=args.stage,rounding=args.rounding,cases=rows,
78                 configs=configs,case_sha256=fixtures_hash,code_sha256=manifest,
79                 catalog_startup_wall_s=catalog_startup,platform=platform.platform(),python=sys.version,
80                 official_rehearsals_run=0,formal_tests_run=0,unseen_validation_cases_run=(i+1 if args.fixtures
↪                 else 0),config_dir=args.config_dir)
81     (out/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
82     if any(r['false_complete'] for r in rows):raise SystemExit(2)
83
84 if __name__=='__main__':main()

```

程序 107

对应文件: question3/innovation/frozen/question3/run_self.py; question3/results/dispatch_20260913/development/code_snapshot/question3/run_self.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/run_self.py; question3/results/round13_adverse_main/code_snapshot/question3/run_self.py; question3/results/round13_adverse_rounding/code_snapshot/question3/run_self.py; question3/results/round13_dev_main/code_snapshot/question3/run_self.py; question3/results/round13_dev_rounding/code_snapshot/question3/run_self.py; question3/results/round13_holdout_main/code_snapshot/question3/run_self.py; question3/results/round13_holdout_rounding/code_snapshot/question3/run_self.py; question3/results/round13_outer_main/code_snapshot/question3/run_self.py; question3/results/round13_outer_rounding/code_snapshot/question3/run_self.py; question3/results/round14_adverse_main/code_snapshot/question3/run_self.py; question3/results/round14_adverse_rounding/code_snapshot/question3/run_self.py; question3/results/round14_dev_main/code_snapshot/question3/run_self.py; question3/results/round14_dev_rounding/code_snapshot/question3/run_self.py; question3/results/round14_holdout_main/code_snapshot/question3/run_self.py; question3/results/round14_holdout_rounding/code_snapshot/question3/run_self.py; question3/results/round14_initial_dev_main/code_snapshot/question3/run_self.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/run_self.py; question3/results/round14_outer_main/code_snapshot/question3/run_self.py; question3/results/round14_outer_rounding/code_snapshot/question3/run_self.py; question3/results/round17_adverse_main/code_snapshot/question3/run_self.py; question3/results/round17_adverse_rounding/code_snapshot/question3/run_self.py; question3/results/round17_dev_main/code_snapshot/question3/run_self.py; question3/results/round17_dev_rounding/code_snapshot/question3/run_self.py; question3/results/round18_adverse_main/code_snapshot/question3/run_self.py; question3/results/round18_adverse_rounding/code_snapshot/question3/run_self.py; question3/results/round18_dev_main/code_snapshot/question3/run_self.py; question3/results/round18_dev_rounding/code_snapshot/question3/run_self.py; question3/results/round18_holdout_main/code_snapshot/question3/run_self.py; question3/results/round18_holdout_rounding/code_snapshot/question3/run_self.py; question3/results/round18_outer_main

```

/code_snapshot/question3/run_self.py; question3/results/round18_outer_rou
nding/code_snapshot/question3/run_self.py; question3/results/round19_ad
verse_main/code_snapshot/question3/run_self.py; question3/results/round1
9_adverse_rounding/code_snapshot/question3/run_self.py; question3/result
s/round19_dev_main/code_snapshot/question3/run_self.py; question3/result
s/round19_dev_rounding/code_snapshot/question3/run_self.py; question3/re
sults/round19_holdout_main/code_snapshot/question3/run_self.py; question
3/results/round19_holdout_rounding/code_snapshot/question3/run_self.py;
question3/results/round19_outer_main/code_snapshot/question3/run_self.py;
question3/results/round19_outer_rounding/code_snapshot/question3/run_sel
f.py; question3/results/round1_main/code_snapshot/question3/run_self.py;
question3/results/round1_rounding/code_snapshot/question3/run_self.py;
question3/run_self.py

```

```

1 """Run development-only SELF fixtures. Hidden truth is saved for evaluator replay,
2 never passed to strategy. Holdout comparison is deferred until user approves innovation.
3 """
4 import argparse
5 import hashlib
6 import json
7 import platform
8 import sys
9 import time
10 from pathlib import Path
11 from .cases import development_cases
12 from .simulator import Simulator
13 from .interface import Client, JsonlLog
14 from .strategy import run
15
16
17 def main():
18     parser=argparse.ArgumentParser();parser.add_argument('--output',default='question3/results/baseline_dev')
19     parser.add_argument('--limit',type=int);parser.add_argument('--rounding',choices=['bounded','pre_round_stress'],d
↵     efault='bounded')
20     args=parser.parse_args();out=Path(args.output);out.mkdir(parents=True,exist_ok=True)
21     config=json.loads(Path('question3/configs/baseline.json').read_text());rows=[]
22     (out/'config.json').write_text(json.dumps(config,indent=2))
23     manifest={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted(Path('question3').glob('*.py'))}
24     manifest.update({str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in
↵     [Path('question1/solve.py'),Path('question2/strategy.py')]})
25     for index,case in enumerate(development_cases()):
26         if args.limit is not None and index>=args.limit:break
27         folder=out/case['id'];folder.mkdir(exist_ok=True)
28         (folder/'evaluator_hidden_case.json').write_text(json.dumps(case,indent=2))
29         log=JsonlLog(folder/'actions.jsonl');events=JsonlLog(folder/'strategy.jsonl')
30         env=Simulator(case,rounding=args.rounding)
31         client=Client(env,'SELF',log,reserve_s=config['real_time_reserve_s'])
32         started=time.perf_counter()
33         try:result=run(client,config,events)
34         except Exception as exc:result=dict(status='incomplete',failure=f'{type(exc).__name__}:
↵         {exc}',metrics=client.metrics())
35         elapsed=time.perf_counter()-started
36         truth=env.evaluation() # Evaluation only, after strategy turn.
37         row=dict(case_id=case['id'],kind='SELF_DEVELOPMENT',rounding=args.rounding,
38                 status=result['status'],failure=result.get('failure'),elapsed_wall_s=elapsed,
39                 total=truth['total'],cleared=truth['cleared'],remaining_channels=truth['remaining_channels'],
40                 clearance_ratio=truth['cleared']/truth['total'],
41                 average_virtual_per_cleared=truth['virtual_time_s']/truth['cleared'] if truth['cleared'] else None,
42                 metrics=result['metrics'])
43         if result['status']=='complete' and truth['remaining_channels']:raise AssertionError('False completion!')
44         for k,v in truth['components'].items():
45             if abs(v-result['metrics']['components'][k])>1e-6:raise AssertionError('Accounting mismatch '+k)
46         (folder/'result.json').write_text(json.dumps(dict(summary=row,strategy=result),ensure_ascii=False,indent=2))
47         rows.append(row);log.close();events.close()

```

```

48     print(case['id'],row['status'],f'{truth["cleared"]}/{truth["total"]}',f'{truth["virtual_time_s"]:.2f}s',flush)
    ↪ =True)
49     summary=dict(kind='SELF_DEVELOPMENT_ONLY',python=sys.version,platform=platform.platform(),config=config,
50                 code_sha256=manifest,cases=rows,official_rehearsals_run=0,formal_tests_run=0,
51                 note='No innovation comparisons or held-out cases run; awaiting human review.')
52     (out/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2))
53     if any(row['status']!='complete' or row['clearance_ratio']!=1 for row in rows):raise SystemExit(1)
54
55 if __name__=='__main__':main()

```

程序 108

对应文件: question3/innovation/frozen/question3/scan_route.py; question3/results/dispatch_20260913/development/code_snapshot/question3/scan_route.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/scan_route.py; question3/results/round17_adverse_main/code_snapshot/question3/scan_route.py; question3/results/round17_adverse_rounding/code_snapshot/question3/scan_route.py; question3/results/round17_dev_main/code_snapshot/question3/scan_route.py; question3/results/round17_dev_rounding/code_snapshot/question3/scan_route.py; question3/results/round18_adverse_main/code_snapshot/question3/scan_route.py; question3/results/round18_adverse_rounding/code_snapshot/question3/scan_route.py; question3/results/round18_dev_main/code_snapshot/question3/scan_route.py; question3/results/round18_dev_rounding/code_snapshot/question3/scan_route.py; question3/results/round18_holdout_main/code_snapshot/question3/scan_route.py; question3/results/round18_holdout_rounding/code_snapshot/question3/scan_route.py; question3/results/round18_outer_main/code_snapshot/question3/scan_route.py; question3/results/round18_outer_rounding/code_snapshot/question3/scan_route.py; question3/results/round19_adverse_main/code_snapshot/question3/scan_route.py; question3/results/round19_adverse_rounding/code_snapshot/question3/scan_route.py; question3/results/round19_dev_main/code_snapshot/question3/scan_route.py; question3/results/round19_dev_rounding/code_snapshot/question3/scan_route.py; question3/results/round19_holdout_main/code_snapshot/question3/scan_route.py; question3/results/round19_holdout_rounding/code_snapshot/question3/scan_route.py; question3/results/round19_outer_main/code_snapshot/question3/scan_route.py; question3/results/round19_outer_rounding/code_snapshot/question3/scan_route.py; question3/scan_route.py

```

1  """Shortest open path through remaining public scan stations; no source data.
2
3  Exact for the stated static path proxy only. Localization interrupts the route,
4  and the count certificate may remove scans, so this is not total-time optimality.
5  """
6  from functools import lru_cache
7  import math
8
9
10 def remaining_open_path(current,cover,remaining):
11     ids=tuple(sorted(remaining))
12     if len(ids)!=len(set(ids)) or len(ids)>7:raise ValueError('Expected at most seven distinct stations')
13     if not ids:return (),0.
14     @lru_cache(maxsize=None)
15     def tail(last,mask):

```

```

16         if not mask: return 0., ()
17         choices=[]
18         for j,k in enumerate(ids):
19             if mask&(1<<j):
20                 cost,path=tail(k,mask^(1<<j))
21                 choices.append((math.dist(cover[last],cover[k])+cost,(k,)+path))
22         return min(choices)
23     choices=[];mask=(1<<len(ids))-1
24     for j,k in enumerate(ids):
25         cost,path=tail(k,mask^(1<<j))
26         choices.append((math.dist(current,cover[k])+cost,(k,)+path))
27     cost,path=min(choices)
28     return path,cost

```

程序 109

对应文件: question3/innovation/frozen/question3/self_server.py; question3/results/dispatch_20260913/development/code_snapshot/question3/self_server.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/self_server.py; question3/results/round13_adverse_main/code_snapshot/question3/self_server.py; question3/results/round13_adverse_rounding/code_snapshot/question3/self_server.py; question3/results/round13_dev_main/code_snapshot/question3/self_server.py; question3/results/round13_dev_rounding/code_snapshot/question3/self_server.py; question3/results/round13_holdout_main/code_snapshot/question3/self_server.py; question3/results/round13_holdout_rounding/code_snapshot/question3/self_server.py; question3/results/round13_outer_main/code_snapshot/question3/self_server.py; question3/results/round13_outer_rounding/code_snapshot/question3/self_server.py; question3/results/round14_adverse_main/code_snapshot/question3/self_server.py; question3/results/round14_adverse_rounding/code_snapshot/question3/self_server.py; question3/results/round14_dev_main/code_snapshot/question3/self_server.py; question3/results/round14_dev_rounding/code_snapshot/question3/self_server.py; question3/results/round14_holdout_main/code_snapshot/question3/self_server.py; question3/results/round14_holdout_rounding/code_snapshot/question3/self_server.py; question3/results/round14_initial_dev_main/code_snapshot/question3/self_server.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/self_server.py; question3/results/round14_outer_main/code_snapshot/question3/self_server.py; question3/results/round14_outer_rounding/code_snapshot/question3/self_server.py; question3/results/round17_adverse_main/code_snapshot/question3/self_server.py; question3/results/round17_adverse_rounding/code_snapshot/question3/self_server.py; question3/results/round17_dev_main/code_snapshot/question3/self_server.py; question3/results/round17_dev_rounding/code_snapshot/question3/self_server.py; question3/results/round18_adverse_main/code_snapshot/question3/self_server.py; question3/results/round18_adverse_rounding/code_snapshot/question3/self_server.py; question3/results/round18_dev_main/code_snapshot/question3/self_server.py; question3/results/round18_dev_rounding/code_snapshot/question3/self_server.py; question3/results/round18_holdout_main/code_snapshot/question3/self_server.py; question3/results/round18_

holdout_rounding/code_snapshot/question3/self_server.py; question3/results/round18_outer_main/code_snapshot/question3/self_server.py; question3/results/round18_outer_rounding/code_snapshot/question3/self_server.py; question3/results/round19_adverse_main/code_snapshot/question3/self_server.py; question3/results/round19_adverse_rounding/code_snapshot/question3/self_server.py; question3/results/round19_dev_main/code_snapshot/question3/self_server.py; question3/results/round19_dev_rounding/code_snapshot/question3/self_server.py; question3/results/round19_holdout_main/code_snapshot/question3/self_server.py; question3/results/round19_holdout_rounding/code_snapshot/question3/self_server.py; question3/results/round19_outer_main/code_snapshot/question3/self_server.py; question3/results/round19_outer_rounding/code_snapshot/question3/self_server.py; question3/results/round1_main/code_snapshot/question3/self_server.py; question3/results/round1_rounding/code_snapshot/question3/self_server.py; question3/self_server.py

```

1  """Local SELF HTTP fixture, default port 2027 (official default is 2026)."""
2  import argparse
3  import json
4  from http.server import BaseHTTPRequestHandler, HTTPServer
5  from .cases import random_case
6  from .simulator import Simulator
7
8
9  def make_server(env, port=0):
10     class Handler(BaseHTTPRequestHandler):
11         def log_message(self, *args): pass
12         def do_POST(self):
13             status=400; response=dict(accepted=False, real_timestamp_ms=0, virtual_time_s=0)
14             try:
15                 length=int(self.headers.get('Content-Length', '0'))
16                 if length>65536: status=413
17                 elif self.headers.get('Content-Type', '').lower() not in ('application/json', 'application/json;
↳ charset=utf-8'):
18                     status=415
19                 else:
20                     def pairs(items):
21                         d={}
22                         for k,v in items:
23                             if k in d: raise ValueError('Duplicate JSON key')
24                             d[k]=v
25                         return d
26                     payload=json.loads(self.rfile.read(length).decode('utf-8'), object_pairs_hook=pairs,
27                                     parse_constant=lambda x: (_ for _ in []).throw(ValueError(x)))
28                     if isinstance(payload, dict): status, response=env.request(self.path, payload)
29             except (ValueError, UnicodeError): pass
30             except ConnectionError:
31                 self.close_connection=True; return
32             data=json.dumps(response, allow_nan=False).encode('utf-8')
33             self.send_response(status); self.send_header('Content-Type', 'application/json')
34             self.send_header('Content-Length', str(len(data))); self.end_headers(); self.wfile.write(data)
35     return HTTPServer(('127.0.0.1', port), Handler)
36
37
38 def main():
39     p=argparse.ArgumentParser(); p.add_argument('--port', type=int, default=2027); p.add_argument('--seed', type=int, default=0)
40     args=p.parse_args(); env=Simulator(random_case(args.seed)); server=make_server(env, args.port)
41     print(f'SELF ONLY http://127.0.0.1:{server.server_port}, robot_id=SELF', flush=True)
42     try: server.serve_forever()
43     except KeyboardInterrupt: pass
44     finally: server.server_close()
45

```

```
46 if __name__ == '__main__':main()
```

程序 110

对应文件:question3/innovation/frozen/question3/shared_cost.py;question3/results/dispatch_20260913/development/code_snapshot/question3/shared_cost.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/shared_cost.py; question3/results/round13_adverse_main/code_snapshot/question3/shared_cost.py; question3/results/round13_adverse_rounding/code_snapshot/question3/shared_cost.py; question3/results/round13_dev_main/code_snapshot/question3/shared_cost.py; question3/results/round13_dev_rounding/code_snapshot/question3/shared_cost.py; question3/results/round13_holdout_main/code_snapshot/question3/shared_cost.py; question3/results/round13_holdout_rounding/code_snapshot/question3/shared_cost.py; question3/results/round13_outer_main/code_snapshot/question3/shared_cost.py; question3/results/round13_outer_rounding/code_snapshot/question3/shared_cost.py;question3/results/round14_adverse_main/code_snapshot/question3/shared_cost.py;question3/results/round14_adverse_rounding/code_snapshot/question3/shared_cost.py; question3/results/round14_dev_main/code_snapshot/question3/shared_cost.py; question3/results/round14_dev_rounding/code_snapshot/question3/shared_cost.py; question3/results/round14_holdout_main/code_snapshot/question3/shared_cost.py; question3/results/round14_holdout_rounding/code_snapshot/question3/shared_cost.py; question3/results/round14_initial_dev_main/code_snapshot/question3/shared_cost.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/shared_cost.py; question3/results/round14_outer_main/code_snapshot/question3/shared_cost.py; question3/results/round14_outer_rounding/code_snapshot/question3/shared_cost.py; question3/results/round17_adverse_main/code_snapshot/question3/shared_cost.py; question3/results/round17_adverse_rounding/code_snapshot/question3/shared_cost.py; question3/results/round17_dev_main/code_snapshot/question3/shared_cost.py; question3/results/round17_dev_rounding/code_snapshot/question3/shared_cost.py; question3/results/round18_adverse_main/code_snapshot/question3/shared_cost.py; question3/results/round18_adverse_rounding/code_snapshot/question3/shared_cost.py; question3/results/round18_dev_main/code_snapshot/question3/shared_cost.py; question3/results/round18_dev_rounding/code_snapshot/question3/shared_cost.py; question3/results/round18_holdout_main/code_snapshot/question3/shared_cost.py; question3/results/round18_holdout_rounding/code_snapshot/question3/shared_cost.py; question3/results/round18_outer_main/code_snapshot/question3/shared_cost.py; question3/results/round18_outer_rounding/code_snapshot/question3/shared_cost.py; question3/results/round19_adverse_main/code_snapshot/question3/shared_cost.py; question3/results/round19_adverse_rounding/code_snapshot/question3/shared_cost.py; question3/results/round19_dev_main/code_snapshot/question3/shared_cost.py; question3/results/round19_dev_rounding/code_snapshot/question3/shared_cost.py;

st.py; question3/results/round19_holdout_main/code_snapshot/question3/shared_cost.py; question3/results/round19_holdout_rounding/code_snapshot/question3/shared_cost.py; question3/results/round19_outer_main/code_snapshot/question3/shared_cost.py; question3/results/round19_outer_rounding/code_snapshot/question3/shared_cost.py; question3/shared_cost.py

```

1 """G: comparable complete-secondary-task costs for shared measurement selection.
2 A sampled planning proxy, NOT a continuous cost bound or a probability model.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np
7 from question2.strategy import diameters
8 from .modules import choose_second
9
10
11 @lru_cache(maxsize=2048)
12 def shared_shape_proxy(offset,error_deg):
13     delta=math.radians(error_deg)
14     r,a,e=np.meshgrid(np.linspace(5.001,1500,31),np.linspace(-delta,delta,3),np.linspace(-delta,delta,3),indexing='ij')
15     ↪
16     g=np.column_stack([(r*np.cos(a)).ravel(),(r*np.sin(a)).ravel()])
17     d=float(diameters(offset,g,e.ravel(),error_deg).max())
18     cells=math.ceil(d/28)*math.ceil(2*1500*math.sin(delta)/28)
19     return cells,d,g.reshape(31,3,3,2)[:,:,:,:].reshape(-1,2)
20
21 def shared_saving(first,bearing,offset,primary_points,config):
22     cells,d,local=shared_shape_proxy(tuple(offset),config['error_deg'])
23     angle=math.radians(bearing);rot=np.array([[math.cos(angle),-math.sin(angle)],[math.sin(angle),math.cos(angle)]])
24     targets=np.asarray(first)+local@rot.T
25     best_lower=math.inf;travel_at_worst=grid_at_worst=None
26     for endpoint in primary_points:
27         independent,selection=choose_second(first,bearing,endpoint,config)
28         if 'proxy_cells' not in selection:raise ValueError('Shared cost gate requires A metadata')
29         travel=(math.dist(endpoint,independent)+np.linalg.norm(targets-independent,axis=1)-np.linalg.norm(targets-endpoint,axis=1))/5
30         ↪
31         grid=8.6*(selection['proxy_cells']-cells)
32         saving=float(travel.min())+grid
33         if saving<best_lower:best_lower=saving;travel_at_worst=float(travel.min());grid_at_worst=float(grid)
34     return dict(predicted_saving_s=best_lower,travel_proxy_s=travel_at_worst,grid_proxy_s=grid_at_worst,
35                 shared_proxy_cells=cells,shared_sampled_diameter_m=d,terminal_count=len(primary_points),source_sample_s=len(targets))

```

程序 111

对应文件: question3/innovation/frozen/question3/shared_measurement.py; question3/results/dispatch_20260913/development/code_snapshot/question3/shared_measurement.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/shared_measurement.py; question3/results/round19_adverse_main/code_snapshot/question3/shared_measurement.py; question3/results/round19_adverse_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round19_dev_main/code_snapshot/question3/shared_measurement.py; question3/results/round19_dev_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round19_holdout_main/code_snapshot/question3/shared_measurement.py; question3/results/round19_holdout_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round19_outer_main/code_snapshot/question3/shared_measurement.py; question3/results/round19_outer_rounding/code_snapshot/question3/shared_measurement.py; question3/shared_measurement.py

```

1 """S: certify another already-detected source at a shared second point."""
2 import math
3 from question2.strategy import candidate
4
5
6 def compatible_offset(first,bearing,point,error_deg,min_angle):
7     a=math.radians(bearing);dx=point[0]-first[0];dy=point[1]-first[1]
8     q=(dx*math.cos(a)+dy*math.sin(a),-dx*math.sin(a)+dy*math.cos(a))
9     return q if candidate(q,min_angle,error_deg) else None
10
11
12 def choose_shared(primary,point,first,cleared,cached,config,primary_points=None,decision_log=lambda e:None):
13     mode=config.get('shared_selection','lowest_id')
14     if mode not in ('lowest_id','cost_gate'):raise ValueError('Invalid shared selection mode')
15     ranked=[]
16     for channel in sorted(first):
17         if channel==primary or channel in cleared or channel in cached:continue
18         offset=compatible_offset(*first[channel],point,config['error_deg'],config['min_angle_deg'])
19         if offset is None and config.get('shared_range_prior','off')==domain':
20             from .domain_range import range_upper,certified
21             from .current_point import local_offset
22             s,bearing=first[channel];upper=range_upper(s,bearing,config['error_deg'])
23             q=local_offset(s,bearing,point)
24             if upper<1500 and certified(q,upper,config['error_deg'],config['min_angle_deg']):
25                 return channel,dict(mode='domain_shared',offset=q,shared_with=primary,
26                                     source_range_upper_m=upper,range_prior='published_domain_1800',
27                                     error_deg=config['error_deg'],min_angle_deg=config['min_angle_deg'])
28         if offset is not None:
29             selection=dict(mode='shared_certified',offset=offset,shared_with=primary,
30                             error_deg=config['error_deg'],min_angle_deg=config['min_angle_deg'])
31             if mode=='lowest_id':return channel,selection
32             if not primary_points:raise ValueError('Cost gate needs the primary optical plan')
33             from .shared_cost import shared_saving
34             prediction=shared_saving(*first[channel],offset,primary_points,config)
35             ranked.append((channel,**selection,**prediction, 'gate_mode':mode))
36     if mode=='cost_gate':
37         acceptable=[row for row in ranked if row[1]['predicted_saving_s']>1e-8]
38         selected=min(acceptable,key=lambda row:(-row[1]['predicted_saving_s'],row[0])) if acceptable else None
39         decision_log(dict(type='shared_gate',primary_channel=primary,selected_channel=selected[0] if selected else
40                             ↪ None,
41                             candidates=[dict(channel=c,**s) for c,s in ranked]))
42     return selected
43
44 return None

```

程序 112

对应文件: question3/innovation/frozen/question3/simulator.py; question3/results/dispatch_20260913/development/code_snapshot/question3/simulator.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/simulator.py; question3/results/round13_adverse_main/code_snapshot/question3/simulator.py; question3/results/round13_adverse_rounding/code_snapshot/question3/simulator.py; question3/results/round13_dev_main/code_snapshot/question3/simulator.py; question3/results/round13_dev_rounding/code_snapshot/question3/simulator.py; question3/results/round13_holdout_main/code_snapshot/question3/simulator.py; question3/results/round13_holdout_rounding/code_snapshot/question3/simulator.py; question3/results/round13_outer_main/code_snapshot/question3/simulator.py; question3/results/round13_outer_rounding/code_snapshot/question3/simulator.py; question3/results/round14_adverse_main/code_snapshot/question3/simulator.py; question3/results/round14_adverse_rounding/code_snapshot/question3/simulator.py; question3/results/round14_dev_main/code_snapshot/question3/simulator.py; question3/results/round14_dev_rounding/code_snapshot/question3/simulator

.py; question3/results/round14_holdout_main/code_snapshot/question3/simulator.py; question3/results/round14_holdout_rounding/code_snapshot/question3/simulator.py; question3/results/round14_initial_dev_main/code_snapshot/question3/simulator.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/simulator.py; question3/results/round14_outer_main/code_snapshot/question3/simulator.py; question3/results/round14_outer_rounding/code_snapshot/question3/simulator.py; question3/results/round17_adverse_main/code_snapshot/question3/simulator.py; question3/results/round17_adverse_rounding/code_snapshot/question3/simulator.py; question3/results/round17_dev_main/code_snapshot/question3/simulator.py; question3/results/round17_dev_rounding/code_snapshot/question3/simulator.py; question3/results/round18_adverse_main/code_snapshot/question3/simulator.py; question3/results/round18_adverse_rounding/code_snapshot/question3/simulator.py; question3/results/round18_dev_main/code_snapshot/question3/simulator.py; question3/results/round18_dev_rounding/code_snapshot/question3/simulator.py; question3/results/round18_holdout_main/code_snapshot/question3/simulator.py; question3/results/round18_holdout_rounding/code_snapshot/question3/simulator.py; question3/results/round18_outer_main/code_snapshot/question3/simulator.py; question3/results/round18_outer_rounding/code_snapshot/question3/simulator.py; question3/results/round19_adverse_main/code_snapshot/question3/simulator.py; question3/results/round19_adverse_rounding/code_snapshot/question3/simulator.py; question3/results/round19_dev_main/code_snapshot/question3/simulator.py; question3/results/round19_dev_rounding/code_snapshot/question3/simulator.py; question3/results/round19_holdout_main/code_snapshot/question3/simulator.py; question3/results/round19_holdout_rounding/code_snapshot/question3/simulator.py; question3/results/round19_outer_main/code_snapshot/question3/simulator.py; question3/results/round19_outer_rounding/code_snapshot/question3/simulator.py; question3/results/round1_main/code_snapshot/question3/simulator.py; question3/results/round1_rounding/code_snapshot/question3/simulator.py; question3/simulator.py

```

1 """SELF simulation of omnidirectional physics and valid-action protocol.
2 Not official simulator. No login/window UI/encrypted official log implementation.
3 Hidden case data stays here, evaluator accesses it only outside strategy execution.
4 """
5 import copy
6 import hashlib
7 import json
8 import math
9 import time
10
11 class Simulator:
12     def __init__(self, case, robot_id='SELF', remaining_s=1200, rounding='bounded'):
13         self._sources=copy.deepcopy(case['sources']);self._seed=case.get('seed',0)
14         self._error_mode=case.get('error_mode','spatial_hash');self._rounding=rounding
15         self._robot_id=robot_id;self._cache={};self._entered=False;self._exited=False
16         self._position=(0.,0.);self._channel=1;self._virtual_us=0;self._remaining=remaining_s;self._start=None
17         self._cleared=set();self._components=dict(movement_s=0.,switch_s=0.,detection_s=0.,optical_s=0.,laser_s=0.)
18         if len({s['channel'] for s in self._sources})!=len(self._sources):raise ValueError('Duplicate channel')
19         for s in self._sources:
20             if not 1<=s['channel']<=20 or not 1000<=s['radius']<=1500 or math.hypot(s['x'],s['y'])>1800+1e-7:

```

```

21         raise ValueError('Invalid self case')
22     def _response(self,accepted,**extra):
23         return dict(accepted=accepted,real_timestamp_ms=int(time.time()*1000),
24                     virtual_time_s=self._virtual_us/1e6 if accepted else 0,**extra)
25     def _bias(self,p,c):
26         x,y=p
27         # IEEE signed zero describes the same physical location.
28         if x==0:x=0.
29         if y==0:y=0.
30         if self._error_mode=='positive':return 1.
31         if self._error_mode=='negative':return -1.
32         if self._error_mode=='smooth':return math.sin(x/173+y/251+c+self._seed)
33         if self._error_mode=='zero':return 0.
34         if self._error_mode!='spatial_hash':raise ValueError('Unknown spatial error mode')
35         # Same location and channel => same bias, independent of call count/time.
36         data=f'{self._seed}:{c}:{float(x).hex()}:{float(y).hex()}'.encode()
37         value=int.from_bytes(hashlib.sha256(data).digest()[8],'big')
38         return 2*value/(2**64-1)-1
39     def _reading(self,p,s):
40         true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
41         tick=round((true+self._bias(p,s['channel']))*100)
42         if self._rounding=='bounded':
43             # Honour the stated final +/-1deg bound and two-decimal serialization together.
44             tick=min(math.floor((true+1)*100),max(math.ceil((true-1)*100),tick))
45         elif self._rounding!='pre_round_stress':raise ValueError('Unknown rounding model')
46         return (tick%36000)/100
47     def request(self,path,payload,timeout=5.):
48         b=copy.deepcopy(payload)
49         if path not in ('/enter','/measure','/clear','/exit'):return 404,self._response(False)
50         required={'arena_id','robot_id','request_id'}|({'position','channel'} if path in ('/measure','/clear') else
51         ↪ set())
52         if not required<=b.keys():return 400,self._response(False)
53         if set(b)!=required or b['arena_id']!='default' or b['robot_id']!=self._robot_id:return
54         ↪ 200,self._response(False)
55         if not isinstance(b['request_id'],str) or not b['request_id']:return 400,self._response(False)
56         if 'position' in b:
57             p=b['position'];c=b['channel']
58             if not isinstance(p,dict) or set(p)!={'x','y'}:return 400,self._response(False)
59             if any(isinstance(p[k],bool) or not isinstance(p[k],(int,float)) or not math.isfinite(p[k]) or
60             ↪ abs(p[k])>2000000 for k in ('x','y')):
61                 return 400,self._response(False)
62             if isinstance(c,bool) or not isinstance(c,(int,float)) or not math.isfinite(c) or int(c)!=c or not
63             ↪ 1<=c<=20:
64                 return 400,self._response(False)
65         signature=(path,json.dumps(b,sort_keys=True,allow_nan=False))
66         if b['request_id'] in self._cache:
67             old,result=self._cache[b['request_id']]
68             return copy.deepcopy(result) if signature==old else (409,self._response(False))
69         if self._exited:raise ConnectionError('Self API closed')
70         if self._start is not None and (time.monotonic()-self._start>=self._remaining or
71         ↪ self._virtual_us>=360000000000):
72             self._exited=True;raise ConnectionError('Self deadline expired')
73         if (path=='/enter' and self._entered) or (path!='/enter' and not self._entered):return
74         ↪ 200,self._response(False)
75         if path=='/enter':
76             self._entered=True;self._start=time.monotonic()
77             response=self._response(True,max_virtual_duration_s=360000,max_real_duration_s=1200,
78                                     remaining_real_duration_s=self._remaining)
79         elif path=='/exit':
80             self._exited=True;response=self._response(True,exit_reason='user_exit')
81         else:
82             p=(float(b['position']['x']),float(b['position']['y']));c=int(b['channel'])
83             move=math.dist(self._position,p)/5;switch=float(path=='/measure' and c!=self._channel)
84             parts=dict(movement_s=move,switch_s=switch,detection_s=0.,optical_s=0.,laser_s=0.)
85             s=next((s for s in self._sources if s['channel']==c and c not in self._cleared),None)
86             distance=math.inf if s is None else math.hypot(s['x']-p[0],s['y']-p[1])
87             if path=='/measure':
88                 parts['detection_s']=5.;self._channel=c
89                 if s is None or distance>s['radius']:result=dict(measure_result='no_signal')
90                 elif distance<=5:result=dict(measure_result='near')
91                 else:result=dict(measure_result='direction',svd_deg=self._reading(p,s))
92             else:
93                 parts['optical_s']=3.

```

```

88         if distance<=20:
89             self._cleared.add(c);parts['laser_s']=2.;result=dict(clear_result='success')
90         else:result=dict(clear_result='no_target_in_range')
91         for k,v in parts.items():self._components[k]+=v
92         self._virtual_us+=round(sum(parts.values())*1e6);self._position=p
93         response=self._response(True,**result)
94         result=(200,response);self._cache[b['request_id']]=(signature,copy.deepcopy(result));return result
95     def evaluation(self):
96         """Evaluator-only: never exposed by /enter, /measure, /clear, /exit."""
97         return dict(total=len(self._sources),cleared=len(self._cleared),
98                     remaining_channels=sorted(s['channel'] for s in self._sources if s['channel'] not in
99                     ↪ self._cleared),
100                    components=self._components.copy(),virtual_time_s=self._virtual_us/1e6)

```

程序 113

对应文件: question3/innovation/frozen/question3/strategy.py; question3/results/dispatch_20260913/development/code_snapshot/question3/strategy.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/strategy.py; question3/results/round19_adverse_main/code_snapshot/question3/strategy.py; question3/results/round19_adverse_rounding/code_snapshot/question3/strategy.py; question3/results/round19_dev_main/code_snapshot/question3/strategy.py; question3/results/round19_dev_rounding/code_snapshot/question3/strategy.py; question3/results/round19_holdout_main/code_snapshot/question3/strategy.py; question3/results/round19_holdout_rounding/code_snapshot/question3/strategy.py; question3/results/round19_outer_main/code_snapshot/question3/strategy.py; question3/results/round19_outer_rounding/code_snapshot/question3/strategy.py; question3/strategy.py

```

1  """Common complete workflow with optional modules A/B/C. Defaults preserve baseline_v1.
2  No imports of case generation/simulator; all decisions use public observations only.
3  """
4  import math
5  from .geometry import stations, station_cover_bound, optical_plan
6  from .modules import choose_second,EvidenceCover
7  from .lookahead import decide
8  from .optical_shortcut import single_circle
9  from .shared_measurement import choose_shared
10 from .order_routing import decide_order
11 from .cached_routing import reorder_cached
12 from .optical_order import nearer_end
13 from .clear_skip import constraints,failure_certificate,radius_failure_certificate
14 from .ring_phase import align_first_entry
15 from .batch_scheduling import gate as batch_gate
16
17 class Incomplete(RuntimeError):pass
18
19
20 def run(client, config, event=lambda data:None):
21     opportunity=config.get('second_point_opportunity','off')
22     if opportunity not in ('off','current_if_cheaper','current_if_certified'):
23         raise ValueError('Unknown second_point_opportunity')
24     if opportunity!='off' and not config.get('modules',{}).get('A',False):
25         raise ValueError('Current-point comparison requires A')
26     opportunity_timing=config.get('opportunity_timing','before_schedule')
27     if opportunity_timing not in ('before_schedule','after_schedule'):
28         raise ValueError('Unknown opportunity_timing')
29     planning_config={**config,'second_point_opportunity':'off'} if opportunity_timing=='after_schedule' else config
30     range_prior=config.get('second_range_prior','off')
31     if range_prior not in ('off','domain'):raise ValueError('Unknown second_range_prior')
32     if range_prior!='off':
33         if opportunity_timing!='after_schedule' or not config.get('modules',{}).get('A',False):
34             raise ValueError('Domain second-range prior requires A and after-schedule timing')
35     planning_config={**planning_config,'second_range_prior':'off'}

```

```

36 shared_range_prior=config.get('shared_range_prior','off')
37 if shared_range_prior not in ('off','domain'):raise ValueError('Unknown shared_range_prior')
38 if shared_range_prior!='off' and (not config.get('modules',{}).get('S',False) or
↪ config.get('shared_selection','lowest_id')!='lowest_id'):
39     raise ValueError('Shared domain range requires S and lowest-id selection')
40 count_stop=config.get('count_stop','off')
41 if count_stop not in ('off','cleared_16','detected_16'):
42     raise ValueError('Unknown count_stop; Q3 source upper bound is fixed at 16')
43 search_radius=config.get('search_radius_m',1500.)
44 if station_cover_bound(search_radius)>=1000:raise ValueError('Invalid search coverage')
45 empty_radius=config.get('empty_origin_radius_m')
46 if empty_radius is not None and station_cover_bound(empty_radius)>=1000:
47     raise ValueError('Invalid empty-origin search coverage')
48 cover=stations(search_radius)
49 negative={c:set() for c in range(1,21)};first={};cleared=set();regions=[];visited=set()
50 modules=config.get('modules',{});joint=modules.get('B',False);evidence=modules.get('C',False)
51 shared=modules.get('S',False);order=modules.get('O',False);cached={}
52 phase_mode=config.get('ring_phase','fixed');phase_decided=False;station_phase=0.
53 if phase_mode not in ('fixed','first_entry_radial'):raise ValueError('Unknown ring_phase')
54 if phase_mode!='fixed' and not joint:raise ValueError('Adaptive ring phase requires B task selection')
55 batch_mode=config.get('search_interleaving','current')
56 if batch_mode not in ('current','cached_batch','known_batch','nearest_finite'):raise ValueError('Unknown
↪ search_interleaving')
57 clear_skip=config.get('clear_skip','off');skip_planes={}
58 if clear_skip not in ('off','separating_halfplane','separating_pair'):raise ValueError('Unknown clear_skip')
59 if clear_skip!='off' and evidence:raise ValueError('Clear skipping requires the fixed optical path; C is
↪ incompatible')
60 radius_priors=config.get('clear_radius_priors','off')
61 if radius_priors not in ('off','domain','received','both'):raise ValueError('Unknown clear_radius_priors')
62 if radius_priors!='off' and clear_skip=='off':raise ValueError('Radius priors require clear_skip')
63 optical_order=config.get('optical_order','original')
64 if optical_order not in ('original','nearer_end'):
65     raise ValueError('Unknown optical_order')
66 if optical_order!='original' and evidence:
67     raise ValueError('Optical reversal requires a fixed optical path; C is incompatible')
68 cache_route=config.get('cache_routing','nearest')
69 if batch_mode!='current' and (not joint or order or cache_route!='nearest' or
↪ config.get('routing',{}).get('mode','nearest')!='nearest'):
70     raise ValueError('Batch scheduling requires B nearest routing without O/cache insertion')
71 if batch_mode=='cached_batch' and not shared:raise ValueError('Cached batch requires S')
72 scan_order=config.get('scan_order','nearest')
73 if scan_order not in ('nearest','remaining_open_path'):raise ValueError('Unknown scan_order')
74 if scan_order!='nearest' and (not joint or order or cache_route!='nearest' or
↪ config.get('routing',{}).get('mode','nearest')!='nearest' or phase_mode!='fixed'):
75     raise ValueError('Scan path ordering requires B nearest routing, fixed stations, no O/cache insertion')
76 if cache_route not in ('nearest','station_insertion'):
77     raise ValueError('Unknown cache_routing')
78 if cache_route!='nearest' and (not shared or not joint or order or modules.get('C',False)):
79     raise ValueError('Cache insertion requires B/S with fixed optical paths and O disabled')
80 shared_limit=config.get('shared_batch_limit',1)
81 if type(shared_limit) is not int or not 1<=shared_limit<=19:
82     raise ValueError('shared_batch_limit must be an integer in [1,19]')
83 current_shared_limit=config.get('current_point_shared_limit',shared_limit)
84 if type(current_shared_limit) is not int or not 0<=current_shared_limit<=19:
85     raise ValueError('current_point_shared_limit must be an integer in [0,19]')
86 if (shared or order) and (not joint or config.get('routing',{}).get('mode','nearest')!='nearest'):
87     raise ValueError('S/O require joint B with nearest routing; legacy H scoring is incompatible')
88 if evidence and config['optical_cell_m']!=28:raise ValueError('Evidence module currently certifies 28m coarse
↪ cells only')
89 client.enter();failure=None;exit_confirmed=False;coverage_emitted=False;consecutive_localizations=0
90 stopping_certificate=None;count_discovery_emitted=False
91 def scan(station_id):
92     nonlocal phase_decided,station_phase
93     if phase_mode!='fixed' and station_id!=0 and not phase_decided:
94         # Only origin evidence exists. No already-observed outer point may move.
95         if visited!={0} or any(ids!={0} for ids in negative.values()):
96             raise Incomplete('Cannot rotate a ring after outer station evidence')
97         old=cover[station_id];replacement,station_phase=align_first_entry(cover,station_id,client.position)
98         cover[:]=replacement;phase_decided=True
99         event(dict(type='ring_phase_selected',phase_rad=station_phase,station_id=station_id,
100                 position=client.position,old_station=old,new_station=cover[station_id],
101                 visited_before=sorted(visited),

```



```

102         local_movement_saved_s=(math.dist(client.position,old)-math.dist(client.position,cover[station_id]
↪ _id]))/5))
103 s=cover[station_id]
104 for channel in range(1,21):
105     # Q3 and Appendix 2 explicitly bound sources by 16 distinct channels.
106     # Returning early does NOT mark this partially scanned station complete.
107     if count_stop!='off' and len(cleared)==16:return
108     if count_stop=='detected_16' and len(set(first)|cleared)==16:return
109     if channel in first or channel in cleared:continue
110     response=client.measure(s,channel,phase='search');kind=response['measure_result']
111     if kind=='no_signal':negative[channel].add(station_id)
112     elif kind=='near':
113         if client.clear(s,channel,phase='near')['clear_result']!='success':raise Incomplete('near followed by
↪ failed clear')
114         cleared.add(channel)
115     elif kind=='direction':first[channel]=(s,response['svd_deg'])
116     else:raise Incomplete('Unknown measure result')
117 visited.add(station_id)
118 if station_id==0 and empty_radius is not None and not first and not cleared:
119     # Origin remains unchanged: its negatives stay valid for either certified ring.
120     cover[:]=stations(empty_radius)
121     event(dict(type='search_ring_switch',reason='all_twenty_origin_channels_no_signal',
122             configured_radius_m=search_radius,selected_radius_m=empty_radius))
123 def prepare(channel,q,r,selection):
124     s,theta=first[channel]
125     planner=None
126     if r['measure_result']=='near':points=[q];certificate=dict(planned_clear_calls=1,covering_radius=5)
127     elif r['measure_result']=='direction':
128         points,certificate=optical_plan(s,theta,q,r['svd_deg'],config['error_deg'],config['optical_cell_m'],confi
↪ g['max_optical_points'])
129         if clear_skip!='off':skip_planes[channel]=constraints(s,theta,q,r['svd_deg'],config['error_deg'])
130         shortcut=single_circle(certificate['vertices']) if modules.get('D',False) and len(points)>1 else None
131         if shortcut:
132             centre,radius=shortcut
133             certificate.update(single_circle_centre=centre,original_grid_calls=len(points),
134                             planned_clear_calls=1,covering_radius=radius)
135             points=[centre]
136         elif evidence and len(points)<=128:
137             planner=EvidenceCover(points,certificate,s,theta,q,r['svd_deg'],[cover[k] for k in
↪ sorted(negative[channel])],config['error_deg'])
138         elif evidence:
139             certificate['evidence_fallback']='more_than_128_coarse_cells_use_original_cover'
140         else:raise Incomplete('Guaranteed-reception second point returned no_signal')
141         certificate.update(channel=channel,first_position=s,second_position=q,selection=selection)
142         regions.append(certificate);event(dict(type='localization',**certificate))
143         return points,certificate,planner
144 def orient(channel,points):
145     if optical_order=='original':return points
146     chosen=nearer_end(points,client.position)
147     event(dict(type='optical_path_order',channel=channel,reversed=chosen is not points,
148             original_entry=points[0],selected_entry=chosen[0],
149             entry_distance_saved_m=math.dist(client.position,points[0])-math.dist(client.position,chosen[0])))
150     return chosen
151 def localize(channel,q,selection):
152     if channel in cached:
153         points,certificate,planner=cached.pop(channel)
154         points=orient(channel,points)
155         event(dict(type='shared_cache_use',channel=channel,measurement_point=certificate['second_position'],
156             optical_entry=points[0]))
157     else:
158         if opportunity_timing=='after_schedule':
159             planned=q
160             q,selection=choose_second(*first[channel],client.position,config)
161             event(dict(type='second_point_after_schedule',channel=channel,scheduled_point=planned,
162                 actual_point=q,selection_mode=selection['mode'],point_changed=q!=planned))
163             r=client.measure(q,channel,phase='localization')
164             points,certificate,planner=prepare(channel,q,r,selection)
165             points=orient(channel,points)
166             batch_limit=current_shared_limit if selection['mode'] in ('opportunistic_current','domain_current') else
↪ shared_limit
167         for _ in range(batch_limit if shared else 0):
168             opportunity=choose_shared(channel,q,first,cleared,cached,config,primary_points=points,decision_log=ev
↪ ent)

```

```

169         if opportunity is None:break
170         other,sel=opportunity
171         response=client.measure(q,other,phase='localization_shared')
172         cached[other]=prepare(other,q,response,sel)
173         event(dict(type='shared_cache_store',channel=other,primary_channel=channel,point=q,
174                 cached_channels=sorted(cached)))
175     for i in range(len(points)):
176         if planner:
177             k,p,info=planner.next_point(client.position);event(dict(type='evidence_choice',channel=channel,point=
178                 ↪ p,**info))
179         else:p=points[i]
180         if clear_skip!='off':
181             proof=failure_certificate(p,skip_planes.get(channel),combine=clear_skip=='separating_pair')
182             if proof is None and radius_priors!='off':
183                 proof=radius_failure_certificate(p,first[channel][0],certificate['second_position'],radius_priors)
184             if proof:
185                 event(dict(type='certified_clear_skip',channel=channel,point=p,original_index=i,error_deg=config[
186                 ↪ 'error_deg'],**proof))
187                 continue
188         if client.clear(p,channel,phase='optical')['clear_result']=='success':
189             cleared.add(channel)
190             if planner:
191                 certificate['evidence']=planner.metrics();event(dict(type='evidence_result',channel=channel,**pla
192                 ↪ nner.metrics()))
193             break
194         if planner:planner.failed(k)
195     else:raise Incomplete('All optical covering points failed: model/protocol inconsistency')
196 try:
197     while True:
198         known=set(first)|cleared
199         if count_stop!='off' and len(known)>16:
200             raise Incomplete('Observed source channels contradict Q3 upper bound 16')
201         if count_stop!='off' and len(cleared)==16:break
202         pending=sorted(set(first)-cleared);unvisited=sorted(set(range(len(cover)))-visited)
203         count_saturated=count_stop=='detected_16' and len(known)==16
204         remaining=[] if count_saturated else unvisited
205         if count_saturated and not count_discovery_emitted:
206             event(dict(type='source_upper_bound_saturated',source_upper_bound=16,
207                 observed_channels=sorted(known),still_to_clear=pending,
208                 unvisited_station_ids=unvisited))
209             count_discovery_emitted=True
210         if not unvisited and not coverage_emitted:
211             event(dict(type='coverage_complete',detected=sorted(first),cleared=sorted(cleared),
212                 absent=[c for c in negative if len(negative[c])==len(cover)]));coverage_emitted=True
213         if not pending and not remaining:break
214         if not joint:
215             if remaining:scan(remaining[0]);continue
216             channel=pending[0];s,theta=first[channel];q,selection=choose_second(s,theta,client.position,planning_
217             ↪ config)
218             localize(channel,q,selection);continue
219         # B: compare task starts; optional gates govern scan interleaving.
220         jobs=[]
221         ready_cache={c:(nearer_end(p,client.position),cert,planner) for c,(p,cert,planner) in cached.items()} if
222         ↪ optical_order=='nearer_end' else cached
223         for c in pending:
224             if c in cached:
225                 q=ready_cache[c][0][0];sel=dict(mode='shared_ready',ready=True)
226                 else:q,sel=choose_second(*first[c],client.position,planning_config)
227                 jobs.append((math.dist(client.position,q),c,q,sel))
228             scheduled_jobs,scheduled_remaining,forced=batch_info=batch_gate(batch_mode,pending,cached,jobs,remaining,
229             ↪ consecutive_localizations)
230             if batch_info:event(dict(type='localization_batch',**batch_info))
231             args=(client.position,client.channel,cover,scheduled_remaining,scheduled_jobs,first,negative,
232                 [c for c in range(1,21) if c not in first and c not in cleared],config)
233             decision=decide_order(*args,cached=ready_cache,forced=forced) if order else decide(*args,forced=forced)
234             if cache_route=='station_insertion':
235                 decision=reorder_cached(decision,client.position,cover,remaining,jobs,ready_cache)
236             task,target,info=decision
237             if task=='scan':
238                 event(dict(type='schedule',task='scan',station_id=target,**info))
239                 scan(target);consecutive_localizations=0
240             else:
241                 _,c,q,sel=target;event(dict(type='schedule',task='localize',channel=c,**info))

```

```

236         localize(c,q,sel);consecutive_localizations+=1
237     if count_stop!='off' and len(cleared)==16:
238         stopping_certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
239             successful_channels=sorted(cleared),
240             inferred_absent_channels=sorted(set(range(1,21))-cleared))
241     elif all(c in cleared or len(negative[c])==len(cover) for c in range(1,21)):
242         stopping_certificate=dict(method='seven_station_channel_cover')
243     else:
244         raise Incomplete('Stopping certificate is incomplete')
245     if not 10<=len(cleared)<=16:raise Incomplete('Cleared count contradicts Q3 count bounds')
246     exit_confirmed=client.exit()['exit_reason']=='user_exit'
247 except (RuntimeError, ValueError, KeyError) as exc:
248     failure=f'{type(exc).__name__}: {exc}'
249 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
250     exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
251     absent_channels=sorted(set(c for c in negative if len(negative[c])==len(cover))|
252         set((stopping_certificate or {}).get('inferred_absent_channels',[]))),
253     negative_station_ids={str(c):sorted(v) for c,v in negative.items()},station_positions=cover,
254     stopping_certificate=stopping_certificate,
255     regions=regions,metrics=client.metrics(),modules=modules)
256 if phase_decided:result['station_phase_rad']=station_phase
257 event(dict(type='strategy_result',**result));return result

```

程序 114

对应文件: question3/innovation/package_smoke/bounded/code_snapshot/self_http_check.py

```

1 """Actual localhost self HTTP check; bind an OS-assigned port, never port 2026."""
2 import json
3 import threading
4 from .experiment import ROOT,PublicClient,dump
5 from .strategy import run
6 from question3.simulator import Simulator
7 from question3.cases import random_case,clustered_case
8 from question3.interface import Client,HttpTransport,JsonLog
9 from question3.self_server import make_server
10 from question3.audit import audit_folder
11 from .experiment import audit_new
12
13 def main():
14     config=json.loads((ROOT/'best_candidate.json').read_text())
15     rows=[]
16     for case in [random_case(0),clustered_case()]:
17         folder=ROOT/'self_http'/case['id'];folder.mkdir(parents=True,exist_ok=False)
18         env=Simulator(case);server=make_server(env,0)
19         thread=threading.Thread(target=server.serve_forever,daemon=True);thread.start()
20         log=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
21         try:
22             assert server.server_port!=2026
23             client=Client(HttpTransport(f'http://127.0.0.1:{server.server_port}'),'SELF',log)
24             result=run(PublicClient(client),config,events)
25         finally:
26             server.shutdown();server.server_close();thread.join();log.close();events.close()
27         truth=env.evaluation();assert result['status']=='complete' and not truth['remaining_channels']
28         dump(folder/'evaluator_hidden_case.json',case)
29         summary=dict(case_id=case['id'],rounding='bounded',status=result['status'],virtual_time_s=truth['virtual_time']
30             ↳ _s'],
31             total=truth['total'],cleared=truth['cleared'],self_port=server.server_port,official_runs=0)
32         dump(folder/'result.json',dict(summary=summary,strategy=result))
33         summary['audited_actions']=audit_folder(folder);summary['events']=audit_new(folder);rows.append(summary)
34         dump(ROOT/'self_http_check.json',rows);print(json.dumps(rows,indent=2))
35 if __name__=='__main__':main()

```

程序 115

对应文件: question3/innovation/package_smoke/bounded/code_snapshot/test_adversarial.py; question3/innovation/test_adversarial.py; question3/results/

dispatch_20260913/development/code_snapshot/question3/innovation/test_adversarial.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/test_adversarial.py

```

1 """Self-only tests for nontrivial geometry, capability boundaries and safe failure."""
2 import ast
3 import json
4 import math
5 import random
6 import unittest
7 from pathlib import Path
8 import numpy as np
9 from .experiment import ROOT, PublicClient
10 from .geometry import initial, wedge, bbox, parallax_point, contains, worthwhile, enclosing
11 from .coverage import covered, verify_cover_boxes
12 from .strategy import run
13 from question3.interface import Client
14 from question3.simulator import Simulator
15 from question3.cases import random_case, boundary_case, late_source_case, near_case, clustered_case
16 from question3.geometry import stations
17
18 BASE=json.loads((ROOT/'frozen/question3/configs/recommended_omni.json').read_text())
19 CONFIG=json.loads((ROOT/'best_candidate.json').read_text()) if (ROOT/'best_candidate.json').exists() else
    ↳ dict(baseline_config=BASE, mechanism='gated_tour_shared_parallax')
20
21 class Adversarial(unittest.TestCase):
22     def test_extreme_bounded_geometry(self):
23         rng=random.Random(611)
24         for _ in range(600):
25             theta=rng.uniform(0,360);r=rng.choice([5.0001,999.999,1000.,1499.999,1500.])
26             angle=math.radians(theta);s=np.array([0.,0.]);g=np.array([r*math.cos(angle),r*math.sin(angle)])
27             read=(theta+rng.choice([-1.005,1.005]))%360
28             poly=initial(s,read)
29             for step in range(12):
30                 self.assertTrue(contains(poly,g))
31                 q,R=bbox(poly,read)[:2]
32                 if R<20-1e-5:
33                     self.assertLess(math.dist(q,g),20);break
34                 if step==0;q,R=parallax_point(poly,read,s)
35                 self.assertLess(R,1000)
36                 actual=math.degrees(math.atan2(*(g-q)[::-1]))
37                 read=(actual+rng.choice([-1.005,1.005]))%360
38                 poly=wedge(poly,q,read)
39             else:self.fail('No contraction to terminal set in 12 steps')
40
41     def test_wrap_and_near_boundary(self):
42         for theta in [0.,.005,89.999,180.,270.,359.995]:
43             for sign in [-1,1]:
44                 s=np.array([1125.,0.]);g=np.array([1800.,0.])
45                 actual=math.degrees(math.atan2(*(g-s)[::-1]))
46                 p=initial(s,(actual+sign*1.005)%360)
47                 self.assertTrue(contains(p,g))
48                 angle=math.radians(theta)
49                 g=np.array([1500*math.cos(angle),1500*math.sin(angle)])
50                 self.assertTrue(contains(initial([0,0],theta+1.005),g))
51
52     def test_information_gate_covers_outputs(self):
53         rng=random.Random(911)
54         for _ in range(80):
55             theta=rng.uniform(0,360);poly=initial([0,0],theta);q,R=parallax_point(poly,theta,[0,0])
56             if not worthwhile(poly,theta,q):continue
57             old=bbox(poly,theta)[1]
58             for g in list(poly)+[(a+b)/2 for a,b in zip(poly,np.roll(poly,-1,axis=0))]:
59                 bearing=math.degrees(math.atan2(*(g-q)[::-1]))
60                 for error in (-1.005,0,1.005):
61                     post=wedge(poly,q,bearing+error)
62                     # Gate proves existence of a covering circle, not the radius of a newly rotated box.
63                     self.assertLessEqual(enclosing(post)[1],max(20,old/2)+1e-5)
64
65     def test_coverage_certificate_independent(self):
66         pts=stations(1125)
67         self.assertTrue(covered(pts))

```

```

68     self.assertGreater(verify_cover_boxes(pts)['covered_boxes'],0)
69     for missing in range(7):self.assertFalse(covered(pts[:missing]+pts[missing+1:]))
70     shifted=list(pts);shifted[-1]=(shifted[-1][0]+.01,shifted[-1][1])
71     self.assertTrue(covered(shifted));verify_cover_boxes(shifted)
72
73     def test_policy_has_no_oracle_or_file_access(self):
74         forbidden={'open','eval','exec','getattr','setattr','__import__'}
75         for name in ('strategy.py','geometry.py','routing.py','coverage.py'):
76             tree=ast.parse((ROOT/name).read_text())
77             for node in ast.walk(tree):
78                 if isinstance(node,ast.Call) and
79                 ↪ isinstance(node.func,ast.Name):self.assertNotIn(node.func.id,forbidden)
80                 if isinstance(node,(ast.Import,ast.ImportFrom)):
81                     names=[a.name for a in node.names] if isinstance(node,ast.Import) else [node.module or '']
82                     self.assertFalse(any(any(k in n for k in
83                     ↪ ['simulator','cases','fixtures','pathlib','os','random','experiment']) for n in names))
84                 if isinstance(node,ast.Attribute):self.assertNotIn(node.attr,{'transport','_sources','_seed','_error_']
85                 ↪ mode','evaluation','__dict__'})
86             c=PublicClient(Client(Simulator(random_case(0)),'SELF'))
87             self.assertFalse(hasattr(c,'transport'));self.assertFalse(hasattr(c,'evaluation'))
88
89     def test_same_point_error_is_fixed(self):
90         for mode in ['positive','negative','smooth','spatial_hash']:
91             env=Simulator(random_case(0,mode));p=(100.125,-232.5)
92             self.assertEqual(env._bias(p,7),env._bias(p,7))
93             self.assertEqual(env._bias((0.,0.),7),env._bias((-0.,0.),7))
94
95     def test_complete_adversarial_families(self):
96         for case in [boundary_case(),late_source_case(),near_case(),clustered_case()]:
97             env=Simulator(case);result=run(PublicClient(Client(env,'SELF')),CONFIG)
98             self.assertEqual(result['status'],'complete',result['failure'])
99             self.assertEqual(env.evaluation()['remaining_channels'],[])
100             self.assertTrue(result['exit_confirmed'])
101
102     def test_deadline_never_reports_complete(self):
103         env=Simulator(random_case(0),remaining_s=0)
104         result=run(PublicClient(Client(env,'SELF')),CONFIG)
105         self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed'])
106
107     def test_rejected_clear_never_reports_complete(self):
108         class Rejected(Simulator):
109             def request(self,path,payload,timeout=5):
110                 if path=='/clear':return 200,dict(accepted=False,virtual_time_s=0)
111                 return super().request(path,payload,timeout)
112         env=Rejected(random_case(0));result=run(PublicClient(Client(env,'SELF')),CONFIG)
113         self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed'])
114
115     def test_certified_no_signal_never_reports_complete(self):
116         class Corrupted(Simulator):
117             def request(self,path,payload,timeout=5):
118                 status,response=super().request(path,payload,timeout)
119                 if path=='/measure':
120                     p=(payload['position']['x'],payload['position']['y'])
121                     if min(math.dist(p,s) for s in stations(1125))>1e-4:
122                         response['measure_result']='no_signal';response.pop('svd_deg',None)
123                     return status,response
124         env=Corrupted(random_case(0));result=run(PublicClient(Client(env,'SELF')),CONFIG)
125         self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed'])
126
127 if __name__=='__main__':unittest.main(verbosity=2)

```

程序 116

对应文件: question3/innovation/round01_homing/bounded/code_snapshot/experiment.py; question3/innovation/round01_homing/bounded_full/code_snapshot/experiment.py; question3/innovation/round01_homing/pre_round_stress/code_snapshot/experiment.py; question3/innovation/round02_parallax/bounded/code_snapshot/experiment.py; question3/innovation/round03_refresh/bounded/code_snapshot/experiment.py; question3/innovation/round04_shared/bounded/code_s

napshot/experiment.py; question3/innovation/round05_strip/bounded/code_snapshot/experiment.py; question3/innovation/round06_belief_tour/bounded/code_snapshot/experiment.py

```

1  """Evaluator-only self experiments. Strategy inputs exclude cases and evaluator state."""
2  import argparse
3  import hashlib
4  import json
5  import math
6  import shutil
7  import statistics
8  import sys
9  import time
10 from pathlib import Path
11 import numpy as np
12
13 ROOT=Path(__file__).resolve().parent
14 sys.path.insert(0,str(ROOT/'frozen'))
15 from question3.cases import development_cases
16 from question3.simulator import Simulator
17 from question3.interface import Client,JsonLog
18 from question3.strategy import run as baseline_run
19 from question3.audit import audit_folder
20 from .strategy import run as candidate_run
21
22 class PublicClient:
23     """Narrow public observation/action capability; no transport/evaluation accessor."""
24     __slots__=('__client',)
25     def __init__(self,client):self.__client=client
26     @property
27     def position(self):return self.__client.position
28     @property
29     def channel(self):return self.__client.channel
30     def enter(self):return self.__client.enter()
31     def measure(self,*a,**kw):return self.__client.measure(*a,**kw)
32     def clear(self,*a,**kw):return self.__client.clear(*a,**kw)
33     def exit(self):return self.__client.exit()
34     def metrics(self):return self.__client.metrics()
35
36 def dump(path,data):
37     path.parent.mkdir(parents=True,exist_ok=True)
38     path.write_text(json.dumps(data,ensure_ascii=False,indent=2))
39
40 def audit_new(folder):
41     case=json.loads((folder/'evaluator_hidden_case.json').read_text())
42     truth={s['channel']:np.array([s['x'],s['y']]) for s in case['sources']}
43     events=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
44     counts={}
45     for e in events:
46         kind=e['type'];counts[kind]=counts.get(kind,0)+1
47         if kind not in ('feasible_update','certified_measure','certified_clear'):
48             continue
49         v=np.asarray(e['vertices']);g=truth[e['channel']]
50         assert len(v)>0 and np.isfinite(v).all()
51         edges=np.roll(v,-1,axis=0)-v;delta=g-v
52         cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
53         assert np.all(cross>=-1e-4) or np.all(cross<=1e-4),('source excluded',e)
54         if kind in ('certified_measure','certified_clear'):
55             p=np.asarray(e['point']);radius=e['radius']
56             assert np.linalg.norm(v-p,axis=1).max()<=radius+1e-8
57             assert radius<(1000 if kind=='certified_measure' else 20)
58     return counts
59
60 def aggregate(rows,reference='BASE'):
61     groups={n:[r for r in rows if r['variant']==n] for n in sorted({r['variant'] for r in rows})}
62     ref={r['case_id']:r for r in groups[reference]}
63     result={}
64     for n,rs in groups.items():
65         t=[r['penalized_virtual_s'] for r in rs]
66         deltas=[(r['penalized_virtual_s']/ref[r['case_id']]['penalized_virtual_s']-1,r['case_id']) for r in rs]
67         result[n]=dict(runs=len(rs),complete=sum(r['success'] for r in rs),
68             failed=sum(not r['success'] for r in rs),timeouts=sum(r['timeout'] for r in rs),

```

```

69         false_complete=sum(r['false_complete'] for r in rs),
70         mean=float(np.mean(t)),p95=float(np.percentile(t,95)),maximum=max(t),
71         worst_case=rs[int(np.argmax(t))]['case_id'],worst_paired_regression=max(deltas),
72         clearance_ratio_mean=float(np.mean([r['cleared']/r['total'] for r in rs])),
73         raw_mean=float(np.mean([r['metrics']['virtual_time_s'] for r in rs])),
74         movement_detection_etc={k:float(np.mean([r['metrics']['components'][k] for r in rs])) for k in
        ↪ rs[0]['metrics']['components']},
75         phase_means={k:float(np.mean([r['metrics']['phases'].get(k,0) for r in rs])) for k in
        ↪ sorted(set().union(*([r['metrics']['phases'] for r in rs])))},
76         mean_per_cleared=float(np.mean([r['metrics']['virtual_time_s']/max(1,r['cleared']) for r in rs])),
77         wall_p95=float(np.percentile([r['elapsed_wall_s'] for r in rs],95)),
78         clear_failure_mean=float(np.mean([r['metrics']['counts']['clear_failure'] for r in rs]))
79     for n,v in result.items():
80         v['mean_change_pct']=100*(v['mean']/result[reference]['mean']-1)
81         v['significant']=n!=reference and v['mean_change_pct']<=-5 and v['complete']>=result[reference]['complete']
        ↪ and v['p95']<=result[reference]['p95']
82     return result
83
84 def main():
85     p=argparse.ArgumentParser()
86     p.add_argument('--round',required=True);p.add_argument('--variants',default='BASE,HOMING')
87     p.add_argument('--rounding',default='bounded',choices=['bounded','pre_round_stress'])
88     p.add_argument('--fixtures');p.add_argument('--limit',type=int)
89     p.add_argument('--stage',default='development');p.add_argument('--label')
90     args=p.parse_args()
91     rounddir=R00T/args.round
92     out=rounddir/(args.label or args.rounding)
93     out.mkdir(parents=True,exist_ok=False)
94     baseline=json.loads((R00T/'frozen/question3/configs/recommended_omni.json').read_text())
95     names=args.variants.split(',')
96     configs={n:(baseline if n=='BASE' else dict(baseline_config=baseline,mechanism=n.lower())) for n in names}
97     dump(out/'configs.json',configs)
98     manifest={str(x.relative_to(R00T)):hashlib.sha256(x.read_bytes()).hexdigest() for x in R00T.glob('*.py')}
99     dump(out/'code_sha256.json',manifest)
100    for file in R00T.glob('*.py'):
101        dest=out/'code_snapshot'/file.name;dest.parent.mkdir(exist_ok=True);shutil.copy2(file,dest)
102    cases=json.loads(Path(args.fixtures).read_text()) if args.fixtures else list(development_cases())
103    if args.limit:cases=cases[:args.limit]
104    dump(out/'fixtures.json',cases)
105    dump(out/'metadata.json',dict(stage=args.stage,rounding=args.rounding,
        distinct_layouts=len({json.dumps(c['sources'],sort_keys=True) for c in cases}),
        case_sha256={c['id']:hashlib.sha256(json.dumps(c,sort_keys=True).encode()).hexdigest() for c in cases},
        official_runs=0,formal_runs=0,numeric_parameter_search_trials=0))
106    rows=[]
107    for i,case in enumerate(cases):
108        order=names[i%len(names):]+names[:i%len(names)]
109        for name in order:
110            folder=out/name/case['id'];folder.mkdir(parents=True)
111            dump(folder/'evaluator_hidden_case.json',case)
112            log=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
113            env=Simulator(case,rounding=args.rounding)
114            client=Client(env,'SELF',log,5.)
115            start=time.perf_counter()
116            try:
117                result=(baseline_run if name=='BASE' else candidate_run)(PublicClient(client),configs[name],events)
118            except Exception as exc:
119                result=dict(status='incomplete',failure=f'{type(exc).__name__}:
                ↪ {exc}',metrics=client.metrics(),regions=[])
120            wall=time.perf_counter()-start
121            truth=env.evaluation() # First evaluator access, after the policy returned.
122            row=dict(variant=name,case_id=case['id'],stage=args.stage,rounding=args.rounding,
                status=result['status'],failure=result.get('failure'),elapsed_wall_s=wall,
                total=truth['total'],cleared=truth['cleared'],metrics=result['metrics'])
123            row['false_complete']=result['status']=='complete' and bool(truth['remaining_channels'])
124            row['timeout']=bool(result.get('failure') and any(x in result['failure'].lower() for x in
                ↪ ('deadline','timeout','budget')))
125            row['success']=result['status']=='complete' and not truth['remaining_channels']
126            dump(folder/'result.json',dict(summary=row,strategy=result))
127            log.close();events.close()
128            try:
129                if row['success']:
130                    row['audited_actions']=audit_folder(folder)
131                    row['event_counts']=audit_new(folder)
132

```

```

137         row['audit']='pass'
138     else:row['audit']='incomplete retained'
139 except Exception as exc:
140     row['audit']=f'FAIL {type(exc).__name__}: {exc}'
141     row['success']=False;row['false_complete']=True
142     row['penalized_virtual_s']=truth['virtual_time_s'] if row['success'] else
    ↪ max(360000,truth['virtual_time_s'])
143     dump(folder/'result.json',dict(summary=row,strategy=result))
144     rows.append(row)
145     dump(out/'summary.json',dict(rows=rows,aggregates=aggregate(rows)))
146     print(f'{i+1}/{len(cases)} {case["id"]} '+'', '.join(f'{r["variant"]}'={r["penalized_virtual_s"]:.1f}'+('
    ↪ '+str(r['failure'])'+ ' '+r['audit'] if not r['success'] else '') for r in rows[-len(names):]),flush=True)
147 stats=aggregate(rows);dump(out/'comparison.json',stats)
148 print(json.dumps({n:{k:v[k] for k in ('complete','runs','mean','mean_change_pct','p95','maximum','significant')}}
    ↪ for n,v in stats.items()),indent=2),flush=True)
149
150 if __name__=='__main__':main()

```

程序 117

对应文件: question3/innovation/round01_homing/bounded/code_snapshot/geomet
ry.py; question3/innovation/round01_homing/bounded_full/code_snapshot/geo
metry.py; question3/innovation/round01_homing/pre_round_stress/code_snapsh
ot/geometry.py

```

1 """Conservative convex sets from public bounded bearings. No environment access."""
2 import itertools
3 import math
4 import numpy as np
5
6 SLACK = 1e-7
7
8 def clip(poly, normal, bound):
9     normal = np.asarray(normal, float)
10    result = []
11    for a,b in zip(poly, np.roll(poly,-1,axis=0)):
12        da, db = float(normal@a-bound), float(normal@b-bound)
13        if da <= 0:
14            result.append(a)
15        if (da <= 0) != (db <= 0):
16            result.append(a + (b-a)*(da/(da-db)))
17    if not result:
18        raise RuntimeError('Empty conservative feasible polygon')
19    return np.asarray(result)
20
21 def disk(poly, centre, radius):
22     # Tangent halfplanes OUTSIDE the disk; never use an inscribed approximation.
23     centre = np.asarray(centre)
24     for k in range(32):
25         t = 2*math.pi*k/32
26         n = np.array([math.cos(t),math.sin(t)])
27         poly = clip(poly,n,float(n@centre+radius+SLACK))
28     return poly
29
30 def wedge(poly, point, bearing, error=1.005):
31     for sign in (-1,1):
32         t = math.radians(bearing+sign*error)
33         n = np.array([-sign*math.sin(t),sign*math.cos(t)])
34         poly = clip(poly,n,float(n@np.asarray(point)+SLACK))
35     return poly
36
37 def initial(point, bearing, error=1.005):
38     poly = np.array([[-1800.,-1800.],[1800.,-1800.],[1800.,1800.],[-1800.,1800.]])
39     poly = wedge(poly,point,bearing,error)
40     return disk(disk(poly,(0,0),1800),point,1500)
41
42 def bbox(poly, bearing):
43     t = math.radians(bearing)
44     rot = np.array([[math.cos(t),-math.sin(t)],[math.sin(t),math.cos(t)]]

```



```

45     local = poly@rot
46     lo,hi = local.min(axis=0)-SLACK,local.max(axis=0)+SLACK
47     centre = ((lo+hi)/2)@rot.T
48     radius = float(np.linalg.norm(poly-centre,axis=1).max()+SLACK)
49     return centre,radius,lo,hi,rot
50
51 def enclosing(poly):
52     # Small convex polygons: enumerate every support pair/triple, then verify all vertices.
53     candidates = [p for p in poly]
54     candidates += [(a+b)/2 for a,b in itertools.combinations(poly,2)]
55     for a,b,c in itertools.combinations(poly,3):
56         u,v = b-a,c-a
57         A=2*np.stack([u,v])
58         if abs(float(np.linalg.det(A))) < 1e-9:
59             continue
60         p=a+np.linalg.solve(A,[u@u,v@v])
61         if np.isfinite(p).all():
62             candidates.append(p)
63     best=min((float(np.linalg.norm(poly-p,axis=1).max()),tuple(p)) for p in candidates)
64     return np.array(best[1]),best[0]+SLACK
65
66 def grid(poly,bearing):
67     _,_,lo,hi,rot=bbox(poly,bearing)
68     sizes=np.maximum(1,np.ceil((hi-lo)/28).astype(int))
69     if int(np.prod(sizes))>2000:
70         raise RuntimeError('Optical finite fallback guard')
71     for j in range(sizes[1]):
72         for i in (range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))):
73             yield (lo+np.array([i+.5,j+.5])*(hi-lo)/sizes)@rot.T
74
75 def contains(poly,p,tol=1e-4):
76     e=np.roll(poly,-1,axis=0)-poly
77     v=np.asarray(p)-poly
78     cross=e[:,0]*v[:,1]-e[:,1]*v[:,0]
79     return bool(np.all(cross>=tol) or np.all(cross<=tol))

```

程序 118

对应文件: question3/innovation/round01_homing/bounded/code_snapshot/strategy.py; question3/innovation/round01_homing/bounded_full/code_snapshot/strategy.py; question3/innovation/round01_homing/pre_round_stress/code_snapshot/strategy.py

```

1  """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2  import math
3  import numpy as np
4  from question3.geometry import stations,station_cover_bound
5  from question3.modules import choose_second
6  from .geometry import initial,wedge,bbox,enclosing,grid
7
8  def run(client,config,event=lambda row=None):
9      cover=stations(1125)
10     assert station_cover_bound(1125)<1000
11     first={};polys={};bearings={};measured={};cleared=set();visited=set()
12     negative={c:set() for c in range(1,21)}
13     failure=None;exit_confirmed=False;certificate=None
14     base=config['baseline_config']
15     mode=config.get('mechanism','homing')
16     def centre(c):
17         return bbox(polys[c],bearings[c])[2:]
18     def update(c,p,response):
19         kind=response['measure_result']
20         if kind=='near':
21             if client.clear(p,c,phase='near')['clear_result']!='success':
22                 raise RuntimeError('Near feedback contradicted by failed clear')
23             cleared.add(c)
24             return
25         if kind!='direction':
26             raise RuntimeError('Certified reception returned no_signal')

```

```

27     theta=response['svd_deg']
28     if c not in first:
29         first[c]=(tuple(p),theta)
30         polys[c]=initial(p,theta,base['error_deg'])
31         measured[c]=set()
32     else:
33         polys[c]=wedge(polys[c],p,theta,base['error_deg'])
34         measured[c].add(tuple(p));bearings[c]=theta
35         event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
36 def scan(k):
37     p=cover[k]
38     for c in range(1,21):
39         if len(set(first)|cleared)==16:
40             return
41         if c in first or c in cleared:
42             continue
43         r=client.measure(p,c,phase='search')
44         if r['measure_result']=='no_signal':negative[c].add(k)
45         else:update(c,p,r)
46     visited.add(k)
47 def localize(c):
48     for step in range(16):
49         p,radius=centre(c)
50         if radius<=20*1e-5:
51             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
52             ↵ ))
53             if client.clear(p,c,phase='optical')['clear_result']!='success':
54                 raise RuntimeError('Certified containing circle clear failed')
55             cleared.add(c);return
56         if radius>=1000*1e-5:
57             raise RuntimeError('Adaptive measurement lacks reception certificate')
58         if tuple(p) in measured[c]:
59             break
60         event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
61         update(c,p,client.measure(p,c,phase='adaptive_localization'))
62         if c in cleared:return
63         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
64         for p in grid(polys[c],bearings[c]):
65             if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
66                 cleared.add(c);return
67         raise RuntimeError('Finite optical cover exhausted without success')
68 client.enter()
69 try:
70     while True:
71         known=set(first)|cleared
72         if len(known)>16:raise RuntimeError('Source count upper bound contradicted')
73         pending=sorted(set(first)-cleared)
74         remaining=[] if len(known)==16 else sorted(set(range(7))-visited)
75         if not pending and not remaining:break
76         jobs=[]
77         for c in pending:
78             # R1 keeps the baseline's pre-schedule second-point proxy.
79             planning=(*base,'second_point_opportunity':'off','second_range_prior':'off')
80             q,_=choose_second(*first[c],client.position,planning)
81             jobs.append((math.dist(client.position,q),'localize',c))
82             jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]
83             _,task,target=min(jobs)
84             event(dict(type='schedule',task=task,target=target))
85             if task=='scan':scan(target)
86             else:localize(target)
87         if len(cleared)==16:
88             certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
89                             successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
90         elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
91             certificate=dict(method='seven_station_channel_cover')
92         else:raise RuntimeError('Incomplete stopping certificate')
93         if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
94         exit_confirmed=client.exit()['exit_reason']=='user_exit'
95 except (RuntimeError,ValueError,KeyError) as exc:
96     failure=f'{type(exc).__name__}: {exc}'
97 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
98            exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
99            absent_channels=sorted(c for c in negative if len(negative[c])==7),

```

```

99         negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
100         station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
101     event(dict(type='strategy_result',**result))
102     return result

```

程序 119

对应文件: question3/innovation/round02_parallax/bounded/code_snapshot/geometry.py; question3/innovation/round03_refresh/bounded/code_snapshot/geometry.py; question3/innovation/round04_shared/bounded/code_snapshot/geometry.py; question3/innovation/round05_strip/bounded/code_snapshot/geometry.py; question3/innovation/round06_belief_tour/bounded/code_snapshot/geometry.py; question3/innovation/round07_reception_bracket/bounded/code_snapshot/geometry.py; question3/innovation/round08_survey/bounded/code_snapshot/geometry.py

```

1  """Conservative convex sets from public bounded bearings. No environment access."""
2  import itertools
3  import math
4  import numpy as np
5
6  SLACK = 1e-7
7
8  def clip(poly, normal, bound):
9      normal = np.asarray(normal, float)
10     result = []
11     for a,b in zip(poly, np.roll(poly,-1,axis=0)):
12         da, db = float(normal@a-bound), float(normal@b-bound)
13         if da <= 0:
14             result.append(a)
15         if (da <= 0) != (db <= 0):
16             result.append(a + (b-a)*(da/(da-db)))
17     if not result:
18         raise RuntimeError('Empty conservative feasible polygon')
19     return np.asarray(result)
20
21 def disk(poly, centre, radius):
22     # Tangent halfplanes OUTSIDE the disk; never use an inscribed approximation.
23     centre = np.asarray(centre)
24     for k in range(32):
25         t = 2*math.pi*k/32
26         n = np.array([math.cos(t),math.sin(t)])
27         poly = clip(poly,n,float(n@centre+radius+SLACK))
28     return poly
29
30 def wedge(poly, point, bearing, error=1.005):
31     for sign in (-1,1):
32         t = math.radians(bearing+sign*error)
33         n = np.array([-sign*math.sin(t),sign*math.cos(t)])
34         poly = clip(poly,n,float(n@np.asarray(point)+SLACK))
35     return poly
36
37 def initial(point, bearing, error=1.005):
38     poly = np.array([[ -1800., -1800.],[1800., -1800.],[1800.,1800.],[ -1800.,1800.]])
39     poly = wedge(poly,point,bearing,error)
40     return disk(disk(poly,(0,0),1800),point,1500)
41
42 def bbox(poly, bearing):
43     t = math.radians(bearing)
44     rot = np.array([math.cos(t),-math.sin(t)],[math.sin(t),math.cos(t)])
45     local = poly@rot
46     lo,hi = local.min(axis=0)-SLACK,local.max(axis=0)+SLACK
47     centre = ((lo+hi)/2)@rot.T
48     radius = float(np.linalg.norm(poly-centre,axis=1).max()+SLACK)
49     return centre,radius,lo,hi,rot
50

```

```

51 def enclosing(poly):
52     # Small convex polygons: enumerate every support pair/triple, then verify all vertices.
53     candidates = [p for p in poly]
54     candidates += [(a+b)/2 for a,b in itertools.combinations(poly,2)]
55     for a,b,c in itertools.combinations(poly,3):
56         u,v = b-a,c-a
57         A=2*np.stack([u,v])
58         if abs(float(np.linalg.det(A))) < 1e-9:
59             continue
60         p=a+np.linalg.solve(A,[u@u,v@v])
61         if np.isfinite(p).all():
62             candidates.append(p)
63     best=min((float(np.linalg.norm(poly-p,axis=1).max()),tuple(p)) for p in candidates)
64     return np.array(best[1]),best[0]+SLACK
65
66 def grid(poly,bearing):
67     _,_,lo,hi,rot=bbox(poly,bearing)
68     sizes=np.maximum(1,np.ceil((hi-lo)/28).astype(int))
69     if int(np.prod(sizes))>2000:
70         raise RuntimeError('Optical finite fallback guard')
71     for j in range(sizes[1]):
72         for i in (range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))):
73             yield (lo+np.array([i+.5,j+.5])*(hi-lo)/sizes)@rot.T
74
75 def contains(poly,p,tol=1e-4):
76     e=np.roll(poly,-1,axis=0)-poly
77     v=np.asarray(p)-poly
78     cross=e[:,0]*v[:,1]-e[:,1]*v[:,0]
79     return bool(np.all(cross>=-tol) or np.all(cross<=tol))
80
81 def parallax_point(poly,bearing,current):
82     """Geometry-scaled parallax: balance cross-track detour and angular uncertainty.
83
84     Baseline scale derives from r*sin(delta), with 20m as the physical accuracy target.
85     No source-distance or error-mode oracle. Candidates require a full vertex reception check.
86     """
87     centre,radius,*_=bbox(poly,bearing)
88     t=math.radians(bearing);side=np.array([-math.sin(t),math.cos(t)])
89     # sqrt(range * optical radius) gives a scale between optical accuracy and range.
90     offset=math.sqrt(radius*20)
91     candidates=[centre+sign*offset*side for sign in (-1,1)]
92     valid=[p for p in candidates if np.linalg.norm(poly-p,axis=1).max()<1000-1e-5]
93     if not valid:return centre,radius
94     p=min(valid,key=lambda p:math.dist(current,p))
95     return p,float(np.linalg.norm(poly-p,axis=1).max()+SLACK)

```

程序 120

对应文件: question3/innovation/round02_parallax/bounded/code_snapshot/strategy.py; question3/innovation/round03_refresh/bounded/code_snapshot/strategy.py; question3/innovation/round04_shared/bounded/code_snapshot/strategy.py; question3/innovation/round05_strip/bounded/code_snapshot/strategy.py

```

1 """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2 import math
3 import numpy as np
4 from question3.geometry import stations,station_cover_bound
5 from question3.modules import choose_second
6 from .geometry import initial,wedge,bbox,enclosing,grid,parallax_point
7
8 def run(client,config,event=lambda row:None):
9     cover=stations(1125)
10    assert station_cover_bound(1125)<1000
11    first={};polys={};bearings={};measured={};cleared=set();visited=set()
12    negative={c:set() for c in range(1,21)}
13    failure=None;exit_confirmed=False;certificate=None
14    base=config['baseline_config']
15    mode=config.get('mechanism','homing')
16    refresh=mode in ('refresh','refresh_parallax','shared','shared_parallax')

```

```

17 parallax=mode in ('parallax','refresh_parallax','shared_parallax')
18 sharing=mode in ('shared','shared_parallax')
19 def centre(c):
20     return bbox(polys[c],bearings[c])[2]
21 def update(c,p,response):
22     kind=response['measure_result']
23     if kind=='near':
24         if client.clear(p,c,phase='near')['clear_result']!='success':
25             raise RuntimeError('Near feedback contradicted by failed clear')
26         cleared.add(c)
27         return
28     if kind!='direction':
29         raise RuntimeError('Certified reception returned no_signal')
30     theta=response['svd_deg']
31     if c not in first:
32         first[c]=(tuple(p),theta)
33         polys[c]=initial(p,theta,base['error_deg'])
34         measured[c]=set()
35     else:
36         polys[c]=wedge(polys[c],p,theta,base['error_deg'])
37     measured[c].add(tuple(p));bearings[c]=theta
38     event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
39 def scan(k):
40     p=cover[k]
41     for c in range(1,21):
42         if len(set(first)|cleared)==16:
43             return
44         if c in first or c in cleared:
45             continue
46         r=client.measure(p,c,phase='search')
47         if r['measure_result']=='no_signal':negative[c].add(k)
48         else:update(c,p,r)
49     visited.add(k)
50 def localize(c):
51     if mode=='strip':
52         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
53         for p in grid(polys[c],bearings[c]):
54             if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
55                 cleared.add(c);return
56         raise RuntimeError('Single-bearing finite cover exhausted')
57     if refresh and tuple(client.position) not in measured[c]:
58         radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
59         if radius<1000-1e-5:
60             event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↳ s[c].tolist()))
61             update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
62             if c in cleared:return
63     for step in range(16):
64         p,radius=centre(c)
65         if radius<=20-1e-5:
66             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↳ ))
67             if client.clear(p,c,phase='optical')['clear_result']!='success':
68                 raise RuntimeError('Certified containing circle clear failed')
69             cleared.add(c);return
70     if parallax and step==0:
71         p,radius=parallax_point(polys[c],bearings[c],client.position)
72     if radius>=1000-1e-5:
73         raise RuntimeError('Adaptive measurement lacks reception certificate')
74     if tuple(p) in measured[c]:
75         break
76     event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
77     update(c,p,client.measure(p,c,phase='adaptive_localization'))
78     if c in cleared:return
79     if sharing:
80         for other in sorted(set(first)-cleared-{c}):
81             if tuple(client.position) in measured[other]:continue
82             rr=float(np.linalg.norm(polys[other]-np.asarray(client.position),axis=1).max()+1e-7)
83             if rr>=1000-1e-5:continue
84             event(dict(type='certified_measure',channel=other,point=list(client.position),radius=rr,vertices=
↳ polys[other].tolist()))
85             update(other,client.position,client.measure(client.position,other,phase='multi_target_refinement'
↳ ))

```

```

86     event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
87     for p in grid(polys[c],bearings[c]):
88         if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
89             cleared.add(c);return
90     raise RuntimeError('Finite optical cover exhausted without success')
91 client.enter()
92 try:
93     while True:
94         known=set(first)|cleared
95         if len(known)>16:raise RuntimeError('Source count upper bound contradicted')
96         pending=sorted(set(first)-cleared)
97         remaining=[] if len(known)==16 else sorted(set(range(7))-visited)
98         if not pending and not remaining:break
99         jobs=[]
100        for c in pending:
101            # R1 keeps the baseline's pre-schedule second-point proxy.
102            planning=(*base,'second_point_opportunity':'off','second_range_prior':'off')
103            q,_=choose_second(*first[c],client.position,planning)
104            jobs.append((math.dist(client.position,q),'localize',c))
105        jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]
106        _,task,target=min(jobs)
107        event(dict(type='schedule',task=task,target=target))
108        if task=='scan':scan(target)
109        else:localize(target)
110    if len(cleared)==16:
111        certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
112            successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
113    elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
114        certificate=dict(method='seven_station_channel_cover')
115    else:raise RuntimeError('Incomplete stopping certificate')
116    if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
117    exit_confirmed=client.exit()['exit_reason']=='user_exit'
118 except (RuntimeError,ValueError,KeyError) as exc:
119     failure=f'{type(exc).__name__}: {exc}'
120 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
121     exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
122     absent_channels=sorted(c for c in negative if len(negative[c])==7),
123     negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
124     station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
125 event(dict(type='strategy_result',**result))
126 return result

```

程序 121

对应文件: question3/innovation/round06_belief_tour/bounded/code_snapshot/s
trategy.py

```

1 """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2 import math
3 import numpy as np
4 from question3.geometry import stations,station_cover_bound
5 from question3.modules import choose_second
6 from .geometry import initial,wedge,bbox,enclosing,grid,parallax_point
7 from .routing import choose as choose_route
8
9 def run(client,config,event=lambda row:None):
10     cover=stations(1125)
11     assert station_cover_bound(1125)<1000
12     first={};polys={};bearings={};measured={};cleared=set();visited=set()
13     negative={c:set() for c in range(1,21)}
14     failure=None;exit_confirmed=False;certificate=None
15     base=config['baseline_config']
16     mode=config.get('mechanism','homing')
17     route_mode='global' if mode.startswith('tour_') else ('nearest' if mode.startswith('nearest_') else None)
18     if route_mode:mode=mode.split('_',1)[1]
19     refresh=mode in ('refresh','refresh_parallax','shared','shared_parallax')
20     parallax=mode in ('parallax','refresh_parallax','shared_parallax')
21     sharing=mode in ('shared','shared_parallax')
22     def centre(c):
23         return bbox(polys[c],bearings[c])[2]
24     def update(c,p,response):

```

```

25     kind=response['measure_result']
26     if kind=='near':
27         if client.clear(p,c,phase='near')['clear_result']!='success':
28             raise RuntimeError('Near feedback contradicted by failed clear')
29         cleared.add(c)
30         return
31     if kind!='direction':
32         raise RuntimeError('Certified reception returned no_signal')
33     theta=response['svd_deg']
34     if c not in first:
35         first[c]=(tuple(p),theta)
36         polys[c]=initial(p,theta,base['error_deg'])
37         measured[c]=set()
38     else:
39         polys[c]=wedge(polys[c],p,theta,base['error_deg'])
40     measured[c].add(tuple(p));bearings[c]=theta
41     event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
42 def scan(k):
43     p=cover[k]
44     for c in range(1,21):
45         if len(set(first)|cleared)==16:
46             return
47         if c in first or c in cleared:
48             continue
49         r=client.measure(p,c,phase='search')
50         if r['measure_result']=='no_signal':negative[c].add(k)
51         else:update(c,p,r)
52     visited.add(k)
53 def localize(c):
54     if mode=='strip':
55         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
56         for p in grid(polys[c],bearings[c]):
57             if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
58                 cleared.add(c);return
59             raise RuntimeError('Single-bearing finite cover exhausted')
60     if refresh and tuple(client.position) not in measured[c]:
61         radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
62         if radius<1000-1e-5:
63             event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↳ s[c].tolist()))
64             update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
65             if c in cleared:return
66     for step in range(16):
67         p,radius=centre(c)
68         if radius<=20-1e-5:
69             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↳ ))
70             if client.clear(p,c,phase='optical')['clear_result']!='success':
71                 raise RuntimeError('Certified containing circle clear failed')
72             cleared.add(c);return
73     if parallax and step==0:
74         p,radius=parallax_point(polys[c],bearings[c],client.position)
75         if radius>=1000-1e-5:
76             raise RuntimeError('Adaptive measurement lacks reception certificate')
77     if tuple(p) in measured[c]:
78         break
79     event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
80     update(c,p,client.measure(p,c,phase='adaptive_localization'))
81     if c in cleared:return
82     if sharing:
83         for other in sorted(set(first)-cleared-{c}):
84             if tuple(client.position) in measured[other]:continue
85             rr=float(np.linalg.norm(polys[other]-np.asarray(client.position),axis=1).max()+1e-7)
86             if rr>=1000-1e-5:continue
87             event(dict(type='certified_measure',channel=other,point=list(client.position),radius=rr,vertices=
↳ polys[other].tolist()))
88             update(other,client.position,client.measure(client.position,other,phase='multi_target_refinement'
↳ ))
89     event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
90     for p in grid(polys[c],bearings[c]):
91         if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
92             cleared.add(c);return
93     raise RuntimeError('Finite optical cover exhausted without success')

```

```

94     client.enter()
95     try:
96         while True:
97             known=set(first)|cleared
98             if len(known)>16:raise RuntimeError('Source count upper bound contradicted')
99             pending=sorted(set(first)-cleared)
100             remaining=[] if len(known)==16 else sorted(set(range(7))-visited)
101             if not pending and not remaining:break
102             jobs=[]
103             for c in pending:
104                 # R1 keeps the baseline's pre-schedule second-point proxy.
105                 planning={**base,'second_point_opportunity':'off','second_range_prior':'off'}
106                 q,_choose_second(*first[c],client.position,planning)
107                 jobs.append((math.dist(client.position,q),'localize',c))
108             jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]
109             if route_mode:
110                 tasks=[('localize',c,centre(c)[0]) for c in pending]
111                 tasks += [('scan',k,cover[k]) for k in remaining]
112                 (task,target,_),tour=choose_route(client.position,tasks,route_mode=='global')
113                 event(dict(type='belief_route',mode=route_mode,route=tour))
114             else:
115                 _,task,target=min(jobs)
116                 event(dict(type='schedule',task=task,target=target))
117                 if task=='scan':scan(target)
118                 else:localize(target)
119             if len(cleared)==16:
120                 certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
121                                 successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
122             elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
123                 certificate=dict(method='seven_station_channel_cover')
124             else:raise RuntimeError('Incomplete stopping certificate')
125             if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
126             exit_confirmed=client.exit()['exit_reason']=='user_exit'
127         except (RuntimeError,ValueError,KeyError) as exc:
128             failure=f'{type(exc).__name__}: {exc}'
129         result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
130                    exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
131                    absent_channels=sorted(c for c in negative if len(negative[c])==7),
132                    negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
133                    station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
134         event(dict(type='strategy_result',**result))
135         return result

```

程序 122

对应文件: question3/innovation/round07_reception_bracket/bounded/code_snapshot/experiment.py; question3/innovation/round08_survey/bounded/code_snapshot/experiment.py; question3/innovation/round09_information/bounded/code_snapshot/experiment.py

```

1  """Evaluator-only self experiments. Strategy inputs exclude cases and evaluator state."""
2  import argparse
3  import hashlib
4  import json
5  import math
6  import shutil
7  import statistics
8  import sys
9  import time
10 from pathlib import Path
11 import numpy as np
12
13 ROOT=Path(__file__).resolve().parent
14 sys.path.insert(0,str(ROOT/'frozen'))
15 from question3.cases import development_cases
16 from question3.simulator import Simulator
17 from question3.interface import Client,JsonlLog
18 from question3.strategy import run as baseline_run
19 from question3.audit import audit_folder

```



```

20 from .strategy import run as candidate_run
21
22 class PublicClient:
23     """Narrow public observation/action capability; no transport/evaluation accessor."""
24     __slots__=('__client',)
25     def __init__(self,client):self.__client=client
26     @property
27     def position(self):return self.__client.position
28     @property
29     def channel(self):return self.__client.channel
30     def enter(self):return self.__client.enter()
31     def measure(self,*a,**kw):return self.__client.measure(*a,**kw)
32     def clear(self,*a,**kw):return self.__client.clear(*a,**kw)
33     def exit(self):return self.__client.exit()
34     def metrics(self):return self.__client.metrics()
35
36 def dump(path,data):
37     path.parent.mkdir(parents=True,exist_ok=True)
38     path.write_text(json.dumps(data,ensure_ascii=False,indent=2))
39
40 def audit_new(folder):
41     case=json.loads((folder/'evaluator_hidden_case.json').read_text())
42     truth={s['channel']:np.array([s['x'],s['y']]) for s in case['sources']}
43     events=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
44     counts={}
45     for e in events:
46         kind=e['type'];counts[kind]=counts.get(kind,0)+1
47         if kind not in ('feasible_update','certified_measure','certified_clear','range_bracket'):
48             continue
49         v=np.asarray(e['vertices']);g=truth[e['channel']]
50         assert len(v)>0 and np.isfinite(v).all()
51         edges=np.roll(v,-1,axis=0)-v;delta=g-v
52         cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
53         assert np.all(cross>=-1e-4) or np.all(cross<=1e-4),('source excluded',e)
54         if kind=='range_bracket':
55             z=(g-np.asarray(e['first']))@np.asarray(e['axis'])
56             assert e['lower']-1e-6<=z<=e['upper']+1e-6
57         if kind in ('certified_measure','certified_clear'):
58             p=np.asarray(e['point']);radius=e['radius']
59             assert np.linalg.norm(v-p,axis=1).max()<=radius+1e-8
60             assert radius<(1000 if kind=='certified_measure' else 20)
61     return counts
62
63 def aggregate(rows,reference='BASE'):
64     groups={n:[r for r in rows if r['variant']==n] for n in sorted({r['variant'] for r in rows})}
65     ref={r['case_id']:r for r in groups[reference]}
66     result={}
67     for n,rs in groups.items():
68         t=[r['penalized_virtual_s'] for r in rs]
69         deltas=[(r['penalized_virtual_s']/ref[r['case_id']]['penalized_virtual_s']-1,r['case_id']) for r in rs]
70         result[n]=dict(runs=len(rs),complete=sum(r['success'] for r in rs),
71             failed=sum(not r['success'] for r in rs),timeouts=sum(r['timeout'] for r in rs),
72             false_complete=sum(r['false_complete'] for r in rs),
73             mean=float(np.mean(t)),p95=float(np.percentile(t,95)),maximum=max(t),
74             worst_case=rs[int(np.argmax(t))]['case_id'],worst_paired_regression=max(deltas),
75             clearance_ratio_mean=float(np.mean([r['cleared']/r['total'] for r in rs])),
76             raw_mean=float(np.mean([r['metrics']['virtual_time_s'] for r in rs])),
77             movement_detection_etc={k:float(np.mean([r['metrics']['components'][k] for r in rs])) for k in
78                 ↪ rs[0]['metrics']['components']},
79             phase_means={k:float(np.mean([r['metrics']['phases'].get(k,0) for r in rs])) for k in
80                 ↪ sorted(set().union(*(r['metrics']['phases'] for r in rs)))},
81             mean_per_cleared=float(np.mean([r['metrics']['virtual_time_s']/max(1,r['cleared']) for r in rs])),
82             wall_p95=float(np.percentile([r['elapsed_wall_s'] for r in rs],95)),
83             clear_failure_mean=float(np.mean([r['metrics']['counts']['clear_failure'] for r in rs]))
84     for n,v in result.items():
85         v['mean_change_pct']=100*(v['mean']/result[reference]['mean']-1)
86         v['significant']=n!=reference and v['mean_change_pct']<=-5 and v['complete']>=result[reference]['complete']
87         ↪ and v['p95']<=result[reference]['p95']
88     return result
89
90 def main():
91     p=argparse.ArgumentParser()
92     p.add_argument('--round',required=True);p.add_argument('--variants',default='BASE,HOMING')

```

```

90 p.add_argument('--rounding',default='bounded',choices=['bounded','pre_round_stress'])
91 p.add_argument('--fixtures');p.add_argument('--limit',type=int)
92 p.add_argument('--stage',default='development');p.add_argument('--label')
93 args=p.parse_args()
94 rounddir=R00T/args.round
95 out=rounddir/(args.label or args.rounding)
96 out.mkdir(parents=True,exist_ok=False)
97 baseline=json.loads((R00T/'frozen/question3/configs/recommended_omni.json').read_text())
98 names=args.variants.split(',')
99 configs={n:(baseline if n=='BASE' else dict(baseline_config=baseline,mechanism=n.lower())) for n in names}
100 dump(out/'configs.json',configs)
101 manifest={str(x.relative_to(R00T)):hashlib.sha256(x.read_bytes()).hexdigest() for x in R00T.glob('*.py')}
102 dump(out/'code_sha256.json',manifest)
103 for file in R00T.glob('*.py'):
104     dest=out/'code_snapshot'/file.name;dest.parent.mkdir(exist_ok=True);shutil.copy2(file,dest)
105 cases=json.loads(Path(args.fixtures).read_text()) if args.fixtures else list(development_cases())
106 if args.limit:cases=cases[:args.limit]
107 dump(out/'fixtures.json',cases)
108 dump(out/'metadata.json',dict(stage=args.stage,rounding=args.rounding,
109     distinct_layouts=len({json.dumps(c['sources'],sort_keys=True) for c in cases}),
110     case_sha256={c['id']:hashlib.sha256(json.dumps(c,sort_keys=True).encode()).hexdigest() for c in cases},
111     official_runs=0,formal_runs=0,numeric_parameter_search_trials=0))
112 rows=[]
113 for i,case in enumerate(cases):
114     order=names[i%len(names):]+names[:i%len(names)]
115     for name in order:
116         folder=out/name/case['id'];folder.mkdir(parents=True)
117         dump(folder/'evaluator_hidden_case.json',case)
118         log=JsonlLog(folder/'actions.jsonl');events=JsonlLog(folder/'strategy.jsonl')
119         env=Simulator(case,rounding=args.rounding)
120         client=Client(env,'SELF',log,5.)
121         start=time.perf_counter()
122         try:
123             result=(baseline_run if name=='BASE' else candidate_run)(PublicClient(client),configs[name],events)
124         except Exception as exc:
125             result=dict(status='incomplete',failure=f'{type(exc).__name__}:
126                 ↳ {exc}',metrics=client.metrics(),regions=[])
127         wall=time.perf_counter()-start
128         truth=env.evaluation() # First evaluator access, after the policy returned.
129         row=dict(variant=name,case_id=case['id'],stage=args.stage,rounding=args.rounding,
130             status=result['status'],failure=result.get('failure'),elapsed_wall_s=wall,
131             total=truth['total'],cleared=truth['cleared'],metrics=result['metrics'])
132         row['false_complete']=result['status']=='complete' and bool(truth['remaining_channels'])
133         row['timeout']=bool(result.get('failure') and any(x in result['failure'].lower() for x in
134             ↳ ('deadline','timeout','budget')))
135         row['success']=result['status']=='complete' and not truth['remaining_channels']
136         dump(folder/'result.json',dict(summary=row,strategy=result))
137         log.close();events.close()
138         try:
139             if row['success']:
140                 row['audited_actions']=audit_folder(folder)
141                 row['event_counts']=audit_new(folder)
142                 row['audit']='pass'
143             else:row['audit']='incomplete retained'
144         except Exception as exc:
145             row['audit']=f'FAIL {type(exc).__name__}: {exc}'
146             row['success']=False;row['false_complete']=True
147             row['penalized_virtual_s']=truth['virtual_time_s'] if row['success'] else
148             ↳ max(360000,truth['virtual_time_s'])
149             dump(folder/'result.json',dict(summary=row,strategy=result))
150             rows.append(row)
151         dump(out/'summary.json',dict(rows=rows,aggregates=aggregate(rows)))
152         print(f'{i+1}/{len(cases)} {case["id"]} '+', '.join(f'{r["variant"]}={r["penalized_virtual_s"]:.1f}'+('
153             ↳ '+str(r['failure'])+' '+r['audit'] if not r['success'] else '') for r in rows[-len(names):]),flush=True)
154 stats=aggregate(rows);dump(out/'comparison.json',stats)
155 print(json.dumps({n:{k:v[k] for k in ('complete','runs','mean','mean_change_pct','p95','maximum','significant')}}
156     ↳ for n,v in stats.items()),indent=2,flush=True)
157
158 if __name__=='__main__':main()

```

程序 123

对应文件: question3/innovation/round07_reception_bracket/bounded/code_snapshot/strategy.py; question3/innovation/round08_survey/bounded/code_snapshot/strategy.py

```
1 """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2 import math
3 import numpy as np
4 from question3.geometry import stations,station_cover_bound
5 from question3.modules import choose_second
6 from .geometry import initial,wedge,bbox,enclosing,grid,parallax_point,clip
7 from .routing import choose as choose_route
8
9 def run(client,config,event=lambda row:None):
10     cover=stations(1125)
11     assert station_cover_bound(1125)<1000
12     first={};polys={};bearings={};measured={};cleared=set();visited=set()
13     negative={c:set() for c in range(1,21)}
14     failure=None;exit_confirmed=False;certificate=None
15     base=config['baseline_config']
16     mode=config.get('mechanism','homing')
17     route_mode='global' if mode.startswith('tour_') else ('nearest' if mode.startswith('nearest_') else None)
18     if route_mode:mode=mode.split('_')[1]
19     refresh=mode in ('refresh','refresh_parallax','shared','shared_parallax')
20     parallax=mode in ('parallax','refresh_parallax','shared_parallax')
21     sharing=mode in ('shared','shared_parallax')
22     survey=mode=='survey'
23     def centre(c):
24         return bbox(polys[c],bearings[c])[:2]
25     def update(c,p,response):
26         kind=response['measure_result']
27         if kind=='near':
28             if client.clear(p,c,phase='near')['clear_result']!='success':
29                 raise RuntimeError('Near feedback contradicted by failed clear')
30             cleared.add(c)
31             return
32         if kind=='direction':
33             raise RuntimeError('Certified reception returned no_signal')
34         theta=response['svd_deg']
35         if c not in first:
36             first[c]=(tuple(p),theta)
37             polys[c]=initial(p,theta,base['error_deg'])
38             measured[c]=set()
39         else:
40             polys[c]=wedge(polys[c],p,theta,base['error_deg'])
41             measured[c].add(tuple(p));bearings[c]=theta
42             event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
43     def scan(k):
44         p=cover[k]
45         for c in range(1,21):
46             if not survey and len(set(first)|cleared)==16:
47                 return
48             if c in cleared or (c in first and not survey):
49                 continue
50             if survey and len(set(first)|cleared)==16 and c not in first:continue
51             r=client.measure(p,c,phase='search')
52             if r['measure_result']=='no_signal':negative[c].add(k)
53             else:update(c,p,r)
54         visited.add(k)
55     def localize(c):
56         if mode=='reception_bracket':
57             s,theta=first[c];s=np.asarray(s)
58             t=math.radians(theta);axis=np.array([math.cos(t),math.sin(t)])
59             bounds=[]
60             for sign in (-1,1):
61                 inside=0.;outside=3001.
62                 while outside-inside>10:
63                     mid=(inside+outside)/2;p=s+sign*mid*axis
64                     r=client.measure(p,c,phase='reception_range')
65                     if r['measure_result']=='near':
```

```

66         update(c,p,r);return
67         if r['measure_result']=='no_signal':outside=mid
68         else:inside=mid
69         bounds.append(sorted([sign*inside,sign*outside]))
70         lower=(bounds[0][0]+bounds[1][0])/2
71         upper=(bounds[0][1]+bounds[1][1])/2
72         polys[c]=clip(polys[c],axis,float(axis@upper+1e-7))
73         polys[c]=clip(polys[c],-axis,float(-axis@s-lower+1e-7))
74         event(dict(type='range_bracket',channel=c,first=list(s),axis=axis.tolist(),lower=lower,upper=upper,vertic
↪ es=polys[c].tolist()))
75         for p in grid(polys[c],theta):
76             if client.clear(p,c,phase='bracket_optical')['clear_result']=='success':
77                 cleared.add(c);return
78         raise RuntimeError('Reception bracket optical cover exhausted')
79     if mode=='strip':
80         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
81         for p in grid(polys[c],bearings[c]):
82             if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
83                 cleared.add(c);return
84         raise RuntimeError('Single-bearing finite cover exhausted')
85     if refresh and tuple(client.position) not in measured[c]:
86         radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
87         if radius<1000-1e-5:
88             event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↪ s[c].tolist()))
89             update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
90             if c in cleared:return
91     for step in range(16):
92         p,radius=centre(c)
93         if radius<=20-1e-5:
94             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↪ )
95             if client.clear(p,c,phase='optical')['clear_result']!='success':
96                 raise RuntimeError('Certified containing circle clear failed')
97             cleared.add(c);return
98     if parallax and step==0:
99         p,radius=parallax_point(polys[c],bearings[c],client.position)
100     if radius>=1000-1e-5:
101         raise RuntimeError('Adaptive measurement lacks reception certificate')
102     if tuple(p) in measured[c]:
103         break
104     event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
105     update(c,p,client.measure(p,c,phase='adaptive_localization'))
106     if c in cleared:return
107     if sharing:
108         for other in sorted(set(first)-cleared-{c}):
109             if tuple(client.position) in measured[other]:continue
110             rr=float(np.linalg.norm(polys[other]-np.asarray(client.position),axis=1).max()+1e-7)
111             if rr>=1000-1e-5:continue
112             event(dict(type='certified_measure',channel=other,point=list(client.position),radius=rr,vertices=
↪ polys[other].tolist()))
113             update(other,client.position,client.measure(client.position,other,phase='multi_target_refinement')
↪ )
114     event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
115     for p in grid(polys[c],bearings[c]):
116         if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
117             cleared.add(c);return
118     raise RuntimeError('Finite optical cover exhausted without success')
119 client.enter()
120 try:
121     while True:
122         known=set(first)|cleared
123         if len(known)>16:raise RuntimeError('Source count upper bound contradicted')
124         pending=sorted(set(first)-cleared)
125         remaining=[] if len(known)==16 and not survey else sorted(set(range(7))-visited)
126         if not pending and not remaining:break
127         jobs=[]
128         for c in pending:
129             # R1 keeps the baseline's pre-schedule second-point proxy.
130             planning={**base,'second_point_opportunity':'off','second_range_prior':'off'}
131             q,_choose_second(*first[c],client.position,planning)
132             jobs.append((math.dist(client.position,q),'localize',c))
133         jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]

```

```

134         if survey:
135             tasks=[('scan',k,cover[k]) for k in remaining] if remaining else
136                 [('localize',c,centre(c)[0]) for c in pending])
137             (task,target,_),tour=choose_route(client.position,tasks,True)
138             event(dict(type='survey_route',route=tour))
139         elif route_mode:
140             tasks=[('localize',c,centre(c)[0]) for c in pending]
141             tasks += [('scan',k,cover[k]) for k in remaining]
142             (task,target,_),tour=choose_route(client.position,tasks,route_mode=='global')
143             event(dict(type='belief_route',mode=route_mode,route=tour))
144         else:
145             _,task,target=min(jobs)
146             event(dict(type='schedule',task=task,target=target))
147             if task=='scan':scan(target)
148             else:localize(target)
149         if len(cleared)==16:
150             certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
151                             successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
152             elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
153                 certificate=dict(method='seven_station_channel_cover')
154             else:raise RuntimeError('Incomplete stopping certificate')
155             if not 10<=len(cleared)<=16:raise RuntimeError('Clear count outside Q3 bounds')
156             exit_confirmed=client.exit()['exit_reason']=='user_exit'
157         except (RuntimeError,ValueError,KeyError) as exc:
158             failure=f'{type(exc).__name__}: {exc}'
159         result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
160                   exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
161                   absent_channels=sorted(c for c in negative if len(negative[c])==7),
162                   negative_station_ids=[str(c):sorted(v) for c,v in negative.items()],
163                   station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
164         event(dict(type='strategy_result',**result))
165         return result

```

程序 124

对应文件: question3/innovation/round09_information/bounded/code_snapshot/strategy.py

```

1 """Experimental public-feedback policy. Imports no cases, simulator, files or RNG."""
2 import math
3 import numpy as np
4 from question3.geometry import stations,station_cover_bound
5 from question3.modules import choose_second
6 from .geometry import initial,wedge,bbox,enclosing,grid,parallax_point,clip,worthwhile
7 from .routing import choose as choose_route
8
9 def run(client,config,event=lambda row=None):
10     cover=stations(1125)
11     assert station_cover_bound(1125)<1000
12     first={};polys={};bearings={};measured={};cleared=set();visited=set()
13     negative={c:set() for c in range(1,21)}
14     failure=None;exit_confirmed=False;certificate=None
15     base=config['baseline_config']
16     mode=config.get('mechanism','homing')
17     information_gate=mode.startswith('gated_')
18     if information_gate:mode=mode[len('gated_'):]
19     route_mode='global' if mode.startswith('tour_') else ('nearest' if mode.startswith('nearest_') else None)
20     if route_mode:mode=mode.split('_')[1]
21     refresh=mode in ('refresh','refresh_parallax','shared','shared_parallax')
22     parallax=mode in ('parallax','refresh_parallax','shared_parallax')
23     sharing=mode in ('shared','shared_parallax')
24     survey=mode=='survey'
25     def centre(c):
26         return bbox(polys[c],bearings[c]):2]
27     def update(c,p,response):
28         kind=response['measure_result']
29         if kind=='near':
30             if client.clear(p,c,phase='near')['clear_result']!='success':
31                 raise RuntimeError('Near feedback contradicted by failed clear')
32             cleared.add(c)
33         return

```

```

34     if kind!='direction':
35         raise RuntimeError('Certified reception returned no_signal')
36     theta=response['svd_deg']
37     if c not in first:
38         first[c]=(tuple(p),theta)
39         polys[c]=initial(p,theta,base['error_deg'])
40         measured[c]=set()
41     else:
42         polys[c]=wedge(polys[c],p,theta,base['error_deg'])
43         measured[c].add(tuple(p));bearings[c]=theta
44         event(dict(type='feasible_update',channel=c,point=list(p),bearing=theta,vertices=polys[c].tolist()))
45 def scan(k):
46     p=cover[k]
47     for c in range(1,21):
48         if not survey and len(set(first)|cleared)==16:
49             return
50         if c in cleared or (c in first and not survey):
51             continue
52         if survey and len(set(first)|cleared)==16 and c not in first:continue
53         r=client.measure(p,c,phase='search')
54         if r['measure_result']=='no_signal':negative[c].add(k)
55         else:update(c,p,r)
56     visited.add(k)
57 def localize(c):
58     if mode=='reception_bracket':
59         s,theta=first[c];s=np.asarray(s)
60         t=math.radians(theta);axis=np.array([math.cos(t),math.sin(t)])
61         bounds=[]
62         for sign in (-1,1):
63             inside=0.;outside=3001.
64             while outside-inside>10:
65                 mid=(inside+outside)/2;p=s+sign*mid*axis
66                 r=client.measure(p,c,phase='reception_range')
67                 if r['measure_result']=='near':
68                     update(c,p,r);return
69                 if r['measure_result']=='no_signal':outside=mid
70                 else:inside=mid
71             bounds.append(sorted([sign*inside,sign*outside]))
72         lower=(bounds[0][0]+bounds[1][0])/2
73         upper=(bounds[0][1]+bounds[1][1])/2
74         polys[c]=clip(polys[c],axis,float(axis@s+upper+1e-7))
75         polys[c]=clip(polys[c],-axis,float(-axis@s-lower+1e-7))
76         event(dict(type='range_bracket',channel=c,first=list(s),axis=axis.tolist(),lower=lower,upper=upper,vertic
↪ es=polys[c].tolist()))
77         for p in grid(polys[c],theta):
78             if client.clear(p,c,phase='bracket_optical')['clear_result']=='success':
79                 cleared.add(c);return
80             raise RuntimeError('Reception bracket optical cover exhausted')
81     if mode=='strip':
82         event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
83         for p in grid(polys[c],bearings[c]):
84             if client.clear(p,c,phase='single_bearing_sweep')['clear_result']=='success':
85                 cleared.add(c);return
86             raise RuntimeError('Single-bearing finite cover exhausted')
87     if refresh and tuple(client.position) not in measured[c]:
88         radius=float(np.linalg.norm(polys[c]-np.asarray(client.position),axis=1).max()+1e-7)
89         if radius<1000-1e-5:
90             event(dict(type='certified_measure',channel=c,point=list(client.position),radius=radius,vertices=poly
↪ s[c].tolist()))
91             update(c,client.position,client.measure(client.position,c,phase='stationary_refinement'))
92             if c in cleared:return
93     for step in range(16):
94         p,radius=centre(c)
95         if radius<=20-1e-5:
96             event(dict(type='certified_clear',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist())
↪ )
97             if client.clear(p,c,phase='optical')['clear_result']!='success':
98                 raise RuntimeError('Certified containing circle clear failed')
99             cleared.add(c);return
100     if parallax and step==0:
101         p,radius=parallax_point(polys[c],bearings[c],client.position)
102     if radius>=1000-1e-5:
103         raise RuntimeError('Adaptive measurement lacks reception certificate')

```

```

104         if tuple(p) in measured[c]:
105             break
106         event(dict(type='certified_measure',channel=c,point=p.tolist(),radius=radius,vertices=polys[c].tolist()))
107         update(c,p,client.measure(p,c,phase='adaptive_localization'))
108         if c in cleared: return
109         if sharing:
110             for other in sorted(set(first)-cleared-{c}):
111                 if tuple(client.position) in measured[other]: continue
112                 rr=float(np.linalg.norm(polys[other]-np.asarray(client.position),axis=1).max()+1e-7)
113                 if rr>=1000-1e-5: continue
114                 if information_gate and not
115                     ↪ worthwhile(polys[other],bearings[other],client.position,base['error_deg']):
116                     event(dict(type='information_gate_skip',channel=other)); continue
117                     event(dict(type='certified_measure',channel=other,point=list(client.position),radius=rr,vertices=
118                         ↪ polys[other].tolist()))
119                     update(other,client.position,client.measure(client.position,other,phase='multi_target_refinement'
120                         ↪ ))
121             event(dict(type='finite_optical_fallback',channel=c,vertices=polys[c].tolist()))
122             for p in grid(polys[c],bearings[c]):
123                 if client.clear(p,c,phase='optical_fallback')['clear_result']=='success':
124                     cleared.add(c); return
125             raise RuntimeError('Finite optical cover exhausted without success')
126         client.enter()
127     try:
128         while True:
129             known=set(first)|cleared
130             if len(known)>16: raise RuntimeError('Source count upper bound contradicted')
131             pending=sorted(set(first)-cleared)
132             remaining=[] if len(known)==16 and not survey else sorted(set(range(7))-visited)
133             if not pending and not remaining: break
134             jobs=[]
135             for c in pending:
136                 # R1 keeps the baseline's pre-schedule second-point proxy.
137                 planning={**base,'second_point_opportunity':'off','second_range_prior':'off'}
138                 q,_=choose_second(*first[c],client.position,planning)
139                 jobs.append((math.dist(client.position,q),'localize',c))
140             jobs += [(math.dist(client.position,cover[k]),'scan',k) for k in remaining]
141             if survey:
142                 tasks=[(['scan',k,cover[k]) for k in remaining] if remaining else
143                     [(['localize',c,centre(c)[0]) for c in pending])
144                 (task,target,_),tour=choose_route(client.position,tasks,True)
145                 event(dict(type='survey_route',route=tour))
146             elif route_mode:
147                 tasks=[(['localize',c,centre(c)[0]) for c in pending]
148                 tasks += [(['scan',k,cover[k]) for k in remaining]
149                 (task,target,_),tour=choose_route(client.position,tasks,route_mode=='global')
150                 event(dict(type='belief_route',mode=route_mode,route=tour))
151             else:
152                 _,task,target=min(jobs)
153                 event(dict(type='schedule',task=task,target=target))
154                 if task=='scan': scan(target)
155                 else: localize(target)
156             if len(cleared)==16:
157                 certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
158                     successful_channels=sorted(cleared),inferred_absent_channels=sorted(set(range(1,21))-cleared))
159                 elif all(c in cleared or len(negative[c])==7 for c in range(1,21)):
160                     certificate=dict(method='seven_station_channel_cover')
161                 else: raise RuntimeError('Incomplete stopping certificate')
162             if not 10<=len(cleared)<=16: raise RuntimeError('Clear count outside Q3 bounds')
163             exit_confirmed=client.exit()['exit_reason']=='user_exit'
164         except (RuntimeError,ValueError,KeyError) as exc:
165             failure=f'{type(exc).__name__}: {exc}'
166             result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
167                 exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
168                 absent_channels=sorted(c for c in negative if len(negative[c])==7),
169                 negative_station_ids={str(c):sorted(v) for c,v in negative.items()},
170                 station_positions=cover,stopping_certificate=certificate,regions=[],metrics=client.metrics())
171         event(dict(type='strategy_result',**result))
172     return result

```

程序 125

对应文件: question3/innovation/run_self.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/run_self.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/run_self.py

```
1 """Direct-file entry point: replay both algorithms against the frozen baseline."""
2 import sys
3 from pathlib import Path
4
5 ROOT = Path(__file__).resolve().parent
6 sys.path.insert(0, str(ROOT.parent))
7 from innovation.experiment import main
8 import question3.strategy
9
10 if Path(question3.strategy.__file__).resolve() != ROOT / 'frozen/question3/strategy.py':
11     raise RuntimeError('Replay must import the frozen baseline; run this file directly.')
12
13 if __name__ == '__main__':
14     main()
```

程序 126

对应文件: question3/innovation/validate.py; question3/results/dispatch_20260913/development/code_snapshot/question3/innovation/validate.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/innovation/validate.py

```
1 """Validate frozen bytes and run the selected candidate's self-only adversarial tests."""
2 import hashlib
3 import json
4 import sys
5 import unittest
6 from pathlib import Path
7
8 ROOT = Path(__file__).resolve().parent
9 sys.path.insert(0, str(ROOT.parent))
10 from innovation import test_adversarial
11 import question3.strategy
12
13
14 def main():
15     if Path(question3.strategy.__file__).resolve() != ROOT / 'frozen/question3/strategy.py':
16         raise RuntimeError('Run this file directly to use the frozen baseline.')
17     lock = json.loads((ROOT / 'validation_lock.json').read_text(encoding='utf-8'))
18     for directory, key in [(ROOT, 'code_sha256'), (ROOT / 'frozen', 'baseline_sha256')]:
19         for name, digest in lock[key].items():
20             if hashlib.sha256((directory / name).read_bytes()).hexdigest() != digest:
21                 raise RuntimeError('Locked source changed: ' + name)
22     suite = unittest.defaultTestLoader.loadTestsFromModule(test_adversarial)
23     result = unittest.TextTestRunner(verbosity=2).run(suite)
24     raise SystemExit(0 if result.wasSuccessful() else 1)
25
26
27 if __name__ == '__main__':
28     main()
```

程序 127

对应文件: question3/results/dispatch_20260913/development/code_snapshot/question3/run_innovation_http.py; question3/results/dispatch_20260913/smoke/code_snapshot/question3/run_innovation_http.py; question3/run_innovation_http.py


```

1 """Independent candidate entry using the repository's existing official HTTP client.
2
3 Only self HTTP or a human-opened Q3 rehearsal; never starts the simulator.
4 The original question3.run_http entry and recommended configuration are unchanged.
5 """
6 import argparse
7 import hashlib
8 import json
9 import time
10 from pathlib import Path
11 from .interface import Client, HttpTransport, JsonLog
12 from .innovation.strategy import run
13
14
15 class PublicClient:
16     """Expose only public action/observation capabilities to the candidate."""
17     __slots__ = ('__client',)
18
19     def __init__(self, client): self.__client = client
20     @property
21     def position(self): return self.__client.position
22     @property
23     def channel(self): return self.__client.channel
24     def enter(self): return self.__client.enter()
25     def measure(self, *args, **kwargs): return self.__client.measure(*args, **kwargs)
26     def clear(self, *args, **kwargs): return self.__client.clear(*args, **kwargs)
27     def exit(self): return self.__client.exit()
28     def metrics(self): return self.__client.metrics()
29
30
31 def main():
32     root = Path(__file__).resolve().parent
33     parser = argparse.ArgumentParser(description=__doc__)
34     parser.add_argument('--mode', choices=['self-http', 'official-rehearsal'], default='self-http')
35     parser.add_argument('--url')
36     parser.add_argument('--robot-id', default='SELF')
37     parser.add_argument('--confirm-rehearsal-ui', action='store_true')
38     parser.add_argument('--output', type=Path, required=True)
39     args = parser.parse_args()
40     if args.mode == 'official-rehearsal' and (not args.confirm_rehearsal_ui or args.robot_id == 'SELF'):
41         parser.error('Confirm official Q3 REHEARSAL UI and supply team robot-id. Formal mode is not implemented.')
42     config = json.loads((root / 'innovation/best_candidate.json').read_text(encoding='utf-8'))
43     url = args.url or ('http://127.0.0.1:2027' if args.mode == 'self-http' else 'http://127.0.0.1:2026')
44     args.output.mkdir(parents=True, exist_ok=False)
45
46     def save(name, value):
47         (args.output / name).write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding='utf-8')
48
49     save('config.json', config)
50     files = [Path(__file__), root / 'interface.py', root / 'geometry.py', root / 'modules.py']
51     files += sorted((root / 'innovation').glob('*.py'))
52     save('code_sha256.json', {str(p.relative_to(root)): hashlib.sha256(p.read_bytes()).hexdigest() for p in files})
53     log = JsonLog(args.output / 'actions.jsonl')
54     events = JsonLog(args.output / 'strategy.jsonl')
55     client = Client(HttpTransport(url), args.robot_id, log, config['baseline_config']['real_time_reserve_s'])
56     start = time.perf_counter()
57     try:
58         try:
59             result = run(PublicClient(client), config, events)
60         except Exception as exc:
61             result = dict(status='incomplete', failure=f'{type(exc).__name__}: {exc}', metrics=client.metrics())
62             result.update(kind=args.mode, config=config, elapsed_wall_s=time.perf_counter() - start,
63                           official_total=None, clearance_ratio=None,
64                           note='Official count must be read from rehearsal UI after exit; no hidden count in HTTP API.')
65             save('result.json', result)
66     finally:
67         log.close()
68         events.close()
69     print(json.dumps(result, ensure_ascii=False, indent=2))
70     if result['status'] != 'complete':
71         raise SystemExit(1)
72
73

```

```

74 if __name__ == '__main__':
75     main()

```

程序 128

对应文件: question3/results/round13_adverse_main/code_snapshot/question3/audit.py;question3/results/round13_adverse_rounding/code_snapshot/question3/audit.py;question3/results/round13_dev_main/code_snapshot/question3/audit.py;question3/results/round13_dev_rounding/code_snapshot/question3/audit.py;question3/results/round13_holdout_main/code_snapshot/question3/audit.py;question3/results/round13_holdout_rounding/code_snapshot/question3/audit.py;question3/results/round13_outer_main/code_snapshot/question3/audit.py;question3/results/round13_outer_rounding/code_snapshot/question3/audit.py

```

1  """Post-run oracle audit: evaluator only, NEVER called by the strategy."""
2  import argparse
3  import json
4  import math
5  import statistics
6  from pathlib import Path
7  import numpy as np
8  from question2.strategy import candidate
9  from .geometry import stations,station_cover_bound
10
11
12 def audit_folder(folder):
13     case=json.loads((folder/'evaluator_hidden_case.json').read_text(encoding='utf-8'))
14     result=json.loads((folder/'result.json').read_text(encoding='utf-8'))
15     sources={s['channel']:s for s in case['sources']};requests={};cleared=set();negative={c:set() for c in
↪ range(1,21)}
16     position=(0.,0.);channel=1;vt=0.;done=set();bearings={};measure_moves={};checks=0
17     for line in (folder/'actions.jsonl').read_text(encoding='utf-8').splitlines():
18         row=json.loads(line)
19         if row['type']=='request':requests[row['payload']]['request_id']=row
20         elif row['type']=='response' and row['response'].get('accepted') is True:
21             rid=row['request_id']
22             if rid in done:continue
23             done.add(rid);req=requests[rid];b=req['payload'];r=row['response'];path=req['path']
24             if path in ('/measure','/clear'):
25                 p=(b['position']['x'],b['position']['y']);c=b['channel'];move=math.dist(position,p);vt+=move/5;positi
↪ on=p
26                 s=sources.get(c);distance=math.inf if s is None or c in cleared else
↪ math.hypot(s['x']-p[0],s['y']-p[1])
27                 if path=='/measure':
28                     measure_moves[p,c]=move
29                     vt+=5+(c!=channel);channel=c
30                     expected='no_signal' if s is None or c in cleared or distance>s['radius'] else ('near' if
↪ distance<=5 else 'direction')
31                     assert r['measure_result']==expected,(folder,expected,r)
32                     if expected=='no_signal' and req['phase']=='search':negative[c].add(p)
33                     if expected=='direction':
34                         true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
35                         envelope=1. if result['summary']['rounding']=='bounded' else 1.005
36                         assert abs((r['svd_deg']-true+180)%360-180)<=envelope+1e-9
37                         key=(p,c)
38                         if key in bearings:assert bearings[key]==r['svd_deg']
39                         bearings[key]=r['svd_deg']
40                 else:
41                     vt+=3
42                     if distance<=20:
43                         assert r['clear_result']=='success';cleared.add(c);vt+=2
44                         else:assert r['clear_result']=='no_target_in_range'
45                     checks+=1
46                     assert abs(vt-r['virtual_time_s'])<=len(done)*1e-6+1e-4
47     strategy=result['strategy']
48     assert strategy['status']=='complete' and strategy['exit_confirmed']

```

```

49     assert set(strategy['cleared_channels'])==cleared
50     cover={tuple(s) for s in strategy['station_positions']}
51     assert len(cover)==7
52     radius=math.dist((0,0),strategy['station_positions'][1])
53     assert station_cover_bound(radius)<1000
54     assert np.allclose(strategy['station_positions'],stations(radius),atol=1e-7,rtol=0)
55     certificate=strategy.get('stopping_certificate') or {}
56     if certificate.get('method')=='source_count_upper_bound':
57         # Count accepted SUCCESS feedback on distinct channels, never attempts or detections.
58         assert certificate['source_upper_bound']==16
59         assert len(certificate['successful_channels'])==16
60         assert len(cleared)==16 and set(certificate['successful_channels'])==cleared
61         assert set(certificate['inferred_absent_channels'])==set(range(1,21))-cleared
62         assert len(sources)<=16 # Evaluator checks the published model's applicability.
63     else:
64         for c in range(1,21):assert c in cleared or cover<=negative[c],(folder,c)
65     assert cleared==set(sources)
66     for region in strategy['regions']:
67         selection=region.get('selection',{})
68         if selection.get('mode') in ('shared_certified','opportunistic_current'):
69             first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
70             angle=math.radians(bearings[first,c]);dx=second[0]-first[0];dy=second[1]-first[1]
71             offset=(dx*math.cos(angle)+dy*math.sin(angle),-dx*math.sin(angle)+dy*math.cos(angle))
72             assert np.allclose(offset,selection['offset'],atol=1e-7,rtol=0)
73             assert candidate(offset,selection['min_angle_deg'],selection['error_deg'])
74             assert (second,c) in bearings
75             if selection['mode']=='opportunistic_current':assert measure_moves[second,c]==0
76         if 'vertices' not in region:continue
77         g=np.array([sources[region['channel']][x],sources[region['channel']][y]]);v=np.array(region['vertices'])
78         edges=np.roll(v,-1,axis=0)-v;rel=g-v
79         cross=edges[:,0]*rel[:,1]-edges[:,1]*rel[:,0]
80         assert np.all(cross>=-1e-5) or np.all(cross<=1e-5)
81         assert region['covering_radius']<20
82         if 'single_circle_centre' in region:
83             centre=np.array(region['single_circle_centre'])
84             assert np.linalg.norm(v-centre,axis=1).max()<=20-1e-5+1e-9
85             assert np.linalg.norm(g-centre)<=20
86             assert region['planned_clear_calls']==1
87     # Independently recompute each omitted action's separating witness from accepted bearings.
88     from question1.solve import bearing_halfplanes
89     regions={r['channel']:r for r in strategy['regions']}
90     for line in (folder/'strategy.jsonl').read_text(encoding='utf-8').splitlines():
91         event=json.loads(line)
92         if event['type']!='certified_clear_skip':continue
93         c=event['channel'];region=regions[c];first=tuple(region['first_position']);second=tuple(region['second_positi
↵ on'])
94         if 'radius_prior' in event:
95             expected={'domain':((0,.0),1800.),'first_received':(first,1500.),'second_received':(second,1500.)}
96             centre,radius=expected[event['radius_prior']]
97             assert tuple(event['centre'])==centre and event['containing_radius_m']==radius
98             if event['radius_prior']!='domain':assert (centre,c) in bearings
99             gap=math.dist(event['point'],centre)-radius
100             assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
101             assert abs(gap-event['separation_m'])<1e-8
102             g=(sources[c][x],sources[c][y])
103             assert math.dist(g,centre)<=radius+1e-7 and math.dist(g,event['point'])>20
104             continue
105         assert event['error_deg']>=(1. if result['summary']['rounding']=='bounded' else 1.005)
106         A,b=bearing_halfplanes([dict(x=p[0],y=p[1],bearing_deg=bearings[p,c]) for p in
↵ [first,second]],event['error_deg'])
107         p=np.asarray(event['point'])
108         if 'halfplane_weights' in event:
109             weights=np.asarray(event['halfplane_weights'])
110             assert weights.shape==(len(A),) and np.isfinite(weights).all() and np.all(weights>=0)
111             assert abs(float(weights.sum())-1)<1e-12
112             normal=weights@A;bound=float(weights@b);length=float(np.linalg.norm(normal))
113             assert length>=1e-8
114             gap=float((normal@p-bound)/length)
115             assert event['slack_m']>=1e-5/length
116         else:
117             k=event['halfplane_index'];gap=float((A[k]@p-b[k])/np.linalg.norm(A[k]))
118             assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
119             assert abs(gap-event['separation_m'])<1e-8

```

```

120         assert math.dist(p,(sources[c]['x'],sources[c]['y']))>20
121     return checks
122
123
124 def main():
125     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)
126     summary=json.loads((root/'summary.json').read_text(encoding='utf-8'));rows=summary['cases']
127     checks=sum(audit_folder(root/r['case_id']) for r in rows)
128     aggregate=dict(kind='SELF_POST_RUN_AUDIT',runs=len(rows),actions_checked=checks,
129                   all_stopping_certificates_valid=True,all_physics_and_time_checks_passed=True,
130                   mean_virtual_s=statistics.mean(r['metrics']['virtual_time_s'] for r in rows),
131                   mean_case_average_per_clear_s=statistics.mean(r['average_virtual_per_cleared'] for r in rows),
132                   pooled_virtual_per_clear_s=sum(r['metrics']['virtual_time_s'] for r in rows)/sum(r['cleared'] for
133                   ↪ r in rows),
134                   total_cleared_over_runs=sum(r['cleared'] for r in rows),
135                   worst_total_case=max(rows,key=lambda r:r['metrics']['virtual_time_s']),
136                   worst_per_clear_case=max(rows,key=lambda r:r['average_virtual_per_cleared']),
137                   most_failed_clear_case=max(rows,key=lambda r:r['metrics']['counts']['clear_failure']),
138                   component_means={k:statistics.mean(r['metrics']['components'][k] for r in rows) for k in
139                   ↪ rows[0]['metrics']['components']},
140                   min_wall_s=min(r['elapsed_wall_s'] for r in rows),max_wall_s=max(r['elapsed_wall_s'] for r in
141                   ↪ rows))
142     (root/'audit.json').write_text(json.dumps(aggregate,ensure_ascii=False,indent=2),encoding='utf-8')
143     print(json.dumps({k:v for k,v in aggregate.items() if not isinstance(v,dict)},ensure_ascii=False,indent=2))
144
145 if __name__=='__main__':main()

```

程序 129

对应文件: question3/results/round13_adverse_main/code_snapshot/question3/g
eometry.py; question3/results/round13_adverse_rounding/code_snapshot/quest
ion3/geometry.py; question3/results/round13_dev_main/code_snapshot/questio
n3/geometry.py; question3/results/round13_dev_rounding/code_snapshot/quest
ion3/geometry.py; question3/results/round13_holdout_main/code_snapshot/que
stion3/geometry.py; question3/results/round13_holdout_rounding/code_snapsh
ot/question3/geometry.py; question3/results/round13_outer_main/code_snapsh
ot/question3/geometry.py; question3/results/round13_outer_rounding/code_sn
apshot/question3/geometry.py

```

1  """Public geometry only; reuse Q1 and Q2, no simulator imports."""
2  import math
3  import numpy as np
4  from question1.solve import solve
5  from question2.strategy import candidate, to_world
6
7
8  def stations(radius=1500.):
9      return [(0.,0.)]+[(radius*math.cos(k*math.pi/3),radius*math.sin(k*math.pi/3)) for k in range(6)]
10
11
12  def station_cover_bound(radius=1500.):
13      if not math.isfinite(radius) or radius<=0:raise ValueError('Invalid search ring radius')
14      # Centre covers radius <=900. Remaining annulus: nearest ring angle <=30deg.
15      return max(900.,*(math.sqrt(r*r+radius**2-2*r*radius*math.cos(math.pi/6)) for r in [900,1800]))
16
17
18  def second_point(first, bearing, offset, error_deg, min_angle):
19      if not candidate(offset,min_angle,error_deg):
20          raise ValueError('Second offset lacks analytic reception/angle certificate')
21      if min_angle <= 4*error_deg:
22          raise ValueError('Angle margin insufficient for bounded two-wedge intersection')
23      return tuple(to_world(offset,first,bearing))
24
25
26  def optical_plan(first, bearing, second, reading, error_deg, cell_m, max_points):
27      """Cover an inflated rotated bounding box by <=28m cells. No hidden G.

```

```

28
29 Cells have diagonal/2 <20m. Testing every centre guarantees a hit if G is in P.
30 """
31 if not 0 < cell_m < 20*math.sqrt(2):
32     raise ValueError('Optical cells must fit strictly inside a radius-20 circle')
33 result=solve([dict(x=first[0],y=first[1],bearing_deg=bearing),
34               dict(x=second[0],y=second[1],bearing_deg=reading)],error_deg)
35 if result['status']!= 'bounded':
36     raise RuntimeError('Invalid localization region: '+result['status'])
37 vertices=np.array(result['vertices'])
38 a=math.radians(bearing)
39 rot=np.array([[math.cos(a),-math.sin(a)],[math.sin(a),math.cos(a)]])
40 local=(vertices-np.array(first))@rot
41 lo=local.min(axis=0)-1e-5;hi=local.max(axis=0)+1e-5
42 sizes=np.maximum(1,np.ceil((hi-lo)/cell_m).astype(int))
43 n=int(np.prod(sizes))
44 if n>max_points:
45     raise RuntimeError(f'Optical plan exceeds configured guard: {n}')
46 points=[]
47 for j in range(sizes[1]):
48     cols=range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))
49     for i in cols:
50         p=lo+(np.array([i+.5,j+.5])/sizes)*(hi-lo)
51         points.append(tuple(np.array(first)+rot@p))
52 certificate=dict(diameter=result['diameter'],vertices=result['vertices'],
53                 bbox_widths=(hi-lo).tolist(),planned_clear_calls=n,
54                 covering_radius=float(np.linalg.norm((hi-lo)/sizes)/2))
55 return points,certificate

```

程序 130

对应文件: question3/results/round13_adverse_main/code_snapshot/question3/lookahead.py; question3/results/round13_adverse_rounding/code_snapshot/question3/lookahead.py; question3/results/round13_dev_main/code_snapshot/question3/lookahead.py; question3/results/round13_dev_rounding/code_snapshot/question3/lookahead.py; question3/results/round13_holdout_main/code_snapshot/question3/lookahead.py; question3/results/round13_holdout_rounding/code_snapshot/question3/lookahead.py; question3/results/round13_outer_main/code_snapshot/question3/lookahead.py; question3/results/round13_outer_rounding/code_snapshot/question3/lookahead.py; question3/results/round14_adverse_main/code_snapshot/question3/lookahead.py; question3/results/round14_adverse_rounding/code_snapshot/question3/lookahead.py; question3/results/round14_dev_main/code_snapshot/question3/lookahead.py; question3/results/round14_dev_rounding/code_snapshot/question3/lookahead.py; question3/results/round14_holdout_main/code_snapshot/question3/lookahead.py; question3/results/round14_holdout_rounding/code_snapshot/question3/lookahead.py; question3/results/round14_initial_dev_main/code_snapshot/question3/lookahead.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/lookahead.py; question3/results/round14_outer_main/code_snapshot/question3/lookahead.py; question3/results/round14_outer_rounding/code_snapshot/question3/lookahead.py

```

1 """Round2: stable numerical ties and one-step terminal-region routing proxy.
2 No truth access. Scenario filtering affects scores only, never safety/stop proofs.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np

```

```

7
8
9 def stable_min(items,score,identity,tolerance=0.):
10     if not items: return None
11     best=min(score(x) for x in items)
12     return min((x for x in items if score(x)<=best+tolerance),key=identity)
13
14
15 @lru_cache(maxsize=4096)
16 def source_region_samples(first,bearing,negative_positions,error_deg):
17     delta=math.radians(error_deg);angle=math.radians(bearing)
18     r,a=np.meshgrid(np.linspace(5.001,1500,31),angle+np.linspace(-delta,delta,3),indexing='ij')
19     points=np.asarray(first)+np.column_stack([(r*np.cos(a)).ravel(),(r*np.sin(a)).ravel()])
20     valid=np.linalg.norm(points,axis=1)<=1800+1e-7
21     for pos in negative_positions: valid &= np.linalg.norm(points-pos,axis=1)>1000-1e-7
22     filtered=points[valid]
23     fallback=len(filtered)==0
24     if fallback: filtered=points # Coarse samples are not an exact feasibility test.
25     return filtered,fallback
26
27
28 def decide(current,current_channel,cover,remaining,jobs,first,negative,unknown,config,forced=False):
29     """Jobs have (move_distance, channel, certified_q, A_selection).
30     End samples are geometric proxies, not a probability distribution or error bound.
31     """
32     options=config.get('routing',{});tolerance=options.get('tie_tolerance_m',0.)
33     mode=options.get('mode','nearest');weight=float(options.get('lookahead_weight',0.))
34     if mode not in ('nearest','region','point'): raise ValueError('Unknown routing mode')
35     if weight<0 or not math.isfinite(weight): raise ValueError('Invalid lookahead weight')
36     if mode=='nearest':
37         station=stable_min(remaining,lambda k:math.dist(current,cover[k]),lambda k:k,tolerance)
38         job=stable_min(jobs,lambda j:j[0],lambda j:j[1],tolerance)
39         choose_scan=station is not None and (job is None or forced or
40         ↪ math.dist(current,cover[station])<=job[0]+tolerance)
41         if choose_scan: return 'scan',station,dict(mode=mode,forced=forced)
42         return 'localize',job,dict(mode=mode,forced=False)
43     anchors=[('scan',k,cover[k]) for k in remaining]+[('localize',j[1],j[2]) for j in jobs]
44     candidates=[]
45     for k in remaining:
46         other=[p for kind,ident,p in anchors if not (kind=='scan' and ident==k)]
47         after=min((math.dist(cover[k],p) for p in other),default=0.)
48         scan_cost=5*len(unknown)+max(0,len(unknown)-1)+(bool(unknown) and unknown[0]!=current_channel)
49         score=math.dist(current,cover[k])/5+scan_cost+weight*after/5
50         candidates.append(dict(kind='scan',id=k,score_s=float(score),forced=forced))
51     if not forced:
52         for _,c,q,selection in jobs:
53             s,bearing=first[c]
54             points,fallback=source_region_samples(tuple(s),bearing,tuple(tuple(cover[k]) for k in
55             ↪ sorted(negative[c])),config['error_deg'])
56             if mode=='point': points=points.mean(axis=0,keepdims=True)
57             other=np.array([p for kind,ident,p in anchors if not (kind=='localize' and ident==c)])
58             after=np.linalg.norm(points[:,None,:]-other[None,:,:],axis=2).min(axis=1) if len(other) else
59             ↪ np.zeros(len(points))
60             transit=np.linalg.norm(points-np.asarray(q),axis=1)+weight*after
61             cells=selection.get('proxy_cells')
62             if cells is None: raise ValueError('Lookahead currently requires A cost metadata')
63             score=math.dist(current,q)/5+5+8.6*cells+float(transit.max())/5
64             candidates.append(dict(kind='localize',id=c,score_s=score,sample_count=len(points),sample_fallback=fallback
65             ↪ ck))
66     chosen=stable_min(candidates,lambda x:x['score_s'],lambda x:(x['kind']!='scan',x['id']),tolerance/5)
67     if chosen is None: raise RuntimeError('No routing task available')
68     info=dict(mode=mode,weight=weight,forced=forced,chosen_score_s=chosen['score_s'],candidates=candidates)
69     if chosen['kind']=='scan': return 'scan',chosen['id'],info
70     return 'localize',next(j for j in jobs if j[1]==chosen['id']),info

```

程序 131

对应文件: question3/results/round13_adverse_main/code_snapshot/question3/modules.py; question3/results/round13_adverse_rounding/code_snapshot/question3/modules.py; question3/results/round13_dev_main/code_snap

shot/question3/modules.py; question3/results/round13_dev_rounding/code_snapshot/question3/modules.py; question3/results/round13_holdout_main/code_snapshot/question3/modules.py; question3/results/round13_holdout_rounding/code_snapshot/question3/modules.py; question3/results/round13_outer_main/code_snapshot/question3/modules.py; question3/results/round13_outer_rounding/code_snapshot/question3/modules.py; question3/results/round14_adverse_main/code_snapshot/question3/modules.py; question3/results/round14_adverse_rounding/code_snapshot/question3/modules.py; question3/results/round14_dev_main/code_snapshot/question3/modules.py; question3/results/round14_dev_rounding/code_snapshot/question3/modules.py; question3/results/round14_holdout_main/code_snapshot/question3/modules.py; question3/results/round14_holdout_rounding/code_snapshot/question3/modules.py; question3/results/round14_initial_dev_main/code_snapshot/question3/modules.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/modules.py; question3/results/round14_outer_main/code_snapshot/question3/modules.py; question3/results/round14_outer_rounding/code_snapshot/question3/modules.py; question3/results/round17_adverse_main/code_snapshot/question3/modules.py; question3/results/round17_adverse_rounding/code_snapshot/question3/modules.py; question3/results/round17_dev_main/code_snapshot/question3/modules.py; question3/results/round17_dev_rounding/code_snapshot/question3/modules.py

```

1 """Optional public-information modules A (local cost), B (route via strategy), C (evidence).
2 No simulator/case imports. All design constants fixed before round-one comparison.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np
7 from question2.strategy import candidate, diameters, to_world
8 from question1.solve import bearing_halfplanes
9 from .geometry import second_point
10 from .current_point import local_offset,current_proxy
11
12 OFFSETS=((400,500),(500,400),(500,500),(600,400),(700,450),(750,400),(800,605),(850,450))
13
14 @lru_cache(maxsize=8)
15 def design_catalog(error_deg=1.005, min_angle=20.):
16     delta=math.radians(error_deg)
17     r,alpha,error=np.meshgrid(np.linspace(5.001,1500,31),np.linspace(-delta,delta,3),
18                               np.linspace(-delta,delta,3),indexing='ij')
19     g=np.column_stack([(r*np.cos(alpha)).ravel(),(r*np.sin(alpha)).ravel()]);error=error.ravel()
20     catalog=[]
21     for a,b in OFFSETS:
22         if not candidate([a,b],min_angle,error_deg):continue
23         d=float(diameters([a,b],g,error,error_deg).max())
24         h=a*math.cos(delta)-b*math.sin(delta)
25         max_range=math.sqrt(max(a*a+b*b+25-10*h,a*a+b*b+1500**2-3000*h))
26         # Surrogate only: diagonal-based box count can overcount or undercount.
27         # Never used as a coverage/feasibility certificate or a probabilistic expectation.
28         cells=math.ceil(d/28)*math.ceil(2*1500*math.sin(delta)/28)
29         continuation=max_range/5+(3+28/5)*cells
30         for sign in [1,-1]:
31             catalog.append(dict(offset=(a,sign*b),sampled_worst_diameter=d,
32                                max_source_distance=max_range,proxy_cells=cells,continuation_proxy_s=continuation))
33     return tuple(catalog)
34
35
36 def choose_second(first,bearing,current,config):
37     if not config.get('modules',{}).get('A',False):

```

```

38     q=second_point(first,bearing,config['second_offset'],config['error_deg'],config['min_angle_deg'])
39     return q,dict(mode='fixed',offset=config['second_offset'])
40 best=None
41 for row in design_catalog(config['error_deg'],config['min_angle_deg']):
42     q=tuple(to_world(row['offset'],first,bearing))
43     score=math.dist(current,q)/5+5+row['continuation_proxy_s']
44     key=(score,row['offset'])
45     if best is None or key<best[0]:best=(key,q,row)
46 if best is None:raise RuntimeError('No certified second-point candidates')
47 opportunity=config.get('second_point_opportunity','off')
48 if opportunity!='off':
49     offset=local_offset(first,bearing,current)
50     row=current_proxy(offset,config['error_deg'],config['min_angle_deg'])
51     if row is not None:
52         score=5+row['continuation_proxy_s']
53         if opportunity=='current_if_certified' or score<best[0][0]-1e-8:
54             return tuple(current),dict(mode='opportunistic_current',score_s=score,**row,
55                 min_angle_deg=config['min_angle_deg'],error_deg=config['error_deg'],
56                 selection_rule=opportunity,catalog_score_s=best[0][0],catalog_point=best[1],
57                 catalog_entry_distance_m=math.dist(current,best[1]),
58                 catalog_sampled_diameter=best[2]['sampled_worst_diameter'])
59 return best[1],dict(mode='cost_proxy',score_s=best[0][0],**best[2])
60
61
62 class EvidenceCover:
63     """Finite covering debt. Fine cells are excluded only by whole-cell proofs.
64
65     Every fine cell has its original coarse-grid centre within 20m of all corners.
66     Failures remove cells fully inside failed-clear disks; they never infer absence
67     from a bare centre test. Point selection uses covered geometric area per cost,
68     not a probability of source presence. No fine-grid point estimate of truth.
69     """
70     def __init__(self,points,certificate,first,bearing,second,reading,negative_positions,error_deg,subdiv=4):
71         v=np.asarray(certificate['vertices']);origin=np.asarray(first)
72         angle=math.radians(bearing);rot=np.array([[math.cos(angle),-math.sin(angle)],[math.sin(angle),math.cos(angle)]]
73         ↪ ))
74         local=(v-origin)@rot;lo=local.min(axis=0)-1e-5;hi=local.max(axis=0)+1e-5
75         sizes=np.maximum(1,np.ceil((hi-lo)/28).astype(int));fine=sizes*subdiv
76         corners=[]
77         for j in range(fine[1]):
78             for i in range(fine[0]):
79                 cell=lo+(np.array([[i,j],[i+1,j],[i+1,j+1],[i,j+1]])/fine)*(hi-lo)
80                 corners.append(origin+cell@rot.T)
81 self.corners=np.array(corners);self.points=np.asarray(points);self.alive=np.ones(len(corners),bool)
82 self.untried=np.ones(len(points),bool);self.initial_count=len(corners);self.prior_dropped=0;self.failed_dropp
83 ↪ ed=0
84 A,b=bearing_halfplanes([dict(x=first[0],y=first[1],bearing_deg=bearing),
85     dict(x=second[0],y=second[1],bearing_deg=reading)],error_deg)
86 projections=np.einsum('kd,nvd->nkv',A,self.corners)
87 self.alive &= ~np.any(projections.min(axis=2)>b[None,:]+1e-7,axis=1)
88 centres=self.corners.mean(axis=1);radii=np.linalg.norm(self.corners-centres[:,None,:],axis=2).max(axis=1)
89 for centre,radius in [((0,0),1800),(first,1500),(second,1500)]:
90     self.alive &= np.linalg.norm(centres-centre,axis=1)-radii <= radius+1e-7
91 for p in negative_positions:
92     self.alive &= np.linalg.norm(self.corners-np.asarray(p),axis=2).max(axis=1)>1000-1e-7
93 self.prior_dropped=int(self.initial_count-self.alive.sum())
94 self.covers=np.linalg.norm(self.points[:,None,None,:]-self.corners[None,:,:,:],axis=3).max(axis=2)<=20-1e-7
95 if not np.all(self.covers.any(axis=0)):raise RuntimeError('Fine cells lack original optical coverage')
96 if not self.alive.any():raise RuntimeError('All source cells excluded by prior evidence')
97 def next_point(self,current):
98     if not self.alive.any():raise RuntimeError('All feasible cells removed without success')
99     gain=self.covers[:,self.alive].sum(axis=1)
100     cost=3+np.linalg.norm(self.points-np.asarray(current),axis=1)/5
101     scores=np.where(self.untried & (gain>0),gain/cost,-1)
102     k=int(np.argmax(scores))
103     if scores[k]<0:raise RuntimeError('Optical covering debt cannot be satisfied')
104     self.untried[k]=False
105     return k,tuple(self.points[k]),dict(remaining_cells=int(self.alive.sum()),fully_covered_cells=int(gain[k]))
106 def failed(self,k):
107     remove=self.alive & self.covers[k];self.failed_dropped+=int(remove.sum());self.alive[remove]=False
108 def metrics(self):return dict(fine_cells=self.initial_count,prior_excluded_cells=self.prior_dropped,
109     failed_clear_excluded_cells=self.failed_dropped,remaining_cells=int(self.alive.sum(
110     ↪ )))

```


程序 132

对应文件: question3/results/round13_adverse_main/code_snapshot/question3/shared_measurement.py; question3/results/round13_adverse_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round13_dev_main/code_snapshot/question3/shared_measurement.py; question3/results/round13_dev_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round13_holdout_main/code_snapshot/question3/shared_measurement.py; question3/results/round13_holdout_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round13_outer_main/code_snapshot/question3/shared_measurement.py; question3/results/round13_outer_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round14_adverse_main/code_snapshot/question3/shared_measurement.py; question3/results/round14_adverse_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round14_dev_main/code_snapshot/question3/shared_measurement.py; question3/results/round14_dev_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round14_holdout_main/code_snapshot/question3/shared_measurement.py; question3/results/round14_holdout_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round14_initial_dev_main/code_snapshot/question3/shared_measurement.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round14_outer_main/code_snapshot/question3/shared_measurement.py; question3/results/round14_outer_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round17_adverse_main/code_snapshot/question3/shared_measurement.py; question3/results/round17_adverse_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round17_dev_main/code_snapshot/question3/shared_measurement.py; question3/results/round17_dev_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round18_adverse_main/code_snapshot/question3/shared_measurement.py; question3/results/round18_adverse_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round18_dev_main/code_snapshot/question3/shared_measurement.py; question3/results/round18_dev_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round18_holdout_main/code_snapshot/question3/shared_measurement.py; question3/results/round18_holdout_rounding/code_snapshot/question3/shared_measurement.py; question3/results/round18_outer_main/code_snapshot/question3/shared_measurement.py; question3/results/round18_outer_rounding/code_snapshot/question3/shared_measurement.py

```
1 """S: certify another already-detected source at a shared second point."""
2 import math
3 from question2.strategy import candidate
4
5
6 def compatible_offset(first,bearing,point,error_deg,min_angle):
7     a=math.radians(bearing);dx=point[0]-first[0];dy=point[1]-first[1]
8     q=(dx*math.cos(a)+dy*math.sin(a),-dx*math.sin(a)+dy*math.cos(a))
9     return q if candidate(q,min_angle,error_deg) else None
```

```

10
11
12 def choose_shared(primary,point,first,cleared,cached,config,primary_points=None,decision_log=lambda e:None):
13     mode=config.get('shared_selection','lowest_id')
14     if mode not in ('lowest_id','cost_gate'):raise ValueError('Invalid shared selection mode')
15     ranked=[]
16     for channel in sorted(first):
17         if channel==primary or channel in cleared or channel in cached:continue
18         offset=compatible_offset(*first[channel],point,config['error_deg'],config['min_angle_deg'])
19         if offset is not None:
20             selection=dict(mode='shared_certified',offset=offset,shared_with=primary,
21                             error_deg=config['error_deg'],min_angle_deg=config['min_angle_deg'])
22             if mode=='lowest_id':return channel,selection
23             if not primary_points:raise ValueError('Cost gate needs the primary optical plan')
24             from .shared_cost import shared_saving
25             prediction=shared_saving(*first[channel],offset,primary_points,config)
26             ranked.append((channel,**selection,**prediction, 'gate_mode':mode))
27     if mode=='cost_gate':
28         acceptable=[row for row in ranked if row[1]['predicted_saving_s']>1e-8]
29         selected=min(acceptable,key=lambda row:(-row[1]['predicted_saving_s'],row[0])) if acceptable else None
30         decision_log(dict(type='shared_gate',primary_channel=primary,selected_channel=selected[0] if selected else
31                             ↪ None,
32                             candidates=[dict(channel=c,**s) for c,s in ranked]))
33     return selected
34     return None

```

程序 133

对应文件: question3/results/round13_adverse_main/code_snapshot/question3/strategy.py; question3/results/round13_adverse_rounding/code_snapshot/question3/strategy.py; question3/results/round13_dev_main/code_snapshot/question3/strategy.py; question3/results/round13_dev_rounding/code_snapshot/question3/strategy.py; question3/results/round13_holdout_main/code_snapshot/question3/strategy.py; question3/results/round13_holdout_rounding/code_snapshot/question3/strategy.py; question3/results/round13_outer_main/code_snapshot/question3/strategy.py; question3/results/round13_outer_rounding/code_snapshot/question3/strategy.py

```

1 """Common complete workflow with optional modules A/B/C. Defaults preserve baseline_v1.
2 No imports of case generation/simulator; all decisions use public observations only.
3 """
4 import math
5 from .geometry import stations, station_cover_bound, optical_plan
6 from .modules import choose_second,EvidenceCover
7 from .lookahead import decide
8 from .optical_shortcut import single_circle
9 from .shared_measurement import choose_shared
10 from .order_routing import decide_order
11 from .cached_routing import reorder_cached
12 from .optical_order import nearer_end
13 from .clear_skip import constraints,failure_certificate,radius_failure_certificate
14
15 class Incomplete(RuntimeError):pass
16
17
18 def run(client, config, event=lambda data:None):
19     opportunity=config.get('second_point_opportunity','off')
20     if opportunity not in ('off','current_if_cheaper','current_if_certified'):
21         raise ValueError('Unknown second_point_opportunity')
22     if opportunity!='off' and not config.get('modules',{}).get('A',False):
23         raise ValueError('Current-point comparison requires A')
24     opportunity_timing=config.get('opportunity_timing','before_schedule')
25     if opportunity_timing not in ('before_schedule','after_schedule'):
26         raise ValueError('Unknown opportunity_timing')
27     planning_config={**config,'second_point_opportunity':'off'} if opportunity_timing=='after_schedule' else config
28     count_stop=config.get('count_stop','off')

```

```

29     if count_stop not in ('off','cleared_16','detected_16'):
30         raise ValueError('Unknown count_stop; Q3 source upper bound is fixed at 16')
31     search_radius=config.get('search_radius_m',1500.)
32     if station_cover_bound(search_radius)>=1000:raise ValueError('Invalid search coverage')
33     empty_radius=config.get('empty_origin_radius_m')
34     if empty_radius is not None and station_cover_bound(empty_radius)>=1000:
35         raise ValueError('Invalid empty-origin search coverage')
36     cover=stations(search_radius)
37     negative={c:set() for c in range(1,21)};first={};cleared=set();regions=[];visited=set()
38     modules=config.get('modules',{});joint=modules.get('B',False);evidence=modules.get('C',False)
39     shared=modules.get('S',False);order=modules.get('O',False);cached={}
40     clear_skip=config.get('clear_skip','off');skip_planes={}
41     if clear_skip not in ('off','separating_halfplane','separating_pair'):raise ValueError('Unknown clear_skip')
42     if clear_skip!='off' and evidence:raise ValueError('Clear skipping requires the fixed optical path; C is
↳ incompatible')
43     radius_priors=config.get('clear_radius_priors','off')
44     if radius_priors not in ('off','domain','received','both'):raise ValueError('Unknown clear_radius_priors')
45     if radius_priors!='off' and clear_skip=='off':raise ValueError('Radius priors require clear_skip')
46     optical_order=config.get('optical_order','original')
47     if optical_order not in ('original','nearer_end'):
48         raise ValueError('Unknown optical_order')
49     if optical_order!='original' and evidence:
50         raise ValueError('Optical reversal requires a fixed optical path; C is incompatible')
51     cache_route=config.get('cache_routing','nearest')
52     if cache_route not in ('nearest','station_insertion'):
53         raise ValueError('Unknown cache_routing')
54     if cache_route!='nearest' and (not shared or not joint or order or modules.get('C',False)):
55         raise ValueError('Cache insertion requires B/S with fixed optical paths and O disabled')
56     shared_limit=config.get('shared_batch_limit',1)
57     if type(shared_limit) is not int or not 1<=shared_limit<=19:
58         raise ValueError('shared_batch_limit must be an integer in [1,19]')
59     current_shared_limit=config.get('current_point_shared_limit',shared_limit)
60     if type(current_shared_limit) is not int or not 0<=current_shared_limit<=19:
61         raise ValueError('current_point_shared_limit must be an integer in [0,19]')
62     if (shared or order) and (not joint or config.get('routing',{}).get('mode','nearest')!='nearest'):
63         raise ValueError('S/O require joint B with nearest routing; legacy H scoring is incompatible')
64     if evidence and config['optical_cell_m']!=28:raise ValueError('Evidence module currently certifies 28m coarse
↳ cells only')
65     client.enter();failure=None;exit_confirmed=False;coverage_emitted=False;consecutive_localizations=0
66     stopping_certificate=None;count_discovery_emitted=False
67     def scan(station_id):
68         s=cover[station_id]
69         for channel in range(1,21):
70             # Q3 and Appendix 2 explicitly bound sources by 16 distinct channels.
71             # Returning early does NOT mark this partially scanned station complete.
72             if count_stop!='off' and len(cleared)==16:return
73             if count_stop=='detected_16' and len(set(first)|cleared)==16:return
74             if channel in first or channel in cleared:continue
75             response=client.measure(s,channel,phase='search');kind=response['measure_result']
76             if kind=='no_signal':negative[channel].add(station_id)
77             elif kind=='near':
78                 if client.clear(s,channel,phase='near')['clear_result']!='success':raise Incomplete('near followed by
↳ failed clear')
79                 cleared.add(channel)
80             elif kind=='direction':first[channel]=(s,response['svd_deg'])
81             else:raise Incomplete('Unknown measure result')
82         visited.add(station_id)
83         if station_id==0 and empty_radius is not None and not first and not cleared:
84             # Origin remains unchanged: its negatives stay valid for either certified ring.
85             cover[:]=stations(empty_radius)
86             event(dict(type='search_ring_switch',reason='all_twenty_origin_channels_no_signal',
87                 configured_radius_m=search_radius,selected_radius_m=empty_radius))
88     def prepare(channel,q,r,selection):
89         s,theta=first[channel]
90         planner=None
91         if r['measure_result']=='near':points=[q];certificate=dict(planned_clear_calls=1,covering_radius=5)
92         elif r['measure_result']=='direction':
93             points,certificate=optical_plan(s,theta,q,r['svd_deg'],config['error_deg'],config['optical_cell_m'],confi
↳ g['max_optical_points'])
94             if clear_skip!='off':skip_planes[channel]=constraints(s,theta,q,r['svd_deg'],config['error_deg'])
95             shortcut=single_circle(certificate['vertices']) if modules.get('D',False) and len(points)>1 else None
96             if shortcut:
97                 centre,radius=shortcut

```

```

98         certificate.update(single_circle_centre=centre,original_grid_calls=len(points),
99                             planned_clear_calls=1,covering_radius=radius)
100         points=[centre]
101     elif evidence and len(points)<=128:
102         planner=EvidenceCover(points,certificate,s,theta,q,r['svd_deg'],[cover[k] for k in
            ↪ sorted(negative[channel])],config['error_deg'])
103     elif evidence:
104         certificate['evidence_fallback']='more_than_128_coarse_cells_use_original_cover'
105     else:raise Incomplete('Guaranteed-reception second point returned no_signal')
106     certificate.update(channel=channel,first_position=s,second_position=q,selection=selection)
107     regions.append(certificate);event(dict(type='localization',**certificate))
108     return points,certificate,planner
109 def orient(channel,points):
110     if optical_order=='original':return points
111     chosen=nearer_end(points,client.position)
112     event(dict(type='optical_path_order',channel=channel,reversed=chosen is not points,
113               original_entry=points[0],selected_entry=chosen[0],
114               entry_distance_saved_m=math.dist(client.position,points[0])-math.dist(client.position,chosen[0])))
115     return chosen
116 def localize(channel,q,selection):
117     if channel in cached:
118         points,certificate,planner=cached.pop(channel)
119         points=orient(channel,points)
120         event(dict(type='shared_cache_use',channel=channel,measurement_point=certificate['second_position'],
121                   optical_entry=points[0]))
122     else:
123         if opportunity_timing=='after_schedule':
124             planned=q
125             q,selection=choose_second(*first[channel],client.position,config)
126             event(dict(type='second_point_after_schedule',channel=channel,scheduled_point=planned,
127                       actual_point=q,selection_mode=selection['mode'],point_changed=q!=planned))
128             r=client.measure(q,channel,phase='localization')
129             points,certificate,planner=prepare(channel,q,r,selection)
130             points=orient(channel,points)
131             batch_limit=current_shared_limit if selection['mode']=='opportunistic_current' else shared_limit
132             for _ in range(batch_limit if shared else 0):
133                 opportunity=choose_shared(channel,q,first,cleared,cached,config,primary_points=points,decision_log=ev
            ↪ ent)
134                 if opportunity is None:break
135                 other,sel=opportunity
136                 response=client.measure(q,other,phase='localization_shared')
137                 cached[other]=prepare(other,q,response,sel)
138                 event(dict(type='shared_cache_store',channel=other,primary_channel=channel,point=q,
139                           cached_channels=sorted(cached)))
140         for i in range(len(points)):
141             if planner:
142                 k,p,info=planner.next_point(client.position);event(dict(type='evidence_choice',channel=channel,point=
            ↪ p,**info))
143             else:p=points[i]
144             if clear_skip!='off':
145                 proof=failure_certificate(p,skip_planes.get(channel),combine=clear_skip=='separating_pair')
146                 if proof is None and radius_priors!='off':
147                     proof=radius_failure_certificate(p,first[channel][0],certificate['second_position'],radius_priors)
148                 if proof:
149                     event(dict(type='certified_clear_skip',channel=channel,point=p,original_index=i,error_deg=config[
            ↪ 'error_deg'],**proof))
150                     continue
151             if client.clear(p,channel,phase='optical')['clear_result']=='success':
152                 cleared.add(channel)
153                 if planner:
154                     certificate['evidence']=planner.metrics();event(dict(type='evidence_result',channel=channel,**pla
            ↪ nner.metrics()))
155                 break
156             if planner:planner.failed(k)
157         else:raise Incomplete('All optical covering points failed: model/protocol inconsistency')
158     try:
159         while True:
160             known=set(first)|cleared
161             if count_stop!='off' and len(known)>16:
162                 raise Incomplete('Observed source channels contradict Q3 upper bound 16')
163             if count_stop!='off' and len(cleared)==16:break
164             pending=sorted(set(first)-cleared);unvisited=sorted(set(range(len(cover)))-visited)
165             count_saturated=count_stop=='detected_16' and len(known)==16

```

```

166         remaining=[] if count_saturated else unvisited
167     if count_saturated and not count_discovery_emitted:
168         event(dict(type='source_upper_bound_saturated',source_upper_bound=16,
169                 observed_channels=sorted(known),still_to_clear=pending,
170                 unvisited_station_ids=unvisited))
171         count_discovery_emitted=True
172     if not unvisited and not coverage_emitted:
173         event(dict(type='coverage_complete',detected=sorted(first),cleared=sorted(cleared),
174                 absent=[c for c in negative if len(negative[c])==len(cover)]));coverage_emitted=True
175     if not pending and not remaining:break
176     if not joint:
177         if remaining:scan(remaining[0]);continue
178         channel=pending[0];s,theta=first[channel];q,selection=choose_second(s,theta,client.position,planning_
179         ↪ config)
180         localize(channel,q,selection);continue
181     # B: choose nearest task start, force coverage after at most two localizations.
182     jobs=[]
183     ready_cache={c:(nearer_end(p,client.position),cert,planner) for c,(p,cert,planner) in cached.items()} if
184     ↪ optical_order=='nearer_end' else cached
185     for c in pending:
186         if c in cached:
187             q=ready_cache[c][0][0];sel=dict(mode='shared_ready',ready=True)
188             else:q,sel=choose_second(*first[c],client.position,planning_config)
189             jobs.append((math.dist(client.position,q),c,q,sel))
190         args=(client.position,client.channel,cover,remaining,jobs,first,negative,
191             [c for c in range(1,21) if c not in first and c not in cleared],config)
192         forced=bool(remaining and consecutive_localizations>=2)
193         decision=decide_order(*args,cached=ready_cache,forced=forced) if order else decide(*args,forced=forced)
194         if cache_route=='station_insertion':
195             decision=reorder_cached(decision,client.position,cover,remaining,jobs,ready_cache)
196         task,target,info=decision
197         if task=='scan':
198             event(dict(type='schedule',task='scan',station_id=target,**info))
199             scan(target);consecutive_localizations+=1
200         else:
201             _,c,q,sel=target;event(dict(type='schedule',task='localize',channel=c,**info))
202             localize(c,q,sel);consecutive_localizations+=1
203     if count_stop!='off' and len(cleared)==16:
204         stopping_certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
205             successful_channels=sorted(cleared),
206             inferred_absent_channels=sorted(set(range(1,21))-cleared))
207     elif all(c in cleared or len(negative[c])==len(cover) for c in range(1,21)):
208         stopping_certificate=dict(method='seven_station_channel_cover')
209     else:
210         raise Incomplete('Stopping certificate is incomplete')
211     if not 10<=len(cleared)<=16:raise Incomplete('Cleared count contradicts Q3 count bounds')
212     exit_confirmed=client.exit()['exit_reason']=='user_exit'
213 except (RuntimeError, ValueError, KeyError) as exc:
214     failure=f'{type(exc).__name__}: {exc}'
215 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
216     exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
217     absent_channels=sorted(set(c for c in negative if len(negative[c])==len(cover))|
218         set((stopping_certificate or {}).get('inferred_absent_channels',[]))),
219     negative_station_ids={str(c):sorted(v) for c,v in negative.items()},station_positions=cover,
220     stopping_certificate=stopping_certificate,
221     regions=regions,metrics=client.metrics(),modules=modules)
222 event(dict(type='strategy_result',**result));return result

```

程序 134

对应文件: question3/results/round14_adverse_main/code_snapshot/question3/audit.py;question3/results/round14_adverse_rounding/code_snapshot/question3/audit.py;question3/results/round14_dev_main/code_snapshot/question3/audit.py;question3/results/round14_dev_rounding/code_snapshot/question3/audit.py;question3/results/round14_holdout_main/code_snapshot/question3/audit.py;question3/results/round14_holdout_rounding/code_snapshot/question3/audit.py;question3/results/round14_outer_main/code_snapshot/question3/audit.py;

question3/results/round14_outer_rounding/code_snapshot/question3/audit.py;
question3/results/round17_adverse_main/code_snapshot/question3/audit.py;
question3/results/round17_adverse_rounding/code_snapshot/question3/audit
.py; question3/results/round17_dev_main/code_snapshot/question3/audit.py;
question3/results/round17_dev_rounding/code_snapshot/question3/audit.py

```

1  """Post-run oracle audit: evaluator only, NEVER called by the strategy."""
2  import argparse
3  import json
4  import math
5  import statistics
6  from pathlib import Path
7  import numpy as np
8  from question2.strategy import candidate
9  from .geometry import stations, station_cover_bound
10
11
12 def audit_folder(folder):
13     case=json.loads((folder/'evaluator_hidden_case.json').read_text(encoding='utf-8'))
14     result=json.loads((folder/'result.json').read_text(encoding='utf-8'))
15     sources={s['channel']:s for s in case['sources']};requests={};cleared=set();negative={c:set() for c in
↪ range(1,21)}
16     position=(0.,0.);channel=1;vt=0.;done=set();bearings={};measure_moves={};checks=0;outer_entry=None
17     for line in (folder/'actions.jsonl').read_text(encoding='utf-8').splitlines():
18         row=json.loads(line)
19         if row['type']=='request':requests[row['payload']]['request_id']=row
20         elif row['type']=='response' and row['response'].get('accepted') is True:
21             rid=row['request_id']
22             if rid in done:continue
23             done.add(rid);req=requests[rid];b=req['payload'];r=row['response'];path=req['path']
24             if path in ('/measure','/clear'):
25                 p=(b['position']['x'],b['position']['y']);c=b['channel'];move=math.dist(position,p)
26                 if path=='/measure' and req['phase']=='search' and math.hypot(*p)>1e-7 and outer_entry is None:
27                     outer_entry=(position,p)
28                 vt+=move/5;position=p
29                 s=sources.get(c);distance=math.inf if s is None or c in cleared else
↪ math.hypot(s['x']-p[0],s['y']-p[1])
30             if path=='/measure':
31                 measure_moves[p,c]=move
32                 vt+=5*(c!=channel);channel=c
33                 expected='no_signal' if s is None or c in cleared or distance>s['radius'] else ('near' if
↪ distance<=5 else 'direction')
34                 assert r['measure_result']==expected,(folder,expected,r)
35                 if expected=='no_signal' and req['phase']=='search':negative[c].add(p)
36                 if expected=='direction':
37                     true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
38                     envelope=1. if result['summary']['rounding']=='bounded' else 1.005
39                     assert abs((r['svd_deg']-true+180)%360-180)<=envelope+1e-9
40                     key=(p,c)
41                     if key in bearings:assert bearings[key]==r['svd_deg']
42                     bearings[key]=r['svd_deg']
43             else:
44                 vt+=3
45                 if distance<=20:
46                     assert r['clear_result']=='success';cleared.add(c);vt+=2
47                 else:assert r['clear_result']=='no_target_in_range'
48             checks+=1
49             assert abs(vt-r['virtual_time_s'])<=len(done)*1e-6+1e-4
50     strategy=result['strategy']
51     assert strategy['status']=='complete' and strategy['exit_confirmed']
52     assert set(strategy['cleared_channels'])==cleared
53     cover={tuple(s) for s in strategy['station_positions']}
54     assert len(cover)==7
55     radius=math.dist((0,0),strategy['station_positions'][1])
56     assert station_cover_bound(radius)<1000
57     phase=strategy.get('station_phase_rad',0.)
58     assert math.isfinite(phase)
59     assert np.allclose(strategy['station_positions'],stations(radius,phase),atol=1e-7,rtol=0)
60     certificate=strategy.get('stopping_certificate') or {}
61     if certificate.get('method')=='source_count_upper_bound':

```

```

62     # Count accepted SUCCESS feedback on distinct channels, never attempts or detections.
63     assert certificate['source_upper_bound']==16
64     assert len(certificate['successful_channels'])==16
65     assert len(cleared)==16 and set(certificate['successful_channels'])==cleared
66     assert set(certificate['inferred_absent_channels'])==set(range(1,21))-cleared
67     assert len(sources)<=16 # Evaluator checks the published model's applicability.
68 else:
69     for c in range(1,21):assert c in cleared or cover<=negative[c],(folder,c)
70 assert cleared==set(sources)
71 phase_events=[e for e in map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()) if
↪ e['type']=='ring_phase_selected']
72 assert len(phase_events)==(1 if 'station_phase_rad' in strategy else 0)
73 if phase_events:
74     e=phase_events[0];i=e['station_id'];assert 1<=i<=6 and e['visited_before']==[0]
75     assert e['phase_rad']==phase and outer_entry is not None
76     assert tuple(e['position'])==outer_entry[0] and tuple(e['new_station'])==outer_entry[1]
77     assert np.allclose(e['old_station'],stations(radius)[i],atol=1e-7,rtol=0)
78     assert tuple(strategy['station_positions'][i])==outer_entry[1]
79     shortest=abs(math.hypot(*outer_entry[0])-radius)
80     assert abs(math.dist(*outer_entry)-shortest)<1e-7
81     saved=(math.dist(outer_entry[0],e['old_station'])-math.dist(*outer_entry))/5
82     assert saved>=-1e-7 and abs(saved-e['local_movement_saved_s'])<1e-7
83 for region in strategy['regions']:
84     selection=region.get('selection',{})
85     if selection.get('mode') in ('shared_certified','opportunistic_current'):
86         first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
87         angle=math.radians(bearings[first,c]);dx=second[0]-first[0];dy=second[1]-first[1]
88         offset=(dx*math.cos(angle)+dy*math.sin(angle),-dx*math.sin(angle)+dy*math.cos(angle))
89         assert np.allclose(offset,selection['offset'],atol=1e-7,rtol=0)
90         assert candidate(offset,selection['min_angle_deg'],selection['error_deg'])
91         assert (second,c) in bearings
92         if selection['mode']=='opportunistic_current':assert measure_moves[second,c]==0
93     if 'vertices' not in region:continue
94     g=np.array([sources[region['channel']][['x']],sources[region['channel']][['y']]);v=np.array(region['vertices'])
95     edges=np.roll(v,-1,axis=0)-v;rel=g-v
96     cross=edges[:,0]*rel[:,1]-edges[:,1]*rel[:,0]
97     assert np.all(cross>=-1e-5) or np.all(cross<=1e-5)
98     assert region['covering_radius']<20
99     if 'single_circle_centre' in region:
100         centre=np.array(region['single_circle_centre'])
101         assert np.linalg.norm(v-centre,axis=1).max()<=20-1e-5+1e-9
102         assert np.linalg.norm(g-centre)<=20
103         assert region['planned_clear_calls']==1
104 # Independently recompute each omitted action's separating witness from accepted bearings.
105 from question1.solve import bearing_halfplanes
106 regions={r['channel']:r for r in strategy['regions']}
107 for line in (folder/'strategy.jsonl').read_text(encoding='utf-8').splitlines():
108     event=json.loads(line)
109     if event['type']!='certified_clear_skip':continue
110     c=event['channel'];region=regions[c];first=tuple(region['first_position']);second=tuple(region['second_positi
↪ on'])
111     if 'radius_prior' in event:
112         expected={'domain':((0.,0.),1800.),'first_received':(first,1500.),'second_received':(second,1500.)}
113         centre,radius=expected[event['radius_prior']]
114         assert tuple(event['centre'])==centre and event['containing_radius_m']==radius
115         if event['radius_prior']!='domain':assert (centre,c) in bearings
116         gap=math.dist(event['point'],centre)-radius
117         assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
118         assert abs(gap-event['separation_m'])<1e-8
119         g=(sources[c][['x']],sources[c][['y']])
120         assert math.dist(g,centre)<=radius+1e-7 and math.dist(g,event['point'])>20
121         continue
122     assert event['error_deg']>=(1. if result['summary']['rounding']=='bounded' else 1.005)
123     A,b=bearing_halfplanes([dict(x=p[0],y=p[1],bearing_deg=bearings[p,c]) for p in
↪ [first,second]],event['error_deg'])
124     p=np.asarray(event['point'])
125     if 'halfplane_weights' in event:
126         weights=np.asarray(event['halfplane_weights'])
127         assert weights.shape==(len(A),) and np.isfinite(weights).all() and np.all(weights>=0)
128         assert abs(float(weights.sum())-1)<1e-12
129         normal=weights@A;bound=float(weights@b);length=float(np.linalg.norm(normal))
130         assert length>=1e-8
131         gap=float((normal@p-bound)/length)

```

```

132         assert event['slack_m']>=1e-5/length
133     else:
134         k=event['halfplane_index'];gap=float((A[k]@p-b[k])/np.linalg.norm(A[k]))
135         assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
136         assert abs(gap-event['separation_m'])<1e-8
137         assert math.dist(p,(sources[c]['x'],sources[c]['y']))>20
138     return checks
139
140
141 def main():
142     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)
143     summary=json.loads((root/'summary.json').read_text(encoding='utf-8'));rows=summary['cases']
144     checks=sum(audit_folder(root/r['case_id']) for r in rows)
145     aggregate=dict(kind='SELF_POST_RUN_AUDIT',runs=len(rows),actions_checked=checks,
146                   all_stopping_certificates_valid=True,all_physics_and_time_checks_passed=True,
147                   mean_virtual_s=statistics.mean(r['metrics']['virtual_time_s'] for r in rows),
148                   mean_case_average_per_clear_s=statistics.mean(r['average_virtual_per_cleared'] for r in rows),
149                   pooled_virtual_per_clear_s=sum(r['metrics']['virtual_time_s'] for r in rows)/sum(r['cleared'] for
150                   ↪ r in rows),
151                   total_cleared_over_runs=sum(r['cleared'] for r in rows),
152                   worst_total_case=max(rows,key=lambda r:r['metrics']['virtual_time_s']),
153                   worst_per_clear_case=max(rows,key=lambda r:r['average_virtual_per_cleared']),
154                   most_failed_clear_case=max(rows,key=lambda r:r['metrics']['counts']['clear_failure']),
155                   component_means={k:statistics.mean(r['metrics']['components'][k] for r in rows) for k in
156                   ↪ rows[0]['metrics']['components']},
157                   min_wall_s=min(r['elapsed_wall_s'] for r in rows),max_wall_s=max(r['elapsed_wall_s'] for r in
158                   ↪ rows))
159     (root/'audit.json').write_text(json.dumps(aggregate,ensure_ascii=False,indent=2),encoding='utf-8')
160     print(json.dumps({k:v for k,v in aggregate.items() if not isinstance(v,dict)},ensure_ascii=False,indent=2))
161
162 if __name__=='__main__':main()

```

程序 135

对应文件: question3/results/round14_adverse_main/code_snapshot/question3/strategy.py; question3/results/round14_adverse_rounding/code_snapshot/question3/strategy.py; question3/results/round14_dev_main/code_snapshot/question3/strategy.py; question3/results/round14_dev_rounding/code_snapshot/question3/strategy.py; question3/results/round14_holdout_main/code_snapshot/question3/strategy.py; question3/results/round14_holdout_rounding/code_snapshot/question3/strategy.py; question3/results/round14_initial_dev_main/code_snapshot/question3/strategy.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/strategy.py; question3/results/round14_outer_main/code_snapshot/question3/strategy.py; question3/results/round14_outer_rounding/code_snapshot/question3/strategy.py

```

1 """Common complete workflow with optional modules A/B/C. Defaults preserve baseline_v1.
2 No imports of case generation/simulator; all decisions use public observations only.
3 """
4 import math
5 from .geometry import stations, station_cover_bound, optical_plan
6 from .modules import choose_second,EvidenceCover
7 from .lookahead import decide
8 from .optical_shortcut import single_circle
9 from .shared_measurement import choose_shared
10 from .order_routing import decide_order
11 from .cached_routing import reorder_cached
12 from .optical_order import nearer_end
13 from .clear_skip import constraints,failure_certificate,radius_failure_certificate
14 from .ring_phase import align_first_entry
15
16 class Incomplete(RuntimeError):pass
17

```



```

18
19 def run(client, config, event=lambda data:None):
20     opportunity=config.get('second_point_opportunity','off')
21     if opportunity not in ('off','current_if_cheaper','current_if_certified'):
22         raise ValueError('Unknown second_point_opportunity')
23     if opportunity!='off' and not config.get('modules',{}).get('A',False):
24         raise ValueError('Current-point comparison requires A')
25     opportunity_timing=config.get('opportunity_timing','before_schedule')
26     if opportunity_timing not in ('before_schedule','after_schedule'):
27         raise ValueError('Unknown opportunity_timing')
28     planning_config={**config,'second_point_opportunity':'off'} if opportunity_timing=='after_schedule' else config
29     count_stop=config.get('count_stop','off')
30     if count_stop not in ('off','cleared_16','detected_16'):
31         raise ValueError('Unknown count_stop; Q3 source upper bound is fixed at 16')
32     search_radius=config.get('search_radius_m',1500.)
33     if station_cover_bound(search_radius)>=1000:raise ValueError('Invalid search coverage')
34     empty_radius=config.get('empty_origin_radius_m')
35     if empty_radius is not None and station_cover_bound(empty_radius)>=1000:
36         raise ValueError('Invalid empty-origin search coverage')
37     cover=stations(search_radius)
38     negative={c:set() for c in range(1,21)};first={};cleared=set();regions=[];visited=set()
39     modules=config.get('modules',{});joint=modules.get('B',False);evidence=modules.get('C',False)
40     shared=modules.get('S',False);order=modules.get('O',False);cached={}
41     phase_mode=config.get('ring_phase','fixed');phase_decided=False;station_phase=0.
42     if phase_mode not in ('fixed','first_entry_radial'):raise ValueError('Unknown ring_phase')
43     if phase_mode!='fixed' and not joint:raise ValueError('Adaptive ring phase requires B task selection')
44     clear_skip=config.get('clear_skip','off');skip_planes={}
45     if clear_skip not in ('off','separating_halfplane','separating_pair'):raise ValueError('Unknown clear_skip')
46     if clear_skip!='off' and evidence:raise ValueError('Clear skipping requires the fixed optical path; C is
↳ incompatible')
47     radius_priors=config.get('clear_radius_priors','off')
48     if radius_priors not in ('off','domain','received','both'):raise ValueError('Unknown clear_radius_priors')
49     if radius_priors!='off' and clear_skip=='off':raise ValueError('Radius priors require clear_skip')
50     optical_order=config.get('optical_order','original')
51     if optical_order not in ('original','nearer_end'):
52         raise ValueError('Unknown optical_order')
53     if optical_order!='original' and evidence:
54         raise ValueError('Optical reversal requires a fixed optical path; C is incompatible')
55     cache_route=config.get('cache_routing','nearest')
56     if cache_route not in ('nearest','station_insertion'):
57         raise ValueError('Unknown cache_routing')
58     if cache_route!='nearest' and (not shared or not joint or order or modules.get('C',False)):
59         raise ValueError('Cache insertion requires B/S with fixed optical paths and O disabled')
60     shared_limit=config.get('shared_batch_limit',1)
61     if type(shared_limit) is not int or not 1<=shared_limit<=19:
62         raise ValueError('shared_batch_limit must be an integer in [1,19]')
63     current_shared_limit=config.get('current_point_shared_limit',shared_limit)
64     if type(current_shared_limit) is not int or not 0<=current_shared_limit<=19:
65         raise ValueError('current_point_shared_limit must be an integer in [0,19]')
66     if (shared or order) and (not joint or config.get('routing',{}).get('mode','nearest')!='nearest'):
67         raise ValueError('S/O require joint B with nearest routing; legacy H scoring is incompatible')
68     if evidence and config['optical_cell_m']!=28:raise ValueError('Evidence module currently certifies 28m coarse
↳ cells only')
69     client.enter();failure=None;exit_confirmed=False;coverage_emitted=False;consecutive_localizations=0
70     stopping_certificate=None;count_discovery_emitted=False
71     def scan(station_id):
72         nonlocal phase_decided,station_phase
73         if phase_mode!='fixed' and station_id!=0 and not phase_decided:
74             # Only origin evidence exists. No already-observed outer point may move.
75             if visited!={0} or any(ids-{0} for ids in negative.values()):
76                 raise Incomplete('Cannot rotate a ring after outer station evidence')
77             old=cover[station_id];replacement,station_phase=align_first_entry(cover,station_id,client.position)
78             cover[:]=replacement;phase_decided=True
79             event(dict(type='ring_phase_selected',phase_rad=station_phase,station_id=station_id,
80                 position=client.position,old_station=old,new_station=cover[station_id],
81                 visited_before=sorted(visited),
82                 local_movement_saved_s=(math.dist(client.position,old)-math.dist(client.position,cover[station_id]
↳ _id))/5))
83             s=cover[station_id]
84             for channel in range(1,21):
85                 # Q3 and Appendix 2 explicitly bound sources by 16 distinct channels.
86                 # Returning early does NOT mark this partially scanned station complete.
87                 if count_stop!='off' and len(cleared)==16:return

```

```

88         if count_stop=='detected_16' and len(set(first)|cleared)==16:return
89         if channel in first or channel in cleared:continue
90         response=client.measure(s,channel,phase='search');kind=response['measure_result']
91         if kind=='no_signal':negative[channel].add(station_id)
92         elif kind=='near':
93             if client.clear(s,channel,phase='near')['clear_result']!='success':raise Incomplete('near followed by
↳ failed clear')
94             cleared.add(channel)
95             elif kind=='direction':first[channel]=(s,response['svd_deg'])
96             else:raise Incomplete('Unknown measure result')
97         visited.add(station_id)
98         if station_id==0 and empty_radius is not None and not first and not cleared:
99             # Origin remains unchanged: its negatives stay valid for either certified ring.
100             cover[:]=stations(empty_radius)
101             event(dict(type='search_ring_switch',reason='all_twenty_origin_channels_no_signal',
102                 configured_radius_m=search_radius,selected_radius_m=empty_radius))
103     def prepare(channel,q,r,selection):
104         s,theta=first[channel]
105         planner=None
106         if r['measure_result']=='near':points=[q];certificate=dict(planned_clear_calls=1,covering_radius=5)
107         elif r['measure_result']=='direction':
108             points,certificate=optical_plan(s,theta,q,r['svd_deg'],config['error_deg'],config['optical_cell_m'],confi
↳ g['max_optical_points'])
109             if clear_skip!='off':skip_planes[channel]=constraints(s,theta,q,r['svd_deg'],config['error_deg'])
110             shortcut=single_circle(certificate['vertices']) if modules.get('D',False) and len(points)>1 else None
111             if shortcut:
112                 centre,radius=shortcut
113                 certificate.update(single_circle_centre=centre,original_grid_calls=len(points),
114                     planned_clear_calls=1,covering_radius=radius)
115                 points=[centre]
116             elif evidence and len(points)<=128:
117                 planner=EvidenceCover(points,certificate,s,theta,q,r['svd_deg'],[cover[k] for k in
↳ sorted(negative[channel])],config['error_deg'])
118             elif evidence:
119                 certificate['evidence_fallback']='more_than_128_coarse_cells_use_original_cover'
120             else:raise Incomplete('Guaranteed-reception second point returned no_signal')
121             certificate.update(channel=channel,first_position=s,second_position=q,selection=selection)
122             regions.append(certificate);event(dict(type='localization',**certificate))
123             return points,certificate,planner
124     def orient(channel,points):
125         if optical_order=='original':return points
126         chosen=nearer_end(points,client.position)
127         event(dict(type='optical_path_order',channel=channel,reversed=chosen is not points,
128             original_entry=points[0],selected_entry=chosen[0],
129             entry_distance_saved_m=math.dist(client.position,points[0])-math.dist(client.position,chosen[0])))
130         return chosen
131     def localize(channel,q,selection):
132         if channel in cached:
133             points,certificate,planner=cached.pop(channel)
134             points=orient(channel,points)
135             event(dict(type='shared_cache_use',channel=channel,measurement_point=certificate['second_position'],
136                 optical_entry=points[0]))
137         else:
138             if opportunity_timing=='after_schedule':
139                 planned=q
140                 q,selection=choose_second(*first[channel],client.position,config)
141                 event(dict(type='second_point_after_schedule',channel=channel,scheduled_point=planned,
142                     actual_point=q,selection_mode=selection['mode'],point_changed=q!=planned))
143                 r=client.measure(q,channel,phase='localization')
144                 points,certificate,planner=prepare(channel,q,r,selection)
145                 points=orient(channel,points)
146                 batch_limit=current_shared_limit if selection['mode']=='opportunistic_current' else shared_limit
147                 for _ in range(batch_limit if shared else 0):
148                     opportunity=choose_shared(channel,q,first,cleared,cached,config,primary_points=points,decision_log=ev
↳ ent)
149                     if opportunity is None:break
150                     other,sel=opportunity
151                     response=client.measure(q,other,phase='localization_shared')
152                     cached[other]=prepare(other,q,response,sel)
153                     event(dict(type='shared_cache_store',channel=other,primary_channel=channel,point=q,
154                         cached_channels=sorted(cached)))
155             for i in range(len(points)):
156                 if planner:

```

```

157         k,p,info=planner.next_point(client.position);event(dict(type='evidence_choice',channel=channel,point=
        ↪ p,**info))
158     else:p=points[i]
159     if clear_skip!='off':
160         proof=failure_certificate(p,skip_planes.get(channel),combine=clear_skip=='separating_pair')
161         if proof is None and radius_priors!='off':
162             proof=radius_failure_certificate(p,first[channel][0],certificate['second_position'],radius_priors)
163         if proof:
164             event(dict(type='certified_clear_skip',channel=channel,point=p,original_index=i,error_deg=config[
        ↪ 'error_deg'],**proof))
165             continue
166     if client.clear(p,channel,phase='optical')['clear_result']=='success':
167         cleared.add(channel)
168         if planner:
169             certificate['evidence']=planner.metrics();event(dict(type='evidence_result',channel=channel,**pla
        ↪ ner.metrics()))
170         break
171     if planner:planner.failed(k)
172 else:raise Incomplete('All optical covering points failed: model/protocol inconsistency')
173 try:
174     while True:
175         known=set(first)|cleared
176         if count_stop!='off' and len(known)>16:
177             raise Incomplete('Observed source channels contradict Q3 upper bound 16')
178         if count_stop!='off' and len(cleared)==16:break
179         pending=sorted(set(first)-cleared);unvisited=sorted(set(range(len(cover)))-visited)
180         count_saturated=count_stop=='detected_16' and len(known)==16
181         remaining=[] if count_saturated else unvisited
182         if count_saturated and not count_discovery_emitted:
183             event(dict(type='source_upper_bound_saturated',source_upper_bound=16,
184                 observed_channels=sorted(known),still_to_clear=pending,
185                 unvisited_station_ids=unvisited))
186             count_discovery_emitted=True
187         if not unvisited and not coverage_emitted:
188             event(dict(type='coverage_complete',detected=sorted(first),cleared=sorted(cleared),
189                 absent=[c for c in negative if len(negative[c])==len(cover)]));coverage_emitted=True
190         if not pending and not remaining:break
191         if not joint:
192             if remaining:scan(remaining[0]);continue
193             channel=pending[0];s,theta=first[channel];q,selection=choose_second(s,theta,client.position,planning_
        ↪ config)
194             localize(channel,q,selection);continue
195         # B: choose nearest task start, force coverage after at most two localizations.
196         jobs=[]
197         ready_cache={c:(nearer_end(p,client.position),cert,planner) for c,(p,cert,planner) in cached.items()} if
        ↪ optical_order=='nearer_end' else cached
198         for c in pending:
199             if c in cached:
200                 q=ready_cache[c][0][0];sel=dict(mode='shared_ready',ready=True)
201                 else:q,sel=choose_second(*first[c],client.position,planning_config)
202                 jobs.append((math.dist(client.position,q),c,q,sel))
203             args=(client.position,client.channel,cover,remaining,jobs,first,negative,
204                 [c for c in range(1,21) if c not in first and c not in cleared],config)
205             forced=bool(remaining and consecutive_localizations>=2)
206             decision=decide_order(*args,cached=ready_cache,forced=forced) if order else decide(*args,forced=forced)
207             if cache_route=='station_insertion':
208                 decision=reorder_cached(decision,client.position,cover,remaining,jobs,ready_cache)
209             task,target,info=decision
210             if task=='scan':
211                 event(dict(type='schedule',task='scan',station_id=target,**info))
212                 scan(target);consecutive_localizations+=1
213             else:
214                 _,c,q,sel=target;event(dict(type='schedule',task='localize',channel=c,**info))
215                 localize(c,q,sel);consecutive_localizations+=1
216         if count_stop!='off' and len(cleared)==16:
217             stopping_certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
218                 successful_channels=sorted(cleared),
219                 inferred_absent_channels=sorted(set(range(1,21))-cleared))
220         elif all(c in cleared or len(negative[c])==len(cover) for c in range(1,21)):
221             stopping_certificate=dict(method='seven_station_channel_cover')
222         else:
223             raise Incomplete('Stopping certificate is incomplete')
224         if not 10<=len(cleared)<=16:raise Incomplete('Cleared count contradicts Q3 count bounds')

```

```

225         exit_confirmed=client.exit()['exit_reason']=='user_exit'
226     except (RuntimeError, ValueError, KeyError) as exc:
227         failure=f'{type(exc).__name__}: {exc}'
228     result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete', failure=failure,
229                exit_confirmed=exit_confirmed, cleared_channels=sorted(cleared),
230                absent_channels=sorted(set(c for c in negative if len(negative[c])==len(cover))),
231                set((stopping_certificate or {}).get('inferred_absent_channels', []))),
232                negative_station_ids={str(c):sorted(v) for c,v in negative.items()}, station_positions=cover,
233                stopping_certificate=stopping_certificate,
234                regions=regions, metrics=client.metrics(), modules=modules)
235     if phase_decided: result['station_phase_rad']=station_phase
236     event(dict(type='strategy_result', **result)); return result

```

程序 136

对应文件: question3/results/round14_initial_dev_main/code_snapshot/question3/audit.py; question3/results/round14_initial_dev_rounding/code_snapshot/question3/audit.py

```

1  """Post-run oracle audit: evaluator only, NEVER called by the strategy."""
2  import argparse
3  import json
4  import math
5  import statistics
6  from pathlib import Path
7  import numpy as np
8  from question2.strategy import candidate
9  from .geometry import stations, station_cover_bound
10
11
12  def audit_folder(folder):
13      case=json.loads((folder/'evaluator_hidden_case.json').read_text(encoding='utf-8'))
14      result=json.loads((folder/'result.json').read_text(encoding='utf-8'))
15      sources={s['channel']:s for s in case['sources']}; requests={}; cleared=set(); negative={c:set() for c in
16      ↪ range(1,21)}
17      position=(0.,0.); channel=1; vt=0.; done=set(); bearings={}; measure_moves={}; checks=0; outer_entry=None
18      for line in (folder/'actions.jsonl').read_text(encoding='utf-8').splitlines():
19          row=json.loads(line)
20          if row['type']=='request': requests[row['payload']['request_id']]=row
21          elif row['type']=='response' and row['response'].get('accepted') is True:
22              rid=row['request_id']
23              if rid in done: continue
24              done.add(rid); req=requests[rid]; b=req['payload']; r=row['response']; path=req['path']
25              if path in ('/measure', '/clear'):
26                  p=(b['position']['x'], b['position']['y']); c=b['channel']; move=math.dist(position, p)
27                  if path=='/measure' and req['phase']=='search' and math.hypot(*p)>1e-7 and outer_entry is None:
28                      outer_entry=(position, p)
29                  vt+=move/5; position=p
30                  s=sources.get(c); distance=math.inf if s is None or c in cleared else
31                  ↪ math.hypot(s['x']-p[0], s['y']-p[1])
32                  if path=='/measure':
33                      measure_moves[p, c]=move
34                      vt+=5+(c!=channel); channel=c
35                      expected='no_signal' if s is None or c in cleared or distance>s['radius'] else ('near' if
36                      ↪ distance<=5 else 'direction')
37                      assert r['measure_result']==expected, (folder, expected, r)
38                      if expected=='no_signal' and req['phase']=='search': negative[c].add(p)
39                      if expected=='direction':
40                          true=math.degrees(math.atan2(s['y']-p[1], s['x']-p[0]))
41                          envelope=1. if result['summary']['rounding']=='bounded' else 1.005
42                          assert abs((r['svd_deg']-true+180)%360-180)<=envelope+1e-9
43                          key=(p, c)
44                          if key in bearings: assert bearings[key]==r['svd_deg']
45                          bearings[key]=r['svd_deg']
46          else:
47              vt+=3
48              if distance<=20:
49                  assert r['clear_result']=='success'; cleared.add(c); vt+=2
50              else: assert r['clear_result']=='no_target_in_range'
51      checks+=1

```

```

49         assert abs(vt-r['virtual_time_s'])<=len(done)*1e-6+1e-4
50     strategy=result['strategy']
51     assert strategy['status']=='complete' and strategy['exit_confirmed']
52     assert set(strategy['cleared_channels'])==cleared
53     cover={tuple(s) for s in strategy['station_positions']}
54     assert len(cover)==7
55     radius=math.dist((0,0),strategy['station_positions'][1])
56     assert station_cover_bound(radius)<1000
57     phase=strategy.get('station_phase_rad',0.)
58     assert math.isfinite(phase)
59     assert np.allclose(strategy['station_positions'],stations(radius,phase),atol=1e-7,rtol=0)
60     phase_events=[e for e in map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()) if
    ↪ e['type']=='ring_phase_selected']
61     assert len(phase_events)==(1 if 'station_phase_rad' in strategy else 0)
62     if phase_events:
63         e=phase_events[0];i=e['station_id'];assert 1<=i<=6 and e['visited_before']==[0]
64         assert e['phase_rad']==phase and outer_entry is not None
65         assert tuple(e['position'])==outer_entry[0] and tuple(e['new_station'])==outer_entry[1]
66         assert np.allclose(e['old_station'],stations(radius)[i],atol=1e-7,rtol=0)
67         assert tuple(strategy['station_positions'][i])==outer_entry[1]
68         shortest=abs(math.hypot(*outer_entry[0])-radius)
69         assert abs(math.dist(*outer_entry)-shortest)<1e-7
70         saved=(math.dist(outer_entry[0],e['old_station'])-math.dist(*outer_entry))/5
71         assert saved>=-1e-7 and abs(saved-e['local_movement_saved_s'])<1e-7
72     certificate=strategy.get('stopping_certificate') or {}
73     if certificate.get('method')=='source_count_upper_bound':
74         # Count accepted SUCCESS feedback on distinct channels, never attempts or detections.
75         assert certificate['source_upper_bound']==16
76         assert len(certificate['successful_channels'])==16
77         assert len(cleared)==16 and set(certificate['successful_channels'])==cleared
78         assert set(certificate['inferred_absent_channels'])==set(range(1,21))-cleared
79         assert len(sources)<=16 # Evaluator checks the published model's applicability.
80     else:
81         for c in range(1,21):assert c in cleared or cover<=negative[c],(folder,c)
82     assert cleared==set(sources)
83     for region in strategy['regions']:
84         selection=region.get('selection',{})
85         if selection.get('mode') in ('shared_certified','opportunistic_current'):
86             first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
87             angle=math.radians(bearings[first,c]);dx=second[0]-first[0];dy=second[1]-first[1]
88             offset=(dx*math.cos(angle)+dy*math.sin(angle),-dx*math.sin(angle)+dy*math.cos(angle))
89             assert np.allclose(offset,selection['offset'],atol=1e-7,rtol=0)
90             assert candidate(offset,selection['min_angle_deg'],selection['error_deg'])
91             assert (second,c) in bearings
92             if selection['mode']=='opportunistic_current':assert measure_moves[second,c]==0
93         if 'vertices' not in region:continue
94         g=np.array([sources[region['channel']][x],sources[region['channel']][y]]);v=np.array(region['vertices'])
95         edges=np.roll(v,-1,axis=0)-v;rel=g-v
96         cross=edges[:,0]*rel[:,1]-edges[:,1]*rel[:,0]
97         assert np.all(cross>=-1e-5) or np.all(cross<=1e-5)
98         assert region['covering_radius']<20
99         if 'single_circle_centre' in region:
100             centre=np.array(region['single_circle_centre'])
101             assert np.linalg.norm(v-centre,axis=1).max()<=20-1e-5+1e-9
102             assert np.linalg.norm(g-centre)<=20
103             assert region['planned_clear_calls']==1
104     # Independently recompute each omitted action's separating witness from accepted bearings.
105     from question1.solve import bearing_halfplanes
106     regions={r['channel']:r for r in strategy['regions']}
107     for line in (folder/'strategy.jsonl').read_text(encoding='utf-8').splitlines():
108         event=json.loads(line)
109         if event['type']!='certified_clear_skip':continue
110         c=event['channel'];region=regions[c];first=tuple(region['first_position']);second=tuple(region['second_positi
    ↪ on'])
111         if 'radius_prior' in event:
112             expected={ 'domain':((0.,0.),1800.), 'first_received':(first,1500.), 'second_received':(second,1500.)}
113             centre,radius=expected[event['radius_prior']]
114             assert tuple(event['centre'])==centre and event['containing_radius_m']==radius
115             if event['radius_prior']!='domain':assert (centre,c) in bearings
116             gap=math.dist(event['point'],centre)-radius
117             assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
118             assert abs(gap-event['separation_m'])<1e-8
119             g=(sources[c][x],sources[c][y])

```

```

120         assert math.dist(g,centre)<=radius+1e-7 and math.dist(g,event['point'])>20
121         continue
122         assert event['error_deg']>=(1. if result['summary']['rounding']=='bounded' else 1.005)
123         A,b=bearing_halfplanes([dict(x=p[0],y=p[1],bearing_deg=bearings[p,c]) for p in
↪ [first,second]],event['error_deg'])
124         p=np.asarray(event['point'])
125         if 'halfplane_weights' in event:
126             weights=np.asarray(event['halfplane_weights'])
127             assert weights.shape==(len(A),) and np.isfinite(weights).all() and np.all(weights>=0)
128             assert abs(float(weights.sum())-1)<1e-12
129             normal=weights@A;bound=float(weights@b);length=float(np.linalg.norm(normal))
130             assert length>=1e-8
131             gap=float((normal@p-bound)/length)
132             assert event['slack_m']>=1e-5/length
133         else:
134             k=event['halfplane_index'];gap=float((A[k]@p-b[k])/np.linalg.norm(A[k]))
135             assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
136             assert abs(gap-event['separation_m'])<1e-8
137             assert math.dist(p,(sources[c]['x'],sources[c]['y']))>20
138     return checks
139
140
141 def main():
142     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)
143     summary=json.loads((root/'summary.json').read_text(encoding='utf-8'));rows=summary['cases']
144     checks=sum(audit_folder(root/r['case_id']) for r in rows)
145     aggregate=dict(kind='SELF_POST_RUN_AUDIT',runs=len(rows),actions_checked=checks,
146                   all_stopping_certificates_valid=True,all_physics_and_time_checks_passed=True,
147                   mean_virtual_s=statistics.mean(r['metrics']['virtual_time_s'] for r in rows),
148                   mean_case_average_per_clear_s=statistics.mean(r['average_virtual_per_cleared'] for r in rows),
149                   pooled_virtual_per_clear_s=sum(r['metrics']['virtual_time_s'] for r in rows)/sum(r['cleared'] for
↪ r in rows),
150                   total_cleared_over_runs=sum(r['cleared'] for r in rows),
151                   worst_total_case=max(rows,key=lambda r:r['metrics']['virtual_time_s']),
152                   worst_per_clear_case=max(rows,key=lambda r:r['average_virtual_per_cleared']),
153                   most_failed_clear_case=max(rows,key=lambda r:r['metrics']['counts']['clear_failure']),
154                   component_means={k:statistics.mean(r['metrics']['components'][k] for r in rows) for k in
↪ rows[0]['metrics']['components']},
155                   min_wall_s=min(r['elapsed_wall_s'] for r in rows),max_wall_s=max(r['elapsed_wall_s'] for r in
↪ rows))
156     (root/'audit.json').write_text(json.dumps(aggregate,ensure_ascii=False,indent=2),encoding='utf-8')
157     print(json.dumps({k:v for k,v in aggregate.items() if not isinstance(v,dict)},ensure_ascii=False,indent=2))
158
159 if __name__=='__main__':main()

```

程序 137

对应文件: question3/results/round17_adverse_main/code_snapshot/question3/strategy.py; question3/results/round17_adverse_rounding/code_snapshot/question3/strategy.py; question3/results/round17_dev_main/code_snapshot/question3/strategy.py; question3/results/round17_dev_rounding/code_snapshot/question3/strategy.py

```

1 """Common complete workflow with optional modules A/B/C. Defaults preserve baseline_v1.
2 No imports of case generation/simulator; all decisions use public observations only.
3 """
4 import math
5 from .geometry import stations, station_cover_bound, optical_plan
6 from .modules import choose_second,EvidenceCover
7 from .lookahead import decide
8 from .optical_shortcut import single_circle
9 from .shared_measurement import choose_shared
10 from .order_routing import decide_order
11 from .cached_routing import reorder_cached
12 from .optical_order import nearer_end
13 from .clear_skip import constraints,failure_certificate,radius_failure_certificate
14 from .ring_phase import align_first_entry
15 from .batch_scheduling import gate as batch_gate
16

```

```

17 class Incomplete(RuntimeError):pass
18
19
20 def run(client, config, event=lambda data:None):
21     opportunity=config.get('second_point_opportunity','off')
22     if opportunity not in ('off','current_if_cheaper','current_if_certified'):
23         raise ValueError('Unknown second_point_opportunity')
24     if opportunity!='off' and not config.get('modules',{}).get('A',False):
25         raise ValueError('Current-point comparison requires A')
26     opportunity_timing=config.get('opportunity_timing','before_schedule')
27     if opportunity_timing not in ('before_schedule','after_schedule'):
28         raise ValueError('Unknown opportunity_timing')
29     planning_config={**config,'second_point_opportunity':'off'} if opportunity_timing=='after_schedule' else config
30     count_stop=config.get('count_stop','off')
31     if count_stop not in ('off','cleared_l6','detected_l6'):
32         raise ValueError('Unknown count_stop; Q3 source upper bound is fixed at l6')
33     search_radius=config.get('search_radius_m',1500.)
34     if station_cover_bound(search_radius)>=1000:raise ValueError('Invalid search coverage')
35     empty_radius=config.get('empty_origin_radius_m')
36     if empty_radius is not None and station_cover_bound(empty_radius)>=1000:
37         raise ValueError('Invalid empty-origin search coverage')
38     cover=stations(search_radius)
39     negative={c:set() for c in range(1,21)};first={};cleared=set();regions=[];visited=set()
40     modules=config.get('modules',{});joint=modules.get('B',False);evidence=modules.get('C',False)
41     shared=modules.get('S',False);order=modules.get('O',False);cached={}
42     phase_mode=config.get('ring_phase','fixed');phase_decided=False;station_phase=0.
43     if phase_mode not in ('fixed','first_entry_radial'):raise ValueError('Unknown ring_phase')
44     if phase_mode!='fixed' and not joint:raise ValueError('Adaptive ring phase requires B task selection')
45     batch_mode=config.get('search_interleaving','current')
46     if batch_mode not in ('current','cached_batch','known_batch','nearest_finite'):raise ValueError('Unknown
↪ search_interleaving')
47     clear_skip=config.get('clear_skip','off');skip_planes={}
48     if clear_skip not in ('off','separating_halfplane','separating_pair'):raise ValueError('Unknown clear_skip')
49     if clear_skip!='off' and evidence:raise ValueError('Clear skipping requires the fixed optical path; C is
↪ incompatible')
50     radius_priors=config.get('clear_radius_priors','off')
51     if radius_priors not in ('off','domain','received','both'):raise ValueError('Unknown clear_radius_priors')
52     if radius_priors!='off' and clear_skip=='off':raise ValueError('Radius priors require clear_skip')
53     optical_order=config.get('optical_order','original')
54     if optical_order not in ('original','nearer_end'):
55         raise ValueError('Unknown optical_order')
56     if optical_order!='original' and evidence:
57         raise ValueError('Optical reversal requires a fixed optical path; C is incompatible')
58     cache_route=config.get('cache_routing','nearest')
59     if batch_mode!='current' and (not joint or order or cache_route!='nearest' or
↪ config.get('routing',{}).get('mode','nearest')!='nearest'):
60         raise ValueError('Batch scheduling requires B nearest routing without O/cache insertion')
61     if batch_mode=='cached_batch' and not shared:raise ValueError('Cached batch requires S')
62     scan_order=config.get('scan_order','nearest')
63     if scan_order not in ('nearest','remaining_open_path'):raise ValueError('Unknown scan_order')
64     if scan_order!='nearest' and (not joint or order or cache_route!='nearest' or
↪ config.get('routing',{}).get('mode','nearest')!='nearest' or phase_mode!='fixed'):
65         raise ValueError('Scan path ordering requires B nearest routing, fixed stations, no O/cache insertion')
66     if cache_route not in ('nearest','station_insertion'):
67         raise ValueError('Unknown cache_routing')
68     if cache_route!='nearest' and (not shared or not joint or order or modules.get('C',False)):
69         raise ValueError('Cache insertion requires B/S with fixed optical paths and O disabled')
70     shared_limit=config.get('shared_batch_limit',1)
71     if type(shared_limit) is not int or not 1<=shared_limit<=19:
72         raise ValueError('shared_batch_limit must be an integer in [1,19]')
73     current_shared_limit=config.get('current_point_shared_limit',shared_limit)
74     if type(current_shared_limit) is not int or not 0<=current_shared_limit<=19:
75         raise ValueError('current_point_shared_limit must be an integer in [0,19]')
76     if (shared or order) and (not joint or config.get('routing',{}).get('mode','nearest')!='nearest'):
77         raise ValueError('S/O require joint B with nearest routing; legacy H scoring is incompatible')
78     if evidence and config['optical_cell_m']!=28:raise ValueError('Evidence module currently certifies 28m coarse
↪ cells only')
79     client.enter();failure=None;exit_confirmed=False;coverage_emitted=False;consecutive_localizations=0
80     stopping_certificate=None;count_discovery_emitted=False
81     def scan(station_id):
82         nonlocal phase_decided,station_phase
83         if phase_mode!='fixed' and station_id!=0 and not phase_decided:
84             # Only origin evidence exists. No already-observed outer point may move.

```

```

85         if visited!={0} or any(ids-{0} for ids in negative.values()):
86             raise Incomplete('Cannot rotate a ring after outer station evidence')
87         old=cover[station_id];replacement,station_phase=align_first_entry(cover,station_id,client.position)
88         cover[:]=replacement;phase_decided=True
89         event(dict(type='ring_phase_selected',phase_rad=station_phase,station_id=station_id,
90                 position=client.position,old_station=old,new_station=cover[station_id],
91                 visited_before=sorted(visited),
92                 local_movement_saved_s=(math.dist(client.position,old)-math.dist(client.position,cover[station_id]
93                 ↪ _id))/5))
94         s=cover[station_id]
95         for channel in range(1,21):
96             # Q3 and Appendix 2 explicitly bound sources by 16 distinct channels.
97             # Returning early does NOT mark this partially scanned station complete.
98             if count_stop!='off' and len(cleared)==16:return
99             if count_stop=='detected_16' and len(set(first)|cleared)==16:return
100            if channel in first or channel in cleared:continue
101            response=client.measure(s,channel,phase='search');kind=response['measure_result']
102            if kind=='no_signal':negative[channel].add(station_id)
103            elif kind=='near':
104                if client.clear(s,channel,phase='near')['clear_result']!='success':raise Incomplete('near followed by
105                ↪ failed clear')
106                cleared.add(channel)
107            elif kind=='direction':first[channel]=(s,response['svd_deg'])
108            else:raise Incomplete('Unknown measure result')
109            visited.add(station_id)
110            if station_id==0 and empty_radius is not None and not first and not cleared:
111                # Origin remains unchanged: its negatives stay valid for either certified ring.
112                cover[:]=stations(empty_radius)
113                event(dict(type='search_ring_switch',reason='all_twenty_origin_channels_no_signal',
114                        configured_radius_m=search_radius,selected_radius_m=empty_radius))
115
116    def prepare(channel,q,r,selection):
117        s,theta=first[channel]
118        planner=None
119        if r['measure_result']=='near':points=[q];certificate=dict(planned_clear_calls=1,covering_radius=5)
120        elif r['measure_result']=='direction':
121            points,certificate=optical_plan(s,theta,q,r['svd_deg'],config['error_deg'],config['optical_cell_m'],config
122            ↪ g['max_optical_points'])
123            if clear_skip!='off':skip_planes[channel]=constraints(s,theta,q,r['svd_deg'],config['error_deg'])
124            shortcut=single_circle(certificate['vertices']) if modules.get('D',False) and len(points)>1 else None
125            if shortcut:
126                centre,radius=shortcut
127                certificate.update(single_circle_centre=centre,original_grid_calls=len(points),
128                                planned_clear_calls=1,covering_radius=radius)
129                points=[centre]
130            elif evidence and len(points)<=128:
131                planner=EvidenceCover(points,certificate,s,theta,q,r['svd_deg'],[cover[k] for k in
132                ↪ sorted(negative[channel])],config['error_deg'])
133            elif evidence:
134                certificate['evidence_fallback']='more_than_128_coarse_cells_use_original_cover'
135            else:raise Incomplete('Guaranteed-reception second point returned no_signal')
136            certificate.update(channel=channel,first_position=s,second_position=q,selection=selection)
137            regions.append(certificate);event(dict(type='localization',**certificate))
138            return points,certificate,planner
139
140    def orient(channel,points):
141        if optical_order=='original':return points
142        chosen=nearer_end(points,client.position)
143        event(dict(type='optical_path_order',channel=channel,reversed=chosen is not points,
144                original_entry=points[0],selected_entry=chosen[0],
145                entry_distance_saved_m=math.dist(client.position,points[0])-math.dist(client.position,chosen[0])))
146        return chosen
147
148    def localize(channel,q,selection):
149        if channel in cached:
150            points,certificate,planner=cached.pop(channel)
151            points=orient(channel,points)
152            event(dict(type='shared_cache_use',channel=channel,measurement_point=certificate['second_position'],
153                    optical_entry=points[0]))
154        else:
155            if opportunity_timing=='after_schedule':
156                planned=q
157                q,selection=choose_second(*first[channel],client.position,config)
158                event(dict(type='second_point_after_schedule',channel=channel,scheduled_point=planned,
159                        actual_point=q,selection_mode=selection['mode'],point_changed=q!=planned))
160            r=client.measure(q,channel,phase='localization')

```



```

154     points, certificate, planner=prepare(channel, q, r, selection)
155     points=orient(channel, points)
156     batch_limit=current_shared_limit if selection['mode']=='opportunistic_current' else shared_limit
157     for _ in range(batch_limit if shared else 0):
158         opportunity=choose_shared(channel, q, first, cleared, cached, config, primary_points=points, decision_log=ev
            ↪ ent)
159         if opportunity is None:break
160         other, sel=opportunity
161         response=client.measure(q, other, phase='localization_shared')
162         cached[other]=prepare(other, q, response, sel)
163         event(dict(type='shared_cache_store', channel=other, primary_channel=channel, point=q,
164             cached_channels=sorted(cached)))
165     for i in range(len(points)):
166         if planner:
167             k, p, info=planner.next_point(client.position); event(dict(type='evidence_choice', channel=channel, point=
            ↪ p, **info))
168         else: p=points[i]
169         if clear_skip!='off':
170             proof=failure_certificate(p, skip_planes.get(channel), combine=clear_skip=='separating_pair')
171             if proof is None and radius_priors!='off':
172                 proof=radius_failure_certificate(p, first[channel][0], certificate['second_position'], radius_priors)
173             if proof:
174                 event(dict(type='certified_clear_skip', channel=channel, point=p, original_index=i, error_deg=config[
            ↪ 'error_deg'], **proof))
175                 continue
176         if client.clear(p, channel, phase='optical')['clear_result']=='success':
177             cleared.add(channel)
178             if planner:
179                 certificate['evidence']=planner.metrics(); event(dict(type='evidence_result', channel=channel, **pla
            ↪ nner.metrics()))
180             break
181         if planner: planner.failed(k)
182     else: raise Incomplete('All optical covering points failed: model/protocol inconsistency')
183 try:
184     while True:
185         known=set(first)|cleared
186         if count_stop!='off' and len(known)>16:
187             raise Incomplete('Observed source channels contradict Q3 upper bound 16')
188         if count_stop!='off' and len(cleared)==16: break
189         pending=sorted(set(first)-cleared); unvisited=sorted(set(range(len(cover)))-visited)
190         count_saturated=count_stop=='detected_16' and len(known)==16
191         remaining=[] if count_saturated else unvisited
192         if count_saturated and not count_discovery_emitted:
193             event(dict(type='source_upper_bound_saturated', source_upper_bound=16,
194                 observed_channels=sorted(known), still_to_clear=pending,
195                 unvisited_station_ids=unvisited))
196             count_discovery_emitted=True
197         if not unvisited and not coverage_emitted:
198             event(dict(type='coverage_complete', detected=sorted(first), cleared=sorted(cleared),
199                 absent=[c for c in negative if len(negative[c])==len(cover)])); coverage_emitted=True
200         if not pending and not remaining: break
201         if not joint:
202             if remaining: scan(remaining[0]); continue
203             channel=pending[0]; s, theta=first[channel]; q, selection=choose_second(s, theta, client.position, planning_
            ↪ config)
204             localize(channel, q, selection); continue
205     # B: compare task starts; optional gates govern scan interleaving.
206     jobs=[]
207     ready_cache={c: (nearer_end(p, client.position), cert, planner) for c, (p, cert, planner) in cached.items()} if
            ↪ optical_order=='nearer_end' else cached
208     for c in pending:
209         if c in cached:
210             q=ready_cache[c][0][0]; sel=dict(mode='shared_ready', ready=True)
211             else: q, sel=choose_second(*first[c], client.position, planning_config)
212             jobs.append((math.dist(client.position, q), c, q, sel))
213     scheduled_jobs, scheduled_remaining, forced, batch_info=batch_gate(batch_mode, pending, cached, jobs, remaining,
            ↪ consecutive_localizations)
214     if batch_info: event(dict(type='localization_batch', **batch_info))
215     args=(client.position, client.channel, cover, scheduled_remaining, scheduled_jobs, first, negative,
216         [c for c in range(1, 21) if c not in first and c not in cleared], config)
217     decision=decide_order(*args, cached=ready_cache, forced=forced) if order else decide(*args, forced=forced)
218     if cache_route=='station_insertion':

```

```

219         decision=reorder_cached(decision,client.position,cover,remaining,jobs,ready_cache)
220     task,target,info=decision
221     if task=='scan':
222         event(dict(type='schedule',task='scan',station_id=target,**info))
223         scan(target);consecutive_localizations+=1
224     else:
225         _,c,q,sel=target;event(dict(type='schedule',task='localize',channel=c,**info))
226         localize(c,q,sel);consecutive_localizations+=1
227     if count_stop!='off' and len(cleared)==16:
228         stopping_certificate=dict(method='source_count_upper_bound',source_upper_bound=16,
229             successful_channels=sorted(cleared),
230             inferred_absent_channels=sorted(set(range(1,21))-cleared))
231     elif all(c in cleared or len(negative[c])==len(cover) for c in range(1,21)):
232         stopping_certificate=dict(method='seven_station_channel_cover')
233     else:
234         raise Incomplete('Stopping certificate is incomplete')
235     if not 10<=len(cleared)<=16:raise Incomplete('Cleared count contradicts Q3 count bounds')
236     exit_confirmed=client.exit()['exit_reason']=='user_exit'
237 except (RuntimeError, ValueError, KeyError) as exc:
238     failure=f'{type(exc).__name__}: {exc}'
239 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
240     exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
241     absent_channels=sorted(set(c for c in negative if len(negative[c])==len(cover))|
242         set((stopping_certificate or {}).get('inferred_absent_channels',[]))),
243     negative_station_ids={str(c):sorted(v) for c,v in negative.items()},station_positions=cover,
244     stopping_certificate=stopping_certificate,
245     regions=regions,metrics=client.metrics(),modules=modules)
246 if phase_decided:result['station_phase_rad']=station_phase
247 event(dict(type='strategy_result',**result));return result

```

程序 138

对应文件: question3/results/round18_adverse_main/code_snapshot/question3/audit.py;question3/results/round18_adverse_rounding/code_snapshot/question3/audit.py;question3/results/round18_dev_main/code_snapshot/question3/audit.py;question3/results/round18_dev_rounding/code_snapshot/question3/audit.py;question3/results/round18_holdout_main/code_snapshot/question3/audit.py;question3/results/round18_holdout_rounding/code_snapshot/question3/audit.py;question3/results/round18_outer_main/code_snapshot/question3/audit.py;question3/results/round18_outer_rounding/code_snapshot/question3/audit.py

```

1 """Post-run oracle audit: evaluator only, NEVER called by the strategy."""
2 import argparse
3 import json
4 import math
5 import statistics
6 from pathlib import Path
7 import numpy as np
8 from question2.strategy import candidate
9 from .geometry import stations,station_cover_bound
10
11
12 def audit_folder(folder):
13     case=json.loads((folder/'evaluator_hidden_case.json').read_text(encoding='utf-8'))
14     result=json.loads((folder/'result.json').read_text(encoding='utf-8'))
15     sources={s['channel']:s for s in case['sources']};requests={};cleared=set();negative={c:set() for c in
16     ↪ range(1,21)}
17     position=(0.,0.);channel=1;vt=0.;done=set();bearings={};measure_moves={};checks=0;outer_entry=None
18     for line in (folder/'actions.jsonl').read_text(encoding='utf-8').splitlines():
19         row=json.loads(line)
20         if row['type']=='request':requests[row['payload']]['request_id']=row
21         elif row['type']=='response' and row['response'].get('accepted') is True:
22             rid=row['request_id']
23             if rid in done:continue
24             done.add(rid);req=requests[rid];b=req['payload'];r=row['response'];path=req['path']
25             if path in ('/measure','/clear'):

```

```

25     p=(b['position']['x'],b['position']['y']);c=b['channel'];move=math.dist(position,p)
26     if path=='/measure' and req['phase']=='search' and math.hypot(*p)>1e-7 and outer_entry is None:
27         outer_entry=(position,p)
28     vt+=move/5;position=p
29     s=sources.get(c);distance=math.inf if s is None or c in cleared else
    ↪ math.hypot(s['x']-p[0],s['y']-p[1])
30     if path=='/measure':
31         measure_moves[p,c]=move
32         vt+=5+(c!=channel);channel=c
33         expected='no_signal' if s is None or c in cleared or distance>s['radius'] else ('near' if
    ↪ distance<=5 else 'direction')
34         assert r['measure_result']==expected,(folder,expected,r)
35         if expected=='no_signal' and req['phase']=='search':negative[c].add(p)
36         if expected=='direction':
37             true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
38             envelope=1. if result['summary']['rounding']=='bounded' else 1.005
39             assert abs((r['svd_deg']-true+180)%360-180)<=envelope+1e-9
40             key=(p,c)
41             if key in bearings:assert bearings[key]==r['svd_deg']
42             bearings[key]=r['svd_deg']
43     else:
44         vt+=3
45         if distance<=20:
46             assert r['clear_result']=='success';cleared.add(c);vt+=2
47             else:assert r['clear_result']=='no_target_in_range'
48         checks+=1
49         assert abs(vt-r['virtual_time_s'])<=len(done)*1e-6+1e-4
50     strategy=result['strategy']
51     assert strategy['status']=='complete' and strategy['exit_confirmed']
52     assert set(strategy['cleared_channels'])==cleared
53     cover={tuple(s) for s in strategy['station_positions']}
54     assert len(cover)==7
55     radius=math.dist((0,0),strategy['station_positions'][1])
56     assert station_cover_bound(radius)<1000
57     phase=strategy.get('station_phase_rad',0.)
58     assert math.isfinite(phase)
59     assert np.allclose(strategy['station_positions'],stations(radius,phase),atol=1e-7,rtol=0)
60     certificate=strategy.get('stopping_certificate') or {}
61     if certificate.get('method')=='source_count_upper_bound':
62         # Count accepted SUCCESS feedback on distinct channels, never attempts or detections.
63         assert certificate['source_upper_bound']==16
64         assert len(certificate['successful_channels'])==16
65         assert len(cleared)==16 and set(certificate['successful_channels'])==cleared
66         assert set(certificate['inferred_absent_channels'])==set(range(1,21))-cleared
67         assert len(sources)<=16 # Evaluator checks the published model's applicability.
68     else:
69         for c in range(1,21):assert c in cleared or cover<=negative[c],(folder,c)
70     assert cleared==set(sources)
71     phase_events=[e for e in map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()) if
    ↪ e['type']=='ring_phase_selected']
72     assert len(phase_events)==(1 if 'station_phase_rad' in strategy else 0)
73     if phase_events:
74         e=phase_events[0];i=e['station_id'];assert 1<=i<=6 and e['visited_before']==[0]
75         assert e['phase_rad']==phase and outer_entry is not None
76         assert tuple(e['position'])==outer_entry[0] and tuple(e['new_station'])==outer_entry[1]
77         assert np.allclose(e['old_station'],stations(radius)[i],atol=1e-7,rtol=0)
78         assert tuple(strategy['station_positions'][i])==outer_entry[1]
79         shortest=abs(math.hypot(*outer_entry[0])-radius)
80         assert abs(math.dist(*outer_entry)-shortest)<1e-7
81         saved=(math.dist(outer_entry[0],e['old_station'])-math.dist(*outer_entry))/5
82         assert saved>=-1e-7 and abs(saved-e['local_movement_saved_s'])<1e-7
83     for region in strategy['regions']:
84         selection=region.get('selection',{})
85         if selection.get('mode') in ('domain_cost_proxy','domain_current'):
86             first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
87             theta=math.radians(bearings[first,c])%(2*math.pi);d=math.radians(selection['error_deg'])
88             assert selection['error_deg']>=1.005 and selection['range_prior']=='published_domain_1800'
89             assert 4*selection['error_deg']<selection['min_angle_deg']<90
90             rho=math.hypot(*first);assert rho<=1800
91             directions=[theta-d,theta+d];opposite=math.atan2(first[1],first[0])+math.pi
92             directions.extend(opposite+2*k*math.pi for k in range(-2,3) if theta-d<=opposite+2*k*math.pi<=theta+d)
93             projection=min(first[0]*math.cos(t)+first[1]*math.sin(t) for t in directions)
94             bound=min(1500.,-projection+math.sqrt(max(0.,projection**2+1800**2-rho**2)))

```

```

95     upper=selection['source_range_upper_m'];assert 5<upper<1500 and upper>=bound-1e-7
96     g=(sources[c]['x'],sources[c]['y']);assert math.dist(g,first)<=upper+1e-7
97     dx=second[0]-first[0];dy=second[1]-first[1]
98     a=dx*math.cos(theta)+dy*math.sin(theta);b=-dx*math.sin(theta)+dy*math.cos(theta)
99     assert np.allclose((a,b),selection['offset'],atol=1e-7,rtol=0) and a>=-1e-7
100    h=a*math.cos(d)-abs(b)*math.sin(d);n=a+b*b
101    radii=[5,upper]+([1000] if upper>1000 else [])
102    for radius in radii:assert n+radius**2-2*radius*h<=max(1000,radius)**2+1e-6
103    maximum=math.sqrt(max(n+25-10*h,n+upper**2-2*upper*h))
104    cross=abs(b)*math.cos(d)-a*math.sin(d)
105    assert cross>5 and cross>=maximum*math.sin(math.radians(selection['min_angle_deg']))-1e-7
106    assert (second,c) in bearings
107    if selection['mode']=='domain_current':assert measure_moves[second,c]==0
108    if selection.get('mode') in ('shared_certified','opportunistic_current'):
109        first=tuple(region['first_position']);second=tuple(region['second_position']);c=region['channel']
110        angle=math.radians(bearings[first,c]);dx=second[0]-first[0];dy=second[1]-first[1]
111        offset=(dx*math.cos(angle)+dy*math.sin(angle),-dx*math.sin(angle)+dy*math.cos(angle))
112        assert np.allclose(offset,selection['offset'],atol=1e-7,rtol=0)
113        assert candidate(offset,selection['min_angle_deg'],selection['error_deg'])
114        assert (second,c) in bearings
115        if selection['mode']=='opportunistic_current':assert measure_moves[second,c]==0
116    if 'vertices' not in region:continue
117    g=np.array([sources[region['channel']]['x'],sources[region['channel']]['y']]);v=np.array(region['vertices'])
118    edges=np.roll(v,-1,axis=0)-v;rel=g-v
119    cross=edges[:,0]*rel[:,1]-edges[:,1]*rel[:,0]
120    assert np.all(cross>=-1e-5) or np.all(cross<=1e-5)
121    assert region['covering_radius']<20
122    if 'single_circle_centre' in region:
123        centre=np.array(region['single_circle_centre'])
124        assert np.linalg.norm(v-centre,axis=1).max()<=20-1e-5+1e-9
125        assert np.linalg.norm(g-centre)<=20
126        assert region['planned_clear_calls']==1
127    # Independently recompute each omitted action's separating witness from accepted bearings.
128    from question1.solve import bearing_halfplanes
129    regions={r['channel']:r for r in strategy['regions']}
130    for line in (folder/'strategy.jsonl').read_text(encoding='utf-8').splitlines():
131        event=json.loads(line)
132        if event['type']!='certified_clear_skip':continue
133        c=event['channel'];region=regions[c];first=tuple(region['first_position']);second=tuple(region['second_positi
↵ on'])
134        if 'radius_prior' in event:
135            expected={('domain':((0.,0.),1800.),'first_received':(first,1500.),'second_received':(second,1500.))
136            centre,radius=expected[event['radius_prior']]
137            assert tuple(event['centre'])==centre and event['containing_radius_m']==radius
138            if event['radius_prior']!='domain':assert (centre,c) in bearings
139            gap=math.dist(event['point'],centre)-radius
140            assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
141            assert abs(gap-event['separation_m'])<1e-8
142            g=(sources[c]['x'],sources[c]['y'])
143            assert math.dist(g,centre)<=radius+1e-7 and math.dist(g,event['point'])>20
144            continue
145        assert event['error_deg']>=(1. if result['summary']['rounding']=='bounded' else 1.005)
146        A,b=bearing_halfplanes([dict(x=p[0],y=p[1],bearing_deg=bearings[p,c]) for p in
↵ [first,second]],event['error_deg'])
147        p=np.asarray(event['point'])
148        if 'halfplane_weights' in event:
149            weights=np.asarray(event['halfplane_weights'])
150            assert weights.shape==(len(A),) and np.isfinite(weights).all() and np.all(weights>=0)
151            assert abs(float(weights.sum())-1)<1e-12
152            normal=weights@A;bound=float(weights@b);length=float(np.linalg.norm(normal))
153            assert length>=1e-8
154            gap=float((normal@p-bound)/length)
155            assert event['slack_m']>=1e-5/length
156        else:
157            k=event['halfplane_index'];gap=float((A[k]@p-b[k])/np.linalg.norm(A[k]))
158            assert event['slack_m']>=1e-5 and gap>20+event['slack_m']
159            assert abs(gap-event['separation_m'])<1e-8
160            assert math.dist(p,(sources[c]['x'],sources[c]['y']))>20
161    return checks
162
163
164 def main():
165     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)

```

```

166 summary=json.loads((root/'summary.json').read_text(encoding='utf-8'));rows=summary['cases']
167 checks=sum(audit_folder(root/r['case_id']) for r in rows)
168 aggregate=dict(kind='SELF_POST_RUN_AUDIT',runs=len(rows),actions_checked=checks,
169               all_stopping_certificates_valid=True,all_physics_and_time_checks_passed=True,
170               mean_virtual_s=statistics.mean(r['metrics']['virtual_time_s'] for r in rows),
171               mean_case_average_per_clear_s=statistics.mean(r['average_virtual_per_cleared'] for r in rows),
172               pooled_virtual_per_clear_s=sum(r['metrics']['virtual_time_s'] for r in rows)/sum(r['cleared'] for
               ↪ r in rows),
173               total_cleared_over_runs=sum(r['cleared'] for r in rows),
174               worst_total_case=max(rows,key=lambda r:r['metrics']['virtual_time_s']),
175               worst_per_clear_case=max(rows,key=lambda r:r['average_virtual_per_cleared']),
176               most_failed_clear_case=max(rows,key=lambda r:r['metrics']['counts']['clear_failure']),
177               component_means={k:statistics.mean(r['metrics']['components'][k] for r in rows) for k in
               ↪ rows[0]['metrics']['components']},
178               min_wall_s=min(r['elapsed_wall_s'] for r in rows),max_wall_s=max(r['elapsed_wall_s'] for r in
               ↪ rows))
179 (root/'audit.json').write_text(json.dumps(aggregate,ensure_ascii=False,indent=2),encoding='utf-8')
180 print(json.dumps({k:v for k,v in aggregate.items() if not isinstance(v,dict)},ensure_ascii=False,indent=2))
181
182 if __name__=='__main__':main()

```

程序 139

对应文件: question3/results/round18_adverse_main/code_snapshot/question3/strategy.py; question3/results/round18_adverse_rounding/code_snapshot/question3/strategy.py; question3/results/round18_dev_main/code_snapshot/question3/strategy.py; question3/results/round18_dev_rounding/code_snapshot/question3/strategy.py; question3/results/round18_holdout_main/code_snapshot/question3/strategy.py; question3/results/round18_holdout_rounding/code_snapshot/question3/strategy.py; question3/results/round18_outer_main/code_snapshot/question3/strategy.py; question3/results/round18_outer_rounding/code_snapshot/question3/strategy.py

```

1  """Common complete workflow with optional modules A/B/C. Defaults preserve baseline_v1.
2  No imports of case generation/simulator; all decisions use public observations only.
3  """
4  import math
5  from .geometry import stations, station_cover_bound, optical_plan
6  from .modules import choose_second,EvidenceCover
7  from .lookahead import decide
8  from .optical_shortcut import single_circle
9  from .shared_measurement import choose_shared
10 from .order_routing import decide_order
11 from .cached_routing import reorder_cached
12 from .optical_order import nearer_end
13 from .clear_skip import constraints,failure_certificate,radius_failure_certificate
14 from .ring_phase import align_first_entry
15 from .batch_scheduling import gate as batch_gate
16
17 class Incomplete(RuntimeError):pass
18
19
20 def run(client, config, event=lambda data:None):
21     opportunity=config.get('second_point_opportunity','off')
22     if opportunity not in ('off','current_if_cheaper','current_if_certified'):
23         raise ValueError('Unknown second_point_opportunity')
24     if opportunity!='off' and not config.get('modules',{}).get('A',False):
25         raise ValueError('Current-point comparison requires A')
26     opportunity_timing=config.get('opportunity_timing','before_schedule')
27     if opportunity_timing not in ('before_schedule','after_schedule'):
28         raise ValueError('Unknown opportunity_timing')
29     planning_config={**config,'second_point_opportunity':'off'} if opportunity_timing=='after_schedule' else config
30     range_prior=config.get('second_range_prior','off')
31     if range_prior not in ('off','domain'):raise ValueError('Unknown second_range_prior')
32     if range_prior!='off':
33         if opportunity_timing!='after_schedule' or not config.get('modules',{}).get('A',False):

```

```

34         raise ValueError('Domain second-range prior requires A and after-schedule timing')
35         planning_config=**planning_config,'second_range_prior':'off'}
36     count_stop=config.get('count_stop','off')
37     if count_stop not in ('off','cleared_16','detected_16'):
38         raise ValueError('Unknown count_stop; Q3 source upper bound is fixed at 16')
39     search_radius=config.get('search_radius_m',1500.)
40     if station_cover_bound(search_radius)>=1000:raise ValueError('Invalid search coverage')
41     empty_radius=config.get('empty_origin_radius_m')
42     if empty_radius is not None and station_cover_bound(empty_radius)>=1000:
43         raise ValueError('Invalid empty-origin search coverage')
44     cover=stations(search_radius)
45     negative={c:set() for c in range(1,21)};first={};cleared=set();regions=[];visited=set()
46     modules=config.get('modules',{});joint=modules.get('B',False);evidence=modules.get('C',False)
47     shared=modules.get('S',False);order=modules.get('O',False);cached={}
48     phase_mode=config.get('ring_phase','fixed');phase_decided=False;station_phase=0.
49     if phase_mode not in ('fixed','first_entry_radial'):raise ValueError('Unknown ring_phase')
50     if phase_mode!='fixed' and not joint:raise ValueError('Adaptive ring phase requires B task selection')
51     batch_mode=config.get('search_interleaving','current')
52     if batch_mode not in ('current','cached_batch','known_batch','nearest_finite'):raise ValueError('Unknown
↪ search_interleaving')
53     clear_skip=config.get('clear_skip','off');skip_planes={}
54     if clear_skip not in ('off','separating_halfplane','separating_pair'):raise ValueError('Unknown clear_skip')
55     if clear_skip!='off' and evidence:raise ValueError('Clear skipping requires the fixed optical path; C is
↪ incompatible')
56     radius_priors=config.get('clear_radius_priors','off')
57     if radius_priors not in ('off','domain','received','both'):raise ValueError('Unknown clear_radius_priors')
58     if radius_priors!='off' and clear_skip=='off':raise ValueError('Radius priors require clear_skip')
59     optical_order=config.get('optical_order','original')
60     if optical_order not in ('original','nearer_end'):
61         raise ValueError('Unknown optical_order')
62     if optical_order!='original' and evidence:
63         raise ValueError('Optical reversal requires a fixed optical path; C is incompatible')
64     cache_route=config.get('cache_routing','nearest')
65     if batch_mode!='current' and (not joint or order or cache_route!='nearest' or
↪ config.get('routing',{}).get('mode','nearest')!='nearest'):
66         raise ValueError('Batch scheduling requires B nearest routing without O/cache insertion')
67     if batch_mode=='cached_batch' and not shared:raise ValueError('Cached batch requires S')
68     scan_order=config.get('scan_order','nearest')
69     if scan_order not in ('nearest','remaining_open_path'):raise ValueError('Unknown scan_order')
70     if scan_order!='nearest' and (not joint or order or cache_route!='nearest' or
↪ config.get('routing',{}).get('mode','nearest')!='nearest' or phase_mode!='fixed'):
71         raise ValueError('Scan path ordering requires B nearest routing, fixed stations, no O/cache insertion')
72     if cache_route not in ('nearest','station_insertion'):
73         raise ValueError('Unknown cache_routing')
74     if cache_route!='nearest' and (not shared or not joint or order or modules.get('C',False)):
75         raise ValueError('Cache insertion requires B/S with fixed optical paths and O disabled')
76     shared_limit=config.get('shared_batch_limit',1)
77     if type(shared_limit) is not int or not 1<=shared_limit<=19:
78         raise ValueError('shared_batch_limit must be an integer in [1,19]')
79     current_shared_limit=config.get('current_point_shared_limit',shared_limit)
80     if type(current_shared_limit) is not int or not 0<=current_shared_limit<=19:
81         raise ValueError('current_point_shared_limit must be an integer in [0,19]')
82     if (shared or order) and (not joint or config.get('routing',{}).get('mode','nearest')!='nearest'):
83         raise ValueError('S/O require joint B with nearest routing; legacy H scoring is incompatible')
84     if evidence and config['optical_cell_m']!=28:raise ValueError('Evidence module currently certifies 28m coarse
↪ cells only')
85     client.enter();failure=None;exit_confirmed=False;coverage_emitted=False;consecutive_localizations=0
86     stopping_certificate=None;count_discovery_emitted=False
87     def scan(station_id):
88         nonlocal phase_decided,station_phase
89         if phase_mode!='fixed' and station_id!=0 and not phase_decided:
90             # Only origin evidence exists. No already-observed outer point may move.
91             if visited!={} or any(ids-{} for ids in negative.values()):
92                 raise Incomplete('Cannot rotate a ring after outer station evidence')
93             old=cover[station_id];replacement,station_phase=align_first_entry(cover,station_id,client.position)
94             cover[:]=replacement;phase_decided=True
95             event(dict(type='ring_phase_selected',phase_rad=station_phase,station_id=station_id,
96                 position=client.position,old_station=old,new_station=cover[station_id],
97                 visited_before=sorted(visited),
98                 local_movement_saved_s=(math.dist(client.position,old)-math.dist(client.position,cover[station
↪ _id]))/5))
99             s=cover[station_id]
100         for channel in range(1,21):

```

```

101     # Q3 and Appendix 2 explicitly bound sources by 16 distinct channels.
102     # Returning early does NOT mark this partially scanned station complete.
103     if count_stop!='off' and len(cleared)==16: return
104     if count_stop=='detected_16' and len(set(first)|cleared)==16: return
105     if channel in first or channel in cleared: continue
106     response=client.measure(s,channel,phase='search'); kind=response['measure_result']
107     if kind=='no_signal': negative[channel].add(station_id)
108     elif kind=='near':
109         if client.clear(s,channel,phase='near')['clear_result']!='success': raise Incomplete('near followed by
            ↪ failed clear')
110         cleared.add(channel)
111     elif kind=='direction': first[channel]=(s,response['svd_deg'])
112     else: raise Incomplete('Unknown measure result')
113 visited.add(station_id)
114 if station_id==0 and empty_radius is not None and not first and not cleared:
115     # Origin remains unchanged: its negatives stay valid for either certified ring.
116     cover[:]=stations(empty_radius)
117     event(dict(type='search_ring_switch', reason='all_twenty_origin_channels_no_signal',
118             configured_radius_m=search_radius, selected_radius_m=empty_radius))
119 def prepare(channel,q,r,selection):
120     s,theta=first[channel]
121     planner=None
122     if r['measure_result']=='near': points=[q]; certificate=dict(planned_clear_calls=1,covering_radius=5)
123     elif r['measure_result']=='direction':
124         points,certificate=optical_plan(s,theta,q,r['svd_deg'],config['error_deg'],config['optical_cell_m'],confi
            ↪ g['max_optical_points'])
125         if clear_skip!='off': skip_planes[channel]=constraints(s,theta,q,r['svd_deg'],config['error_deg'])
126         shortcut=single_circle(certificate['vertices']) if modules.get('D',False) and len(points)>1 else None
127         if shortcut:
128             centre,radius=shortcut
129             certificate.update(single_circle_centre=centre,original_grid_calls=len(points),
130                     planned_clear_calls=1,covering_radius=radius)
131             points=[centre]
132         elif evidence and len(points)<=128:
133             planner=EvidenceCover(points,certificate,s,theta,q,r['svd_deg'],[cover[k] for k in
            ↪ sorted(negative[channel])],config['error_deg'])
134         elif evidence:
135             certificate['evidence_fallback']='more_than_128_coarse_cells_use_original_cover'
136         else: raise Incomplete('Guaranteed-reception second point returned no_signal')
137         certificate.update(channel=channel,first_position=s,second_position=q,selection=selection)
138         regions.append(certificate); event(dict(type='localization',**certificate))
139         return points,certificate,planner
140 def orient(channel,points):
141     if optical_order=='original': return points
142     chosen=nearer_end(points,client.position)
143     event(dict(type='optical_path_order',channel=channel,reversed=chosen is not points,
144             original_entry=points[0],selected_entry=chosen[0],
145             entry_distance_saved_m=math.dist(client.position,points[0])-math.dist(client.position,chosen[0])))
146     return chosen
147 def localize(channel,q,selection):
148     if channel in cached:
149         points,certificate,planner=cached.pop(channel)
150         points=orient(channel,points)
151         event(dict(type='shared_cache_use',channel=channel,measurement_point=certificate['second_position'],
152                 optical_entry=points[0]))
153     else:
154         if opportunity_timing=='after_schedule':
155             planned=q
156             q,selection=choose_second(*first[channel],client.position,config)
157             event(dict(type='second_point_after_schedule',channel=channel,scheduled_point=planned,
158                     actual_point=q,selection_mode=selection['mode'],point_changed=q!=planned))
159             r=client.measure(q,channel,phase='localization')
160             points,certificate,planner=prepare(channel,q,r,selection)
161             points=orient(channel,points)
162             batch_limit=current_shared_limit if selection['mode'] in ('opportunistic_current','domain_current') else
            ↪ shared_limit
163             for _ in range(batch_limit if shared else 0):
164                 opportunity=choose_shared(channel,q,first,cleared,cached,config,primary_points=points,decision_log=ev
            ↪ ent)
165                 if opportunity is None: break
166                 other,sel=opportunity
167                 response=client.measure(q,other,phase='localization_shared')
168                 cached[other]=prepare(other,q,response,sel)

```

```

169         event(dict(type='shared_cache_store', channel=other, primary_channel=channel, point=q,
170                    cached_channels=sorted(cached)))
171     for i in range(len(points)):
172         if planner:
173             k,p,info=planner.next_point(client.position);event(dict(type='evidence_choice', channel=channel, point=
↪ p,**info))
174         else:p=points[i]
175         if clear_skip!='off':
176             proof=failure_certificate(p, skip_planes.get(channel), combine=clear_skip=='separating_pair')
177             if proof is None and radius_priors!='off':
178                 proof=radius_failure_certificate(p, first[channel][0], certificate['second_position'], radius_priors)
179             if proof:
180                 event(dict(type='certified_clear_skip', channel=channel, point=p, original_index=i, error_deg=config[
↪ 'error_deg'],**proof))
181                 continue
182         if client.clear(p, channel, phase='optical')['clear_result']=='success':
183             cleared.add(channel)
184             if planner:
185                 certificate['evidence']=planner.metrics();event(dict(type='evidence_result', channel=channel,**pla
↪ nner.metrics()))
186         break
187         if planner:planner.failed(k)
188     else:raise Incomplete('All optical covering points failed: model/protocol inconsistency')
189 try:
190     while True:
191         known=set(first)|cleared
192         if count_stop!='off' and len(known)>16:
193             raise Incomplete('Observed source channels contradict Q3 upper bound 16')
194         if count_stop!='off' and len(cleared)==16:break
195         pending=sorted(set(first)-cleared);unvisited=sorted(set(range(len(cover)))-visited)
196         count_saturated=count_stop=='detected_16' and len(known)==16
197         remaining=[] if count_saturated else unvisited
198         if count_saturated and not count_discovery_emitted:
199             event(dict(type='source_upper_bound_saturated', source_upper_bound=16,
200                      observed_channels=sorted(known), still_to_clear=pending,
201                      unvisited_station_ids=unvisited))
202             count_discovery_emitted=True
203         if not unvisited and not coverage_emitted:
204             event(dict(type='coverage_complete', detected=sorted(first), cleared=sorted(cleared),
205                      absent=[c for c in negative if len(negative[c])==len(cover)]));coverage_emitted=True
206         if not pending and not remaining:break
207         if not joint:
208             if remaining:scan(remaining[0]);continue
209             channel=pending[0];s,theta=first[channel];q,selection=choose_second(s,theta,client.position,planning_
↪ config)
210             localize(channel,q,selection);continue
211         # B: compare task starts; optional gates govern scan interleaving.
212         jobs=[]
213         ready_cache={c:(nearer_end(p,client.position),cert,planner) for c,(p,cert,planner) in cached.items()} if
↪ optical_order=='nearer_end' else cached
214         for c in pending:
215             if c in cached:
216                 q=ready_cache[c][0][0];sel=dict(mode='shared_ready', ready=True)
217                 else:q,sel=choose_second(*first[c],client.position,planning_config)
218                 jobs.append((math.dist(client.position,q),c,q,sel))
219             scheduled_jobs,scheduled_remaining,forced,batch_info=batch_gate(batch_mode,pending,cached,jobs,remaining,
↪ consecutive_localizations)
220             if batch_info:event(dict(type='localization_batch',**batch_info))
221             args=(client.position,client.channel,cover,scheduled_remaining,scheduled_jobs,first,negative,
222                   [c for c in range(1,21) if c not in first and c not in cleared],config)
223             decision=decide_order(*args,cached=ready_cache,forced=forced) if order else decide(*args,forced=forced)
224             if cache_route=='station_insertion':
225                 decision=reorder_cached(decision,client.position,cover,remaining,jobs,ready_cache)
226             task,target,info=decision
227             if task=='scan':
228                 event(dict(type='schedule', task='scan', station_id=target,**info))
229                 scan(target);consecutive_localizations+=1
230             else:
231                 _,c,q,sel=target;event(dict(type='schedule', task='localize', channel=c,**info))
232                 localize(c,q,sel);consecutive_localizations+=1
233         if count_stop!='off' and len(cleared)==16:
234             stopping_certificate=dict(method='source_count_upper_bound', source_upper_bound=16,
235                                     successful_channels=sorted(cleared),

```



```

236         inferred_absent_channels=sorted(set(range(1,21))-cleared))
237     elif all(c in cleared or len(negative[c])==len(cover) for c in range(1,21)):
238         stopping_certificate=dict(method='seven_station_channel_cover')
239     else:
240         raise Incomplete('Stopping certificate is incomplete')
241     if not 10<=len(cleared)<=16:raise Incomplete('Cleared count contradicts Q3 count bounds')
242     exit_confirmed=client.exit()['exit_reason']=='user_exit'
243 except (RuntimeError, ValueError, KeyError) as exc:
244     failure=f'{type(exc).__name__}: {exc}'
245 result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
246            exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
247            absent_channels=sorted(set(c for c in negative if len(negative[c])==len(cover)))|
248            set((stopping_certificate or {}).get('inferred_absent_channels',[]))),
249            negative_station_ids={str(c):sorted(v) for c,v in negative.items()},station_positions=cover,
250            stopping_certificate=stopping_certificate,
251            regions=regions,metrics=client.metrics(),modules=modules)
252 if phase_decided:result['station_phase_rad']=station_phase
253 event(dict(type='strategy_result',**result));return result

```

程序 140

对应文件: question3/results/round1_main/code_snapshot/question3/analyze_matrix.py; question3/results/round1_rounding/code_snapshot/question3/analyze_matrix.py

```

1 """Report every case, failure penalty, tails and factorial conditional effects."""
2 import argparse
3 import csv
4 import json
5 from pathlib import Path
6 import numpy as np
7 from .run_matrix import VARIANTS
8
9
10 def good(row):return row['status']=='complete' and row['clearance_ratio']==1 and row.get('audit')=='pass' and not
    ↳ row.get('false_complete')
11 def objective(row):return row['metrics']['virtual_time_s'] if good(row) else
    ↳ max(360000,row['metrics']['virtual_time_s'])
12
13 def main():
14     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)
15     data=json.loads((root/'summary.json').read_text());lookup={r['variant'],r['case_id']}:r for r in data['cases']}
16     ids=sorted({r['case_id'] for r in data['cases']});stats=[]
17     for name in VARIANTS:
18         rows=[lookup[name,i] for i in ids];base=[lookup['baseline',i] for i in ids]
19         values=np.array([objective(r) for r in rows]);raw=np.array([r['metrics']['virtual_time_s'] for r in rows])
20         improvement=1-values/np.array([objective(r) for r in base]);wall=[r['elapsed_wall_s'] for r in rows]
21         worst=int(np.argmax(values));regress=int(np.argmin(improvement));per_clear=[r['average_virtual_per_cleared']
    ↳ for r in rows]
22         stats.append(dict(variant=name,runs=len(rows),complete=sum(good(r) for r in rows),
23                          failed=sum(not good(r) for r in rows),timeouts=sum(r['timeout'] for r in rows),
24                          false_complete=sum(r['false_complete'] for r in rows),
25                          mean_clearance_ratio=float(np.mean([r['clearance_ratio'] for r in rows])),
26                          mean_virtual_s=float(values.mean()),raw_all_case_mean_virtual_s=float(raw.mean()),
27                          p95_virtual_s=float(np.percentile(values,95)),cvar_worst10_s=float(np.sort(values)[-max(1,i)
    ↳ nt(np.ceil(.1*len(rows))))].mean()),
28                          max_virtual_s=float(values.max()),max_virtual_case=ids[worst],
29                          mean_case_average_per_clear_s=None if any(x is None for x in per_clear) else
    ↳ float(np.mean(per_clear)),
30                          pooled_virtual_per_clear_s=float(raw.sum()/sum(r['cleared'] for r in rows)) if
    ↳ sum(r['cleared'] for r in rows) else None,
31                          median_paired_improvement=float(np.median(improvement)),
32                          worst_case_improvement=float(improvement.min()),worst_regression_case=ids[regress],
33                          worse_than_baseline=int(np.sum(improvement < -1e-9)),wins=int(np.sum(improvement>1e-9)),
34                          wall_p95_s=float(np.percentile(wall,95)),wall_max_s=max(wall),
35                          mean_failed_clears=float(np.mean([r['metrics']['counts']['clear_failure'] for r in rows])),
36                          max_failed_clears=max(r['metrics']['counts']['clear_failure'] for r in rows),
37                          mean_components={k:float(np.mean([r['metrics']['components'][k] for r in rows])) for k in
    ↳ rows[0]['metrics']['components']})
38     effects=[]

```

```

39     for module in 'ABC':
40         for without in VARIANTS:
41             if module in without or without=='baseline' and False:continue
42             letters='' if without=='baseline' else without
43             with_name=''.join(sorted(letters+module))
44             if with_name not in VARIANTS:continue
45             before=np.array([objective(lookup[without,i]) for i in
↪ ids]);after=np.array([objective(lookup[with_name,i]) for i in ids])
46             change=1-after/before;k=int(np.argmin(change))
47             effects.append(dict(module=module,without=without,with_variant=with_name,mean_saved_s=float(np.mean(befor
↪ e-after))),
48                             median_paired_improvement=float(np.median(change)),wins=int(np.sum(change>1e-9)),
49                             losses=int(np.sum(change< -1e-9)),worst_change=float(change[k]),worst_case=ids[k]))
50     interactions=[]
51     mean={s['variant']:s['mean_virtual_s'] for s in stats}
52     for a,b in (('A','B'),('A','C'),('B','C')):
53         interactions.append(dict(pair=a+b,additive_interaction_s=mean[a+b]-mean[a]-mean[b]+mean['baseline'],
54                                 definition='T_AB - T_A - T_B + T_baseline; negative = savings exceed additive
↪ single-module savings'))
55     failures=[r for r in data['cases'] if not good(r)]
56     report=dict(kind=data['kind'],rounding=data['rounding'],stats=stats,effects=effects,interactions=interactions,fai
↪ lures=failures,
57                 notes=[f'All runs retained. Failure/timeout/noncertified runs ranked with >=360000s penalty.',
58                         f'Percentiles include all cases. Fixtures are development cases, NOT held-out validation.'])
59     (root/'comparison.json').write_text(json.dumps(report,indent=2),encoding='utf-8')
60     with (root/'paired_cases.csv').open('w',newline='',encoding='utf-8') as f:
61         writer=csv.writer(f);writer.writerow(['case_id']+VARIANTS)
62         for i in ids:writer.writerow([i]+[objective(lookup[n,i]) for n in VARIANTS])
63     for s in stats:print(s['variant'],f'{s["complete"]}/{s["runs"]}',f'mean={s["mean_virtual_s"]:.2f}',f'p95={s["p95_
↪ virtual_s"]:.2f}',
64                             f'max={s["max_virtual_s"]:.2f}',f'median_gain={s["median_paired_improvement"]:.2%}',
65                             f'worst_gain={s["worst_case_improvement"]:.2%}',s['worst_regression_case'])
66     print('EFFECTS',json.dumps(effects));print('INTERACTIONS',json.dumps(interactions))
67
68 if __name__=='__main__':main()

```

程序 141

对应文件: question3/results/round1_main/code_snapshot/question3/audit.py;
question3/results/round1_rounding/code_snapshot/question3/audit.py

```

1  """Post-run oracle audit: evaluator only, NEVER called by the strategy."""
2  import argparse
3  import json
4  import math
5  import statistics
6  from pathlib import Path
7  import numpy as np
8
9
10 def audit_folder(folder):
11     case=json.loads((folder/'evaluator_hidden_case.json').read_text(encoding='utf-8'))
12     result=json.loads((folder/'result.json').read_text(encoding='utf-8'))
13     sources={s['channel']:s for s in case['sources']};requests={};cleared=set();negative={c:set() for c in
↪ range(1,21)}
14     position=(0.,0.);channel=1;vt=0.;done=set();bearings={};checks=0
15     for line in (folder/'actions.jsonl').read_text(encoding='utf-8').splitlines():
16         row=json.loads(line)
17         if row['type']=='request':requests[row['payload']]['request_id']=row
18         elif row['type']=='response' and row['response'].get('accepted') is True:
19             rid=row['request_id']
20             if rid in done:continue
21             done.add(rid);req=requests[rid];b=req['payload'];r=row['response'];path=req['path']
22             if path in ('/measure','/clear'):
23                 p=(b['position']['x'],b['position']['y']);c=b['channel'];vt+=math.dist(position,p)/5;position=p
24                 s=sources.get(c);distance=math.inf if s is None or c in cleared else
↪ math.hypot(s['x']-p[0],s['y']-p[1])
25                 if path=='/measure':
26                     vt+=5+(c!=channel);channel=c
27                     expected='no_signal' if s is None or c in cleared or distance>s['radius'] else ('near' if
↪ distance<=5 else 'direction')

```

```

28         assert r['measure_result']==expected,(folder,expected,r)
29         if expected=='no_signal' and req['phase']=='search':negative[c].add(p)
30         if expected=='direction':
31             true=math.degrees(math.atan2(s['y']-p[1],s['x']-p[0]))
32             envelope=1. if result['summary']['rounding']=='bounded' else 1.005
33             assert abs((r['svd_deg']-true+180)%360-180)<=envelope+1e-9
34             key=(p,c)
35             if key in bearings:assert bearings[key]==r['svd_deg']
36             bearings[key]=r['svd_deg']
37         else:
38             vt+=3
39             if distance<=20:
40                 assert r['clear_result']=='success';cleared.add(c);vt+=2
41             else:assert r['clear_result']=='no_target_in_range'
42             checks+=1
43             assert abs(vt-r['virtual_time_s'])<=len(done)*1e-6+1e-4
44             strategy=result['strategy']
45             assert strategy['status']=='complete' and strategy['exit_confirmed']
46             cover={tuple(s) for s in strategy['station_positions']}
47             assert len(cover)==7
48             for c in range(1,21):assert c in cleared or cover<=negative[c],(folder,c)
49             assert cleared==set(sources)
50             for region in strategy['regions']:
51                 if 'vertices' not in region:continue
52                 g=np.array([sources[region['channel']]['x'],sources[region['channel']]['y']]);v=np.array(region['vertices'])
53                 edges=np.roll(v,-1,axis=0)-v;rel=g-v
54                 cross=edges[:,0]*rel[:,1]-edges[:,1]*rel[:,0]
55                 assert np.all(cross>=-1e-5) or np.all(cross<=1e-5)
56                 assert region['covering_radius']<20
57             return checks
58
59
60 def main():
61     p=argparse.ArgumentParser();p.add_argument('folder');args=p.parse_args();root=Path(args.folder)
62     summary=json.loads((root/'summary.json').read_text(encoding='utf-8'));rows=summary['cases']
63     checks=sum(audit_folder(root/r['case_id']) for r in rows)
64     aggregate=dict(kind='SELF_POST_RUN_AUDIT',runs=len(rows),actions_checked=checks,
65                   all_stopping_certificates_valid=True,all_physics_and_time_checks_passed=True,
66                   mean_virtual_s=statistics.mean(r['metrics']['virtual_time_s'] for r in rows),
67                   mean_case_average_per_clear_s=statistics.mean(r['average_virtual_per_cleared'] for r in rows),
68                   pooled_virtual_per_clear_s=sum(r['metrics']['virtual_time_s'] for r in rows)/sum(r['cleared'] for
69                   ↪ r in rows),
70                   total_cleared_over_runs=sum(r['cleared'] for r in rows),
71                   worst_total_case=max(rows,key=lambda r:r['metrics']['virtual_time_s']),
72                   worst_per_clear_case=max(rows,key=lambda r:r['average_virtual_per_cleared']),
73                   most_failed_clear_case=max(rows,key=lambda r:r['metrics']['counts']['clear_failure']),
74                   component_means={k:statistics.mean(r['metrics']['components'][k] for r in rows) for k in
75                   ↪ rows[0]['metrics']['components']},
76                   min_wall_s=min(r['elapsed_wall_s'] for r in rows),max_wall_s=max(r['elapsed_wall_s'] for r in
77                   ↪ rows))
78     (root/'audit.json').write_text(json.dumps(aggregate,ensure_ascii=False,indent=2),encoding='utf-8')
79     print(json.dumps({k:v for k,v in aggregate.items() if not isinstance(v,dict)},ensure_ascii=False,indent=2))
80
81 if __name__=='__main__':main()

```

程序 142

对应文件: question3/results/round1_main/code_snapshot/question3/geometry.py; question3/results/round1_rounding/code_snapshot/question3/geometry.py

```

1 """Public geometry only; reuse Q1 and Q2, no simulator imports."""
2 import math
3 import numpy as np
4 from question1.solve import solve
5 from question2.strategy import candidate, to_world
6
7
8 def stations():
9     return [(0.,0.)]+[(1500*math.cos(k*math.pi/3),1500*math.sin(k*math.pi/3)) for k in range(6)]
10
11

```

```

12 def station_cover_bound():
13     # Centre covers radius <=900. Remaining annulus: nearest ring angle <=30deg.
14     return max(900.,*(math.sqrt(r*r+1500**2-2*r*1500*math.cos(math.pi/6)) for r in [900,1800]))
15
16
17 def second_point(first, bearing, offset, error_deg, min_angle):
18     if not candidate(offset,min_angle,error_deg):
19         raise ValueError('Second offset lacks analytic reception/angle certificate')
20     if min_angle <= 4*error_deg:
21         raise ValueError('Angle margin insufficient for bounded two-wedge intersection')
22     return tuple(to_world(offset,first,bearing))
23
24
25 def optical_plan(first, bearing, second, reading, error_deg, cell_m, max_points):
26     """Cover an inflated rotated bounding box by <=28m cells. No hidden G.
27
28     Cells have diagonal/2 <20m. Testing every centre guarantees a hit if G is in P.
29     """
30     if not 0 < cell_m < 20*math.sqrt(2):
31         raise ValueError('Optical cells must fit strictly inside a radius-20 circle')
32     result=solve([dict(x=first[0],y=first[1],bearing_deg=bearing),
33                  dict(x=second[0],y=second[1],bearing_deg=reading)],error_deg)
34     if result['status']!='bounded':
35         raise RuntimeError('Invalid localization region: '+result['status'])
36     vertices=np.array(result['vertices'])
37     a=math.radians(bearing)
38     rot=np.array([[math.cos(a),-math.sin(a)],[math.sin(a),math.cos(a)]])
39     local=(vertices-np.array(first))@rot
40     lo=local.min(axis=0)-1e-5;hi=local.max(axis=0)+1e-5
41     sizes=np.maximum(1,np.ceil((hi-lo)/cell_m).astype(int))
42     n=int(np.prod(sizes))
43     if n>max_points:
44         raise RuntimeError(f'Optical plan exceeds configured guard: {n}')
45     points=[]
46     for j in range(sizes[1]):
47         cols=range(sizes[0]) if j%2==0 else reversed(range(sizes[0]))
48         for i in cols:
49             p=lo+(np.array([i+.5,j+.5])/sizes)*(hi-lo)
50             points.append(tuple(np.array(first)+rot@p))
51     certificate=dict(diameter=result['diameter'],vertices=result['vertices'],
52                     bbox_widths=(hi-lo).tolist(),planned_clear_calls=n,
53                     covering_radius=float(np.linalg.norm((hi-lo)/sizes)/2))
54     return points,certificate

```

程序 143

对应文件: question3/results/round1_main/code_snapshot/question3/modules.py;
question3/results/round1_rounding/code_snapshot/question3/modules.py

```

1 """Optional public-information modules A (local cost), B (route via strategy), C (evidence).
2 No simulator/case imports. All design constants fixed before round-one comparison.
3 """
4 import math
5 from functools import lru_cache
6 import numpy as np
7 from question2.strategy import candidate, diameters, to_world
8 from question1.solve import bearing_halfplanes
9 from .geometry import second_point
10
11 OFFSETS=((400,500),(500,400),(500,500),(600,400),(700,450),(750,400),(800,605),(850,450))
12
13 @lru_cache(maxsize=8)
14 def design_catalog(error_deg=1.005, min_angle=20.):
15     delta=math.radians(error_deg)
16     r,alpha,error=np.meshgrid(np.linspace(5.001,1500,31),np.linspace(-delta,delta,3),
17                               np.linspace(-delta,delta,3),indexing='ij')
18     g=np.column_stack([(r*np.cos(alpha)).ravel(),(r*np.sin(alpha)).ravel()]);error=error.ravel()
19     catalog=[]
20     for a,b in OFFSETS:
21         if not candidate([a,b],min_angle,error_deg):continue
22         d=float(diameters([a,b],g,error,error_deg).max())

```

```

23     h=a*math.cos(delta)-b*math.sin(delta)
24     max_range=math.sqrt(max(a*a+b*b+25-10*h,a*a+b*b+1500**2-3000*h))
25     # Surrogate only: diagonal-based box count can overcount or undercount.
26     # Never used as a coverage/feasibility certificate or a probabilistic expectation.
27     cells=math.ceil(d/28)*math.ceil(2*1500*math.sin(delta)/28)
28     continuation=max_range/5+(3+28/5)*cells
29     for sign in [1,-1]:
30         catalog.append(dict(offset=(a,sign*b),sampled_worst_diameter=d,
31                             max_source_distance=max_range,proxy_cells=cells,continuation_proxy_s=continuation))
32     return tuple(catalog)
33
34
35 def choose_second(first,bearing,current,config):
36     if not config.get('modules',{}).get('A',False):
37         q=second_point(first,bearing,config['second_offset'],config['error_deg'],config['min_angle_deg'])
38         return q,dict(mode='fixed',offset=config['second_offset'])
39     best=None
40     for row in design_catalog(config['error_deg'],config['min_angle_deg']):
41         q=tuple(to_world(row['offset'],first,bearing))
42         score=math.dist(current,q)/5+5+row['continuation_proxy_s']
43         key=(score,row['offset'])
44         if best is None or key<best[0]:best=(key,q,row)
45     if best is None:raise RuntimeError('No certified second-point candidates')
46     return best[1],dict(mode='cost_proxy',score_s=best[0][0],**best[2])
47
48
49 class EvidenceCover:
50     """Finite covering debt. Fine cells are excluded only by whole-cell proofs.
51
52     Every fine cell has its original coarse-grid centre within 20m of all corners.
53     Failures remove cells fully inside failed-clear disks; they never infer absence
54     from a bare centre test. Point selection uses covered geometric area per cost,
55     not a probability of source presence. No fine-grid point estimate of truth.
56     """
57     def __init__(self,points,certificate,first,bearing,second,reading,negative_positions,error_deg,subdiv=4):
58         v=np.asarray(certificate['vertices']);origin=np.asarray(first)
59         angle=math.radians(bearing);rot=np.array([[math.cos(angle),-math.sin(angle)],[math.sin(angle),math.cos(angle)]]
60         ↪ ])
61         local=(v-origin)@rot;lo=local.min(axis=0)-1e-5;hi=local.max(axis=0)+1e-5
62         sizes=np.maximum(1,np.ceil((hi-lo)/28).astype(int));fine=sizes*subdiv
63         corners=[]
64         for j in range(fine[1]):
65             for i in range(fine[0]):
66                 cell=lo+(np.array([[i,j],[i+1,j],[i+1,j+1],[i,j+1]])/fine)*(hi-lo)
67                 corners.append(origin+cell@rot.T)
68         self.corners=np.array(corners);self.points=np.asarray(points);self.alive=np.ones(len(corners),bool)
69         self.untried=np.ones(len(points),bool);self.initial_count=len(corners);self.prior_dropped=0;self.failed_dropp
70         ↪ ed=0
71         A,b=bearing_halfplanes([dict(x=first[0],y=first[1],bearing_deg=bearing),
72                                dict(x=second[0],y=second[1],bearing_deg=reading)],error_deg)
73         projections=np.einsum('kd,nvd->nkv',A,self.corners)
74         self.alive &= ~np.any(projections.min(axis=2)>b[None,:]+1e-7,axis=1)
75         centres=self.corners.mean(axis=1);radii=np.linalg.norm(self.corners-centres[:,None,:],axis=2).max(axis=1)
76         for centre,radius in [(0,0),1800),(first,1500),(second,1500)]:
77             self.alive &= np.linalg.norm(centres-centre,axis=1)-radii <= radius+1e-7
78         for p in negative_positions:
79             self.alive &= np.linalg.norm(self.corners-np.asarray(p),axis=2).max(axis=1)>1000-1e-7
80         self.prior_dropped=int(self.initial_count-self.alive.sum())
81         self.covers=np.linalg.norm(self.points[:,None,None,:]-self.corners[None,:, :, :],axis=3).max(axis=2)<=20-1e-7
82         if not np.all(self.covers.any(axis=0)):raise RuntimeError('Fine cells lack original optical coverage')
83         if not self.alive.any():raise RuntimeError('All source cells excluded by prior evidence')
84     def next_point(self,current):
85         if not self.alive.any():raise RuntimeError('All feasible cells removed without success')
86         gain=self.covers[:,self.alive].sum(axis=1)
87         cost=3*np.linalg.norm(self.points-np.asarray(current),axis=1)/5
88         scores=np.where(self.untried & (gain>0),gain/cost,-1)
89         k=int(np.argmax(scores))
90         if scores[k]<0:raise RuntimeError('Optical covering debt cannot be satisfied')
91         self.untried[k]=False
92         return k,tuple(self.points[k]),dict(remaining_cells=int(self.alive.sum()),fully_covered_cells=int(gain[k]))
93     def failed(self,k):
94         remove=self.alive & self.covers[k];self.failed_dropped+=int(remove.sum());self.alive[remove]=False
95     def metrics(self):return dict(fine_cells=self.initial_count,prior_excluded_cells=self.prior_dropped,

```

```

94                                     failed_clear_excluded_cells=self.failed_dropped,remaining_cells=int(self.alive.sum(
↪      )))

```

程序 144

对应文件: question3/results/round1_main/code_snapshot/question3/run_http.py; question3/results/round1_rounding/code_snapshot/question3/run_http.py

```

1  """Run strategy against self HTTP or an already-open official Q3 REHEARSAL.
2  No formal-test mode; no simulator UI automation; no exe launch.
3  """
4  import argparse
5  import json
6  import time
7  from pathlib import Path
8  from .interface import Client,HttpTransport,JsonLog
9  from .strategy import run
10
11
12 def main():
13     p=argparse.ArgumentParser()
14     p.add_argument('--mode',choices=['self-http','official-rehearsal'],default='self-http')
15     p.add_argument('--url');p.add_argument('--robot-id',default='SELF')
16     p.add_argument('--confirm-rehearsal-ui',action='store_true',help='Human confirms Q3 rehearsal, NOT formal UI')
17     p.add_argument('--output',required=True)
18     args=p.parse_args()
19     if args.mode=='official-rehearsal' and (not args.confirm_rehearsal_ui or args.robot_id=='SELF'):
20         p.error('Confirm official Q3 REHEARSAL UI and supply team robot-id. Formal mode is not implemented.')
21     url=args.url or ('http://127.0.0.1:2027' if args.mode=='self-http' else 'http://127.0.0.1:2026')
22     out=Path(args.output)
23     out.mkdir(parents=True,exist_ok=False)
24     config=json.loads(Path(__file__).with_name('configs').joinpath('baseline.json').read_text())
25     log=JsonLog(out/'actions.jsonl');events=JsonLog(out/'strategy.jsonl')
26     client=Client(HttpTransport(url),args.robot_id,log,config['real_time_reserve_s'])
27     start=time.perf_counter()
28     try:result=run(client,config,events)
29     except Exception as e:result=dict(status='incomplete',failure=f'{type(e).__name__}: {e}',metrics=client.metrics())
30     result.update(kind=args.mode,config=config,elapsed_wall_s=time.perf_counter()-start,
31                 official_total=None,clearance_ratio=None,
32                 note='Official count must be read from rehearsal UI after exit; no hidden count in HTTP API.')
33     (out/'result.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
34     log.close();events.close();print(json.dumps(result,ensure_ascii=False,indent=2))
35     if result['status']!='complete':raise SystemExit(1)
36
37 if __name__=='__main__':main()

```

程序 145

对应文件: question3/results/round1_main/code_snapshot/question3/run_matrix.py; question3/results/round1_rounding/code_snapshot/question3/run_matrix.py

```

1  """First-round fixed factorial experiment. Saves EVERY run including failures.
2  Case truth accessible to evaluator only, after strategy. No holdout generation.
3  """
4  import argparse
5  import hashlib
6  import json
7  import platform
8  import sys
9  import time
10 from pathlib import Path
11 from .cases import development_cases
12 from .simulator import Simulator
13 from .interface import Client,JsonLog
14 from .strategy import run
15 from .modules import design_catalog

```

```

16 from .audit import audit_folder
17
18 VARIANTS=['baseline','A','B','C','AB','AC','BC','ABC']
19
20 def main():
21     parser=argparse.ArgumentParser();parser.add_argument('--output',required=True)
22     parser.add_argument('--rounding',choices=['bounded','pre_round_stress'],default='bounded')
23     parser.add_argument('--limit',type=int)
24     args=parser.parse_args();out=Path(args.output);out.mkdir(parents=True,exist_ok=False)
25     configs={n:json.loads(Path(f'question3/configs/round1/{n}.json').read_text()) for n in VARIANTS}
26     fixtures=list(development_cases());rows=[]
27     startup=time.perf_counter();design_catalog();catalog_startup=time.perf_counter()-startup
28     files=list(Path('question3').glob('*.py'))+[Path('question1/solve.py'),Path('question2/strategy.py')]
29     manifest={str(p):hashlib.sha256(p.read_bytes()).hexdigest() for p in files}
30     snapshot=out/'code_snapshot';snapshot.mkdir()
31     for p in files:
32         dest=snapshot/p;dest.parent.mkdir(parents=True,exist_ok=True);dest.write_bytes(p.read_bytes())
33     fixtures_hash={c['id']:hashlib.sha256(json.dumps(c,sort_keys=True).encode()).hexdigest() for c in fixtures}
34     (out/'configs.json').write_text(json.dumps(configs,indent=2))
35     for i,case in enumerate(fixtures):
36         if args.limit is not None and i>=args.limit:break
37         # Rotate config order: no module consistently absorbs first-run overhead.
38         order=VARIANTS[i%8:] + VARIANTS[:i%8]
39         for name in order:
40             cfg=configs[name];folder=out/name/case['id'];folder.mkdir(parents=True)
41             (folder/'evaluator_hidden_case.json').write_text(json.dumps(case,indent=2),encoding='utf-8')
42             log=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
43             env=Simulator(case,rounding=args.rounding);client=Client(env,'SELF',log,cfg['real_time_reserve_s'])
44             start=time.perf_counter()
45             try:result=run(client,cfg,events)
46             except Exception as exc:result=dict(status='incomplete',failure=f'{type(exc).__name__}:
↳ {exc}',metrics=client.metrics(),regions=[])
47             wall=time.perf_counter()-start;truth=env.evaluation()
48             row=dict(variant=name,case_id=case['id'],kind='SELF_ROUND1_DEVELOPMENT',rounding=args.rounding,
49                     status=result['status'],failure=result.get('failure'),elapsed_wall_s=wall,
50                     total=truth['total'],cleared=truth['cleared'],remaining_channels=truth['remaining_channels'],
51                     clearance_ratio=truth['cleared']/truth['total'],
52                     average_virtual_per_cleared=truth['virtual_time_s']/truth['cleared'] if truth['cleared'] else
↳ None,
53                     metrics=result['metrics'])
54             row['false_complete']=result['status']=='complete' and bool(truth['remaining_channels'])
55             row['timeout']=bool(result.get('failure') and any(s in result['failure'].lower() for s in
↳ ('deadline','timeout','budget')))
56             row['penalized_virtual_s']=truth['virtual_time_s'] if result['status']=='complete' and not
↳ truth['remaining_channels'] else max(360000,truth['virtual_time_s'])
57             (folder/'result.json').write_text(json.dumps(dict(summary=row,strategy=result),ensure_ascii=False,indent=2),
↳ encoding='utf-8')
58             log.close();events.close()
59             if row['status']=='complete':
60                 try:row['audited_actions']=audit_folder(folder);row['audit']='pass'
61                 except Exception as exc:row['audit']='FAIL: '+str(exc);row['false_complete']=True
62             else:row['audit']='incomplete run retained; no complete certificate asserted'
63             rows.append(row)
64             if row['status']!='complete' or
↳ row['false_complete']:print('FAIL',name,case['id'],row['failure'],row['audit'],flush=True)
65             print(f'{i+1}/{len(fixtures)} {case["id"]}: '+,
↳ '.join(f'{r["variant"]]={r["metrics"]}[\"virtual_time_s\"]:.0f}' for r in rows[-8:]),flush=True)
66             summary=dict(kind='SELF_ROUND1_DEVELOPMENT_NOT_HOLDOUT',rounding=args.rounding,cases=rows,
67                         configs=configs,case_sha256=fixtures_hash,code_sha256=manifest,
68                         catalog_startup_wall_s=catalog_startup,platform=platform.platform(),python=sys.version,
69                         official_rehearsals_run=0,formal_tests_run=0,unseen_validation_cases_run=0)
70             (out/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
71             if any(r['false_complete'] for r in rows):raise SystemExit(2)
72
73 if __name__=='__main__':main()

```

程序 146

对应文件: question3/results/round1_main/code_snapshot/question3/strategy.py; question3/results/round1_rounding/code_snapshot/question3/strategy.py

```

1 """Common complete workflow with optional modules A/B/C. Defaults preserve baseline_v1.
2 No imports of case generation/simulator; all decisions use public observations only.
3 """
4 import math
5 from .geometry import stations, station_cover_bound, optical_plan
6 from .modules import choose_second, EvidenceCover
7
8 class Incomplete(RuntimeError):pass
9
10
11 def run(client, config, event=lambda data:None):
12     cover=stations()
13     if station_cover_bound()>=1000:raise ValueError('Invalid search coverage')
14     negative={c:set() for c in range(1,21)};first={};cleared=set();regions=[];visited=set()
15     modules=config.get('modules',{});joint=modules.get('B',False);evidence=modules.get('C',False)
16     if evidence and config['optical_cell_m']!=28:raise ValueError('Evidence module currently certifies 28m coarse
    ↪ cells only')
17     client.enter();failure=None;exit_confirmed=False;coverage_emitted=False;consecutive_localizations=0
18     def scan(station_id):
19         s=cover[station_id]
20         for channel in range(1,21):
21             if channel in first or channel in cleared:continue
22             response=client.measure(s,channel,phase='search');kind=response['measure_result']
23             if kind=='no_signal':negative[channel].add(station_id)
24             elif kind=='near':
25                 if client.clear(s,channel,phase='near')['clear_result']!='success':raise Incomplete('near followed by
    ↪ failed clear')
26                 cleared.add(channel)
27             elif kind=='direction':first[channel]=(s,response['svd_deg'])
28             else:raise Incomplete('Unknown measure result')
29         visited.add(station_id)
30     def localize(channel,q,selection):
31         s,theta=first[channel];r=client.measure(q,channel,phase='localization')
32         planner=None
33         if r['measure_result']=='near':points=[q];certificate=dict(planned_clear_calls=1,covering_radius=5)
34         elif r['measure_result']=='direction':
35             points,certificate=optical_plan(s,theta,q,r['svd_deg'],config['error_deg'],config['optical_cell_m'],confi
    ↪ g['max_optical_points'])
36             if evidence and len(points)<=128:
37                 planner=EvidenceCover(points,certificate,s,theta,q,r['svd_deg'],[cover[k] for k in
    ↪ sorted(negative[channel])],config['error_deg'])
38             elif evidence:
39                 certificate['evidence_fallback']='more_than_128_coarse_cells_use_original_cover'
40             else:raise Incomplete('Guaranteed-reception second point returned no_signal')
41             certificate.update(channel=channel,first_position=s,second_position=q,selection=selection)
42             regions.append(certificate);event(dict(type='localization',**certificate))
43             for i in range(len(points)):
44                 if planner:
45                     k,p,info=planner.next_point(client.position);event(dict(type='evidence_choice',channel=channel,point=
    ↪ p,**info))
46                 else:p=points[i]
47                 if client.clear(p,channel,phase='optical')['clear_result']=='success':
48                     cleared.add(channel)
49                     if planner:
50                         certificate['evidence']=planner.metrics();event(dict(type='evidence_result',channel=channel,**pla
    ↪ nner.metrics()))
51                     break
52                 if planner:planner.failed(k)
53             else:raise Incomplete('All optical covering points failed: model/protocol inconsistency')
54     try:
55         while True:
56             pending=sorted(set(first)-cleared);remaining=sorted(set(range(len(cover)))-visited)
57             if not remaining and not coverage_emitted:
58                 event(dict(type='coverage_complete',detected=sorted(first),cleared=sorted(cleared),
59                 absent=[c for c in negative if len(negative[c])==len(cover)]));coverage_emitted=True
60             if not pending and not remaining:break
61             if not joint:
62                 if remaining:scan(remaining[0]);continue
63                 channel=pending[0];s,theta=first[channel];q,selection=choose_second(s,theta,client.position,config)
64                 localize(channel,q,selection);continue
65             # B: choose nearest task start, force coverage after at most two localizations.
66             station_id=min(remaining,key=lambda k:(math.dist(client.position,cover[k]),k)) if remaining else None
67             jobs=[]

```



```

68         for c in pending:
69             q,sel=choose_second(*first[c],client.position,config)
70             jobs.append((math.dist(client.position,q),c,q,sel))
71             job=min(jobs,key=lambda x:(x[0],x[1])) if jobs else None
72             if station_id is not None and (job is None or consecutive_localizations>=2 or
73             ↪ math.dist(client.position,cover[station_id])<=job[0]):
74                 event(dict(type='schedule',task='scan',station_id=station_id,forced=consecutive_localizations>=2))
75                 scan(station_id);consecutive_localizations=0
76             else:
77                 _,c,q,sel=job;event(dict(type='schedule',task='localize',channel=c))
78                 localize(c,q,sel);consecutive_localizations+=1
79             if not all(c in cleared or len(negative[c])==len(cover) for c in range(1,21)):
80                 raise Incomplete('Stopping certificate is incomplete')
81             if not 10<=len(cleared)<=16:raise Incomplete('Cleared count contradicts Q3 count bounds')
82             exit_confirmed=client.exit()['exit_reason']=='user_exit'
83         except (RuntimeError, ValueError, KeyError) as exc:
84             failure=f'{type(exc).__name__}: {exc}'
85             result=dict(status='complete' if failure is None and exit_confirmed else 'incomplete',failure=failure,
86                         exit_confirmed=exit_confirmed,cleared_channels=sorted(cleared),
87                         absent_channels=[c for c in negative if len(negative[c])==len(cover)],
88                         negative_station_ids={str(c):sorted(v) for c,v in negative.items()},station_positions=cover,
89                         regions=regions,metrics=client.metrics(),modules=modules)
90         event(dict(type='strategy_result',**result));return result

```

程序 147

对应文件: question3/tests/test_baseline.py

```

1 import copy
2 import json
3 import math
4 import threading
5 import unittest
6 from pathlib import Path
7 import numpy as np
8 from question2.strategy import candidate
9 from question3.cases import random_case,late_source_case,boundary_case,near_case
10 from question3.geometry import stations,station_cover_bound,optical_plan
11 from question3.interface import Client,HttpTransport,ProtocolError,TransportUncertain
12 from question3.simulator import Simulator
13 from question3.strategy import run
14 from question3.self_server import make_server
15
16 CONFIG=json.loads(Path('question3/configs/baseline.json').read_text())
17
18 def payload(i,**kw):return dict(arena_id='default',robot_id='SELF',request_id=str(i),**kw)
19
20 def source(c,x,y,r=1000):return dict(channel=c,x=x,y=y,radius=r)
21
22 class PhysicsTests(unittest.TestCase):
23     def test_appendix_199_seconds_and_clear_channel(self):
24         env=Simulator(dict(sources=[]));c=Client(env,'SELF');c.enter()
25         self.assertEqual(c.measure((300,400),1)['virtual_time_s'],105)
26         self.assertEqual(c.measure((300,400),2)['virtual_time_s'],111)
27         self.assertEqual(c.clear((300,0),3)['virtual_time_s'],194)
28         self.assertEqual(c.measure((300,0),2)['virtual_time_s'],199)
29         self.assertEqual(c.channel,2);self.assertEqual(c.exit()['virtual_time_s'],199)
30     def test_radius_and_near_and_clear_boundaries(self):
31         c=Client(Simulator(dict(sources=[source(1,0,0)]),'SELF');c.enter()
32         self.assertEqual(c.measure((1000,0),1)['measure_result'],'direction')
33         self.assertEqual(c.measure((1000.0001,0),1)['measure_result'],'no_signal')
34         self.assertEqual(c.measure((5,0),1)['measure_result'],'near')
35         self.assertEqual(c.measure((5.0001,0),1)['measure_result'],'direction')
36         self.assertEqual(c.clear((20.0001,0),1)['clear_result'],'no_target_in_range')
37         self.assertEqual(c.clear((20,0),1)['clear_result'],'success')
38         self.assertEqual(c.clear((0,0),1)['clear_result'],'no_target_in_range')
39     def test_bias_fixed_at_same_location_and_bounded(self):
40         for mode in ['spatial_hash','smooth','positive','negative']:
41             c=Client(Simulator(dict(sources=[source(1,123,321,1500)],error_mode=mode)), 'SELF');c.enter()
42             self.assertEqual(c.measure((0.,0.),1)['svd_deg'],c.measure((-0.,-0.),1)['svd_deg'])
43             for p in [(0,0),(700,100),(-100,-200)]:

```

```

44         a=c.measure(p,1)['svd_deg'];b=c.measure(p,1)['svd_deg']
45         self.assertEqual(a,b)
46         true=math.degrees(math.atan2(321-p[1],123-p[0]))
47         self.assertLessEqual(abs((a-true+180)%360-180),1+1e-10)
48         self.assertAlmostEqual(a*100,round(a*100),places=7)
49     def test_rounding_stress_enveloped(self):
50         c=Client(Simulator(dict(sources=[source(1,123,321,1500)],error_mode='positive'),rounding='pre_round_stress'),
51         ↪ 'SELF');c.enter()
52         a=c.measure((0,0),1)['svd_deg'];true=math.degrees(math.atan2(321,123))
53         self.assertLessEqual(abs((a-true+180)%360-180),1.005+1e-10)
54     def test_no_truth_fields(self):
55         c=Client(Simulator(random_case(0)), 'SELF')
56         r=c.enter()
57         self.assertEqual(set(r),{'accepted','real_timestamp_ms','virtual_time_s','max_virtual_duration_s','max_real_d
58         ↪ uration_s','remaining_real_duration_s'})
59         r=c.measure((0,0),1)
60         self.assertTrue(set(r)<={'accepted','real_timestamp_ms','virtual_time_s','measure_result','svd_deg'})
61
62     class ProtocolTests(unittest.TestCase):
63     def test_idempotency_and_conflict(self):
64         e=Simulator(dict(sources=[]));e.request('/enter',payload('e'))
65         p=payload('m',position=dict(x=300,y=400),channel=1)
66         a=e.request('/measure',p);b=e.request('/measure',p)
67         self.assertEqual(a,b);self.assertEqual(e.evaluation()['virtual_time_s'],105)
68         p['channel']=2;self.assertEqual(e.request('/measure',p)[0],409)
69     def test_lost_response_same_id_exactly_once(self):
70         e=Simulator(dict(sources=[]));seen=[]
71         class Lost:
72         def request(self,path,payload,timeout):
73             r=e.request(path,payload,timeout);seen.append((path,copy.deepcopy(payload)))
74             if path=='/measure' and len([x for x in seen if x[0]=='/measure'])==1:raise ConnectionError('Lost
75             ↪ after commit')
76             return r
77         c=Client(Lost(),'SELF');c.enter();c.measure((300,400),1)
78         self.assertEqual(seen[-1],seen[-2]);self.assertEqual(c.virtual,105)
79         self.assertEqual(c.counts['measure'],1);self.assertEqual(c.counts['retries'],1)
80     def test_rejected_clock_zero_does_not_reset(self):
81         e=Simulator(dict(sources=[]));c=Client(e,'SELF');c.enter();c.measure((0,0),1)
82         c.robot_id='WRONG'
83         with self.assertRaises(ProtocolError):c.measure((0,0),2)
84         self.assertEqual(c.virtual,5);self.assertEqual(c.channel,1)
85     def test_failed_http_status_not_accepted(self):
86         class Bad:
87         def request(self,*args):return 500,dict(accepted=True,virtual_time_s=0)
88         with self.assertRaises(ProtocolError):Client(Bad(),'SELF').enter()
89     def test_uncertain_action_blocks_new_actions(self):
90         class Broken:
91         def request(self,*args):raise ConnectionError('down')
92         c=Client(Broken(),'SELF')
93         with self.assertRaises(TransportUncertain):c.enter()
94         with self.assertRaises(ProtocolError):c.enter()
95         self.assertEqual(c.virtual,0)
96     def test_invalid_coordinate_no_effect(self):
97         e=Simulator(dict(sources=[]));e.request('/enter',payload('e'))
98         r=e.request('/measure',payload('m',position=dict(x=float('nan'),y=0),channel=1))
99         self.assertEqual(r[0],400);self.assertEqual(e.evaluation()['virtual_time_s'],0)
100     def test_unknown_field_rejected_without_id_consumption(self):
101         e=Simulator(dict(sources=[]))
102         p=payload('e',wrong=1);self.assertFalse(e.request('/enter',p)[1]['accepted'])
103         del p['wrong'];self.assertTrue(e.request('/enter',p)[1]['accepted'])
104     def test_deadline_incomplete(self):
105         e=Simulator(random_case(0),remaining_s=0);c=Client(e,'SELF');r=run(c,CONFIG)
106         self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
107     def test_virtual_guard(self):
108         c=Client(Simulator(dict(sources=[])), 'SELF');c.enter();c.max_virtual=4
109         with self.assertRaises(ProtocolError):c.measure((0,0),1)
110         self.assertEqual(c.virtual,0)
111
112     class StrategyTests(unittest.TestCase):
113     def test_seven_station_cover(self):
114         self.assertLess(station_cover_bound(),1000)
115         angles=np.linspace(0,2*np.pi,721);r=np.linspace(0,1800,41)
116         points=np.array([(x*math.cos(a),x*math.sin(a)) for x in r for a in angles])

```

```

114     nearest=np.linalg.norm(points[:,None,:]-np.array(stations())[None,:,:],axis=2).min(axis=1)
115     self.assertLess(nearest.max(),1000)
116 def test_q2_extended_error_certificate(self):
117     self.assertTrue(candidate([800,605],20,1.005))
118     self.assertFalse(candidate([800,0],20,1.005))
119 def test_full_cases_stop_certificate(self):
120     for case in [random_case(),boundary_case(),late_source_case(),near_case()]:
121         e=Simulator(case);r=run(Client(e,'SELF'),CONFIG)
122         self.assertEqual(r['status'],'complete');self.assertEqual(e.evaluation()['remaining_channels'],[])
123         cleared=set(r['cleared_channels'])
124         for c in range(1,21):
125             self.assertTrue(c in cleared or len(r['negative_station_ids'][str(c)])==7)
126 def test_late_eleventh_source_not_omitted(self):
127     e=Simulator(late_source_case());events=[];r=run(Client(e,'SELF'),CONFIG,events.append)
128     self.assertIn(20,r['cleared_channels']);self.assertEqual(len(r['cleared_channels']),11)
129     self.assertIn(20,next(row for row in events if row['type']=='coverage_complete')['detected'])
130 def test_localization_contract_violation_incomplete(self):
131     e=Simulator(random_case());events=[]
132     class BadSecond:
133         def request(self,path,payload,timeout):
134             status,r=e.request(path,payload,timeout)
135             if events and events[-1]['type']=='coverage_complete' and path=='/measure':
136                 r['measure_result']='no_signal';r.pop('svd_deg',None)
137             return status,r
138     r=run(Client(BadSecond(),'SELF'),CONFIG,events.append)
139     self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
140 def test_optical_failure_is_not_success(self):
141     e=Simulator(random_case())
142     class BadClear:
143         def request(self,path,payload,timeout):
144             status,r=e.request(path,payload,timeout)
145             if path=='/clear':
146                 # Mimic impossible feedback AND accounting consistently.
147                 if r.get('clear_result')=='success':
148                     e._virtual_us-=2000000;e._components['laser_s']-=2
149                     r['virtual_time_s']-=2;r['clear_result']='no_target_in_range'
150             return status,r
151     r=run(Client(BadClear(),'SELF'),CONFIG)
152     self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
153 def test_optical_grid_covers_polygon(self):
154     first=(0,0);g=np.array([1499.,20.]);q=(800,605)
155     b=math.degrees(math.atan2(g[1]-q[1],g[0]-q[0]))+.8
156     points,cert=optical_plan(first,0,q,b,1.005,28,2000)
157     self.assertLess(cert['covering_radius'],20)
158     self.assertLessEqual(min(np.linalg.norm(np.array(p)-g) for p in points),20)
159     # Random convex combinations cover interior, independent of chosen true G.
160     rng=np.random.default_rng(5);v=np.array(cert['vertices'])
161     w=rng.dirichlet(np.ones(len(v)),1000);samples=w@v
162     self.assertLess(np.linalg.norm(samples[:,None,:]-np.array(points)[None,:,:],axis=2).min(axis=1).max(),20)
163 def test_strategy_does_not_import_simulator(self):
164     import ast
165     root=ast.parse(Path('question3/strategy.py').read_text())
166     imports=[n.module or '' for n in ast.walk(root) if isinstance(n,ast.ImportFrom)]
167     self.assertTrue(all('simulator' not in x and 'cases' not in x for x in imports))
168
169 class HttpTests(unittest.TestCase):
170     def test_complete_over_real_loopback_http(self):
171         e=Simulator(late_source_case());server=make_server(e)
172         thread=threading.Thread(target=server.serve_forever,daemon=True);thread.start()
173         try:
174             client=Client(HttpTransport(f'http://127.0.0.1:{server.server_port}'),'SELF')
175             r=run(client,CONFIG)
176             self.assertEqual(r['status'],'complete');self.assertEqual(e.evaluation()['remaining_channels'],[])
177         finally:server.shutdown();server.server_close();thread.join()
178
179 if __name__=='__main__':unittest.main(verbosity=2)

```

程序 148

对应文件: question3/tests/test_batch_scheduling.py

```

1 """Finite batches preserve evidence, do not fake visits, and fail honestly."""
2 import json
3 from pathlib import Path
4 import unittest
5 from question3.batch_scheduling import gate
6 from question3.cases import random_case,late_source_case
7 from question3.interface import Client
8 from question3.simulator import Simulator
9 from question3.strategy import run
10
11
12 def config(name):return json.loads(Path('question3/configs/round15_dev',name+'.json').read_text())
13
14
15 class BatchSchedulingTests(unittest.TestCase):
16     def test_cached_batch_filters_only_ready_jobs_without_mutating_evidence(self):
17         jobs=[(1,2,(0,0),{}),(2,3,(1,1),{})];remaining=[1,2,3]
18         selected,allowed,forced,info=gate('cached_batch',[2,3],{3:None},jobs,remaining,20)
19         self.assertEqual([j[1] for j in selected],[3]);self.assertEqual(allowed,[]);self.assertFalse(forced)
20         self.assertEqual(remaining,[1,2,3]);self.assertEqual(info['deferred_stations'],remaining)
21         selected,allowed,forced,info=gate('cached_batch',[2],{},{},jobs[:1],remaining,20)
22         self.assertEqual(allowed,remaining);self.assertTrue(forced);self.assertIsNone(info)
23
24     def test_known_batch_keeps_all_known_jobs_and_returns_to_search(self):
25         jobs=[(1,2,(0,0),{})]
26         self.assertEqual(gate('known_batch',[2],{},{},jobs,[1],20)[:3],(jobs,[],False))
27         self.assertEqual(gate('known_batch',[1],{},{},[1],20)[:3],([], [1],True))
28
29     def test_batch_selections_clear_at_most_sixteen_distinct_targets_between_scans(self):
30         for mode in ['CACHE','KNOWN']:
31             events=[];env=Simulator(random_case(0));r=run(Client(env,'SELF'),config(mode),events.append)
32             self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
33             active=None;chain=[];batch_steps=0
34             for e in events:
35                 if e['type']=='localization_batch':active=e;batch_steps+=1
36                 if e['type']=='schedule':
37                     if active:
38                         self.assertEqual(e['task'],'localize');self.assertIn(e['channel'],active['batch_channels']);a
39                             ↪ ctive=None
40                     if e['task']=='scan':chain=[]
41                     else:
42                         self.assertNotIn(e['channel'],chain);chain.append(e['channel']);self.assertLessEqual(len(chai
43                             ↪ n),16)
44             self.assertGreater(batch_steps,0)
45
46     def test_late_channel_and_absence_certificate_not_replaced_by_queue_exhaustion(self):
47         for mode in ['CACHE','KNOWN']:
48             r=run(Client(Simulator(late_source_case()),'SELF'),config(mode))
49             self.assertEqual(r['status'],'complete');self.assertIn(20,r['cleared_channels'])
50             self.assertEqual(r['stopping_certificate']['method'],'seven_station_channel_cover')
51             for c in r['absent_channels']:self.assertEqual(len(r['negative_station_ids'][str(c)]),7)
52
53     def test_deadline_during_batch_is_not_complete(self):
54         for mode in ['CACHE','KNOWN']:
55             client=Client(Simulator(random_case(0)),'SELF');seen=[]
56             def event(e):
57                 if e['type']=='localization_batch':seen.append(e);client.deadline=0
58             r=run(client,config(mode),event)
59             self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
60
61     def test_invalid_and_incompatible_mode_before_enter(self):
62         for c in [{**config('KNOWN'),'search_interleaving':'both'},
63                 {**config('KNOWN'),'modules':{**config('KNOWN')['modules'],'B':False}},
64                 {**config('CACHE'),'modules':{**config('CACHE')['modules'],'S':False}}]:
65             client=Client(Simulator(random_case(0)),'SELF')
66             with self.assertRaises(ValueError):run(client,c)
67             self.assertFalse(client.entered)
68
69     def test_explicit_current_equals_omitted_setting(self):
70         c=config('REF');a=run(Client(Simulator(random_case(0)),'SELF'),c)
71         c['search_interleaving']='current';b=run(Client(Simulator(random_case(0)),'SELF'),c)
72         self.assertEqual(a,b)

```

程序 149

对应文件: question3/tests/test_cached_routing.py

```
1 import json
2 import math
3 from pathlib import Path
4 import unittest
5 from question3.cached_routing import insertion_cost, reorder_cached
6 from question3.cases import late_source_case, random_case
7 from question3.interface import Client
8 from question3.simulator import Simulator
9 from question3.strategy import run
10
11
12 class CachedRoutingTests(unittest.TestCase):
13     def test_full_path_bounds_all_successful_prefixes(self):
14         current=(3,8); anchor=(100,20)
15         for points in [[(10,0)], [(-30,20), (90,20), (45,60)], [(2,1), (2,1), (2,1)]]:
16             bound=insertion_cost(current, points, anchor)
17             for k in range(1, len(points)+1):
18                 prefix=insertion_cost(current, points[:k], anchor)
19                 self.assertLessEqual(prefix, bound+1e-9)
20             direct=(math.dist(current, points[0])+sum(math.dist(a,b) for a,b in zip(points, points[1:])))
21                 +math.dist(points[-1], anchor)-math.dist(current, anchor))/5+3*len(points)+2
22             self.assertAlmostEqual(bound, direct)
23
24     def test_insertion_can_choose_farther_entry(self):
25         current=(0,0); cover=[(100,0)]; cached={1:([(10,0), (-100,0)], {}, None), 2:([(20,0), (90,0)], {}, None)}
26         jobs=[(10,1, (10,0), {}), (20,2, (20,0), {})] ; base=('localize', jobs[0], {'forced': False})
27         task, target, info=reorder_cached(base, current, cover, [0], jobs, cached)
28         self.assertEqual(target[1], 2); self.assertTrue(info['cache_order_changed'])
29         for decision, remaining in [ (('scan', 0, {'forced': True}), [0]), (('scan', 0, {'forced': False}), [0]), (base, [1]) ]:
30             self.assertEqual(reorder_cached(decision, current, cover, remaining, jobs, cached), decision)
31         other=('localize', (1,3, (1,0), {}), {'forced': False})
32         self.assertEqual(reorder_cached(other, current, cover, [0], jobs, cached), other)
33
34     def test_existing_choice_on_equal_cost(self):
35         job=(10,2, (10,0), {}); jobs=[(10,1, (10,0), {}), job]
36         cache={1:([(10,0)], {}, None), 2:([(10,0)], {}, None)}
37         answer=reorder_cached(('localize', job, {}), (0,0), [(100,0)], [0], jobs, cache)
38         self.assertEqual(answer[1][1], 2)
39
40     def test_full_flow_and_search_fairness(self):
41         config=json.loads(Path('question3/configs/round6_dev/I.json').read_text())
42         for case in [late_source_case(), random_case(6, 'smooth')]:
43             events=[]; env=Simulator(case); result=run(Client(env, 'SELF'), config, events.append)
44             self.assertEqual(result['status'], 'complete'); self.assertFalse(env.evaluation()['remaining_channels'])
45             scans=chain=0
46             for e in events:
47                 if e['type']!='schedule': continue
48                 if e['task']=='scan': scans+=1; chain=0
49                 else:
50                     chain+=1
51                     if scans<7: self.assertLessEqual(chain, 2)
52             self.assertEqual(scans, 7)
53
54     def test_incompatible_before_enter(self):
55         config=json.loads(Path('question3/configs/round6_dev/I.json').read_text())
56         for flag in ['C', 'O']:
57             c=**config, 'modules':{**config['modules'], flag:True}
58             client=Client(Simulator(random_case(0)), 'SELF')
59             with self.assertRaises(ValueError): run(client, c)
60             self.assertFalse(client.entered)
```

程序 150

对应文件: question3/tests/test_clear_skip.py

```
1 """Geometric boundaries and whole-flow equivalence for execution-only skipping."""
2 import json
```

```

3 from pathlib import Path
4 import unittest
5 import tempfile
6 from unittest.mock import patch
7 import numpy as np
8 from question3.clear_skip import failure_certificate
9 from question3.cases import random_case,late_source_case
10 from question3.interface import Client,JsonLLog
11 from question3.audit import audit_folder
12 from question3.simulator import Simulator
13 from question3.strategy import run
14
15
16 def config(on=True):
17     c=json.loads(Path('question3/configs/round11_dev/REF.json').read_text())
18     if on:c['clear_skip']='separating_halfplane'
19     return c
20
21
22 class ClearSkipTests(unittest.TestCase):
23     def test_strict_disk_separation_and_inconclusive_corner(self):
24         planes=(np.array([[1.,0.],[0.,1.]]),np.zeros(2))
25         for p in [(20,0),(20+1e-6,0),(0,0),(-100,-100),(15,15)]:
26             self.assertIsNone(failure_certificate(p,planes))
27             self.assertIsNotNone(failure_certificate((20.001,0),planes))
28             self.assertIsNone(failure_certificate((float('nan'),0),planes))
29             self.assertIsNone(failure_certificate((100,100),None))
30
31     def test_rotation_translation_normal_scaling(self):
32         angle=.789;normal=np.array([np.cos(angle),np.sin(angle)])
33         origin=np.array([1000000.,-1000000.])
34         planes=(normal[None,:]*7,np.array([normal@origin*7]))
35         self.assertIsNone(failure_certificate(origin+20*normal,planes))
36         self.assertIsNotNone(failure_certificate(origin+20.01*normal,planes))
37
38     def test_same_success_positions_and_measurements_on_fixed_cases(self):
39         skipped=0
40         for case in [random_case(i) for i in range(4)]+[late_source_case()]:
41             runs=[]
42             for enabled in [False,True]:
43                 req=[];es=[];env=Simulator(case)
44                 r=run(Client(env,'SELF',req.append),config(enabled),es.append)
45                 self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
46                 requests={x['payload']['request_id']:x for x in req if x['type']=='request'}
47                 retained=[]
48                 for e in req:
49                     if e['type']!='response':continue
50                     q=requests[e['request_id']]
51                     if q['path']=='/measure' or q['path']=='/clear' and e['response'].get('clear_result')=='success':
52                         retained.append((q['path'],q['payload']['position'],q['payload']['channel']))
53                 runs.append((r,retained,es))
54                 self.assertEqual(runs[0][1],runs[1][1])
55                 self.assertEqual(runs[1][0]['metrics']['virtual_time_s'],runs[0][0]['metrics']['virtual_time_s']+1e-5)
56                 skipped+=sum(e['type']=='certified_clear_skip' for e in runs[1][2])
57             self.assertGreater(skipped,0)
58
59     def test_exhausted_skip_plan_never_claims_success(self):
60         with patch('question3.strategy.failure_certificate',return_value=dict(halfplane_index=0,separation_m=100.,sla_
↳ ck_m=1e-5)):
61             r=run(Client(Simulator(random_case(0)),'SELF'),config())
62             self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
63             self.assertIn('All optical',r['failure'])
64
65     def test_deadline_after_a_skip_is_incomplete(self):
66         client=Client(Simulator(random_case(1)),'SELF');seen=[]
67         def event(e):
68             if e['type']=='certified_clear_skip':seen.append(e);client.deadline=0
69         r=run(client,config(),event)
70         self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
71
72     def test_independent_audit_rejects_forged_skip_witness(self):
73         with tempfile.TemporaryDirectory() as temp:
74             folder=Path(temp);case=random_case(1)

```

```

75         actions=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
76         r=run(Client(Simulator(case), 'SELF',actions),config(),events)
77         actions.close();events.close()
78         (folder/'evaluator_hidden_case.jsonl').write_text(json.dumps(case))
79         (folder/'result.jsonl').write_text(json.dumps(dict(summary=dict(rounding='bounded'),strategy=r)))
80         self.assertGreater(audit_folder(folder),0)
81         rows=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
82         skips=[e for e in rows if e['type']=='certified_clear_skip'];self.assertTrue(skips)
83         skips[0]['separation_m']=1000000
84         (folder/'strategy.jsonl').write_text('\n'.join(json.dumps(e) for e in rows)+'\n')
85         with self.assertRaises(AssertionError):audit_folder(folder)
86
87     def test_invalid_and_dynamic_plan_config_before_enter(self):
88         for c in [{**config(),'clear_skip':'unknown'},]**config(),{'modules':**config()['modules'],'C':True}}:
89             client=Client(Simulator(random_case(0)), 'SELF')
90             with self.assertRaises(ValueError):run(client,c)
91             self.assertFalse(client.entered)

```

程序 151

对应文件: question3/tests/test_clear_skip_pair.py

```

1  """Combined constraints are only proposals until their nonnegative witness passes."""
2  import unittest
3  import json
4  import tempfile
5  from pathlib import Path
6  import numpy as np
7  from question3.clear_skip import failure_certificate
8  from question3.cases import random_case
9  from question3.interface import Client,JsonLog
10 from question3.simulator import Simulator
11 from question3.strategy import run
12 from question3.audit import audit_folder
13
14
15 class ClearSkipPairTests(unittest.TestCase):
16     def test_corner_missed_by_each_edge_is_certified_by_pair(self):
17         A=np.eye(2);b=np.zeros(2)
18         self.assertIsNone(failure_certificate((15,15),(A,b)))
19         proof=failure_certificate((15,15),(A,b),combine=True)
20         self.assertIsNotNone(proof)
21         w=np.array(proof['halfplane_weights']);self.assertTrue(np.all(w>=0));self.assertAlmostEqual(w.sum(),1)
22         self.assertAlmostEqual(proof['separation_m'],15*np.sqrt(2))
23
24     def test_tangent_and_uncertain_corner_are_retained(self):
25         planes=(np.eye(2),np.zeros(2))
26         for r in [0,19.99,20,20+1e-6]:
27             self.assertIsNone(failure_certificate(np.ones(2)*r/np.sqrt(2),planes,combine=True))
28             self.assertIsNotNone(failure_certificate(np.ones(2)*20.001/np.sqrt(2),planes,combine=True))
29
30     def test_parallel_and_opposing_normals_fail_closed(self):
31         for A,b,p in [(np.array([[1.,0.],[1.,0.]]),np.array([0.,1.]),(15.,15.)),
32                     (np.array([[1.,0.],[-1.,0.]]),np.zeros(2),(0.,100.))]:
33             self.assertIsNone(failure_certificate(p,(A,b),combine=True))
34
35     def test_single_witness_is_exactly_preserved(self):
36         planes=(np.eye(2),np.zeros(2))
37         self.assertEqual(failure_certificate((30,15),planes),failure_certificate((30,15),planes,combine=True))
38
39     def test_rotation_translation_and_valid_region_points(self):
40         angle=.437;R=np.array([[np.cos(angle),-np.sin(angle)],[np.sin(angle),np.cos(angle)]])
41         origin=np.array([1e6,-1e6]);A=np.eye(2)@R.T;b=A@origin;p=origin+R@np.array([15.,15.])
42         proof=failure_certificate(p,(A,b),combine=True);self.assertIsNotNone(proof)
43         for x in np.linspace(-40,0,9):
44             for y in np.linspace(-40,0,9):
45                 g=origin+R@np.array([x,y]);self.assertGreater(np.linalg.norm(g-p),20)
46                 self.assertIsNone(failure_certificate(origin-R@np.ones(2),(A,b),combine=True))
47
48     def test_auditor_recomputes_weights_and_rejects_negative_weight(self):
49         with tempfile.TemporaryDirectory() as temp:

```

```

50         folder=Path(temp);case=random_case(1)
51         config=json.loads(Path('question3/configs/round12_dev/Q.json').read_text())
52         actions=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
53         r=run(Client(Simulator(case),'SELF',actions),config,events);actions.close();events.close()
54         (folder/'evaluator_hidden_case.json').write_text(json.dumps(case))
55         (folder/'result.json').write_text(json.dumps(dict(summary=dict(rounding='bounded'),strategy=r)))
56         rows=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
57         proof=next(e for e in rows if e['type']=='certified_clear_skip')
58         # A single inequality is also a valid nonnegative-combination witness.
59         k=proof.pop('halfplane_index');proof['halfplane_weights']=[float(i==k) for i in range(4)]
60         path=folder/'strategy.jsonl';path.write_text('\n'.join(map(json.dumps,rows))+'\n')
61         self.assertGreater(audit_folder(folder),0)
62         proof['halfplane_weights'][(k+1)%4]=-0.1
63         path.write_text('\n'.join(map(json.dumps,rows))+'\n')
64         with self.assertRaises(AssertionError):audit_folder(folder)

```

程序 152

对应文件: question3/tests/test_count_stop.py

```

1 import json
2 from pathlib import Path
3 import tempfile
4 import threading
5 import unittest
6 from question3.audit import audit_folder
7 from question3.cases import late_source_case
8 from question3.interface import Client,HttpTransport,JsonLog
9 from question3.self_server import make_server
10 from question3.simulator import Simulator
11 from question3.strategy import run
12
13
14 def cfg(name):return json.loads(Path('question3/configs/round7_count',name+'.json').read_text())
15 def fixture(n=16,near=False):
16     return dict(id='count_contract',seed=991,error_mode='positive',sources=[
17         dict(channel=c,x=1*c if near else 300+5*c,y=0.,radius=1000.) for c in range(1,n+1)])
18
19
20 class CountStopTests(unittest.TestCase):
21     def test_sixteen_successes_stop_on_action_prefix(self):
22         case=fixture(near=True);base_log=[];short_log=[]
23         base=run(Client(Simulator(case),'SELF',base_log.append),cfg('REF'))
24         env=Simulator(case);short=run(Client(env,'SELF',short_log.append),cfg('U'))
25         def actions(log):return [(e['path'],e['payload'].get('position'),e['payload'].get('channel'))
26             for e in log if e['type']=='request' and e['path'] in ('/measure','/clear'))
27         before=actions(base_log);after=actions(short_log)
28         self.assertEqual(before[:len(after)],after);self.assertLess(len(after),len(before))
29         self.assertEqual(short['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
30         self.assertEqual(short['stopping_certificate']['method'],'source_count_upper_bound')
31         self.assertEqual(short['absent_channels'],[17,18,19,20])
32         self.assertTrue(all(len(short['negative_station_ids'][str(c)])<7 for c in [17,18,19,20]))
33         self.assertLess(short['metrics']['virtual_time_s'],base['metrics']['virtual_time_s'])
34
35     def test_sixteen_detected_are_not_sixteen_cleared(self):
36         class AlwaysFail(Client):
37             def clear(self,p,c,phase='clear'):
38                 result=super().clear(p,c,phase)
39                 return {'**result','clear_result':'no_target_in_range'}
40         events=[];r=run(AlwaysFail(Simulator(fixture()),'SELF'),cfg('K'),events.append)
41         saturated=[e for e in events if e['type']=='source_upper_bound_saturated']
42         self.assertEqual(len(saturated),1);self.assertEqual(len(saturated[0]['still_to_clear']),16)
43         self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
44         self.assertIsNone(r['stopping_certificate']);self.assertEqual(r['cleared_channels'],[])
45
46     def test_fewer_than_sixteen_keep_coverage_and_late_source(self):
47         for case in [fixture(15),late_source_case()]:
48             before=run(Client(Simulator(case),'SELF'),cfg('REF'))
49             for variant in ['U','K']:
50                 r=run(Client(Simulator(case),'SELF'),cfg(variant))
51                 self.assertEqual(r['metrics'],before['metrics']);self.assertEqual(r['status'],'complete')

```



```

52         self.assertEqual(r['stopping_certificate']['method'],'seven_station_channel_cover')
53         for c in r['absent_channels']:self.assertEqual(len(r['negative_station_ids'][str(c)]),7)
54         if case['id']=='late_channel120_after_ten':self.assertIn(20,r['cleared_channels'])
55
56     def test_deadline_after_discovery_cannot_complete(self):
57         client=Client(Simulator(fixture()),'SELF');seen=[]
58         def event(e):
59             if e['type']=='source_upper_bound_saturated':seen.append(e);client.deadline=0
60             r=run(client,cfg('DK'),event)
61             self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
62
63     def test_last_success_retry_counts_distinct_channels_once(self):
64         env=Simulator(fixture(near=True));dropped=False
65         class DropLast:
66             def request(self,path,body,timeout):
67                 nonlocal dropped
68                 response=env.request(path,body,timeout)
69                 if path=='/clear' and body['channel']==16 and not dropped:
70                     dropped=True;raise OSError('last accepted clear response lost')
71                 return response
72         r=run(Client(DropLast(),'SELF'),cfg('U'))
73         self.assertTrue(dropped);self.assertEqual(r['status'],'complete')
74         self.assertEqual(r['metrics']['counts']['clear_success'],16)
75         self.assertEqual(r['metrics']['counts']['retries'],1)
76
77     def test_auditor_rejects_forged_sixteen_certificate(self):
78         case=fixture(15)
79         with tempfile.TemporaryDirectory() as d:
80             folder=Path(d);log=JsonLog(folder/'actions.jsonl')
81             r=run(Client(Simulator(case),'SELF',log),cfg('REF'));log.close()
82             r['stopping_certificate']=dict(method='source_count_upper_bound',source_upper_bound=16,
83                 successful_channels=list(range(1,17)),inferred_absent_channels=list(range(17,21)))
84             (folder/'evaluator_hidden_case.json').write_text(json.dumps(case))
85             (folder/'result.json').write_text(json.dumps(dict(summary={'rounding':'bounded'},strategy=r)))
86             with self.assertRaises(AssertionError):audit_folder(folder)
87
88     def test_count_mode_real_self_http(self):
89         env=Simulator(fixture());server=make_server(env)
90         thread=threading.Thread(target=server.serve_forever,daemon=True);thread.start()
91         try:
92             r=run(Client(HttpTransport(f'http://127.0.0.1:{server.server_port}'),'SELF'),cfg('DK'))
93             self.assertEqual(r['status'],'complete');self.assertEqual(r['stopping_certificate']['method'],'source_cou
↳ nt_upper_bound')
94             self.assertFalse(env.evaluation()['remaining_channels'])
95             finally:server.shutdown();server.server_close();thread.join()
96
97     def test_unrecognized_bound_mode_before_enter(self):
98         c=cfg('U');c['count_stop']='cleared_10';client=Client(Simulator(fixture()),'SELF')
99         with self.assertRaises(ValueError):run(client,c)
100         self.assertFalse(client.entered)

```

程序 153

对应文件: question3/tests/test_current_point.py

```

1 import json
2 import math
3 from pathlib import Path
4 import unittest
5 import numpy as np
6 from question2.strategy import candidate,to_world
7 from question3.current_point import current_proxy,local_offset
8 from question3.modules import design_catalog,choose_second
9 from question3.cases import random_case,late_source_case
10 from question3.interface import Client
11 from question3.simulator import Simulator
12 from question3.strategy import run
13
14
15 def cfg(n='J'):return json.loads(Path('question3/configs/round8_dev',n+'.json').read_text())
16

```

```

17
18 class CurrentPointTests(unittest.TestCase):
19     def test_rotation_and_exact_current_position(self):
20         first=(234.,-567.);bearing=37.;current=tuple(to_world((500,500),first,bearing))
21         np.testing.assert_allclose(local_offset(first,bearing,current),(500,500),atol=1e-10,rtol=0)
22         q,s=choose_second(first,bearing,current,cfg('Z'))
23         self.assertEqual(q,current);self.assertEqual(s['mode'],'opportunistic_current')
24         self.assertTrue(candidate(s['offset'],20,1.005))
25
26     def test_same_proxy_as_catalog_and_invalid_points_excluded(self):
27         for row in design_catalog():
28             p=current_proxy(row['offset'],1.005,20)
29             self.assertAlmostEqual(p['sampled_worst_diameter'],row['sampled_worst_diameter'],places=7)
30             self.assertEqual(p['proxy_cells'],row['proxy_cells'])
31             self.assertAlmostEqual(p['continuation_proxy_s'],row['continuation_proxy_s'],places=7)
32         for point in [(0,0),(700,0),(-500,500),(1200,1200)]:
33             self.assertIsNone(current_proxy(point,1.005,20))
34             base=choose_second((0,0),0,point,cfg('REF'))
35             self.assertEqual(choose_second((0,0),0,point,cfg('J')),base)
36
37     def test_cheaper_gate_rejects_costly_admissible_current(self):
38         # Fixed development geometry: test both possible branches without fitting a threshold.
39         decisions=[]
40         for a in [200,400,600,800]:
41             for b in [300,500,700]:
42                 if not candidate((a,b),20,1.005):continue
43                 q,s=choose_second((0,0),0,(a,b),cfg('J'))
44                 old_q,old=choose_second((0,0),0,(a,b),cfg('REF'))
45                 self.assertLessEqual(s['score_s'],old['score_s']+1e-8)
46                 decisions.append(s['mode'])
47             self.assertIn('opportunistic_current',decisions);self.assertIn('cost_proxy',decisions)
48
49     def test_complete_flow_no_move_and_late_source(self):
50         for case in [random_case(0),late_source_case()]:
51             requests=[];events=[];env=Simulator(case)
52             r=run(Client(env,'SELF',requests.append),cfg('J'),events.append)
53             self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
54             selected=[e for e in events if e['type']=='localization' and
55 ↪ e['selection']['mode']=='opportunistic_current']
56             if case['id']=='dev_0_spatial_hash':self.assertTrue(selected)
57             position=(0.,0.);moves={}
58             for e in requests:
59                 if e['type']=='request' and e['path'] in ('/measure','/clear'):
60                     p=e['payload']['position'];q=(p['x'],p['y'])
61                     if e['path']=='/measure' and
62                     ↪ e['phase']=='localization':moves[e['payload']['channel']]=math.dist(position,q)
63                     position=q
64             for e in selected:self.assertEqual(moves[e['channel']],0)
65             if case['id']=='late_channel20_after_ten':self.assertIn(20,r['cleared_channels'])
66
67     def test_contradictory_current_reading_is_incomplete(self):
68         env=Simulator(random_case(0));hit=False
69         class BadClient(Client):
70             def measure(self,p,c,phase='measure'):
71                 nonlocal hit
72                 no_move=tuple(p)==tuple(self.position)
73                 r=super().measure(p,c,phase)
74                 if no_move and phase=='localization':
75                     hit=True;return {'**r','measure_result':'no_signal'}
76                 return r
77         r=run(BadClient(env,'SELF'),cfg('J'))
78         self.assertTrue(hit);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
79
80     def test_invalid_configuration_before_enter(self):
81         for bad in [{"**cfg(),'second_point_opportunity':'unknown'},
82 ↪ {"**cfg(),'modules':{'**cfg()['modules'],'A':False}}]:
83             client=Client(Simulator(random_case(0)),'SELF')
84             with self.assertRaises(ValueError):run(client,bad)
85             self.assertFalse(client.entered)

```

程序 154

对应文件: question3/tests/test_current_sharing.py

```
1 """Selective sharing must preserve ordinary batches and honest failure handling."""
2 import json
3 from pathlib import Path
4 import unittest
5 from question3.cases import random_case, late_source_case
6 from question3.interface import Client
7 from question3.simulator import Simulator
8 from question3.strategy import run
9
10
11 def config(limit=None):
12     c=json.loads(Path('question3/configs/round10_dev/REF.json').read_text())
13     if limit is not None:c['current_point_shared_limit']=limit
14     return c
15
16
17 class CurrentSharingTests(unittest.TestCase):
18     def test_current_cap_preserves_ordinary_batches_and_cached_measurements(self):
19         for limit in [0,1,19]:
20             events=[];requests=[];env=Simulator(random_case(0))
21             result=run(Client(env,'SELF',requests.append),config(limit),events.append)
22             modes={e['channel']:e['selection']['mode'] for e in events if e['type']=='localization'}
23             stores=[e for e in events if e['type']=='shared_cache_store'];batches={}
24             self.assertIn('opportunistic_current',modes.values())
25             for e in stores:batches[e['primary_channel']]=batches.get(e['primary_channel'],0)+1
26             for c,n in batches.items():
27                 if modes[c]=='opportunistic_current':self.assertLessEqual(n,limit)
28             self.assertGreater(max(n for c,n in batches.items() if modes[c]!='opportunistic_current'),1)
29             if limit:self.assertTrue(any(modes[c]=='opportunistic_current' for c in batches))
30             for e in stores:
31                 ms=[q for q in requests if q['type']=='request' and q['path']=='/measure'
32                    and q['payload']['channel']==e['channel'] and q['phase'].startswith('localization')]
33                 self.assertEqual(len(ms),1)
34             self.assertEqual(result['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
35             self.assertTrue(result['exit_confirmed'])
36
37     def test_omitted_limit_is_exactly_existing_all_sharing(self):
38         a=run(Client(Simulator(random_case(0)),'SELF'),config())
39         b=run(Client(Simulator(random_case(0)),'SELF'),config(19))
40         self.assertEqual(a,b)
41
42     def test_late_channel_still_requires_coverage(self):
43         r=run(Client(Simulator(late_source_case()),'SELF'),config(0))
44         self.assertEqual(r['status'],'complete');self.assertIn(20,r['cleared_channels'])
45         self.assertEqual(r['stopping_certificate']['method'],'seven_station_channel_cover')
46
47     def test_current_measurement_contradiction_cannot_complete(self):
48         client=Client(Simulator(random_case(0)),'SELF');original=client.measure;seen=[]
49         def measure(p,c,phase='measure'):
50             current=p==client.position
51             r=original(p,c,phase)
52             if phase=='localization' and current:
53                 seen.append(c);return {'*r','measure_result':'no_signal'}
54             return r
55         client.measure=measure
56         r=run(client,config(0))
57         self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
58
59     def test_bad_limit_rejected_before_enter(self):
60         for limit in [-1,20,1.5,True,None]:
61             c=config();c['current_point_shared_limit']=limit;client=Client(Simulator(random_case(0)),'SELF')
62             with self.assertRaises(ValueError):run(client,c)
63             self.assertFalse(client.entered)
```

程序 155

对应文件: question3/tests/test_domain_range.py

```

1 """Continuous-certificate boundary checks and public-region invariants."""
2 import json
3 import math
4 from pathlib import Path
5 import unittest
6 import numpy as np
7 from question2.strategy import candidate,to_world
8 from question3.domain_range import range_upper,certified,proxy
9 from question3.modules import choose_second,design_catalog
10 from question3.cases import random_case,late_source_case,boundary_case
11 from question3.interface import Client
12 from question3.simulator import Simulator
13 from question3.strategy import run
14
15
16 def config():return json.loads(Path('question3/configs/round18_dev/DOMAIN.json').read_text())
17
18
19 class DomainRangeTests(unittest.TestCase):
20     def test_range_bound_encloses_dense_boundary_rays_and_wraparound(self):
21         for first in [(0,0),(1125,0),(-1125,1e-8),(0,1800),(1800-1e-8,0)]:
22             for bearing in np.linspace(-361,361,39):
23                 upper=range_upper(first,bearing,1.005)
24                 for angle in np.radians(bearing+np.linspace(-1.005,1.005,101)):
25                     projection=first[0]*math.cos(angle)+first[1]*math.sin(angle)
26                     bound=min(1500,-projection+math.sqrt(max(0,projection**2+1800**2-math.hypot(*first)**2)))
27                     self.assertGreaterEqual(upper+1e-8,bound)
28             self.assertEqual(range_upper((1801,0),0,1.005),1500)
29             self.assertEqual(range_upper((0,0),0,1.005),1500)
30             self.assertLess(range_upper((1125,0),0,1.005),677)
31
32     def test_certified_sector_reception_and_angle_dense_extremes(self):
33         accepted=0
34         for upper in [6,20,100,675,999,1000,1001,1200,1500]:
35             offsets=[tuple(row['offset']) for row in design_catalog()]
36             offsets += [(a*upper/1500,b*upper/1500) for a,b in offsets]
37             for q in offsets:
38                 if not certified(q,upper,1.005,20):continue
39                 accepted+=1
40                 r,t=np.meshgrid(np.linspace(5,upper,61),np.radians(np.linspace(-1.005,1.005,31)),indexing='ij')
41                 g=np.stack([r*np.cos(t),r*np.sin(t)],axis=-1);v=g-q;distance=np.linalg.norm(v,axis=-1)
42                 self.assertTrue(np.all(distance<=np.maximum(1000,r)+1e-7))
43                 sine=np.abs(g[...0]*v[...1]-g[...1]*v[...0])/(r*distance)
44                 self.assertGreaterEqual(float(sine.min()),math.sin(math.radians(20))-1e-9)
45                 self.assertGreater(float(distance.min()),5)
46             self.assertGreater(accepted,100)
47
48     def test_old_invalid_offset_can_be_valid_on_domain_but_not_assumed_globally(self):
49         q=(300.,250.)
50         self.assertFalse(candidate(q,20,1.005));self.assertTrue(certified(q,676,1.005,20))
51         self.assertFalse(certified(q,1500,1.005,20))
52         for q in [(0,0),(-200,300),(500,0),(5000,5000),(float('nan'),400)]:
53             self.assertFalse(certified(q,676,1.005,20))
54
55     def test_full_range_fallback_exact_and_current_opportunity(self):
56         c=config();old={**c,'second_range_prior':'off'}
57         self.assertEqual(choose_second((0,0),45,(22,33),c),choose_second((0,0),45,(22,33),old))
58         first=(1125.,0.);current=tuple(to_world((300,250),first,0))
59         q,s=choose_second(first,0,current,c)
60         self.assertEqual(q,current);self.assertEqual(s['mode'],'domain_current')
61         self.assertGreaterEqual(s['source_range_upper_m'],675)
62
63     def test_whole_workflow_still_clears_late_and_boundary_sources(self):
64         for case in [random_case(0),late_source_case(),boundary_case()]:
65             env=Simulator(case);events=[];r=run(Client(env,'SELF'),config(),events.append)
66             self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
67             self.assertLessEqual(sum(e['type']=='schedule' for e in events),23)
68             if case['id']=='late_channel20_after_ten':self.assertIn(20,r['cleared_channels'])
69
70     def test_contradictory_domain_second_reading_never_completes(self):
71         active=False;hit=False
72         class BadClient(Client):
73             def measure(self,p,c,phase='measure'):

```

```

74         nonlocal hit
75         r=super().measure(p,c,phase)
76         if active and phase=='localization':hit=True;return {**r,'measure_result':'no_signal'}
77         return r
78     def event(e):
79         nonlocal active
80         if e['type']=='second_point_after_schedule':active=e['selection_mode'].startswith('domain_')
81         r=run(BadClient(Simulator(random_case(0))),'SELF'),config(),event)
82         self.assertTrue(hit);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
83
84     def test_invalid_configuration_before_enter(self):
85         for change in [dict(second_range_prior='bad'),dict(opportunity_timing='before_schedule'),dict(modules={})]:
86             client=Client(Simulator(random_case(0)),'SELF')
87             with self.assertRaises(ValueError):run(client,**change)
88             self.assertFalse(client.entered)

```

程序 156

对应文件: question3/tests/test_domain_shared.py

```

1  """Channel-specific public ranges, full-sector fallback and shared-workflow failures."""
2  import json
3  from pathlib import Path
4  import unittest
5  from question3.shared_measurement import choose_shared
6  from question3.cases import random_case,clustered_case,late_source_case
7  from question3.interface import Client
8  from question3.simulator import Simulator
9  from question3.strategy import run
10
11
12  def config(name='BOTH'):return json.loads(Path('question3/configs/round19_dev',name+'.json').read_text())
13
14
15  class DomainSharedTests(unittest.TestCase):
16      def test_each_channel_uses_own_first_range_and_exclusions(self):
17          first={2:((1125.,0.),0.),1:((1125.,0.),0.),3:((0.,0.),0.)};point=(1425.,250.)
18          self.assertIsNone(choose_shared(2,point,first,set(),{}),config('PRIMARY'))
19          c,s=choose_shared(2,point,first,set(),{}),config()
20          self.assertEqual(c,1);self.assertEqual(s['mode'],'domain_shared');self.assertEqual(s['source_range_upper_m'],
↳ 676)
21          self.assertIsNone(choose_shared(2,point,first,{1},{}),config())
22          self.assertIsNone(choose_shared(2,point,first,set(),{1:None}),config())
23
24      def test_old_eligibility_metadata_preserved(self):
25          first={2:((1125.,0.),0.),1:((1125.,0.),0.)};point=(1625.,500.)
26          a=choose_shared(2,point,first,set(),{}),config('PRIMARY')
27          b=choose_shared(2,point,first,set(),{}),config()
28          self.assertEqual(a,b);self.assertEqual(b[1]['mode'],'shared_certified')
29
30      def test_shared_retains_no_move_and_complete_late_source(self):
31          for case in [random_case(0),late_source_case()]:
32              env=Simulator(case);events=[];actions=[];result=run(Client(env,'SELF',actions.append),config(),events.app
↳ end)
33              self.assertEqual(result['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
34              position=(0.,0.);moves={}
35              for e in actions:
36                  if e['type']=='request' and e['path'] in ('/measure','/clear'):
37                      p=e['payload']['position'];q=(p['x'],p['y']);c=e['payload']['channel']
38                      if e['path']=='/measure' and e['phase']=='localization_shared':moves[c]=(position,q)
39                      position=q
40              for e in events:
41                  if e['type']=='localization' and e['selection']['mode']=='domain_shared':
42                      a,b=moves[e['channel']];self.assertEqual(a,b)
43              if case['id']=='late_channel20_after_ten':self.assertIn(20,result['cleared_channels'])
44
45      def test_cluster_recovers_fifteen_shared_targets(self):
46          times={}
47          for variant in ['PRIMARY','BOTH']:
48              env=Simulator(clustered_case());events=[];r=run(Client(env,'SELF'),config(variant),events.append)
49              self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])

```

```

50         times[variant]=r['metrics']['virtual_time_s']
51         shares=[e for e in events if e['type']=='shared_cache_store']
52         self.assertEqual(len(shares),15 if variant=='BOTH' else 0)
53         self.assertLess(times['BOTH'],times['PRIMARY'])
54
55     def test_failed_shared_measurement_is_incomplete(self):
56         hit=False
57         class BadClient(Client):
58             def measure(self,p,c,phase='measure'):
59                 nonlocal hit
60                 r=super().measure(p,c,phase)
61                 if phase=='localization_shared':hit=True;return {'r','measure_result':'no_signal'}
62                 return r
63         r=run(BadClient(Simulator(clustered_case()),'SELF'),config())
64         self.assertTrue(hit);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
65
66     def test_invalid_configuration_before_enter(self):
67         for change in
        ↪ [dict(shared_range_prior='wrong'),dict(shared_selection='cost_gate'),dict(modules={'A':True,'B':True})]:
68             client=Client(Simulator(random_case(0)),'SELF')
69             with self.assertRaises(ValueError):run(client,**config(),**change)
70             self.assertFalse(client.entered)

```

程序 157

对应文件: question3/tests/test_http_cli.py

```

1 import json
2 from pathlib import Path
3 import subprocess
4 import sys
5 import tempfile
6 import threading
7 import unittest
8 from question3.cases import random_case
9 from question3.simulator import Simulator
10 from question3.self_server import make_server
11
12 class HttpCliTests(unittest.TestCase):
13     def test_explicit_AB_over_real_http(self):
14         env=Simulator(random_case(0));server=make_server(env)
15         worker=threading.Thread(target=server.serve_forever,daemon=True);worker.start()
16         try:
17             with tempfile.TemporaryDirectory() as tmp:
18                 out=Path(tmp)/'AB 演练'
19                 proc=subprocess.run([sys.executable,'-m','question3.run_http','--mode','self-http',
20                                     '--url',f'http://127.0.0.1:{server.server_port}',
21                                     '--config','question3/configs/round1/AB.json','--output',str(out)],capture_output=True,timeout=30)
22                 self.assertEqual(proc.returncode,0,proc.stderr.decode('utf-8',errors='replace'))
23                 r=json.loads((out/'result.json').read_text(encoding='utf-8'))
24                 self.assertEqual(r['status'],'complete')
25                 self.assertEqual(r['config']['modules'],dict(A=True,B=True,C=False))
26                 self.assertEqual(r['metrics']['virtual_time_s'],6429.685179)
27                 self.assertEqual(env.evaluation()['remaining_channels'],[])
28                 self.assertEqual(json.loads((out/'config.json').read_text(encoding='utf-8')),r['config'])
29         finally:
30             server.shutdown();server.server_close();worker.join()
31     def test_rehearsal_requires_explicit_ui_confirmation(self):
32         with tempfile.TemporaryDirectory() as tmp:
33             out=Path(tmp)/'must_not_exist'
34             p=subprocess.run([sys.executable,'-m','question3.run_http','--mode','official-rehearsal',
35                               '--robot-id','TEST','--output',str(out)],capture_output=True,timeout=10)
36             self.assertEqual(p.returncode,2);self.assertFalse(out.exists())

```

程序 158

对应文件: question3/tests/test_innovation_http_cli.py

```

1 """Self HTTP only: preserve the old entry and replay the candidate's locked metrics."""
2 import json

```

```

3 from pathlib import Path
4 import subprocess
5 import sys
6 import tempfile
7 import threading
8 import unittest
9 from question3.cases import random_case
10 from question3.simulator import Simulator
11 from question3.self_server import make_server
12
13
14 class InnovationHttpCliTests(unittest.TestCase):
15     def test_original_and_candidate_match_frozen_metrics(self):
16         root = Path(__file__).resolve().parents[1]
17         expected = json.loads((root / 'innovation/package_smoke/bounded/summary.json').read_text(encoding='utf-8'))
18         cases = {row['variant']: row for row in expected['rows'] if row['case_id'] == 'dev_0_spatial_hash'}
19         for module, variant, extra in [
20             ('question3.run_http', 'BASE', ['--config', str(root / 'configs/recommended_omni.json')]),
21             ('question3.run_innovation_http', 'GATED_TOUR_MULTI_PARALLAX', []),
22         ]:
23             with self.subTest(variant=variant):
24                 env = Simulator(random_case(0))
25                 server = make_server(env)
26                 worker = threading.Thread(target=server.serve_forever, daemon=True)
27                 worker.start()
28                 try:
29                     with tempfile.TemporaryDirectory() as tmp:
30                         out = Path(tmp) / '自建接口结果'
31                         proc = subprocess.run([
32                             sys.executable, '-m', module, '--mode', 'self-http',
33                             '--url', f'http://127.0.0.1:{server.server_port}', '--output', str(out), *extra,
34                         ], cwd=root.parent, capture_output=True, timeout=30)
35                         self.assertEqual(proc.returncode, 0, proc.stderr.decode('utf-8', errors='replace'))
36                         result = json.loads((out / 'result.json').read_text(encoding='utf-8'))
37                         self.assertEqual(result['status'], 'complete')
38                         self.assertEqual(result['metrics'], cases[variant]['metrics'])
39                         self.assertEqual(env.evaluation()['remaining_channels'], [])
40                         self.assertTrue(result['exit_confirmed'])
41                         self.assertIsNone(result['official_total'])
42                         self.assertIsNone(result['clearance_ratio'])
43                         if variant != 'BASE':
44                             self.assertTrue((out / 'code_sha256.json').is_file())
45                     finally:
46                         server.shutdown()
47                         server.server_close()
48                         worker.join()
49
50     def test_candidate_requires_explicit_rehearsal_confirmation(self):
51         with tempfile.TemporaryDirectory() as tmp:
52             out = Path(tmp) / 'must_not_exist'
53             proc = subprocess.run([
54                 sys.executable, '-m', 'question3.run_innovation_http', '--mode', 'official-rehearsal',
55                 '--robot-id', 'TEST', '--output', str(out),
56             ], capture_output=True, timeout=10)
57             self.assertEqual(proc.returncode, 2)
58             self.assertFalse(out.exists())

```

程序 159

对应文件: question3/tests/test_lookahead.py

```

1 import json
2 import math
3 import unittest
4 from pathlib import Path
5 from question3.geometry import stations
6 from question3.lookahead import stable_min, source_region_samples
7 from question3.strategy import run
8 from question3.interface import Client
9 from question3.simulator import Simulator
10 from question3.cases import random_case, clustered_case

```

```

11
12 AB=json.loads(Path('question3/configs/round1/AB.json').read_text())
13
14 class LookaheadTests(unittest.TestCase):
15     def test_tie_fix_real_station_geometry(self):
16         ring=stations();ids=list(range(1,7))
17         old=stable_min(ids,lambda k:math.dist((0,0),ring[k]),lambda k:k)
18         new=stable_min(ids,lambda k:math.dist((0,0),ring[k]),lambda k:k,1e-7)
19         rev=stable_min(ids[::-1],lambda k:math.dist((0,0),ring[k]),lambda k:k,1e-7)
20         self.assertEqual(old,3);self.assertEqual(new,1);self.assertEqual(rev,1)
21     def test_material_distance_not_overridden(self):
22         data=[(1,1500.), (2,1499.)]
23         self.assertEqual(stable_min(data,lambda x:x[1],lambda x:x[0],1e-7)[0],2)
24     def test_old_AB_unchanged(self):
25         r=run(Client(Simulator(random_case(0)), 'SELF'),AB)
26         self.assertEqual(r['metrics']['virtual_time_s'],6429.685179)
27     def test_tie_cluster_regression(self):
28         cfg={**AB, 'routing':dict(mode='nearest',tie_tolerance_m=1e-7)}
29         r=run(Client(Simulator(clustered_case()), 'SELF'),cfg)
30         self.assertEqual(r['status'],'complete');self.assertEqual(r['metrics']['virtual_time_s'],2239.)
31     def test_all_modes_complete(self):
32         for mode,weight in (('region',0),('region',.5),('region',1),('point',1)):
33             cfg={**AB, 'routing':dict(mode=mode,lookahead_weight=weight,tie_tolerance_m=1e-7)}
34             e=Simulator(random_case(0));r=run(Client(e, 'SELF'),cfg)
35             self.assertEqual(r['status'],'complete',r['failure']);self.assertEqual(e.evaluation()['remaining_channels']
36                                     ↪ ' ',[])
37     def test_sampling_fallback_is_not_infeasibility(self):
38         p,fallback=source_region_samples((0,0),0,((500.,0.), (1400.,0.)),1.005)
39         self.assertTrue(fallback);self.assertGreater(len(p),0)
40 if __name__=='__main__':unittest.main(verbosity=2)

```

程序 160

对应文件: question3/tests/test_modules.py

```

1 import copy
2 import json
3 import math
4 import threading
5 import unittest
6 from pathlib import Path
7 import numpy as np
8 from question2.strategy import candidate
9 from question3.modules import EvidenceCover,design_catalog,choose_second
10 from question3.geometry import optical_plan
11 from question3.cases import random_case,late_source_case
12 from question3.interface import Client
13 from question3.interface import HttpTransport
14 from question3.self_server import make_server
15 from question3.simulator import Simulator
16 from question3.strategy import run
17
18 BASE=json.loads(Path('question3/configs/baseline.json').read_text())
19
20 class ModuleTests(unittest.TestCase):
21     def test_A_all_candidates_analytically_safe(self):
22         for row in design_catalog():self.assertTrue(candidate(row['offset'],20,1.005))
23     def test_A_uses_current_position(self):
24         cfg={**BASE, 'modules':dict(A=True)}
25         q1,_=choose_second((0,0),0,(800,1000),cfg)
26         q2,_=choose_second((0,0),0,(800,-1000),cfg)
27         self.assertGreater(q1[1],0);self.assertLess(q2[1],0)
28     def test_default_baseline_frozen_time(self):
29         env=Simulator(random_case(0));r=run(Client(env, 'SELF'),BASE)
30         self.assertEqual(r['metrics']['virtual_time_s'],10526.473544)
31     def test_all_combinations_same_complete_contract(self):
32         for name in ['baseline','A','B','C','AB','AC','BC','ABC']:
33             cfg=json.loads(Path(f'question3/configs/round1/{name}.json').read_text())
34             e=Simulator(late_source_case());r=run(Client(e, 'SELF'),cfg)
35             self.assertEqual(r['status'],'complete',(name,r['failure']))

```



```

36         self.assertIn(20, r['cleared_channels']); self.assertEqual(e.evaluation()['remaining_channels'], [])
37     def test_B_no_search_starvation(self):
38         cfg = {'**BASE', 'modules': dict(B=True)}; events = []
39         r = run(Client(Simulator(random_case(0)), 'SELF'), cfg, events.append)
40         scans = 0; chain = 0
41         for e in events:
42             if e['type'] != 'schedule': continue
43             if e['task'] == 'scan': scans += 1; chain = 0
44             else:
45                 chain += 1
46                 if scans < 7: self.assertLessEqual(chain, 2)
47         self.assertEqual(scans, 7); self.assertEqual(r['status'], 'complete')
48     def test_C_retains_true_cell_until_success(self):
49         g = np.array([1499., 20.]); s = (0, 0); q = (800, 605); reading = math.degrees(math.atan2(g[1] - q[1], g[0] - q[0])) + .8
50         points, cert = optical_plan(s, 0, q, reading, 1.005, 28, 2000)
51         planner = EvidenceCover(points, cert, s, 0, q, reading, [(0, -1000)], 1.005)
52         contains = np.all(g >= planner.corners.min(axis=1) - 1e-8, axis=1) & np.all(g <= planner.corners.max(axis=1) + 1e-8, axis=1)
53         self.assertTrue(np.any(contains & planner.alive)); current = q; success = False
54         for _ in points:
55             k, p, _ = planner.next_point(current)
56             if math.dist(p, g) <= 20: success = True; break
57             planner.failed(k); current = p
58         self.assertTrue(np.any(contains & planner.alive))
59         self.assertTrue(success)
60     def test_C_whole_cell_cover_not_centre_heuristic(self):
61         g = np.array([1499., 20.]); q = (800, 605); reading = math.degrees(math.atan2(g[1] - q[1], g[0] - q[0]))
62         points, cert = optical_plan((0, 0), 0, q, reading, 1.005, 28, 2000)
63         planner = EvidenceCover(points, cert, (0, 0), 0, q, reading, [], 1.005)
64         for k in range(len(points)):
65             idx = planner.covers[k]
66             self.assertTrue(np.all(np.linalg.norm(planner.corners[idx] - points[k], axis=2) <= 20))
67             self.assertTrue(np.all(planner.covers.any(axis=0)))
68     def test_combined_budget_failure_not_complete(self):
69         cfg = {'**BASE', 'modules': dict(A=True, B=True, C=True)}
70         result = run(Client(Simulator(random_case(0), remaining_s=0), 'SELF'), cfg)
71         self.assertEqual(result['status'], 'incomplete'); self.assertFalse(result['exit_confirmed'])
72     def test_ABC_over_real_loopback_without_truth_fields(self):
73         cfg = {'**BASE', 'modules': dict(A=True, B=True, C=True)}
74         env = Simulator(late_source_case()); server = make_server(env)
75         thread = threading.Thread(target=server.serve_forever, daemon=True); thread.start()
76         try:
77             result = run(Client(HttpTransport(f'http://127.0.0.1:{server.server_port}'), 'SELF'), cfg)
78             self.assertEqual(result['status'], 'complete')
79             self.assertEqual(env.evaluation()['remaining_channels'], [])
80         finally: server.shutdown(); server.server_close(); thread.join()
81
82 if __name__ == '__main__': unittest.main(verbosity=2)

```

程序 161

对应文件: question3/tests/test_nearest_finite.py

```

1  """Remove the counter constraint while preserving the complete choice set."""
2  import json
3  from pathlib import Path
4  import unittest
5  from unittest.mock import patch
6  from question3.batch_scheduling import gate
7  from question3.lookahead import decide
8  from question3.cases import random_case, late_source_case, boundary_case
9  from question3.interface import Client
10 from question3.simulator import Simulator
11 from question3.strategy import run
12
13
14 def config(): return json.loads(Path('question3/configs/round16_dev/FREE.json').read_text())
15
16
17 class NearestFiniteTests(unittest.TestCase):
18     def test_no_filter_or_forcing_even_with_long_counter(self):
19         jobs = [(1, 2, (0, 0), {})]; remaining = [1, 2]

```

```

20     a,b,forced,info=gate('nearest_finite',[2],[2:None],jobs,remaining,100)
21     self.assertIs(a,jobs);self.assertIs(b,remaining);self.assertFalse(forced);self.assertIsNone(info)
22
23     def test_controller_receives_all_options_and_no_forcing(self):
24         seen=[]
25         def spy(current,channel,cover,remaining,jobs,first,negative,unknown,c,forced=False):
26             self.assertFalse(forced);seen.append((len(remaining),len(jobs)))
27             return decide(current,channel,cover,remaining,jobs,first,negative,unknown,c,forced)
28         with patch('question3.strategy.decide',side_effect=spy):
29             r=run(Client(Simulator(random_case(0)),'SELF'),config())
30             self.assertEqual(r['status'],'complete');self.assertTrue(any(a and b for a,b in seen))
31
32     def test_finite_distinct_tasks_and_complete_stop_including_late_channel(self):
33         for case in [random_case(0),late_source_case(),boundary_case()]:
34             es=[];env=Simulator(case);r=run(Client(env,'SELF'),config(),es.append)
35             self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
36             tasks=[e for e in es if e['type']=='schedule'];scans=[e['station_id'] for e in tasks if e['task']=='scan']
37             locs=[e['channel'] for e in tasks if e['task']=='localize']
38             self.assertEqual(len(scans),len(set(scans)));self.assertLessEqual(len(scans),7)
39             self.assertEqual(len(locs),len(set(locs)));self.assertLessEqual(len(locs),16)
40             self.assertLessEqual(len(tasks),23);self.assertFalse(any(e['forced'] for e in tasks))
41             if case['id']=='late_channel20_after_ten':
42                 self.assertIn(20,r['cleared_channels']);self.assertEqual(r['stopping_certificate']['method'],'seven_s
↳ tation_channel_cover')
43
44     def test_deadline_during_extended_localization_never_completes(self):
45         client=Client(Simulator(random_case(0)),'SELF');selected=[]
46         def event(e):
47             if e['type']=='schedule' and e['task']=='localize':
48                 selected.append(e)
49                 if len(selected)==3:client.deadline=0
50         r=run(client,config(),event)
51         self.assertEqual(len(selected),3);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirm']
↳ ed'])

```

程序 162

对应文件: question3/tests/test_opportunity_timing.py

```

1 import json
2 from pathlib import Path
3 import unittest
4 from unittest.mock import patch
5 from question3.cases import random_case,late_source_case
6 from question3.interface import Client
7 from question3.simulator import Simulator
8 from question3.strategy import run
9 from question3.lookahead import decide
10 from question3.modules import choose_second
11
12
13 def cfg(n):return json.loads(Path('question3/configs/round9_dev',n+'.json').read_text())
14
15
16 class OpportunityTimingTests(unittest.TestCase):
17     def test_scheduler_sees_only_original_catalog_jobs(self):
18         calls=[];config=cfg('ZL');events=[]
19         def spy(current,channel,cover,remaining,jobs,first,negative,unknown,c,forced=False):
20             for _,source,q,selection in jobs:
21                 if selection['mode']=='shared_ready':continue
22                 original,_=choose_second(*first[source],current,**c,'second_point_opportunity':'off'})
23                 self.assertEqual(q,original);self.assertEqual(selection['mode'],'cost_proxy')
24                 result=decide(current,channel,cover,remaining,jobs,first,negative,unknown,c,forced)
25                 calls.append(result);return result
26         with patch('question3.strategy.decide',side_effect=spy):
27             r=run(Client(Simulator(random_case(0)),'SELF'),config,events.append)
28             self.assertTrue(calls);self.assertEqual(r['status'],'complete')
29             last=None;checks=0
30             for e in events:
31                 if e['type']=='schedule':last=e
32                 if e['type']=='second_point_after_schedule':

```

```

33         self.assertEqual(last['task'],'localize');self.assertEqual(last['channel'],e['channel']);checks+=1
34     self.assertGreater(checks,0)
35
36     def test_cached_jobs_not_remeasured_and_late_source_found(self):
37         for n in ['JL','ZL']:
38             es=[];req=[];env=Simulator(late_source_case());r=run(Client(env,'SELF',req.append),cfg(n),es.append)
39             self.assertEqual(r['status'],'complete');self.assertIn(20,r['cleared_channels'])
40             cached={e['channel'] for e in es if e['type']=='shared_cache_store'}
41             late={e['channel'] for e in es if e['type']=='second_point_after_schedule'}
42             self.assertFalse(cached & late)
43             for c in cached:
44                 measurements=[e for e in req if e['type']=='request' and e['path']=='/measure'
45                             and e['payload']['channel']==c and e['phase'].startswith('localization')]
46             self.assertEqual(len(measurements),1)
47
48     def test_deadline_between_schedule_and_execution_is_incomplete(self):
49         client=Client(Simulator(random_case(0)),'SELF');seen=[]
50         def event(e):
51             if e['type']=='second_point_after_schedule':seen.append(e);client.deadline=0
52         r=run(client,cfg('ZL'),event)
53         self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
54
55     def test_no_op_and_invalid_timing(self):
56         c=cfg('REF');base=run(Client(Simulator(random_case(0)),'SELF'),c)
57         c['opportunity_timing']='after_schedule'
58         actual=run(Client(Simulator(random_case(0)),'SELF'),c)
59         self.assertEqual(base,actual)
60         c['opportunity_timing']='invalid';client=Client(Simulator(random_case(0)),'SELF')
61         with self.assertRaises(ValueError):run(client,c)
62         self.assertFalse(client.entered)

```

程序 163

对应文件: question3/tests/test_optical_order.py

```

1 import json
2 import math
3 from pathlib import Path
4 import unittest
5 from question3.optical_order import nearer_end
6 from question3.cases import random_case,late_source_case
7 from question3.interface import Client
8 from question3.simulator import Simulator
9 from question3.strategy import run
10
11
12 def cfg(name='V'):
13     return json.loads(Path('question3/configs/round6_factorial',name+'.json').read_text())
14
15
16 class OpticalOrderTests(unittest.TestCase):
17     def test_same_cover_and_nonincreasing_full_traversal(self):
18         p=[(10,0),(10,20),(0,20),(0,0)]
19         for current in [(0,-1),(20,0),(5,0),(100,90)]:
20             q=nearer_end(p,current)
21             self.assertEqual(sorted(q),sorted(p))
22             length=lambda points:math.dist(current,points[0])+sum(math.dist(a,b) for a,b in zip(points,points[1:]))
23             self.assertEqual(length(q),length(p)+1e-9)
24         self.assertIs(nearer_end(p,(5,0)),p)
25
26     def test_order_events_match_actual_first_clear_including_cache(self):
27         for name in ['V','IV']:
28             merged=[];env=Simulator(random_case(0));r=run(Client(env,'SELF',merged.append),cfg(name),merged.append)
29             self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
30             first_clears=set();entry={};cache_used=set();reversed_count=0
31             for e in merged:
32                 if e['type']=='optical_path_order':
33                     entry[e['channel']]=e['selected_entry'];reversed_count+=e['reversed']
34                     self.assertGreaterEqual(e['entry_distance_saved_m'],-1e-8)
35                 if e['type']=='shared_cache_use':
36                     cache_used.add(e['channel']);self.assertEqual(entry[e['channel']],e['optical_entry'])

```

```

37         if e['type']=='request' and e['path']=='/clear' and e['phase']=='optical':
38             c=e['payload']['channel']
39             if c not in first_clears:
40                 p=e['payload']['position'];self.assertEqual(tuple(entry[c]),(p['x'],p['y']));first_clears.add_
                     ↪ (c)
41         self.assertTrue(cache_used);self.assertGreater(reversed_count,0)
42
43     def test_reversed_path_exhaustion_is_incomplete(self):
44         class BadClient(Client):
45             def clear(self,p,c,phase='clear'):
46                 r=super().clear(p,c,phase)
47                 return {'r','clear_result':'no_target_in_range'} if phase=='optical' else r
48         r=run(BadClient(Simulator(random_case(0))),'SELF'),cfg())
49         self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
50         self.assertIn('All optical covering points failed',r['failure'])
51
52     def test_late_channel_and_incompatible_evidence(self):
53         result=run(Client(Simulator(late_source_case()),'SELF'),cfg('IV'))
54         self.assertEqual(result['status'],'complete');self.assertIn(20,result['cleared_channels'])
55         c=cfg();c['modules']['C']=True;client=Client(Simulator(random_case(0)),'SELF')
56         with self.assertRaises(ValueError):run(client,c)
57         self.assertFalse(client.entered)

```

程序 164

对应文件: question3/tests/test_optical_shortcut.py

```

1 import math
2 import unittest
3 import numpy as np
4 from question3.optical_shortcut import single_circle
5
6 class SingleCircleTests(unittest.TestCase):
7     def test_diameter_circle_counterexample(self):
8         # D=38 <40 is insufficient: the equilateral circumradius is >20.
9         self.assertIsNone(single_circle([[0,0],[38,0],[19,19*math.sqrt(3)]]))
10    def test_single_circle_covers_all_convex_combinations(self):
11        v=np.array([[0,0],[30,0],[15,15*math.sqrt(3)]])
12        p,r=single_circle(v);self.assertLess(r,20)
13        rng=np.random.default_rng(11);w=rng.dirichlet([1,1,1],1000)
14        self.assertLessEqual(np.linalg.norm(w@v-p,axis=1).max(),r+1e-8)
15    def test_boundary_has_margin_and_fallback(self):
16        self.assertIsNone(single_circle([[20,0],[20,0]]))
17        self.assertIsNotNone(single_circle([[-19.999,0],[19.999,0]]))
18    def test_collinear_and_translated(self):
19        p,r=single_circle([[1000,1000],[1010,1000],[1020,1000]])
20        self.assertAlmostEqual(r,10);self.assertEqual(p,(1010.,1000.))

```

程序 165

对应文件: question3/tests/test_origin_ring.py

```

1 import json
2 from pathlib import Path
3 import unittest
4 from question3.cases import clustered_case,random_case,boundary_case
5 from question3.interface import Client
6 from question3.simulator import Simulator
7 from question3.strategy import run
8
9
10 def cfg(name):return json.loads(Path('question3/configs/round4_adapt',name+'.json').read_text())
11
12 class OriginRingTests(unittest.TestCase):
13     def test_cluster_and_outer_ring_fallback(self):
14         for case in [clustered_case(),boundary_case()]:
15             reference=run(Client(Simulator(case),'SELF'),cfg('ABS'))
16             events=[];e=Simulator(case);actual=run(Client(e,'SELF'),cfg('ABSR1125E'),events.append)
17             self.assertEqual(actual['metrics'],reference['metrics'])

```

```

18         self.assertEqual(actual['status'],'complete');self.assertTrue(any(x['type']=='search_ring_switch' for x
    ↪ in events))
19         self.assertEqual(actual['station_positions'][1],[1500.,0.] if
    ↪ isinstance(actual['station_positions'][1],list) else (1500.,0.))
20     def test_positive_origin_keeps_compact_ring(self):
21         case=random_case(0);ref=run(Client(Simulator(case),'SELF'),cfg('ABSR1125'))
22         es=[];r=run(Client(Simulator(case),'SELF'),cfg('ABSR1125E'),es.append)
23         self.assertEqual(r['metrics'],ref['metrics']);self.assertFalse(any(x['type']=='search_ring_switch' for x in
    ↪ es))
24     def test_fallback_still_needs_coverage_certificate(self):
25         c=cfg('ABSR1125E');c['empty_origin_radius_m']=1100
26         client=Client(Simulator(random_case(0)),'SELF')
27         with self.assertRaises(ValueError):run(client,c)
28         self.assertFalse(client.entered)

```

程序 166

对应文件: question3/tests/test_radius_priors.py

```

1  """Published containing-circle bounds, independent from the unknown actual radius."""
2  import json
3  import tempfile
4  from pathlib import Path
5  import unittest
6  from question3.clear_skip import radius_failure_certificate
7  from question3.cases import random_case,late_source_case,clustered_case
8  from question3.interface import Client,JsonLog
9  from question3.simulator import Simulator
10 from question3.strategy import run
11 from question3.audit import audit_folder
12
13
14 def config(name='BOTH'):
15     return json.loads(Path('question3/configs/round13_dev',name+'.json').read_text())
16
17
18 class RadiusPriorTests(unittest.TestCase):
19     def test_domain_tangent_and_uncertain_boundary_are_retained(self):
20         for x in [0,1800,1820,1820+1e-6]:
21             self.assertIsNone(radius_failure_certificate((x,0),(0,0),(100,0),'domain'))
22             self.assertEqual(radius_failure_certificate((1820.001,0),(0,0),(100,0),'domain')['radius_prior'],'domain')
23
24     def test_received_upper_bound_is_1500_not_minimum_radius(self):
25         for x in [1100,1500,1520,1520+1e-6]:
26             self.assertIsNone(radius_failure_certificate((x,0),(0,0),(0,0),'received'))
27             proof=radius_failure_certificate((1520.001,0),(0,0),(0,0),'received')
28             self.assertEqual(proof['containing_radius_m'],1500)
29
30     def test_second_received_circle_and_mode_isolation(self):
31         p=(0,0);first=(0,0);second=(1600,0)
32         self.assertIsNone(radius_failure_certificate(p,first,second,'domain'))
33         self.assertIsNone(radius_failure_certificate(p,first,second,'off'))
34         self.assertEqual(radius_failure_certificate(p,first,second,'received')['radius_prior'],'second_received')
35
36     def test_actual_skip_modes_keep_complete_clearance_and_measurements(self):
37         for case in [random_case(0,'negative'),random_case(1,'smooth'),late_source_case(),clustered_case()]:
38             trajectories=[];times=[];skipped=0
39             for name in ['REF','DOMAIN','RANGE','BOTH']:
40                 records=[];events=[];env=Simulator(case)
41                 r=run(Client(env,'SELF',records.append),config(name),events.append)
42                 self.assertEqual(r['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
43                 req={e['payload']['request_id']:e for e in records if e['type']=='request'};trace=[]
44                 for e in records:
45                     if e['type']!='response':continue
46                     q=req[e['request_id']]
47                     if q['path']=='/measure' or e['response'].get('clear_result')=='success':
48                         trace.append((q['path'],q['payload'].get('channel'),q['payload'].get('position')))
49                     trajectories.append(trace);times.append(r['metrics']['virtual_time_s'])
50                     skipped+=sum(e['type']=='certified_clear_skip' and 'radius_prior' in e for e in events)
51                 self.assertTrue(all(t==trajectories[0] for t in trajectories))
52                 self.assertTrue(all(t<=times[0]+1e-5 for t in times))

```

```

53         if case['id'].startswith('dev_'):self.assertGreater(skipped,0)
54
55     def test_deadline_after_radius_skip_is_incomplete(self):
56         client=Client(Simulator(random_case(0,'negative')), 'SELF');seen=[]
57         def event(e):
58             if e['type']=='certified_clear_skip' and 'radius_prior' in e:seen.append(e);client.deadline=0
59         r=run(client,config(),event)
60         self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
61
62     def test_forged_smaller_radius_rejected_by_independent_audit(self):
63         with tempfile.TemporaryDirectory() as temp:
64             folder=Path(temp);case=random_case(0,'negative')
65             actions=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
66             r=run(Client(Simulator(case), 'SELF',actions),config(),events);actions.close();events.close()
67             (folder/'evaluator_hidden_case.jsonl').write_text(json.dumps(case))
68             (folder/'result.jsonl').write_text(json.dumps(dict(summary=dict(rounding='bounded'),strategy=r)))
69             self.assertGreater(audit_folder(folder),0)
70             rows=list(map(json.loads,(folder/'strategy.jsonl').read_text().splitlines()))
71             e=next(e for e in rows if e['type']=='certified_clear_skip' and 'radius_prior' in e)
72             e['containing_radius_m']-=100
73             (folder/'strategy.jsonl').write_text('\n'.join(map(json.dumps,rows))+'\n')
74             with self.assertRaises(AssertionError):audit_folder(folder)
75
76     def test_invalid_or_disabled_parent_rejected_before_enter(self):
77         for c in [{"**config(),'clear_radius_priors':'unknown'},{"**config(),'clear_skip':'off'}]:
78             client=Client(Simulator(random_case(0)), 'SELF')
79             with self.assertRaises(ValueError):run(client,c)
80             self.assertFalse(client.entered)

```

程序 167

对应文件: question3/tests/test_ring_phase.py

```

1  """Rotation invariant coverage, once-only evidence handling, and failure paths."""
2  import json
3  import math
4  from pathlib import Path
5  import tempfile
6  import unittest
7  import numpy as np
8  from question3.geometry import stations,station_cover_bound
9  from question3.ring_phase import align_first_entry
10 from question3.cases import random_case,late_source_case,clustered_case,boundary_case
11 from question3.interface import Client,JsonLog
12 from question3.simulator import Simulator
13 from question3.strategy import run
14 from question3.audit import audit_folder
15
16
17 def config(name='PHASE'):
18     return json.loads(Path('question3/configs/round14_dev',name+'.json').read_text())
19
20
21 class RingPhaseTests(unittest.TestCase):
22     def test_radial_alignment_attains_continuous_entry_minimum(self):
23         for p in [(0.,0.), (984.,-462.), (-100.,-2000.), (1125.,0.)]:
24             for i in range(1,7):
25                 cover,phase=align_first_entry(stations(1125),i,p)
26                 self.assertAlmostEqual(math.dist(p,cover[i]),abs(math.hypot(*p)-1125),places=8)
27                 self.assertEqual(cover[0],(0.,0.));self.assertTrue(math.isfinite(phase))
28
29     def test_rotated_boundary_coverage_retains_strict_margin(self):
30         bound=station_cover_bound(1125);self.assertLess(bound,1000)
31         for phase in [-8.7,-.31,0,1.73,12.4]:
32             cover=np.array(stations(1125,phase))
33             for radius in [0,900,1800]:
34                 for theta in np.linspace(0,2*math.pi,721)+phase:
35                     p=radius*np.array([math.cos(theta),math.sin(theta)])
36                     self.assertLessEqual(np.linalg.norm(cover-p,axis=1).min(),bound+1e-7)
37
38     def test_shift_occurs_once_before_any_outer_search_evidence(self):

```

```

39     records=[];events=[];client=Client(Simulator(random_case(0)),'SELF',records.append)
40     def event(e):
41         events.append(e)
42         if e['type']=='ring_phase_selected':
43             self.assertEqual(e['visited_before'],[0])
44             self.assertEqual(events[-2]['type'],'schedule');self.assertEqual(events[-2]['task'],'scan')
45             for r in records:
46                 if r['type']=='request' and r['path']=='/measure' and r['phase']=='search':
47                     self.assertEqual(r['payload']['position'],dict(x=0.,y=0.))
48             result=run(client,config(),event)
49             self.assertEqual(result['status'],'complete')
50             self.assertEqual(sum(e['type']=='ring_phase_selected' for e in events),1)
51             cover={tuple(p) for p in result['station_positions']}
52             for r in records:
53                 if r['type']=='request' and r['path']=='/measure' and r['phase']=='search':
54                     self.assertIn(tuple(r['payload']['position'][k] for k in ['x','y']),cover)
55
56     def test_late_channel_still_requires_complete_certificate(self):
57         r=run(Client(Simulator(late_source_case()),'SELF'),config())
58         self.assertEqual(r['status'],'complete');self.assertIn(20,r['cleared_channels'])
59         self.assertEqual(r['stopping_certificate']['method'],'seven_station_channel_cover')
60
61     def test_no_outer_scan_does_not_invent_phase_event(self):
62         case=dict(id='all_near_16',seed=14,error_mode='spatial_hash',
63                 sources=[dict(channel=c,x=c/10,y=0,radius=1000) for c in range(1,17)])
64         events=[];r=run(Client(Simulator(case),'SELF'),config(),events.append)
65         self.assertEqual(r['status'],'complete')
66         self.assertFalse(any(e['type']=='ring_phase_selected' for e in
67                               ↪ events));self.assertNotIn('station_phase_rad',r)
68
69     def test_deadline_after_phase_selection_is_incomplete(self):
70         client=Client(Simulator(random_case(0)),'SELF');seen=[]
71         def event(e):
72             if e['type']=='ring_phase_selected':seen.append(e);client.deadline=0
73         r=run(client,config(),event)
74         self.assertTrue(seen);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
75
76     def test_empty_origin_makes_the_entire_policy_a_no_op(self):
77         for case in [clustered_case(),boundary_case()]:
78             a=run(Client(Simulator(case),'SELF'),config('REF'))
79             events=[];b=run(Client(Simulator(case),'SELF'),config(),events.append)
80             phases=[e for e in events if e['type']=='ring_phase_selected']
81             self.assertEqual(len(phases),1);self.assertEqual(phases[0]['phase_rad'],0)
82             self.assertEqual(phases[0]['position'],(0.,0.))
83             self.assertEqual(a['metrics'],b['metrics']);self.assertEqual(a['station_positions'],b['station_positions']
84                               ↪ )
85
86     def test_auditor_rejects_forged_station_phase(self):
87         with tempfile.TemporaryDirectory() as temp:
88             folder=Path(temp);case=random_case(0)
89             actions=JsonLog(folder/'actions.jsonl');events=JsonLog(folder/'strategy.jsonl')
90             r=run(Client(Simulator(case),'SELF',actions),config(),events);actions.close();events.close()
91             (folder/'evaluator_hidden_case.jsonl').write_text(json.dumps(case))
92             document=dict(summary=dict(rounding='bounded'),strategy=r)
93             path=folder/'result.jsonl';path.write_text(json.dumps(document));self.assertGreater(audit_folder(folder),0)
94             r['station_phase_rad']+=.1;path.write_text(json.dumps(document))
95             with self.assertRaises(AssertionError):audit_folder(folder)
96
97     def test_invalid_phase_mode_or_nonjoint_before_enter(self):
98         for c in [{**config(),'ring_phase':'always'},{**config(),'modules':{**config()['modules'],'B':False}}]:
99             client=Client(Simulator(random_case(0)),'SELF')
100             with self.assertRaises(ValueError):run(client,c)
101             self.assertFalse(client.entered)

```

程序 168

对应文件: question3/tests/test_round3_modules.py

```

1 import copy
2 import json
3 import math

```

```

4 from pathlib import Path
5 import threading
6 import unittest
7 import numpy as np
8 from question3.cases import random_case,late_source_case
9 from question3.interface import Client,HttpTransport
10 from question3.simulator import Simulator
11 from question3.self_server import make_server
12 from question3.strategy import run
13 from question3.shared_measurement import compatible_offset,choose_shared
14 from question3.order_routing import incremental_regrets
15
16
17 def cfg(name):return json.loads(Path('question3/configs/round3_factorial',name+'.json').read_text())
18
19
20 class Round3Tests(unittest.TestCase):
21     def test_shared_geometric_membership_and_exclusion(self):
22         config=cfg('ABS');first={1:((0,0),0),2:((0,0),0),3:((0,0),0)}
23         self.assertIsNotNone(compatible_offset((0,0),0,(800,605),1.005,20))
24         self.assertIsNone(compatible_offset((0,0),0,(700,0),1.005,20))
25         c,s=choose_shared(1,(800,605),first,set(),{2:True},config);self.assertEqual(c,3)
26         self.assertIsNone(choose_shared(1,(800,605),first,{2},{3:True},config))
27     def test_regret_matches_two_explicit_routes(self):
28         x=(12,44);station=(700,100);q=(400,300);anchor=(-400,600);g=np.array([[1,2],[500,550]])
29         rj,rs,d=incremental_regrets(x,station,q,g,anchor)
30         direct=[]
31         for z in g:
32             jf=math.dist(x,q)+math.dist(q,z)+math.dist(z,station)+math.dist(station,anchor)
33             sf=math.dist(x,station)+math.dist(station,q)+math.dist(q,z)+math.dist(z,anchor)
34             direct.append((sf-jf)/5)
35         np.testing.assert_allclose(d,direct);self.assertEqual(rj,max(0,-min(direct)));self.assertEqual(rs,
↪         ,max(0,max(direct)))
36     def test_all_eight_complete_and_late_source(self):
37         for path in sorted(Path('question3/configs/round3_factorial').glob('*.json')):
38             env=Simulator(late_source_case());r=run(Client(env,'SELF'),cfg(path.stem))
39             self.assertEqual(r['status'],'complete',(path.stem,r['failure']));self.assertIn(20,r['cleared_channels'])
40             self.assertEqual(env.evaluation()['remaining_channels'],[])
41     def test_shared_cached_once_no_extra_remeasure(self):
42         events=[];requests=[];env=Simulator(random_case(0))
43         r=run(Client(env,'SELF',requests.append),cfg('ABSOD'),events.append)
44         stored=[e['channel'] for e in events if e['type']=='shared_cache_store'];used=[e['channel'] for e in events
↪         if e['type']=='shared_cache_use']
45         self.assertGreater(len(stored),0);self.assertEqual(sorted(stored),sorted(used));self.assertEqual(len(stored),
↪         len(set(stored)))
46         for c in stored:
47             actions=[e for e in requests if e['type']=='request' and e['path']=='/measure' and
↪             e['payload']['channel']==c and e['phase'].startswith('localization')]
48             self.assertEqual(len(actions),1);self.assertEqual(actions[0]['phase'],'localization_shared')
49             self.assertEqual(r['status'],'complete')
50     def test_search_cannot_be_starved_by_cached_jobs(self):
51         events=[];r=run(Client(Simulator(random_case(0)),'SELF'),cfg('ABSOD'),events.append)
52         scans=chain=0
53         for e in events:
54             if e['type']!='schedule':continue
55             if e['task']=='scan':scans+=1;chain=0
56             else:
57                 chain+=1
58                 if scans<7:self.assertLessEqual(chain,2)
59             self.assertEqual(scans,7);self.assertEqual(r['status'],'complete')
60     def test_no_signal_on_shared_measure_is_incomplete(self):
61         class BadClient(Client):
62             def measure(self,p,c,phase='measure'):
63                 r=super().measure(p,c,phase)
64                 if phase=='localization_shared':r={'**r','measure_result':'no_signal'}
65                 return r
66         r=run(BadClient(Simulator(random_case(0)),'SELF'),cfg('ABSOD'))
67         self.assertEqual(r['status'],'incomplete');self.assertIn('Guaranteed-reception',r['failure']);self.assertFals
↪         e(r['exit_confirmed'])
68     def test_deadline_after_cache_is_not_complete(self):
69         client=Client(Simulator(random_case(0)),'SELF');stored=[]
70         def event(e):
71             if e['type']=='shared_cache_store':stored.append(e);client.deadline=0

```



```

72     r=run(client,cfg('ABSOD'),event)
73     self.assertTrue(stored);self.assertEqual(r['status'],'incomplete');self.assertFalse(r['exit_confirmed'])
74 def test_shared_response_retry_is_idempotent(self):
75     env=Simulator(random_case(0));state={'phase':None,'lost':False}
76     class DropOnce:
77         def request(self,path,body,timeout):
78             status,r=env.request(path,body,timeout)
79             if not state['lost'] and path=='/measure' and state['phase']=='localization_shared':
80                 state['lost']=True;raise OSError('accepted shared response lost')
81             return status,r
82     def log(e):
83         if e['type']=='request':state['phase']=e['phase']
84     r=run(Client(DropOnce(),'SELF',log),cfg('ABSOD'))
85     self.assertTrue(state['lost']);self.assertEqual(r['status'],'complete');self.assertEqual(r['metrics']['counts']
↪     'retries'],1)
86 def test_legacy_incompatible_config_rejected_before_enter(self):
87     config=cfg('ABSOD');config['routing']={'mode':'region'};env=Simulator(random_case(0))
88     with self.assertRaises(ValueError):run(Client(env,'SELF'),config)
89     self.assertEqual(env.evaluation()['virtual_time_s'],0)
90 def test_ABSOD_real_http(self):
91     env=Simulator(random_case(0));server=make_server(env);thread=threading.Thread(target=server.serve_forever,dae
↪     mon=True);thread.start()
92     try:
93         r=run(Client(HttpTransport(f'http://127.0.0.1:{server.server_port}'),'SELF'),cfg('ABSOD'))
94         self.assertEqual(r['status'],'complete');self.assertEqual(env.evaluation()['remaining_channels'],[])
95     finally:server.shutdown();server.server_close();thread.join()

```

程序 169

对应文件: question3/tests/test_round4.py

```

1 import json
2 import math
3 from pathlib import Path
4 import threading
5 import unittest
6 import numpy as np
7 from question3.geometry import stations,station_cover_bound
8 from question3.strategy import run
9 from question3.interface import Client,HttpTransport
10 from question3.simulator import Simulator
11 from question3.cases import random_case
12 from question3.shared_measurement import choose_shared
13 from question3.self_server import make_server
14
15
16 def cfg(name):return json.loads(Path('question3/configs/round4_dev',name+'.json').read_text())
17
18
19 class Round4Tests(unittest.TestCase):
20     def test_compressed_rings_continuous_bound_against_grid(self):
21         r,a=np.meshgrid(np.linspace(0,1800,41),np.linspace(0,2*math.pi,721))
22         points=np.column_stack([(r*np.cos(a)).ravel(),(r*np.sin(a)).ravel()])
23         for rho in [1125,1200,1500]:
24             bound=station_cover_bound(rho)
25             d=np.linalg.norm(points[:,None,:]-np.asarray(stations(rho))[None,:],axis=2).min(axis=1)
26             self.assertLess(bound,1000);self.assertEqual(float(d.max()),bound+1e-8)
27     def test_boundary_midpoint_is_close_to_guarantee(self):
28         p=(1800*math.cos(math.pi/6),1800*math.sin(math.pi/6))
29         d=min(math.dist(p,q) for q in stations(1125))
30         self.assertAlmostEqual(d,999.1106618753616);self.assertLess(d,1000)
31     def test_unsafe_ring_rejected_before_enter(self):
32         for rho in [1100,-1,float('nan')]:
33             c=cfg('AB');c['search_radius_m']=rho;client=Client(Simulator(random_case(0)),'SELF')
34             with self.assertRaises(ValueError):run(client,c)
35             self.assertFalse(client.entered)
36     def test_shared_gate_can_reject_a_geometrically_legal_point(self):
37         first=[1:((0,0),0),2:((0,0),0)];points=[(0,0),(100,0)]
38         old=choose_shared(1,(400,500),first,set(),{ },cfg('ABS'),points)
39         gated=choose_shared(1,(400,500),first,set(),{ },cfg('ABSG'),points)
40         self.assertIsNotNone(old);self.assertIsNone(gated)

```

```

41         self.assertIsNotNone(choose_shared(1, (800, 605), first, set(), {}, cfg('ABSG'), points))
42     def test_adversarial_outer_midpoints_complete(self):
43         sources=[]
44         for c in range(1,13):
45             a=(c%6+.5)*math.pi/3;sources.append(dict(channel=c,x=1800*math.cos(a),y=1800*math.sin(a),radius=1000.))
46         case=dict(id='unit_outer_midpoints',seed=91,error_mode='positive',sources=sources)
47         for name in ['ABR1125','ABSR1125G']:
48             env=Simulator(case);r=run(Client(env,'SELF'),cfg(name))
49             self.assertEqual(r['status'],'complete',r['failure']);self.assertEqual(env.evaluation()['remaining_channels'],
↳ ls'),[])
50     def test_all_dev_configs_complete(self):
51         for path in Path('question3/configs/round4_dev').glob('*.json'):
52             env=Simulator(random_case(0));r=run(Client(env,'SELF'),cfg(path.stem))
53             self.assertEqual(r['status'],'complete',(path.stem,r['failure']));self.assertEqual(env.evaluation()['remaining_channels'],
↳ ing_channels'],[])
54     def test_compressed_gate_over_real_http(self):
55         env=Simulator(random_case(0));server=make_server(env);thread=threading.Thread(target=server.serve_forever,daemon=True);thread.start()
56         try:
57             r=run(Client(HttpTransport(f'http://127.0.0.1:{server.server_port}'),'SELF'),cfg('ABSR1125G'))
58             self.assertEqual(r['status'],'complete');self.assertEqual(env.evaluation()['remaining_channels'],[])
59         finally:server.shutdown();server.server_close();thread.join()

```

程序 170

对应文件: question3/tests/test_scan_route.py

```

1  """Verify the finite path proxy independently and preserve complete workflow."""
2  import itertools
3  import json
4  import math
5  from pathlib import Path
6  import random
7  import unittest
8  from question3.scan_route import remaining_open_path
9  from question3.lookahead import decide
10 from question3.geometry import stations
11 from question3.cases import random_case,late_source_case
12 from question3.interface import Client
13 from question3.simulator import Simulator
14 from question3.strategy import run
15
16
17 def config():return json.loads(Path('question3/configs/round17_dev/ROUTE.json').read_text())
18
19
20 def cost(p,cover,path):
21     total=0.
22     for k in path:total+=math.dist(p,cover[k]);p=cover[k]
23     return total
24
25
26 class ScanRouteTests(unittest.TestCase):
27     def test_exact_cost_against_exhaustive_permutations(self):
28         rng=random.Random(831);cover=stations(1125)
29         for n in range(8):
30             ids=rng.sample(range(7),n);p=(rng.uniform(-1800,1800),rng.uniform(-1800,1800))
31             route,d=remaining_open_path(p,cover,ids)
32             self.assertEqual(set(route),set(ids));self.assertEqual(len(route),n)
33             self.assertAlmostEqual(d,cost(p,cover,route),places=8)
34             self.assertAlmostEqual(d,min(cost(p,cover,x) for x in itertools.permutations(ids)),places=8)
35
36     def test_greedy_counterexample_and_deterministic_tie(self):
37         route,d=remaining_open_path((0,0),[(1,0),(-2,0),(3,0)],[(0,1,2)])
38         self.assertEqual(route,(1,0,2));self.assertEqual(d,7)
39         self.assertEqual(remaining_open_path((0,0),[(-1,0),(1,0)],[(0,1),((0,1),3.)])
40
41     def test_empty_singleton_and_duplicate_rejection(self):
42         self.assertEqual(remaining_open_path((0,0),[],[]),((0,0),))
43         self.assertEqual(remaining_open_path((0,0),[(3,4)],[]),((0,0),5.))
44         with self.assertRaises(ValueError):remaining_open_path((0,0),[(3,4)],[(0,0)])

```

```

45
46 def test_longer_first_scan_still_competes_with_localization(self):
47     cover=[(1,0),(-2,0),(3,0)];job=(1.5,10,(0,1.5),{})
48     args=((0,0),None,cover,[0,1,2],[job],{},{},[],config())
49     task,target,info=decide(*args)
50     self.assertEqual(task,'localize');self.assertEqual(target,job)
51     self.assertEqual(info['scan_route'],[1,0,2]);self.assertEqual(info['scan_route_cost_m'],7)
52     task,target,_=decide(*args[:-1],{})
53     self.assertEqual((task,target),('scan',0))
54
55 def test_real_controller_logs_valid_routes_and_retains_late_source(self):
56     for case in [random_case(0),late_source_case()]:
57         client=Client(Simulator(case),'SELF');cover=stations(config()['search_radius_m']);tasks=[]
58         def check(e):
59             if e['type']!='schedule':return
60             tasks.append(e);route=e['scan_route']
61             self.assertEqual(len(route),len(set(route)))
62             self.assertAlmostEqual(e['scan_route_cost_m'],cost(client.position,cover,route),places=7)
63             self.assertAlmostEqual(e['scan_route_cost_m'],min(cost(client.position,cover,p) for p in
↳ itertools.permutations(route)),places=7)
64             if e['task']=='scan':self.assertEqual(route[0],e['station_id'])
65         result=run(client,config(),check)
66         self.assertEqual(result['status'],'complete');self.assertEqual(len(tasks),23)
67         if case['id']=='late_channel20_after_ten':
68             self.assertIn(20,result['cleared_channels'])
69             self.assertEqual(result['stopping_certificate']['method'],'seven_station_channel_cover')
70
71 def test_deadline_on_scan_never_reports_complete(self):
72     client=Client(Simulator(late_source_case()),'SELF');n=0
73     def expire(e):
74         nonlocal n
75         if e['type']=='schedule' and e['task']=='scan':
76             n+=1
77             if n==3:client.deadline=0
78     result=run(client,config(),expire)
79     self.assertEqual(n,3);self.assertEqual(result['status'],'incomplete');self.assertFalse(result['exit_confirmed']
↳ '')
80
81 def test_incompatible_or_unknown_config_rejected(self):
82     for edits in [dict(scan_order='wrong'),dict(ring_phase='first_entry_radial'),dict(modules={})]:
83         with self.assertRaises(ValueError):run(Client(Simulator(random_case(0)),'SELF'),{**config(),**edits})

```

程序 171

对应文件: question3/tests/test_shared_batch.py

```

1 """Batch invariants and adversarial interrupted-batch paths, without truth in policy."""
2 import json
3 from pathlib import Path
4 import unittest
5 from question3.cases import clustered_case,late_source_case
6 from question3.interface import Client
7 from question3.simulator import Simulator
8 from question3.strategy import run
9
10
11 def config():
12     return json.loads(Path('question3/configs/round5_dev/COMPACT_BATCH.json').read_text())
13
14
15 class SharedBatchTests(unittest.TestCase):
16     def test_multiple_shared_cached_once_and_full_certificate(self):
17         events=[];requests=[];env=Simulator(clustered_case())
18         result=run(Client(env,'SELF',requests.append),config(),events.append)
19         stores=[e for e in events if e['type']=='shared_cache_store']
20         self.assertGreater(max(len(e['cached_channels']) for e in stores),1)
21         channels=[e['channel'] for e in stores]
22         self.assertEqual(len(channels),len(set(channels)))
23         self.assertEqual(sorted(channels),sorted(e['channel'] for e in events if e['type']=='shared_cache_use'))
24         for c in channels:
25             ms=[e for e in requests if e['type']=='request' and e['path']=='/measure'

```

```

26         and e['payload']['channel']==c and e['phase'].startswith('localization')]
27         self.assertEqual(len(ms),1)
28         self.assertEqual(result['status'],'complete');self.assertFalse(env.evaluation()['remaining_channels'])
29         for c in range(1,21):
30             self.assertTrue(c in result['cleared_channels'] or len(result['negative_station_ids'][str(c)])==7)
31
32     def test_second_shared_contradiction_cannot_certify_or_exit(self):
33         class BadClient(Client):
34             shared_count=0
35             def measure(self,p,c,phase='measure'):
36                 r=super().measure(p,c,phase)
37                 if phase=='localization_shared':
38                     self.shared_count+=1
39                     if self.shared_count==2:r={'**r','measure_result':'no_signal'}
40                 return r
41         client=BadClient(Simulator(clustered_case()),'SELF')
42         r=run(client,config())
43         self.assertEqual(client.shared_count,2);self.assertEqual(r['status'],'incomplete')
44         self.assertFalse(r['exit_confirmed']);self.assertIn('Guaranteed-reception',r['failure'])
45
46     def test_deadline_mid_batch_does_not_clear_cached_channels(self):
47         client=Client(Simulator(clustered_case()),'SELF');stores=[]
48         def event(e):
49             if e['type']=='shared_cache_store':
50                 stores.append(e)
51                 if len(stores)==2:client.deadline=0
52         r=run(client,config(),event)
53         self.assertEqual(len(stores),2);self.assertEqual(r['status'],'incomplete')
54         self.assertFalse(r['exit_confirmed'])
55         self.assertTrue(all(e['channel'] not in r['cleared_channels'] for e in stores))
56
57     def test_late_source_and_search_fairness(self):
58         events=[];r=run(Client(Simulator(late_source_case()),'SELF'),config(),events.append)
59         scans=chain=0
60         for e in events:
61             if e['type']!='schedule':continue
62             if e['task']=='scan':scans+=1;chain=0
63             else:
64                 chain+=1
65                 if scans<7:self.assertEqual(chain,2)
66         self.assertEqual(scans,7);self.assertIn(20,r['cleared_channels']);self.assertEqual(r['status'],'complete')
67
68     def test_invalid_batch_limit_before_enter(self):
69         for limit in [0,20,1.5,True,None]:
70             c=config();c['shared_batch_limit']=limit;client=Client(Simulator(clustered_case()),'SELF')
71             with self.assertRaises(ValueError):run(client,c)
72             self.assertFalse(client.entered)
73
74     def test_explicit_one_preserves_default(self):
75         c=config();c.pop('shared_batch_limit')
76         a=run(Client(Simulator(clustered_case()),'SELF'),c)
77         c['shared_batch_limit']=1;b=run(Client(Simulator(clustered_case()),'SELF'),c)
78         self.assertEqual(a,b)

```